과제번호: 10-IT-01

차량용 임베디드 시스템 기술 개발

Development of a Vehicle Embedded System

대구경북과학기술원

교육과학기술부

과제번호 : 10-11-01

차량용 임베디드 시스템 기술 개발

교육과학기술부

제 출 문

과학기술부 장관 귀하

본 보고서를 “차량용 임베디드 시스템 기술 개발”과제의 보고서로 제출합니다.

2011 . 1 .

주관연구기관명 : 대구경북과학기술원

주관연구책임자 : 진 성 호

연 구 원 : 조 희 섭
〃 : 조 성 래
〃 : 조 국 래
〃 : 손 종 욱
〃 : 박 세 권
〃 : 조 현 철
〃 : 김 정 면
〃 : 김 병 준

보고서 초록

<table>
<tr><td>과제관리번호</td><td>10-IT-01</td><td>연구기간</td><td>208.1.1~2010.12.31</td><td>단계 구분</td><td>3단계 / 3단계</td></tr>
<tr><td rowspan="2">연구사업명</td><td>중 사 업 명</td><td colspan="4">기관고유사업</td></tr>
<tr><td>세부사업명</td><td colspan="4">미래산업융합기술연구사업</td></tr>
<tr><td rowspan="2">연구과제명</td><td>중 과 제 명</td><td colspan="4"></td></tr>
<tr><td>세부(단위)과제명</td><td colspan="4">차량용 임베디드 시스템 기술 개발</td></tr>
<tr><td>연구책임자</td><td>진 성 호</td><td>해당단계
참여연구원수</td><td>총 : 9 명
내부 : 5 명
외부 : 4 명</td><td>해당단계
연구비</td><td>정부: 2,515,246 천원
기업: 0 천원
계: 2,515,246 천원</td></tr>
<tr><td>연구기관명 및
소속부서명</td><td colspan="2">대구경북과학기술원
미래산업융합기술연구부</td><td colspan="2">참여기업명</td><td></td></tr>
<tr><td>국제공동연구</td><td colspan="5">상대국명 : 상대국연구기관명 :</td></tr>
<tr><td>위 탁 연 구</td><td colspan="5">연구기관명 : 한국과학기술원 연구책임자 : 조 동 호
포항공과대학교 염 대 현</td></tr>
<tr><td colspan="4">요약(연구결과를 중심으로 개조식 500자이내)</td><td>보고서면수</td><td></td></tr>
<tr><td colspan="6">1. 표준화된 차량용 소프트웨어 플랫폼 구조 연구
o 컴포넌트 실행환경 개발
- AUTOSAR R3.0기반의 RTE 세부 기능 구현 및 테스트
o OS 개발
- AUTOSAR OS 안정화
- OSEK OS 인증
o Safety Critical 서비스 개발
- 시스템 서비스 모듈 안정화(ECU State Manager)
- 내결함 통신 서비스 설계 및 구현(SM, NM, TP)
- 내결함 통신 서비스 안정화(COM, PDU Router)
o Safety Critical 하드웨어 추상화(Hardware Abstraction, HAL) 기법 연구
- 내결함 통신 하드웨어 추상화 계층 안정화
- 내결함 통신 드라이버 안정화
2. 차량용 소프트웨어 플랫폼 설계 도구 개발
o ECU 환경 설정 도구 개발
o BSW 코드 생성 도구 개발
3. 차량용 소프트웨어 플랫폼 기반 응용시스템 개발
o 지능형 인테리어 라이팅 시스템 개발
4. ITS를 위한 효율적인 인증 프로토콜 개발
o 효율적인 해쉬 체인 운행 알고리즘
- 해쉬 체인 운행 기법의 기존 연구 분석
- 일반화한 해쉬 체인 운행 알고리즘 설계
o 양방 인증을 위한 거리 한정 알고리즘
- 거리 한정 알고리즘의 기존 연구 분석
- False accept rate를 최소화한 거리 한정 알고리즘 설계
- Mutual challenge를 이용한 거리 한정 알고리즘 설계
5. ITS를 위한 효율적인 차량간 통신 방안 연구
o BDMA 기반의 V2I 통신 시스템 연구
- 효율적인 초기 접속 및 충돌 회피 방안 연구
- 차량의 grouping을 이용한 자원할당 방안 제안
o 인프라와 릴레이의 협력을 통한 재전송 기술 연구
- 차량간 통신에서 C-ARQ mechanism의 성능 분석</td></tr>
<tr><td rowspan="2">색 인 어
(각 5개 이상)</td><td>한 글</td><td colspan="4">오토사, 소프트웨어플랫폼, 운영체제, 베이직소프트웨어, 런타임환경, 차량간 통신</td></tr>
<tr><td>영 어</td><td colspan="4">AUTOSAR, Software Platform, OS, BSW, RTE, V2V</td></tr>
</table>

요 약 문

Ⅰ. 제 목

차량용 임베디드 시스템 기술 개발

Ⅱ. 연구개발의 목적 및 필요성

최근 거의 90% 정도의 자동차 기술의 혁신은 소프트웨어와 전자장치에 의해 이루어지고 있다. 소프트웨어 및 하드웨어 기술의 발전으로 지난 몇 년간 자동차 산업은 많은 변화를 거듭하고 있고 향후 수십 년 간 이러한 변화는 계속될 것으로 예측된다. 또한, 차량 가격에서 차지하는 소프트웨어의 가격이 2006년 기준 8.8% 정도로 추정되고, 2010년 후에는 12.8%까지 늘어날 것으로 예측되며 차량 소프트웨어의 시장 또한 급속히 팽창하고 있다. 또한, 차량 전자장치의 증가로 인하여 차량내의 ECU (Electronic Control Unit) 증가 및 ECU간의 상호 연동이 필요한 기능이 점차 확대되고 있으나 ECU 사이의 인터페이스 표준의 부족으로 전자 제어 시스템의 통합이 어려운 실정이다. 이러한 문제점을 해결하기 위해 표준화된 통합 소프트웨어 플랫폼의 도입이 필요하며, 통합 소프트웨어 플랫폼은 각 소프트웨어 모듈의 작동 환경, 통신 방식, ECU 제약 조건 등을 단일화된 방식으로 기술하도록 하여 객체 지향적이고 확장성이 용이한 소프트웨어를 제작할 수 있게 해주는 개발 환경을 제공한다.

표준화된 소프트웨어 플랫폼과 설계 도구를 개발하여 ECU 개발에 소요되는 시간 및 노력을 절감하고 표준화된 인터페이스와 운영 체계를 사용함으로써, 차종에 따라 차량 응용소프트웨어를 개발하여야 하는 문제점을 해결하고 하드웨어에 관계없이 소프트웨어를 독립적으로 개발할 수 있도록 하고자 한다.

지능형 교통시스템(ITS : Intelligent Transport System)은 기존의 교통체계를 정보통신, 전자, 제어 컴퓨터등의 첨단 기술과 접목시켜 교통의 이동성, 안전성, 효율성 및 교통환경을 혁신적으로 개선하는 신 교통체계이다. 최근 들어, 텔레매틱스 기술 고도화와 관련 산업의 확산에 따라 실시간 차량정보 기반 컨버전스 서비스의 중요성이 높아지고 있으며 특히, 고속 이동체인 차량과 차량 사이 및 외부 통신 인프라 사이의 일관적이며 끊김 없는 차량 정보의 수집, 처리 및 구조화, 그리고 체계적인 관리의 필요성이 대두되고 있다.

이미 미국/유럽에서는 국가나 민간 주도의 다양한 프로젝트들이 존재하여 Vehicle-to-Vehicle/ Vehicle-to-Infrastructure (V2V/V2I or V2X) 통신을 위한 선행 연구와 표준화가 진행 중에 있다. 2008년도 까지 V2X 통신의 핵심이 되는 Physical 계층과 MAC 계층은 IEEE 802.11p로 상위 계층은 IEEE 1609.x 표준을 중심으로 상당 부분 표준화가 진행되었다. 따라서 앞으로의 V2X 연구는 양산과 상품화로 직결되는 실용적 기술개발과 구현과 상품화에 관련된 연구가 필요하다.

기존 네트워크의 기반구조만을 이용하는 서비스의 형태들이 수요 창출의 한계를 드러내고 있는 상태에서 V2V, V2I를 모두 지원하고 있는 차량통신은 새로운 과학 기술 응용 영역을 개척하는 중요한 의미를 가진다. 기존 시스템과 차량 네트워크의 차이점은 통신의 주체가 차량이라는 점과 고속 이동성이 가지는 특성에 기반을 둔다. 차량에서는 송신 전력에 큰 제약이 없으며 GPS와 같은 많은 부가 기능을 추가할 수 있다. 반면, 고속 이동성으로 인해 fading 채널이 존재하며, 차량 네트워크의 토폴로지가 급격하게 변화하게 된다. 따라서 이와 같은 차량 네트워크의 특성을 이해하여 기존 네트워크 기반의 기술들을 보완, 발전시키는 연구과정이 필요한다.

본 연구개발과제에서는 지능형 교통시스템을 위한 효율적인 차량 간 통신을 위한 핵심 요소 기술 개발을 최종 목표로 하여 빔 분할 다중 접속(BDMA) 기반 V2I시스템 연구를 통한 차량간 통신 방안과 인프라와 릴레이의 협력을 통한 인프라 기반의 재전송 기술을 개발하며 이와 함께 통신망 보안을 위해서 고속으로 주행하는 차량 간의 효율적인 데이터 인증을 위한 알고리즘을 개발하여 차량 간에 송?수신되는 메시지에 대한 신뢰성을 확보하고, 효율적인 메시지 처리를 가능하게 하는 기능을 개발하기 위한

연구를 수행하고자 한다.

Ⅲ. 연구개발의 내용 및 범위

1. 표준화된 차량용 소프트웨어 플랫폼 구조 연구
 1.1 컴포넌트 실행환경 개발
  o AUTOSAR R3.0기반의 RTE 세부 기능 구현
 1.2 OS 개발
  o AUTOSAR OS 테스트 및 안정화
  o OSEK OS 인증 획득
  o 플랫폼 확장 및 상용화
 1.3 Safety Critical 서비스 개발
  o 시스템 서비스 모듈 안정화(ECU State Manager)
  o 내결함 통신 서비스 설계 및 구현(SM, NM, TP)
  o 내결함 통신 서비스 안정화(COM, PDU Router)
 1.4 Safety Critical 하드웨어 추상화(Hardware Abstraction, HAL) 기법 연구
  o 내결함 통신 하드웨어 추상화 계층 안정화
  o 내결함 통신 드라이버 안정화

2 차량용 소프트웨어 플랫폼 설계 도구 개발
 2.1 ECU 환경 설정 도구 개발
  o 다중 플랫폼 지원
 2.2 BSW 코드 생성 도구 개발
  o BSW 요구사항 반영 코드생성 기능 개선

3. 차량용 소프트웨어 플랫폼 기반 응용시스템 개발
 3.1 지능형 인테리어 라이팅 시스템 개발
  o AUTOSAR 아키텍처 적용 소프트웨어 설계 및 시스템 통합

4. ITS를 위한 효율적인 인증 프로토콜 개발

4.1 효율적인 해쉬 체인 운행 알고리즘

- o 해쉬 체인 운행 기법의 기존 연구 분석
- o 일반화한 해쉬 체인 운행 알고리즘 설계
- o 일반화한 해쉬 체인 운행 알고리즘 개발

4.2 양방 인증을 위한 거리 한정 알고리즘

- o 거리 한정 알고리즘의 기존 연구 분석
- o False accept rate를 최소화한 거리 한정 알고리즘 설계
- o Mutual challenge를 이용한 거리 한정 알고리즘 설계
- o Mutual challenge를 이용한 거리 한정 알고리즘 개발

5. ITS를 위한 효율적인 차량간 통신 방안 연구

5.1 BDMA 전송 기술 기반의 차량 통신 시스템

- o 기존의 차량 통신 시스템 연구
- o 빔 분할 다중 접속 기술
- o BDMA 기반의 차량 통신 시스템

5.2 BDMA 전송 기술 기반의 차량 시스템에서 차량 스케줄링 방안

- o 기존 연구 및 문제점
- o 제안 방안
- o 성능 분석

5.3 BDMA 기반의 차량 통신 시스템에서 multi-hop relay 방안

- o 기존 연구 및 문제점
- o 제안 방안
- o 성능 분석

5.4 인프라와 릴레이의 협력을 통한 재전송 기술

- o 차량간 통신에서 C-ARQ mechanism 연구

Ⅳ. 연구개발결과

1. 표준화된 차량용 소프트웨어 플랫폼 구조 연구

1.1 컴포넌트 실행환경 개발

o AUTOSAR R3.0기반의 RTE 세부 기능 구현

- AUTOSAR RTE Generator모듈 안정화
- AUTOSAR RTE Template모듈 안정화
- AUTOSAR RTE Generator/Template모듈 연동 테스트

1.2 OS 개발

o AUTOSAR OS 테스트 및 안정화

- OS 커널 소스코드 동적 테스트(SW 커버리지 테스트) 수행

o OSEK OS 인증 획득

- 인증용 OSEK-OS 구현 결과 문서 작성
- Conformance Test 결과를 기반으로 인증 절차 진행 및 인증 획득 완료

o 플랫폼 확장 및 상용화

- 8/16비트 타겟 지원을 위한 확장 개발
- 이식성 향상을 위한 기능 개선
- 상용화 관련 문서 작업(User Guid 문서 및 상표 등록 등)
- 기술이전 완료 1건

1.3 Safety Critical 서비스 개발

o 시스템 서비스 모듈 안정화(ECU State Manager)

o 내결함 통신 서비스 설계 및 구현(SM, NM, TP)

o 내결함 통신 서비스 안정화(COM, PDU Router)

1.4 Safety Critical 하드웨어 추상화(Hardware Abstraction, HAL) 기법 연구

o 내결함 통신 하드웨어 추상화 계층 안정화

o 내결함 통신 드라이버 안정화

2 차량용 소프트웨어 플랫폼 설계 도구 개발

2.1 ECU 환경 설정 도구 개발

o 다중 플랫폼 지원

- 8/16비트 타겟 MCU 환경 설정 기능 개발

2.2 BSW 코드 생성 도구 개발

o BSW 요구사항 반영 코드생성 기능 개선
- 코드생성 도구 통합을 위한 개선 및 최적화

3. 차량용 소프트웨어 플랫폼 기반 응용시스템 개발
3.1 지능형 인테리어 라이팅 시스템 개발
o AUTOSAR기반 지능형 인테리어 라이팅 시스템 프로토타입 개발

4. ITS를 위한 효율적인 인증 프로토콜 개발
4.1 효율적인 해쉬 체인 운행 알고리즘
o 기존 기술 연구
o 일반화한 해쉬 체인 운행 알고리즘 설계
o 일반화한 해쉬 체인 운행 알고리즘 개발
- 알고리즘의 구현 및 테스트

4.2 양방 인증을 위한 거리 한정 알고리즘
o 공격 유형 분석
o 기존 기술 연구
o False accept rate을 최소화한 거리 한정 알고리즘 설계
o Mutual challenge를 이용한 거리 한정 알고리즘 설계
o 릴레이 공격에 강한 효율적인 거리 한정 프로토콜 개발

5. ITS를 위한 효율적인 차량간 통신 방안 연구
5.1 BDMA 전송 기술 기반의 차량 통신 시스템
o 프레임 구조 설계
o 효율적인 초기 접속 및 충돌 회피 알고리즘 설계
o 차량의 grouping을 이용한 적응적 자원할당 알고리즘 설계
5.2 BDMA 전송 기술 기반의 차량 시스템에서 차량 스케줄링 방안
o V2V2I 통신 시스템에서 V2V통신 스케줄링 방안 설계
5.3 BDMA 기반의 차량 통신 시스템에서 multi-hop relay 방안
o 간섭을 고려한 최적의 relay 차량 선택 방안 설계
5.4 인프라와 릴레이의 협력을 통한 재전송 기술

o 차량간 통신에서 C-ARQ mechanism 설계

V. 연구개발결과의 활용계획

구 분	구체적인 단계 및 내용
기술의 응용분야 및 활용방법	o 차량용 소프트웨어 플랫폼 - 자동차, 부품 모듈 개발에 활용 - 지능형/무인 자동차의 소프트웨어 플랫폼으로 활용 - 선박, 항공 우주 산업, 의료, 로봇, 플랜트 분야의 임베디드시스템에 적용 가능 o 차량용 소프트웨어 플랫폼 설계 도구 및 운영체제 설계 도구 - 자동차 제조사 및 부품 업체의 전기전자시스템 개발도구로 활용 - 기타 임베디드시스템의 SW플랫폼 설정 도구로 사용 가능 o ITS를 위한 효율적인 인증 프로토콜 - 차량과 기지국간의 송수신 메시지 인증 모듈 확보 - 차량간의 송수신 메시지 인증 모듈 확보 - 차량과 IT 통신기술의 융합은 차량이 무선 통신망에 접속되어 운전자에게 교통정보안내, 긴급구조, 돌발 상황 경고, 차량간 데이터 통신, 지역 정보 제공을 통하여 편리함과 안정성을 증대시켜 새로운 부가가치를 얻을 수 있음. o ITS를 위한 효율적인 차량 통신 방안 - BDMA 기법을 이용해서 서비스 커버리지를 확장시키고 시스템 용량을 증대 시킬수 있는 차세대 차량간 통신 방안 원천 기술 확보
제품/서비스의 예상 수요자	o 자동차 제조사, 자동차부품업체 o 자동차 소프트웨어지원업체 또는 개발도구지원업체
시장 도입 소요 시간	3~6년

S U M M A R Y

At the beginning of the third millennium, the automotive industry is facing a new challenge. Electronics make 90% of the innovations, 80% out of that in the area of software. This fact means a big change for the development of electronics. More and more highly connected functionality has to be brought into series production while development time is getting shorter and shorter. This change is believed to keep for the next decades of years. And it is estimated that the portion of the software cost in the vehicle price was 8.8% in 2006 and it is expected to increase up to 12.8% by 2010, leading to rapid expansion of the vehicular software market. The rise of the vehicular electronic device gradually requires various functions necessary for the interactive operation between the increased ECUs (Electronic Control Unit). However, the lack of the interface standard between ECU makes it difficult to integrate the electronic control system. Therefore the introduction of a standardized integration software platform is required to resolve this problem. This integration software platform which takes each software module's functions such as operation environment, communication method, ECU limitation condition and so on, to be described in a single simplified way, plays a role as integrated development environment that helps to build an object-oriented and easily expandable software.

Due to the development of telematics technologies, the demand for wireless vehicular communication has been growing. Vehicular communication is able to significantly improve upon the weaknesses of a conventional transportation system with respect to safety, efficiency, and convenience. The wireless vehicular communication can be classified into two categories of a vehicle-to-vehicle (V2V) and vehicle-to-infrastructure (V2I) communications. In the V2V communication, each vehicle communicates with other vehicles. In the V2I communication, vehicles exchange information with an infrastructure (henceforth, infra) that is a fixed base station generally

located at the side of the road. The infra may connect vehicles to other data networks to provide various services. Wireless access in vehicular environment (WAVE) refers to a set of emerging standards to support V2V and V2I communications for intelligent transportation system (ITS). WAVE defines enhancements to the IEEE 802.11a (WLAN) from an indoor environment to a vehicular environment with high mobility, fast fading channels, and larger transmission range. The medium access control (MAC) and physical (PHY) layers of WAVE are covered by the IEEE 802.11p. WAVE systems operate in the licensed frequency band of 5.85-5.925 GHz and use an orthogonal frequency division multiplexing (OFDM) modulation to achieve high data rate up to 27 Mbps. The MAC mechanism in WAVE is equivalent to the IEEE 802.11e enhanced distributed channel access (EDCA) that uses carrier sense multiple access with collision avoidance (CSMA/CA) and binary exponential backoff methods. The upper layers of WAVE are covered by the IEEE 1609 that is the family of standards for resource management, security, connection setup and management, and multi-channel operation. To satisfy the demand for high data rate and adaptability to the changing network conditions, smart antenna technology is considered as one of key technologies. By using the smart antenna technology that utilizes multiple antennae and beamforming, wireless systems can extend service coverage, achieve diversity gain which improves link quality against multipath fading, and increase spectral efficiency.

In this research, we propose the integration development system which solves the problem to produce the same application software differently for a target vehicle according to each vehicle model and makes it possible to develop the software without depending on the hardware. This system is accomplished through the development of the standardized software platform and the design tool which reduces both time and effort spent for ECU development and uses the standardized interface and the operating system. Also, to provide assistance to the automotive industry in the creation and application of safe and reliable software in vehicle systems. And we also propose the V2I and V2V communication system with a time division

multiple-access (TDMA)-based frame structure that uses smart antenna. By using smart antenna, the proposed system can increase service coverage and system throughput although the computational complexity of infra is increased.

C O N T E N T S

Chapter 1. Introduction ····· 1
Section 1. Purpose of the research ····· 1
Section 2. Necessity and Scope of the research ····· 2

Chapter 2. State of the arts ····· 9
Section 1. State of the arts on domestic research ····· 9
Section 2. State of the arts on overseas research ····· 9

Chapter 3. Contents and Results of the research ····· 13
Section 1. Goal of the research ····· 13
Section 2. Contents of the research ····· 13
Section 3. Results of the research ····· 15

Chapter 4. Achievements and Contributions ····· 363
Section 1. Achievements ····· 363
Section 2. Contributions ····· 372

Chapter 5. The plan of the practical use ····· 374

Chapter 6. Technical information ····· 376

Chapter 7. References ····· 388

목 차

제 1 장 연구개발과제의 개요 ····· 1
제 1 절 연구개발의 목적 ····· 1
제 2 절 연구개발의 필요성 및 범위 ····· 2

제 2 장 국내외 기술개발 현황 ····· 9
제 1 절 국내 기술개발 현황 ····· 9
제 2 절 국외 기술개발 현황 ····· 9

제 3 장 연구개발수행 내용 및 결과 ····· 13
제 1 절 연구개발 목표 ····· 13
제 2 절 연구개발 내용 ····· 13
제 3 절 연구개발 수행 결과 ····· 15

제 4 장 목표달성도 및 관련분야에의 기여도 ····· 363
제 1 절 연구개발목표의 달성도 ····· 363
제 2 절 관련분야의 기여도 ····· 372

제 5 장 연구개발결과의 활용계획 ····· 374

제 6 장 연구개발과정에서 수집한 해외과학기술정보 ····· 376

제 7 장 참고문헌 ····· 388

제 1 장 연구개발과제의 개요

제 1 절 연구개발의 목적

저연비, 저 배기가스, 안전 및 편의에 대한 소비자의 요구 및 법규 규제로 인하여 자동차 산업은 커다란 변화에 직면하고 있으며, 이러한 요구를 해결할 수 있는 상당 부분의 기술은 소프트웨어와 전자장치에 의존 하고 있다. 최근 거의 90% 정도의 자동차 기술의 혁신은 소프트웨어와 전자장치에 의해 이루어지고 있으며, 소프트웨어 및 하드웨어 기술의 발전으로 지난 몇 년간 자동차 산업은 많은 변화를 거듭하고 있고 향후 수십 년 간 이러한 변화는 계속될 것으로 예측된다. 또한, 차량 가격에서 차지하는 소프트웨어의 가격이 2006년 기준 8.8% 정도로 추정되고, 2010년 이후 12.8% 까지 늘어날 것으로 예측되며 차량 소프트웨어의 시장 또한 급속히 팽창하고 있다. 또한, 차량 전자장치의 증가로 인하여 차량내의 ECU (Electronic Control Unit) 증가 및 ECU간의 상호 연동이 필요한 기능이 점차 확대되고 있으나 ECU 사이의 인터페이스 표준의 부족으로 전자 제어 시스템의 통합이 어려운 실정이다. 이러한 문제점을 해결하기 위해 표준화된 통합 소프트웨어 플랫폼의 도입이 필요하며, 통합 소프트웨어 플랫폼은 각 소프트웨어 모듈의 작동 환경, 통신 방식, ECU 제약 조건 등을 단일화된 방식으로 기술하도록 하여 객체 지향적이고 확장성이 용이한 소프트웨어를 제작할 수 있게 해주는 개발 환경을 제공한다.

표준화된 소프트웨어 플랫폼과 설계 도구를 개발하여 ECU 개발에 소요되는 시간 및 노력을 절감하고 표준화된 인터페이스와 운영 체계를 사용함으로써, 차종에 따라 차량 응용소프트웨어를 개발하여야 하는 문제점을 해결하고 하드웨어에 관계없이 소프트웨어를 독립적으로 개발할 수 있도록 하고자 한다.

지능형 교통시스템(ITS : Intelligent Transport System)은 기존의 교통 체계를 정보통신, 전자, 제어 컴퓨터등의 첨단 기술과 접목시켜 교통의 이동성, 안전성, 효율성 및 교통환경을 혁신적으로 개선하는 신 교통체계이다. 최근 들어, 텔레매틱스 기술 고도화와 관련 산업의 확산에 따라 실시간 차량정보 기반 컨버전스 서비스의 중요성이 높아지고 있으며 특히, 고속 이동체인 차량과 차량 사이 및 외부 통신 인프라 사이의 일관적이며 끊김 없는 차량 정보의 수집, 처리 및 구조화, 그리고 체계적인 관리의 필요성이 대두되고 있다.

이미 미국/유럽에서는 국가나 민간 주도의 다양한 프로젝트들이 존재하여

Vehicle-to-Vehicle/ Vehicle-to-Infrastructure (V2V/V2I or V2X) 통신을 위한 선행 연구와 표준화가 진행 중에 있다. 2008년도 까지 V2X 통신의 핵심이 되는 Physical 계층과 MAC 계층은 IEEE 802.11p로 상위계층은 IEEE 1609.x 표준을 중심으로 상당 부분 표준화가 진행되었다. 따라서 앞으로의 V2X 연구는 양산과 상품화로 직결되는 실용적 기술개발과 구현과 상품화에 관련된 연구가 필요하다. 기존 네트워크의 기반구조만을 이용하는 서비스의 형태들이 수요 창출의 한계를 드러내고 있는 상태에서 V2V, V2I를 모두 지원하고 있는 차량통신은 새로운 과학 기술 응용 영역을 개척하는 중요한 의미를 가진다. 기존 시스템과 차량 네트워크의 차이점은 통신의 주체가 차량이라는 점과 고속 이동성이 가지는 특성에 기반을 둔다. 차량에서는 송신 전력에 큰 제약이 없으며 GPS와 같은 많은 부가기능을 추가할 수 있다. 반면, 고속 이동성으로 인해 fading 채널이 존재하며, 차량 네트워크의 토폴로지가 급격하게 변화하게 된다. 따라서 이와 같은 차량 네트워크의 특성을 이해하여 기존 네트워크 기반의 기술들을 보완, 발전시키는 연구과정이 필요하다.

본 연구개발과제에서는 지능형 교통시스템을 위한 효율적인 차량 간 통신을 위한 핵심 요소 기술 개발을 최종 목표로 하여 빔 분할 다중 접속(BDMA) 기반 V2I시스템 연구를 통한 차량간 통신 방안과 인프라와 릴레이의 협력을 통한 인프라 기반의 재전송 기술을 개발하며 이와 함께 통신망 보안을 위해서 고속으로 주행하는 차량 간의 효율적인 데이터 인증을 위한 알고리즘을 개발하여 차량 간에 송/수신되는 메시지에 대한 신뢰성을 확보하고, 효율적인 메시지 처리를 가능하게 하는 기능을 개발하기 위한 연구를 수행하고자 한다.

제 2 절 연구개발의 필요성 및 범위

1. 연구개발의 필요성

- 차량 전자 장치 증가 및 소프트웨어의 중요성 대두
  - 저연비, 저 배기가스, 안전 및 편의에 대한 소비자의 요구 및 법규 규제로 인하여 자동차 산업은 커다란 변화에 직면하고 있으며, 이러한 요구를 해결할 수 있는 상당 부분의 기술은 소프트웨어와 전자장치에 의존 하고 있음
  - 최근 거의 90% 정도의 자동차 기술의 혁신은 소프트웨어와 전자장치에 의해 이루어지고 있으며, 소프트웨어 및 하드웨어 기술의 발전으로 지난 몇 년간 자동차 산업은 많은 변화를 거듭하고 있고, 향후 수십 년 간 이러한

변화는 계속될 것으로 예측 됨

- 차량 가격에서 차지하는 소프트웨어의 가격이 2006년 기준 8.8% 정도로 추정되고, 2010년 까지 12.8% 까지 늘어날 것으로 예측되며, 차량 소프트웨어의 시장 또한 급속히 팽창하고 있음
- 기존의 기계적 부품을 가벼운 전자 제어 장치로 대체되고 있으며, 차량 소프트웨어 시장은 급속히 팽창하고 있음
- 최근 도요다 자동차의 심각한 리콜사태로 인해 차량의 전자장치 및 소프트웨어의 신뢰성 문제가 대두됨.

- 차량용 소프트웨어 표준 플랫폼의 필요성
  - 임베디드 시스템의 복잡성 증가로 인하여 시스템 통합과 소프트웨어 개발에 큰 어려움에 직면하고 있으며, 시스템이 점차 복잡해질수록 소프트웨어의 신뢰성이 중요해지고 있음
  - 자동차의 전기 전자적 구조가 점점 복잡해짐에 따라, 이를 제어하기 위한 소프트웨어의 양과 복잡도 또한 함께 증가하고 있으며, 소프트웨어를 개발하는 데 소요되는 기간 역시 늘어나고 있으며 소프트웨어의 결함이 발생할 확률도 증가하고 있음
  - 복잡하고 거대해진 차량용 소프트웨어를 효과적으로 개발하기 위해 표준화된 통합 소프트웨어 플랫폼이 필요함
  - 차량 전자장치의 증가로 인하여 차량내의 ECU(Electronic Control Unit) 증가 및 ECU간의 상호 연동이 필요한 기능이 점차 확대되고 있으나, ECU 사이의 인터페이스 표준의 부족으로 전자 제어 시스템의 통합이 어려움.
  - 이러한 문제점을 해결하기 위해, 표준화된 통합 소프트웨어 플랫폼이 필요하며, 통합 소프트웨어 플랫폼은 각 소프트웨어 모듈의 작동 환경, 통신 방식, ECU 제약 조건 등을 단일화된 방식으로 기술하도록 하여 객체 지향적이고 확장성이 용이한 소프트웨어를 제작할 수 있게 해주는 개발 환경
  - 소프트웨어 플랫폼을 표준화함으로써 ECU 개발에 소요되는 시간 및 노력의 절감 가능
  - 표준화된 인터페이스와 운영 체계를 사용함으로써 동일한 응용 소프트웨어를 다양한 차종에 따라 개발해야 하는 문제점을 해결하고, 하드웨어에 독립적으로 소프트웨어의 개발 가능
  - 표준화의 영향으로 재사용성 향상 및 업그레이드가 용이하며, 새로운 기능 개발 시 새로운 ECU를 개발이 쉬워지며, 표준을 따름으로써 신뢰성 향상

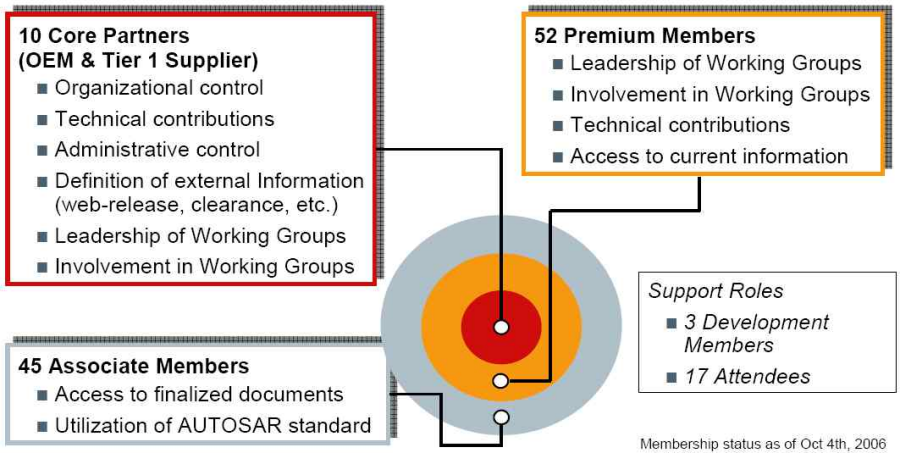


그림 1-1. AUTOSAR Partnership Structure

- AUTOSAR(Automotive Open System Architecture) 차량 소프트웨어 표준 플랫폼
  - 자동차 시장을 선도하는 유럽계 자동차 산업체들을 중심으로 소프트웨어 구조에 대한 공개 표준 확립을 위하여 AUTOSAR 협력체가 탄생
  - AUTOSAR는 차량용 소프트웨어의 구조에 대한 실질적인 산업 표준을 개발, 확립하기 위해 거의 모든 메이저 제조사 자동차 제조업체들과 부품 제조업체들이 공동으로 참여하는 협력체 임
  - AUTOSAR 명세의 Release 1.0이 2005년 초에 발표되었으며, Release 1.0을 기반으로 2005년 말에 Release 2.0이, 2006년 말에는 Release 2.1, 2007년 말에는 Release 3.0이 발표되었음.
  - AUTOSAR는 차량 소프트웨어 산업의 지속적인 발전 및 전문화 기틀 마련
    ; 차량 제조사는 핵심 어플리케이션 소프트웨어의 개발 및 최종 차량 통합 연구에 집중
    ; 시스템 통합 업체는 대부분의 어플리케이션을 및 ECU 개발, 소프트웨어 개발 업체는 어플리케이션 소프트웨어만 공급
    ; Mathworks, dSpace, Telelogic, ETAS 등과 같은 업체는 소프트웨어 개발 환경 및 시뮬레이션 툴 개발에 집중
    ; 반도체 업체는 임베디드 솔루션 및 configurable 플랫폼 개발에 집중
  - 표준화된 runtime environment 및 ECU 통합 표준 인터페이스 구축으로 시스템 통합이 쉬워짐
  - 플랫폼 및 ECU의 위치에 독립적인 응용 소프트웨어 개발 가능
  - 재사용성 증대로 약간의 변경으로 새로운 시스템 개발 및 업그레이드 가능

하며, engineering cost 절감, 개발 기간 단축 및 소프트웨어의 신뢰성 향상

· 차량용 application 소프트웨어 시장의 형성 및 성장

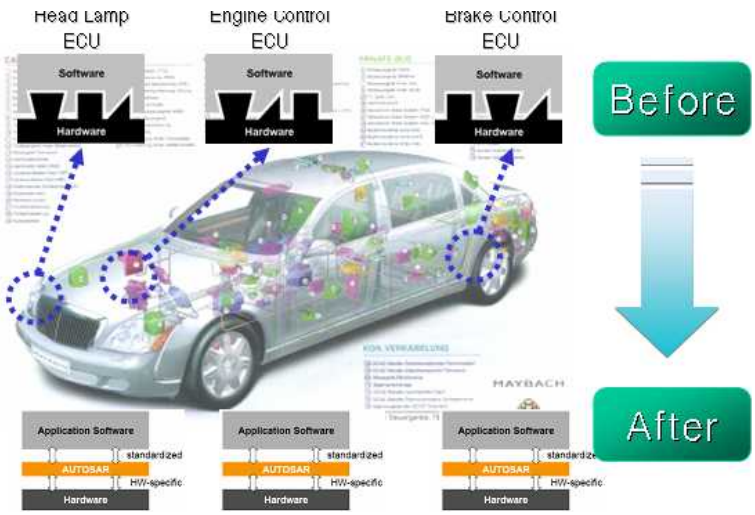


그림 1-2. 차량 전장용 임베디드 SW 플랫폼 개념

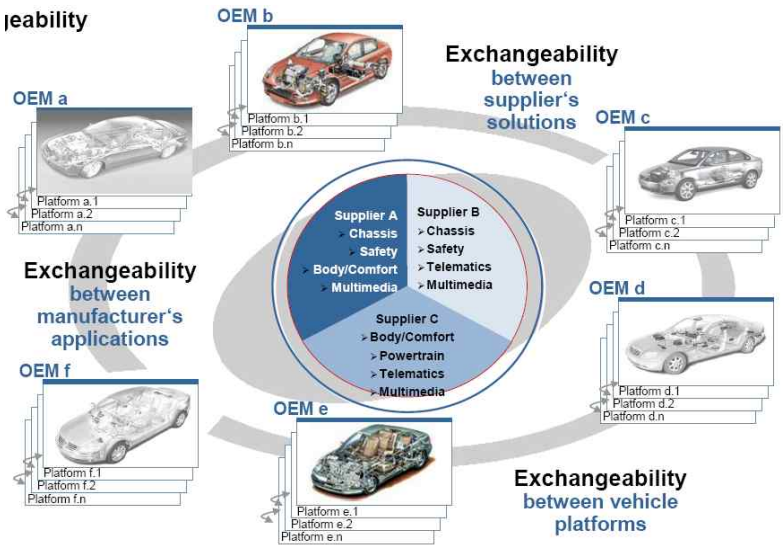


그림 1-3. AUTOSAR Exchangeability

- 국내 자동차 산업의 경쟁력 향상을 위하여 AUTOSAR 표준 소프트웨어 플랫폼 표준 연구 필요

  · 국내 자동차 업체들 중에는 현대/기아자동차, 만도, 대우정밀, 현대오토에버시스템즈, 대구경북과학기술연구원이 AUTOSAR에 가입하여 활동하고 있

음

- 국내는 AUTOSAR 연구들이 초기 수준에 머물러 있으며 일부 선도 기업에서 해외 상용제품을 사용을 계획하고 있는 것이 현실
- 국내의 독자적인 AUTOSAR 플랫폼을 개발함으로써 향후 예상되는 수요 증대에 대응 및 관련 기술의 국산화
- 지능형 자동차 소프트웨어 기술에서 가장 중요한 비중을 차지하게 될 AUTOSAR 기반 설계 능력 확보 및 통합 개발 환경을 제공함으로써 국제 경쟁력을 갖춘 자동차 소프트웨어 설계 기술의 초석을 마련할 필요 있음

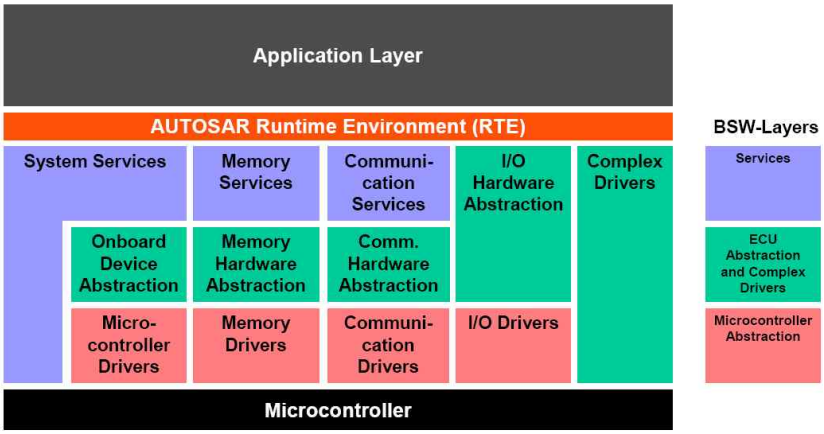


그림 1-4. AUTOSAR Layered Software Architecture

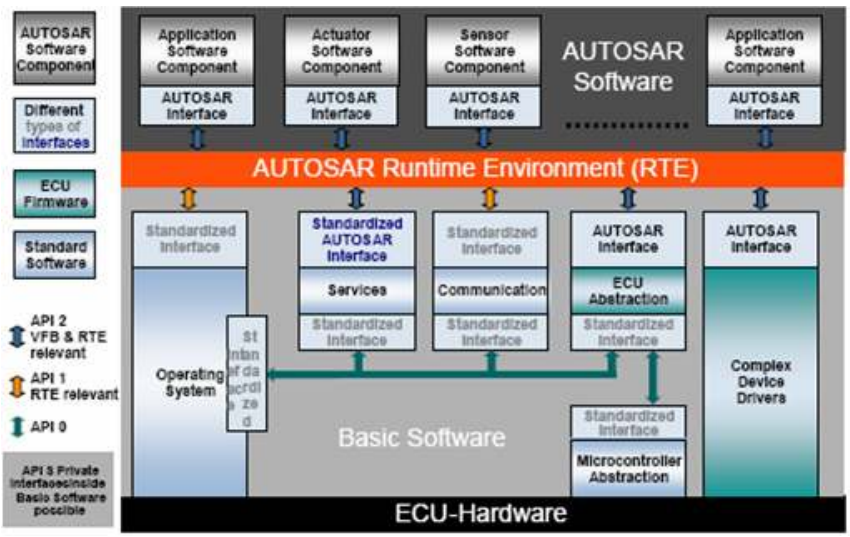


그림 1-5. AUTOSAR Architecture

- 전체 차량 IT기술의 세계시장 규모는 지속적으로 성장하고 있으며, 1980년대에는 차량

내부의 전자장치 비율이 1%에 불과했으나 2005년도에는 20%로 증가하였으며, 2015년에는 40%까지 증가하며 2000억 달러 규모의 세계시장을 형성할 것으로 전망된다(McKinsey, 2004). 국내 차량 IT 융합시장은 2008년도 40억 달러 규모에서 지속적으로 성장하여 2013년 95억 달러, 2018년 158억 달러의 시장을 형성할 것으로 전망된다(ETRI, 2008). 따라서 차량 통신기술을 선도하고 이와 같은 시장을 선점하기 위하여 차량 통신 연구 추진하여 핵심 원천 기술을 확보하고 지능형 교통시스템에 통합해 활용도를 높일 수 있다.

차량에 대한 전통적인 공격 방법은 자동차의 유리를 깨고 자동차를 탈취하거나 자동차 도어락을 불법으로 해지하는 등의 물리적 공격이 대부분이었다. 따라서 공격자는 공격하고자 하는 대상 자동차에 물리적으로 접근하여야 하는 부담이 있었다. 하지만, 정보통신 기술이 융합된 지능형 교통 시스템에서는 공격자가 인터넷을 통해 원격으로 차량을 공격할 수 있게 된다. 데이터 통신망에 대한 기존의 공격이 데이터의 유출 등과 같은 간접적 손실을 야기하였다면, 자동차 통신망에 대한 공격은 자동차 사고 등으로 이어져 차량 및 인명에 대한 직접적 피해가 예상된다. 그러므로 미래 선도 기술인 지능형 교통 시스템이 실생활에 적용되기 위해서는 자동차 통신망을 공격자로부터 보호하기 위한 보안 기술 개발이 반드시 선행되어야 한다.

하지만, 기존의 인증 보안 알고리즘들은 전자 서명 및 인증서와 같은 공개키 암호 시스템에 기반하고 있기 때문에, 고속으로 움직이는 차량들로 구성된 VANET에서의 인증 알고리즘으로는 적합하지 않다. 이러한 문제를 해결하기 위해 공개키 암호 시스템에 비해 효율이 뛰어난 해쉬 체인 알고리즘 등을 VANET에 적용하고자 하는 연구들이 제시되고 있지만, 연구 단계가 아직 초창기이기 때문에 만족할 만한 성능의 인증 알고리즘의 제시가 부족한 현실이다. 또한 인터넷과 달리 VANET은 물리적 위치 정보에 대한 인증 기술이 필요한데 이러한 부문에 대한 연구 또한 아직은 초보 단계라고 할 수 있다. 따라서 자동차 통신망에 적합한 효율적인 인증 프로토콜의 개발은 국가 인프라 보호를 위한 중요한 요소일 뿐 아니라, 학문적으로 초창기에 있는 VANET에 대한 원천 기술 확보 차원에서도 매우 중요한 주제이다.

2. 연구개발의 내용 및 범위

- 표준화된 차량용 소프트웨어 플랫폼 구조 연구
  - · 컴포넌트 실행환경 개발
    ; AUTOSAR R3.0기반의 RTE 세부 기능 구현
  - · OS 개발
    ; AUTOSAR OS 테스트 및 안정화

; OSEK OS 인증 획득
; 플랫폼 확장 및 상용화

· Safety Critical 서비스 개발
; 시스템 서비스 모듈 안정화(ECU State Manager)
; 내결함 통신 서비스 설계 및 구현(SM, NM, TP)
; 내결함 통신 서비스 안정화(COM, PDU Router)

· Safety Critical 하드웨어 추상화(Hardware Abstraction, HAL) 기법 연구
; 내결함 통신 하드웨어 추상화 계층 안정화
; 내결함 통신 드라이버 안정화

- 차량용 소프트웨어 플랫폼 설계 도구 개발
· ECU 환경 설정 도구 개발
; 다중 플랫폼 지원
· BSW 코드 생성 도구 개발
; BSW 요구상 반영 코드생성 기능 개선

- 차량용 소프트웨어 플랫폼 기반 응용시스템 개발
· 지능형 인테리어 라이팅 시스템 개발
; AUTOSAR 아키텍처 적용 소프트웨어 설계 및 시스템 통합

- ITS의 통신망 보안 위한 효율적인 데이터 인증 및 위치 정보 인증 알고리즘 개발 필요
· 필요에 따라 일정한 간격으로 위치한 노드들만을 효율적으로 출력하는 해쉬 체인 운행 (Hash Chain Traversal) 알고리즘을 개발
· 둘째 차량 상호간에 신뢰할 수 있는 거리 정보를 제공하기 위해 양방 인증 기능을 갖는 효율적인 거리 한정 프로토콜(Distance Bounding Protocol)을 개발

- ITS를 효율적인 차량간 통신 알고리즘 개발
· 서비스 커버리지 및 시스템 용량 증대를 위해 BDMA 기반의 차량 통신 방안 연구
· BDMA 기반의 multi-hop relay 방안 연구

제 2 장 국내외 기술개발 현황

제 1 절 국내 기술개발 현황

국내 차량용 임베디드 소프트웨어 플랫폼 기술 수준은 현재 기술 개발 초기 단계로, 일부 자동차 관련 기업 및 연구기관 주도로 운영체제, 소프트웨어플랫폼 등 차량용 임베디드 기술을 국산화하려는 작업을 시도하고 있다. 또한, 일부 완성차업체 및 자동차부품 업체에서 차량용 임베디드 소프트웨어 플랫폼을 사용한 응용시스템 개발을 선행연구로 진행중이다.

ETRI에서는 지식 경제부와 같이 Vehicle Multi-hop Communication(VMC) 프로젝트를 2007년부터 약 2년 6개월간 진행 중에 있다. 연구 목표는 V2X 통신 기술에 있어서 5.8GHz근방의 RF, OFDM 모뎀, CSMA/TDMA 혼합의 MAC 기술, 라우팅 기술을 포함한다. 시스템의 요구사항으로는 Broadcast, Multicast, Unicast 통신 방법을 포함하며 최대 200km/h의 이동성을 지원하며 패킷의 latency를 100ms이하로 한다. 주요 특징으로는 WAVE 기반의 시스템이라는 점과 높은 이동성의 지원을 위한 Mid-amble 채널 추정 방법과 최대 12Mbps까지 지원하는 전송 속도와 priority 컨트롤을 포함하는 Hybrid MAC 방식의 사용에 있다.

현재 관련 터미널 플랫폼과 통신 모듈을 포함한 시제품을 개발하여 Multi hop 방식의 V2V 테스트와 실제 도로에서 180km/h의 주행속도로 1km 라디오 영역에서 비디오 정보를 전송하는 시스템 시험을 수행하였다. 이를 바탕으로 스마트 하이웨이 적용, 국제 표준 특허 반영, 국내 주파수 할당을 추진 중에 있다.

제 2 절 국외 기술개발 현황

AUTOSAR 관련 국외 기술개발 현황은 개념정립 단계를 넘어서 기업화 단계에 있으며 주요 기관의 연구개발 동향은 다음 표와 같다.

연구수행 기관	연구개발의 내용	연구개발성과의 활용현황
Bosch	AUTOSAR플랫폼 개발	AUTOSAR플랫폼 시제품(CuBas)개발
Vector(독일)	AUTOSAR플랫폼 개발	AUTOSAR플랫폼 시제품(DaVinci)개발
TNI-Software (프랑스)	AUTOSAR플랫폼 개발	AUTOSAR플랫폼 시제품(AUTOSAR RM, AUTOSAR Builder)개발

표 2-1. 국외 기술개발 현황

미국에서는 교통국 주관으로 자동차 제조업체들이 참여한 Vehicle Infrastructure Integration Program (VII), Cooperative Intersection Collision Avoidance System (CICAS)와 같은 프로젝트가 진행 중에 있다. VII 프로젝트는 차량과 주변 사물 또는 차량 간의 통신을 제공하여 도로교통 안전성을 높이고, 교통 효율성을 증대시켜, 편리한 교통 시스템을 제공하려는 목표를 가진다.

IEEE를 중심으로 진행되고 있는 공공 안전과 ITS 서비스를 위한 V2V/V2I 차량 네트워크 기술을 포함하는 V2X 프로토콜을 Wireless Access for the Vehicular Environment (WAVE)라 불리고 있다. Physical 계층과 MAC 계층이 IEEE 802.11p에서 상위 계층의 통신 표준은 IEEE 1609.x시리즈에서 정의된다.

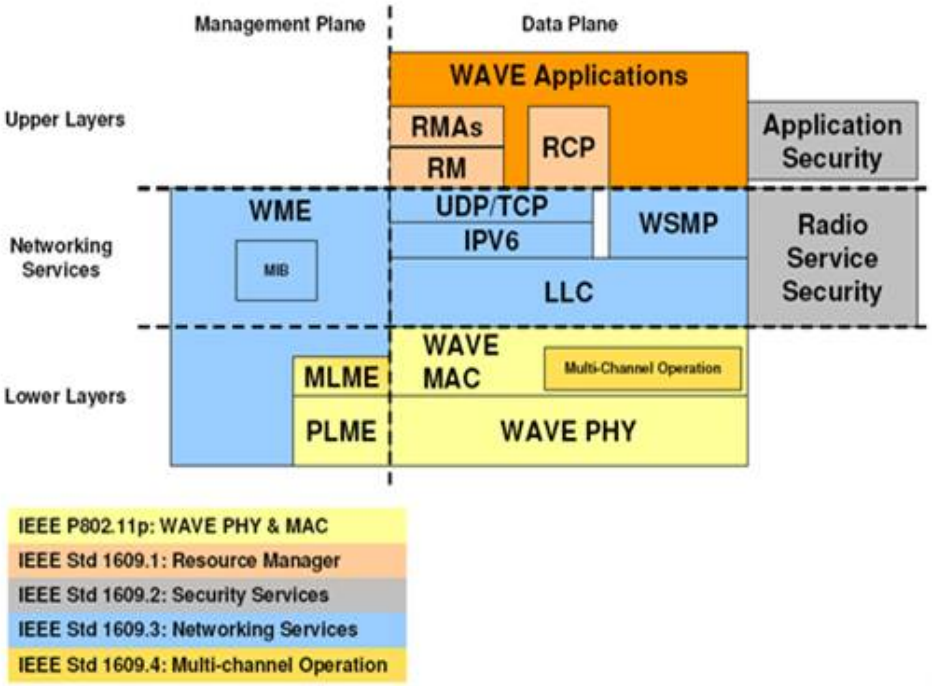


그림 2-1 WAVE 표준 프로토콜 구조

IEEE 802.11p의 성능 목표는 V2X 통신을 지원하여 최대 통달거리 1km, 차량 이동속도 200km/h를 지원하여 최대 54Mbps의 데이터 속도를 구현하는데 있다. 이에 해당하는 채널 환경으로는 도심지, 교외, 고속도로의 환경 등이 포함된다. 주파수 대역은 5.850GHz에서 5.925GHz까지 75MHz를 포함하며 이 사이에 데이터 통신과 컨트롤을 위한 7개의 채널이 존재하게 된다.

1609.x시리즈는 MAC계층과 Network계층의 다양한 영역을 포함하는데 각각의 기능은 아래와 같이 정리된다. IEEE 1609.1는 응용 서비스의 리소스 매니저로 다수의 데이터 스트림, 메모리 등의 각종 시스템 자원 관리를 수행하며, IEEE 1609.2는 보안 서비스를 책임져 Security가 보장되는 WAVE 메시지를 생성한다. IEEE 1609.3는 WAVE의 네트워킹 서비스와 프로토콜을 수행하여 IP 네트워킹과 안전 메시지 처리 프로토콜(WSMP)기능을 가지고 있다. IEEE 1609.4는 멀티 WAVE 채널 기능과 데이터와 제어 정보 채널을 효율적으로 관리한다.

유럽에서의 V2X 통신 연구 개발은 공적 자금으로 이루어지는 연구 프로젝트들이나 Car-2-Car Communication Consortium (C2C-CC)에 의해 진행된다. 많은 정부 지원의 연구 프로젝트에서 V2X 관련된 선행 연구 결과들이 나오고 있으며, 앞으로의 연구 프로젝트 개발은 주로 선행 연구 결과를 실제로 검증하는 필드 테스트나 테스트 베드 기반의 연구가 될 것으로 예상된다. C2C-CC는 유럽 주도의 표준화를 진행하는 단체로써, 산학연

이 다양한 연구 프로젝트의 결과를 공유하고 표준화를 진행하여 필요한 주파수 분배를 주도하는 것을 목적으로 한다. 현재 C2C-CC는 5.9GHz 대역에 30 MHz 대역폭을 V2X 통신 용도로 확보하였다

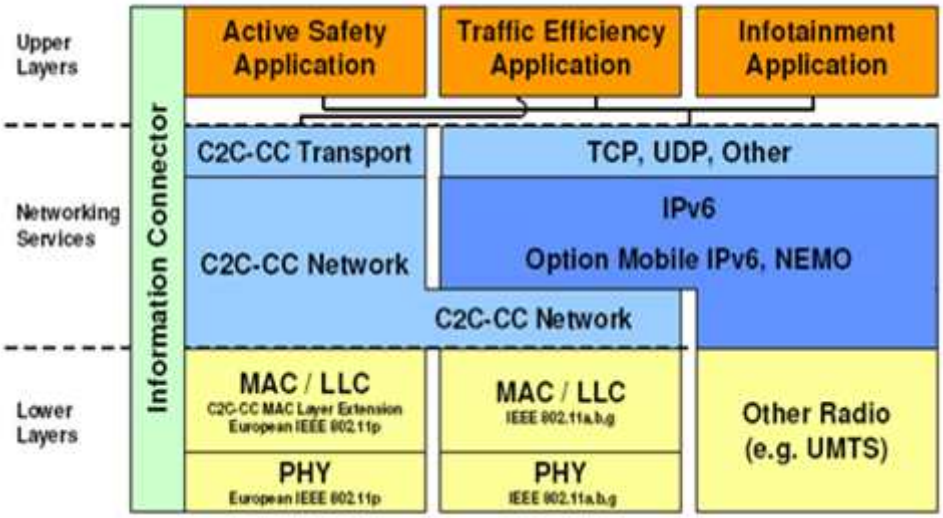


그림 2-2. C2C-CC 주도의 프로토콜 구조

위의 그림과 같이, C2C-CC 주도의 V2X표준도 Physical 계층과 MAC 계층은 IEEE 802.11p를 확장하여 구현하는 것을 기본으로 한다. 하지만, C2C-CC 주도의 V2X 표준이 가지는 미국 표준과의 가장 큰 차이점은 V2V 통신에 기반을 둔 기술들이 주로 구성되어, 상대적으로 V2I 통신을 고려한 기술은 크게 고려되고 있지 않다. 따라서 C2C-CC Transport와 Network 계층의 프로토콜에는 multi-hop 기능 및 라우팅 기능이 포함된다.

국내 기술 개발 현황을 살펴보면 현재 TTA에서 차량 간 통신 Physical 계층과 MAC 계층 규격을 제정하는 단계에 있으며 2007년 TTA PG310 차량 간 통신 실무반 구성 표준 개발에 착수 하였다. 미국/유럽과 동일한 IEEE 802.11p 규격을 기반으로 핵심 기술을 추가하는 것을 목표로 한다. 핵심 기술로는 MAC규격에 Time slot DCF 기능을 추가하여 Time slot운용에 따른 Priority, Back-off 동기 기능을 포함하는 방안과 PHY규격에 Mid-amble기능을 추가하여 Long Packet(1KB이상)에서 채널 추정 실패로 발생하는 에러를 개선하는 방안이 있다. 아직 국내에서는 차량 통신을 위한 정확한 주파수의 할당은 이루어지지 않았으나 5.875~5.905GHz까지의 30MHz를 일단 반영하여 개발이 이루어지고 있다

제 3 장 연구개발 수행 내용 및 결과

제 1 절 연구개발 목표

- AUTOSAR기반의 차량용 임베디드 SW 플랫폼 설계도구, RTE, BSW 개발

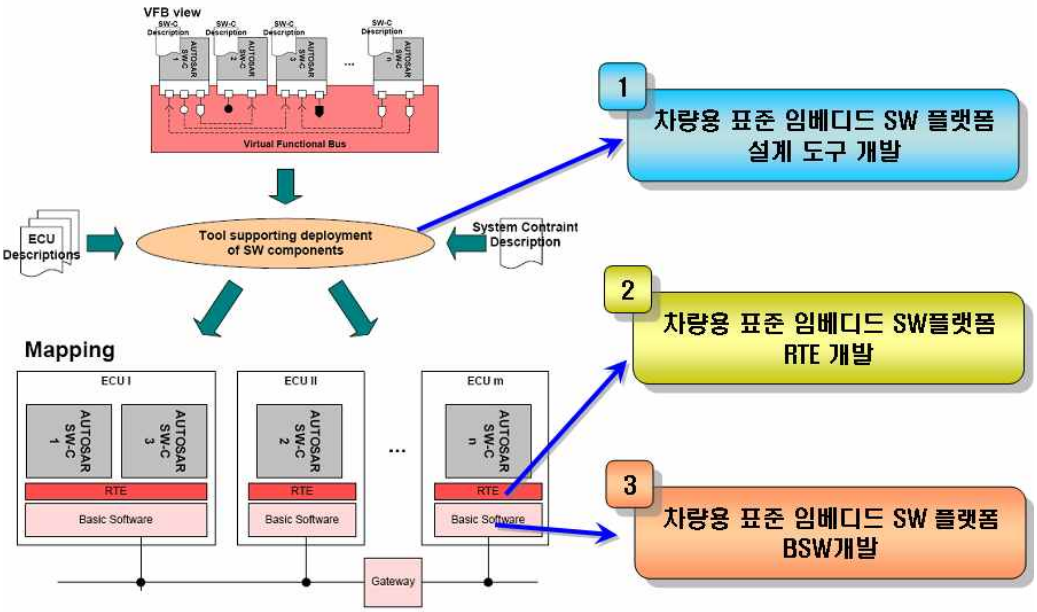


그림 3-1. 연구개발 최종 목표

- ITS를 위한 효율적인 인증 프로토콜 개발
- ITS를 위한 효율적인 통신 방안 연구

제 2 절 연구개발 내용

- 표준화된 차량용 소프트웨어 플랫폼 구조 연구
  - 컴포넌트 실행환경 개발
    ; AUTOSAR R3.0기반의 RTE 세부 기능 구현
  - OS 개발
    ; AUTOSAR OS 테스트 및 안정화
    ; OSEK OS 인증 획득

; 플랫폼 확장 및 상용화

· Safety Critical 서비스 개발

; 시스템 서비스 모듈 안정화(ECU State Manager)

; 내결함 통신 서비스 설계 및 구현(SM, NM, TP)

; 내결함 통신 서비스 안정화(COM, PDU Router)

· Safety Critical 하드웨어 추상화(Hardware Abstraction, HAL) 기법 연구

; 내결함 통신 하드웨어 추상화 계층 안정화

; 내결함 통신 드라이버 안정화

- 차량용 소프트웨어 플랫폼 설계 도구 개발

· ECU 환경 설정 도구 개발

; 다중 플랫폼 지원

· BSW 코드 생성 도구 개발

; BSW 요구사항 반영 코드생성 기능 개선

- 차량용 소프트웨어 플랫폼 기반 응용시스템 개발

· 지능형 인테리어 라이팅 시스템 개발

; AUTOSAR 아키텍처 적용 소프트웨어 설계 및 시스템 통합

- ITS를 위한 효율적인 인증 프로토콜 개발

· 효율적인 해쉬 체인 운행 알고리즘

; 해쉬 체인 운행 기법의 기존 연구 분석

; 일반화한 해쉬 체인 운행 알고리즘 설계

; 일반화한 해쉬 체인 운행 알고리즘 개발

· 양방 인증을 위한 거리 한정 알고리즘

; 거리 한정 알고리즘의 기존 연구 분석

; False accept rate를 최소화한 거리 한정 알고리즘 설계

; Mutual challenge를 이용한 거리 한정 알고리즘 설계

; relay 공격에 강한 거리 한정 프로토콜 개발

- ITS를 위한 효율적인 차량간 통신 방안 연구

· BDMA 기반의 V2I 통신 시스템

; 효율적인 초기 접속 및 충돌 회피 방안 연구

; 차량의 grouping을 이용한 자원할당 방안 연구

; BDMA jsthd 기술 기반의 차량 시스템에서 차량 스케줄링 방안
; BDMA 기반의 차량 통신 시스템에서 multi-hop relay 방안

· 인프라와 릴레이의 협력을 통한 재전송 기술
; 차량간 통신에서 C-ARQ mechanism의 성능 분석
; 차량간 통신에서 C-ARQ mechanism 설계

제 3 절 연구개발 수행 결과

1. 표준화된 차량용 소프트웨어 플랫폼 구조 연구

가. 컴포넌트 실행환경

(1) RTE컨텍스트 다이어그램 및 개요 설명

RTE Generator은 ECU Configuration Description에 설정된 정보에 근거하여 AUTOSAR VFB의 구현결과를 생성해주는 도구이다.

ECU Configuration Description은 한 ECU내에 있는 모든 BSW 모듈에 관련된 설정정보를 포함한다. 각 BSW 모듈 Generator은 관련된 설정데이터를 추출하여 필요한 모듈을 생성한다. RTE Generator은 Description파일로부터 다음과 같은 파라메터들을 입력받는다.

- SWC 및 instance에 관련된 정보
- Runnable entity 및 task에 관련된 정보
- Port 및 interface에 관련된 정보
- RTE 통신에 관련된 정보
- Data Consistency에 관련된 정보
- RTE configuration에 관련된 정보
- Measurement 및 calibration에 관련된 정보
- Mode Management에 관련된 정보

RTE생성은 Contract Phase와 Generation Phase 두 단계를 거치며, Contract Phase에서는 컴포넌트 API파일을 생성하고, Generation Phase에서는 RTE소스 및 헤더파일을 생성한다.

RTE Generator은 RTE Contract Phase에서 'SW-Component Internal Behavior Description'로부터 컴포넌트 API를 생성한다. SWC Internal

Behavior Description에는 SWC 타입과 포트, 인터페이스 및 Runnable entity, RTE Event 등이 정의된다. 생성된 컴포넌트 API는 RTE API와 RTE에 필요한 데이터 구조 등을 정의한다.

RTE Generator은 RTE Generation에서 'ECU Configuration Description'로부터 RTE소스코드 및 헤더파일을 생성한다. ECU Configuration Description은 ECU에 있는 모든 BSW 모듈의 설정을 포함하고 있다. 따라서 ECU Configuration Description파일은 여러 파일에 분산되어 있으며 SWC Description이나 System Configuration Description과 같은 파일을 참조하기도 한다. 생성된 RTE코드로는 Rte.c, Rte.h, Rte_Main.h, Rte_Cfg.h, Rte_Cbk.h, Rte_Hook.h, Rte_Type.h 등이 있다. 각 코드는 다음 기능을 포함하고 있다.

Rte.c: RTE의 기능을 구현한 코드로 소프트웨어 컴포넌트와 소프튼웨어 컴포넌트 사이, 소프트웨어 컴포넌트와 베이직 소프트웨어 사이 통신기능, Task Body생성기능 등을 포함한다.

생성된 RTE Generator은 최종 RTE Configurator과 연동하게 된다. RTE Configurator은 GUI화면에서 RTE Generator 호출메뉴/아이콘을 제공하며 RTE Configurator은 RTE와 관련된 설정정보, 예하면 VfbTrace설정과 같은 정보와 기타 설정정보를 RTE Generator로 넘겨준다.

RTE Generator은 각 단계에서 입력되는 설정정보 및 입력 파라메터를 체크하며 잘못된 정보 및 파라메터를 에러형식으로 출력한다.

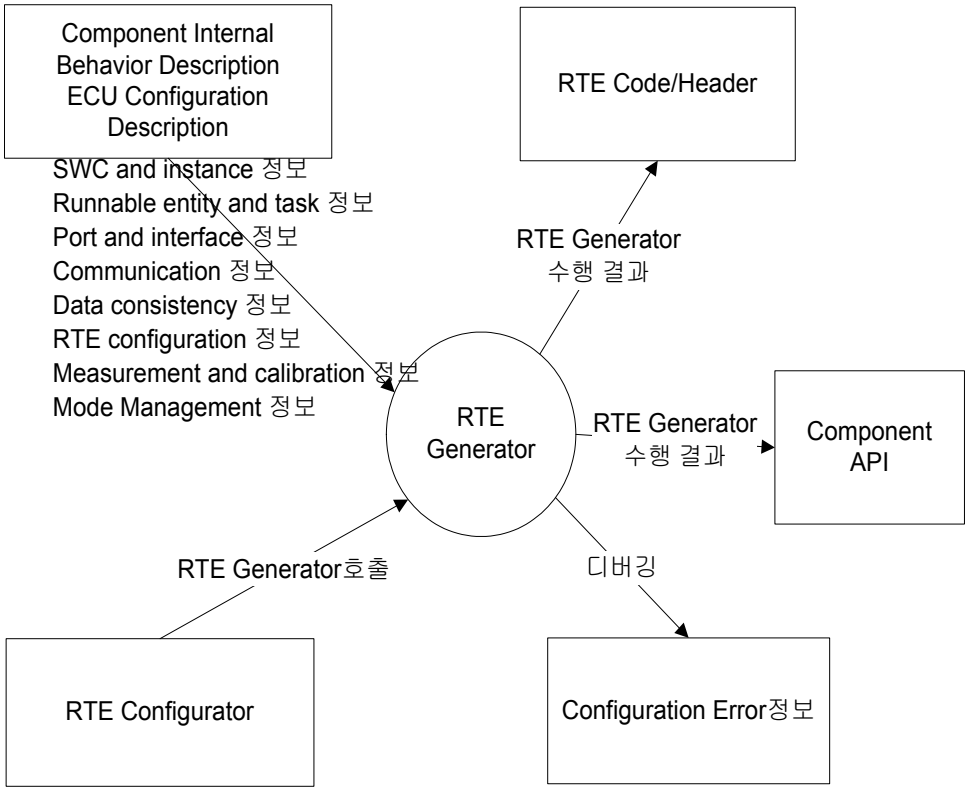


그림 3-2 RTE 생성기 컨텍스트 다이어그램

RTE Generator은 모든 RTE API 및 기능을 구현하기 위한 정보를 담은 XML형식의 Description파일을 입력받는다. AUTOSAR표준은 AUTOSAR정보표현을 위한 XML schema를 제공하여 XML파일을 통하여 제공되는 정보의 상호호환성을 보장한다. AUTOSAR구조를 설명하기 위하여 사용되고 있는 AUTOSAR meta-model은 매우 방대하며, AUTOSAR SW정보표현을 위한 AUTOSAR schema는 meta-model보다도 더욱 복잡하다. AUTOSAR schema는 AUTOSAR meta-model에 명확하게 표현되어 있지만 기타 추가적인 목적으로 사용되는 엘리먼트도 포함된다. RTE Generator구현에 있어서 입력되는 정보에 대한 분석이 선행되어야 하며 이를 위하여 AUTOSAR schema에 대한 정확한 분석이 필요하다. 하지만 AUTOSAR shcema에 대한 모든 분석은 방대한 자료뿐 아니라 읽기도 쉽지 않다. 따라서 본 문서에서는 RTE Generator에 관련된 AUTOSAR schema를 분석하며, RTE표준문서에 정의된 RTE Generator입력정보를 기반으로 AUTOSAR schema를 분석하도록 한다.

(2) 컴포넌트 실행환경 안정화 수행 목적

본 절에서는 응용프로그램을 설계하고 설계된 응용프로그램의 xml기술 명세서에 기반하여 Rte생성기능을 테스트함으로써 Rte생성기에서 지원되어야 할 기능이 정상적으로 동작하는지 여부를 확인하고 비정상적으로 동작하는 기능을 찾아내고 수정함으로써 설계, 개발된 Rte생성기가 안정적으로 요구명세서상의 기능을 지원할 수 있도록 한다. Rte클러스터에 대한 기능을 점검하기 위한 시험환경은 하드웨어 시스템 환경과 소프트웨어 시스템 환경으로 구성된다.

○ 하드웨어 시험환경

본 시험을 위한 소프트웨어 시스템 환경을 구축하기 위해 필요한 하드웨어 시스템 환경은 다음과 같다.

- 개발 PC: Intel Pentium 4 이상
- 개발 보드: Freescale사 EVB9S12XF512 제품
- Debugger: P&E사 USB Debugger

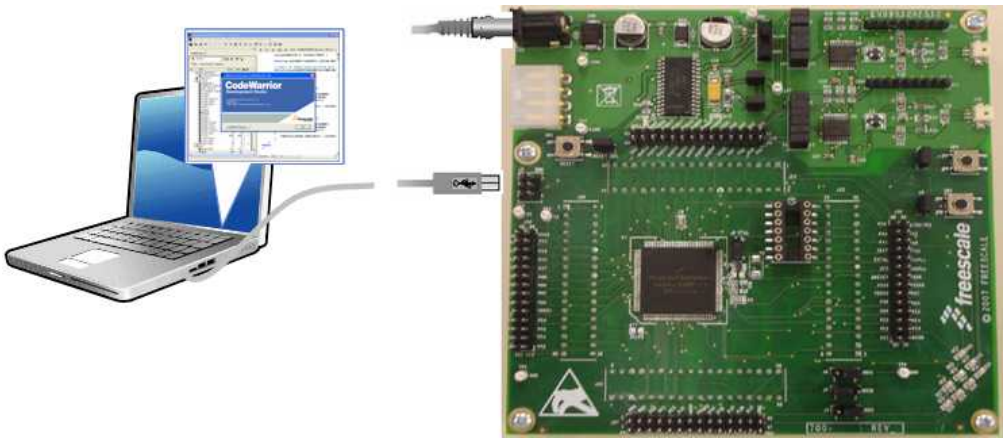


그림 3-3. RTE 클러스터 시험 환경

○ 소프트웨어 시험환경

본 시험을 위한 소프트웨어 시스템 환경은 다음과 같다.

- IDE: Metrowork사 CodeWarrior
- Perl: Active Perl 5.8.9 이상

(2) RTE클러스터 안정화

○ Runnable Entity 할당여부 체크 기능 안정화

- 기능

설 명	o Configurator에서 SW-C와 Runnable Entity를 설계하며 Runnable Entity를 SW-C에 할당한 후 설정을 xml기술 명세서에 저장한다. 만약 AUTOSAR SW-C 인스턴스의 Runnable Entity가 타스크에 할당되지 않은 경우, RTE생성기에서 이 설정을 체크해내는지 확인.

- 절차

안정화 수행 방법	(1) 시험 프로그램은 RTE Generator 프로그램과 RTE Generator에 사용될 XML파일로 구성된다. (2) XML파일은 테스트를 위한SW-C Description파일과 RTE 설정내용을 포함한다. (3) 테스트에 사용될 XML파일을 입력 파라메터로 전달하여 RTE파일을 생성한다. (4) Rte Generator을실행한 후 결과화면에서 미할당된 Entity에 대한 정보를 출력하는지를 확인한다.
세부 분석 및 수정	(1) 생성된 xml파일에 테스트에 적합한 내용의 타스크와 Runnable Entity가 설정되었는지 확인. 입력 xml파일에서는 display Runnable와 SendTask 2개의 타스크 Description을 포함하고 displayRunnable1과 SdrRunnable 및 errorRunnable을 포함함. (2) 입력파일에서 타스크에 할당된 Runnable entity와 타스크에 할당되지 않은 Runnable entity가 있는지 확인. display Runnable1은 display Runnable 타스크에 할당되고 SendTask는 SdrRunnable에 할당됨. errorRunnable은 할당되지 않음. (3) 할당되지 않은 runnable entity를 포함한 xml파일을 입력값으로 RTE Generator을실행하여 할당되지 않은

	runnable entity를 체크하는지 확인. RTE Generator실행 화면(도스)에서 에러메시지 출력.

○ Task body구성 기능 안정화

- 기능

설 명	o ECU Description파일에는 ECU관련 설정을 포함하고 있다. RTE에서 구성해야 할 Task Body는 OS에서 설정하며 설정결과는 ECU Description파일에 포함된다. RTE Generator가 ECU Description파일의 OS항목에서 설정한 항목대로 task body를 구성하는지 여부 확인.

- 절차

안정화 수행 방법	(1) 시험 프로그램은 RTE Generator 프로그램과 RTE Generator에 사용될 XML파일로 구성된다. (2) XML파일은 테스트를 위한 SW-C Description파일과 RTE설정내용을 포함한다. (3) 테스트에 사용될 XML파일을 입력 파라메터로 전달하여 RTE Generator파일을 생성한다. (4) XML파일의 Description내용과 RTE소스코드를 비교분석하여 Task body가 설정대로 생성되는지를 확인한다. (5) RTE소스코드에서 설정된 Task body가 제대로 생성되지 않았으면 XML처리모듈을 검사 및 수정한 후 (3), (4)과정을 반복한다.
세부 분석 및 수정	(1) 우선 Configurator을 통하여 OS에서 어떤 Task를 설정하였는지를 확인. 테스트에 사용된 어플리케이션에서는 displayRunnable와 SendTask 2개의 타스크를 설정함. 다음 Configurator로부터 생성된 xml파일에서 Configurator설정대로 Task body내용을 저장하였는지 확인함. xml파일에서는 displayRunnable와 SendTask 2개의 타스크가 설정됨.

	(2) 상기 Configurator파일을 이용하여 RTE생성기로부터 RTE코드를 생성함. 생성된 RTE소스파일에서 구성한 task body 내용 확인. RTE소스파일에서 displayRunnable와 SendTask 2개의 타스크가 생성되어 있음을 확인할 수 있음.

○ INIT_VALUE값 초기화 기능 안정화

- 기능

설 명	o 전장 소프트웨어 설정과정에서 각각의 소프트웨어 컴포넌트 상의 포트는 송/수신 할 데이터 엘리먼트를 동시에 설정한다. 이 포트의 데이터 엘리먼트에서 데이터의 초기값을 설정하고 이 값대로 데이터 초기화가 이루어지는지를 체크.

- 절차

안정화 수행 방법	(1) 시험 프로그램은 RTE Generator 프로그램과 RTE Generator에 사용될 XML파일로 구성된다. (2) XML파일은 테스트를 위한 SW-C Description파일과 RTE설정내용을 포함한다. (3) 테스트에 사용될 XML파일을 입력 파라메터로 전달하여 RTE Generator파일을 생성한다. (4) XML파일의 Description내용과 RTE Generator생성결과를 분석하여 초기화값이 설정대로 소스로 출력되는지 확인한다. (5) 설정에서 데이터 엘리먼트 초기값을 지정하였고 XML파일에서도 초기화값이 존재하지만 RTE소스코드에서 데이터 초기화가 이루어지지 않으면 RTE템플릿 및 XML처리모듈을 검사 및 수정 후, (3)~(4)과정을 반복하여 초기화 기능이 안정적으로 동작하도록 한다.
세부 분석	(1) 테스트에서 사용되는 응용프로그램에서 SenderSWc의

및 수정	Pport2에서 displayData1 데이터 엘리먼트의 초기값을 0으로 설정하도록 한다. 따라서 xml파일에서 설계 및 설정된 초기화값이 제대로 저장되었는지 확인한다. displayData1값은 0으로 초기화됨을 확인. (2) 초기화값이 정상적으로 저장된 xml파일을 입력파일로 하여 설계, 구현된 RTE생성기를 이용하여 RTE소스코드를 생성하고 생성된 Rte소스파일에서 displayData1값이 초기화되었는지 체크. 생성된 RTE코드의 Rte_Start()함수 내에서 displayData1 = 0으로 초기화됨을 확인.

○ ECU별 코드 생성 기능 안정화

- 기능

설 명	o 한 개의 전장소프트웨어 프로젝트는 1개 혹은 1개이상의 ECU상에서 동작하도록 설계 가능함. RTE는 각각의 ECU상의 소프트웨어 설정에 따라 생성됨으로 RTE생성기가 ECU설정에 따라 ECU별로 코드를 생성하는지 여부 확인.

- 절차

안정화 수행 방법	(1) 시험 프로그램은 RTE Generator 프로그램과 RTE Generator에 사용될 XML파일로 구성된다. (2) XML파일은 테스트를 위한 SW-C Description파일과 RTE설정내용을 포함한다. (3) 테스트에 사용될 XML파일을 입력 파라메터로 전달하여 RTE Generator파일을 생성한다. (4) 1개의 ECU Description파일로 1개의 Rte소스코드가 생성되는지 확인한다. (5) 1개의 ECU Description파일에서는 1개의 Rte소스코드만 생성되어야 함. (6) 2개의 ECU Description파일을 이용하여 2개의 Rte소스코드가 생성되는지 확인한다. 2개의 Description파일로부터 2개의 Rte소스코드가 생성되지 않으면 xml처리

	모듈에서 ECU별로 설정파일을 추출하지 못하는 것이므로 xml처리모듈을 수정한다.
세부 분석 및 수정	(1) 테스트에서 사용되어지는 응용프로그램을 2개의 ECU를 포함하도록 설계한다. 따라서 출력된 xml파일은 2개의 ECU에 대한 설정정보를 포함한다. xml파일에서 2개의 ECU정보가 저장되어 있음을 확인. (2) 1개의 ECU정보만을 분리하여 RTE생성기의 입력파일로 하여 RTE를 생성한다. 1개의 RTE코드가 생성되는 것을 확인. Node1 ECU설정파일 입력으로 Node1 RTE 코드가 생성됨. (3) 2개의 ECU정보가 포함되어 있는 ECU Description파일을 입력으로 하여 RTE생성기로부터 RTE코드를 생성. RTE에서 2개의 RTE코드를 생성하는 것을 확인.

○ 동일 입력에 대한 동일 출력 기능 안정화

- 기능

설 명	o RTE생성기는 동일한 설정에 대하여 동일한 RTE소스코드를 생성해야만 전장 소프트웨어 동작의 안정성을 보장할 수 있음. RTE Generator에서 동일한 입력파일을 입력 시 동일한 소스파일을 생성하는지 체크.

- 절차

안정화 수행 방법	(1) 시험 프로그램은 RTE Generator 프로그램과RTE Generator에 사용될 XML파일로 구성된다. (2) XML파일은 테스트를 위한 SW-C Description파일과 RTE설정내용을 포함한다. (3) 테스트에 사용될 XML파일을 입력 파라메터로 전달하여 RTE Generator파일을 생성한다. (4) 동일한 XML파일을 입력으로 Rte Generator를 여러 번

	실행하여 생성된 소스가 동일한지 확인한다. (5) 응용프로그램 설정을 변경하고 변경내용을 XML파일로 저장한 후 (3)~(4)과정을 반복하여 확인한다.
세부 분석 및 수정	(1) 우선 동일한 설정에 대하여 동일한 XML Description파일로 저장되는지 확인한다. 우선 설정파일을 Description1.xml파일로 저장한 후 설정을 변경하지 않고 Description2.xml파일로 저장한다. 2개의 기술명세서 내용이 완전히 일치한지를 확인함. (2) 설계, 개발된 RTE생성기를 이용하여 Description1.xml 파일과 Description2.xml파일에 대해 각각 Rte1.c와 Rte2.c, Rte1.h와 Rte2.h 등 RTE코드를 생성한다. 생성된 RTE코드를 비교하여 생성된 코드가 완전히 일치하는지 확인함. (3) 생성된 파일이 동일한 파일임을 확인. 설정을 변경하여 새로운 Description파일로 상기 과정을 반복하여 RTE 코드를 생성하고 동일한 입력에 대하여 동일한 코드가 생성됨을 확인함.

○ ANSI C/ISO C++ 지원 기능 안정화

- 기능

설 명	o 생성된 RTE에서 C/C++ 지원하는지 체크. 체크내용은 RTE 소스코드가 C/C++프로그램언어로 작성되었는지 와 파일확장명, 타입정의 등 문법이 C/C++문법에 부합되는지 확인.

- 절차

안정화 수행 방법	(1) 시험 프로그램은 RTE Generator 프로그램과RTE Generator에 사용될 XML파일로 구성된다. (2) XML파일은 테스트를 위한 SW-C Description파일과 RTE설정내용을 포함한다. (3) 테스트에 사용될 XML파일을 입력 파라메터로 전달하여 RTE Generator파일을 생성한다. (4) RTE Generator을 이용하여 생성된 Rte코드가 C/C++

	지원에 대한 내용을 포함했는지 확인한다.
세부 분석 및 수정	(1) RTE생성기로 xml 기술명세서를 이용하여 RTE코드를 생성함. 생성된 RTE코드의 AUTOSAR 타입 헤더파일에서 각각의 타입이 typedef으로 정의되는 확인. Typedef을 사용하여 타입이 정의됨을 확인. (2) 생성된 RTE코드 중에서 메인RTE파일은 C언어를 지원하는 확장자로 되어있는지 확인. Rte생성파일의 명칭이 RTE.c임을 확인. (3) Rte는 C언어형식으로 작성되어야 함. 생성된 RTE소스코드가 C언어문법에 부합되는지 확인하고 코드워리어 임베디드 C컴파일러를 이용하여 문법상의 오류가 없는지 확인함. 생성된 Rte는 C언어로 작성됨을 확인.

○ Rte생성기 입력파일 지원 기능 안정화

- 기능

설 명	o Rte Generator은 0개 혹은 그 이상의 파일을 입력받아 처리하고 입력파일 설정에 따라 Rte를 생성 할 수 있어야 한다.

- 절차

안정화 수행 방법	(1) 시험 프로그램은 RTE Generator 프로그램과RTE Generator에 사용될 XML파일로 구성된다. (2) XML파일은 테스트를 위한 SW-C Description파일과 RTE설정내용을 포함한다. (3) 테스트에 사용될 XML파일을 입력 파라메터로 전달하여 RTE Generator파일을 생성한다. (4) 0개의 xml파일을 입력값으로 하여 RTE를 생성 시, 에러없이 Rte코드가 생성되는지 확인한다. (4) 1개이상의 xml파일을 입력값으로 RTE를 생성 시에도

	에러없이 Rte코드가 생성되는지 확인한다.
세부 분석 및 수정	(1) 우선 0개의 xml입력에 대해서도 RTE생성기가 정상적으로 RTE코드를 생성하는지 여부를 체크한다. 0개의 xml 파일을 입력 시 RTE생성기는 어떠한 설정정보도 추출할 수 없으므로 Task Body, Communication API 등을 생성할 수 없고, RTE Lifecycle API만을 포함해야 한다. (2) 0개의 입력파일을 입력으로 생성된 RTE소스코드에는 RTE Lifecycle API만을 포함하고 기타 ECU설정과 관련된 코드는 생성되지 않음을 확인. (3) 테스트를 위한 어플리케이션을 설정하고 저장된 xml파일을 입력으로 하여 RTE소스코드를 생성한다. RTE생성기는 설정에 따라 정상적으로 RTE소스코드를 생성함을 확인.

○ 데이터 타입 재정의 방지 기능 안정화

- 기능

설 명	o 생성된 여러 개의 RTE파일에서 데이터 재 정의로 인한 오류를 방지하기 위하여 RTE 생성기는 기 정의된 데이터 타입의 재 정의를 방지해야 한다.

- 절차

안정화 수행 방법	(1) 시험 프로그램은 RTE Generator 프로그램과RTE Generator에 사용될 XML파일로 구성된다. (2) XML파일은 테스트를 위한 SW-C Description파일과 RTE설정내용을 포함한다. (3) 테스트에 사용될 XML파일을 입력 파라메터로 전달하여 RTE Generator파일을 생성한다.

	(4) 생성된 어플리케이션 헤더파일에서 데이터 타입 재정의 방지내용이 포함되었는지 확인한다. (5) 생성된 어플리케이션 헤더파일에 데이터 타입 재정의 방지내용이 없으면 템플릿 파일에서 정확하게 정의되었는지 확인 및 수정하고 (3)~(4)과정을 반복한다.
세부 분석 및 수정	(1) 테스트를 위한 어플리케이션을 설계하고 생성된 xml파일을 이용하여 RTE코드를 생성한다. 소프트웨어 컴포넌트 수와 같은 어플리케이션 헤더파일이 생성되는지 확인한다. (2) 각각의 어플리케이션 헤더파일에 어플리케이션 헤더파일에서 Rtetype.h Include하였는지 여부 확인. (3) Rtetype.h에서 #ifdef ~ #define~#end를 이용하여 데이터 재정의를 방지하였는지 확인.

○ Runnable Entity 순차적 활성화 기능 안정화

- 기능

설 명	o RTE설정에서 SW-C는 여러 개의 Runnable Entity를 포함할 수 있으며 설정 시 순서에 따라 SW-C에 할당된다. RTE생성기는 설정 Description에 저장된 Runnable Entity의 순서대로 Runnable entity를 활성하는지 여부 체크.

- 절차

안정화 수행 방법	(1) 시험 프로그램은 RTE Generator 프로그램과RTE Generator에 사용될 XML파일로 구성된다. (2) XML파일은 테스트를 위한 SW-C Description파일과 RTE설정내용을 포함한다. (3) 테스트에 사용될 XML파일을 입력 파라메터로 전달하여 RTE Generator파일을 생성한다. (4) XML파일 내의 Runnable entity정의순서와 Task body

	내의 Runnable entity생성순서가 일치한지 확인한다. (5) 설정순서대로 Runnable entity가 생성되지 않으면 xml로부터 Runnable entity정보를 추출 시 순서정보를 포함하지 않았을 가능성이 크므로 xml모듈에서 Runnable entity순서정보를 추출하는 API를 수정 혹은 추가 후 (3)~(4)과정을 반복한다.
세부 분석 및 수정	(1) 테스트에 사용될 어플리케이션을 설정하고 xml기술 명세서에 설정내용을 저장한다. 본 테스트에 사용하기 위하여 한개의 소프트웨어 컴포넌트에는 2개의 Runnable entity를 설정 및 매핑한다. 저장된 xml파일은 설정한 소프트웨어 컴포넌트, Runnable entity 및 Runnable entity순서정보를 정확힌 저장하였음을 확인. (2) 저장된 xml파일을 이용하여 RTE코드를 생성한다. 생성된 RTE코드의 Task Body코드 중에서 Runnable entity가 설정한 순서대로 호출되는지 확인. Runnable entity가 Task body내에서 순서대로 호출됨을 확인 (3) 소프트웨어 컴포넌트 내의 Runnable entity 수량 및 순서를 변경하고 테스틀 반복하면서 설정한 순서대로 Runnable entity가 생성되는지 확인. 다양한 테스트에서 Runnable entity가 설정변경에 따라 RTE코드에서의 호출순서가 설정에 맞게 변경됨을 확인.

○ Runnable Entity 초기화/종료 기능 안정화

- 기능

설 명	o RTE는 초기화/종료 API를 생성함으로써 하위 서비스 컴포넌트에서 RTE를 호출 및 종료할 수 있도록 해야 한다. Rte생성기에서 Rte 초기화/종결 메커니즘을 제공하는지 여부 체크.

- 절차

안정화 수행 방법	(1) 시험 프로그램은 RTE Generator 프로그램과 RTE Generator에 사용될 XML파일로 구성된다. (2) XML파일은 테스트를 위한 SW-C Description파일과 RTE설정내용을 포함한다. (3) 테스트에 사용될 XML파일을 입력 파라메터로 전달하여 RTE Generator파일을 생성한다. (4) 생성된 Rte소스코드에 Rte시작함수와 종료함수를 포함하는지 확인한다. (5) RTE시작 및 종료 API에서 RTE를 시작하기 위한 시스템 자원할당 및 초기화 부분이 포함되었는지 확인.
세부 분석 및 수정	(1) Rte생성기에서 Rte_Start()함수를 생성하는지 여부 체크. Rte_Start()함수 생성. (2) Rte생성기에서 Rte_Stop()함수를 생성하는지 여부 체크. Rte_Stop()함수 생성.

○ 입력파일 및 생성파일 기능 안정화

- 기능

설 명	o Rte 생성기는 각종 기술서 파일을 입력받아 템플릿에 적용 후 여러 Rte파일을 생성한다.

- 절차

안정화 수행 방법	(1) 시험 프로그램은 RTE Generator 프로그램과 RTE Generator에 사용될 XML파일로 구성된다. (2) XML파일은 테스트를 위한 SW-C Description파일과 RTE설정내용을 포함한다. (3) 테스트에 사용될 XML파일을 입력 파라메터로 전달하여

	RTE Generator파일을 생성한다. (4) XML입력파일을 입력받아 요구하는 Rte소스코드를 각각 생성하는지 체크한다.
세부 분석 및 수정	(1) Rte.c생성여부 체크. Rte.c를 생성 확인. (2) Rte_Type.h헤드파일 생성여부 체크. Rte_Type.h파일 생성 확인. (3) Rte_Cbk.h헤드파일 생성여부 체크. Rte_Cbk.h파일 생성 확인. (4) Rte_Cfg.h헤드파일 생성여부 체크. Rte_Cfg.h파일 생성 확인. (5) 기술서 파일의 설정내용대로 파라메터에 저장되는지 파라메터를 출력하여 확인. 설정내용대로 파라메터가 설정됨을 확인. (6) 템플릿 파일에서 파라메터를 입력 혹은 파라메터로 전달받는지 확인. 저장된 파라메터와 템플릿에서 사용되는 파라메터가 동일함.

○ 컴포넌트 리스트 생성API 기능 안정화

- 기능

설 명	o RTE생성기는 xml파일을 처리하여 전장 소프트웨어 설정정보를 추출하고 필요한 RTE소스코드를 생성한다. 본 기능은 전장 소프트웨어 설정과정에서 설정한 컴포넌트정보를 추출하고 필요한 데이터 포맷으로 저장하는 기능을 확인.

- 절차

안정화 수행 방법	(1) 시험 프로그램은 RTE Generator 프로그램과 RTE Generator에 사용될 XML파일로 구성된다. (2) XML파일은 테스트를 위한 SW-C Description파일과 RTE설정내용을 포함한다.

	(3) 테스트에 사용될 XML파일을 입력 파라메터로 전달하여 RTE Generator파일을 생성한다. (4) 프로그램 메인루프에서 컴포넌트 리스트 API 호출 후 컴포넌트 리스트 해시데이터를 출력하여 설정한 컴포넌트 정보가 데이터에 저장되었는지 확인한다. (5) 설정한 컴포넌트 명을 이용한 어플리케이션 헤더파일이 생성되었는지 확인한다. (6) RTE소스크도에서 컴포넌트의 통신API가 설정한 컴포넌트에 따라 생성되었는지 확인한다.
세부 분석 및 수정	(1) 어플리케이션 소프트웨어 컴포넌트를 설정하고 xml기술명세서에 저장한 후 이를 입력으로 RTE생성기를 이용하여 RTE를 생성한다. RTE코드가 생성됨. (2) RTE생성기의 메인루프에서 컴포넌트 리스트 API를 호출한 후 컴포넌트 정보를 출력함. 출력된 컴포넌트 정보에 설정한 컴포넌트 정보가 출력됨을 확인. (3) RTE생성기를 이용하여 RTE코드를 생성하고 생성된 RTE코드에 설정된 소프트웨어 컴포넌트 명을 이용한 어플리케이션 헤더파일이 있는 것을 확인.

○ 데이터 타입 생성 API 안정화

- 기능

설 명	o RTE는 표준 타입 헤더파일에 정의되어 있는 데이터 타입 뿐만 아니라 사용자가 base타입에 기반하여 정의한 데이터 타입도 정의하며 본 API에서 소프트웨어 설정과정에서 설정한 데이터 타입에 따라 RTE에서 데이터 타입을 정의하는지 확인.

- 절차

안정화 수행 방법	(1) 시험 프로그램은 RTE Generator 프로그램과 RTE Generator에 사용될 XML파일로 구성된다.

	(2) XML파일은 테스트를 위한 SW-C Description파일과 RTE설정내용을 포함한다. (3) 테스트에 사용될 XML파일을 입력 파라메터로 전달하여 RTE Generator파일을 생성한다. (4) 설정에 따른 데이터 타입이 생성되지 않으면 설정된 데이터 타입의 base타입에 근거하여 Array, Bool, char 등 서브 함수 API를 수정하고 RTE를 다시 생성 및 확인한다.
세부 분석 및 수정	(1) 전장 소프트웨어 설정에서 데이터 엘리먼트에 사용될 데이터 타입을 정의한다. (2) 정의한 데이터 타입에 따라 xml파일에 데이터 타입이 정확하게 정의되었는지 확인. (3) xml파일을 이용하여 RTE코드를 생성. 생성된 코드의 datatype.h파일에 설정한 데이터 타입종류가 있는지 확인. 설정에 따라 데이터 타입이 정의되어 있는 것을 확인. (4) 데이터 타입 정의에서 사용된 base타입을 변경하여 RTE를 다시 생성한다.. 생성된 RTE datatype.h에서 변경값에 따라 정의된 데이터 타입의 basetype이 변경되는지 확인. 설정된 데이터 타입의 변경에 따라 datatype.h코드도 변경됨을 확인.

○ 포트 리스트 생성 API 안정화

- 기능

설 명	o 소프트웨어 컴포넌트는 외부와의 통신 시 소프트웨어 컴포넌트에 정의되어 있는 포트를 통하여 데이터를 주고 받는다. 포트 리스트 생성 API는 설정정보에 맞게 모든 포트관련 RTE API를 안정적으로 생성하는지 확인.

- 절차

안정화 수행 방법	(1) 시험 프로그램은 RTE Generator 프로그램과 RTE Generator에 사용될 XML파일로 구성된다. (2) XML파일은 테스트를 위한 SW-C Description파일과 RTE설정내용을 포함한다. (3) 테스트에 사용될 XML파일을 입력 파라메터로 전달하여 RTE Generator파일을 생성한다. (4) 생성된 RTE파일에서 포트와 관련된 통신 API 등 RTE 코드가 정상적으로 생성되는 확인. 정상적으로 생성되지 않으면 Rte템플릿 파일에서 통신 API가 정상적으로 구성되었는지와 포트리스트 생성 API에서 포트리스트를 정상적으로 추출하는지를 체크.
세부 분석 및 수정	(1) 테스트를 위하여 2개의 소프트웨어와 각각의 소프트웨어에 Send포트 및 Receive포트를 가지고 데이터를 주기적으로 전송하는 어플리케이션을 설계하고 설정한 후 xml파일로 저장. xml파일이 정상적으로 생성됨. (2) 생성된 xml파일을 open하여 설정한 내용대로 포트정보가 저장되었는지 확인. 포트정보가 정상적으로 저장됨. (3) xml파일을 이용하여 RTE코드를 생성. 생성된 코드의 Rte_send API와 Rte_Recive API의 구성이 저장된 포트정보에 따라 구성되었는지 확인. xml파일에 저장된 내용대로 RTE Comm. API를 구성함을 확인.

○ 타스크 리스트 생성 API 안정화

- 기능

설 명	o RTE task body 등을 구성하기 위하여 타스크 리스트 API를 이용하여 RTE task와 관련된 정보를 정확히 추출하여 Task body를 구성해야 함..

- 절차

안정화 수행 방법	(1) 시험 프로그램은 RTE Generator 프로그램과 RTE Generator에 사용될 XML파일로 구성된다. (2) XML파일은 테스트를 위한 SW-C Description파일과 RTE설정내용을 포함한다. (3) 테스트에 사용될 XML파일을 입력 파라메터로 전달하여 RTE Generator파일을 생성한다. (4) 생성된 RTE파일에서 Task body가 설정정보에 따라 정확하게 구성되었는지 확인. 만약 정상적으로 구성되지 않았으면 xml파일정보를 확인하여 데이터가 정확하게 저장되었는지 확인. 다음 RTE이벤트, Runnable entity, Runnableentity mapping등 타스크 리스트 정보 처리 API가 정확하게 구현되는지 확인 및 수정하고 다시 테스트.
세부 분석 및 수정	(1) RTE에서 필요한 Task를 설정하고 설정한 Runnable entity를 Task에 적절하게 매핑한 후 설정정보를 xml파일로 저장함. xml파일로 저장됨. (2) 생성된 xml파일을 open하여 설정한 내용대로 타스크 리스트정보가 저장되었는지 확인. 타스크 리스트정보가 정상적으로 저장됨. (3) xml파일을 이용하여 RTE코드를 생성. 생성된 RTE코드에 Taskbody코드가 포함되어 있는지 확인. RTE모듈에서 Taskbody API가 포함되어 있음을 확인. Taskbody API가 소프트웨어 설정에 맞게 생성됨을 확인. () TaskBody내에서 할당된 Runnable entity가 정상적으로 호출되는지 확인. Taskbody는 자기의 영역속에서 Runnablen entity를 호출함을 확인.

나. OS 개발

(1) AUTOSAR OS 테스트 및 안정화

(가) AUTOSAR OS 안정화의 필요성

소프트웨어 및 하드웨어 기술의 발전으로 지난 몇 년간 자동차 산업은 많은 변화를 거듭하고 있고 향후 수십 년 간 이러한 변화는 계속될 것으로 예측된다. 이러한 차량용 소프트웨어는 일반적인 소프트웨어와는 달리 사용자의

안전과 연계되어 있어 안정성 및 신뢰성이 더욱 강조되어 진다. 부동 소수 계산 오류로 인한 아리안 5호 폭발사건은 SW 신뢰성 및 품질문제에 대한 중요성을 잘 나타내주고 있다.



그림 3-4. 부동소수 계산 오류로 인한 아리안 5호 폭발 사건

이러한 문제해결을 위해서는 소프트웨어 개발 초기단계에서 충분한 테스팅을 수행함으로써 SW 안정화 및 신뢰성을 향상시킬 수 있다.

(나) 소프트웨어 테스팅 개요

현재 소프트웨어 개발 과정에서 시행 되고 있는 소프트웨어 테스트의 분류는 기준에 따라 다양하게 존재하고 있으며, 이를 각각 기준에 따라 분류하자면 표 2와 같다.

분류	테스트	내용
테스트 단계	단위(Unit test)/모듈(Module test)	컴파일된 독립 모듈의 완전성 테스트
	통합 테스트(Integration test)	모듈들을 결합하여 하나의 시스템을 만드는 테스트
	시스템 테스트(System test),	통합된 시스템의 테스트
	인수 테스트(Acceptance test)	사용자가 수용할 만한 품질을 갖추었는지 여부를 테스트
	설치 테스트(Installation test)	현장에 시스템을 설치하여 가동시켜 보면서 하는 테스트
목적	기능 테스트(Functional test)	주어진 입력에 대해 기대되는 출력이 제공되는지 테스트
	성능 테스트(Performance test)	응답속도, 처리량, 메모리 관리 능력, 처리속도 등을 테스트
	스트레스 테스트(Stress test)	과다한 정보 거래량이 부과되거나, 최저 조건 미달 혹은 최고 조건 초과의 상황, 물리적인 충격이나 변화 시의 반응 정도로 신뢰성을 테스트
	구조 테스트(Structure test)	소프트웨어에 내재되어 있는 논리 경로의 복잡도를 평가모듈 테스트, 통합 테스트 시스템 테스트에 광범위하게 적용되는 테스트
시각	검증(Verification)	개발자 혹은 테스트자의 시각
	확인(Validation)	사용자 시각
기법	블랙박스 테스트(black box test)	소프트웨어 내면은 알 수 없는 블랙박스로 규정하고, 외부에서 기능과 성능 등을 테스트
	화이트박스 테스트(white box test)	소프트웨어 내면을 테스트
작동 방식	동적 테스트(Dynamic test)	소스코드를 직접 실행시키면서 테스트
	정적 테스트(Static test)	소스코드를 실행시키지 않으며 테스트

표 3-1. 소프트웨어 테스팅의 분류

이 같은 다양한 테스트의 분류 중에서 테스트 단계와 목적에 대한 분류가 테스트 방법론의 변화를 주는 것은 아니다. 즉, 개발이 진행됨에 따라 단위 테스트와 통합 테스트를 수행하면 되고, 릴리즈와 시스템 통합의 일정에 따라 인수 테스트와 설치 테스트를 수행하면 되며, 목적에 따라 테스트 케이스를 달리하여 테스트함으로써 기능 테스트, 성능 테스트, 스트레스 테스트 등을 수행하면 되는 것이다. 테스터들이 최선을 다하여 작업을 수행함에도 불구하고 이렇게 간단히 말할 수 있는 이유는, 그 모든 것들이 기법으로 보자면 동적 테스트에 한정되어 있다는 이유에서다. 더욱 세분화 하자면, 내부의 소프트웨어 구조는 모르고, 계획해 놓은 동작을 실행해 보면서 문제점을 발견하는 '동적 블랙박스 테스트(Dynamic black box test)'만을 수행하고 있는 것이다.

블랙박스 테스트는 소프트웨어 내부의 구조를 모른 채 외부에서 보이는 것으로만 검사를 하는 것으로 테스터가 볼 수 있는 것은 입력 값과 출력 값, 그리고 개발 명세서 등의 문서들이다. 반대로 화이트박스 테스트의 경우, 소프트웨어 내부를 보면서 테스트하는 것으로 아키텍처에서 소스코드 한 라인, 한 라인까지 섬세하게 살펴보며 테스트하는 방법이다. 또한 그 둘의 분류 안에서

해당 소스코드를 컴파일 및 실행시키면서 테스트를 진행하는 지의 유무에 따라 동적 테스트와 정적 테스트로 나뉘게 된다. 보다 더 자세히 살펴보면 다음과 같다.

① 동적 블랙박스 테스트(Dynamic black box test): 실제 수행하며 테스트 하는 의미로 동작 테스트(Behavioral test)라고도 한다. 이는 테스트 케이스를 정하고 이에 따라 작동을 시켜보면서 문제점을 발견한다. 이 중 성공/실패 테스트 방법론으로 각각 나누어지며, 전자는 최소한의 정상적 수행과 완료의 유무를 검사하는 부분이며, 후자는 하드코어한 작동에서 발생하는 문제를 검출하기 위한 방법을 의미한다. 당연히 이 세상에 존재하는 모든 입력 값을 대입시킬 수는 없기 때문에 '적당한' 테스트 케이스 수립이 필수 요소이며, 그 기술로는 동등 분할(equivalence partitioning), 경계 값 분석(boundary condition), 실패를 위한 상태 테스트(반복, 스트레스, 부하) 등이 있다.

② 정적 블랙박스 테스트(Static black box test): 소스코드를 파악 할 수도 없고, 실행시키지도 않는 검사로 명세서 테스트가 이에 해당한다. 제품 개발의 방향을 잡아주는 개발 명세서의 완결성, 정확성, 정밀성, 일관성, 연관성, 실행 가능성, 코드와의 무관성, 테스트 가능성 등을 판단한다.

③ 동적 화이트박스 테스트(Dynamic white box test): 자동으로 프로그램이 실행되는 스크립트(script)를 작성하여 소스코드를 보면서 작동을 검사한다. 오류부분의 격리 측면에서 테스트 영역의 동적 화이트박스 테스트와 프로그래밍 영역의 디버거(Debugger)와 유사한 부분이 있으나, 그 검출의 목표와 추후 작업의 측면에서 다른 점이 있다.

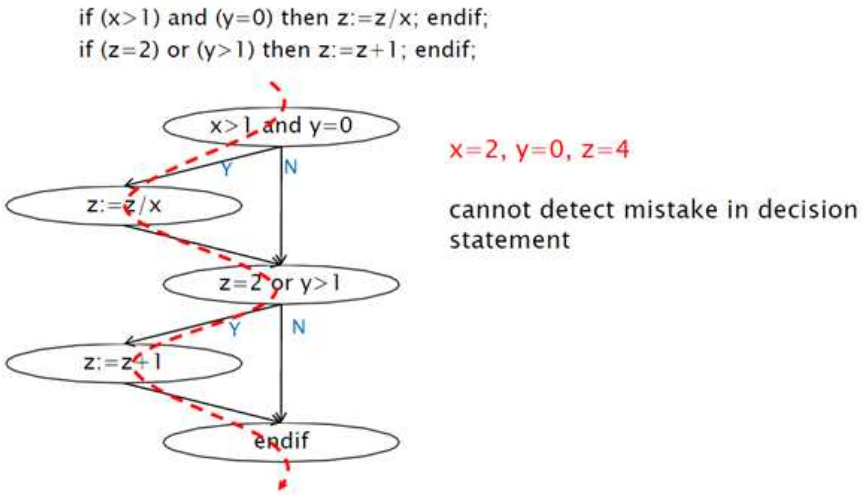
④ 정적 화이트박스 테스트(Static white box test): 프로그램이 실행되지 않은 상태에서도 대상 소프트웨어의 설계, 아키텍처, 코드 등에서 상세하게 버그를 찾을 수 있는 방법으로, 방법론적으로는 동료 검토(Peer Reviews), 워크스루(Walkthroughs), 검사(Inspections)등이 있다. 본 테스트를 위해서는 일반적으로 데이터 참조 오류, 데이터 선언 오류, 연산 오류, 비교 오류, 제어흐름 오류, 서브루틴 파라미터 오류, 입출력 오류 등과 같은 코드 검토 체크리스트를 사전에 정해 놓고 검사한다.

(다) 동적 화이트박스 테스트(Dynamic White box Test)

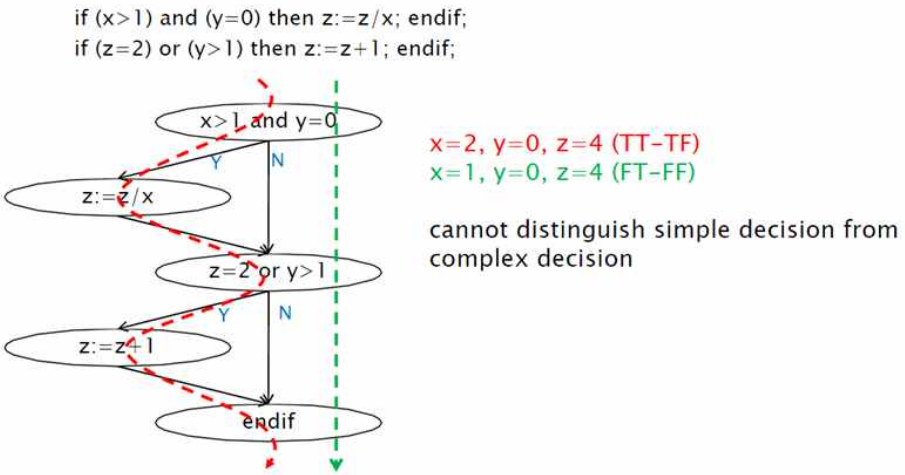
동적화이트박스 테스트는 소스코드를 보면서 프로그램을 테스트하는 방법으로 일반적으로 테스트슈트(Test Suite)를 이용해서 대량의 테스트케이스(Test

case)를 적용하여 테스트를 수행한다. 이때 테스트의 달성정도를 나타내는 것으로 커버리지(Coverage)라는 개념을 사용하며 커버리지 테스트의 종류는 다음과 같다

① 구문 커버리지(Statement Coverage) : 테스트슈트에 의해 실행된 구문이 몇% 인지를 측정하는 것으로 적은 개수의 테스트케이스로 쉽게 달성가능하나 코드상의 많은 부분을 체크할 수 없는 보장성이 낮은 커버리지 이며 포함관계가 더 큰 커버리지를 달성하면 저절로 달성됨.

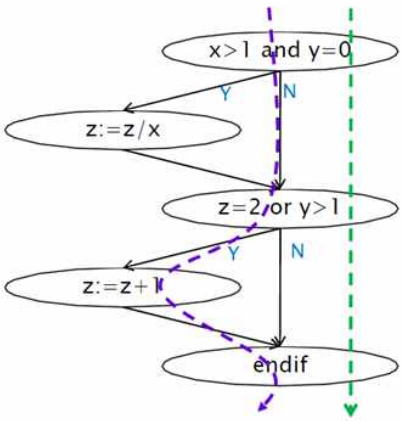


② 분기 커버리지(Decision Coverage) : 분기문의 가능한 결과값(참, 거짓)이 테스트케이스에 의해서 한번이상 수행되었는지 판단



③ 조건 커버리지(Condition Coverage) : 결정문내의 모든 조건의 결과값(참, 거짓)이 테스트케이스에 의해서 한번 수행되는지 판단

```
if (x>1) and (y=0) then z:=z/x; endif;
if (z=2) or (y>1) then z:=z+1; endif;
```

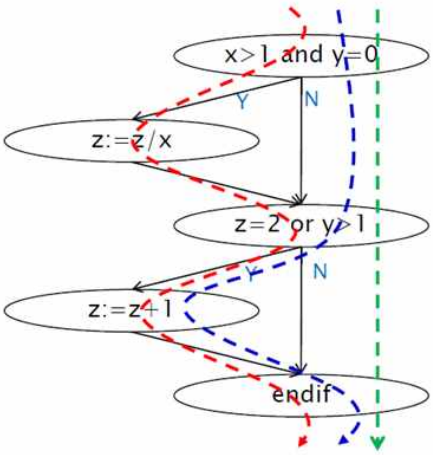


x=2, y=2, z=2 (TF-TT)
x=1, y=0, z=4 (FT-FF)

cannot satisfy even statement coverage

④ 변경 조건/분기 커버리지(MC/DC) : 각개별 조건식이 다른 개별 조건식에 영향을 받지 않고 전체 조건식의 결과에 독립적으로 영향을 주도록 함으로써, 조건/결정 커버리지를 향상 시킨 것.

```
if (x>1) and (y=0) then z:=z/x; endif;
if (z=2) or (y>1) then z:=z+1; endif;
```



x=2, y=0, z=4 (TT-TF)
x=2, y=2, z=4 (TF-FT)
x=1, y=0, z=4 (FT-FF)

for decisions with a large number of inputs, MC/DC requires considerably more test cases than C/DC

⑤ 다중 조건 커버리지(Multiple Condition Coverage) : 분기문내의 각 모든 개별 조건식의 모든 가능한 논리적인 조합을 고려한 강력한 커버리지로 테스트케이스의 양이 방대해지므로 현실적으로 매우 간단한 SW 구조가 아니면 달성하기 어려움.

본 과제에서는 차량용 AUTOSAR OS의 안정화를 위해 동적테스팅 방법으로 커버리지 테스트를 수행하였고 이를 위해서 (주)슈어소프트에서 공급하는 CodeScroll Controller 도구를 사용하였다. CodeScroll Controller는 문장, 분기, MC/DC 커버리지 외에도 함수 호출(Function Call) 관계를 GUI로 보여주며 소스코드를 분석하여 초기 테스트케이스를 자동생성해 주는 기능을 가지고 있다.

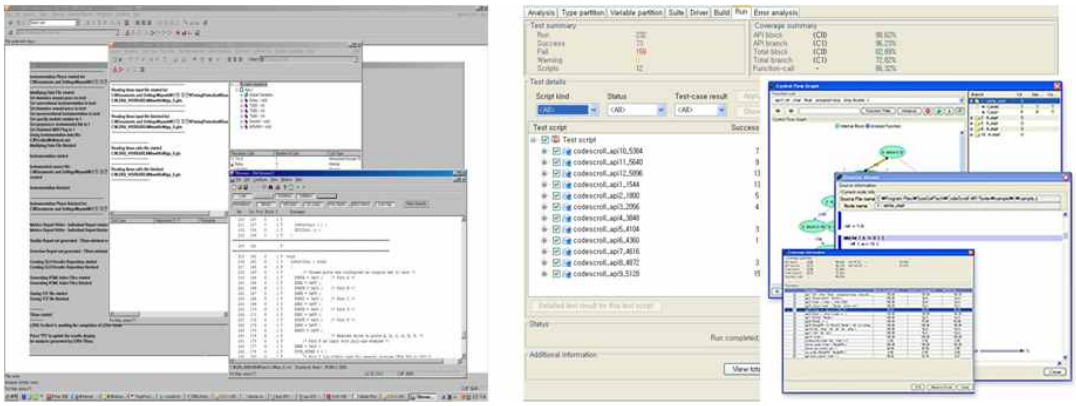


그림 3-5. CodeScroll Controller 실행 화면

동적테스트는 AUTOSAR OS의 소스파일을 대상을 실시하였으며 Block(Statement), Branch(Decision), MCDC 테스트 커버리지 달성을 목적으로 수행되었다. 수행결과는 다음 그림과 같다.

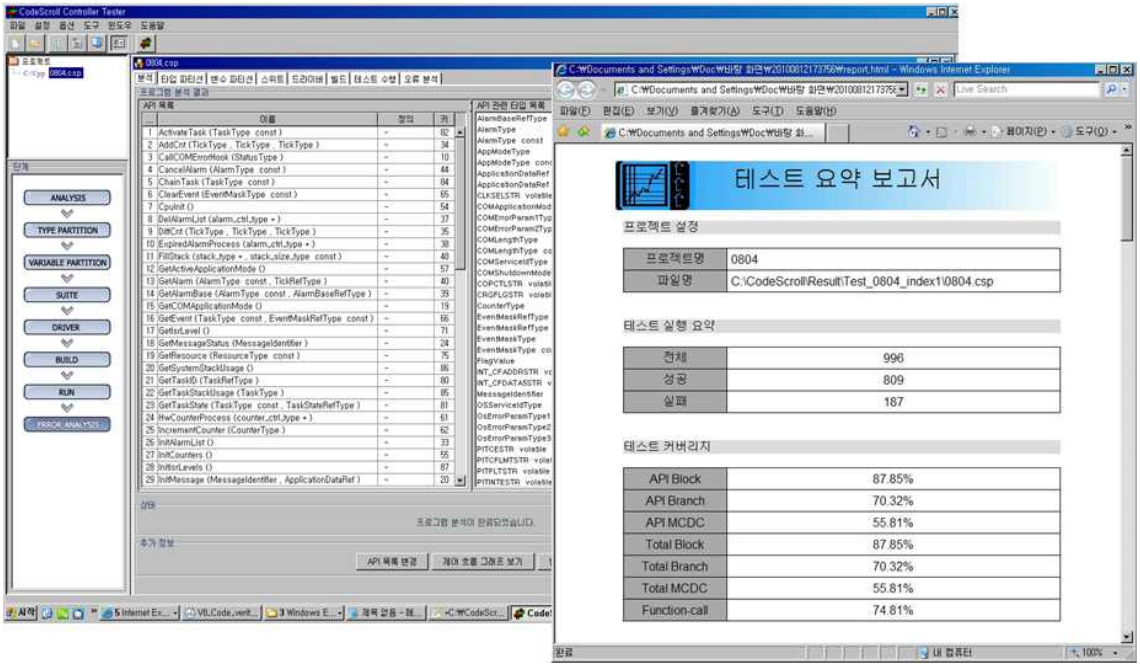


그림 3-6. 동적 테스트 수행 결과

(2) OSEK OS 인증 획득

(가) 개요

2차년도에 인증추진을 위해 OSEK OS Testsuite를 적용하여 개선 및 안정화 작업을 수행한 OS 소스코드 및 코드생성도구를 이용해서 OSEK 인증절차 및 상표등록을 수행하였다.

(나) OSEK OS 인증을 위해 필요한 필수 항목

OSEK 인증 절차(Part1)은 다음과 같다.

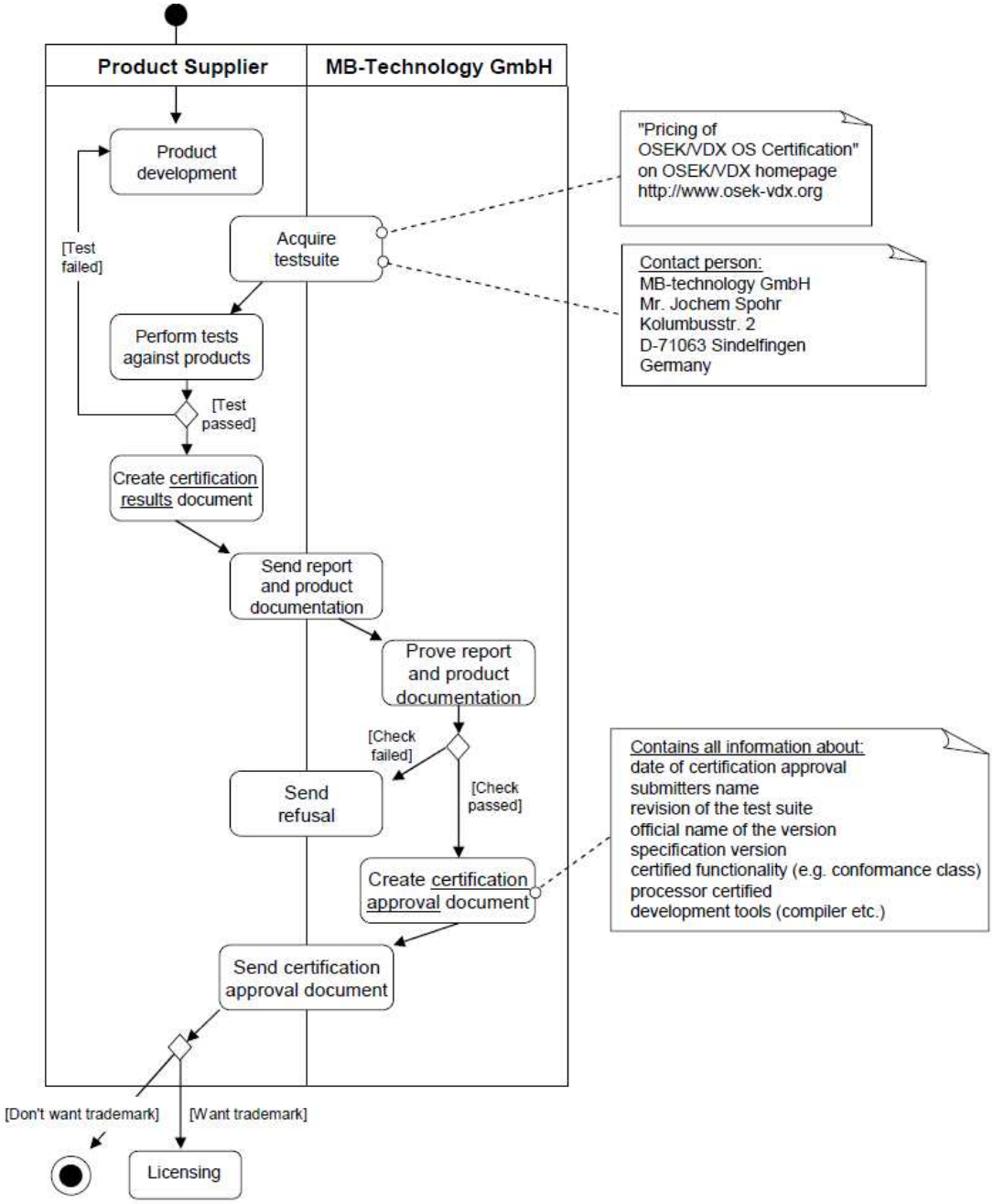


그림 3-7. OSEK OS 인증절차 (Part 1)

testsuites를 이용해서 테스트가 통과되면 테스팅 과정에서 생성되는 결과파일, 타겟에 맞게 포팅된 testsuite, OS Implementation Document, Requested Information for OSEK Certification Approval 문서를 MBTech에 송부한다.

MBTech에서는 문서 및 결과파일을 검토하여 문제가 발생되지 않으면 인증승인 문서(Certification approval Document)를 보내다. 아래 그림은 MBTech에서 보내온 인증승인 문서이다.

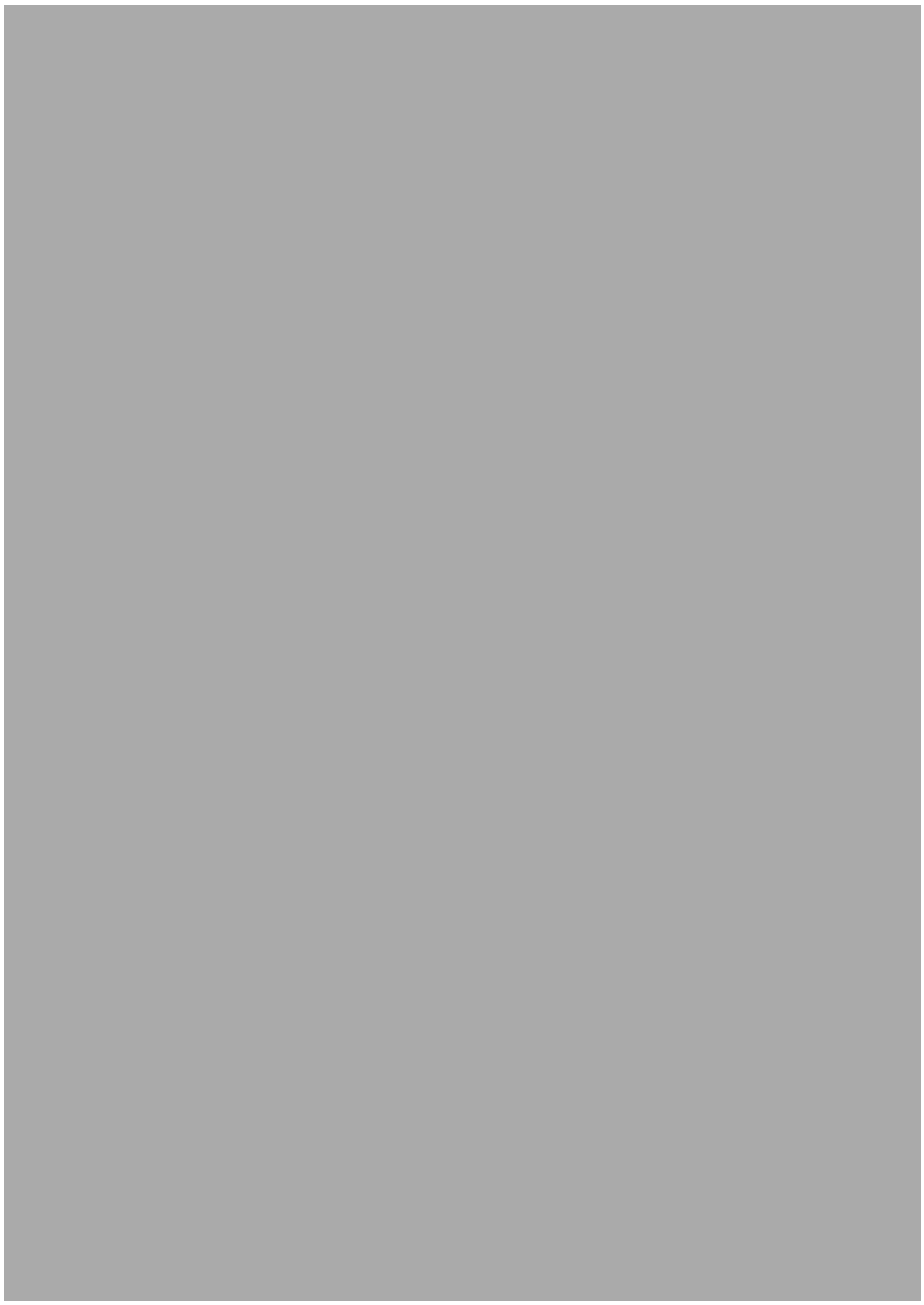


그림 3-8. Certification Approval Document

이와 동시에 OSEK 공식 홈페이지에 다음과 같이 등록된다.

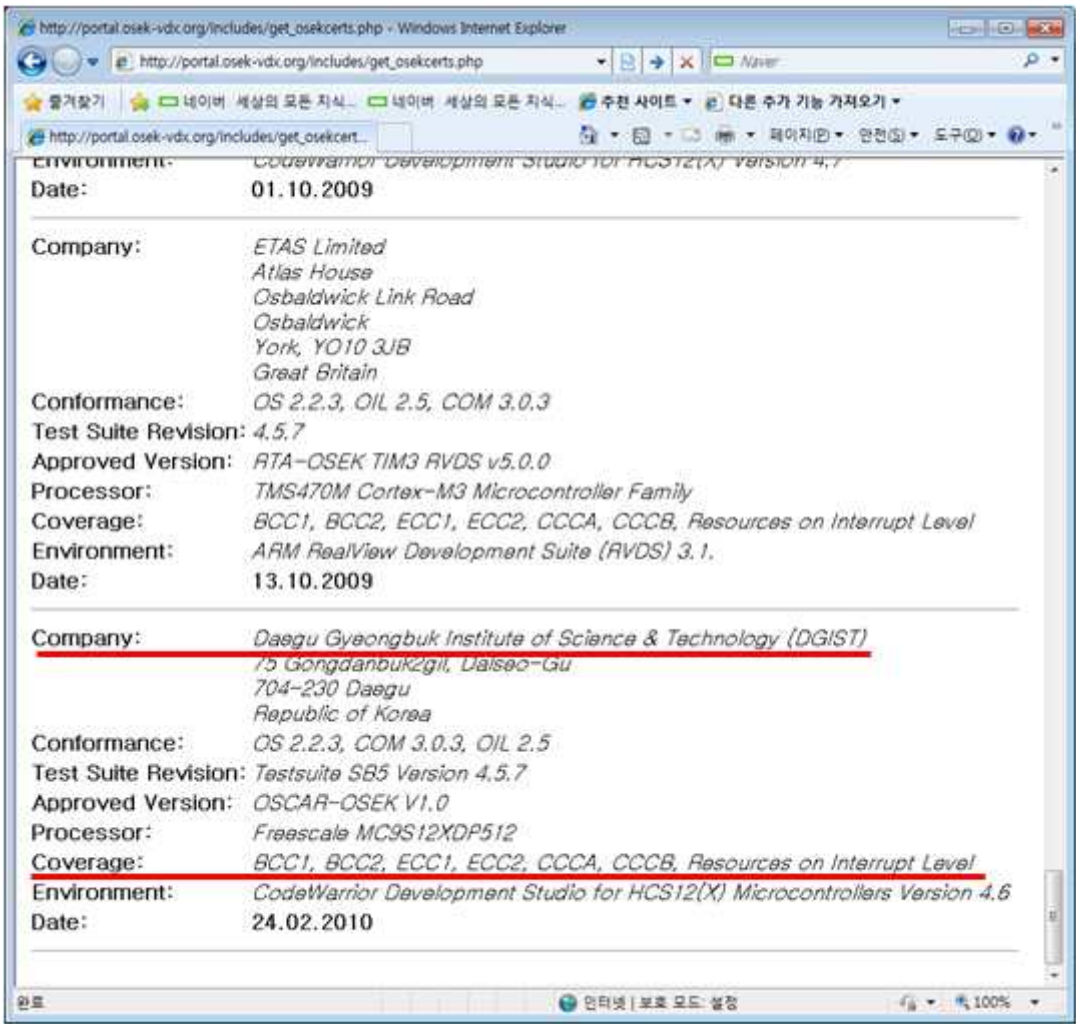


그림 3-9. OSEK/VDX 홈페이지 등록

Trade Mark Licensing Agreement를 위한 OSEK 인증 절차(Part2)는 다음과 같다.

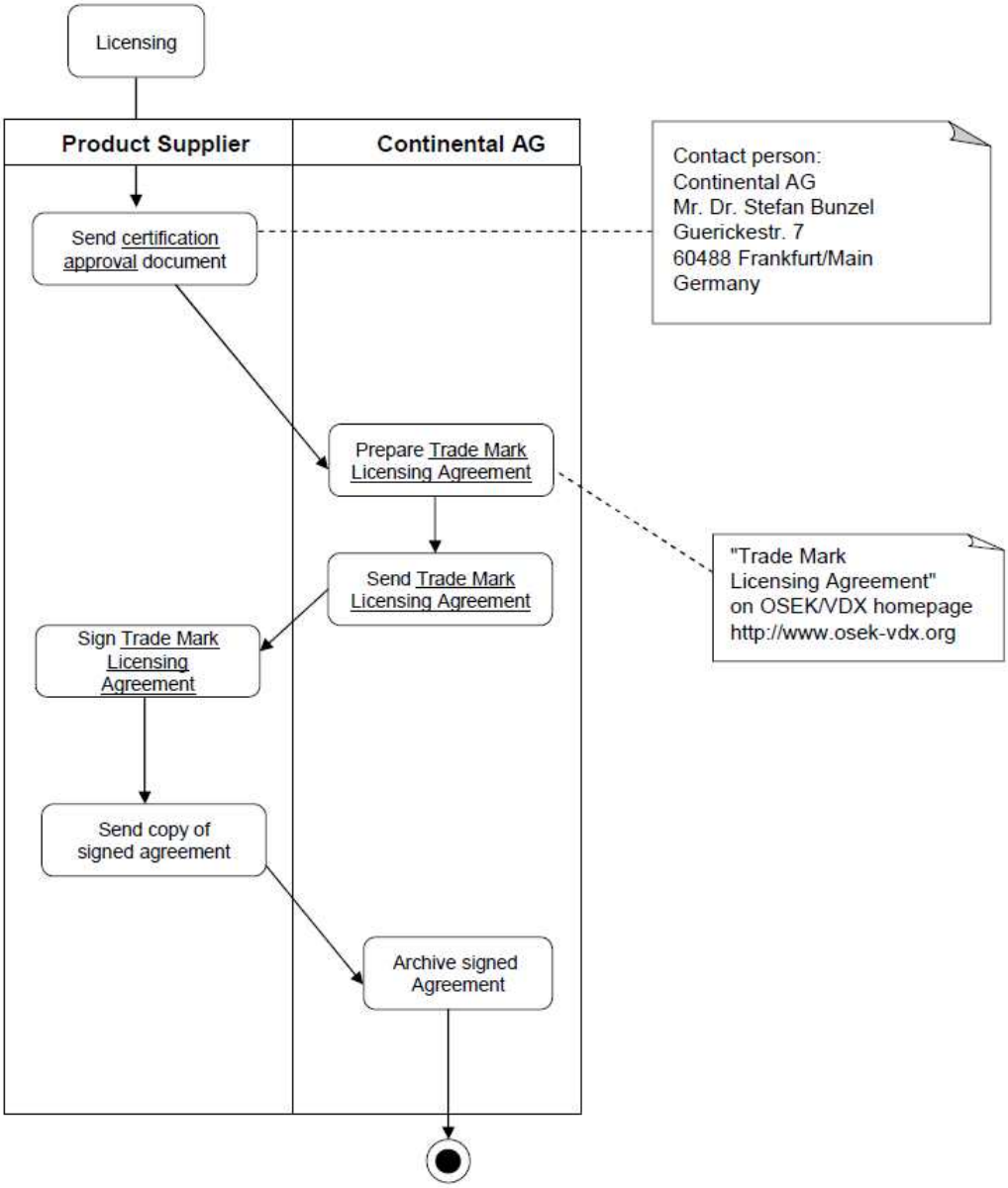
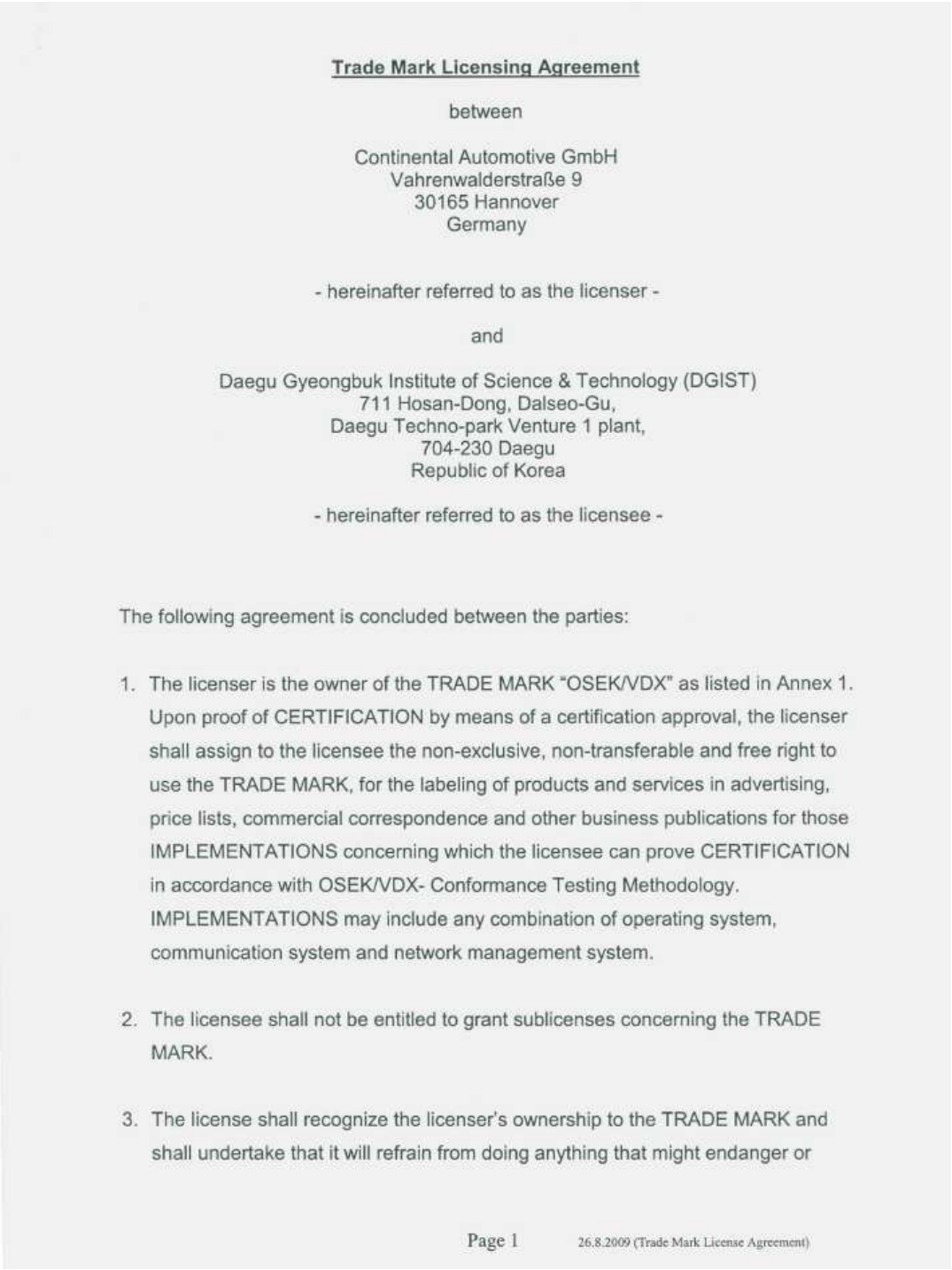


그림 3-10. OSEK OS 인증절차 (Part 2)

Part2의 절차는 비교적 단순하며 Part1을 통해 수령한 Certification approval document를 Continental의 담당자에게 송부하면 협약서를 보내주게 되고 이를 양자 서명해서 보관한다.



Trade Mark Licensing Agreement

between

Continental Automotive GmbH
Vahrenwalderstraße 9
30165 Hannover
Germany

- hereinafter referred to as the licenser -

and

Daegu Gyeongbuk Institute of Science & Technology (DGIST)
711 Hosan-Dong, Dalseo-Gu,
Daegu Techno-park Venture 1 plant,
704-230 Daegu
Republic of Korea

- hereinafter referred to as the licensee -

The following agreement is concluded between the parties:

1. The licenser is the owner of the TRADE MARK "OSEK/VDX" as listed in Annex 1. Upon proof of CERTIFICATION by means of a certification approval, the licenser shall assign to the licensee the non-exclusive, non-transferable and free right to use the TRADE MARK, for the labeling of products and services in advertising, price lists, commercial correspondence and other business publications for those IMPLEMENTATIONS concerning which the licensee can prove CERTIFICATION in accordance with OSEK/VDX- Conformance Testing Methodology. IMPLEMENTATIONS may include any combination of operating system, communication system and network management system.

2. The licensee shall not be entitled to grant sublicenses concerning the TRADE MARK.

3. The license shall recognize the licenser's ownership to the TRADE MARK and shall undertake that it will refrain from doing anything that might endanger or

Page 1 26.8.2009 (Trade Mark License Agreement)

그림 3-11. Trade Mark Licensing Agreement

(다) OS Implementation Document

OS Implementation Document는 개별사가 구현한 OS에 주요 사양 및 특징에 대해서 기술하는 문서이다. 문서형식은 자유형식이고 MBTech에서 인증 승인을 위해 필요한 15개의 필수 체크리스트를 문서에 포함하여 작성하면 된다.
아래 그림은 DGIST에서 개발한 OSEK OS(OSCAR-OSEK)의 Implementation Document의 목차이다. 목차의 순서는 인증 승인 절차를 용이하게 수행하도록 체크리스트의 순서에 맞게 작성하였다.

2 Specifications 5

2.1 Conformance Classes 5
2.2 Official OSEK description 5
2.3 System objects 5
2.4 Particularities of non pre-emptive system 6
2.5 Interrupt processing 6
2.6 Resource management for interrupts 7
2.7 SW priorities and HW interrupt levels 7
2.8 Counter API 8
2.9 Communication conformance classes 8
2.10 System start-up 9
2.11 Error handling 9
2.12 Behaviors of *SetRelAlarm* function 10
2.13 Call to *StartOS* function 10
2.14 Behavior of *ShutdownOS* function 10
2.15 Additional Information 11

OSCAR-OSEK에서 제공하는 OS의 시스템 오브젝트의 지원 사양은 다음 표와 같다. 표의 내용 중에 회색배경으로 표시된 부분은 OSEK OS 표준 규격에서 요구하는 최소사양이다.

	BCC1	BCC2	ECC1	ECC2
Multiple requesting of task activation	no	yes	BT: no ET: no	BT: yes ET: no
OSCAR-OSEK	**no**	**yes**	**BT: no ET: no**	**BT: yes ET: no**
Number of tasks which are not in the *suspended* state	8		16 (any combination of BT/ET)	
OSCAR-OSEK	**256**		**256 (any combination of BT/ET)**	
More than one task per priority	no	yes	no (both BT/ET)	yes (both BT/ET)
OSCAR-OSEK	**no**	**yes**	**no (both BT/ET)**	**yes (both BT/ET)**
Number of events per task	-		8	
OSCAR-OSEK	-		**16**	
Number of task priorities	8		16	
OSCAR-OSEK	**256**			
Resources	RES_SCHEDULER	8(including RES_SCHEDULER)		
OSCAR-OSEK	**256(including RES_SCHEDULER)**			
Internal resources	2			
OSCAR-OSEK	**256**			
Alarm	1			
OSCAR-OSEK	**256(per task), 65536(per system)**			
Application Mode	1			
OSCAR-OSEK	**1**			

표 3-2. OSCAR-OSEK 지원 사양

(3) 플랫폼 확장 및 상용화

(가) 개요

DGIST는 2차년까지 OSEK OS 버전 2.2.3 및 OSEK COM 버전 3.0.3 표준규격에 기반한 실시간 운영체제를 개발하고, 대표적인 전장제어 16비트 MCU인 Freescale사의 MC9S12XDP512 타겟보드를 사용하여 호환성 테스트 절차에 따라 OSEK/VDX 인증을 획득하였다. 본 과업은 OSEK/VDX 인증을 획득한 OSEK OS를 로봇, 의료기기 및 다양한 전자제어분야에 활용하기 위해서 8비트 및 32비트 MCU에서 동작 가능하도록 소스코드의 구조를 변경하고 기능의 개선 및 확장이 목적이다.

(나) 개발 범위

① OS 추상화 계층 개발

플랫폼 추상화 계층 개발

컴파일러 추상화 계층 개발

메모리 추상화 계층 개발

② OS 추상화 계층 적용 및 소스코드 개선

OS 소스코드에 추상화 계층 적용

추상화 계층 적용에 따른 OS 소스코드 개선

③ 8비트 MCU 적용 및 소스코드 개선

8비트 MCU 개발환경 구축

추상화 계층을 통해 8비트 MCU에 OS 포팅

8비트 MCU 적용을 위한 소스코드 개선

④ 32비트 MCU 적용 및 소스코드 개선

32비트 MCU 개발환경 구축

추상화 계층을 통해 32비트 MCU에 OS 포팅

32비트 MCU 적용을 위한 소스코드 개선

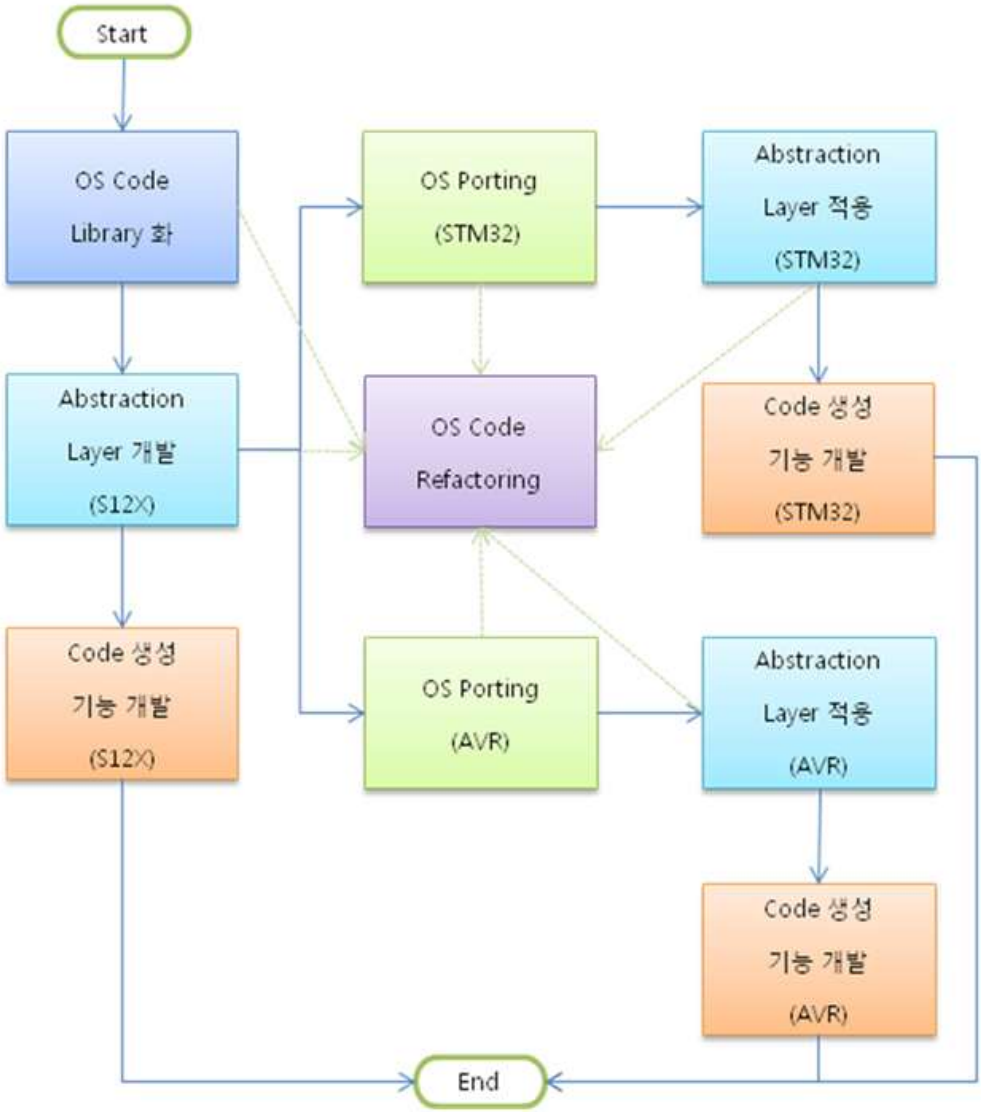
⑤ 타겟 MCU별 코드생성 기능 개발

타켓 MCU 선택 기능 개발

타켓 MCU에 따른 코드생성 기능 개발

(다) 개발 방법

① 개발 절차 및 내용



- OS Code Library화

  배포 및 유지 관리 편의성을 위해 Library화 수행

- Abstraction Layer 개발

  플랫폼 및 개발환경 독립성을 확보하기 위해 추상화 계층 개발

- OS Porting

  기존 16비트 MCU 이외에 8비트 및 32비트 MCU에 OS를 포팅 및 적용하여 다양한 플랫폼 지원 기반 마련

- Code 생성 기능 개발

추상화 및 Library화에 따른 OS Code 변경 내용 적용

- OS Code Refactoring
추상화 및 Library화에 따른 OS Code Refactoring 수행

② 개발 환경
- S12X Target
; MCU
Freescale MC9S12XDP512 16Mhz Clock
; Target Board
Freescale SK - S12XDP512 - A
; Compiler Set
Metrowerks CodeWarrior Development Studio for Freescale HCS12(X) Microcontrollers v4.6
; ICE & Debugger
PE MICRO USB Multilink Interface

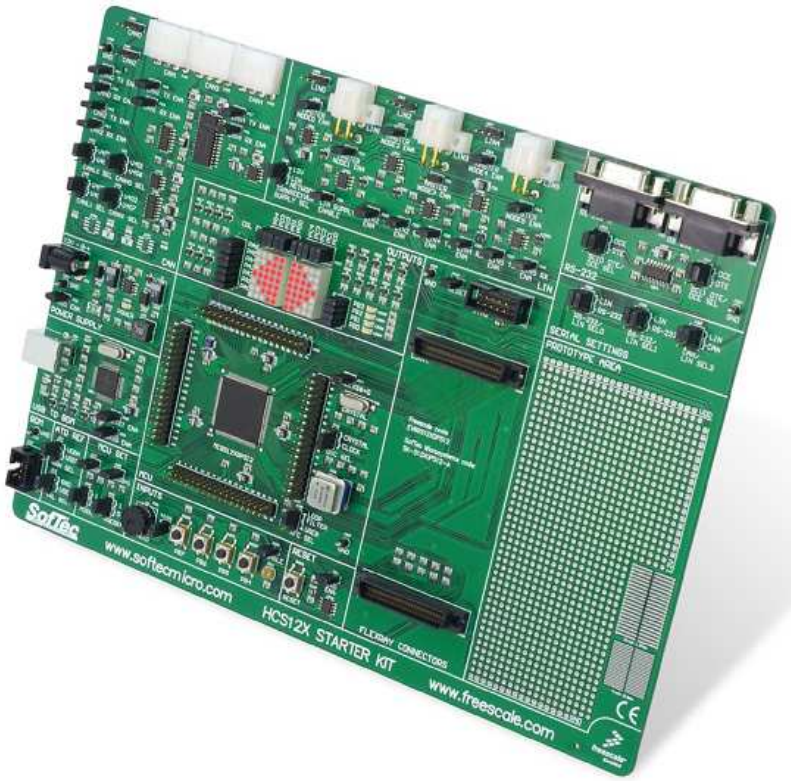


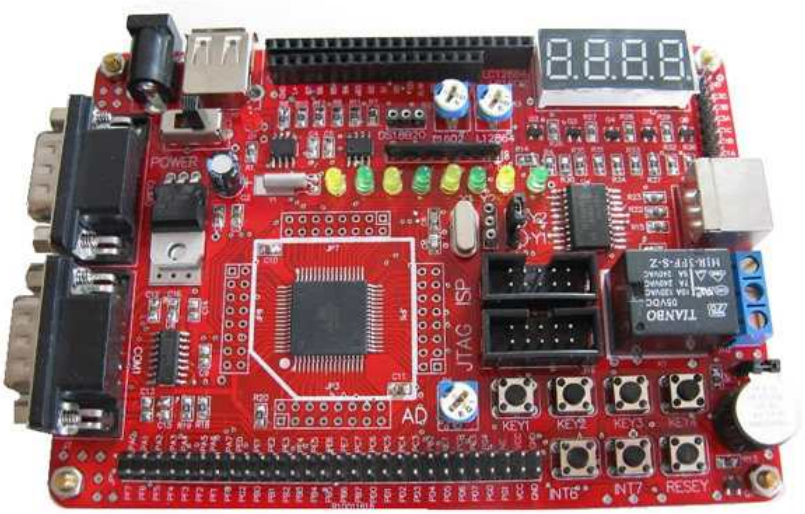
그림 3-12. 16비트 타겟 보드

- STM32 Target
  ; MCU
  ST-Microelectroncis STM32F103RBT6
  48Mhz Clock
  ; Target Board
  한진데이타 STM32-CAR(RX-TFT1.8)
  ; Compiler Set
  KEIL RealView MDK-ARM Version 4.13a
  ; ICE & Debugger
  KEIL ULINK2



그림 3-13. 32비트 타겟 보드

- AVR Target
  ; MCU
  ATMEL ATmega128A
  16Mhz Clock
  ; Target Board
  JK전자 ATMEGA128 STK128_RED
  ; Compiler Set
  WinAVR 20100110
  - AVR GNU Binutils 2.19
  - AVR GNU Compiler Collection 4.3.3
  ; ICE & Debugger
  인터보드 AVR USB-COMBO JTAG&ISP ICE



3-12. 8비트 타겟 보드

(라) 개발 내용

1) Code Library 화

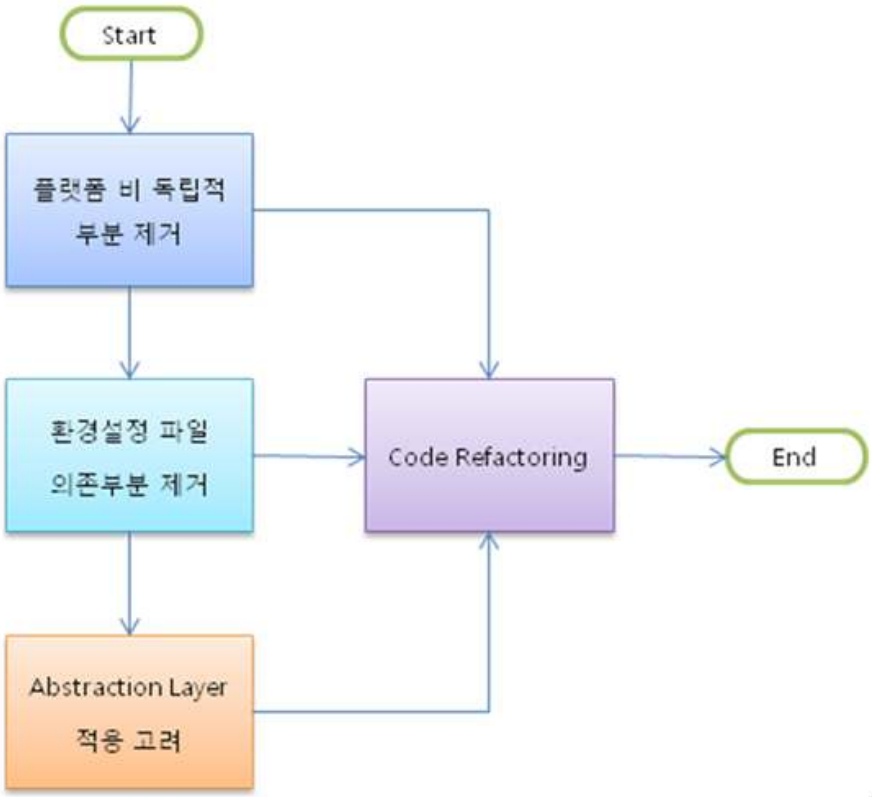
- 목적

가) OS의 외부 공개를 목적으로 함

나) OS Kernel 및 COM 등의 주요 파일의 외부 노출 방지

다) OS 배포 및 포팅 시 유지 관리 편의성 높임

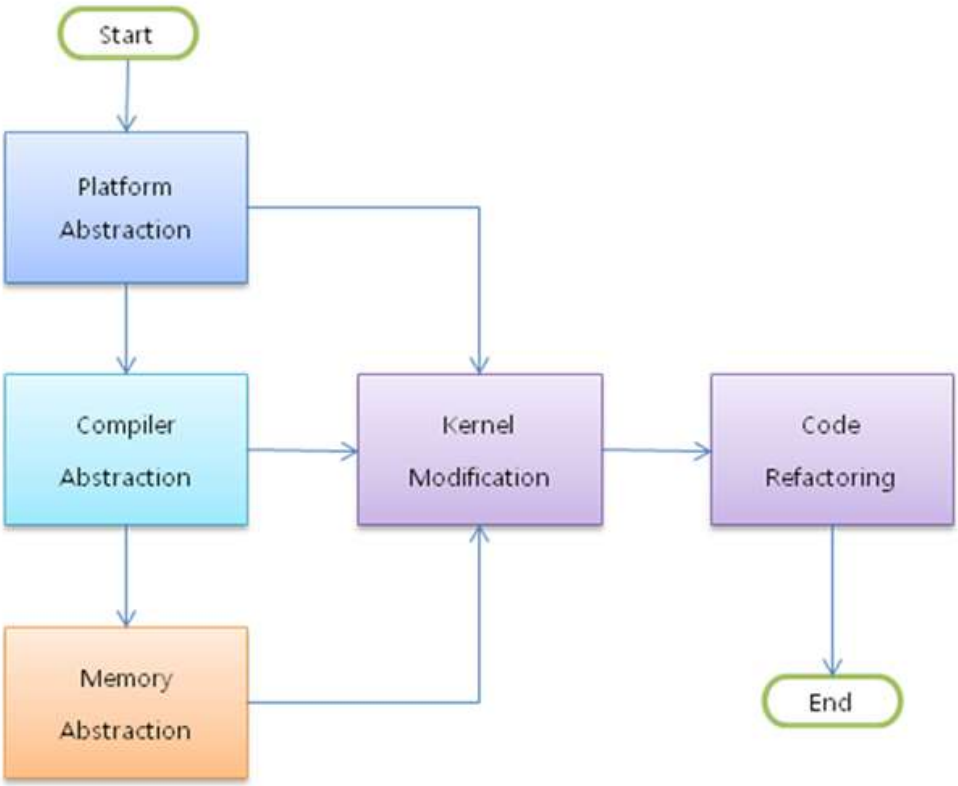
- 개발 절차



- 주요 내용
  가) Kernel code 내의 compile 관련 pre processor 삭제
  나) Kernel code 내의 Macro 처리 부분 삭제 <- 함수로 대체
  다) 환경 설정 파일Os_cgf.h, App_cfg.h 내의 #define 정의 부분을 const variable로 수정 및 Kernel code에 적용
  라) Platform 관련 파일 Cpu.c, Cpu.h, Cpu_interrupt.c, Cpu_interrupt.h의 구조 수정 및 Refactoring

2) Abstraction Layer
- 목적
  가) OS 적용 시 타겟 플랫폼의 하드웨어에 독립성 유지
  나) 타겟 변경 시 OS 소스 코드 변경 최소화
  다) 개발환경 변경 시 문제 발생 최소화

- 개발 절차



- Platform Abstraction
  가) 사용하는 모든 type, symbol 등에 대한 정의를 포함
  나) 타겟 플랫폼에 독립적으로 설계
  다) 타겟 플랫폼 변경 시 OS 소스 코드 변경 최소화

- Compile Abstraction
  가) 데이터나 코드에 대한 선언, Keyword에 대한 추상화
  나) 메모리 모델, 상수형, 변수형, 포인터형, 변수 참조 등에 대한 추상화 제공
  다) 컴파일러 간 차이에 따른 문제점 최소화
  라) Compiler_cfg.h 파일에 OS container 부분 추가

- Memory Abstraction
  가) 코드 및 데이터가 특정 메모리 영역에 맵핑되도록 추상화 제

공

나) 메모리 가용성을 높이고 지정된 메모리 영역 배치 기능 제공

다) 타켓 플랫폼 및 컴파일러 간 차이점 최소화

라) Memmap.h 파일에 OS section 부분 추가 및 default section mapping 부분을 Target 및 컴파일러에 적합하도록 수정

- Kernel Modification

가) Abstraction Layer 적용에 따른 OS Kernel Code 수정

나) 컴파일러 및 타겟 플랫폼 의존적인 부분 제거 및 수정

- Code Refactoring

가) 수정된 OS Code와 환경 설정 파일과 호환성 유지

나) 매크로 및 인라인 함수 부분 제거 및 수정

3) OS Porting

- 개발 절차



- 개발환경 Setup
  가) Target MCU 선정 : 시장 수요, 구매 편의성 고료
  나) 개발환경 구축 비용 고려 : 오픈 소스 개발 툴 사용, 저가 Evaluation Board 사용

- OS Porting
  가) Abstraction Layer 적용 고려
  나) Platform 관련 파일 수정 : Cpu.c, Cpu.h, Cpu_Interrupt.c, Cpu_Interrupt.h
  다) Configuration Tool 및 System Generator 연동 작업 수행

- Code 최적화
  가) 메모리, 코드 사이즈, 응답성 고려

나) OSEK 표준 호환성 고려

4) Configuration Tool 추가 개발

- 목적

가) 지원 Target MCU 확장에 따른 변경 사항 적용

나) Target MCU 별 고유기능 지원

다) Target MCU 별 Code 생성 기능 개발

라) OS Abstraction Layer 적용에 따른 변경 사항 적용

마) OS Source Code Refactoring에 따른 수정 사항 적용

- 추가 개발 내용

가) Target MCU 선택 기능 : S12X, AVR(ATmega Series), ARM(Cortex-M3)

나) Target MCU 별 고유 기능 적용

· Interrupt 선택 및 코드 생성

· Timer/Counter 기능 선택 및 코드 생성

다) 환경설정 파일 구조 수정

· MACRO 정의 부분 삭제 <- 컴파일러 전처리 부분 삭제

· const 변수 정의 및 참조 부분 처리

· Target MCU 및 컴파일러 의존 부분 처리 : OS에서 사용하는 type 정의 부분을 os_type.h에서 os_cfg.h 로 이동

다. 통신클러스터 개발

(1) 통신 클러스터

RTE에서 시그널을 생성하고 이를 환경설정되어 있는 모듈에 전송하는 것은 아래 그림과 같은 과정을 통해 이루어진다. RTE는 AUTOSAR COM에서 제공되는 Com_SendSignal이나 Com_SendSignalGroup을 호출하여 AUTOSAR COM에 시그널을 전송한다. 이 시그널은 시그널 ID와 시그널의 값으로 구성되어 있는데, 시그널 ID는 Com_SignalGroupIdType(uint16)으로 되어 있으며 시그널의 값은 Com_ApplicationDataRefType(unit8)로 정의된다.

시그널을 수신한AUTOSAR COM은 수신한 시그널이 저장되는 I-PDU ID를 검색해서, I-PDU ID와 시그널 값인 unit8을 포함하는 I-PDU를 생성하고, PDU Router에서 제공되는 PduR_ComTransmit 함수를 사용해서 생성한 I-PDU(uint16 I-PDU ID, unit8)을 PDU Router에게 송신한다.

PDU Router는 Transport 프로토콜 모듈을 경유하는 경우와 직접 Interface 모듈에 I-PDU를 전송하는 두가지 경우가 있는데, 여기서는 Interface 모듈을 통해서 자료를 송수신하는 경우를 살펴보고자 한다. Interface 모듈을 거치는 경우, AUTOSAR COM에서 수신한 I-PDU[I-PDU ID, uint8]을 Interface 프로토콜 모듈에서 제공하는 송신용함수인 CanIf_Transmit의 입력값으로 다음과 같이 송신한다. CanIf_Transmit(I-PDU ID, unit8).

CAN 인터페이스 모듈과 CAN Driver의 예를 들어보면, PDU Router는 CAN 인터페이스에서 제공하는 CanIf_Transmit 함수를 호출하여 I-PDU를 Can Interface에 전송한다. Can Interface는 I-PDU를 Data Link 계층의 데이터 포멧인 L-PDU[uint8, PDUInfo[I-PDU ID, uint8]]로 변환해서, Can Driver에서 제공하는 Can_Write 함수의 입력값으로 Can Driver에 L-PDU를 전송한다.

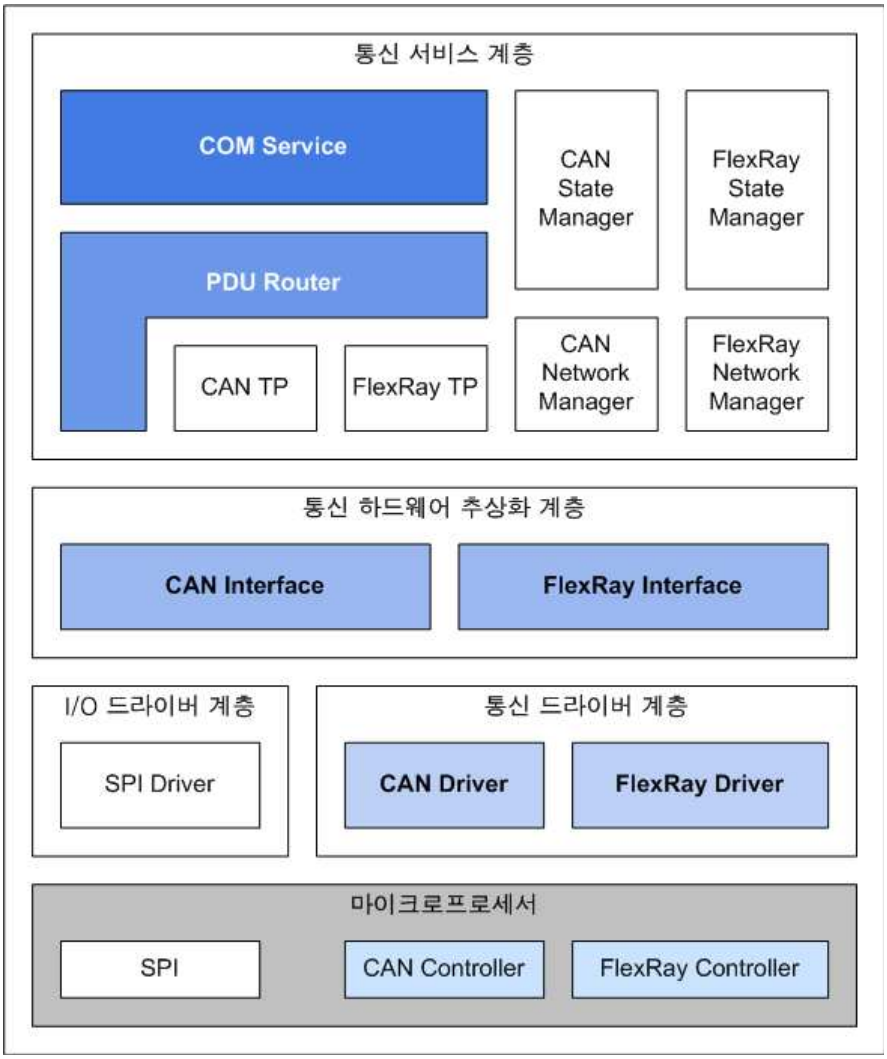


그림 3-14. 통신 클러스터 계층 컨텍스트 다이어그램

AUTOSAR COM에서 사용되는 데이터 Foramt은 크게 세가지를 볼수가 있다. 첫째는 Service Data Unit으로 불리는 SDU인데, 이것은 상위 계층으로부터 전송된 모든 패킷을 말한다. 두번째는 Protocol Control Information이라고 불리는 PCI로써, 패킷을 캡슐화 시키기위해서 기본적으로 필요한 정보들을 저장하고 있는데, 일반적으로 해당 패킷의 소스 및 목적지의 주소를 나타내고 있다.

이 PCI는 계층을 내려갈수록 하나씩 추가되고 난후, 네트워크를 통해 전송되고, 수신측에 도착하면 그에 해당하는 계층에서 하나씩 분해되어 해석된

다. 마지막으로 Protocol Data Unit이라고 불리는 PDU가 있는데, 이것은 상위 계층으로 받은 SDU와 현재 계층에서 부가될 PCI를 결합하여 형성한다. 앞에서 설명한 AUTOSAR COM 데이터 포멧을 살펴보면 아래 그림과 같다.

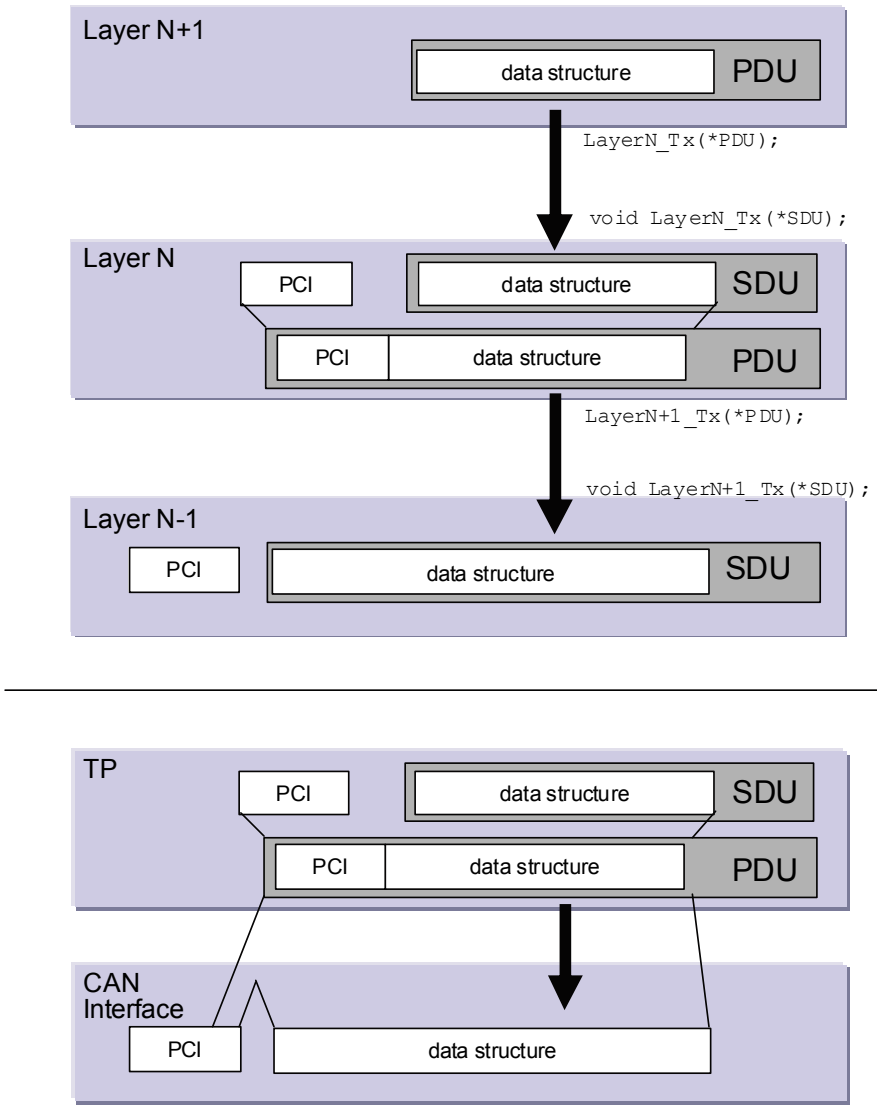


그림 3-15. AUTOSAR COM의 Data 포맷

그러나 실제 AUTOSAR COM 데이터 포멧으로는 각각의 계층 즉, Communication 계층, PDU Router 계층, CAN Interface 계층, CAN Driver 계층에서 실제로 사용되는 것은 다음과 같이 2가지 데이터 포멧을 볼수가

있다. Communicaiton 계층과, PDU Router 계층에서는 I-PDU가 사용되고, Network 계층인 CAN Interface 계층에서는 L-PDU가 사용된다. 이렇게 사용되는 각 데이터 포멧은 다음과 같은 형식을 가지고 있다. 참고로, PCI에 해당하는 부분들은 AUTOSAR 환경 설정 부분에서 미리 세팅되어 있기 때문에, 송수신되는 시그널이나 PDU의 ID만 알고 있으면, 각각의 모듈들은 이 데이터의 소스 및 목적지 데이터 정보등을 알수가 있다.

I-PDU = (uint16 I-PDU ID || unit8 Signal Value)
L-PDU = (uint8 Hrh || I-PDU)

RTE로부터 AUTOSAR COM으로 보내는 시그널은 시그널 ID와 그 값으로 구성되어 있는데, 수신한 시그널 ID를 바탕으로 환경 설정에서 저장된 모든 시그널들을 검색한다. 시그널에 사용되는 전송모드, DM 주기 및 저장되는 I-PDU ID 등은 미리 설정된 환경정보에서 검색이 가능하게 된다. 이렇게 환경 설정에서 결정되는 시그널과 I-PDU의 데이터는 아래와 같은 형식으로 구성되어 있다.

- 송수신되는 시그널 데이터 형식

```
Uint16 Com_SignalIdType // 시그널 ID 타입
Uint8 Com_SignalGroupIdType // 시그널 그룹 ID 타입
Uint8 Com_ApplicationDataRefType // 시그널 데이터 타입
```

- 송수신되는 PDU 데이터 형

```
Uint16 PduIdType: PDU ID 타입
Struct PduInfoType {
  Uint8* SduDataPtr; // Sdu 데이터에 대한 포인터
  PduLengthType SduLength; // Sdu 데이터 길이
}
```

- 환경설정되는 시그널 데이터 형식

```
struct COM_SIGNAL {
```

uint8 COM_SIGNAL_NAME[16]; // 시그널 이름
uint16 COM_SIGNAL_HANDLE_ID; // 시그널 고유번호
float64 COM_SIGNAL_INIT_VALUE ; // 시그널 초기 값
uint8 COM_SIGNAL_LENGTH; // 시그널 길이
uint8 COM_SIGNAL_GROUP_REF[16]; // 시그널이 속한 그룹이름
uint8 COM_SIGNAL_NETWORK_SIGNAL_REF[16]; // 시그널이 속한 네트워크
COM_FILTER com_FILTER; // 시그널에 적용되는 필터 알고리즘
}

- 환경설정되는 네트워크 시그널 형식

struct COM_NETWORK_SIGNAL{
char COM_NETWORK_SIGNAL_NAME[16]; /네트워크 시그널 이름
uint32 COM_NETWORK_SIGNAL_SIZE; //0-64 bits (up to 8 bytes)
uint32 COM_NETWORK_SIGNAL_BIT_POSITION; //0-63 bits, Starting position in I-PDU of network signal.
uint32 COM_NETWORK_SIGNAL_FIRST_TIMEOUT_FACTOR; //DM의 첫번째 타임 아웃 시간을 결정
uint32 COM_NETWORK_SIGNAL_TIMEOUT_FACTOR; //DM의 타임 아웃 시간을 결정
uint8 COM_NETWORK_SIGNAL_IPDU_REF[16]; // 이 시그널이나 시그널 그룹을 포함하고 있는 I-PDU에 대한 참조값
COM_SIGNAL_TRANSFER_PROPERTY com_SIGNAL_
TRANSFER_ PROPERTY; // 시그널의 전송 속성
}

- 환경설정되는 I-PDU 데이터 형식

struct COM_IPDU {
uint8 COM_IPDU_NAME[16]; // IPDU 이름
uint16 COM_IPDU_HANDLE_ID; // IPDU 고유 번호
uint32 COM_IPDU_SIZE; // IPDU 크기
uint8 COM_IPDU_UNUSED_AREAS_DEFAULT // 사용되지 않는 IPU

디폴트값

uint8 COM_IPDU_GROUP_REF[16]; // IPDU가 속한 그룹 이름

C O M _ T R A N S M I S S I O N _ M O D E _ T R U E com_TRANSMISSION_MODE_TRUE;

// IPDU에 설정된 송신 모드

}

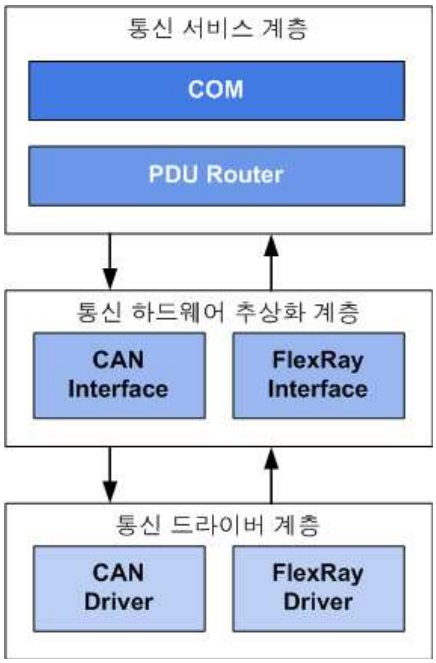


그림 3-16. 통신 클러스터 계층 모듈 다이어그램

통신 클러스터는 통신 드라이버 계층, 통신 하드웨어 추상화 계층, 통신 서비스 계층으로 구성된다. 통신 드라이버 계층은 CAN Driver 모듈로 구성되며, 통신 하드웨어 추상화 계층은 CAN Interface 모듈, 그리고 통신 서비스 계층은 PDU Router와 COM의 두 모듈로 구성된다. 당해년도에는 CAN Driver 모듈, CAN Interface 모듈, PDU Router 모듈, 그리고 COM 모듈을 개발한다.

(가) AUTOSAR COM

① AUTOSAR COM 안정화 수행 항목

이절에서는 COM 모듈에 속한 각 API들을 통합하여, COM 모듈 고유의 기본 기능을 정상적으로 수행하는지를 검증하고, 오류 발견 시 보완하는 작업을 수행하고자 한다. 예를 들어, COM이 수신한 시그널을 I-PDU에 저장하고, 시그널의 타입이 Trigger Property인 경우, 해당 I-PDU를 PDU Router에게 즉시 송신해야 되는 등의 자체 통합 테스트를 수행한다. 이러한 테스트를 통해 문제점이 발견되면, 해당 오류를 디버깅을 통해 수정하는 등의 안정화 과정을 수행한다. 아래 각 항목은 AUTOSAR COM에서 수행한 안정화 항목을 나타낸다.

○ 송신하는 시그널을 I-PDU에 저장하는 함수 동작 안정화
○ I-PDU에서 시그널 데이터를 수신하는 함수 안정화
○ COM이 각각의 I-PDU에 대해서 제공하는 Direct Transmission Mode 안정화
○ COM이 각각의 I-PDU에 대해서 제공하는 None Transmission Mode 안정화
○ I-PDU의 사용되지 않는 구역과 비트들을 환경 설정된 값으로 저장
○ COM이 I-PDU를 직접 송신할 수 있는 함수 안정화
○ COM 모듈이 시그널의 Trigger Property를 지원하는 기능 안정화
○ COM에 새로운 시그널이 수신되면 삽입되는 I-PDU가 업되이트 되었음을 알 수 있는 업데이트 비트를 제공하는 기능 안정화
○ ALWAYS와 NEVER의 기본적인 필터링 알고리즘 안정화
○ COM 모듈이 MASKED_NEW_EQUALS_X, MASKED_NEW_DIFFERS_X, MASKED_NEW_ DIFFERS_MASKED_OLD 필터링 알고리즘을 옵션으로 제공하는 기능 안정화

② AUTOSAR COM 안정화

○ 송신하는 시그널을 I-PDU에 저장하는 함수 동작 안정화

- 기능

설 명	o RTE에서 전송하고자 하는 시그널이 발생하면, 이 시그널을 COM 클러스터에 송신하여 I-PDU 버퍼(ipdu_buffer)에 시그널 값을 저장하고, I-PDU Update 비트를 1로 설정하는지를 확인하고, 문제 발생 시 수정

- 절차

안정화 수행 방법	(1) 시험 프로그램은COM Cluster (COM, PDU Router, CAN Interface, CAN Driver)와 시험을 위한 응용 프로그램으로 구성한다. (2) 응용 프로그램은 COM 모듈의 시그널 전송 API인 COM_SendSignal 함수를 주기적으로 호출한다. (3) 디버거를 통해 I-PDU 버퍼에 RTE로부터 전달된 시그널 데이터가 저장되어 있는지를 확인한다.
세부 분석 및 수정	(1) RTE에서 Com_SendSignal 함수를 호출하여 Signal ID와 SignalDataPtr을 전송하고, Signal ID와 SignalDataPtr 값을 Com_SendSignal 함수에서 확인함 (2) 수신된 시그널의 시그널 핸들 ID(sig[i].COM_SIGNAL_HANDLE_ID)와 시그널이 저장될 I-PDU의 핸들 ID(ipdu[z].COM_IPDU_HANDLE_ID)를 검색한 후, 검색된 시그널 핸들 ID와 I-PDU 핸들 ID 값을 확인 (3) 수신된 시그널 값을 검색된 I-PDU의 시그널 저장 위치에 삽입하고, 업데이트 비트를 1로 설정하여, ipdu_buffer에 수신된 시그널 값인 SignalDataPtr이 저장되어있고, 업데이트 비트가 1로 설정되어 있는지를 확인

○ I-PDU에서 시그널 데이터를 수신하는 함수 안정화

- 기능

설 명	o RTE나 응용프로그램에서, COM 클러스터의 I-PDU에 수신된 시그널을 필터링 알고리즘을 통과시켜 SignalDataPtr에 반환받는 함수를 제공

- 절차

안정화 수행 방법	(1) 시험 프로그램은COM Cluster (COM, PDU Router, CAN Interface, CAN Driver)와 시험을 위한 응용 프로그램으로 구성한다. (2) 송신용 보드의 응용 프로그램은 COM 모듈의 시그널 전송 API인 COM_SendSignal 함수를 주기적으로 호출 (3) 송신용 보드 COM모듈의 시그널 전송 API가 호출됨으로써 AUTOSAR SPEC.에서 규정하는 절차에 따라 PDU Router, CAN Interface 내의 상응하는 API가 호출되도록 한다. (4) 수신용 보드에서는 수신된 데이터를 처리하기 위해서, AUTOSAR SPEC.에서 규정하는 절차에 따라, CAN Interface, PDU Router내의 상응하는 API가 호출되도록 한다. (5) 수신용 보드에서는 COM_ReceiveSignal 함수를 주기적으로 호출하고, 디버거를 통해 COM_RecceiveSignal 함수의 argument인 SignalDataPtr에 시그널 데이터가 제대로 저장되어 있는지를 확인한다.
세부 분석 및 수정	(1) 수신용 보드의 응용프로그램에서 COM_RecceiveSignal 함수를 호출하고 정상 동작 수행을 확인 (2) 인자로 입력된 시그널 ID를 사용해서 시그널이 속한 I-PDU 핸들 ID를 검색하고, 검색된 I-PDU 핸들 ID가 수신된 시그널이 속한 I-PDU의 핸들 ID와 동일한지를 확인 (3) 필터링 알고리즘을 거쳐, I-PDU에 수신된 시그널 값을 SignalDataPtr에 저장하여 반환하여, I-PDU 버퍼에 들어있는 시그널 값이 SignalDataPtr에 제대로 입력 되었는지를 확인

○ COM이 각각의 I-PDU에 대해서 제공하는 Direct Transmission Mode 안정화

- 기능

설 명	o RTE에서 시그널을 송신할때, 시그널의 Transfer Property가 Trigger Property인 경우 COM 모듈에서의 송신모드가 Direct Transmission Mode라면, 해당 I-PDU를 PDU Router로 즉시 송신 할 수 있음

- 절차

안정화 수행 방법	(1) 시험 프로그램은COM Cluster (COM, PDU Router, CAN Interface, CAN Driver)와 시험을 위한 응용 프로그램으로 구성한다. (2) 응용 프로그램은 COM 모듈의 시그널 전송 API를 주기적으로 호출한다. (3) 디버거를 통해 PduR_ComTransmit에 전달된 argument의 값을 확인한다.
세부 분석 및 수정	(1) 응용프로그램에서 Com_SendSignal 함수를 호출하여 정상 동작을 수행하는지 확인 (2) 시그널의 Transfer Property는 Triggeed로 설정하고, I-PDU의 Transmission Mode는 Direct Transmission Mode로 설정하여, 코드 수행 중 조건문에서 I-PDU의 Transmission Mode가 Direct Transmission Mode가 참인 경우, 그 이하의 코드가 수행되는 것을 확인하고, 필터알고리즘을 통과하고 나면, PDU Router에서 제공하는 PduR_Com Transmit 함수가 제대로 수행되는 것을 확인하기 위해서, 이 함수에 전달되는 인자 값을 확인

○ COM이 각각의 I-PDU에 대해서 제공하는 None Transmission Mode 안정화

- 기능

설 명	o RTE에서 시그널을 송신할때, 시그널의 Transfer Property와 관계없이 COM 모듈에서의 송신모드가 None Transmission Mode라면, 해당 I-PDU를 PDU Router로 송신 할 수 없음

- 절차

안정화 수행 방법	(1) 시험 프로그램은COM Cluster (COM, PDU Router, CAN Interface, CAN Driver)와 시험을 위한 응용 프로그램으로 구성한다. (2) 응용 프로그램은 COM 모듈의 시그널 전송 API를 주기적으로 호출한다. (3) 디버거를 통해 PDU Router로 I-PDU가 전송되지 않는 것을 확인한다.
세부 분석 및 수정	(1) 응용프로그램에서 Com_SendSignal 함수를 호출하여 정상 동작 수행인 것을 확인 (2) 디버거를 통해 PDU Router에서 제공하는 PduR_ComTransmit 함수가 수행되는지 확인하여, 시그널이 I-PDU에 저장되었지만, I-PDU가 PDU Router에는 전달되지 않않다는 것을 확인하기 위해서, Com_SendSignal의 마지막 코드에서 테스트 코드를 삽입하여 PduR_ComTransmit 함수가 수행되지 않는것을 확인

○ I-PDU의 사용되지 않는 구역과 비트들을 환경 설정된 값으로 저장

- 기능

설 명	o COM 클러스터에서 사용되는 I-PDU 상태 버퍼, 업데이트 버퍼들은 0으로 초기화시키고, I-PDU 데이터 버퍼들을 1로 초기화 시킴

- 절차

안정화 수행 방법	(1) 시험 프로그램은 COM Cluster (COM, PDU Router, CAN Interface, CAN Driver)와 시험을 위한 응용 프로그램으로 구성 (2) 응용 프로그램에서 COM 모듈 초기화 함수인 Com_Init 함수를 호출 (3) 디버거를 통해 I-PDU 상태버퍼와 업데이트 비트들이 0로 초기화되고, I-PDU 데이터 버퍼가 1로 초기화 되는 것을 확인
세부 분석 및 수정	(1) 응용프로그램에서 Com_Init 함수를 호출하여 정상 동작을 수행하는지 확인 (2) IPDU_State,와 I-PDU_Update _bit를 0으로 설정하여, IPDU_State와 I-PDU_Update_bit가 0으로 설정되었음을 확인 2) I-PDU_Buffer가 1로 초기화 시켜, I-PDU_Buffer가 1로 설정되었음을 확인

○ COM이 I-PDU를 직접 송신할 수 있는 함수 안정화

- 기능

설 명	o 응용프로그램이나, RTE, COM 클러스터 등이 COM 모듈의 I-PDU의 전송을 임의로 요청할 수 있는 함수 제공

- 절차

안정화 수행 방법	(1) 시험 프로그램은COM Cluster (COM, PDU Router, CAN Interface, CAN Driver)와 시험을 위한 응용 프로그램으로 구성한다. (2) 응용 프로그램은 COM 모듈의 시그널 전송 API를 주기적으로 호출한다. (3) 응용 프로그램에서 Com_TriggerIPDUSend 함수를 주기적으로 호출한다. (4) 디버거를 통해 PduR_ComTransmit에 전달된 argument의 값을 확인한다.
세부 분석 및 수정	(1) 응용프로그램에서 Com_SendSignal 함수를 호출하여 정상 동작을 수행하는지 확인 (2) 전송하고자하는 I-PDU ID에 대한 I-PDU 데이터 값을 임시버퍼인 pduinfotype에 저장하여, pduinfotype에 I-PDU값이 제대로 저장되는지를 확인 (3) PduR_ComTransmit 함수 호출하여, PduR_ComTransmit 함수가 호출되고, 그 인자 값이 제대로 디버깅을 통해서 전달되는지 확인

○ COM 모듈이 시그널의 Trigger Property를 지원하는 기능 안정화

- 기능

설 명	o RTE에서 시그널을 송신할 때, COM 모듈에서의 송신모드가 Direct Transmission Mode라면, 시그널의 Transfer Property가 Trigger Property인 경우에 I-PDU를 PDU Router로 즉시 송신할 수 있음

- 절차

안정화 수행 방법	(1) 시험 프로그램은COM Cluster (COM, PDU Router, CAN Interface, CAN Driver)와 시험을 위한 응용 프로그램으로 구성한다. (2) 응용 프로그램은 COM 모듈의 시그널 전송 API를 주기적으로 호출한다. (3) 디버거를 통해 PduR_ComTransmit에 전달된 argument의 값을 확인한다.
세부 분석 및 수정	(1) 응용프로그램에서 Com_SendSignal 함수를 호출하여 정상 동작을 수행하는지 확인 (2) 시그널의 Transfer Property는 Triggeed로 설정하고, I-PDU의 Transmission Mode는 Direct Transmission Mode로 설정한 후, 코드 수행 중 조건문에서 I-PDU의 Transmission Mode가 Direct Transmission Mode가 참인 경우, 그 이하의 코드가 수행되는 것을 확인, 그리고, 필터알고리즘을 통과하고 나면, PDU Router에서 제공하는PduR_Com Transmit 함수가 제대로 수행되는 것을 확인하기 위해서, 이 함수에 전달되는 인자 값을 확인

○ COM에 새로운 시그널이 수신되면 삽입되는 I-PDU가 업되이트 되었음을 알 수 있는 업데이트 비트를 제공하는 기능 안정화

- 기능

설 명	o RTE나 응용프로그램에서 시그널을 전송하거나, COM 모듈에 I-PDU가 수신된 경우 해당 I-PDU가 업데이트 되었음을 확인할 수 있는 업데이트 비트를 각각 제공

- 절차

안정화 수행 방법	(1) 시험 프로그램은COM Cluster (COM, PDU Router, CAN Interface, CAN Driver)와 시험을 위한 응용 프로그램으로 구성한다. (2) 송신용 보드의 응용 프로그램은 COM 모듈의 시그널 전송 API인 COM_SendSignal 함수를 주기적으로 호출한다. (3) 송신용 보드 COM모듈의 시그널 전송 API가 호출됨으로써 AUTOSAR SPEC.에서 규정하는 절차에 따라 PDU Router, CAN Interface 내의 상응하는 API가 호출되도록 한다. (4) 수신용 보드에서는 수신된 데이터를 처리하기 위해서, AUTOSAR SPEC.에서 규정하는 절차에 따라, CAN Interface, PDU Router내의 상응하는 API가 호출되도록 한다. (5) 송신용 보드에서는 Com_SendSignal에서 I-PDU 업데이트 비트가 제대로 세팅되는지 확인하고, 수신용 보드에서는 Com_RxIndication에서 수신되 I-PDU의 수신용 업데이트 비트가 제대로 세팅되는지는 디버거를 통해서 확인한다.
세부 분석 및 수정	(1) 송신용 응용프로그램에서 Com_SendSignal 함수를 호출하여, 정상 동작 수행을 확인 (2) 송신되는 시그널이 저장되는I-PDU의IPDU_Update_Bit를 1로 설정하여, IPDU_Update_Bit가 1로 설정되는것을 디버깅을 통해 확인 (3) 수신용 보드에서 Com_RxIndication 함수를 디버깅하여, 수신된 I-PDU에 대한 IPdu_Rx_ Update_Bit가 1로 세팅 되는 것을 확인

○ ALWAYS와 NEVER의 기본적인 필터링 알고리즘 안정화

- 기능

설 명	o COM 클러스터에서는 시그널을 송신하거나 수신할 때, ALWAYS와 NEVER의 기본적인 필터링 알고리즘들을 제공하여, 시그널의 송수신을 제어할 수 있음

- 절차

안정화 수행 방법	(1) 시험 프로그램은COM Cluster (COM, PDU Router, CAN Interface, CAN Driver)와 시험을 위한 응용 프로그램으로 구성한다. (2) 송신용 보드의 응용 프로그램은 COM 모듈의 시그널 전송 API인 COM_SendSignal 함수를 주기적으로 호출한다. (3) 송신용 보드 COM모듈의 시그널 전송 API가 호출됨으로써 AUTOSAR SPEC.에서 규정하는 절차에 따라 PDU Router, CAN Interface 내의 상응하는 API가 호출되도록 한다. (4) 수신용 보드에서는 수신된 데이터를 처리하기 위해서, AUTOSAR SPEC.에서 규정하는 절차에 따라, CAN Interface, PDU Router내의 상응하는 API가 호출되도록 한다. (5) 송신용 보드에서는 Com_SendSignal 함수에서 시그널 필터 알고리즘의 동작을 검증하고, 수신용 보드에서는 Com_ReceiveSignal 함수에서 시그널 필터링 알고리즘이 제대로 동작되는지 디버거를 통해서 확인한다.
세부 분석 및 수정	(1) 송신용 응용프로그램에서 Com_SendSignal 함수를 호출하여, 정상 동작 수행을 확인 (2) 송신용 보드에서 시그널 필터링 속성을 ALWAYS와 NEVER로 교대로 설정하여 시그널을 전송하여, 시그널의 필터 알고리즘이 ALWAYS인 경우 모든 시그널이 다음 코드를 수행하고, NEVER인 경우에는 COM_SERVICE_ NOT_AVAILABLE을 반환하는지를 확

	인 (3) 수신용 응용프로그램에서 Com_ReceiveSignal 함수를 호출하여 정상 동작이 수행되는지 확인 (4) 수신용 보드에서 시그널 필터링 속성을 ALWAYS와 NEVER로 교대로 설정하여 시그널을 수신하여, 시그널의 필터 알고리즘이 ALWAYS인 경우 모든 시그널이 다음 코드를 수행하고, NEVER인 경우에는 COM_SERVICE_ NOT_AVAILABLE을 반환하는지 확인

○ COM 모듈이 MASKED_NEW_EQUALS_X, MASKED_NEW_DIFFERS_X, MASKED_NEW_ DIFFERS_MASKED_OLD 필터링 알고리즘을 옵션으로 제공하는 기능 안정화

- 기능

설 명	o COM 클러스터에서는 시그널을 송신하거나 수신할 때, MASKED_NEW_EQUALS_X,MASKED_NEW_DIFFERS_X, MASKED _NEW_DIFFERS_MASKED_OLD 의기본적인 필터링 알고리즘들을 제공하여, 시그널의 송수신을 제어할 수 있음

- 절차

안정화 수행 방법	(1) 시험 프로그램은COM Cluster (COM, PDU Router, CAN Interface, CAN Driver)와 시험을 위한 응용 프로그램으로 구성한다. (2) 송신용 보드의 응용 프로그램은 COM 모듈의 시그널 전송 API인 COM_SendSignal 함수를 주기적으로 호출한다. (3) 송신용 보드 COM모듈의 시그널 전송 API가 호출됨으로써 AUTOSAR SPEC.에서 규정하는 절차에 따라 PDU Router, CAN Interface 내의 상응하는 API가 호출되도

	록 한다. (4) 수신용 보드에서는 수신된 데이터를 처리하기 위해서, AUTOSAR SPEC.에서 규정하는 절차에 따라, CAN Interface, PDU Router내의 상응하는 API가 호출되도록 한다. (5) 송신용 보드에서는 Com_SendSignal 함수에서 시그널 필터 알고리즘의 동작을 검증하고, 수신용 보드에서는 Com_ReceiveSignal 함수에서 시그널 필터링 알고리즘이 제대로 동작되는지 디버거를 통해서 확인한다.
세부 분석 및 수정	(1) 송신용 응용프로그램에서 Com_SendSignal 함수를 호출하여 정상 동작 수행을 확인 (2) 송신용 보드에서 시그널 필터링 속성을 MASKED_NEW_EQUALS_X, MASKED_NEW_DIFFERS_X, MASKED_NEW_DIFFERS_MASKED_OLD를 교대로 설정하여 시그널을 전송하여, 송신 시그널의 필터 알고리즘이 MASKED_NEW_EQUALS_X, MASKED_NEW_DIFFERS_X, MASKED_NEW_DIFFERS_MASKED_OLD로 변화 될 때, 조건문의 필터링 알고리즘과 일치하는 부분에서 해당 코드가 제대로 수행되는지를 디버깅을 통해서 확인 (3) 수신용 응용프로그램에서 Com_ReceiveSignal 함수를 호출하여 정상 동작 수행을 확인 (4) 수신용 보드에서 시그널 필터링 속성을 MASKED_NEW_EQUALS_X, MASKED_NEW_DIFFERS_X, MASKED_NEW_DIFFERS_MASKED_OLD를 교대로 설정하여 시그널을 수신하여, 수신 시그널의 필터 알고리즘이 MASKED_NEW_EQUALS_X, MASKED_NEW_DIFFERS_X, MASKED_NEW_DIFFERS_MASKED_OLD로 변화 될 때, 조건문의 필터링 알고리즘과 일치하는 부분에서 해당 코드가 제대로 수행되는지를 디버깅을 통해서 확인

(나) PDU Router

① PDU Router 안정화 수행 항목

이절에서는 PDU Router 모듈에 속한 각 API들을 통합하여, PDU Router 모듈 고유의 기본 기능을 정상적으로 수행하는지를 검증하고, 오류 발견 시 보완하는 작업을 수행하고자 한다. 예를 들어, 하위단인 CAN Interface로부터 PDU가 수신되면, COM 모듈에게 PDU 수신알림 메시지인 Com_RxIndication함수를 호출하여, CAN으로부터 COM 모듈로의 PDU의 전송이 원활하게 동작하는지를 확인하는 등 자체 통합 테스트를 수행한다. 이러한 테스트를 통해 문제점이 발견되면, 해당 오류를 디버깅을 통해 수정하는 등의 안정화 과정을 수행한다. 아래 각 항목은 PDU Router에서 수행한 안정화 항목을 나타낸다.

○ PDU Router에 수신된 I-PDU를 COM 모듈에 전송하는 함수 안정화
○ COM 모듈에게 I-PDU의 성공적인 전송을 알리는 함수 안정화
○ PDU Router 모듈이 I-PDU의 성공적인 전송을 알리는 함수 안정화
○ PDU Router 모듈이 I-PDU가 수신되었음을 알리는 함수 안정화

② PDU Router 안정화

○ PDU Router에 수신된 I-PDU를 COM 모듈에 전송하는 함수 안정화

- 기능

설 명	o I-PDU가 하위계층인 PDU Router에 수신되면, 수신된 I-PDU ID와 그에 속한 시그널 값을 하위계층인 PDU Router에서 COM 모듈로 전달하는 함수를 제공

- 절차

안정화	(1) 시험 프로그램은COM Cluster (COM, PDU Router,

수행 방법	CAN Interface, CAN Driver)와 시험을 위한 응용 프로그램으로 구성한다. (2) 송신용 보드의 응용 프로그램은 COM 모듈의 시그널 전송 API인 COM_SendSignal 함수를 주기적으로 호출한다. (3) 송신용 보드 COM모듈의 시그널 전송 API가 호출됨으로써 AUTOSAR SPEC.에서 규정하는 절차에 따라 PDU Router, CAN Interface 내의 상응하는 API가 호출되도록 한다. (4) 수신용 보드에서는 수신된 데이터를 처리하기 위해서, AUTOSAR SPEC.에서 규정하는 절차에 따라, CAN Interface, PDU Router내의 상응하는 API가 호출되도록 한다. (5) 수신용 보드에서 Com_RxIndication 함수를 호출하고, I-PDU와 ID와 시그널 값이 제대로 전달되는지 디버깅을 통해서 확인한다.
세부 분석 및 수정	(1) 송신용 응용프로그램에서 Com_SendSignal 함수를 호출하여 정상 동작 수행을 확인 (2) 수신용 보드에서 PduR_CanIfRxIndication 함수가 Com_RxIndication 함수를 호출하여, 함수 인자 ComRxPduId와 SduPtr에 제대로 된 값이 들어가 있고, SduPtr이 I-PDU 버퍼인 ipdu_buffer에 저장되었는지 디버깅을 통해 확인

○ COM 모듈애게 I-PDU의 성공적인 전송을 알리는 함수 안정화

- 기능

설 명	o I-PDU가 물리적인 전송 계층인 CAN 드라이버에서 전송되면, 전송이 완료되었음을 알리는 Confirmation을 상위 계층으로 올

	려주는데, PDU Router에서 Confirmation을 COM 모듈에 전달하기 위해서 호출하는 함수를 제공

- 절차

안정화 수행 방법	(1) 시험 프로그램은 COM Cluster (COM, PDU Router, CAN Interface, CAN Driver)와 시험을 위한 응용 프로그램으로 구성한다. (2) 응용 프로그램은 COM 모듈의 시그널 전송 API인 COM_SendSignal 함수를 주기적으로 호출한다. (3) COM모듈의 시그널 전송 API가 호출됨으로써 AUTOSAR SPEC.에서 규정하는 절차에 따라 PDU Router, CAN Interface 내의 상응하는 API가 호출되도록 한다. (4) Com_TxConfirmation 함수를 호출하고, I-PDU ID가 성공적으로 전달되고, 송신 확인용 업데이트비트가 제대로 설정되는지를 디버깅을 통해서 확인한다.
세부 분석 및 수정	(1) 응용프로그램에서 Com_SendSignal 함수를 호출하여 정상 동작 수행을 확인 (2) PduR_CanIfTxConfirmation 함수에서 Com_TxConfirmation 함수를 호출하여, 인자로 전달된 ComTxPduId의 값이 전송 요청한 I-PDU ID 인지를 확인하고, 송신 확인용 Ipdu_Tx_Update_Bit가 1로 설정되었는지 검증

○ PDU Router 모듈이 I-PDU의 성공적인 전송을 알리는 함수 안정화

- 기능

설 명	o I-PDU가 물리적인 전송 계층인 CAN 드라이버에서 전송되면, 전송이 완료되었음을 알리는 Confirmation을 상위 계층으로 올려주는데, CAN Interface 모듈에서 Confirmation을 PDU Router 모듈에 전달하기 위해서 호출하는 함수를 제공

- 절차

안정화 수행 방법	(1) 시험 프로그램은COM Cluster (COM, PDU Router, CAN Interface, CAN Driver)와 시험을 위한 응용 프로그램으로 구성한다. (2) 응용 프로그램은 COM 모듈의 시그널 전송 API인 COM_SendSignal 함수를 주기적으로 호출한다. (3) COM모듈의 시그널 전송 API가 호출됨으로써 AUTOSAR SPEC.에서 규정하는 절차에 따라 PDU Router, CAN Interface 내의 상응하는 API가 호출되도록 한다. (4) PduR_CanIfTxConfirmation 함수를 호출하고, Com_TxConfirmation 함수가 제대로 호출되는지 디버깅을 통해서 확인한다.
세부 분석 및 수정	(1) 응용프로그램에서 Com_SendSignal 함수를 호출하여, 정상 동작을 수행하는지 확인 (2) CanIf_TxConfirmation 함수에서 PduR_CanIfTxConfirmation 함수를 호출하여, 인자로 전달된 CanTxPduId의 값이 전송 요청한 I-PDU ID 인지를 확인하고, PduR_CanIfTxConfirmation 함수에서 Com_TxConfirmation 함수가 성공적으로 호출되는지 검증

○ PDU Router 모듈이 I-PDU가 수신되었음을 알리는 함수 안정화

- 기능

설 명	o I-PDU가 물리적인 전송 계층인 CAN 드라이버에서 수신되면, I-PDU가 수신되었음을 알려는 Indication을 상위 계층으로 전송하는데, CAN Interface 모듈이 수신된 I-PDU의 ID와 그 데이터 값을 PDU Router에게 전송하는 함수를 제공

- 절차

안정화 수행 방법	(1) 시험 프로그램은COM Cluster (COM, PDU Router, CAN Interface, CAN Driver)와 시험을 위한 응용 프로그램으로 구성한다. (2) 송신용 보드의 응용 프로그램은 COM 모듈의 시그널 전송 API인 COM_SendSignal 함수를 주기적으로 호출한다. (3) 송신용 보드 COM모듈의 시그널 전송 API가 호출됨으로써 AUTOSAR SPEC.에서 규정하는 절차에 따라 PDU Router, CAN Interface 내의 상응하는 API가 호출되도록 한다. (4) 수신용 보드에서는 수신된 데이터를 처리하기 위해서, AUTOSAR SPEC.에서 규정하는 절차에 따라, CAN Interface, PDU Router내의 상응하는 API가 호출되도록 한다. (5) 수신용 보드에서 PduR_CanIfRxIndication 함수를 호출하고, I-PDU와 ID와 시그널 값이 제대로 전달되는지 디버깅을 통해서 확인한다.
세부 분석 및 수정	(1) 송신용 응용프로그램에서 Com_SendSignal 함수를 호출하여 정상 동작 수행을 확인 (2) 수신용 보드에서 CanIf_RxIndication 함수가 PduR_CanIfRxIndication 함수를 호출하여, 함수 인자 CanRxPduId와 CanSduPtr에 제대로 된 값이 들어가 있고, Com_RxIndication 함수가 성공적으로 호출되는지 디버깅을 통해 확인

(다) CAN Interface 모듈

① CAN Interface 모듈 안정화 수행 항목

CAN Interface는 하위 계층인 CAN Driver와 상위 계층인 PDU Router, CAN State Manager, ECU State Manager 등과 같은 COM 서비스 계층의 사이에 위치하여 CAN 시그널의 송수신에 관한 표준 인터페이스를 제공한다.

CAN 시그널의 이동 경로는 PDU Router - CAN Interface - CAN Driver 계층 사이에서 구성된다. CAN Interface는 PDU Router로부터 L-PDU의 전송에 대한 전송 요청을 받아들여 하위의 CAN Driver에 이를 전달하며, 반대로 CAN Driver로부터 성공적인 데이터의 수신을 통보 받아 이를 PDU Router에 전달하는 역할을 수행한다.

본 절에서는 아래에 열거한 CAN Interface의 주요 기능에 대하여 정상적인 수행 여부를 검증하고, 오류 발견 시 이를 보완하는 작업을 수행하고자 한다.

○ CAN Interface 전송 요청 서비스
○ CAN Interface 전송 요청 시의 메시지 버퍼링
○ CAN Interface가 상위 계층에 전송 성공을 확인시켜주는 기능
○ CAN Driver에서 성공적으로 수신된 메시지에 대한 CAN Interface의 인지 기능
○ CAN Driver에서 전달된 메시지 수신 여부를 상위 계층인 PDU Router에 전달하는 기능
○ 상위 계층의 요구에 의해 CAN Interface가 CAN 메시지를 읽어 전달하는 기능

② CAN Interface 모듈 안정화

○ CAN Interface 전송 요청 서비스

- 기능

설명	o COM 클러스터에서 signal 전송 요청이 발생할 때에 CAN Interface 계층에서 이에 상응하여 CanIf_Transmit API가 호출, 실행되는 것을 확인

- 절차

안정화 수행 방법	(1) 시험 프로그램은 COM Cluster (COM, PDU Router, CAN Interface, CAN Driver)와 시험을 위한 응용 프로그램으로 구성한다. (2) 응용 프로그램은 COM 모듈의 시그널 전송 API를 주기적으로 호출한다. (3) COM모듈의 시그널 전송 API가 호출됨으로써 AUTOSAR SPEC.에서 규정하는 절차에 따라 PDU Router, CAN Interface 내의 상응하는 API가 호출되도록 한다. (4) CAN Interface 모듈의 전송 요청 서비스인 CanIf_Transmit 함수 내에 LED를 켜는 루틴을 삽입하여 해당 API가 절차에 따라 호출, 수행되었음을 확인하도록 한다. (5) 디버거를 통해 CanIf_Transmint에 전달된 argument의 값을 확인한다.
세부 분석 및 수정	(1) 응용프로그램에서 Com_SendSignal 함수를 호출하여 CanIf_Transmit 함수가 실행될 때 LED가 ON 됨을 확인 (2) 디버거를 통해 CanIf_Transmit의 argument의 값을 확인하여 PduInfoPtr이 지시하는 구조체의 멤버인 SduLength, SduDataPtr의 값이 전송하려는 값과 일치함을 확인하고, CanTxPduId 값이 전송하려는 값과 일치함을 확인함

○ CAN Interface 전송 요청 시의 메시지 버퍼링

- 기능

설명	o CAN Interface에서의 전송 요청이 CAN Driver에 의해 받아들여지지 않을 경우, 전송이 시도된 메시지가 내부 버퍼에 저장되는 것을 확인

- 절차

안정화 수행 방법	(1) 시험 프로그램은 COM Cluster (COM, PDU Router, CAN Interface, CAN Driver)와 시험을 위한 응용 프로그램으로 구성한다. (2) 응용 프로그램은 COM 모듈의 시그널 전송 API를 호출한다. (3) COM모듈의 시그널 전송 API가 호출됨으로써 AUTOSAR SPEC.에서 규정하는 절차에 따라 PDU Router, CAN Interface, CAN Driver 내의 상응하는 API가 호출되도록 한다. (4) CAN Interface의 전송 요청에 의해 호출되는 CAN Driver API인 Can_Wrie 함수가 CAN_BUSY를 반환하도록 수정 (5) Can_Write의 CAN_BUSY 반환에 대해 CAN Interface가 전송을 시도한 메시지를 내부의 버퍼 (TxBuffer)에 저장하는 것을 디버거를 통해 확인한다.
세부 분석 및 수정	(1) 응용프로그램에서 Com_SendSignal 함수를 호출 → CanIf_Transmit 함수가 실행 → Can_Wrtie 함수가 실행되는 것을 확인 (2) Can_Wire에서 CAN_BUSY 가 반환됨에 따라 CanIf_Transmit 함수 내에서 메시지를 내부 버퍼 (TxBuffer[ ])에 저장한 것을 디버거를 통해 확인하여 저장된 메시지와 전송을 요청한 메시지가 동일함을 확인

○ CAN Interface가 상위 계층에 전송 성공을 확인시켜주는 기능

- 기능

설명	o CAN Interface에서 발생한 전송 요청에 의해 CAN Driver가 성공적으로 메시지를 전송했을 경우, CAN Interface가 이를 상위 계층(PDU Router)에 알리는 기능을 확인

- 절차

안정화 수행 방법	(1) 시험 프로그램은 COM Cluster (COM, PDU Router, CAN Interface, CAN Driver)와 시험을 위한 응용 프로그램으로 구성한다. (2) Tx의 응용 프로그램은 COM 모듈의 시그널 전송 API를 호출함으로써 메시지를 전송 한다. (3) COM모듈의 시그널 전송 API가 호출됨으로써 AUTOSAR SPEC.에서 규정하는 절차에 따라 PDU Router, CAN Interface, CAN Driver 내의 상응하는 API가 호출되도록 한다. (4) CAN Interface 모듈의 전송 요청 서비스인 CanIf_Transmit 함수가 호출되면 CAN Driver의 Can_Write 함수를 호출함으로써 CAN 메시지의 물리적인 전송을 구현한다. (5) CAN Driver를 통해 메시지가 성공적으로 전송되었을 때, 전송 인터럽트의 문맥에서 CAN Interface에 전송 성공을 알리는 기능을 구현한다. (CanIf_TxConfirmation) (6) CanIf_TxConfirmation 을 통해 성공적인 전송에 대한 알림이 발생할 경우 이를 PDU Router에 전달하기 위해 PduR_CanIfTxConfirmation 을 호출한다. (7) PduR_CanIfTxConfirmation 내에 LED를 켜는 루틴을 삽입하여 해당 API가 절차에 따라 호출되었음을 확인하도록 한다. (8) PduR_CanIfTxConfirmation 의 인수로 전달되는 메시지의 ID가 CAN Driver에서 전송한 메시지 ID와 동일한 것임을 확인한다.
세부 분석 및 수정	(1) 응용프로그램에서 Com_SendSignal 함수를 호출하여 LED가 ON 됨을 확인 (2) 디버거를 통해 PduR_CanIfTxConfirmation의 argument의 값을 확인하여 CanTxPduId가 CAN Driver에서 전송한 메시지의 ID와 동일함을 확인

○ CAN Driver에서 성공적으로 수신된 메시지에 대한 CAN Interface의 인지 기능

- 기능

설명	o CAN Driver를 통해 메시지가 성공적으로 수신될 경우 CAN Driver는 이를 CAN Interface에 통보하게 되므로 CAN Interface가 전달받게 되는 메시지 관련 정보 즉, 수신된 메시지의 ID, 길이, 및 페이로드를 확인함

- 절차

안정화 수행 방법	(1) 시험 프로그램은 COM Cluster (COM, PDU Router, CAN Interface, CAN Driver)와 시험을 위한 응용 프로그램으로 구성한다. (2) Tx의 응용 프로그램은 COM 모듈의 시그널 전송 API를 호출함으로써 메시지를 전송 한다. (3) Rx의 응용 프로그램은 CAN Driver 모듈에 메시지가 수신될 경우 이를 감지하기 위해 loop 문을 수행하며 인터럽트가 발생되기를 기다린다. (4) Rx에서 CAN 메시지가 버스를 통해 H/W에 의해 수신될 경우 수신 인터럽트가 발생하며, 이를 처리하기 위해 CAN Driver 모듈에 인터럽트 서비스 루틴을 구현한다. (5) 메시지의 수신에 의해 발생한 인터럽트의 서비스 루틴 내에서 CAN Interface 모듈에 메시지의 수신을 알리기 위한 call-back 함수인 CanIf_RxIndicaton 함수를 호출하며 인터럽트가 발생했음을 알리기 위해 LED를 켜도록 한다. (6) CanIf_RxIndication 함수 내에 LED를 켜는 루틴을 삽입하여 해당 API가 절차에 따라 호출되었음을 확인하도록 한다. (7) CanIf_RxIndication 함수의 arguments를 확인하여 CAN Driver에서 수신된 메시지의 정보와 일치하는 지를 확인한다.
세부 분석 및 수정	(1) Tx의 응용프로그램에서 Com_SendSignal 함수를 호출하여 LED가 ON 됨을 확인 (2) Rx의 응용프로그램에서 인터럽트가 발생할 때를 기다리

	기 위해 loop 문을 실행하여, 메시지의 수신에 의해 발생하는 인터럽트에 대한 서비스 루틴에서 LED가 ON 되고 CanIf_RxIndication이 호출될 때 LED가 ON 되는 것을 확인 (3) 디버거를 통해CanIf_RxIndication의 arguments를 확인하여 CanId, CanDlc, CanSduPtr을 CAN Driver에서 수신된 메시지와 비교했을 때에 정확하게 일치함을 확인

○ CAN Driver에서 전달된 메시지 수신 여부를 상위 계층인 PDU Router에 전달하는 기능

- 기능

설명	o CAN Driver를 통해 메시지가 성공적으로 수신될 경우 CAN Driver는 이를 CAN Interface에 통보하게 되고 CAN Interface는 이를 다시 PDU Router에 전달하게 되므로, PDU Router로의 전달여부를 확인하는 동시에 전달되는 메시지 관련 정보 즉, 메시지의 ID와 페이로드를 확인함

- 절차

안정화 수행 방법	(1) 시험 프로그램은 COM Cluster (COM, PDU Router, CAN Interface, CAN Driver)와 시험을 위한 응용 프로그램으로 구성한다. (2) Tx의 응용 프로그램은 COM 모듈의 시그널 전송 API를 호출함으로써 메시지를 전송 한다. (3) Rx의 응용 프로그램은 CAN Driver 모듈에 메시지가 수신될 경우 이를 감지하기 위해 loop 문을 수행하며 인터럽트가 발생되기를 기다린다. (4) Rx에서 CAN 메시지가 버스를 통해 H/W에 의해 수신될 경우 수신 인터럽트가 발생하며, 이를 처리하기 위해 CAN Driver 모듈에 인터럽트 서비스 루틴을 구현한다. (5) 메시지의 수신에 의해 발생한 인터럽트의 서비스 루틴 내에서 CAN Interface 모듈에 메시지의 수신을 알리기

	위한 call-back 함수인 CanIf_RxIndicaton 함수를 호출하도록 한다. (6) CanIf_RxIndication 함수 내에서 PDU Router의 call-back 함수인 PduR_CanIfRxIndication 을 호출하도록 한다. (7) PduR_CanIfRxIndication 함수 내에 LED를 켜는 루틴을 삽입하여 해당 API가 절차에 따라 호출되었음을 확인하도록 한다. (8) PduR_CanIfRxIndication 함수의 arguments를 확인하여 CAN Driver에서 수신된 메시지의 정보와 일치하는 지를 확인한다.
세부 분석 및 수정	(1) Tx의 응용프로그램에서 Com_SendSignal 함수를 호출하여 LED가 ON 됨을 확인 (2) Rx의 응용프로그램에서 인터럽트가 발생할 때를 기다리기 위해 loop 문을 실행하여 PduR_CanIfRxIndication이 호출될 때 LED가 ON 되는 것을 확인 (3) 디버거를 통해PduR_CanIfRxIndication의 arguments를 확인하여 dataId, CanSduPtr을 CAN Driver에서 수신된 메시지와 비교했을 때에 정확하게 일치함을 확인

○ 상위 계층의 요구에 의해 CAN Interface가 CAN 메시지를 읽어 전달하는 기능

- 기능

설명	o CAN Driver의 메시지 수신 인터럽트에 의한 Indication 절차에 따르지 않고 상위 계층의 요구에 의해 CAN 메시지를 읽어 들이는 기능을 확인

- 절차

안정화 수행 방법	(1) 시험 프로그램은 COM Cluster (COM, PDU Router, CAN Interface, CAN Driver)와 시험을 위한 응용 프로그램으로 구성한다. (2) Tx의 응용 프로그램은 COM 모듈의 시그널 전송 API

	를 호출함으로써 메시지를 전송 한다. (3) Rx의 응용 프로그램은 CAN Interface의 API (CanIf_ReadRxPduData)를 호출하여 CAN 메시지를 읽어 들이도록 요청한다. (4) CanIf_ReadRxPduData 함수 내에 LED를 켜는 루틴을 삽입하여 해당 API가 절차에 따라 호출되었음을 확인하도록 한다. (5) CanIf_ReadRxPduData 는 CAN Driver의 Can_Write 함수를 호출하여 CAN 컨트롤러에 입력된 메시지를 읽도록 구현한다. (6) CanIf_ReadRxPduData 함수의 arguments를 확인하여 CAN Driver에서 수신된 메시지의 정보와 일치하는 지를 확인 한다.
세부 분석 및 수정	(1) Tx의 응용프로그램에서 Com_SendSignal 함수를 호출하여 오류없이 실행됨을 확인 (2) Rx의 응용프로그램에서 CAN Interface의 CanIf_ReadRxPduData를 호출하여 메시지 읽기를 요청함에 따라 CanIf_ReadRxPduData이 호출될 때 LED가 ON 됨을 확인 (3) 디버거를 통해CanIf_ReadRxPduData의 arguments를 확인하여 CanRxPduId, PduInfoPtr을 CAN Driver에서 수신된 메시지와 비교했을 때에 정확하게 일치함을 확인

(라) CAN Driver 모듈

① CAN Driver 모듈 안정화 수행 항목

CAN Driver는 AUTOSAR CAN 스택의 최하위에 위치하여 CAN 컨트롤러와 같은 하드웨어에 대한 접근을 담당하며 상위 계층에 하드웨어에 대한 독립적인 API를 제공한다. CAN Driver는 CAN 버스를 통한 데이터의 전송을 실행하며 데이터의 수신과 같은 하드웨어적 이벤트에 대해 CAN Interface의 Callback 함수를 호출하는 역할을 수행한다. 또한 CAN 컨트롤러의 동작 모드와 상태를 제어할 수 있는 여러 가

지 기능을 제공한다.

본 절에서는 아래에 열거한 CAN Driver의 주요 기능에 대하여 정상적인 수행 여부를 검증하고, 오류 발견 시 이를 보완하는 작업을 수행하고자 한다.

○ CAN Driver에 의한 인터럽트 활성화 기능
○ CAN Driver의 전송 요청 서비스
○ CAN Driver가 상위 계층에 전송의 성공을 확인시켜주는 기능
○ CAN Driver가 성공적으로 수신한 메시지에 대해 CAN Interface에 통보해주는 기능

② CAN Driver 모듈 안정화

○ CAN Driver에 의한 인터럽트 활성화 기능

- 기능

설명	o CAN Driver는 메시지의 전송과 수신에 있어서 인터럽트를 활용하게 되며, 인터럽트를 활성화 시키는 기능의 동작 여부를 확인하기 위해 메시지의 전송과 수신 시에 각각 인터럽트가 발생하는 지 확인

- 절차

안정화 수행 방법	(1) 시험 프로그램은 COM Cluster (COM, PDU Router, CAN Interface, CAN Driver)와 시험을 위한 응용 프로그램으로 구성한다. (2) Tx의 응용 프로그램은 COM 모듈의 시그널 전송 API를 호출함으로써 메시지를 전송 한다. (3) Rx의 응용 프로그램은 CAN Driver 모듈에 메시지가 수신될 경우 이를 감지하기 위해 loop 문을 수행하며 인터럽트가 발생되기를 기다린다. (4) Tx와 Rx에서 각각 CAN 컨트롤러의 인터럽트를 활성화 시킨다. (5) Tx에서 CAN 메시지가 성공적으로 전송되었을 때 발생하는 인터럽트를 처리하기 위한 서비스 루틴을 구현한

	다. (6) Rx에서 CAN 메시지가 성공적으로 수신되었을 때 발생하는 인터럽트를 처리하기 위한 서비스 루틴을 구현한다. (7) Tx와 Rx의 인터럽트 서비스 루틴 내에 LED를 켜는 루틴을 삽입하여 해당 인터럽트가 발생되었음을 확인하도록 한다.
세부 분석 및 수정	(1) Tx의 응용프로그램에서Com_SendSignal 함수를 호출하여 Tx에서 전송 인터럽트가 발생함에 따라 이를 처리하기 위한 서비스 루틴이 실행될 때에 LED가 ON 됨을 확인 (2) Rx의 응용프로그램에서 인터럽트가 발생할 때를 기다리기 위해 loop 문을 실행하고, Rx에서 메시지가 성공적으로 수신 되었을 때 수신 인터럽트가 발생함에 따라 이를 처리하기 위한 서비스 루틴이 실행될 때에 LED가 ON 됨을 확인

○ CAN Driver의 전송 요청 서비스

- 기능

설명	o COM 클러스터에서 signal 전송 요청이 발생할 때에 CAN Driver 계층에서 이에 상응하여 Can_Write API가 호출, 실행되는 것을 확인

- 절차

안정화 수행 방법	(1) 시험 프로그램은 COM Cluster (COM, PDU Router, CAN Interface, CAN Driver)와 시험을 위한 응용 프로그램으로 구성한다. (2) 응용 프로그램은 COM 모듈의 시그널 전송 API를 주기적으로 호출한다. (3) COM모듈의 시그널 전송 API가 호출됨으로써 AUTOSAR SPEC.에서 규정하는 절차에 따라 PDU

	Router, CAN Interface, CAN Driver 내의 상응하는 API가 호출되도록 한다. (4) CAN Driver 모듈의 전송 요청 서비스인 Can_Wrtie 함수 내에 LED를 켜는 루틴을 삽입하여 해당 API가 절차에 따라 호출, 수행되었음을 확인하도록 한다. (5) 디버거를 통해 Can_Write에 전달된 argument의 값을 확인한다.
세부 분석 및 수정	(1) 응용프로그램에서 Com_SendSignal 함수를 호출하여 Can_Write 함수가 실행될 때 LED가 ON 됨을 확인 (2) 디버거를 통해 Can_Write의 argument인 PduInfoPtrk가 지시하는 구조체의 멤버인 SduLength, SduDataPtr의 값이 전송하려는 값과 일치함을 확인

○ CAN Driver가 상위 계층에 전송의 성공을 확인시켜주는 기능

- 기능

설명	o CAN Interface에서 발생한 전송 요청에 의해 CAN Driver가 성공적으로 메시지를 전송했을 경우, CAN Driver가 이를 상위 계층 (CAN Interface)에 알리는 기능을 확인

- 절차

안정화 수행 방법	(1) 시험 프로그램은 COM Cluster (COM, PDU Router, CAN Interface, CAN Driver)와 시험을 위한 응용 프로그램으로 구성한다. (2) Tx의 응용 프로그램은 COM 모듈의 시그널 전송 API를 호출함으로써 메시지를 전송 한다. (3) COM모듈의 시그널 전송 API가 호출됨으로써 AUTOSAR SPEC.에서 규정하는 절차에 따라 PDU Router, CAN Interface, CAN Driver 내의 상응하는 API가 호출되도록 한다. (4) CAN Interface 모듈의 전송 요청 서비스인 CanIf_Transmit 함수가 호출되면 CAN Driver의 Can_Write 함수를 호출함으로써 CAN 메시지의 물리적

	인 전송을 구현한다. (5) CAN Driver를 통해 메시지가 성공적으로 전송되었을 때, 전송 인터럽트의 문맥에서 CAN Interface에 전송 성공을 알리는 기능을 구현한다. 이는 CAN Interface의 call-out 함수인 CanIf_TxConfirmation 를 호출하는 것으로 구현된다. (6) CanIf_TxConfirmation 내에 LED를 켜는 루틴을 삽입하여 해당 API가 절차에 따라 호출되었음을 확인하도록 한다. (7) CanIf_TxConfirmation 의 인수로 전달되는 메시지의 ID가 CAN Driver에서 전송한 메시지 ID와 동일한 것임을 확인한다.
세부 분석 및 수정	(1) 응용프로그램에서 Com_SendSignal 함수를 호출하여 LED가 ON 됨을 확인 (2) 디버거를 통해 CanIf_TxConfirmation의 argument인 CanTxPduId가 CAN Driver에서 전송한 메시지의 ID와 동일함을 확인

○ CAN Driver가 성공적으로 수신한 메시지에 대해 CAN Interface에 통보해주는 기능

- 기능

설명	o CAN Driver를 통해 메시지가 성공적으로 수신될 경우 CAN Driver는 이를 CAN Interface에 통보하게 되므로 CAN Interface가 전달받게 되는 메시지 관련 정보 즉, 수신된 메시지의 ID, 길이, 및 페이로드를 확인함

- 절차

안정화 수행 방법	(1) 시험 프로그램은 COM Cluster (COM, PDU Router, CAN Interface, CAN Driver)와 시험을 위한 응용 프로그램으로 구성한다. (2) Tx의 응용 프로그램은 COM 모듈의 시그널 전송 API

	를 호출함으로써 메시지를 전송 한다. (3) Rx의 응용 프로그램은 CAN Driver 모듈에 메시지가 수신될 경우 이를 감지하기 위해 loop 문을 수행하며 인터럽트가 발생되기를 기다린다. (4) Rx에서 CAN 메시지가 버스를 통해 H/W에 의해 수신될 경우 수신 인터럽트가 발생하며, 이를 처리하기 위해 CAN Driver 모듈에 인터럽트 서비스 루틴을 구현한다. (5) 메시지의 수신에 의해 발생한 인터럽트의 서비스 루틴 내에서 CAN Interface 모듈에 메시지의 수신을 알리기 위한 call-back 함수인 CanIf_RxIndicaton 함수를 호출하며 인터럽트가 발생했음을 알리기 위해 LED를 켜도록 한다. (6) CanIf_RxIndication 함수 내에 LED를 켜는 루틴을 삽입하여 해당 API가 절차에 따라 호출되었음을 확인하도록 한다. (7) CanIf_RxIndication 함수의 arguments를 확인하여 CAN Driver에서 수신된 메시지의 정보와 일치하는 지를 확인한다.
세부 분석 및 수정	(1) Tx의 응용프로그램에서 Com_SendSignal 함수를 호출하여 LED가 ON 됨을 확인 (2) Rx의 응용프로그램에서 인터럽트가 발생할 때를 기다리기 위해 loop 문을 실행하고 메시지의 수신에 의해 발생하는 인터럽트에 대한 서비스 루틴에서 LED (D23)가 ON 됨을 확인, CanIf_RxIndication이 호출될 때 LED (D25)가 ON 됨을 확인 (3) 디버거를 통해CanIf_RxIndication의 arguments인 CanId, CanDlc, CanSduPtr을 CAN Driver에서 수신된 메시지와 비교했을 때에 정확하게 일치함을 확인

(마) FlexRay 통신 서비스 계층

① 개요

○ FlexRay 통신 서비스 계층의 정의는 AUTOSAR 표준 2.1을 따름

- FlexRay 통신 서비스 계층은 전송 (Transport) 모듈, 네트워크 관리자 (Network Manager) 모듈, 상태 관리자 (State Manager) 모듈을 지칭함

- FlexRay 통신 서비스 계층의 구현은 타겟 보드의 마이크로프로세서나 컨트롤러와 트랜시버의 특성이나 상황을 고려하여 구현되고 특히 드라이버들은 타겟 보드에 의존적임

- 기능 규격 및 개발 방법을 표준에 의거하여 개발함

- AUTOSAR 규격에서 제시하는 구현 방법과 소프트웨어 설정 방법에 따라서 구현함

- 컴파일 전, 링크, 컴파일 후의 소프트웨어 설정방법과 스케줄링 방법을 준수함

- AUTOSAR의 Overview of Software Layers 표준에 의거하여 제공하는 모든 서비스는 “Standardized Interface"로 정의되어 있으므로 모든 서비스는 C 프로그래밍 언어의 함수로 제공됨

- FlexRay 통신 서비스 계층을 구현하기 위해 필요한 모듈의 이름과 표준스펙정의 문서는 표 1과 같음

[표 1 : FlexRay Service Layer 개발 모듈 및 표준스펙]

개발모듈	표준스펙정의 문서
FlexRay Transport Protocol	Specification of FlexRay Transport Layer V2.2.1
FlexRay State Manager	Specification of FlexRay State Manager V1.0.2
FlexRay Network Manager	Specification of FlexRay Network Management V3.0.2

② FlexRay 통신 서비스 모듈의 세부 내용

- 기본 소프트웨어 계층 내에 구현되는 모듈들은 FlexRay 네트워크의 공통적인 인터페이스를 제공하여, 상위 어플리케이션에서 프로토콜과 메시지 특성들과 관계없이 인터페이스 사용이 가능하도록 해줌

- AUTOSAR 통신 모듈, 일반적인 네트워크 관리자, 인터페이스 모듈, 통신 진단 관리자 (Diagnostic Communication Manager, DCM) 모듈 등은 모든 자동차 네트워크 시스템에 동일하게 구성되며 모든 ECU마다 존재함

- 네트워크 관리자는 시스템 서비스의 통신 관리자와 연결되며 디스패처 (Dispatcher) 역할을 수행함

- 상태 관리자는 하드웨어 장치를 시작, 종료시킴

- PDU 라우터는 서로 다른 추상화된 통신 컨트롤러들과 상위 계층의 모듈들 사이에서 통신 경로를 제공함

- FlexRay 통신 서비스 계층에 대한 세부 규격 요약은 표 3-3과 같음

[표 3-3: FlexRay 통신 서비스 계층]

모듈명	주요 기능
FlexRay State Manager	o 하나의 FlexRay 클러스터에서 통신을 시작, 종료시키기 위하여 AUTOSAR 통신 관리자에게 추상화된 인터페이스를 제공
FlexRay Transport	o PDU Router와 FlexRay 인터페이스 사이에서 데이터 전달 - 송수신 시 데이터 분리 또는 조합 - 데이터 흐름 제어 가능 - 에러 검출 가능 - 확인 응답을 보내고, 재시도 가능 - 1:1, 1:n 연결 가능 - 2Bytes 또는 4Bytes 주소 정보 포함 - 232-1Bytes까지의 payload 전송 가능 - ISO 15765-2와 ISO 15765-4 표준에 준하여 구성
FlexRay Network Management	o 네트워크의 일반 동작모드와 bus-sleep 모드 등을 관리하여 네트워크 변화 상태를 알려줌 o 다른 방식의 네트워크와의 Gateway 역할 수행

③ FlexRay 통신 서비스 계층의 설계

○ 설계를 위한 요구 사항
- 각각 10MBit/s의 속도를 가지는 최대 2개의 채널 지원함
- 다양한 FlexRay Cluster를 제공함
- 동기 및 비동기 통신이 가능하도록 함
- FlexRay 프레임에서 PDU 데이터들의 분리, 조합이 가능해야 함
- 상위 계층에서는 PDU를 기반으로 제어되는 API들로 구성된 인터페이스를 사용할 수 있음
- 네트워크 관리를 위한 기본적인 기능을 제공함
- FlexRay 통신의 상태와 버스의 에러 상태를 제공함

○ FlexRay 네트워크 시스템 분석
- AUTOSAR 기반의 각 모듈들은 계층적 구조를 이루고 있음
- FlexRay 네트워크 관련 기본 소프트웨어 모듈들도 계층적 구조를 이루고 있으며

[그림 3-17]에서 구조 및 인터페이스를 보여주고 있음

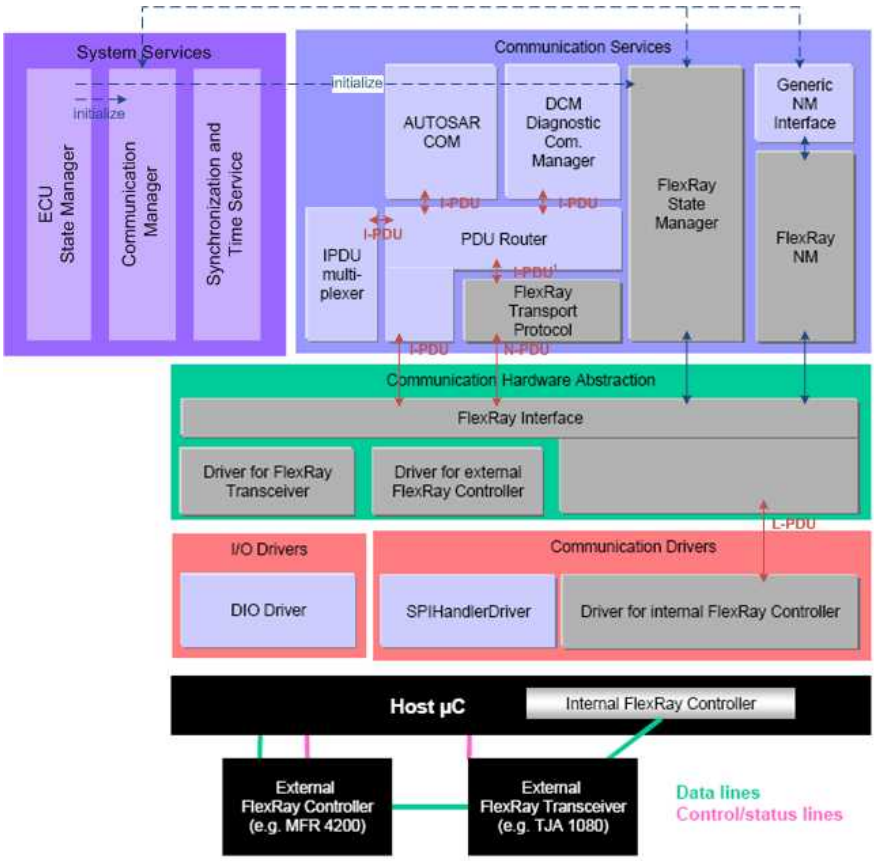


그림 3-17. FlexRay 기본 소프트웨어 모듈 모듈도

- AUTOSAR는 통신을 위하여 자동차 네트워크의 공통적인 기본 인터페이스를 제공함
- FlexRay 네트워크를 사용하는 차량 소프트웨어의 어플리케이션은 계층화된 구조의 모듈들을 이용하여 프로토콜과 메시지 속성을 자세히 알지 못하여도 통신 기능을 사용할 수 있음
- FlexRay 상태 관리자는 초기화 및 종료 등의 제어 기능을 통하여 통신 시스템을 운영 및 관리함
- FlexRay 상태 관리자는 PDU를 보내거나 신호 타임아웃을 체크하는 등 통신상에서 필요한 다양한 옵션 기능들도 수행함
- FlexRay 전송 프로토콜 (Transport Protocol) 모듈은 PDU 라우터와 연결되어 송수신되는 데이터를 분리 또는 조합하여 전달하는 기능을 수행함

- FlexRay 통신 서비스 계층의 하위에는 하드웨어의 제어와 연관이 있는 통신 하드웨어 추상화 계층 (Communication Hardware Abstraction Layer)이 있음
- FlexRay 인터페이스는 FlexRay 드라이버와 연결되어 드라이버의 여러 함수들을 사용할 수 있는 인터페이스를 제공함
- 물리적인 하드웨어를 직접 사용하는 FlexRay 드라이버는 최하위단인 통신 드라이버 계층 (Communication Drivers Layer)의 모듈임
- FlexRay 드라이버는 하드웨어의 레지스터 설정 및 초기화 등 물리적인 제어가 가능하도록 인터페이스를 제공해줌
- 일반적으로 FlexRay 인터페이스의 함수와 FlexRay 드라이버의 함수들은 일대일로 대응되어 인터페이스의 함수를 사용한 드라이버 제어가 가능함
- FlexRay 상태 관리자는 [그림 3-18]와 같이 FlexRay CC (Communication Controller)의 POC (Protocol Operation Control) 상태와 대응되는 관리자 상태가 있으며 이 상태에 따라 FlexRay 프로토콜 네트워크가 동작함

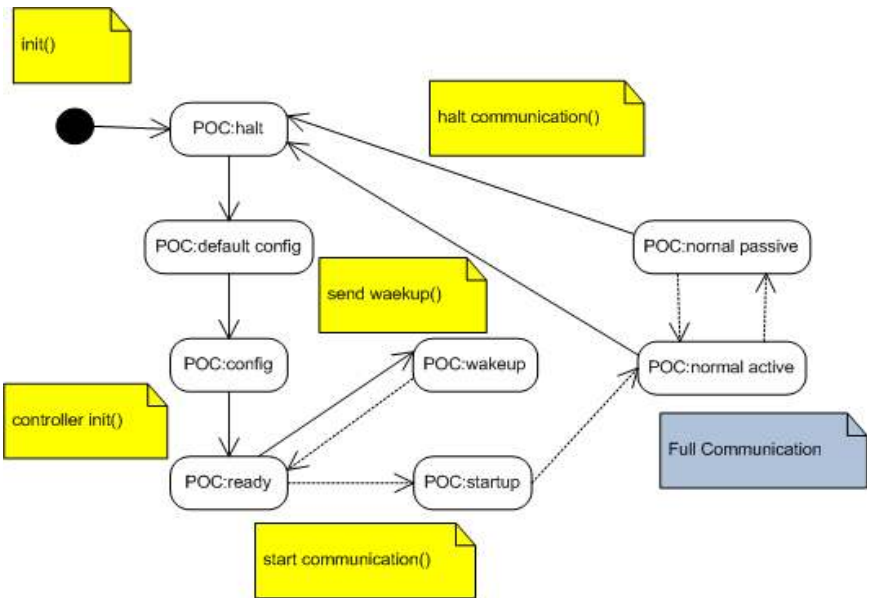


그림 3-18. 동작 함수에 따른 FlexRay CC의 POC 상태 변화

- AUTOSAR 기본 소프트웨어의 각 모듈 및 드라이버들은 AUTOSAR 서비스 계층의 ECU 관리자에 의해 초기화됨
- FlexRay 네트워크는 상태 체크 및 동작을 위해 FlexRay 상태 관리자에 의해 시작

됨

- FlexRay 네트워크는 타임 트리거 (time-triggered) 방식의 네트워크임
- 네트워크의 동작은 BSW 스케쥴러 (Basic Software Scheduler)의 FIXED-CYCLIC 시간에 각 모듈의 메인 함수가 호출되면서 수행됨
- 예를 들어 FlexRay 인터페이스에서는 호출된 메인함수에서 전역 시간 (global time)을 얻어오고, FlexRay 클러스터에 해당하는 FlexRay Job List를 찾은 후, Absolute Timer Interrupt를 사용 가능하도록 만듦

○ FlexRay 통신 서비스 계층의 주요 동작 과정

▸ 통신의 초기화

- 각 모듈들은 필요한 인터페이스를 제공하며 계층적 구조화를 이루어 서로 연결되어 있음
- [그림 3-19]에서는 각 모듈들의 초기화 과정을 나타내었음
- 각 모듈의 초기화는 서비스 계층의 ECU 매니저에 의해 수행됨
- 초기화는 정상적인 하드웨어의 사용을 위해 반드시 선행적으로 이루어져야 함
- 그리고 연관되어 있는 모듈의 사용을 위해 순차적인 초기화가 이루어져야 함
- TP 모듈의 초기화의 경우 ECU 관리자에 의해 초기화된 AUTOSAR 통신 관리자를 통하여 수행됨
- FlexRay CC 하드웨어가 초기화되기 위해서는 FlexRay 상태 관리자가 FlexRay 인터페이스를 통하여 FlexRay 드라이버의 Fr_ControllerInit() 함수를 수행하여야 함
- POC:halt 상태의 FlexRay CC는 FlexRay Cluster 및 노드의 환경 매개변수 값들을 초기화하고 데이터 메시지 버퍼와 같은 송수신에 필요한 리소스들을 초기화하면서 POC:ready 상태로 변경됨

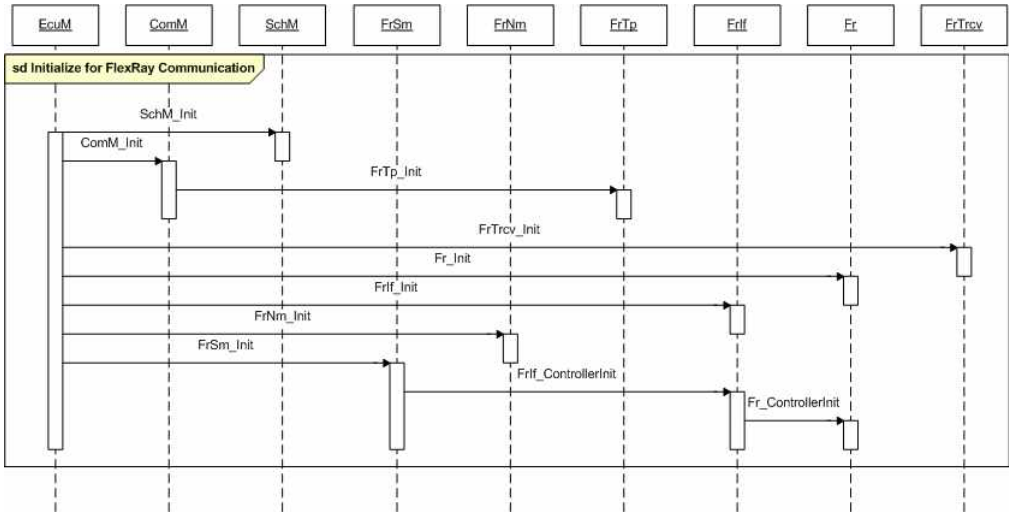


그림 3-19. FlexRay 통신 초기화

▸ 통신 시작 과정

- [그림 3-20]에서는 초기화 과정 이후의 통신 시작 과정을 나타내고 있음
- AUTOSAR 통신 관리자에게서 COMM_FULL_COMMUNICATION 모드 설정으로 변경이 요청되면 FlexRay 상태 관리자는 FlexRay 통신 시작을 준비함
- FlexRay 상태 관리자 동작은 메인함수에 의해 수행됨
- 동작 수행 전 FlexRay 인터페이스를 이용하여 FlexRay CC의 POC 상태를 알아봄
- 만약 통신이 가능한 상태라면 FlexRay 드라이버에서 CHI (Controller Host Interface) 명령을 FlexRay CC에 보내어 FlexRay 통신이 시작됨

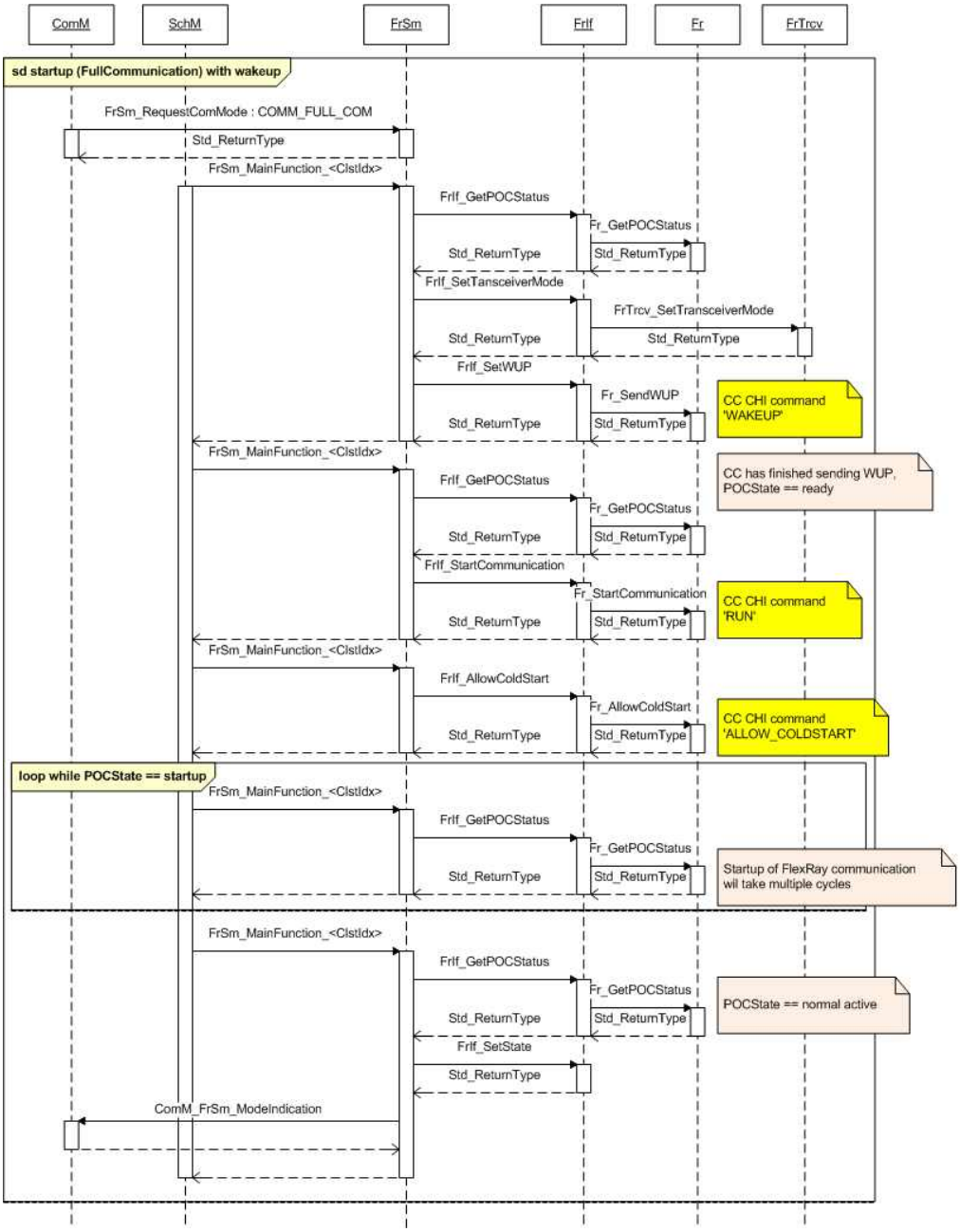


그림 3-20. FlexRay 통신 시작 (Wakeup 포함)

▸ FlexRay TP를 통한 데이터의 송신과 수신

- 통신 시작 과정을 수행하는 동안 FlexRay CC의 POC 상태는 POC:startup을 거쳐 POC:normal active 상태로 변경됨

- POC:normal active 상태는 정상적인 통신 상태로, 다른 통신 모드를 요청하지 않는 다면 FlexRay 프레임 데이터의 송수신이 가능함
- 데이터 송수신은 AUTOSAR의 계층화된 구조의 각 모듈들을 순차적으로 거치면서 수행됨
- [그림 3-21]은 이러한 메시지 송수신 중 TP를 통한 데이터 송신 시퀀스를 나타내고 있음

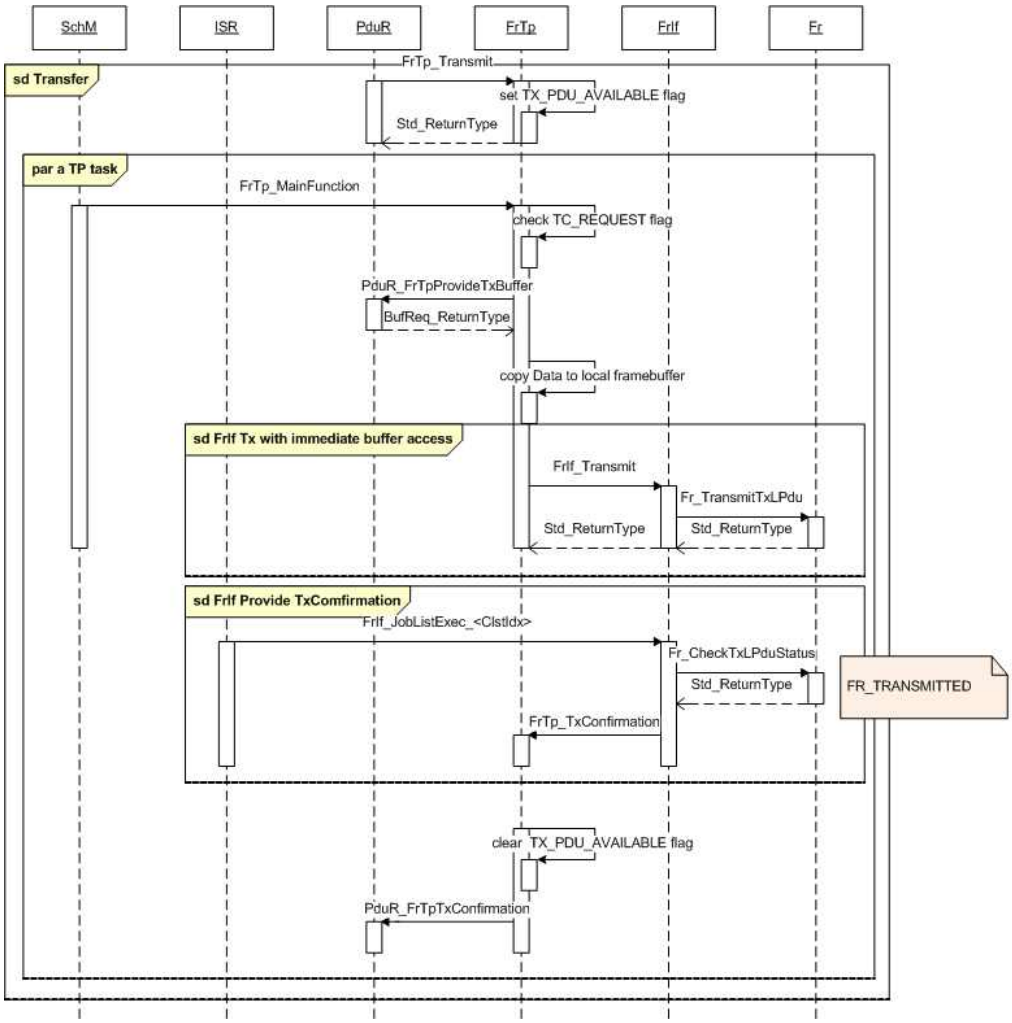


그림 3-21. FlexRay TP를 통한 데이터 송신

- FlexRay 네트워크의 데이터 송신은 상위 어플리케이션에서 보낸 메시지가 AUTOSAR 통신 모듈 중에서 PDU 라우터를 거쳐 전달되면서 이루어짐
- PDU 라우터에서 메시지는 FlexRay 인터페이스로 바로 전달될 수도 있고, TP를 거

친 후 FlexRay 인터페이스로 전달될 수도 있음

- TP로 전달된 데이터는 메시지 길이 및 환경 변수 초기 값에 따라 여러 개의 Segment로 나뉘어져 첫번째 프레임 (First Frame, FF)과 연속 프레임 (Conseccutive Frame, CF)로 전달됨
- 가능하다면 단일 프레임 (Single Frame, SF)으로 전달되기도 함
- TP에서는 필요에 따라 응답 프레임 (Acknowledgement Frame, AF)을 전송하거나 데이터 전송에 실패한 경우 재시도 (Retry) 기능을 제공하기도 함
- 흐름제어 기능을 제공하여 흐름제어 데이터 (Flow Control, FC)에 따라 알맞은 통신 기능을 수행함
- 메시지 수신은 OS의 인터럽트 서비스 루틴 (Interrupt Service Routine, ISR)에 의해 호출되는 Job List Exec 함수에서 수행됨
- Job List Exec 함수는 FlexRay 클러스터 별로 정의되어 있음
- Job List Exec 함수를 위한 인터럽트는 FlexRay CC의 Absolute Timer Interrupt에 의해 발생하며 본 과제에서는 OSEK 등의 OS가 없는 관계로 드라이버의 Absolute Timer Interrupt를 발생 가능하도록 설정함으로써 수행되도록 하였음
- 메시지 수신 동작은 설정에 따라 버퍼에 저장 후 트리거 방식에 의해 전달될 수도 있고, 바로 전달될 수도 있음
- [그림 3-22]에서는 드라이버로부터 전달받은 데이터가 TP를 거쳐 PDU 라우터로 전달되는 과정을 보여주고 있음
- TP에서의 모든 동작 수행은 FIXED_CYCLE time에 수행되는 태스크에서 이루어짐
- FlexRay 인터페이스로부터 메시지가 수신되었음이 알려지면 수신된 데이터를 저장할 수 있는 버퍼를 제공받아 PDU 라우터에게 메시지가 수신되었음을 알림

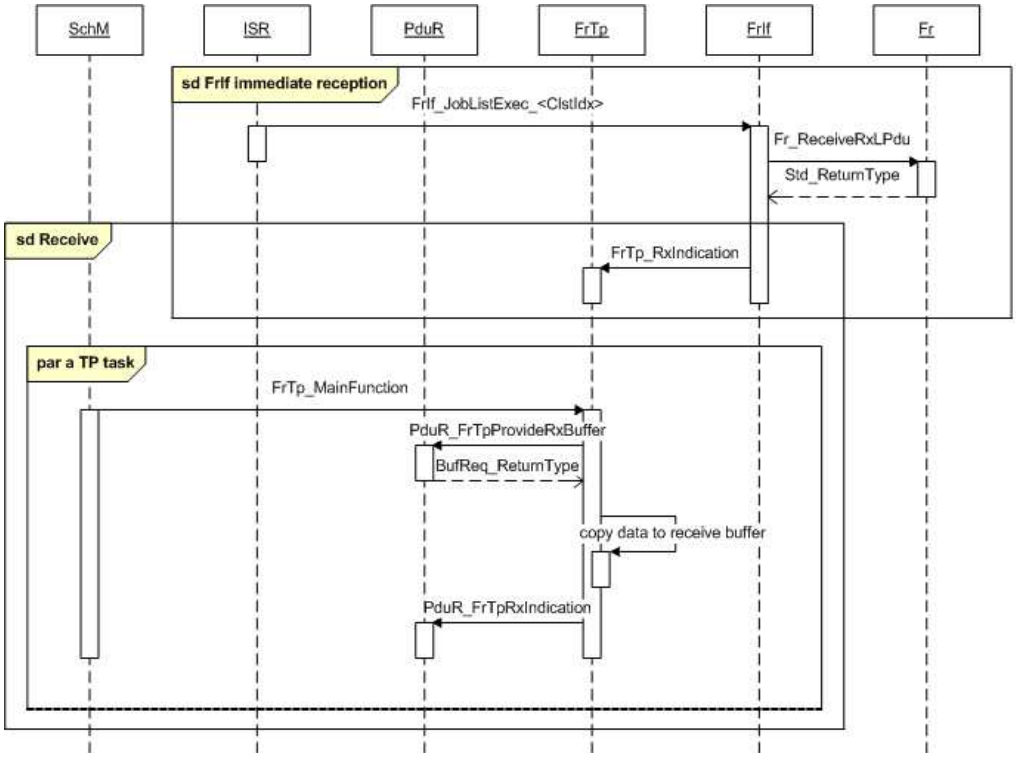


그림 3-22. FlexRay TP를 통한 데이터 수신

○ FlexRay 통신 서비스 계층의 설계

▸ FlexRay 상태 관리자 (SM)의 설계

- FlexRay 상태 관리자 (FlexRay State Manager)는 AUTOSAR의 계층적 소프트웨어 구조에서 ECU 추상화 계층, 엄밀히 말하면 통신 서비스 계층에 해당됨
- FlexRay 상태 관리자는 AUTOSAR 통신 관리자에게 FlexRay Cluster에서 통신을 시작하거나 종료시킬 수 있는 인터페이스를 제공함
- FlexRay 상태 관리자는 FlexRay CC 또는 FlexRay 트랜시버와 같은 하드웨어와 직접 연결되지 않고, FlexRay 인터페이스를 통하여 드라이버에 접근하게 됨
- 인터페이스를 통하여 FlexRay CC의 다양한 상태 즉, POC 상태를 확인하고 이러한 상태를 변경할 수 있는 CHI 커맨드의 명령 수행이 가능함

- FlexRay SM의 주요 함수

• void FrSm_Init (const FrSm_ConfigType *FrSm-ConfigPtr)

이 함수는 ECU 관리자에 의해 사용되며 FlexRay Cluster를 초기화 시키고, FlexRay 인터페이스의 FrIf_ControllerInit() 함수를 사용하여 POC 상태를 POC:config에서 POC:ready로 바꾸어 FlexRay 상태 관리자의 상태 역시 FRSM_READY 상태로 설정함

• Std_ReturnType FrSm_RequestComMode (
NetworkHandleType NetworkHandle,
ComM_ModeType ComM_Mode)

이 함수는 AUTOSAR 통신 관리자에 의해 FlexRay 통신 모드를 변경하고자 할 때 사용됨. 또한 요청된 모드를 저장만 하고, 다음에 수행되는 FrSm_MainFunction() 함수에 의해 모드의 변경 및 현재 모드 확인 등의 작업이 이루어짐. 매개변수로 주어지는 NetworkHandleType은 ComStack_Type 헤더파일에 정의되어 있는 타입으로 uint8형의 0-based 정수 타입임. ComM_ModeType은 통신 관리자의 헤더파일에 정의되어 있으며, NoCom, SilentCom, FullCom 등으로 구성되어 있음. 리턴 타입인 Std_ReturnType은 E_OK, E_NOT_OK으로 일반적인 AUTOSAR API 리턴 타입임

• void FrSm_MainFunction_<Cluster Id> ( )

이 함수는 기본 소프트웨어 스케줄러에서 FIXED_CYCLIC time에 호출된다. FlexRay Cluster와 연관되어 있는 모든 FlexRay CC의 POC 상태를 결정함

▸ FlexRay TP의 설계

- FlexRay TP는 주로 하나의 메시지 길이에 허용되는 사이즈를 넘어서는 경우 메시지를 분리하거나 조합하여 데이터를 재전송해 줌
- 또한 흐름 제어 및 에러 검출, 송수신 확인 및 재시도 등의 기능을 제공하며, 1:1 또는 1:n 통신이 가능함
- AUTOSAR FlexRay TP 규격은 대부분의 자동차 영역에서 사용되는 국제표준인 ISO 15765을 기반으로 하여 구성되었음

- 다음은 FlexRay TP의 주요 함수

• void FrTp_Init ( )

이 함수는 통신 관리자에 의해 사용되며, FlexRay TP 내의 모든 전역 변수들을 초기화함

• void FrTp_Transmit (
PduIdType FrTpTxPduId,
PduInfoType * PduInfoPtr)

이 함수는 PDU 라우터에서 데이터를 전송하고자 할 때 사용함. 현재 전송이 요청되었음을 나타내는 플래그(TX_PDU_AVAILABLE)를 설정하고, 함수의 리턴 값이 E_OK일 경우 다음에 수행되는 메인 함수에서 PDU 라우터의 PduR_FrTpProvideTxBuffer() 함수를 호출하여 데이터 버퍼를 제공받음. 매개변수로 전송되는 N-PDU의 ID와 데이터의 포인터와 길이로 이루어진 PduInfoType의 포인터가 주어져 전송하고자 하는 메시지의 정보가 전달됨.

• void FrTp_TriggerTransmit (
PduIdType FrTxPduId,
uint8 * FrSduPtr)

이 함수는 FlexRay 인터페이스에 의해 호출되는 콜백 함수로서 FlexRay 프레임이 전송되었음을 알려줌. 인터페이스 측에서는 인터럽트 서비스 루틴에 의해 트리거 함수를 호출함. PduId는 전송 요청된 FlexRay N-PDU의 ID이며, FrSduPtr는 L-PDU 전송 버퍼의 포인터임.

• void FrTp_RxIndication (
PduIdType FrRxPduId,
const uint8 * FrSduPtr)

이 함수는 FlexRay 인터페이스 측에서 Fr N-PDU의 수신 후에 호출하는 콜백 함수로서 FlexRay TP에서는 수신된 프레임을 복사하고, 수신된 데이터가 있음을 나타내는 플래그 (RX_PDU_AVAILABLE)를 설정함.

• void FrTp_TxConfirmation (PduIdType FrTxPduId)

이 함수는 데이터의 송신이 완료된 후 FlexRay 인터페이스 측에서 TxLPduStatus가 FR_TRANSMITTED인지 확인 후 호출하는 콜백 함수임

• void FrTp_MainFunction ( )
이 함수는 기본 소프트웨어 스케쥴러에서 FIXED_CYCLIC time에 호출되며, FlexRay 트랜스포트 모듈의 메인함수가 수행됨.

▸ FlexRay 네트워크 관리자 (NM)의 설계

- FlexRay NM은 하드웨어에 무관하게 동작하는 모듈로서 정상적인 통신(Normal Operation) 동작과 버스 휴지(Bus-sleep) 상태에 대한 조절을 담당함
- 부가적으로 각 노드가 통신 네트워크에 접속되어 있는지 여부를 탐지하거나 휴지상태로 전이 될 수 있게 준비되었는지를 탐지할 수 있음
- 상태천이는 FlexRay 노드간의 투표 방식에 의하여 진행됨

- 휴지 상태로 천이를 원하는 노드는 NM-Vote 메시지를 보내고 휴지 상태로 천이하며 NM-Vote 메시지를 받은 노드는 일정시간을 기다리다 NM-Vote 메시지가 도착하지 않으면 휴지 상태로 천이함
- 휴지 상태에서 정상상태로 천이를 원하는 노드는 NM-message를 보내고 FlexRay 상태관리 모듈에 의하여 정상상태로 상태천이를 함

- 다음은 FlexRay NM의 주요 함수

• void FrNm_Init (
FrNm_ConfigType *const nmConfigPtr)

이 함수는 FlexRay NM을 초기화함. 초기화 한 후에 NM-Vote 메시지와 NM-Data 송신을 중단시켜야 함

• void FrNm_PassiveStartup (
const NetworkHandleType NetworkHandle)

이 함수는 FlexRay NM이 수동적으로 FlexRay 노드의 통신을 촉발하기 위한 함수임. 이 함수에 의하여 휴지상태에서 네트워크관리를 시작하는 것을 의미함

• void FrNm_NetworkRequest (
const NetworkHandleType NetworkHandle)

이 함수는 FlexRay NM이 다른 노드로부터 휴지상태를 벗어나기 위한 요청이 있어서 휴지상태를 벗어나기 위한 출발을 시작하라는 의미임. 이 함수에 의하여 휴지상태에서 네트워크관리를 시작하는 것을 의미함.

④ FlexRay 통신 서비스 계층의 주요 API 설계

○ FlexRay TP의 주요 API 설계

- FrTp_Init

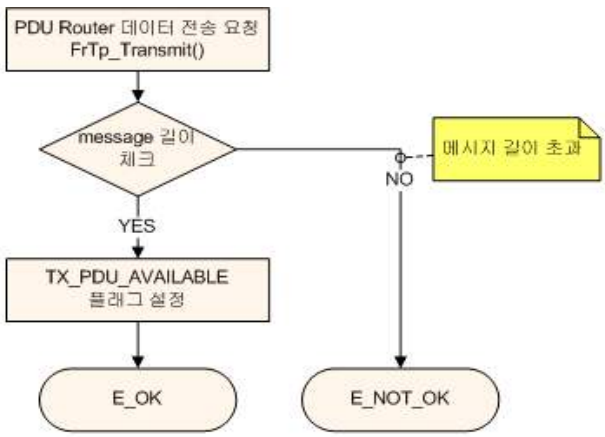
서비스 이름	FrTp_Init
구문 및 형태	void FrTp_Init( )
서비스 ID [hex]	0x00
파라미터(입력)	None
파라미터(입출력)	None
파라미터(출력)	None
리턴값	None
설명	FlexRay Transport 의 모든 전역변수를 초기화하고, idle 상태로 설정한다. 설정시간동안 발견되는 초기 데이터의 소프트웨어 오류 때문에 리턴값이 없다. 또한, 만약 하드웨어 오류가 발생하면 관리자 모듈을 통해 보고한다.
비고	FlexRay Transport 인스턴스 사용하기전에 의무적으로 호출되는 함수

- FrTp_Shutdown

서비스 이름	FrTp_Shutdown
구문 및 형태	void FrTp_Shutdown ( )
서비스 ID [hex]	0x01
파라미터(입력)	None
파라미터(입출력)	None
파라미터(출력)	None
리턴값	None
설명	이 서비스는 정지 연산에 의한 모든 미결정 Transport 프로토콜 연결들을 닫고, 모든 자원들을 해제하고, FrTp 모듈을 정지한다.

- FrTp_Transmit

서비스 이름	FrTp_Transmit

구문 및 형태	Std_ReturnType FrTp_Transmit (PduIdType FrTpTxPduId, PduInfoType* PduInfoPtr)	
서비스 ID [hex]	0x02	
파라미터(입력)	FrTpTxPduId	전송하게 되는 Fr N-SDU의 유일한 식별자
	PduInfoPtr	Fr N-SDU와 관련된 데이터 구조체 포인터: 데이터 길이와 Fr N-SDU 버퍼 포인터
파라미터(입출력)	None	
파라미터(출력)	None	
리턴값	Std_ReturnType	E_OK: 요청 수락 E_NOT_OK: 요청 수락되지 않음, 통신 채널에서 전송중일 때 또는 전송 메시지가 긴 경우 리턴.
설명	이 함수는 데이터 전송을 요청하기 위해 이용된다. 전송 요청을 나타내는 플래그(TX_PDU_AVAILABLE)를 설정한다. 이 함수는 FrTp의 PDU-Id로 호출되어야 한다. PduInfoPtr의 SduLength는 유효하다(데이터가 없음). 만약 이 함수가 E_OK를 리턴하면 보낼 데이터를 가져오기 위해 PduR_FrTpProvideTxBuffer를 호출 한다.	
비고	FrTp_Transmit() 수행절차: 1. PduInfoPtr의 SduLength를 통해 메시지 길이를 체크. 2. 메시지 길이가 초과될 경우 E_NOT_OK 리턴. 3. 길이가 초과되지 않으면 플래그(TX_PDU_AVAILABLE) 설정 4. E_OK 리턴. [흐름도]  	

- FrTp_CancelTransmitRequest

서비스 이름	FrTp_CancelTransmitRequest	
구문 및 형태	Std_ReturnType FrTp_CancelTransmitRequest (PduIdType FrTpTxPduId, FrTp_CancelReasonType FrTpCancelReason)	
서비스 ID [hex]	0x03	
파라미터(입력)	FrTpTxPduId	전송이 취소되는 Fr N-SDU의 유일한 식별자
	FrTpCancelReason	취소 이유
파라미터(입출력)	None	
파라미터(출력)	None	
리턴값	Std_ReturnType	E_OK: 전송(송신 또는 수신) 취소 요청 성공 E_NOT_OK: 전송 취소 요청 거절(즉, 1:n 연결에서 수신자가 취소 요청했을 때 또는 수신자 언세그먼트 전송일 때 또는 해당 채널에 허용되지 않을 때 리턴
설명	이 서비스는 미정의 Fr N-SDU의 전송을 취소할 때 이용된다. FrTpTxSduId 식별자로 연결한다. 이 함수는 FrTp의 PDU-Id와 같이 호출된다. (즉, 상위층에서 해당 메시지에 대해 TP에서 PDU-Id를 갖고 전송한다.)	
비고	만약 취소 요청이 허용되고 송신측에서 전송 취소할 경우, PduR_FrTpTxConfirmation함수는 취소가 끝난(성공 또는 실패) 후 호출되지 않는다. 만약 취소 요청이 허용되고 수신측에서 전송 취소할 경우, PduR_FrTpRxIndication함수는 취소가 끝난(성공 또는 실패) 후 호출되지 않을 것이다. 대신, 취소 요청이 허용되었다면 취소가 끝났을 때(성공 또는 실패) PduR_FrTpCancelTransmitConfirmation이 호출 될 것이다.	

- FrTp_ChangeParameterRequest

서비스 이름	FrTp_ChangeParameterRequest
구문 및 형태	void FrTp_ChangeParameterRequest (PduIdType FrTpTxPduId,

	FrTp_ParameterValueType FrTpParameterValue)	
서비스 ID [hex]	0x04	
파라미터(입력)	FrTpTxPduId	채널 변경에 대한 연결 ID
	FrTpParameterValue	FRTP_STMIN의 새로운 값을 포함한다.
파라미터(입출력)	None	
파라미터(출력)	None	
리턴값	None	
설명	FRTP_STMIN 매개변수값을 변경 요청하는데 사용 새로운 값은 FrTpParameterValue에 의해 주어집니다. 이 함수는 FrTp의 PDU-Id와 호출할 수 있다. 즉, 상위층은 해당하는 메시지를 위해 TP에서 자신의 PDU-Id로 바꾼다.	
비고	ISO 15765-2에 따르면 수신하는 동안 매개변수의 값을 변경하는 것은 불가능	

- FrTp_TriggerTransmit

서비스 이름	FrTp_TriggerTransmit	
구문 및 형태	void FrTp_TriggerTransmit (PduIdType FrTxPduId, uint8* FrSduPtr)	
서비스 ID [hex]	0x07	
파라미터(입력)	FrTxPduId	전송 요청된FlexRay L-PDU의 ID
	FrSduPtr	L-PDU 전송 버퍼 포인터
파라미터(입출력)	None	
파라미터(출력)	None	
리턴값	None	
설명	이 함수는 프레임을보내기 위해 FlexRay 인터페이스에서 호출되는 콜백함수이다. trigger transmit은 FlexRay 스케쥴에 의해 시작된다. 이 함수는 FrTp의 PDU-Id와 호출된다. 즉, FlexRay 인터페이스에서는 FrTp에 해당하는PDU-Id로 쓰인다.	
비고	이 함수는 인터럽트 문맥에서 호출할 수 있다.	

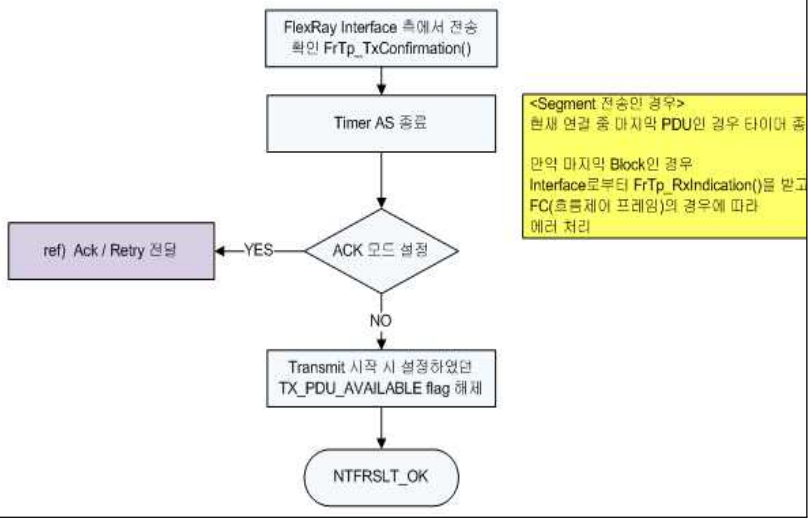
- FrTp_RxIndication

서비스 이름	FrTp_RxIndication

구문 및 형태	void FrTp_RxIndication (PduIdType FrRxPduId, const uint8* FrSduPtr)	
서비스 ID [hex]	0x05	
파라미터(입력)	FrRxPduId	수신된 Fr N-PDU의 식별자
	FrSduPtr	FrIf의 FlexRay SDU 버퍼 포인터
파라미터(입출력)	None	
파라미터(출력)	None	
리턴값	None	
설명	FlexRay 인터페이스에서 Fr N-PDU를 수신한 후에 호출되는 콜백함수이다. 이 함수 내에서 FlexRay Transport 층은 수신된 전송층 프레임을 복사한다. 수신된 데이터가 있음을 나타내는 플래그 (RX_PDU_AVAILABLE)를 설정한다. 이 함수는 FrTp의 PDU-Id와 호출된다. 즉, FlexRay 인터페이스에서는 FrTp에 해당하는 PDU-Id로 쓰인다.	
비고		

- FrTp_TxConfirmation

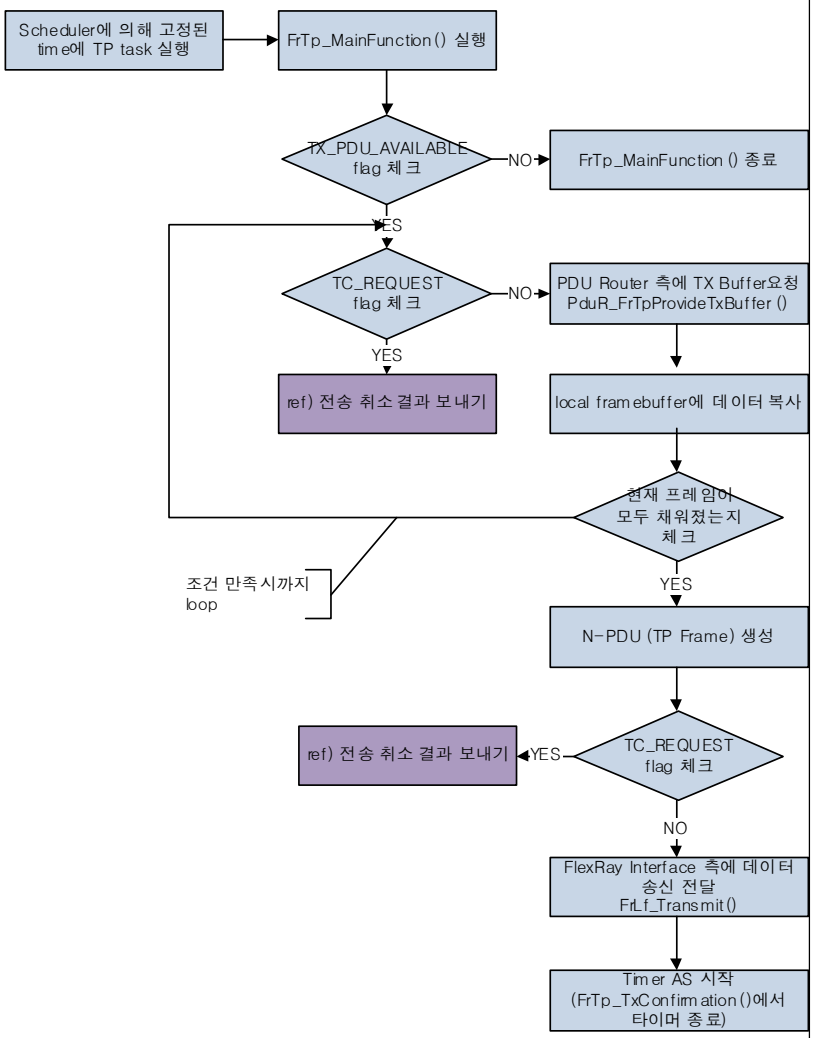
서비스 이름	FrTp_TxConfirmation	
구문 및 형태	void FrTp_TxConfirmation (PduIdType FrTxPduId)	
서비스 ID [hex]	0x06	
파라미터(입력)	FrTxPduId	전송된 Fr N-PDU의 식별자
파라미터(입출력)	None	
파라미터(출력)	None	
리턴값	None	
설명	이 함수는 TP에 관련있는 PDU가 전송되고 난 후에 FlexRay 인터페이스에서 호출된다. 이 함수 내에서, FlexRay TP는 구성된 타겟 전송 연결에 확인을 보낼 것이다. FlexRay TP에 속하고 있는 모든 전송된 프레임들은 이 함수를 사용해 확인할 것이다. 이 함수는 FrTp의 PDU-Id와 호출된다. 즉, FlexRay 인터페이스에서는 FrTp에 해당하는PDU-Id로 쓰인다.	
비고	FrTp_TxConfirmation() 수행절차: 1. ACK 모드 설정 체크.	

	2. 맞으면 ACK 전달하기 위한 동작 수행 3. 아닐 경우 플래그(TX_PDU_AVAILABLE) 해제. 4. NTFRSLT_OK 리턴. [흐름도] 

- FrTp_MainFunction

서비스 이름	FrTp_MainFunction
구문 및 형태	void FrTp_MainFunction ( )
서비스 ID [hex]	0x10
설명	FlexRay TP 스케쥴 (스케쥴 시작점)
비고	FrTp_MainFunction() 수행절차: [송신일 경우] 1. 플래그(TX_PDU_AVAILABLE)를 체크. 2. 플래그 설정 된 경우에는 플래그(TC_REQUEST) 체크 3. 플래그(TC_REQUEST) 설정된 경우 전송 취소 결과 보냄. 4. PDU 라우터에 전송 버퍼 요청(PduR_FrTpProvideTxBuffer()) 5. 루프 실행하면서 버퍼에 데이터 복사 6. 데이터에 맞는 프레임 생성 7. 플래그 설정 된 경우에는 플래그(TC_REQUEST) 체크 8. 플래그(TC_REQUEST) 설정된 경우 전송 취소 결과 보냄. 9. FlexRay 인터페이스에 데이터 송신 전달

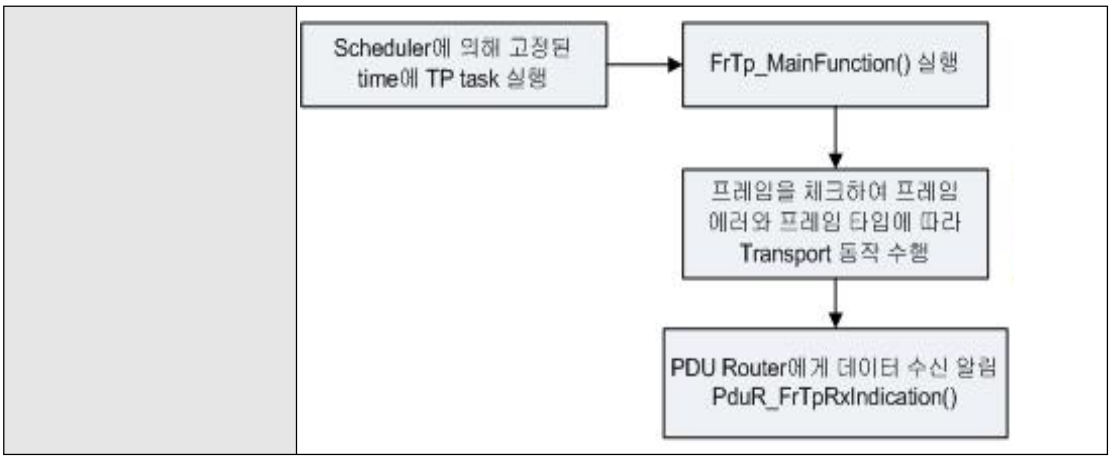
[흐름도]



[수신일 경우]

1. 플래그(RX_PDU_AVAILABLE)를 체크.
2. 프레임에러와 타입 체크
3. PDU 라우터에 데이터 수신 알림(PduR_FrTpRxIndication() 호출)
4. PDU 라우터에 수신버퍼 요청(PduR_FrTpProvideRxBuffer())
5. 버퍼에 수신 데이터 복사
6. 플래그(RX_PDU_AVAILABLE) 해제
7. PDU 라우터에 데이터 수신 알림(PduR_FrTpRxIndication() 호출)

[흐름도]


Scheduler에 의해 고정된
time에 TP task 실행
FrTp_MainFunction() 실행
프레임을 체크하여 프레임
에러와 프레임 타입에 따라
Transport 동작 수행
PDU Router에게 데이터 수신 알림
PduR_FrTpRxIndication()

○ FlexRay NM의 주요 API 설계

- FrNm_Init

서비스 이름	FrNm_Init	
구문 및 형태	void FrNm_Init(FrNm_ConfigType* const nmConfigPtr)	
서비스 ID [hex]	0x00	
파라미터(입력)	nmConfigPtr	선택된 설정셋의 포인터
파라미터(입출력)	None	
파라미터(출력)	None	
리턴값	None	
설명	FlexRay NM과 내부 상태 머신의 초기화	
비고	- FrNm_Init 함수는 FlexRay NM 모듈을 초기화 - FlexRay NM 모듈을 성공적으로 초기화하면, NM_INIT로 설정, 실패하면 NM_UNINIT로 설정 - FrNm_Init 함수는 설정 포인터 파라미터로 주어진 활성화된 설정셋을 선택 - FrNm_Init 함수는 FlexRay NM 모듈의 초기화 후에 NM-Vote와 NM-Data의 주기적인 전송을 비활성화함 (참조 : FRNM100, FRNM137) - FlexRay NM 모듈을 초기화한 후, FrNm_Init 함수는 NM-Data와 NM-Vote PDU의 예약된 바이트를 0으로 설정 - FlexRay NM 모듈을 초기화한 후, FrNm_Init 함수는 사용자 데이터 바이트를 FFh로 설정	

- FrNm_PassiveStartUp

서비스 이름	FrNm_PassiveStartUp	
구문 및 형태	Nm_ReturnType FrNm_PassiveStartUp (const NetworkHandleType NetworkHandle)	
서비스 ID [hex]	0x01	
파라미터(입력)	NetworkHandle	NM-채널의 식별자
파라미터(입출력)	None	
파라미터(출력)	None	
리턴값	Nm_ReturnType	NM_E_OK: 에러 없음

		NM_E_NOT_OK: 네트워크 관리를 시작할 수 없음 (실패) NM_E_NOT_EXECUTED: 네트워크 관리 시작이 허용되지 않음
설명	FlexRay NM을 Passive 시작	
비고	- FrNm_PassiveStartUp 함수는 FlexRay NM을 시작시킴 - FrNm_PassiveStartUp 함수는 반복된 메시지 상태에서 (동기화 상태를 거쳐) 버스-슬립 모드로부터 네트워크 모드로 전환 - FrNm_PassiveStartUp 함수는 버스-슬립 모드가 아닌 FlexRay NM 모듈의 운영 모드에서는 아무런 효과도 발휘하지 않음. 이 때, NM_E_NOT_EXECUTED가 리턴됨 (참고: FRNM138)	

- FrNm_NetworkRequest

서비스 이름	FrNm_NetworkRequest	
구문 및 형태	Nm_ReturnType FrNm_NetworkRequest(const NetworkHandleType NetworkHandle)	
서비스 ID [hex]	0x02	
파라미터(입력)	NetworkHandle	NM채널의 식별자
파라미터(입출력)	None	
파라미터(출력)	None	
리턴값	Nm_ReturnType	NM_E_OK: 에러 없음 NM_E_NOT_OK: 버스 통신 요청이 실패함
설명	ECU가 버스에서 통신할 필요가 있기 때문에, 이 함수로 네트워크를 요청함. 네트워크 상태는 "요청됨' 상태로 변경됨.	
비고	- 만약 FlexRay NM 모듈의 환경이 다음 모드에서 FrNm_NetworkRequest 함수를 호출하고 있으면, 1. 버스-슬립 모드에서 호출 : 버스-슬립 모드에서 벗어나, 네트워크 요청 플래그 FrNm_NetworkRequested를 TRUE로 설정하고, 동기 모드로 들어감 2. 동기 / 네트워크 모드에서 호출 : 네트워크 요청 플래그 FrNm_NetworkRequested를 TRUE로 설정 - FrNm_NetworkRequest 함수는 오직FRNM_PASSIVE_NODE_ENABLED 설정 파라미터가 OFF로 설정되어 있을 때만 사용이 가능함	

- FrNm_NetworkRelease

서비스 이름	FrNm_NetworkRelease	
구문 및 형태	Nm_ReturnType FrNm_NetworkRelease(const NetworkHandleType NetworkHandle)	
서비스 ID [hex]	0x03	
파라미터(입력)	NetworkHandle	NM-채널의 식별자
파라미터(입출력)	None	
파라미터(출력)	None	
리턴값	Nm_ReturnType	NM_E_OK: 에러 없음 NM_E_NOT_OK: 버스 통신 해제를 실패함
설명	ECU가 버스에서 통신을 하지 않을 때, 네트워크를 해제시킴. 네트워크 상태는 '해제' 상태로 변경됨.	
비고	- 만약 FlexRay NM 모듈의 환경이 동기 모드나 네트워크 모드에서 FrNm_NetworkRelease 함수를 호출하고 있으면, 네트워크 요청 플래그 FrNm_NetworkRequested를 FALSE로 설정 - FrNm_NetworkRelease 함수는 오직 FRNM_PASSIVE_NODE_ENABLED 설정 파라미터가 OFF로 설정되어 있을 때만 사용이 가능함	

- FrNm_SetUserData

서비스 이름	FrNm_SetUserData	
구문 및 형태	Nm_ReturnType FrNm_SetUserData(const NetworkHandleType NetworkHandle, const uint8* const nmUserDataPtr)	
서비스 ID [hex]	0x06	
파라미터(입력)	NetworkHandle	NM-채널의 식별자
	nmUserDataPtr	다음 전송될 NM 메시지의 사용자 데이터
파라미터(입출력)	None	
파라미터(출력)	None	
리턴값	Nm_ReturnType	NM_E_OK: 에러 없음 NM_E_NOT_OK: 사용자 데이터 설정 실패
설명	버스에서 다음 전송될 NM-데이터를 위해 사용자 데이터를 설정	

비고	- 만약 취급할 사용자 데이터가 FlexRay NM 모듈에 대해 활성화되어 있으면, FrNm_SetUserData 함수가 사용자 데이터를 설정함 - FrNm_SetUserData 함수는 FrNmUserDataEnabled 파라미터가 ON 되어 있을 때만 사용이 가능함 (참고: FRNM077)

- FrNm_GetUserData

서비스 이름	FrNm_GetUserData	
구문 및 형태	Nm_ReturnType FrNm_GetUserData(const NetworkHandleType NetworkHandle, uint8* const nmUserDataPtr)	
서비스 ID [hex]	0x07	
파라미터(입력)	NetworkHandle	NM-채널의 식별자
파라미터(입출력)	None	
파라미터(출력)	nmUserDataPtr	성공적으로 수신한 NM 메시지로부터 사용자 데이터 위치를 복사할 포인터
리턴값	Nm_ReturnType	NM_E_OK: 에러 없음 NM_E_NOT_OK: 사용자 데이터 가져오기 실패
설명	성공적으로 수신한 NM 메시지로부터 사용자 데이터를 얻음	
비고	- 만약 취급할 사용자 데이터가 FlexRay NM 모듈에 대해 활성화되어 있으면FrNM_GetUserData 함수는 지난번에 받은 NM-Data PDU로부터 사용자 데이터를 제공함. - FrNm_GetUserData 함수는 FrNmUserDataEnabled 설정 파라미터가 ON 되어 있을 때만 사용이 가능함 (참고: FRNM077)	

- FrNm_GetPduData

서비스 이름	FrNm_GetPduData	
구문 및 형태	Nm_ReturnType FrNm_GetPduData(const NetworkHandleType NetworkHandle, uint8* const nmPduData)	
서비스 ID [hex]	0x08	
파라미터(입력)	NetworkHandle	NM-채널의 식별자

파라미터(입출력)	None	
파라미터(출력)	NmPduData	NM PDU가 복사될 포인터
리턴값	Nm_ReturnType	NM_E_OK: 에러 없음 NM_E_NOT_OK: NM PDU 데이터 가져오기 실패
설명	PDU Data를 가져옴	
비고	- FrNm_GetPduData 함수는 가장 최근에 받은 NM 메시지의 전체 PDU 데이터를 가져옴 - FrNm_GetPduData 함수는 FrNmUserDataEnabled 설정 파라미터가 ON 되어 있을 때만 사용이 가능함	

- FrNm_RepeatMessageRequest

서비스 이름	FrNm_RepeatMessageRequest	
구문 및 형태	Nm_ReturnType FrNm_RepeatMessageRequest(const NetworkHandleType NetworkHandle)	
서비스 ID [hex]	0x09	
파라미터(입력)	NetworkHandle	NM-채널의 식별자
파라미터(입출력)	None	
파라미터(출력)	None	
리턴값	Nm_ReturnType	NM_E_OK: 에러 없음 NM_E_NOT_OK: 반복 메시지 요청 실패
설명	버스에서 전송될 다음 반복 메시지 요청	
비고	- 만약 노드 탐지가 활성화되어 있으면, FrNm_RepeatMessageRequest 함수는FlexRay 버스 NM 노드들 상에서 노드 탐지를 요청함. - 만약 FlexRay NM 모듈 환경이 네트워크 모드에서 FrNm_RepeatMessageRequest 함수를 호출하고 있고, 노드 탐지 서비스가 활성화된 상태(FrNmNodeDetectionEnabled)이면 FrNm_RepeatMessage 플래그를 TRUE로 설정함 - FrNm_RepeatMessageRequest 함수는 FrNmNodeDetectionEnabled 설정 파라미터가 ON 되어 있을 때만 사용이 가능함	

- FrNm_GetNodeIdentifier

서비스 이름	FrNm_GetNodeIdentifier	
구문 및 형태	Nm_ReturnType FrNm_GetNodeIdentifier(const NetworkHandleType NetworkHandle, uint8* const nmNodeIdPtr)	
서비스 ID [hex]	0x0a	
파라미터(입력)	NetworkHandle	NM-채널의 식별자
파라미터(입출력)	None	
파라미터(출력)	nmNodeIdPtr	성공적으로 수신한 NM 메시지로부터 노드 식별자 위치를 복사할 포인터
리턴값	Nm_ReturnType	NM_E_OK: 에러 없음 NM_E_NOT_OK: 노드 식별자 가져오기 실패
설명	성공적으로 수신한 NM 메시지로부터 노드 식별자를 가져옴	
비고	- 만약 노드 탐지가 활성화되어 있으면 FrNm_GetNodeIdentifier 함수는 가장 최근에 받은 NM 메시지의 노드 식별자를 제공함 - FrNm_GetNodeIdentifier 함수는 FrNmNodeDetectionEnabled 설정 파라미터가 ON 되어 있을 때만 사용이 가능함 (참고: FRNM170)	

- FrNm_GetLocalNodeIdentifier

서비스 이름	FrNm_GetLocalNodeIdentifier	
구문 및 형태	Nm_ReturnType FrNm_GetLocalNodeIdentifier(const NetworkHandleType NetworkHandle, uint8* const nmNodeIdPtr)	
서비스 ID [hex]	0x0b	
파라미터(입력)	NetworkHandle	NM-채널의 식별자
파라미터(입출력)	None	
파라미터(출력)	nmNodeIdPtr	지역노드 식별자의 위치를 복사할 포인터
리턴값	Nm_ReturnType	NM_E_OK: 에러 없음 NM_E_NOT_OK: 지역노드 식별자 가져오기 실패
설명	지역 노드를 위해 구성된 노드 식별자를 가져옴	
비고	- 만약 노드 탐지가 활성화되어 있으면 FrNm_GetLocalNodeIdentifier 함수는 지역 호스트 노드(FrNmNodeId)를 위해 구성된 노드 식별자를 제공함 - FrNm_GetLocalNodeIdentifier 함수는	

	FrNmNodeDetectionEnabled 설정 파라미터가 ON 되어 있을 때만 사용이 가능함 (참고: FRNM170)

- FrNm_RequestBusSynchronization

서비스 이름	FrNm_RequestBusSynchronization	
구문 및 형태	Nm_ReturnType FrNm_RequestBusSynchronization(const NetworkHandleType NetworkHandle)	
서비스 ID [hex]	0xc0	
파라미터(입력)	NetworkHandle	NM-클러스터의 식별자
파라미터(입출력)	None	
파라미터(출력)	None	
리턴값	NM_ReturnType	NM_E_OK: 에러 없음 NM_E_NOT_OK: 실패
설명	이 함수는 동작하지 않음 (이 서비스는 단지 미래의 확장성과 CAN-NM 인터페이스와의 호환성을 위해서 제공)	
비고	- FlexRay NM 모듈은 어떤 기능도 실행하지 않고 단지 NM_E_OK를 리턴함으로써FrNm_RequestBusSynchronization 함수를 지원함. (설명 : FlexRay 버스는 동기 버스로서, FlexRay NM은 버스 동기화에 영향을 미치지 않는다. 이 기능은 FlexRay 컨트롤러와FlexRay 드라이버에 의해 처리된다. 이 함수가 미래에 확장(예를 들어, 게이트웨이) 사용될 수 있기 때문에, 최소로 사용이 가능하다.) - FrNm_RequestBusSynchronization 함수는 FrNmBusSynchronizationEnabled 설정 파라미터가 ON 되어 있을 때만 사용이 가능함	

- FrNm_CheckRemoteSleepIndication

서비스 이름	FrNm_CheckRemoteSleepIndication
구문 및 형태	Nm_ReturnTypeFrNm_CheckRemoteSleepIndication(const NetworkHandleType NetworkHandle, boolean* const nmRemoteSleepIndPtr)
서비스 ID [hex]	0x0d

파라미터(입력)	NetworkHandle	NM-채널의 식별자
파라미터(입출력)	None	
파라미터(출력)	nmRemoteSleepIndPtr	원격슬립 표시 검사결과 위치를 복사할 포인터
리턴값	NM_ReturnType	NM_E_OK: 에러 없음 NM_E_NOT_OK: 원격슬립 표시 비트 검사 실패 NM_E_NOT_EXECUTED: 원격슬립 표시 검사를 실행할 수 없음
설명	원격슬립 표시가 발생했는지 검사함	
비고	- FlexRay NM 모듈과 Nm 모듈은 FlexRay NM 모듈 환경이 FrNm_CheckRemoteSleepIndication 함수를 호출하기 전에 바르게 초기화됨 - FlexRay NM 함수인 FrNm_CheckRemoteSleepIndication은 원격 슬립 표시의 현재 상태 정보를 제공 (이미 감지되었거나 감지되지 않았음) - FrNm_CheckRemoteSleepIndication 함수는 FrNmRemoteSleepIndicationEnabled 설정 파라미터가 ON 되어 있을 때만 사용이 가능함	

- FrNm_GetState

서비스 이름	FrNm_GetState	
구문 및 형태	Nm_ReturnType FrNm_GetState(const NetworkHandleType NetworkHandle, Nm_StateType* const nmStatePtr, Nm_ModeType* const nmModePtr)	
서비스 ID [hex]	0x0e	
파라미터(입력)	NetworkHandle	NM-채널의 식별자
파라미터(입출력)	None	
파라미터(출력)	nmStatePtr	네트워크 관리 상태의 위치를 복사할 포인터
	nmModePtr	네트워크 모드의 위치를 복사할 포인터
리턴값	Nm_ReturnType	NM_E_OK: 에러 없음 NM_E_NOT_OK: NM 상태 가져오기 실패
설명	네트워크 관리 상태와 모드를 리턴	
비고	- FrNm_GetState 함수는 NM 상태 머신의 현재 상태와 모드에 관한 일관된 정보를 제공	

	(제공되는 값과 상태 및 모드의 현재 값 사이의 일관성이 보장됨)

- FrNm_GetVersionInfo

서비스 이름	FrNm_GetVersionInfo	
구문 및 형태	void FrNm_GetVersionInfo(Std_VersionInfoType* NmVerInfoPtr)	
서비스 ID [hex]	0x0f	
파라미터(입력)	None	
파라미터(입출력)	None	
파라미터(출력)	NmVerInfoPtr	버전 정보의 위치를 복사할 포인터
리턴값	None	
설명	버전 정보를 리턴	
비고	- FrNm_GetVersionInfo 함수는 이 모듈의 버전 정보를 리턴함. 버전 정보는 모듈 ID, 벤더 ID, 벤더 고유 버전 번호 (BSW00407)를 포함하고 있다. - 만약 FrNm_GetVersionInfo의 호출자와 피호출자의 소스 코드가 사용 가능하면, FlexRay NM 모듈은 모듈 헤더 파일의 정의된 매크로로서 실행된다. - FrNm_GetVersionInfo 함수는 FrNmVersionInfoApi 설정 파라미터가 ON 되어 있을 때만 사용이 가능함	

- FrNm_RxIndication

서비스 이름	FrNm_RxIndication	
구문 및 형태	void FrNm_RxIndication(PduIdType FrNmRxPduId, const uint8* FrNmRxSduPtr)	
서비스 ID [hex]	0xe1	
파라미터(입력)	FrNmRxPduId	FlexRay NM PDU ID
	FrNmRxSduId	FlexRay NM L-SDU 데이터 포인터
파라미터(입출력)	None	
파라미터(출력)	None	
리턴값	None	

설명	FlexRay NM PDU가 수신된 후, FlexRay 인터페이스에 의해서 호출됨
비고	- FrNm_RxIndication 함수는 수신한 FlexRay NM PDU를 복사하고FlexRay NM PDU ID에 대해 지역적으로 저장된 NM-메시지로부터 데이터를 처리함 - FrIf 모듈과 FrNm 모듈은 FlexRay NM 모듈 환경이 FrNM_RxIndication 함수를 호출하기 전에 바르게 초기화됨 - FrNm_RxIndication 함수는 인터럽트 컨텍스트에서 FlexRay NM 모듈 환경에 의해 호출된다.

- FrNm_TriggerTransmit

서비스 이름	FrNm_TriggerTransmit	
구문 및 형태	void FrNm_TriggerTransmit(PduIdType FrNmTxPduId, Uint8* FrNmRxSduPtr)	
서비스 ID [hex]	0xe4	
파라미터(입력)	FrNmTxPduId	콜백을 발생시킨 FlexRay NM PDU의 ID
	FrNmRxSduPtr	FlexRay NM L-SDU 데이터의 포인터
파라미터(입출력)	None	
파라미터(출력)	None	
리턴값	None	
설명	FlexRay NM PDU가 전송되었을 때, FlexRay 인터페이스에 의해 호출됨	
비고	- FrNm_TriggerTransmit 함수는 FlexRay NM PDU ID에 관해서 FlexRay NM PDU를 복사함 - FrIf 모듈과 FrNm 모듈은 FlexRay NM 모듈 환경이 FrNm_TriggerTransmit 함수를 호출하기 전에 바르게 초기화됨 - FrNm_TriggerTransmit 함수는 인터럽트 컨텍스트에서 FlexRay NM 모듈 환경에 의해 호출된다.	

- FrNm_TxConfirmation

서비스 이름	FrNm_TxConfirmation
구문 및 형태	void <UL_TxConfirmation> (

	PduIdType FrIf_TxPduId)
서비스 ID [hex]	0x00
파라미터(입력)	FrIf_TxPduId 전송이 확인되고 있는 FlexRay PDU의 ID
파라미터(입출력)	None
파라미터(출력)	None
리턴값	None
설명	상위 레이어 BSW 모듈 <UL>의 API 서비스(예를 들어, PduR, FrTp, FrNm)는 FlexRay 통신 시스템을 거쳐 FrIf_TxPduId 인덱스와 함께 PDU가 전송된 상위 레이어 BSW 모듈로 확인된 FlexRay 인터페이스에 의해 호출된다.
비고	- 이 API 서비스는 FrNm_MainFunction_<NmClstIdx>가 실행되는 동안 호출된다.

- FrNm_MainFunction_<NmClstIdx>

서비스 이름	FrNm_MainFunction_<NmClstIdx>
구문 및 형태	Void FrNm_MainFunction_<NmClstIdx> ( )
서비스 ID [hex]	0xf0
타이밍	FIXED_CYCLIC
설명	FlexRay NM의 메인 함수
비고	- 주기적으로 수행되는 FlexRay 네트워크 관리 API 서비스는 FrNm 상태 머신을 유지하는 것을 목적으로 제공된다. (참고: 7.2, 7.4) - 이 FlexRay 네트워크 관리 API 서비스는 BSW 스케줄러에 의해 제공되는 작업체로부터 주기적으로 호출된다. - FlexRay 사이클의 지속시간은 한 개의 ECU의 두 클러스터 사이에 서로 다를 수 있기 때문에, API 서비스의 호출 시기는 시스템 설정 시간에 각 클러스터에게 독립적으로 구성될 수 있다. - 각 NM 클러스터에 하나의 특정한 목적을 가진 FlexRay NM 메인 함수가 있다. API 이름은 아래와 같다. 1. FlexRay NM 클러스터를 위한 FrNm_MainFunction_0()은 FlexRay NM 클러스터 0과 관련있다. 2. FlexRay NM 클러스터를 위한 FrNm_MainFunction_1()은 FlexRay NM 클러스터 1과 관련있다.

	3. 기타 등등 - 독립된 FrNm 클러스터들의 최대 개수는 FrNmNumberOfClusters FrNm 전역 정의에 의해 정의되어 있다. (참고 : FrNmGlobalConstants 챕터)

- NM_NetworkStartIndication

서비스 이름	Nm_NetworkStartIndication	
구문 및 형태	void Nm_NetworkStartIndication(const Nm_NetworkHandleType nmNetworkHandle)	
서비스 ID [hex]	0x00	
파라미터(입력)	nmNetworkHandle	NM-클러스터의 식별자
파라미터(입출력)	None	
파라미터(출력)	None	
리턴값	None	
설명	상위 레이어 Nm의 이 API 서비스는 NM-메시지가 성공적으로 수신되었고 FrNm이 버스-슬립 모드에 있는, 상위 레이어 BSW 모듈을 나타내기 위해 FlexRay 네트워크 관리에 의해 호출된다.	
비고	- 이 API 서비스는 FlexRay 네트워크 관리 모듈이 실행되고 있는 동안 호출된다. (예를 들어, FrNm_MainFunction_<NmClstIdx>에 의해 호출)	

- NM_PduRxIndication

서비스 이름	<Nm_NetworkStartIndication >	
구문 및 형태	void <Nm_NetworkStartIndication > (const Nm_NetworkHandleType nmNetworkHandle)	
서비스 ID [hex]	0x00	
파라미터(입력)	nmNetworkHandle	NM-클러스터의 식별자
파라미터(입출력)	None	
파라미터(출력)	None	
리턴값	None	
설명	상위 레이어 Nm의 이 API 서비스는 NM-메시지가 성공적으로 수신되었고 FrNm이 버스-슬립 모드에 있는, 상위 레이어 BSW	

	모듈을 나타내기 위해 FlexRay 네트워크 관리에 의해 호출된다.
비고	- 이 API 서비스는 FlexRay 네트워크 관리 모듈이 실행되고 있는 동안 호출된다. (예를 들어, FrNm_MainFunction_<NmClstIdx>에 의해 호출)

- Nm_StateChangeNotification

서비스 이름	< Nm_StateChangeNotification >	
구문 및 형태	void < Nm_StateChangeNotification > (const Nm_NetworkHandleType nmNetworkHandle, const Nm_StateType nmState)	
서비스 ID [hex]	0x00	
파라미터(입력)	nmNetworkHandle	NM-클러스터의 식별자
	nmState	이 NM-클러스터를 위한 FrNm 상태 머신의 (새로운) 상태
파라미터(입출력)	None	
파라미터(출력)	None	
리턴값	None	
설명	상위 레이어 BSW 모듈 Nm의 이 API 서비스는 FlexRay NM 상태 머신에서 상태 변화가 발생한 BSW 모듈을 나타내기 위해 FlexRay 네트워크 관리에 의해 호출된다.	
비고	- 이 API 서비스는 FlexRay 네트워크 관리 모듈이 실행되고 있는 동안 호출된다. (예를 들어, FrNm_MainFunction_<NmClstIdx>에 의해 호출)	

- 옵션 인터페이스

서비스 이름	Det_ReportError
설명	개발 오류를 보고하는 서비스

- <UL>_RemoteSleepCancellation

서비스 이름	< UL_RemoteSleepCancellation >
구문 및 형태	void < UL_RemoteSleepCancellation > (const Nm_NetworkHandleType nmNetworkHandle)
서비스 ID [hex]	0x00

파라미터(입력)	nmNetworkHandle	NM-클러스터의 식별
파라미터(입출력)	None	
파라미터(출력)	None	
리턴값	None	
설명	상위 레이어 BSW 모듈 <UL>(예를 들어, ComM)의 이 API 서비스는 버스가 깨어있음을 나타내는 NM-메시지를 수신함으로서 "원격 슬립"이 취소된 BSW 모듈을 나타내기 위해 FlexRay 네트워크 관리에 의해 호출된다. 그리고, 원격 슬립 표시는 이전에 탐지된다.	
비고	- 이 API 서비스는 FlexRay 네트워크 관리 모듈이 실행되고 있는 동안 호출된다. (예를 들어, FrNm_MainFunction_<NmClstIdx>에 의해 호출)	

- <UL>_RemoteSleepIndication

서비스 이름	< UL_RemoteSleepIndication >	
구문 및 형태	void < UL_RemoteSleepIndication > (const Nm_NetworkHandleType nmNetworkHandle)	
서비스 ID [hex]	0x00	
파라미터(입력)	nmNetworkHandle	NM-클러스터의 식별자
파라미터(입출력)	None	
파라미터(출력)	None	
리턴값	None	
설명	상위 레이어 BSW 모듈 <UL>(예를 들어, ComM)의 이 API 서비스는 클러스터의 모든 다른 노드들이 잠들 준비가 된 BSW 모듈을 나타내기 위해 FlexRay 네트워크 관리에 의해 호출된다. 따라서, 버스가 깨어있음을 나타내는 표시가 없는 NM-메시지들이 일반적인 운영 상태에서 FRNM_REMOTE_SLEEP_IND_TIME 설정 파라미터에 의해 결정된 시간 설정 양을 받는다.	
비고	- 이 API 서비스는 FlexRay 네트워크 관리 모듈이 실행되고 있는 동안 호출된다. (예를 들어, FrNm_MainFunction_<NmClstIdx>에 의해 호출)	

- <UL>_TransmissionTitmeoutException

서비스 이름	< UL_TransmissionTimeoutException >	
구문 및 형태	void < UL_TransmissionTimeoutException > (const Nm_NetworkHandleType nmNetworkHandle)	
서비스 ID [hex]	0x00	
파라미터(입력)	nmNetworkHandle	NM-클러스터의 식별
파라미터(입출력)	None	
파라미터(출력)	None	
리턴값	None	
설명	상위 레이어 BSW 모듈 <UL>의 이 API 서비스는 NM-메시지가FRNM_MSG_TIMEOUT_TIME 시간 간격 내에 성공적으로 전송될 수 없었던 BSW 모듈을 나타내기 위해 FlexRay 네트워크 관리에 의해 호출된다. 그리고, 주기적인 NM-메시지 전송이 계속 된다.	
비고	- 이 API 서비스는 FlexRay 네트워크 관리 모듈이 실행되고 있는 동안 호출된다. (예를 들어, FrNm_MainFunction_<NmClstIdx>에 의해 호출) - 만약 FrNmMsgTimeoutTime > 0 이면, 콜백이 사용 가능하다.	

○ FlexRay SM의 주요 API 설계

- FrSm_Init

서비스 이름	FrSm_Init	
구문 및 형태	void FrSm_Init(const FrSm_ConfigType* FrSm_ConfigPtr)	
서비스 ID [hex]	0x01	
파라미터(입력)	FrSm_ConfigPtr	FlexRay State Manager 설정 구조의 Base Pointer
파라미터(입출력)	None	
파라미터(출력)	None	
리턴값	None	
설명	FlexRay State Manager 초기화	
비고	FrSm_Init 함수 용도 1. FlexRay clusters의 상태를 FRSM_READY의 값으로 초기화시킨다. 2. 내부 저장장치 안의 Data Address 구성들이 다음의 API가 Data 구조에 대한 호출을 가능하도록 시킨다. 3. 만일 개발 실수 탐지 기능이 성공적으로 활성화 된다면 (FrSmDevErrorDetect is ON), 적당한 모듈 초기화를 위해서 다른 API 함수들이 내부적으로 기억이 된다.	

- FrSm_RequestComMode

서비스 이름	FrSm_RequestComMode	
구문 및 형태	Std_ReturnType FrSm_RequestComMode(NetworkHandleType NetworkHandle, ComM_ModeType ComM_Mode)	
서비스 ID [hex]	0x02	
파라미터(입력)	NetworkHandle	Communication Mode를 요청할 수 있기 위한 FlexRay cluster
	ComM_Mode	Communication Mode를 요청하는 값
파라미터(입출력)	None	
파라미터(출력)	None	
리턴값	Std_ReturnType	E_OK : 요청이 가능할 때 E_OK를 리턴 E_NOT_OK : 요청이 불가능할 때 E_NOT_OK를 리턴

설명	Communication Mode를 통하여 FlexRay cluster를 시작하고 중지시킬 수 있도록 명령
비고	※ FrSm_RequestComMode 함수는Communication Mode 값의 요청을 저장한다. 요청된 값이 요청이 되기 위해서는, FrSm_MainFuncion 함수가 cluster에 상등하는 상태 기계가 수행할 때 요청이 된다. ※ 만일 NoCom이 FullCom 값으로 된다면, FrSm_RequestComMode 함수가 상태 기계 변화가 즉시 실행이 된다. • Network Handle의 값에서 문제가 생겼을 때, FRSM_E_INV_HANDLE 에러 값으로 인하여E_NOT_OK 값이 return • ComM_Mode의 값에서 문제가 생겼을 때, FRSM_E_INV_MODE 에러 값으로 인하여E_NOT_OK 값이 return • FlexRay가 아직 FrSm_Init 호출을 하지 않았을 때, FRSM_E_NOT_INITIALIZED 에러 값으로 인하여E_NOT_OK 값이 return

- FrSm_GetCurrentComMode

서비스 이름	FrSm_GetCurrentComMode	
구문 및 형태	Std_ReturnType FrSm_GetCurrentComMode(NetworkHandleType NetworkHandle, ComM_ModeType* ComM_ModePtr)	
서비스 ID [hex]	0x03	
파라미터(입력)	NetworkHandle	Network 상태 핸들
파라미터(입출력)	None	
파라미터(출력)	ComM_ModePtr	현재 Communication Mode의 값에 대한 메모리 위치 주소 Pointer
리턴값	Std_ReturnType	E_OK : 정상적으로 E_OK를 리턴 E_NOT_OK : FrSm이 FrSm_Init 함수를 사용하여 생성하지 않았을 때, E_NOT_OK를 리턴
설명	FlexRay cluster의 현재 Communication Mode의 값을 결정시킬 수 있는 함수	
비고	※ FrSm_GetCurrentComMode 함수는 메모리 위치 안의	

	FlexRay cluster에 상등하는 Communication Mode의 값을 쓴다. ※ 다음과 같을 때 Communication Mode의 값을 결정 - cluster에 대한 상태 기계가 FRSM_ONLINE이고 FrIf가 FRIF_ONLINE일 때, Communiction Mode는 COMM_FULL_COMMUNICATION의 값을 가진다. › 아닐 때, Communication Mode는 COMM_NO_COMMUNICATION의 값을 가진다. • Network Handle의 값에서 문제가 생겼을 때, FRSM_E_INV_HANDLE 에러 값으로 인하여E_NOT_OK 값이 return • ComM_Mode의 값에서 문제가 생겼을 때, FRSM_E_INV_MODE 에러 값으로 인하여E_NOT_OK 값이 return • FlexRay가 아직 FrSm_Init 호출을 하지 않았을 때, FRSM_E_NOT_INITIALIZED 에러 값으로 인하여E_NOT_OK 값이 return

- FrSm_GetVersionInfo

서비스 이름	FrSm_GetVersionInfo	
구문 및 형태	void FrSm_GetVersionInfo(Std_VersionInfoType* versioninfo)	
서비스 ID [hex]	0x04	
파라미터(입력)	None	
파라미터(입출력)	None	
파라미터(출력)	versioninfo	이 모듈의 버전 정보에 대한 Pointer
리턴값	None	
설명	이 모듈의 버전 정보에 대한 값을 받아온다.	
비고	※ 이 모듈의 버전 정보에 대한 값을 받아온다. 버전 정보 값은 Module ID, Vendor ID, Vendor Specific Version Numbers (BSW00407) 등이다. ※ 이 함수는 컴파일 전에 시간 구성을 ON/OFF 시킨다. : FrSmVersionInfoApi	

⑤ 구현 및 결과

▶ 하드웨어 개발 환경

○ AUTOSAR 기반의 FlexRay 프로토콜의 구현은 Freescale Semiconductor사의 EVB9S12XF512E 보드를 기반으로 함

○ EVB9S12XF512E 보드의 메인 칩셋인 MC9S12XF512는 FlexRay 인터페이스가 내장되어 있는 Freescale의 16bit ECU 전용 칩셋으로 표 3-4와 같은 특성을 가짐

[표 3-4: MC9S12XF512 칩셋 특성]

주요특성	• 512K 사이즈의 Flash memory • 변경 가능한 programmable EEPROM. • 관리자/사용자 모드로 시스템의 무결성 제공.
기타특성	• S12X CPU의 50Mhz bus speed 가짐. • 12-bit resolution과 3μs conversion time이 있는 ADC • 16-bit capture buffer를 가지는 Capture Timer • 주기적인 programmable interrupt timer • task scheduling 또는 cyclic wake-up을 위한 RTI (Real Time Interrupt) • Full Stop mode를 포함한 모든 모드에서 가능한 API (Asynchronous Periodic Interrupt) • 오류 검출이 가능한 6개 채널의 PWM(Pulth Width Modulator) • MSCAN module • 최대 각각 2개씩의 SPI(Serial Peripheral Interface)와 SCI (Serial Communication Interface)
FlexRay 특성	• FlexRay V2.1 버전의 Protocol에 따라 구현 • 2개의 각 채널마다 2.5, 2.8, 10MBit/s의 데이터 속도를 제공 • PLL을 이용하여 4MHz~40MHz범위의 FlexRay clock 사용 가능. • 32개로 구성된 메시지 버퍼 • 1개의 채널당 2개의 독립적인 수신 FIFO • 별도의 6가지 interrupt channel 존재 (Receive, receive FIFO channel A, receive FIFO channel B, Transmit, Error, Wake-up)

○ EVB9S12XF512E 보드에는 CAN 트랜시버 및 FlexRay 트랜시버 (NXP TJA1080)가 함께 구성되어 있어, FlexRay 통신과 함께 CAN도 이용하여 다양한 네트워크 환경의 개발이 가능함

○ 개별 동작 모드가 있어 개발 및 테스트에 유용함

○ EVB9S12XF512E의 외관은 [그림 3-23]에서와 같음

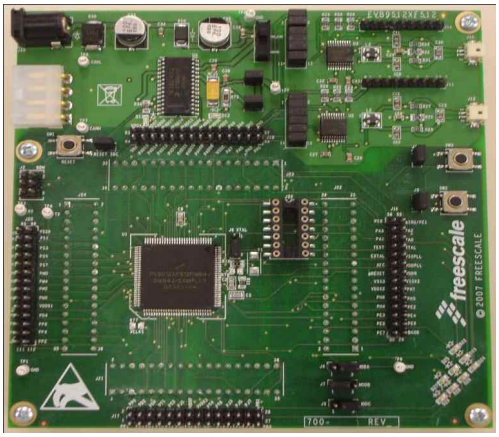


그림 3-23. EVB9S12XF512E 보드

▶ 소프트웨어 개발 환경

○ Freescale의 개발 및 디버깅 툴인 CodeWarrior Development Studio for HC12X을 사용함

○ 디버깅이 가능한 프로그래머로 P&E USB BDM Multilink 사용

○ BDM Multilink의 USB 연결을 통한 개발 호스트 PC와 EVB9S12XF512E 보드와의 연결은 [그림 3-24]에서와 같음

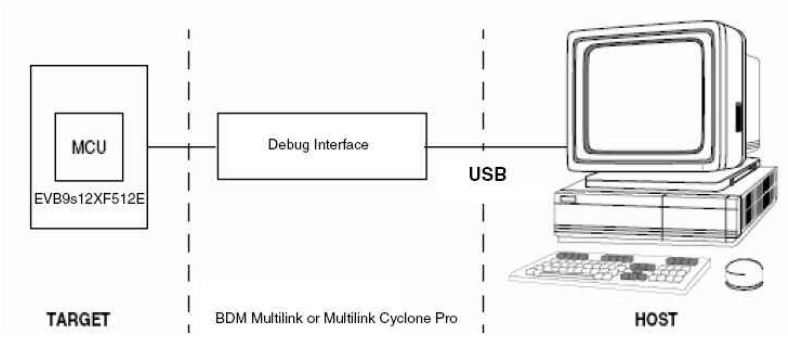


그림 3-24. 개발 호스트 PC와 EVB9S12XF512E 보드와의 연결 구성

○ 소스 코드는 C언어로 구현되었음

▶ FlexRay 통신 서비스 계층의 파일 구조

○ AUTOSAR FlexRay 모듈 관련 파일들을 [그림 3-25]에 나타내었음

○ 이 중에서 TP, NM, SM 모듈이 개발 대상이며 해당 모듈의 파일에는 각각 FrTP_, FrNn, FrSm_ 접두사가 붙어 있음

○ 이외의 파일들은 기존에 개발된 FlexRay 인터페이스 모듈과 FlexRay 드라이버 모듈의 파일들이며 본 구현에서 연동을 위하여 사용하였음

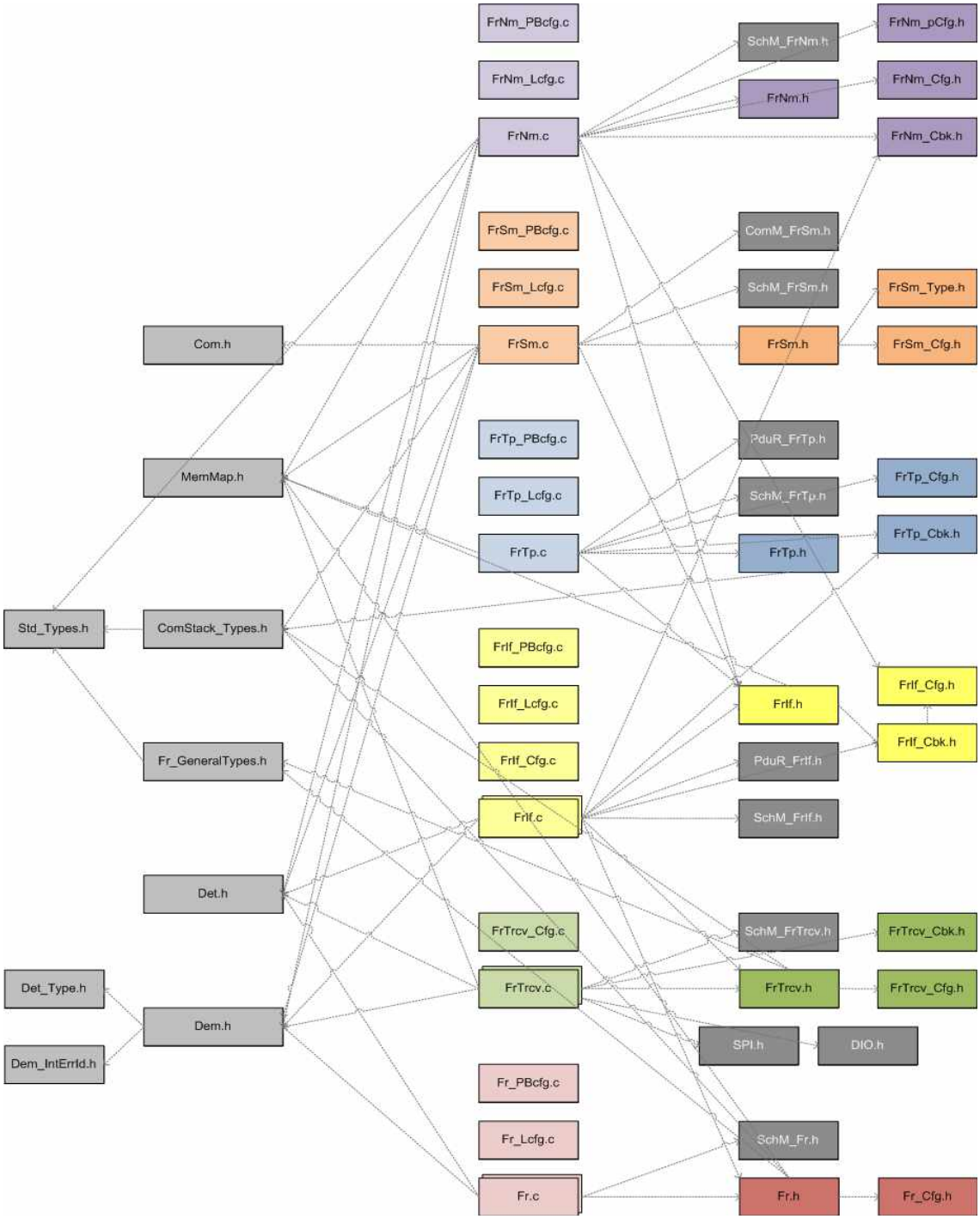


그림 3-25. FlexRay 클러스터의 파일 구조 및 관계

○ 각 모듈의 파일들은 필요에 따라 상위 계층 모듈들의 헤더파일 또는 하위 계층 모

듈들의 헤더파일들을 포함하여 필요 함수를 사용하거나 호출됨

○ 환경 변수의 경우 특성에 따라 pre-compile time, link time, post-build time 등의 세 가지 형태로 구분되어 헤더파일 또는 소스파일에 정의되어 사용됨

○ CodeWarrior 상의 Workspace를 [그림 3-26]에 나타내었음

○ 각 모듈별 파일은 별도 그룹으로 구현하였으며, 기타 필요한 헤더파일들은 임시로 Inc 그룹에 헤더파일만 정의하여 사용하였음

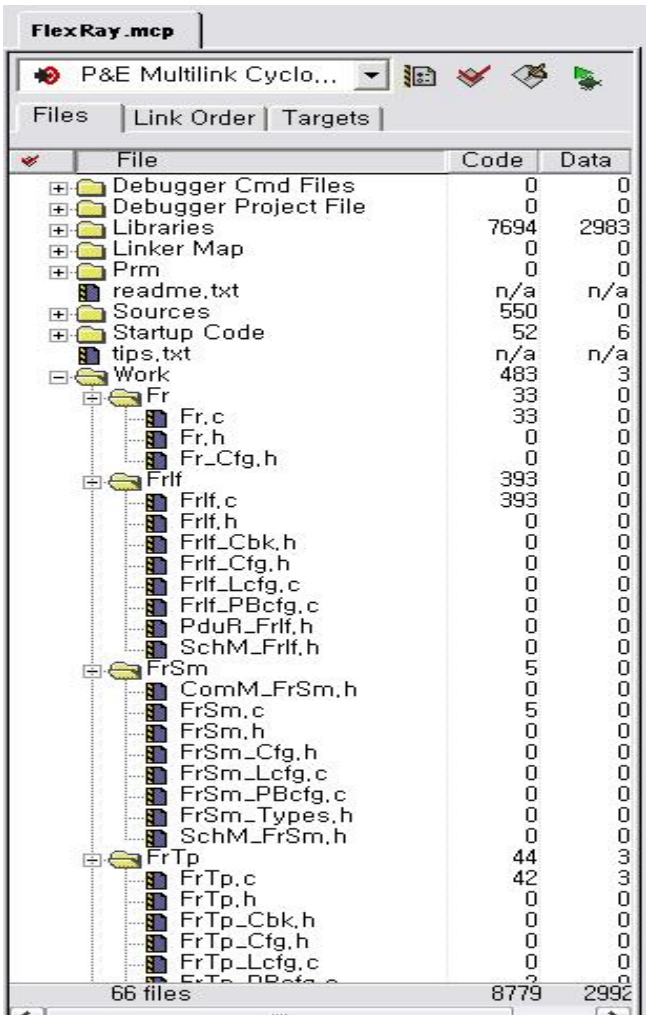


그림 3-26. CodeWarrior에서의 실제 Workspace

▶ FlexRay 통신 서비스 계층의 핵심 기능에 대한 흐름도

○ 각 모듈들은 크게 초기화, 시작, 종료 등의 제어 함수들과 에러 및 상태를 체크하기 위한 상태 확인 함수, 그리고 FlexRay 네트워크 동작을 위한 메시지 송수신 함수로 구성됨

○ 초기화 및 시작 과정은 필요 순서에 따라 ECU 관리자 및 AUTOSAR 네트워크 관리자, FlexRay 상태 관리자에 의해 수행됨

○ FlexRay 상태 관리자는 FlexRay Cluster를 초기화시키고 통신이 수행될 준비를 함

○ FlexRay CC가 초기화되고 정상적으로 통신 준비가 완료되었다면 AUTOSAR 통신 관리자에서 FlexRay 상태 관리자에게 FULL COMMUNICATION 통신 모드를 요청하면서 통신이 시작됨

○ FlexRay TP 모듈은 PDU 라우터와 FlexRay 인터페이스 사이에서 데이터가 전달될 수 있도록 메시지를 분해하거나, 조합하고 흐름을 제어하는 등 다양한 기능을 수행함

○ 대표적인 메시지 송신 과정을 살펴보면 [그림 3-27]에서부터 [그림 3-29]까지의 과정과 같음

▹ 데이터의 전송

- [그림 3-27]에서 보는 바와 같이 PDU 라우터에서 TP에게 데이터 송신을 요청하면 일단 플래그를 설정한 후 리턴하고나면, [그림 3-28]에서처럼 메인 함수에서 적절한 프레임을 생성하여 FlexRay 인터페이스 측에 전달하고, [그림 3-29]에서처럼 전송 완료 후 인터페이스로부터 데이터 전송 확인이 전달되고, 정상 완료되었다면 결과를 PDU 라우터 측에 확인 전달함

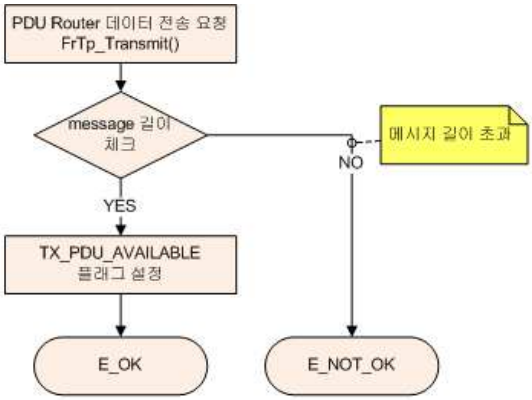


그림 3-27. 데이터 전송 요청

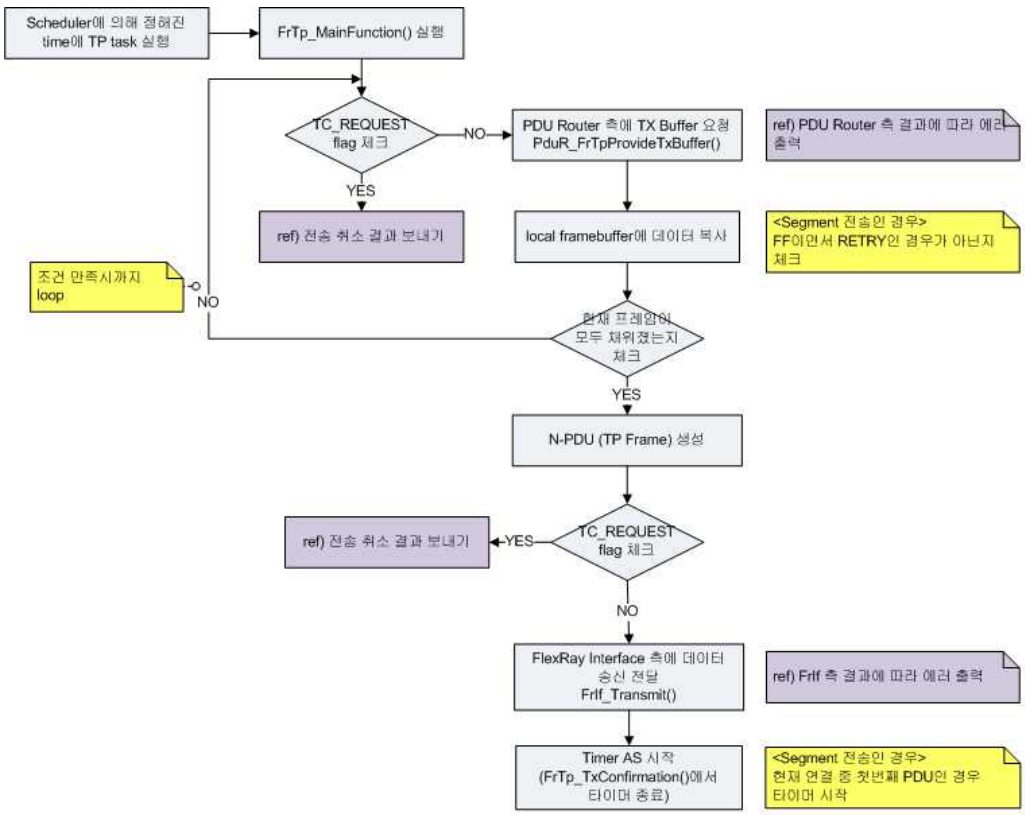


그림 3-28. 데이터의 전송

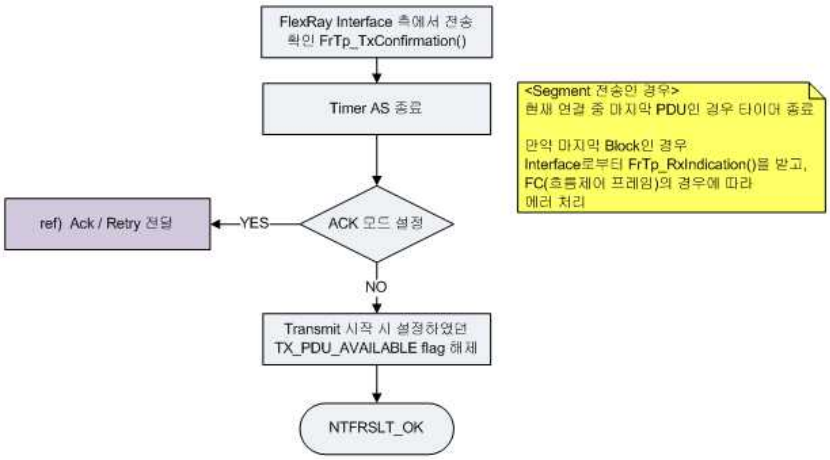


그림 3--29. 전송 확인

▷ 데이터의 수신

- 데이터 수신은 FlexRay 인터페이스의 각 Cluster마다 할당된 JobListExec함수를 통해 수행됨
- FrIf_JobListExec_<Cluster ID> 함수는 연결된 FlexRay CC의 Absolute Timer Interrupt에 의해 발생됨
- FlexRay 드라이버로부터 L-PDU 데이터를 수신하면 FlexRay TP 모듈에게 알리고 데이터를 가져갈 수 있도록 함
- FlexRay TP 모듈에서는 데이터 송신과 마찬가지로 스케쥴러에 의한 메인 함수에서 프레임 데이터를 확인하고 필요하다면 데이터를 조합하여 PDU 라우터 측에 데이터를 전달함
- 그림 3-30에 이러한 과정을 나타내었음

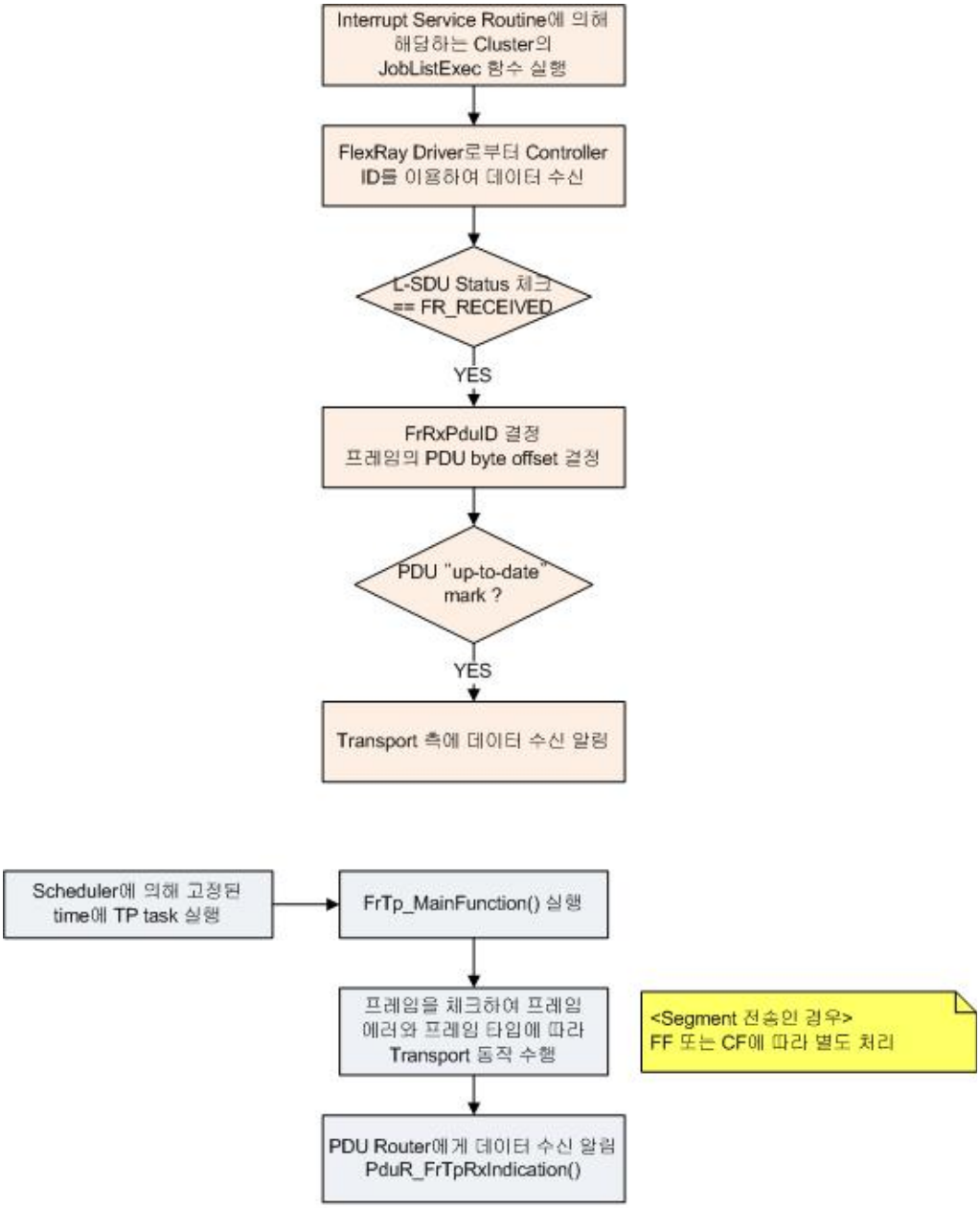


그림 3-30. 데이터 수신 과정

▶ 기능의 검증

- 구현에 대한 테스트는 2개의 ENV9S12XF512E 보드를 각각 FlexRay A, B 두 개의 채널로 연결하여 이루어졌음
- 시험을 위한 보드의 연결은 [그림 3-31]에서와 같음

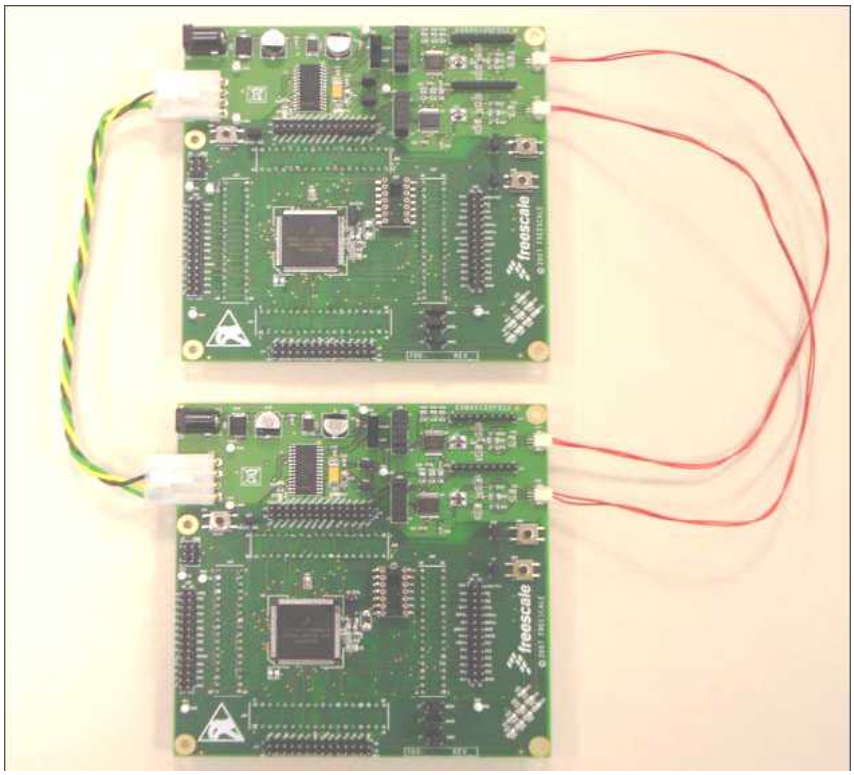


그림 3-31. 2개의 EVB9S12XF512E 보드 연결

- 각 보드는 하나의 ECU와 같이 동작하며, FlexRay 네트워크로 연결되어 있음
- 그림 상의 오른쪽에 위치한 선이 FlexRay를 위한 네트워크 연결선이며 A, B 두개의 채널로 연결되어 있음
- FlexRay를 통하여 송수신된 상태는 보드의 LED의 점멸로 표시됨
- 검증을 위하여 각 모듈은 다음과 같이 동작되도록 하였음

<Node1에서의 LED 동작>

- Slot 1에서의 송신 시 D23 LED ON

- Slot 4에서의 수신 시 D25 LED ON
- Slot 62에서의 수신 시 D28 LED ON

<Node2에서의 LED 동작>

- Slot 1에서의 수신 시 D23 LED ON
- Slot 4에서의 송신 시 D25 LED ON
- Slot 62에서의 송신 시 D28 LED ON

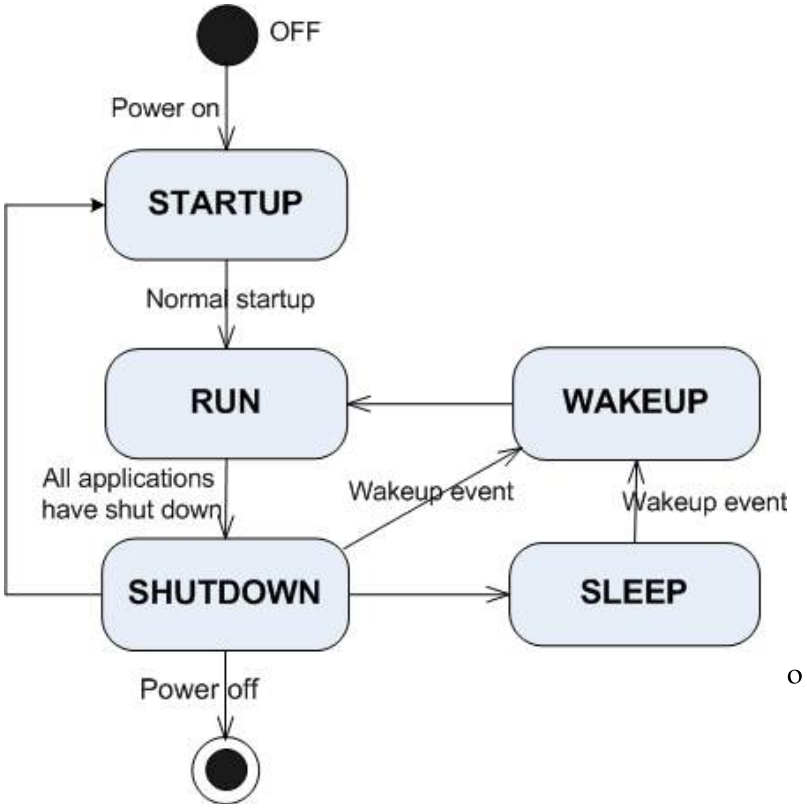
- FlexRay 데이터의 송수신과 관계없이 각 모듈이 정상적으로 동작하고 있는지 여부를 확인하기 위하여 D22, D27, D29의 별도 LED가 지속적으로 점멸되도록 하였음
- 이 시험을 통하여 AUTOSAR 기반으로 구현된 FlexRay 데이터 송수신을 LED 점멸을 통해 확인할 수 있었으며, 계층화된 단계별 모듈 동작 역시 확인 가능하였음
- TP 모듈에 의해 데이터의 분리와 조합을 통한 데이터 전송이 가능한지 확인함
- 네트워크 관리를 위하여 기본적인 제어가 가능한 초기화, 시작, 종료 등의 함수를 제공하고, 이러한 함수들을 이용한 FlexRay CC의 제어가 가능한지 확인함
- FlexRay 통신 상태와 버스의 에러 상태를 확인할 수 있는 인터페이스가 구현되어 있는지 확인하고 이러한 함수들을 이용하여 FlexRay CC의 상태 확인이 가능한지 확인함

라. 서비스 클러스터 개발

(1) ECU 상태 관리자

(가) 개요

ECU 상태 관리자 basic software 모듈을 초기화하고 종료화 하며 ECU의 상태 및 상태 천이를 관리하는 역할을 한다. 아래 [그림 3-32]는 ECU 상태 관리자의 주요 State diagram을 보여준다.



o

그림 3-32. ECU 주요 상태

[그림 3-33]의 상태 천이도는 ECU의 상태가 OFF에서 시작해서 STARTUP, RUN을 거쳐 SLEEP이나 OFF, RESET으로 변경되는 life cycle을 보여준다.

STARTUP 상태의 목적은 BSW 모듈을 초기화하는 것이다. STARTUP 상태는 OS 구동 이전과 OS 구동 이후로 나뉜다. MCU 초기화 같은 가장 기본적이 하드웨어 초기화는 OS 시작이전에 수행되고 나머지 모듈의 초기화는 OS가 구동된 후에 수행된다. RUN 상태는 모든 BSW 모듈이 초기화 된 후에 ECU 상태 관리자에 의해 천이되는 상태이다. RUN 상태가 되면 RTE 위의 SW-C에 BSW가 초기화 되고 동작이 시작되었음을 알려준다. 또한 RUN 상태에서는 응용 소프트웨어의 동기화된 셧다운 시키는 방법을 제공한다. RUN 상태는 응용 프로그램에 의해 요청되며 RUN 요청이 없다면 ECU 상태 관리자는 셧다운을 수행한다. RUN 상태는 두 서브 상태로 나뉘는데 이는 보통의 RUN 상태와 POST RUN 상태이다. POST RUN 상태는 ECU가 SLEEP 상태로 천이하기전에 정리 작업을 수행하기 위해 SW-C에 의해 요청

된다. SHUTDOWN 상태는 BSW 모듈의 셧다운을 수행하며 최종적으로 SLEEP, OFF 혹은 RESET과 같은 셧다운 타겟 중에 하나로 천이된다. SLEEP 상태는 에너지 절약 상태이다. 전형적으로 SLEEP상태는 코드는 수행되지 않지만 전력은 공급되는 상태이다. 설정에 따라서 SLEEP 상태에서 ECU가 wakeup 될수 있다. 아울러 SLEEP 상태에서는 설정가능한 sleep mode 셋을 제공하며 Sleep mode 셋은 하드웨어에 따라 달라질수 있다. WAKEUP 상태는 ECU가 wakeup에 의해 SLEEP 상태에서 빠져 나와서 천이되는 상태로 MCU의 모드를 SLEEP에서 정상 모드로 변경한다. OFF 상태는 전력이 공급되지 않는 상태를 의미한다.

(나) 파일 구조

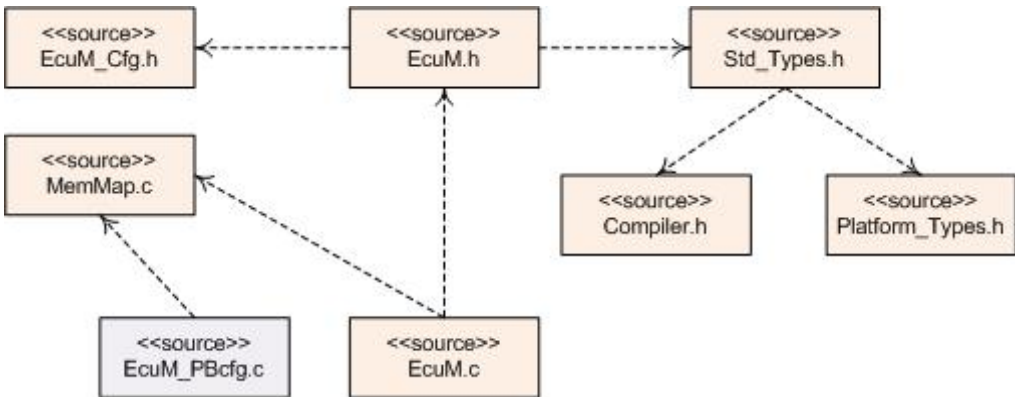


그림 3-33 ECU 상태 관리자 모듈의 파일 구조

ECU 상태 관리자 모듈은 EcuM.c, EcuM.h, EcuM_Cfg.h로 구성된다. EcuM.c 파일에는 ECU 상태 관리자 모듈의 기능적 동작을 위한 API가 구현되어 있고 EcuM.h 파일에는 ECU 상태 관리자 모듈에서 사용하는 데이터 타입이 정의되어 있고, EcuM_Cfg.h 파일에는 사용자가 정의한 ECU 상태 관리자 모듈의 파라미터 및 컨테이너 등 설정 정보를 위한 코드가 포함된다.

(다) 모듈 기능

■ 일반 함수

Name	Description

EcuM_GetVersionInfo	이 모듈의 버전 정보를 리턴함
EcuM_Init	ECU State Manger를 초기화하고 startup 과정을 수행
EcuM_Startup_Two	Startup 2 상태를 구현
EcuM_Shutdown	셧다운 hook를 통해 호출되며 이 함수를 수행 컨트롤을 인계받아서 GO OFF 2 동작을 수행
EcuM_GetState	상태값을 리턴
EcuM_RequestRUN	RUN 상태를 요청
EcuM_ReleaseRUN	RUN 상태를 해제
EcuM_RequestPOST_RUN	POST RUN 상태를 요청
EcuM_ReleasePOST_RUN	POST RUN 상태를 해제
EcuM_SelectShutdownTarget	셧다운 타겟을 설정
EcuM_GetShutdownTarget	Shutdown target을 리턴
EcuM_GetLastShutdownTarget	마지막 Shutdown target을 리턴
EcuM_SelectApplicationMode	OS의 application mode를 설정
EcuM_GetApplicationMode	OS의 application mode를 얻어옴
EcuM_MainFunction	ECU 상태 관리자의 모든 동작을 구현
EcuM_SetWakeupEvent	Wakeup event를 설정하는 함수

(라) ECU State Manager 개발

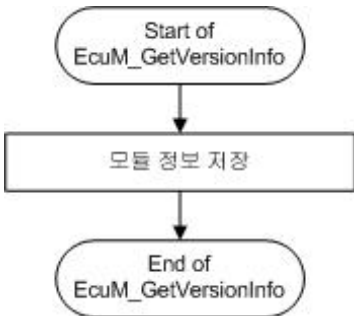
① AUTOSAR ECU 상태 관리자 주요 API 내부 설계

○ EcuM_GetVersionInfo()

- 기능

구문 및 형태	Void EcuM_GetVersionInfo(Std_VersionInfoType* versioninfo)
설 명	o 모듈 버전 정보를 리턴 o *versioninofr는 Std_Types.h에 정의

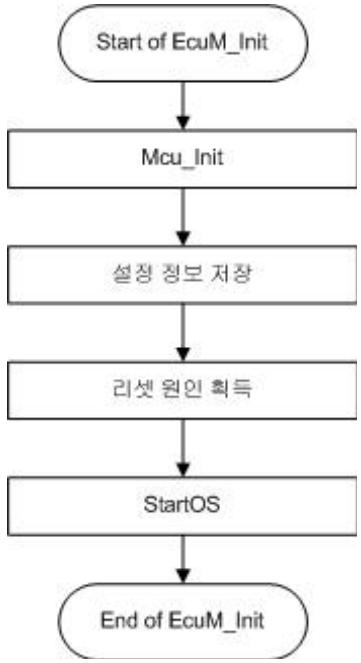
- EcuM_GetVersionInfo 구현 절차

세부 구현 절차	 < EcuM_GetVersionInfo 구현절차 > (1) Vendor ID, Module ID 등 기본적인 모듈 정보 저장한다.

- 기능 ○ EcuM_Init()

기본 문법	Void EcuM_Init()
설 명	o ECU 상태 관리자가 제일 먼저 수행해야 하는 API o 부트 코드 수행이후에 이 함수를 호출함으로써 제어 권한이 ECU 상태 관리자에 넘어감 o Mcu_Init() API를 호출함으로써 MCU를 초기화 한다. o 기본적인 하드웨어(MCU)를 초기한 후에 OS를 시작해야 한다.

- EcuM_Init 구현 절차

세부 구현 절차	 < COM_DeInit 구현 절차 > (1) Mcu_Init - MCU를 초기화 시키는 API (2) 설정 정보 저장 - 셧다운 타겟, 부트타겟 등 설정 정보 등을 읽어서 저장 (3) 리셋 원인 획득 - Mcu_GetResetReason() API를 호출함으로써 리셋 원인 획득 (4) OS 시작 StartOS() API를 호출함으로써 OS를 활성화 시킴

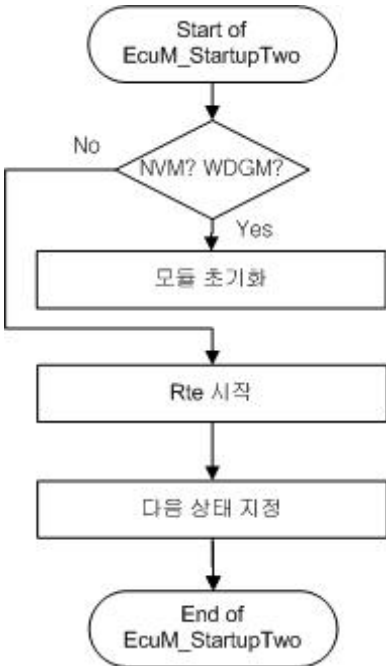
○ EcuM_Startup_Two()

- 기능

구문 및 형태	Void EcuM_Startup_Two()

설명	o OS가 시작되고 난 후에 수행되는 초기화 과정 o Rte 동작을 활성화 시킴 o NVM이나 WDGM 모듈이 ON 상태이면 이 모듈들을 초기화 시킨다.

- EcuM_Startup_Two 구현 절차

세부 구현 절차	 < EcuM_Startup_Two 구현 절차 > (1) BSW 모듈 존재 여부 판단 - 있으면 초기화 (2) RTE 시작 시킴 (3) 다음 상태 설정

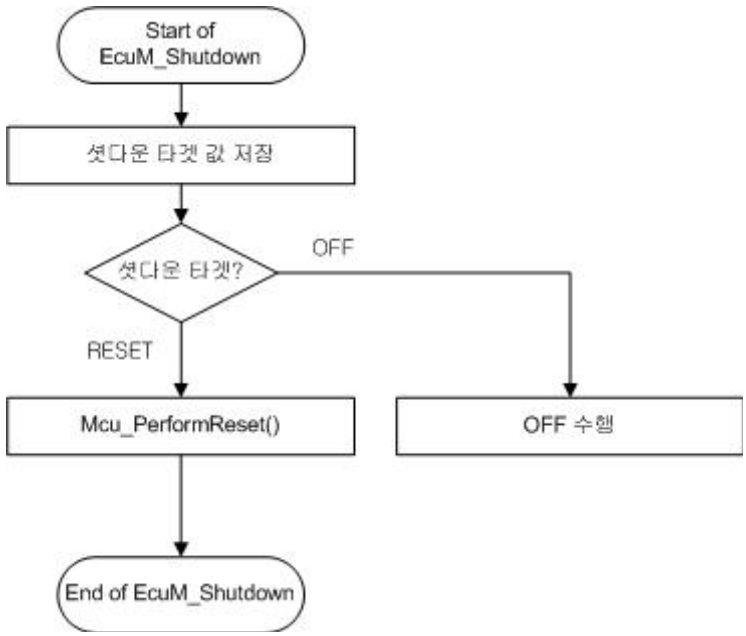
○ EcuM_Shutdown()

- 기능

구문 및 형태	Void EcuM_Shutdown ()
설 명	o 셧다운 타겟값을 사전에 저장하고 있어야 함

	o 저장된 셧다운 타겟 값에 따라 다음 상태로 천이

- EcuM_Shutdown구현 절차

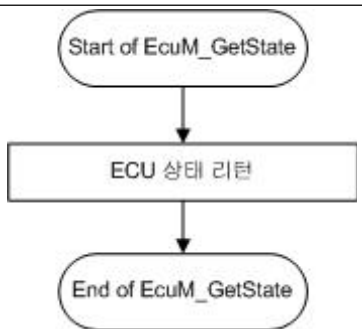
세부 구현 절차	 < EcuM_Shutdown 구현 절차 > (1) 셧다운 타겟 값 저장 (2) 셧다운 타겟값을 비교해서 다음 동작 수행 - RESET인 경우 Mcu_PerformReset을 호출함으로 RESET 수행 - OFF인 경우OFF 동작 수행

○ EcuM_GetState()

- 기능

구문 및 형태	Std_ReturnType EcuM_GetState (EcuM_StateType* state)
설명	o ECU의 현재 상태를 알려준다.

- EcuM_GetState 구현 절차

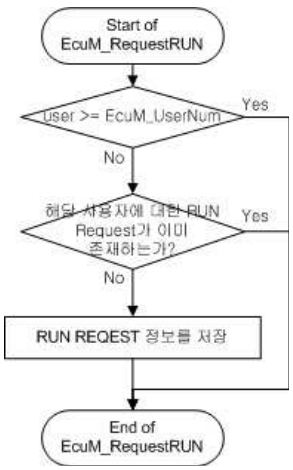
세부 구현 절차	 < EcuM_GetState 구현 절차 > (1) ECU의 상태 정보 리턴

○ EcuM_RequestRUN()

- 기능

구문 및 형태	Std_ReturnType EcuM_RequestRUN (EcuM_UserType user)
설명	o 한 사용자가 RUN Request후에 이에 대한 처리가 되기 전에 다시 RUN을 Request해서는 안된다. o user 정보가 전체 사용자의 정보 범위 내에 존재해야 한다.

- EcuM_RequestRUN 구현 절차

세부 구현 절차	 < EcuM_RequestRUN 구현 절차 >

	(1) 사용자 정보가 전체 사용자 범위 내에 있는지 확인 - 범위 내에 있으면 정상 동작 (2) 해당 사용자에 대한 RUN Request가 이미 존재하는지 확인 - 기존에 RUN Request가 존재하면 오동작으로 간주 (3) 요청된 RUN Request를 저장함

○ EcuM_ReleaseRUN()

- 기능

구문 및 형태	Std_ReturnType EcuM_ReleaseRUN (EcuM_UserType user)
설명	o RUN Release를 요청하는 사용자의 상태는 현재 RUN Request를 요청중이어야 한다. 즉, RUN Request를 요청하지 않은 사용자가 RUN Release를 요청할 경우 오동작으로 인식한다. o user 정보가 전체 사용자의 정보 범위 내에 존재해야 한다.

- EcuM_ReleaseRUN 구현 절차

세부 구현 절차	 < EcuM_ReleaseRUN 구현 절차 >

	(1) 사용자 정보가 전체 사용자 범위 내에 있는지 확인 - 범위 내에 있으면 정상 동작 (2)해당 사용자에 대한 RUN Request가 존재하는지 확인 - 기존에 RUN Request가 존재하지 않으면 오동작으로 간주 (3)요청된 RUN Release를 저장함.

○ EcuM_RequestPOST_RUN()

- 기능

구문 및 형태	Std_ReturnType EcuM_RequestPOST_RUN (EcuM_UserType user)
설명	o 한 사용자가 POST RUN Request후에 이에 대한 처리가 되기 전에 다시 POST RUN을 Request해서는 안된다. o user 정보가 전체 사용자의 정보 범위 내에 존재해야 한다.

- EcuM_RequestPOST_RUN 구현 절차

세부 구현 절차	 < EcuM_RequestPOST_RUN 구현 절차 >

	(1) 사용자 정보가 전체 사용자 범위 내에 있는지 확인 - 범위 내에 있으면 정상 동작 (2) 해당 사용자에 대한 POST RUN Request가 존재하는지 확인 - 기존에 POST RUN Request가 존재하면 오동작으로 간주 (3) 요청된 POST RUN Request를 저장함

○ EcuM_ReleasePOST_RUN()

- 기능

구문 및 형태	Std_ReturnType EcuM_ReleasePOST_RUN (EcuM_UserType user)
설명	o POST RUN Release를 요청하는 사용자의 상태는 현재 RUN Request를 요청중이어야 한다. 즉, POST RUN Request를 요청하지 않은 사용자가 POST RUN Release를 요청할 경우 오동작으로 인식한다. o user 정보가 전체 사용자의 정보 범위 내에 존재해야 한다.

- EcuM_ReleasePOST_RUN 구현 절차

세부 구현 절차	 < EcuM_ReleasePOST_RUN 구현 절차 >

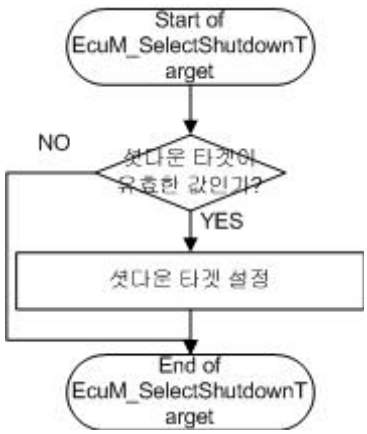
	(1) 사용자 정보가 전체 사용자 범위 내에 있는지 확인 - 범위 내에 있으면 정상 동작 (2)해당 사용자에 대한 POST RUN Request가 존재하는지 확인 - 기존에 POST RUN Request가 존재하지 않으면 오동작으로 간주 (3)요청된 POST RUN Release를 저장함.

○ EcuM_SelectShutdownTarget()

- 기능

구문 및 형태	Std_ReturnType EcuM_SelectShutdownTarget (EcuM_StateType shutdownTarget, uint8 sleepMode)
설 명	o 셧다운 타겟을 설정하는 기능을 제공한다. o 설정하고자 하는 셧다운 타겟은 설정 가능한 범위 내에 있어야 한다.

- EcuM_SelectShutdownTarget 구현 절차

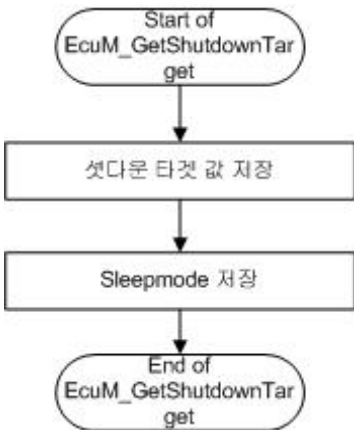
세부 구현 절차	 < EcuM_SelectShutdownTarget 구현 절차 > (1) 셧다운 타겟 값 유효성 검사 (2) 셧다운 타겟 설정

○ EcuM_GetShutdownTarget()

- 기능

구문 및 형태	Std_ReturnType EcuM_GetShutdownTarget (EcuM_StateType* shutdownTarget, uint8* sleepMode)
설명	o ECU 상태 관리자는 셧다운 타겟 값을 알려주는API를 제공한다. o user는 이 함수를 호출함으로써 셧다운 타겟과 sleepmode를 알수 있다.

- EcuM_GetShutdownTarget 구현 절차

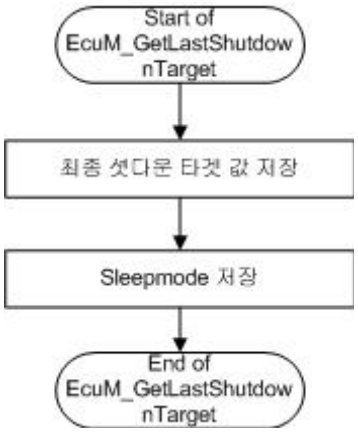
세부 구현 절차	 < EcuM_GetShutdownTarget 구현 절차 > (1) 셧다운 타겟 값 저장 (2) Sleep mode 저장 (3) 저장된 정보는 return 시에 전달됨

○ EcuM_GetLastShutdownTarget()

- 기능

구문 및 형태	Std_ReturnType EcuM_GetLastShutdownTarget (EcuM_StateType* shutdownTarget, uint8* sleepMode)
설명	o ECU 상태 관리자는 최종 셧다운 타겟 값을 알려주는API를 제공한다. o user는 이 함수를 호출함으로써 최종 셧다운 타겟과 sleepmode를 알수 있다.

- EcuM_GetLastShutdownTarget 구현 절차

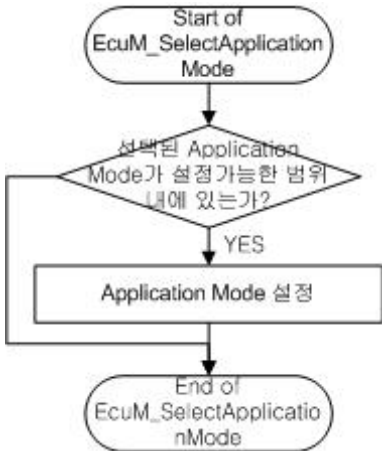
세부 구현 절차	 < EcuM_GetLastShutdownTarget 구현 절차 > (1) 최종 셧다운 타겟 값 저장 (2) Sleep mode 저장 (3) 저장된 정보는 return 시에 전달됨

○ EcuM_SelectApplicationMode()

- 기능

구문 및 형태	Std_ReturnType EcuM_SelectApplicationMode (AppModeType appMode)
설 명	o Application Mode를 설정할 수 있는 기능을 제공한다. o 선택된 Application Mode는 설정가능한 범위내에 있어야 한다.

- EcuM_SelectApplicationMode 구현 절차

세부 구현 절차	 < EcuM_SelectApplicationMode 구현 절차 > (1) 선택된 Application Mode가 설정 가능한 범위에 있는지 확인한다. - 설정가능한 범위 내에 있다면 입려된 application mode로 설정하고 그렇지 않으면 오동작으로 간주

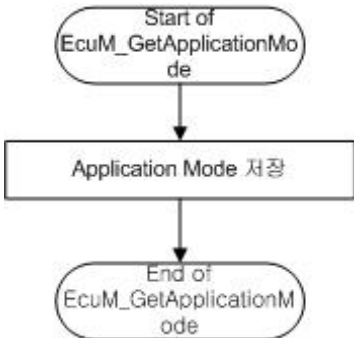
○ EcuM_GetApplicationMode ()

- 기능

구문 및 형태	Std_ReturnType EcuM_GetApplicationMode (AppModeType * appMode)
설 명	o user에게 Application Mode 값을 알려주는 함수이다.

- EcuM_GetApplicationMode 구현 절차

세부 구현 절차	

	 < EcuM_GetApplicationMode 구현 절차 > (1) Application Mode 값을 저장하고 리턴시 이 값이 전달된다.

○ EcuM_KillAllRUNRequests()

- 기능

구문 및 형태	void EcuM_KillAllRUNRequests (void)
설명	o 무조건적으로 RUN과 POST RUN에 대한 모든 요청을 제거하는 함수이다. o 결과적으로 EcuM_RequestRUN과 EcuM_RequestPOST_RUN은 어떤 새로운 요청도 받아들이지 않는다.

- EcuM_GetApplicationMode 구현 절차

세부 구현 절차	 < EcuM_GetApplicationMode 구현 절차 > (1) 모든 RUN 요청을 해제한다

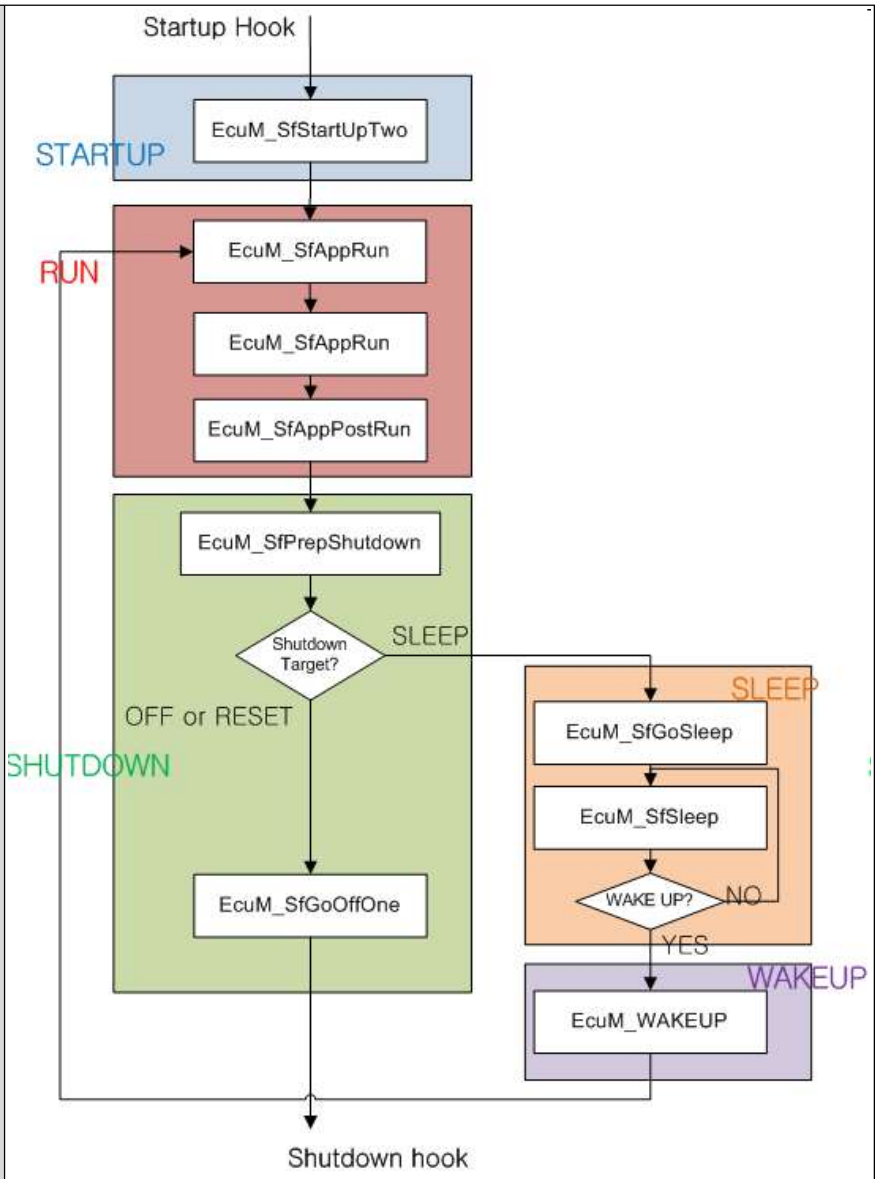
○ EcuM_MainFunction()

- 기능

구문 및 형태	Void EcuM_MainFunction ()
설명	o OS가 시작되고 나서 동작하는 동안 ECU의 상태를 관리하는 역할을 수행 o 이 서비스는 적당한BSW 태스크에서 주기적으로 호출된다 o OS가 동작하고 나면 태스크 단위로 동작한다. o ECU 상태 관리에 관해서 EcuM_Init 서비스와 EcuM_Shutdown 서비스외에 모든 서비스는 EcuM_MainFunction내에서 처리된다.

- EcuM_MainFunction 구현 절차

세부 구현 절차	



< EcuM_MainFunction 구현 절차 >

(1) Startup 에서 EcuM_MainFunction을 호출함으로써 시작함.

(2) 적당한 BSW 태스크에 의해 주기적으로 호출되어야 함.

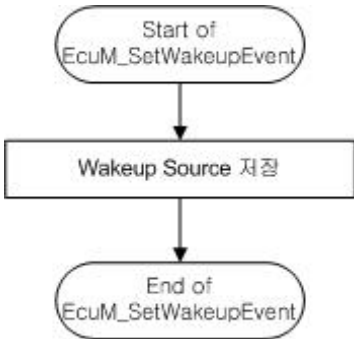
(3) 각 상태를 처리하는 State handler 함수가 존재하며 각 상태에서 해당 동작을 수행함.

○ EcuM_SetWakeupEvent ()

- 기능

구문 및 형태	Void EcuM_SetWakeupEvent (EcuM_WakeupSourceType sources)
설명	o Wakeup event를 설정한다. o Wakeup source 값을 받아서 내부 변수로 저장한다.

- EcuM_SetWakeupEvent 구현 절차

세부 구현 절차	 < EcuM_SetWakeupEvent 구현 절차 > (1) Wakeup source 값을 받아서 내부 변수로 저장한다

(마) ECU State Manager 안정화

이절에서는 서비스 모듈에 속한 각 API들을 통합하여, 서비스 모듈 고유의 기본 기능이 정상적으로 수행하는지를 검증하고, 오류 발견 시 보완하는 작업을 수행하고자 한다. 예를 들어, ECU State Manager를 수행시키고 각 상태마다 동작을 수행하여 테스트를 수행하며 이러한 자체 통합 테스트를 수행한다. 이러한 테스트를 통해 문제점이 발견되면, 해당 오류를 디버깅을 통해 수정하는 등의 안정화 과정을 수행한다. 아래 각 항목은 AUTOSAR Ecu State Manager에서 수행한 안정화 항목을 나타낸다.

○ BSW 모듈에 대한 초기화 루틴과 순서 관계 정보 저장 여부 확인
○ BSW 모듈에 대한 종료화 루틴과 순서 관계 정보 저장 여부 확인
○ 현재 wake up 원인을 저장할 수 있게 하는지 여부 확인
○ ECU 상태 관리자가 초기화 과정에서 OS를 실행시키는지 여부를 확인
○ 마지막 리셋 원인을 확인할 수 있는 API 제공 여부 확인
○ RUN 상태로 머물기 위한 조건 확인
○ Sleep mode를 선택하기 위한 API 제공 여부 확인

① AUTOSAR Ecu State Manager 안정화

○ BSW 모듈에 대한 초기화 루틴과 순서 관계 정보 저장 여부 확인

- 기능

설 명	o ECU 상태 관리자가 현재 ECU 상태 정보를 알려주는 API를 제공하는지 여부를 확인

- 절차

안정화 수행 방법	(1) 시험 프로그램은 시스템 서비스 클러스터 (ECU 상태 관리자, OS, MCU 드라이버)와 시험을 위한 응용 프로그램으로 구성한다. (2) 전원이 ON 되면서 ECU 상태 관리자는 저장된 초기화 루틴과 순서에 따라 BSW 초기화를 수행한다.

	(3) OS 태스크에서 주기적으로 EcuM_MainFunction 함수를 호출한다. (4) 응용 프로그램에서 RUN 요청하는 EcuM_RequestRUN 함수를 호출한다. (5) ECU의 상태가 RUN 상태로 천이된다. (6) 응용 프로그램에서 RUN 요청을 해제하는 EcuM_ReleaseRUN 함수를 호출한다. (7) 위의 과정을 거치는 동안 응용 프로그램에서 주기적으로 ECU 상태 정보를 얻어 오는 EcuM_GetState 함수를 호출해서 ECU 상태 정보를 얻어오고 이 값에 따라 Seven Segment LED에 ECU 상태를 나타낸다
세부 분석 및 수정	(1) 응용 프로그램에서 RUN 요청하는 EcuM_RequestRUN을 호출하고 RUN 상태에서 RUN 요청을 해제하는 EcuM_ReleaseRUN을 호출한다. (2) ECU가 STARTUP에서부터 RUN, SHUTDOWN, WAKEUP 등 모든 상태로 천이하는 동안 응용 프로그램에서 주기적으로 ECU 상태 정보를 얻어오는 EcuM_GetState 함수를 호출했을 때 ECU의 상태가 변함에 따라서 읽어 오는 값도 그에 따라 변함을 디버거를 통해 확인하고 응용 프로그램에서 읽어온 ECU 상태 정보에 따라 Seven Segment LED에 ECU 상태를 나타냄으로써 ECU 상태 정보가 올바르게 얻어짐을 확인한다.

○ BSW 모듈에 대한 종료화 루틴과 순서 관계 정보 저장 여부 확인

- 기능

설 명	o ECU 상태 관리자가 현재 ECU 상태 정보를 알려주는 API를 제공하는지 여부를 확인

- 절차

안정화 수행 방법	(1) 시험 프로그램은 시스템 서비스 클러스터 (ECU 상태 관리자, OS, MCU 드라이버)와 시험을 위한 응용 프로그램으로 구성한다. (2) 전원이 ON 되면서 ECU 상태 관리자는 저장된 초기화 루틴과 순서에 따라 BSW 초기화를 수행한다. (3) 초기화 과정에서 reset 원인이 의도되지 않은 경우에는 EcuM_SelectApplicationMode 함수를 호출함으로써 application mode를 default application mode로 설정한다.
세부 분석 및 수정	(1) 초기화 과정에서 reset 원인이 의도되지 않은 경우에는 EcuM_SelectApplicationMode함수를 호출함으로써 application mode를 default application mode로 설정한다. (2) EcuM_SelectApplicationMode 를 호출해서 application mode를 변경할 때 디버거를 통해 application mode를 저장하는 변수의 값이 원하는 값으로 변경된 것을 확인한다.

○ 현재 wake up 원인을 저장할 수 있게 하는지 여부 확인

- 기능

설 명	o wakeup 이벤트 발생시에 ECU 상태 관리자가 현재 wake up 원인을 저장할 수 있게 하는지 여부 확인

- 절차

안정화 수행 방법	(1) 시험 프로그램은 시스템 서비스 클러스터 (ECU 상태 관리자, OS, MCU 드라이버)와 시험을 위한 응용 프로그램으로 구성한다. (2) 전원이 ON 되면서 ECU 상태 관리자는 저장된 초기화 루틴과 순서에 따라 BSW 초기화를 수행한다. (3) OS 태스크에서 주기적으로 EcuM_MainFunction 함수를 호출한다. (4) ECU의 상태가 SLEEP일 때 응용 프로그램은 외부 인터럽트를 사용해서 wakeup 이벤트를 발생시킨다.
세부 분석 및 수정	(1) SLEEP 상태에서 응용 프로그램은 외부 인터럽트를 사용해서 wake up 이벤트를 발생시킨다. (2) Wake up 이벤트 발생시 ECU 상태 관리자는 EcuM_SetWakeupEvent를 호출함으로써 발생한 wake up 원인을 저장하는 것을 디버거를 통해 확인한다.

○ ECU 상태 관리자가 초기화 과정에서 OS를 실행시키는지 여부를 확인

- 기능

설 명	o ECU 상태 관리자는 STARTUP 상태에서 모든 BSW를 초기화시키는데 이때 OS를 실행시키는지 여부를 확인한다.

- 절차

안정화 수행 방법	(1) 시험 프로그램은 시스템 서비스 클러스터 (ECU 상태 관리자, OS, MCU 드라이버)와 시험을 위한 응용 프로그램으로 구성한다. (2) 전원이 ON 되면서 ECU 상태 관리자는 저장된 초기화 루틴과 순서에 따라 BSW 초기화를 수행한다
세부 분석 및 수정	(1) 전원이 ON 되면서 ECU 상태 관리자는 저장된 초기화 루틴과 순서에 따라 BSW 초기화를 수행한다. (2) ECU 상태 관리자는 STARTUP 상태에서 OS를 실행시키는 API를 호출하는 것을 디버거에서 확인하고 실제로 OS가 생성된 태스크를 정상적으로 수행하는지 확인

○ 마지막 리셋 원인을 확인할 수 있는 API 제공 여부 확인

- 기능

설 명	ECU 상태 관리자가 마지막 리셋 원인을 확인할 수 있는 API를 제공하는지 여부 확인

- 절차

안정화 수행 방법	(1) 시험 프로그램은 시스템 서비스 클러스터 (ECU 상태 리자, OS, MCU 드라이버)와 시험을 위한 응용 프로그

	램으로 구성한다. (2) 전원이 ON 되면서 ECU 상태 관리자는 저장된 초기화 루틴과 순서에 따라 BSW 초기화를 수행한다.
세부 분석 및 수정	(1) 전원이 ON 되면서 ECU 상태 관리자는 저장된 초기화 루틴과 순서에 따라 BSW 초기화를 수행한다. (2) ECU 상태 관리자는 초기화 루틴을 수행할때 Mcu_GetResetReason 함수를 호출함으로써 마지막 리셋 원인을 얻어오는데 이 과정을 디버거를 통해 확인한다.

○ RUN 상태로 머물기 위한 조건 확인

- 기능

설 명	ECU 상태 관리자는 ECU 를 RUN 상태로 유지하기 위해 주어진 조건에 따라 동작하는지 여부를 확인한다.

- 절차

안정화 수행 방법	(1) 시험 프로그램은 시스템 서비스 클러스터 (ECU 상태 관리자, OS, MCU 드라이버)와 시험을 위한 응용 프로그램으로 구성한다. (2) 전원이 ON 되면서 ECU 상태 관리자는 저장된 초기화

	루틴과 순서에 따라 BSW 초기화를 수행한다. (3) OS 태스크에서 주기적으로 EcuM_MainFunction 함수를 호출한다. (4) 응용 프로그램에서 RUN 요청하는 EcuM_RequestRUN 함수를 호출한다. (5) ECU의 상태가 RUN 상태로 천이된다. 응용 프로그램에서 RUN 요청을 해제하는 EcuM_ReleaseRUN 함수를 호출한다.
세부 분석 및 수정	(1) ECU가 SLEEP 상태에 있을때 응용 프로그램에서 RUN 요청하는 EcuM_RequestRUN 함수를 호출한다. (2) ECU 상태가 RUN 상태에 있을때 응용 프로그램에서 RUN 요청을 해제하는 EcuM_ReleaseRUN 함수를 호출한다.

○ Sleep mode를 선택하기 위한 API 제공 여부 확인

- 기능

설 명	o ECU 상태 관리자가 다음 ECU 셧다운에서 선택될 sleep mode를 선택하기 위한 API를 제공하는 여부 확인

- 절차

안정화	(1) 시험 프로그램은 시스템 서비스 클러스터 (ECU 상태

수행 방법	관리자, OS, MCU 드라이버)와 시험을 위한 응용 프로그램으로 구성한다. (2) 전원이 ON 되면서 ECU 상태 관리자는 저장된 초기화 루틴과 순서에 따라 BSW 초기화를 수행한다. (3) OS 태스크에서 주기적으로 EcuM_MainFunction 함수를 호출한다. (4) 응용 프로그램에서 RUN 요청하는 EcuM_RequestRUN 함수를 호출한다. (5) ECU의 상태가 RUN 상태로 천이된다. (6) 응용 프로그램에서 EcuM_SelectShutdownTarget 함수를 호출함으로써 셧다운 타겟을 sleep으로, sleep mode를 하드웨어에서 지원하는 sleep mode 중에 하나로 설정한다. (본 시험에서는 sleep mode가 STOP mode, WAIT mode로 구성됨) (7) 응용 프로그램에서 RUN 요청을 해제하는 EcuM_ReleaseRUN 함수를 호출한다. (8) ECU가 SHUTDOWN 상태 이후에 SLEEP 상태로 천이하고 설정된 sleep mode로 변경됨을 확인하다.
세부 분석 및 수정	(1) 응용 프로그램에서 EcuM_SelectShutdownTarget 함수를 호출함으로써 셧다운 타겟을 sleep으로, sleep mode를 STOP, WAIT 중에 하나로 설정하고 EcuM_ReleaseRUN 함수를 호출한다.

2. 차량용 소프트웨어 플랫폼 설계 도구 개발

가. ECU 환경 설정 도구 개발

(1) 다중플랫폼 지원 개발 개요

2차년도까지 개발한 OSEK OS는 16비트 환경의 Freescale MCU(S12XDP512) 타겟 보드를 지원한다. OS의 저변확대를 위해 8/16비트 MCU용 OS를 개발하였으며 이 MCU들의 환경설정 및 코드생성을 지원하기 위해 다중플랫폼을 지원하는 환경설정도구를 개발하였다.

(2) 개발 목표

OSCAR-OSEK 환경 설정 도구는 커널 모듈의 속성들이 정의된 Implementation 파일을 기반으로 환경 설정 기능을 수행한다. 이 Implementation 파일은 MCU 하드웨어에 의존적인 부분이 많으므로 각 MCU별로 파일의 내용이 달라진다. 따라서, 타겟 보드 선택 기능을 개발하기 위해서는 각 MCU별로 Implementation 파일을 정의하고 GUI에서 MCU를 선택했을 때 해당 파일을 기반으로 환경설정 기능을 수행하도록 해야 한다. 본 기능의 개발 목표는 3차년도에 수행한 OS의 플랫폼 확장 개발에 따라 8/32비트 MCU의 환경설정 및 코드생성 기능을 지원하는 것이다.

(3) 개발 결과

3차년도에 추가 개발한 타겟 보드 MCU인 Atmega128(8비트 MCU)과 STM32(32비트 MCU)을 확장 개발하면서 각 MCU의 타겟 보드를 선택할 수 있도록 GUI 기능을 추가하였다.

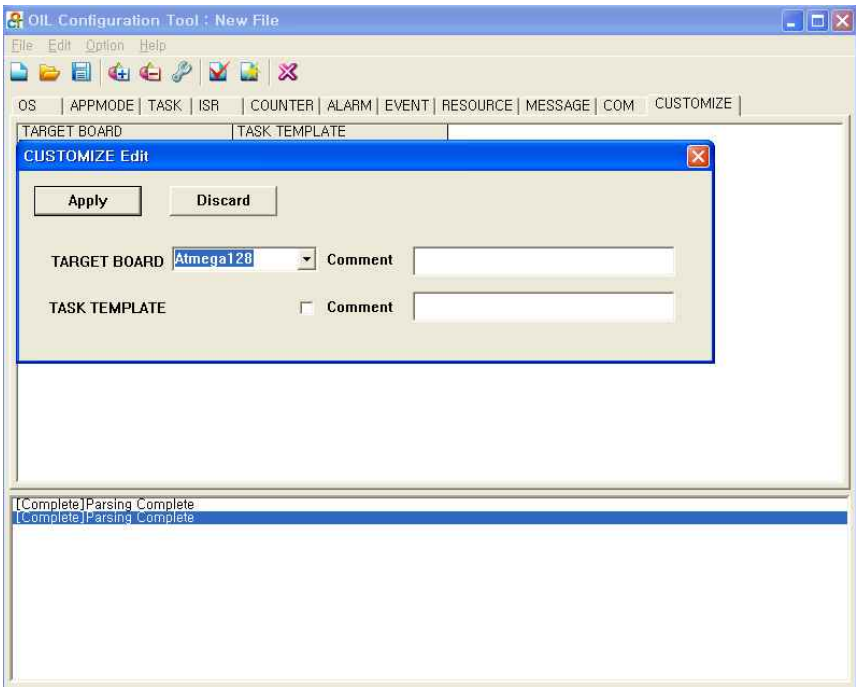


그림 3-34. 타겟 보드 선택 GUI

또한, 타겟 보드 의존적인 정보 처리를 위해 Atmega128 Implementation 파일과 STM32 Implementation 파일을 생성하였다. OSCAR-OSEK 환경 설정 도구는 OS 설정을 위한 OIL 파일 생성을 위해 해당하는 타겟 보드의 MCU Implementation

파일을 선택 할 수 있도록 하였다.

추가로 작성한 Implementation File의 내용은 다음과 같다.

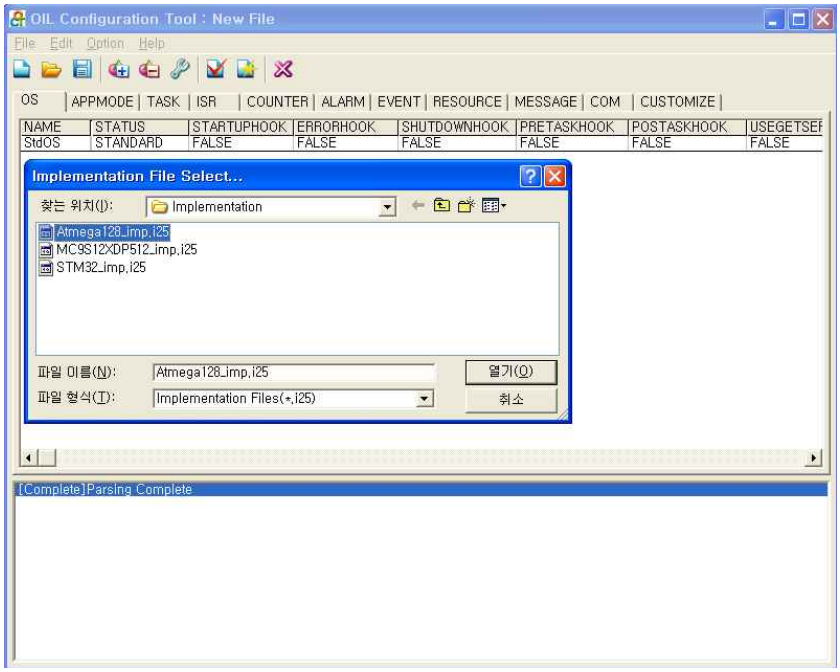


그림 3-35. Implementation 파일 선택

나. BSW 코드생성 도구 개발

(1) 개요

그림 3-36은 ECU 설정 도구에서 하나의 ECU에 있는 모듈을 편집하여 사용하는 과정을 도식화 하였다. 사용자는 ECU 설정 도구에서 AUTOSAR 표준에 부합하는 설정 파라메터 명세(Configuration Parameter Definition)를 바탕으로 ECU설정 명세서를 생성 및 편집을 할 수 있다.

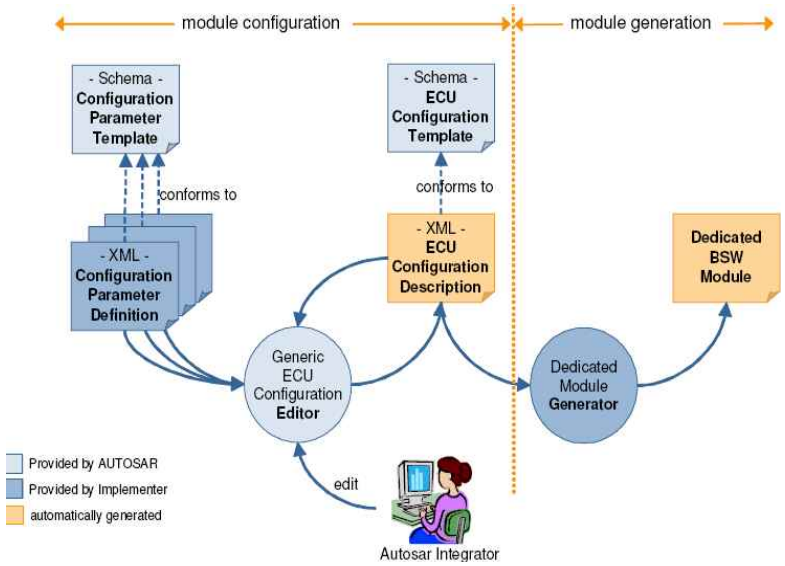


그림 3-36 ECU Configuration Editor

생성된 ECU 설정 명세서는 코드 생성 도구([그림 3-35]의 Dedicated Module Generator)를 이용하여 실행 가능한 코드([그림 3-36]의 Dedicated BSW Module)를 만들어 내게 된다

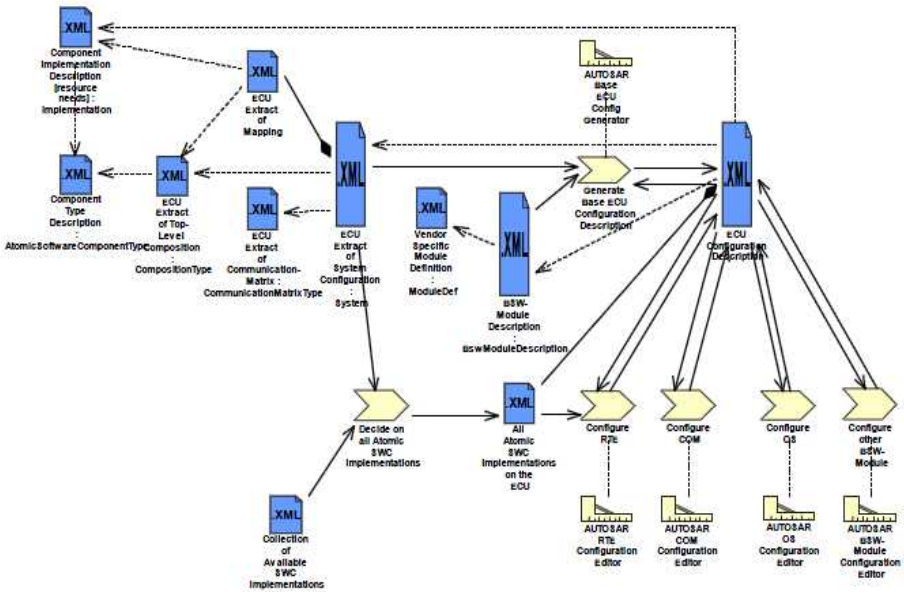
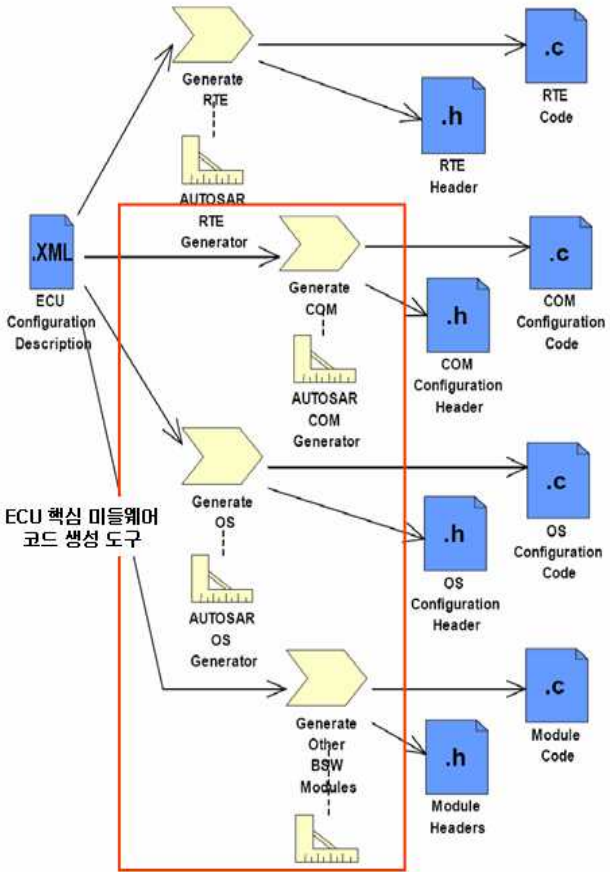


그림 3-37. ECU설정의 세부 표현

(2) 개발의 범위

ECU 설정 도구에서 만들어진 ECU 설정 명세서는 [그림 3-38]와 같이 각 모듈마다 해당되는 생성 도구에 의해 코드 파일(code)과 헤더파일(header)이 만들어지게 된다.



3-32. ECU 핵심 미들웨어 코드 생성

본 과제는 위와 같은 개별 ECU에 따른 ECU 설정 명세서 파일을 기반으로 필요한 소스 코드와 헤더 파일을 만들어 내는 코드 생성 도구의 개발에 목적이 있으며 해당 도구는 AUTOSAR에 정한 XML 버전에 독립적으로 BSW 모듈의 소스 및 헤더파일 생성이 가능하도록 설계하여야 한다. 당해 년도에서는 기존 생성 도구의 OS SC2 코드 생성 기능 추가와 함께 기존 도구의 최적화 및 안정화를 목표로 과업지시서에 포함된 내용을 바탕으로 다음과 같은 세부 사항을 만족하는 도구의 개발을 목적으로 한다

(가) AUTOSAR OS Scalability 2 코드 생성 기능 개발

기존 코드 생성 도구에 OS Scalability 2 코드를 생성할 수 있는 기능을 추가한다.

(나) COM Cluster 코드 생성 기능 수정

기존 코드 생성 도구의 COM Cluster와 관련된 코드 생성 기능을 최적화와 함께 필요한 부분을 수정한다.

(다) JSP 기반 파서 최적화

기존 도구의 JSP 기반 템플릿 파서를 최적화 및 안정화 시킨다.

(3) OS 모듈의 소스코드 구조

AUTOSAR OS(SC2 지원) 모듈은 OSEK Operating System을 기반으로 다음과 같은 확장성을 가진다.

- memory detection
- deadline monitoring
- schedule tables
- enhanced counter structure

(가) 파일 구조

OS 모듈의 종속성은 다음의 가정을 따른다.

① OS 모듈이 카운터를 동작시킬려면 타이머 유닛을 사용한다.

② 사용자가 직접 global time 으로 스케쥴을 동작시킬려면 gobal time에 대한 인터럽트가 필요하다.

③ 사용자가 스케쥴링 테이블을 gobal time으로 동기화시킬려면 OS는 SyncScheduleTable() 서비스를 이용하여야 한다.

[그림 3-39]은 OS 모듈의 헤더파일의 구조이며 필요에 따라 헤더파일을 추가적으로 포함할 수 있다. OS 모듈은 코드파일(code file)의 구조에 대해서는 지정하지 않는다.

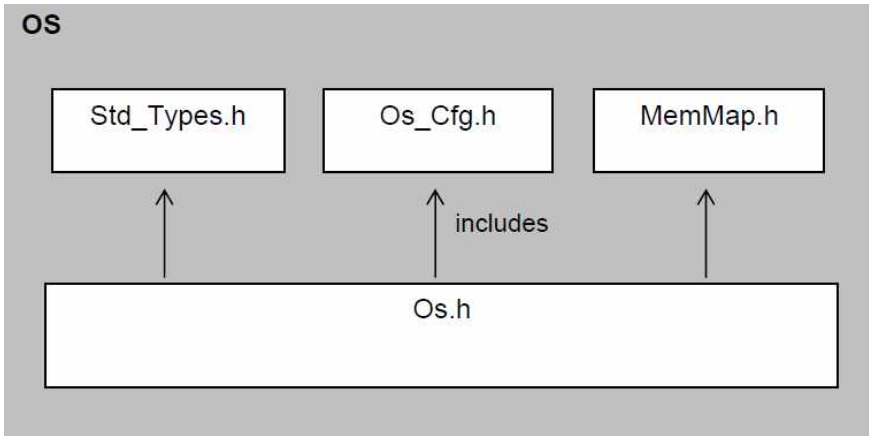


그림 3-39. OS 모듈의 파일 구조

(나) OS 모듈의 Types

OS 모듈에서 사용하는 타입들을 정의한다. 템플릿 파일 작성과 코드 및 헤더 파일을 생성할 경우 사용된다.

이름	ApplicationType	
형태	Scalar	
범위	INVALID_OSAPPLICATION	Invalid
설명	OS-Application의 Identification	
비고	-	

이름	TrustedFunctionIndexType
형태	Scalar
범위	-
설명	Trusted function 의 Identification
비고	

이름	TrustedFunctionParameterRefType
형태	Pointer
범위	-
설명	Trusted Function 호출을 위한 인자를 가지는 구조체의 포인터
비고	-

이름	AccessType
형태	Integral
범위	-
설명	접근 가능한 메모리 영역에 대한 정보
비고	-

이름	ObjectAccessType
형태	Scalar
범위	ACCESS, NO_ACCESS
설명	OS Application 이 해당 Object를 접근 가능 여부
비고	-

이름	ObjectTypeType
형태	Scalar
범위	OBJECT_TASK OBJECT_ISR OBJECT_ALARM OBJECT_RESOURCE OBJECT_COUNTER OBJECT_SCHEDULETABLE
설명	Object의 Identification
비고	-

이름	MemoryStartAddressType
형태	Pointer
범위	-
설명	MCU의 주소 공간 포인터
비고	-

이름	MemorySizeType
형태	Scalar
범위	-
설명	메모리 영역 크기
비고	-

이름	ISRType
형태	Scalar
범위	INVALID_ISR
설명	Interrupt Service Routine 의 Identification
비고	-

이름	ScheduleTableType
형태	Scalar
범위	-
설명	Schedule Table 의 Identification
비고	-

이름	ScheduleTableStatusType
형태	Scalar
범위	SCHEDULETABLE_STOPPED SCHEDULETABLE_NEXT SCHEDULETABLE_WAITING SCHEDULETABLE_RUNNING SCHEDULETABLE_RUNNING_AND_SYNCHRONOUS
설명	Schedule의 상태
비고	-

이름	ScheduleTableStatusRefType
형태	Pointer
범위	-
설명	ScheduleTableStatusType 변수의 포인터
비고	-

이름	CounterType
형태	Scalar
범위	-
설명	Counter의 Identification
비고	-

이름	ProtectionReturnType
형태	Pointer
범위	PRO_IGNORE PRO_TERMINATETASKISR PRO_TERMINATEAPPL PRO_TERMINATEAPPL_RESTART PRO_SHUTDOWN
설명	Protection hook 의 리턴에 대한 OS action의 값
비고	-

이름	RestartType
형태	Scalar
범위	RESTART NO_RESTART
설명	OS Application 의 종료후 Restart 여부
비고	-

이름	PhysicalTimeType
형태	Scalar
범위	-
설명	Conversion macro 에 의해 반환된 값을 사용
비고	-

(다) OS 모듈의 Configuration Parameter

OS 모듈의 설정에 사용되는 컨테이너 및 설정 파라메터들은 다음과 같으며 템플릿 작성 및 코드 작성에 사용된다.

모듈이름	OS
설명	OS 모듈의 설정
컨테이너	OSAlarm: 0..* OsAppMode: 1..* OsApplication: 0..* OsCounter: 0..* OsEvent: 0..* OsIsr: 0..* OsOs: 1 OsResource: 0..* OsScheduleTable: 0..* OsTask: 0..*

컨테이너 이름	OsSetAlarmEvent	
설명	Event 설정을 위한 파라메터들	
서브 컨테이너	없음	
파라메터	이름	OsAlarmSetEventRef {EVENT}
	설명	Alarm action 에 의해 설정될 Event에 대한 참조
	Mul	1
	타입	Reference to OsEvent
	기본값	-
	설정	Pre-Compile time : All Variants
	비고	-
파라메터	이름	OsAlarmSetEventTaskRef {TASK}
	설명	Event에 의해 동작될 Task에 대한 참조
	Mul	1
	타입	Reference to OsTask
	기본값	-
	설정	Pre-Compile time : All Variants
	비고	-

컨테이너 이름	OsAlarm	
설명	비동기적인 정보 전달 및 특정 Task에 사용	
서브 컨테이너	OsAlarmAction: 1 OsAlarmAutoStart: 0..1	
파라메터	이름	OsAlarmAccessingApplication
	설명	접근가능한 가능한 application에 대한 참조
	Mul	0..*
	타입	Reference to OsApplication
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-
파라메터	이름	OsAlarmCounterRef
	설명	Alarm에 사용되는 Counter에 대한 참조
	Mul	1

	타입	Reference to OsCounter
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-

컨테이너 이름	OsAlarm	
설명	비동기적인 정보 전달 및 특정 Task에 사용	
서브 컨테이너	OsAlarmAction: 1 OsAlarmAutoStart: 0..1	
파라메터	이름	OsAlarmAccessingApplication
	설명	접근가능한 가능한 application에 대한 참조
	Mul	0..*
	타입	Reference to OsApplication
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-
파라메터	이름	OsAlarmCounterRef
	설명	Alarm에 사용되는 Counter에 대한 참조
	Mul	1
	타입	Reference to OsCounter
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-

컨테이너 이름	OsAlarmAction
설명	Alarm에 사용되는 notification 의 형태
서브 컨테이너	OsAlarmActivateTask: 0..1 OsAlarmCallback: 0..1 OsAlarmIncrementCounter: 0..1 OsAlarmSetEvent: 0..1

컨테이너 이름	OsAlarmActivateTask	
설명	Task 동작에 사용되는 파라메터 정의	
서브 컨테이너	-	
파라메터	이름	OsAlarmActivateTaskRef

	설명	Alarm에 의해 동작되는 Task의 참조
	Mul	1
	타입	Reference to OsTask
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-

컨테이너 이름	OsAlarmAutostart	
설명	Alarm AutoStart에 대한 정의	
서브 컨테이너	-	
파라메터	이름	OsAlarmAlarmTime
	설명	최초 Alarm이 동작할 relarive 또는 absolute tick
	Mul	1
	타입	IntegerParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-
파라메터	이름	OsAlarmAutostartType
	설명	Alarm autostart의 타입
	Mul	1
	타입	EnumerationParamDef
	범위	ABSOLUTE RELATIVE
	설정	Pre-Compile time : All Variants
	비고	-
파라메터	이름	OsAlarmCycleTime
	설명	주기적인 alarm에 대한 cycle time
	Mul	1
	타입	IntegerParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-

파라메터	이름	OsAlarmAppModeRef
	설명	Application mode에 대한 참조
	Mul	1..*
	타입	Reference to OsAppMode
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-

컨테이너 이름	OsAlarmCallback	
설명	Callback 호출을 위한 파라메터들 포함	
서브 컨테이너	-	
파라메터	이름	OsAlarmCallbackName
	설명	Callback 함수명
	Mul	1
	타입	FunctionNameDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-

컨테이너 이름	OsAlarmIncrementCounter	
설명	Counter 증가를 위한 파라메터들 포함	
서브 컨테이너	-	
파라메터	이름	OsAlarmIncrementCounterRef
	설명	증가시킬 Counter에 대한 참조
	Mul	1
	타입	Reference to OsCounter
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-

컨테이너 이름	OsApplication
설명	Os objects(tasks, interrupts, alarms, hook등)들의 집합
서브 컨테이너	OsApplicationHooks: 1

	OsApplicationTrustedFunction: 0..*	
파라메터	이름	OsTrusted
	설명	Trusted 여부
	Mul	1
	타입	BooleanParamDef
	기본값	false
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU
파라메터	이름	OsAppAlarmRef
	설명	Os Application 에 속하는 OsAlarms
	Mul	0..*
	타입	Reference to OsAlarm
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU
파라메터	이름	OsApplsrRef
	설명	Os Application 에 속하는 Oslsr
	Mul	0..*
	타입	Reference to Oslsr
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU
파라메터	이름	OsAppResourceRef
	설명	Os Application 에 속하는 OsResource
	Mul	0..*
	타입	Reference to OsResource
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU

파라메터	이름	OsAppScheduleTableRef
	설명	Os Application 에 속하는 OsScheduleTable
	Mul	0..*
	타입	Reference to OsScheduleTable
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU
파라메터	이름	OsAppTaskRef
	설명	Os Application 에 속하는 OsTask
	Mul	0..*
	타입	Reference to OsTask
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU
파라메터	이름	OsRestartTask
	설명	Os Application 의 Restart Task
	Mul	0..1
	타입	Reference to OsTask
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU

컨테이너 이름	OsApplicationHooks
설명	OS-Application-specific hooks 의 구조
서브 컨테이너	-
파라메터	이름	OsAppErrorHook
	설명	OS-Application error hook 여부
	Mul	1
	타입	BooleanParamDef
	범위	-
	설정	Pre-Compile time : All Variants

	비고	scope: ECU
파라메터	이름	OsAppShutdownHook
	설명	OS-Application specific shutdown hook 여부
	Mul	1
	타입	BooleanParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU
파라메터	이름	OsAppStartupHook
	설명	OS-Application specific startup hook 여부
	Mul	1
	타입	BooleanParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU

컨테이너 이름	OsApplicationTrustedFunction	
설명	Trusted function 파라메터들의 구조	
서브 컨테이너	-	
파라메터	이름	OsTrustedFunctionName
	설명	다른 OsApplication에 가능한 trusted function
	Mul	1
	타입	FunctionNameDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU

컨테이너 이름	OsAppMode	
설명	OSEK OS application mode 에 대한 OSEK OS 속성 정의.	
서브 컨테이너	-	
파라메터	이름	OsAppModeId

	설명	AppMode의 Internal ID
	Mul	1
	타입	IntegerParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU

컨테이너 이름	OsCounter	
설명	OS Application의 카운터에 대한 정보 설정.	
서브 컨테이너	OsDriver: 0..1 OsTimeConstant: 0..*	
파라메터	이름	OsCounterMaxAllowedValue
	설명	시스템 카운터의 사용가능한 최대 tick
	Mul	1
	타입	IntegerParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-
파라메터	이름	OsCounterMinCycle
	설명	주기적인 alarm에 사용가능한 최소 tick
	Mul	1
	타입	IntegerParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU
파라메터	이름	OsAppModeId
	설명	AppMode의 Internal ID
	Mul	1
	타입	IntegerParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-

파라메터	이름	OsCounterTicksPerBase
	설명	OsCounterTicksPerBase 에 사용되는 tick
	Mul	1
	타입	IntegerParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-
파라메터	이름	OsCounterType
	설명	Counter의 타입
	Mul	1
	타입	EnumerationParamDef
	범위	HARDWARE SOFTWARE
	설정	Pre-Compile time : All Variants
	비고	scope: ECU
파라메터	이름	OsSecondsPerTick
	설명	초당 Tick
	Mul	0..1
	타입	FloatParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU
파라메터	이름	OsCounterAccessingApplication
	설명	접근 가능한 application에 대한 참조
	Mul	0..*
	타입	Reference to OsApplication
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-

컨테이너 이름	OsDriver	
설명	Counter를 동작할 정보 설정.	
서브 컨테이너	-	
파라메터	이름	OsGptChannelRef
	설명	GPT 채널에 대한 참조
	Mul	0..1
	타입	Reference to GptChannelConfiguration
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU

컨테이너 이름	OsEvent	
설명	OS Event 의 Representation	
서브 컨테이너	-	
파라메터	이름	OsEventMask
	설명	OIL에서 설정되면 무시
	Mul	0..1
	타입	IntegerParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-

컨테이너 이름	OsHooks	
설명	OS의 hooks 구조	
서브 컨테이너	-	
파라메터	이름	OsErrorHook
	설명	OSEK Error Hook 정의값
	Mul	1
	타입	BooleanParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-
파라메터	이름	OsPostTaskHook

	설명	OSEK Post-task Hook 정의값
	Mul	1
	타입	BooleanParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-
파라메터	이름	OsPreTaskHook
	설명	OSEK Pre-task Hook 정의값
	Mul	1
	타입	BooleanParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-
파라메터	이름	OsProtectionHook
	설명	Protection hook 사용여부
	Mul	1
	타입	BooleanParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU dependency: Required for scalability class 2,3 and 4
파라메터	이름	OsShutdownHook
	설명	OSEK Shutdown 정의값
	Mul	1
	타입	BooleanParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-
파라메터	이름	OsStartupHook
	설명	OSEK Startup Hook 정의값

	Mul	1
	타입	BooleanParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-

컨테이너 이름	OsIsr	
설명	OSEK interrupt service routine	
서브 컨테이너	OsIsrTimingProtection: 0..1	
파라메터	이름	OsIsrCategory
	설명	ISR Category
	Mul	1
	타입	EnumerationParamDef
	범위	CATEGORY_1 CATEGORY_2
	설정	Pre-Compile time : All Variants
	비고	-
파라메터	이름	OsIsrResourceRef
	설명	ISR에 의해 접근되는 Resource 참조
	Mul	0..*
	타입	Reference to OsResource
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-

컨테이너 이름	OsIsrResourceLock	
설명	OsIsr이 사용하는 Resource 사용시간	
서브 컨테이너	-	
파라메터	이름	OsIsrResourceLockBudget
	설명	인터럽트가 Resource에 대해 사용가능한 최대 시간
	Mul	1
	타입	FloatParamDef
	범위	-
	설정	Pre-Compile time : All Variants

	비고	scope: ECU dependency: Required for scalability class 2 and 4
파라메터	이름	OsIsrResourceLockResourceRef
	설명	Lock Resource에 대한 참조
	Mul	1
	타입	Reference to OsResource
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU dependency: Required for scalability class 2 and 4

컨테이너 이름		OsIsrTimingProtection
설명		Timing protection과 관련된 파라메터들 설정
서브 컨테이너		OsIsrResourceLock: 0..*
파라메터	이름	OsIsrAllInterruptLockBudget
	설명	모든 인터럽트를 lock할 수 있는 최대 시간
	Mul	0..1
	타입	FloatParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU dependency: Required for scalability class 2 and 4
파라메터	이름	OsIsrExecutionBudget
	설명	인터럽트 최대 실행 시간
	Mul	0..1
	타입	FloatParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU dependency: Required for scalability class 2 and 4
파라메터	이름	OsIsrOsInterruptLockBudget
	설명	모든 Category 2 인터럽트를 lock할 수 있는 최대 시간
	Mul	0..1
	타입	FloatParamDef
	범위	-

	설정	Pre-Compile time : All Variants
	비고	scope: ECU dependency: Required for scalability class 2 and 4
파라메터	이름	OsIsrTimeFrame
	설명	연속되는 인터럽트간의 minimum inter-arrival time
	Mul	0..1
	타입	FloatParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU dependency: Required for scalability class 2 and 4

컨테이너 이름		OsOs
설명		OSEK OS 속성 설정
서브 컨테이너		OsHooks: 1
파라메터	이름	OsScalabilityClass
	설명	Scalability class
	Mul	0..1
	타입	EnumerationParamDef
	범위	SC1 SC2 SC3 SC4
	설정	Pre-Compile time : All Variants
	비고	scope: ECU
파라메터	이름	OsStackMonitoring
	설명	Stack monitoring 여부
	Mul	1
	타입	BooleanParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU
파라메터	이름	OsStatus
	설명	System의 상태

	Mul	1
	타입	EnumerationParamDef
	범위	EXTENDED STANDARD
	설정	Pre-Compile time : All Variants
	비고	-
파라메터	이름	OsUseGetServiceId
	설명	OSEK 설정 값
	Mul	1
	타입	BooleanParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-
파라메터	이름	OsUseParameterAccess
	설명	OSEK 설정 값
	Mul	1
	타입	BooleanParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-
파라메터	이름	OsUseResScheduler
	설명	Application 내에서 RES_SCHEDULER 사용여부
	Mul	1
	타입	BooleanParamDef
	기본값	true
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-

컨테이너 이름	OsResourc
설명	OsResource object 속성 설정
서브 컨테이너	OsHooks: 1

파라메터	이름	OsResourceProperty
	설명	Resource 타입
	Mul	1
	타입	EnumerationParamDef
	범위	INTERNAL LINKED STANDARD
	설정	Pre-Compile time : All Variants
	비고	-
파라메터	이름	OsResourceAccessingApplication
	설명	접근가능한 application에 대한 참조
	Mul	0..*
	타입	Reference to OsApplication
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-
파라메터	이름	OsResourceLinkedResourceRef
	설명	The link to the resource
	Mul	0..1
	타입	Reference to OsResource
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-

컨테이너 이름	OsScheduleTable	
설명	알람의 동기화를 위한 설정	
서브 컨테이너	OsScheduleTableAutostart: 0..1 OsScheduleTableExpiryPoint: 1..* OsScheduleTableSync: 0..1	
파라메터	이름	OsScheduleTableDuration
	설명	the duration (modulus) of the schedule table
	Mul	1
	타입	IntegerParamDef

	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU
파라메터	이름	OsScheduleTableRepeating
	설명	ScheduleTable 처리 반복 여부
	Mul	1
	타입	BooleanParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU
파라메터	이름	OsSchTblAccessingApplication
	설명	접근가능한 Application에 대한 참조
	Mul	0..*
	타입	Reference to OsApplication
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU
파라메터	이름	OsScheduleTableCounterRef
	설명	Schedule table을 이용하는 Counter 에 대한 참조
	Mul	1
	타입	Reference to OsCounter
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU

컨테이너 이름	OsScheduleTableAutostart	
설명	Schedule table 자동실행에 대한 설정	
서브 컨테이너	-	
파라메터	이름	OsScheduleTableAbsValue
	설명	Absolute autostart tick
	Mul	1

	타입	IntegerParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU
파라메터	이름	OsScheduleTableAutostartType
	설명	자동실행 타입
	Mul	1
	타입	EnumerationParamDef
	범위	ABSOLUTE RELATIVE
	설정	Pre-Compile time : All Variants
	비고	-
파라메터	이름	OsScheduleTableRelOffset
	설명	Relative offset in ticks
	Mul	1
	타입	IntegerParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU
파라메터	이름	OsScheduleTableAppModeRef
	설명	자동실행시 Application mode 에 대한 참조
	Mul	1..*
	타입	Reference to OsAppMode
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU

컨테이너 이름		OsScheduleTableEventSetting
설명		Schedule table에 의한 Event
서브 컨테이너		-
파라메터	이름	OsScheduleTableSetEventRef

	설명	Action에 의해 설정될 Event에 대한 참조
	Mul	1
	타입	Reference to OsEvent
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-
파라메터	이름	OsScheduleTableSetEventTaskRef
	설명	-
	Mul	1
	타입	Reference to OsTask
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-

컨테이너 이름	OsScheduleTableExpiryPoint	
설명	OS 가 동작시킬 task 또는 event 에 대한 schedule table의 point	
서브 컨테이너	OsScheduleTableEventSetting: 0..* OsScheduleTableTaskActivation: 0..* OsScheduleTblAdjustableExpPoint: 0..1	
파라메터	이름	OsScheduleTblExpPointOffset
	설명	the expiry point
	Mul	1
	타입	IntegerParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-

컨테이너 이름	OsScheduleTableTaskActivation	
설명	schedule table에 의해 동작될 task 설정	
서브 컨테이너	-	
파라메터	이름	OsScheduleTableActivateTaskRef
	설명	schedule table에 의해 동작될 Task에 대한 참조
	Mul	1
	타입	Reference to OsTask

	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU

컨테이너 이름		OsScheduleTableSync
설명		조정가능한 expiry point 설정
서브 컨테이너		-
파라메터	이름	OsScheduleTableMaxAdvance
	설명	The maximum positive adjustment that can be made to the expiry point offset (in ticks).
	Mul	1
	타입	IntegerParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-
파라메터	이름	OsScheduleTableMaxRetard
	설명	The maximum negative adjustment that can be made to the expiry point offset (in ticks).
	Mul	1
	타입	IntegerParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-

컨테이너 이름		OsScheduleTableSync
설명		Schedule table의 동기화 파라메터에 대한 설정
서브 컨테이너		-
파라메터	이름	OsScheduleTblExplicitPrecision
	설명	동기화 타입이 explicit 일 경우 사용
	Mul	0..1
	타입	IntegerParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: System

파라메터	이름	OsScheduleTblSyncStrategy
	설명	동기화 방법
	Mul	1
	타입	EnumerationParamDef
	범위	EXPLICIT IMPLICIT NONE
	설정	Pre-Compile time : All Variants
	비고	scope: System

컨테이너 이름	OsTask
설명	OSEK Task 설정
서브 컨테이너	OsTaskAutostart: 0..1 OsTaskTimingProtection: 0..1

파라메터	이름	OsTaskActivation
	설명	요청 가능한 Task 수
	Mul	1
	타입	IntegerParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-

파라메터	이름	OsTaskPriority
	설명	Task의 우선순위
	Mul	1
	타입	IntegerParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-

파라메터	이름	OsTaskSchedule
	설명	Task의 preemptability 설정
	Mul	1
	타입	EnumerationParamDef
	범위	FULL

		NON
	설정	Pre-Compile time : All Variants
	비고	-
파라메터	이름	OsTaskAccessingApplication
	설명	접근가능한 Application에 대한 참조
	Mul	0..*
	타입	Reference to OsApplication
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-
파라메터	이름	OsTaskEventRef
	설명	Extended Task에 반응할 Event의 목록
	Mul	0..*
	타입	Reference to OsEvent
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-
파라메터	이름	OsTaskResourceRef
	설명	접근가능한 Resource 목록에 대한 참조
	Mul	0..*
	타입	Reference to OsResource
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-

컨테이너 이름	OsTaskAutostart
설명	Task 자동실행에 대한 설정
서브 컨테이너	-
파라메터	이름	OsTaskAppModeRef
	설명	Task가 동작할 시 Application Mode 에 대한 참조
	Mul	1..*

	타입	Reference to OsAppMode
	범위	-
	설정	Pre-Compile time : All Variants
	비고	-

컨테이너 이름		OsTaskResourceLock
설명		Resource를 반환하는데 걸리는 최대 시간 설정
서브 컨테이너		-
파라메터	이름	OsTaskResourceLockBudget
	설명	Task가 resource를 lock 할 수 있는 최대 시간
	Mul	1..*
	타입	FloatParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU dependency: Required for scalability class 2 and 4
파라메터	이름	OsTaskResourceLockResourceRef
	설명	Task에 의해 사용되는 Resource에 대한 참조
	Mul	1
	타입	Reference to OsResource
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU dependency: Required for scalability class 2 and 4

컨테이너 이름		OsTaskTimingProtection
설명		Task의 timing protection에 대한 파라메터 설정
서브 컨테이너		OsTaskResourceLock: 0..*
파라메터	이름	OsTaskAllInterruptLockBudget
	설명	모든 인터럽트를 Lock 할 수 있는 최대 시간
	Mul	0..1
	타입	FloatParamDef
	범위	-
	설정	Pre-Compile time : All Variants

	비고	scope: ECU dependency: Required for scalability class 2 and 4
파라메터	이름	OsTaskExecutionBudget
	설명	Task의 최대 실행 시간
	Mul	0..1
	타입	FloatParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU dependency: Required for scalability class 2 and 4
파라메터	이름	OsTaskOsInterruptLockBudget
	설명	Category 2 인터럽트를 Lock 할 수 있는 최대 시간
	Mul	0..1
	타입	FloatParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU dependency: Required for scalability class 2 and 4
파라메터	이름	OsTaskTimeFrame
	설명	Task의 동작과 끝 사이의 최소 inter-arrival time
	Mul	0..1
	타입	FloatParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU dependency: Required for scalability class 2 and 4

컨테이너 이름	OsTimeConstant
설명	사용자 정의 상수 설정
서브 컨테이너	-
파라메터	이름	OsConstName
	설명	사용자 정의 상수 명
	Mul	1
	타입	StringParamDef
	범위	-

	설정	Pre-Compile time : All Variants
	비고	scope: ECU
파라메터	이름	OsTimeValue
	설명	사용자 정의 시간
	Mul	1
	타입	StringParamDef
	범위	-
	설정	Pre-Compile time : All Variants
	비고	scope: ECU

(4) 개발 환경

개발 환경은 다음과 같으며 개발 OS는 Windows XP 이지만 순수 Java API 만을 사용하여 JVM이 설치될 수 있는 모든 플랫폼에서 동일하게 수행된다. 실행환경은 콘솔(Console)기반이며 필요한 경우 생성 도구 실행 시에 옵션을 사용하여 필요한 동작을 수행하도록 한다.

항목	환경	비고
OS	Windows XP	Operation System
SDK	Java Development Kit 1.6	Compiler
Library	JSP Parser JDK Library	Template Parser ECU Configuration Description Parser Compile Template and Run Compiled Template
IDE	Eclipse 3.5(GALILEO)	IDE

(5) 시스템 설계

ECU 핵심 미들웨어 코드 생성도구가 수행해야 하는 기능은 다음과 같다.

템플릿 파일 명세 언어 정의

- 템플릿 파일은 ECU 설정 명세에 정의된 모듈의 컨테이너 및 파라미터를 코드 및 헤더파일로 생성할 수 있도록 표현할 수 있어야 한다.
- 템플릿 파일은 ECU 설정 명세에 접근할 수 있는 인터페이스를 제공하여야 하며 제공된 인터페이스로 ECU 설정 명세를 해석할 수 있어야 한다.
- 템플릿 파일은 ECU 설정 명세의 버전 및 AUTOSAR 버전에 종속되지 않게 작성될 수 있어야 한다.

- OS, COM Driver, CAN Interface, COM, Pdu Router 모듈의 스펙에 명시된 파라메터를 분석하고 pre-compile time, link time, post-build time 별로 템플릿 파일을 작성한다.(OS 모듈은 pre-compile time만이 존재한다.)
- 템플릿 파일은 pre-compile time, link time, post-build time 별로 관리 및 생성되어야 한다.

템플릿 파일 파싱
- 템플릿 파일을 파싱한 후에는 자바 바이트 코드로 만들어져야 하며 만들어진 자바 바이트 코드가 실행되도록 인터페이스를 제공하여야 한다.
- 템플릿 파일 파싱의 대상인 모듈에 맞는 객체로 해당 모듈들을 파싱할 수 있어야 한다. 현재 OS 모듈을 제외한 COM Cluster 및 기타 모듈은 Common 파서로 파싱이 가능하지만 특화된 모듈 파싱을 위해서 새로운 모듈 파서를 만들어 제공할 수 도 있다.

ECU 설정 명세서 파싱
- AUTOSAR XML 스키마 버전 3.0 파일 및 ECU 설정 파라미터 정의(ECU Configuration Parameter Definition) 파일을 분석하여 ECU 설정 도구에서 만들어 낸 ECU 설정 명세를 파싱할 수 있어야 한다.
- 자바 기반으로 XML 버전에 독립적으로 ECU 설정 명세를 파싱할 수 있어야 한다.
- ECU 설정 명세의 모듈별로 파싱할 수 있어야 하며 파싱된 모듈의 컨테이너 별로 관리할 수 있는 인터페이스를 제공하여야 한다.
- 컨테이너의 파라메터 및 참조값들을 관리할 수 있어야 한다.

코드 파일 생성
- 파싱된 ECU 설정 명세를 바탕으로 해당하는 코드 및 헤더파일을 생성할 수 있어야 한다.

위에 명시된 기능을 바탕으로 한 시스템 다이어그램은 [그림 3-40]와 같다.

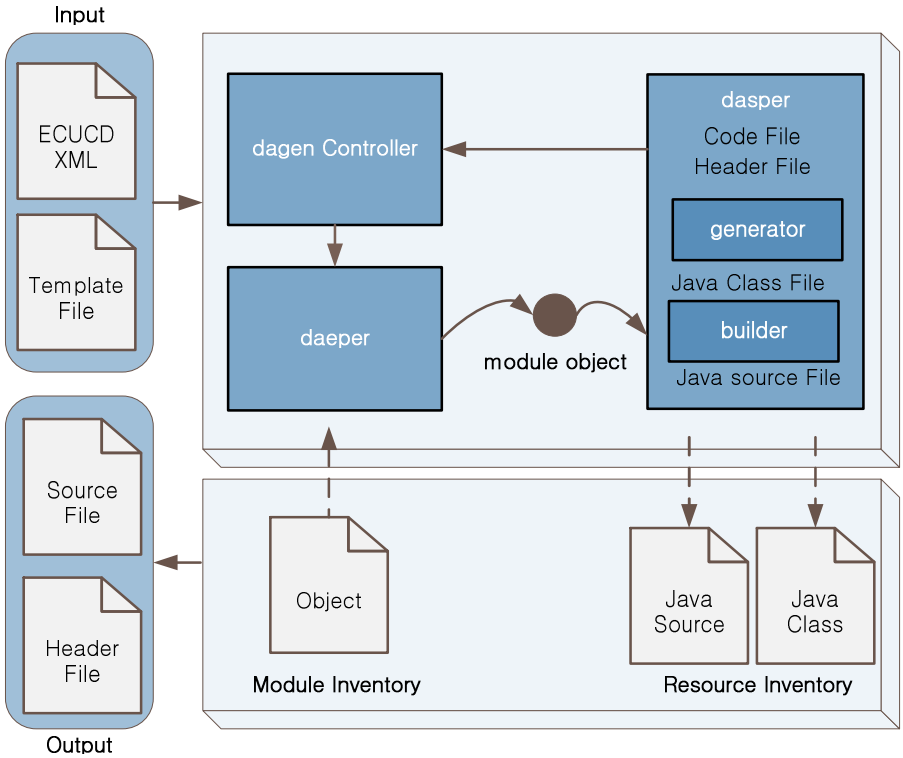
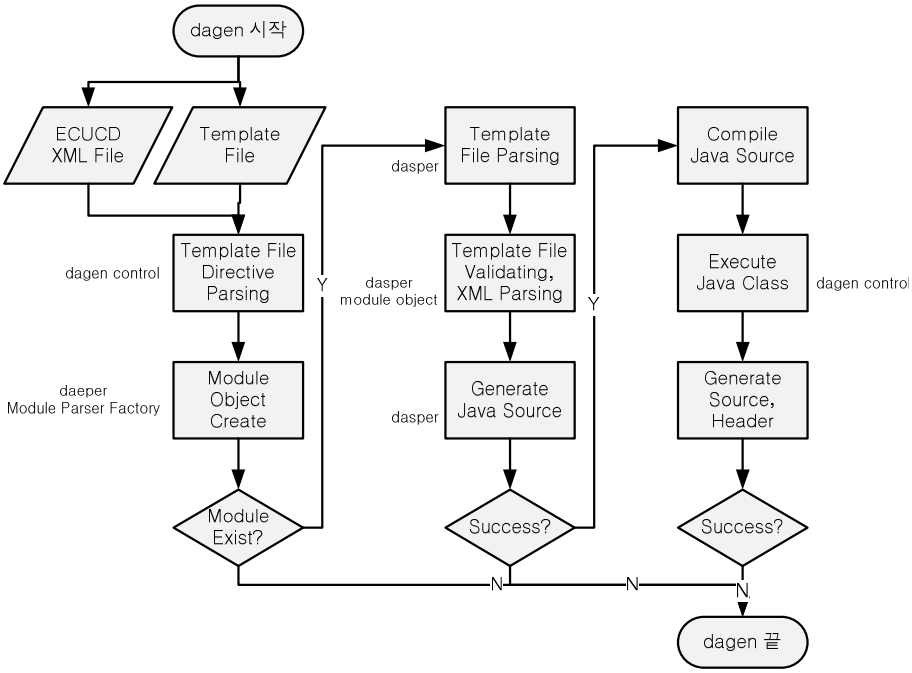


그림 3-40. 시스템 다이어그램

ECUCD(ECU 설정 명세)는 ECU 설정 도구에서 만들어지며 템플릿(Template File)은 ECUCD에 명시된 모듈을 pre-compile time, link time, post-build time별로 처리할 수 있도록 미리 기술되어 있어야 한다. dagen Controller는 각 모듈들을 처리할 수 있는 인터페이스 및 제어를 담당한다. daeper는 dagen Controller로부터 정보를 받아 ECUCD와 Template 파일의 헤더정보를 기반으로 모듈을 파싱하고 dasper에서 사용할 수 있는 module object를 생성한다. dasper는 dagen Controller로부터 정보를 받아 템플릿 파일의 문법 검사 및 module object와의 연동을 통해 자바 바이트 코드를 생성해 낸다. Builder Manager는 dasper가 생성시킨 자바 바이트 코드의 정보를 dagen Con troller에게 넘겨주고 dagen Controller는 해당하는 모듈의 코드를 생성해낸다.

각 모듈들의 처리 순서는 다음과 같다.



[그림 3-41]는 dagen 모듈의 패키지와 패키지간의 관계를 나타낸다.

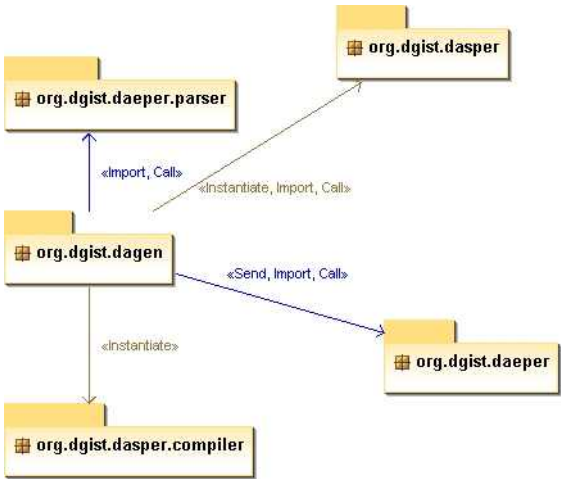


그림 3-41. dagen 모듈의 패키지 다이어그램

org.dgist.dagen패키지(A)와 org.dgist.daeper.parser패키지(B)의 인터페이스			
	데이터 종류	속성	
B → A	ECUCD 파싱 정보	목적	ECUCD의 정보와 파서를 전달
		인터페이스 유형	Import, Call

		데이터 형식	daeper의 parser 객체
		기타	

org.dgist.dagen패키지(A)와 org.dgist.dasper 패키지(B)의 인터페이스			
	데이터 종류	속성	
B → A	dasper 정보	목적	dasper의 정보를 전달과 제어
		인터페이스 유형	Instantiate, Import, Call
		데이터 형식	텍스트
		기타	

org.dgist.dagen패키지(A)와 org.dgist.daeper 패키지(B)의 인터페이스			
	데이터 종류	속성	
B → A	daeper 정보	목적	daeper의 정보를 전달과 제어
		인터페이스 유형	Instantiate
		데이터 형식	
		기타	

org.dgist.dagen패키지(A)와 org.dgist.dasper.compiler 패키지(B)의 인터페이스			
	데이터 종류	속성	
B → A	-	목적	dasper의 compiler 제어
		인터페이스 유형	Send, Import, Call
		데이터 형식	텍스트
		기타	

[그림 3-42]은 daeper 모듈의 패키지와 패키지간의 관계를 나타낸다.

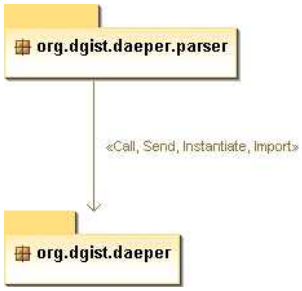


그림 3-42. daeper의 패키지 다이어그램

org.dgist.daeper 패키지(A)와 org.dgist.daeper.parser 패키지(B)의 인터페이스			
	데이터 종류	속성	
A → B	daeper정보 및 제어	목적	daeper 모듈의 제어와 정보 전달
		인터페이스 유형	Call, Send, Instantiate, Import
		데이터 형식	텍스트 및 객체
		기타	

[그림 3-43]은 dasper 모듈의 패키지 다이어그램이다.

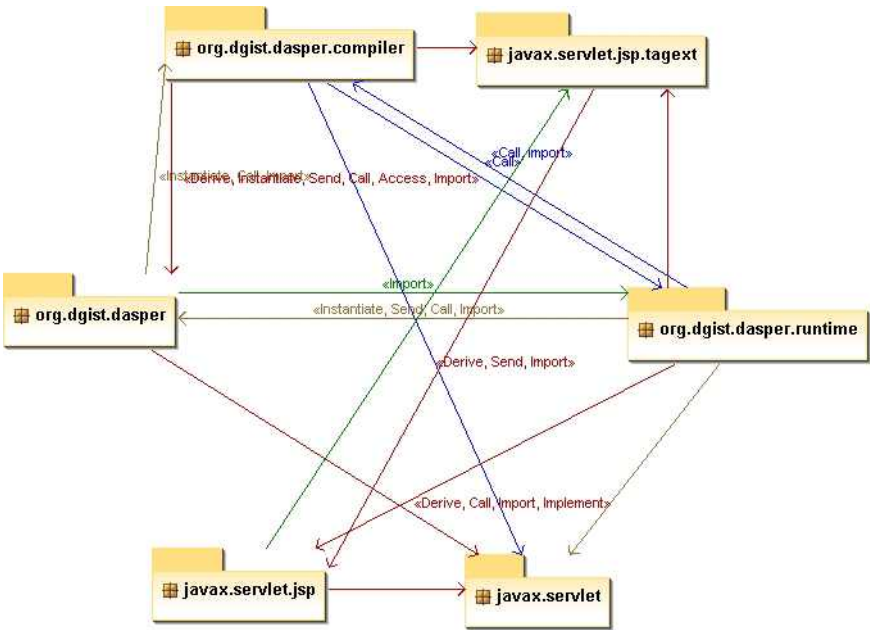


그림 3-43. dasper 모듈의 패키지 다이어그램

org.dgist.dasper패키지(A)와 org.dgist.dasper.compiler 패키지(B)의 인터페이스			
	데이터 종류	속성	
A ↔ B	템플릿 정보 및 객체	목적	템플릿 파싱 제어와 정보 전달
		인터페이스 유형	Derive, Send, Call, Instantiate, Access, Import
		데이터 형식	텍스트 및 객체
		기타	

org.dgist.dasper패키지(A)와 org.dgist.dasper.runtime 패키지(B)의 인터페이스			
	데이터 종류	속성	
A ↔ B	템플릿 파일을 위한 정보 및 객체	목적	템플릿 파일의 자바 코드 생성을 위한 정보 및 객체 전달과 제어
		인터페이스 유형	Send, Call, Instantiate, Import
		데이터 형식	텍스트 및 객체
		기타	

org.dgist.dasper패키지(A)와 javax.servlet 패키지(B)의 인터페이스			
	데이터 종류	속성	
B → A	템플릿 정보	목적	템플릿 파싱을 위한 정보 전달
		인터페이스 유형	Derive, Call, Instantiate, Import
		데이터 형식	텍스트 및 객체
		기타	

(6) 실행 결과

코드생성기 실행 및 결과를 보여주며 해당 소스 및 실행 파일은 본 완료보고서와 함께 CD로 제출한다. 소스파일은 개발 IDE인 eclipse의 workspace 폴더로 제출한다.

(가) 실행 방법

콘솔상에서 실행시키며 옵션은 다음과 같다

-CC : 템플릿의 해당하는 설정 클래스를 지정(ALL : 모든 설정 클래스, PC : Pre-Compile Time, L : Link Time, PB : Post-Build Time)

template_folder : 템플릿이 들어있는 폴더 지정(템플릿의 확장자는 btf 이며 해당 확장자와 일치하는 파일만 처리)

output_folder : 템플릿 파일로부터 생성된 코드가 저장되는 폴더 지정



그림 3-44. dagen 실행 옵션

(나) 실행 화면

실행 화면은 그림 3-45와 같으며 사용자가 지정한 폴더의 모든 템플릿 파일을 순차적으로 처리한다. 처리되는 템플릿 파일은 확장자는 btf(BSW Template File)이다. 템플릿 파일 처리는 제일 먼저 템플릿 파일의 헤더정보가 유효한지를 검사한 다음 템플릿에서 사용하는 모듈이 ECU 설정 명세서(Ecu Configuration Description)에 존재하는지 ConfigurationClass가 일치하는지 확인한다. 해당 모듈이 존재하지 않거나 ConfigurationClass가 일치하지 않으면 해당 템플릿 파일은 무시하고 다음 템플릿을 처리한다.

해당 템플릿 파일이 유효하면 파싱과 함께 컴파일을 실행하여 자바 코드를 생성하며 해당 코드를 컴파일하여 자바 바이트 코드를 생성한다. 그런 다음 ECU 설정 명세서의 정보를 가지고 있는 객체(모듈 파서)를 만들어 내고 이를 자바 바이트 코드와 함께 실행시켜 해당하는 모듈의 코드를 생성해낸다. 실행 중간에 생성되는 자바 코드와 바이트 코드는 BTFClass 라는 폴더에 자동으로 저장된다.

생성되는 코드는 설정 클래스(pre-compile time, link time, post-build time)별로, 소스 및 헤더파일별로 만들어진 템플릿 파일마다 하나씩 생성되며 생성된 코드 및 헤더 파일은 사용자가 지정한 해당 폴더에 생성된다.

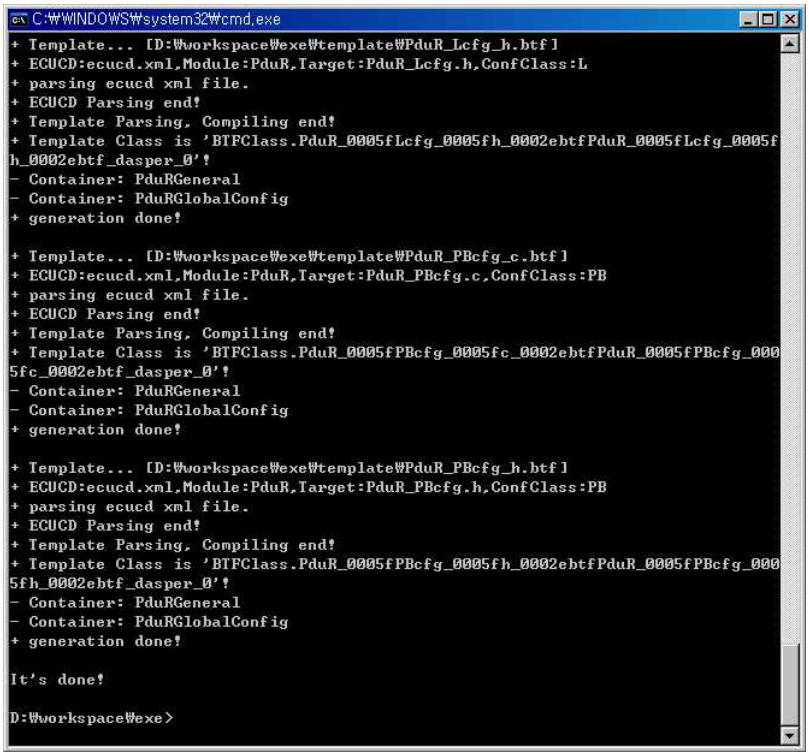


그림 3-45. ECU 핵심 미들웨어 코드 생성 도구 실행 화면

(다) 실행 결과물

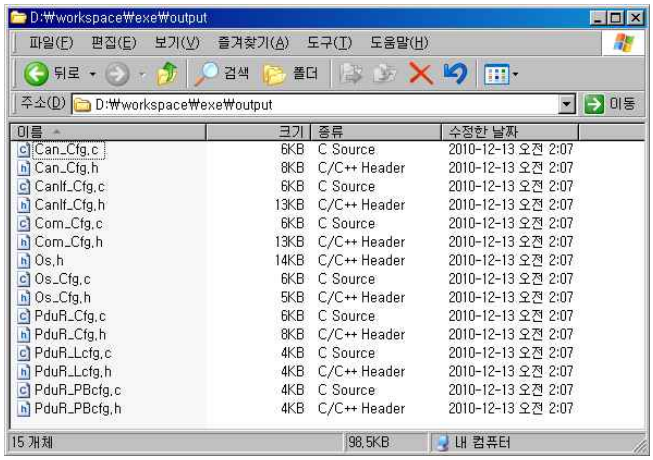


그림 3-46. Output 폴더의 내용

실행이 성공적으로 완료되면 [그림 3-46]과 같이 해당 폴더에 생성된 코드들

이 저장되며 중간 결과물로 만들어진 자바와 자바바이트 코드는 [그림 3-47]과 같이 BTFClass 폴더에 생성된다.

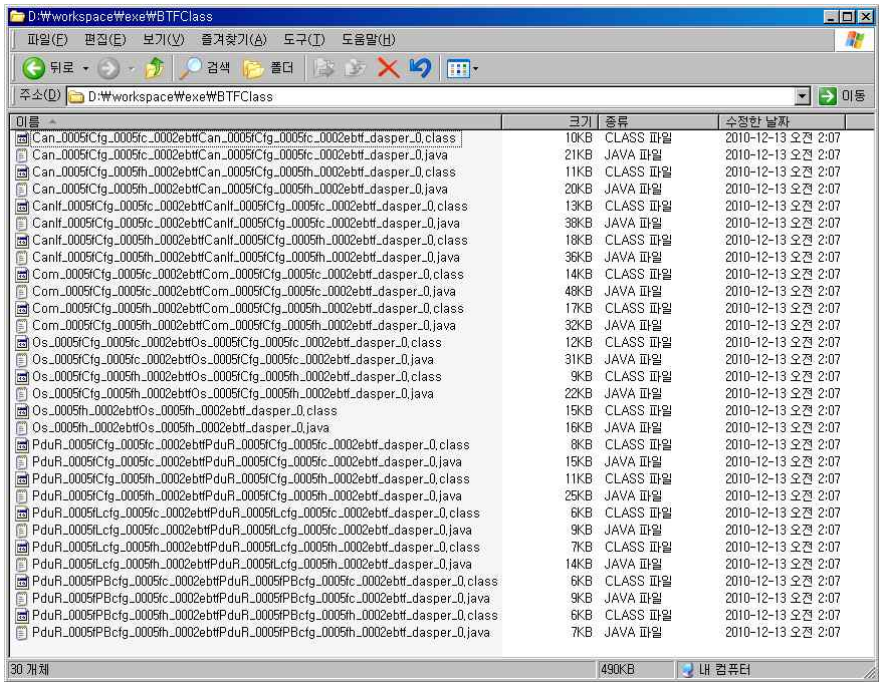


그림 3-47. BTFClass 폴더의 내용

[그림 3-47]에서 확장자가 java는 해당 템플릿로부터 생성된 자바 코드이며 확장자가 class인 파일은 해당 자바 코드를 컴파일을 통해 만들어진 자바 바이트 코드이다. 자바 바이트 코드 실행으로 코드 및 소스 파일을 생성해내게 된다.

3. 차량용 소프트웨어 플랫폼 기반 응용시스템 개발

가. AUTOSAR기반 지능형 인테리어 라이팅 시스템 개발

(1) 시스템 구성

AUTOSAR 소프트웨어 플랫폼을 적용한 지능형 인테리어 라이팅 시스템(Intelligent Interior Lighting System, IILS)의 블록 다이어그램은 [그림 3-48]과 다. 차량의 속도, 운영 모드 등 차량 내외부의 상태를 모니터링 하여 IILS ECU의 운용 상태를 결정하고, 스위치로부터 운전자 및 탑승자의 요구를 받아서 LED실내등을 제어한다. IILS ECU의 입력 및 출력 신호는 그림 59와 같다.

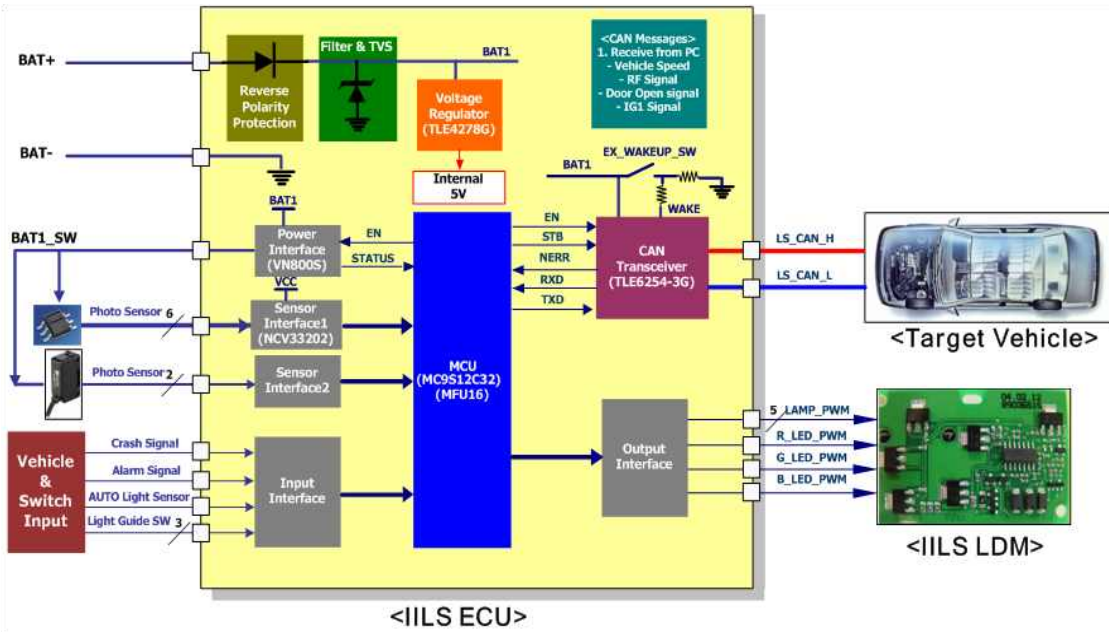


그림 3-48. IILS System Block Diagram

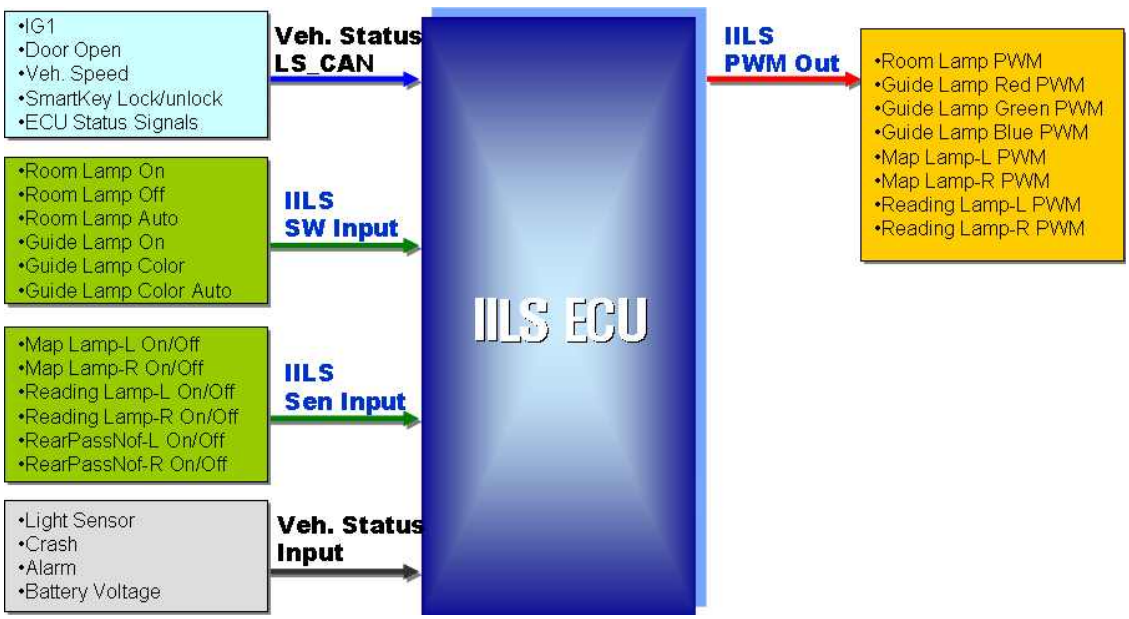


그림 3-49. IILS ECU Input/Output Diagram

스위치 입력 : Room Lamp, Guide Lamp 스위치

비접촉 센서 입력 : Map Lamp Left/Right 스위치,
Reading Lamp Left/Right 스위치,
뒷좌석 좌우 승객 감지 센서
차량 상태 입력(hardwired) : 외부 밝기 감지 센서, Crash 신호, Alarm신호
배터리 전압 신호
CAN Rx : IG1 신호, 도어 오픈 신호, 스마트키 lock/unlock 신호,
다른 ECU의 Normal/Sleep 상태 신호
LED 제어 출력(PWM) : Room Lamp, Guide Lamp Red/Green/Blue,
Map Lamp Left/Right, Reading Lamp Left/Right

(2) 소프트웨어 설계 및 시스템 통합

(가) IILS 적용 AUTOSAR 소프트웨어 아키텍처

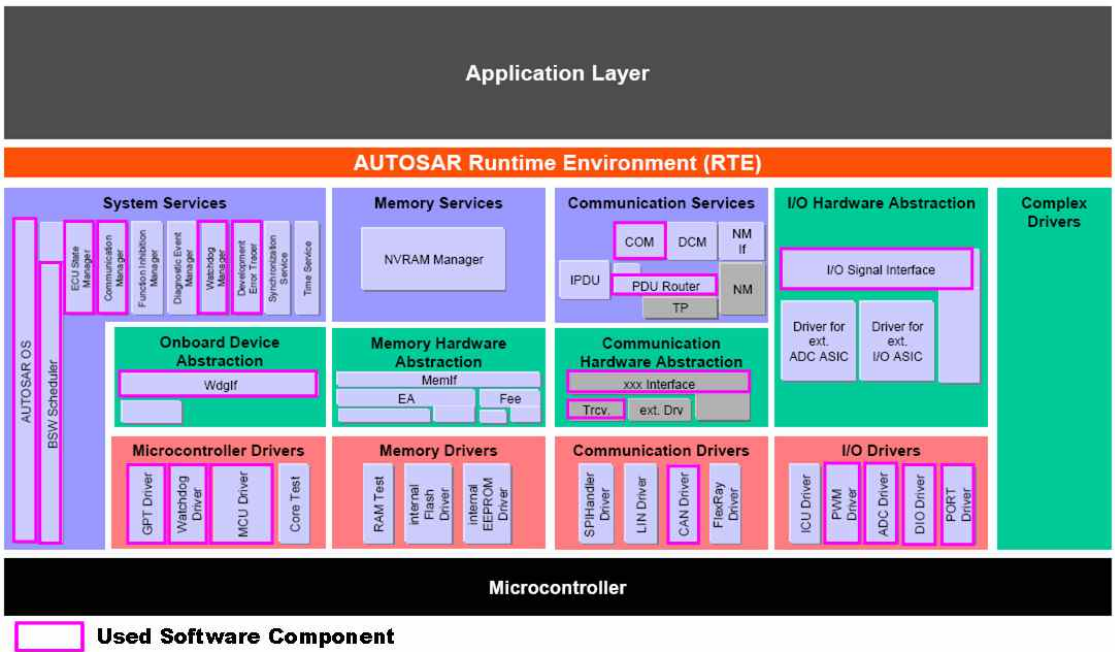


그림 3-50. AUTOSAR Software Components used in the IILS

□ IILS에 적용된 AUTOSAR 소프트웨어 컴포넌트

- RTE
- System Services
  - AUTOSAR OS, BSW Scheduler, ECU State Manager, Watchdog

Manager, Communication Manager, Development Error Tracer

- Onboard Device Abstraction
  - Watchdog Interface
- Microcontroller Driver
  - GPT Driver, Watchdog Driver, MCU Driver
- Communication
  - COM, PDU Router, CAN Interface, CAN Driver
- IoHwAb & IO Drivers
  - IO Hardware Abstraction, PORT Driver, DIO Driver, ADC Driver, PWM Driver

(나) IILS 소프트웨어 아키텍처 개요

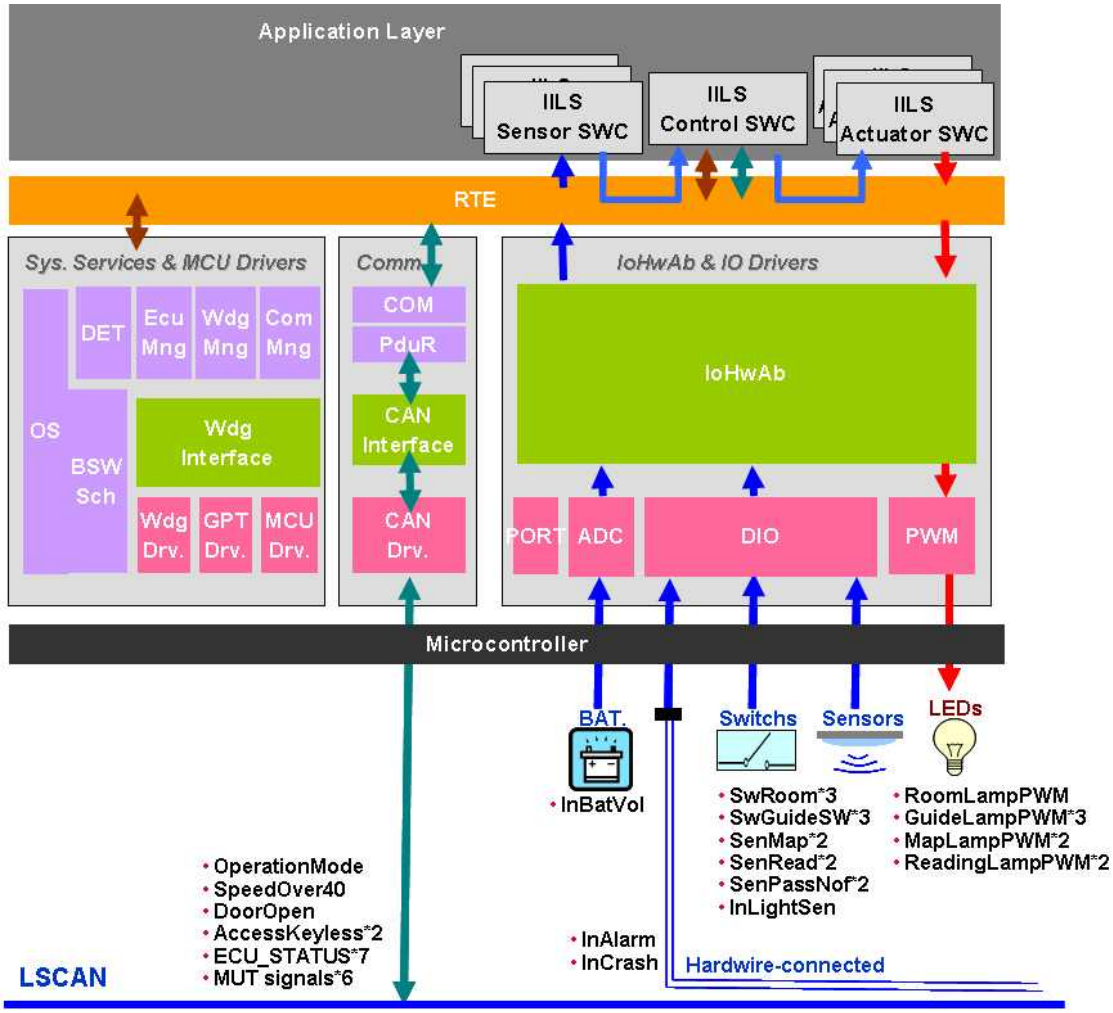


그림 3-51. Overview of IILS Software Architecture

1) Application Layer

- IILS Sensor SW-C
  - 센서로부터 ECU로 입력되는 전기적인 입력을 실제 물리적인 값으로 변환하기 위한 소프트웨어 컴포넌트
  - 시스템에 사용된 센서의 종류에 종속적임
  - Lamp 동작 스위치/센서 입력, 배터리 전압 입력 등을 처리
- IILS Actuator SW-C
  - ECU에서 액추에이터로 출력되는 실제 물리적인 값을 전기적인 출력으로 변환하기 위한 소프트웨어 컴포넌트
  - 시스템에 사용된 액추에이터의 종류에 종속적임

- Lamp 동작을 위한 PWM신호 출력을 처리

• IILS Control SW-C
  - IILS 제어를 위한 소프트웨어 컴포넌트
  - 센서 및 스위치, 차량 상태, ECU 상태 등의 입력을 바탕으로 Room Lamp, Guide Lamp, Map Lamp-L/R, Reading Lamp-L/R을 제어하여 출력 값을 Actuator SW-C로 송신한다.

2) RTE (Run Time Environment)
• ECU 내 SW-C간의 통신, ECU간의 네트워크 통신, Application SW-C와 BSW간의 데이터 교환을 위한 포트와 인터페이스 제공
• Sender/Receiver 인터페이스와 Client/Server 인터페이스 방식이 있음
• Runnable 실행을 위한 트리거 역할

3) IoHwAb & IO Drivers
• IO Hardware Abstraction
  - IO Driver의 추상화로 Application SW-C의 설계를 ECU에 독립적으로 해줌
  - Application SW-C와 마찬가지로 Runnable로 구성됨
  - 스위치 및 센서의 DIO 드라이버 입력, 배터리 전압의 ADC 드라이버 입력을 Sensor SW-C로 전달한다.
  - Actuator SW-C의 PWM 출력 신호를 받아서 PWM 드라이버에 전달한다.
  - 스위치 및 센서의 디바운스 등 입출력 신호의 핸들러를 포함
• PORT Driver
  - 마이크로 컨트롤러의 포트 핀의 입출력 방향 등 설정
  - 포트 핀의 기본 값 설정
• DIO Driver
  - 마이크로 컨트롤러의 포트 핀으로부터 디지털 신호를 입력 및 출력
  - Lamp 작동을 위한 스위치 및 센서 입력, 알람 및 크래쉬 입력
  - 시스템에 사용된 액추에이터의 종류에 종속적임
  - Lamp 동작을 위한 PWM신호 출력을 처리
• ADC Driver
  - 마이크로 컨트롤러 포트 핀의 아날로그 입력을 디지털 값으로 변환
  - 배터리 전압 아날로그 입력을 디지털 값으로 변환
• PWM Driver

- 펄스 폭 변조 신호를 출력으로 디지털 값을 아날로그 형태로 변환
- Lamp의 LED 점등을 위한 출력 신호를 생성

4) Communication

- AUTOSAR COM
  - RTE와의 데이터 교환을 위해 시그널 지향적인 인터페이스 제공
  - I-PDU와의 송수신을 위한 시그널의 패킹 및 언패킹
  - 시그널 모니터링, 송신 모드 설정 등
- PDU Router
  - AUTOSAR COM과 CAN Interface와의 인터페이스
  - 수신 및 송신 PDU의 라우팅
- CAN Interface
  - 송신 버퍼, 수신 통보, Bus off 통보
  - sleep, wakeup 등 모드 변경 등
- CAN Driver
  - CAN 컨트롤러 설정
  - CAN Bus와의 데이터 송수신

5) System Services

- AUTOSAR OS (Operating System)
  - 태스크 스케줄링, 인터럽트 핸들링 등
- BSW Scheduler
  - Basic Software 모듈의 메인 함수를 주기적으로 호출
- DET (Development Error Tracer)
  - 소프트웨어 디버깅을 위한 함수를 제공
  - 에러 발생 모듈, 함수 에러 유형 제공
- Com Manager
  - Communication request 매니지먼트
  - 버스 에러 매니지먼트
- ECU Manager
  - Basic Software 모듈의 초기화
  - Sleep, Wakeup 준비
- Watchdog Manager
  - Alive-supervision

- 하드웨어 와치독 트리거

(다) Application Layer

Sensor/Actuator SW-C, Control SW-C의 설계를 한다.
Application Layer의 아키텍처 설계는 Vector사의 DaVinci를 활용하였으며, Control SW-C의 설계는 Matlab/Simulink를 활용하였다.

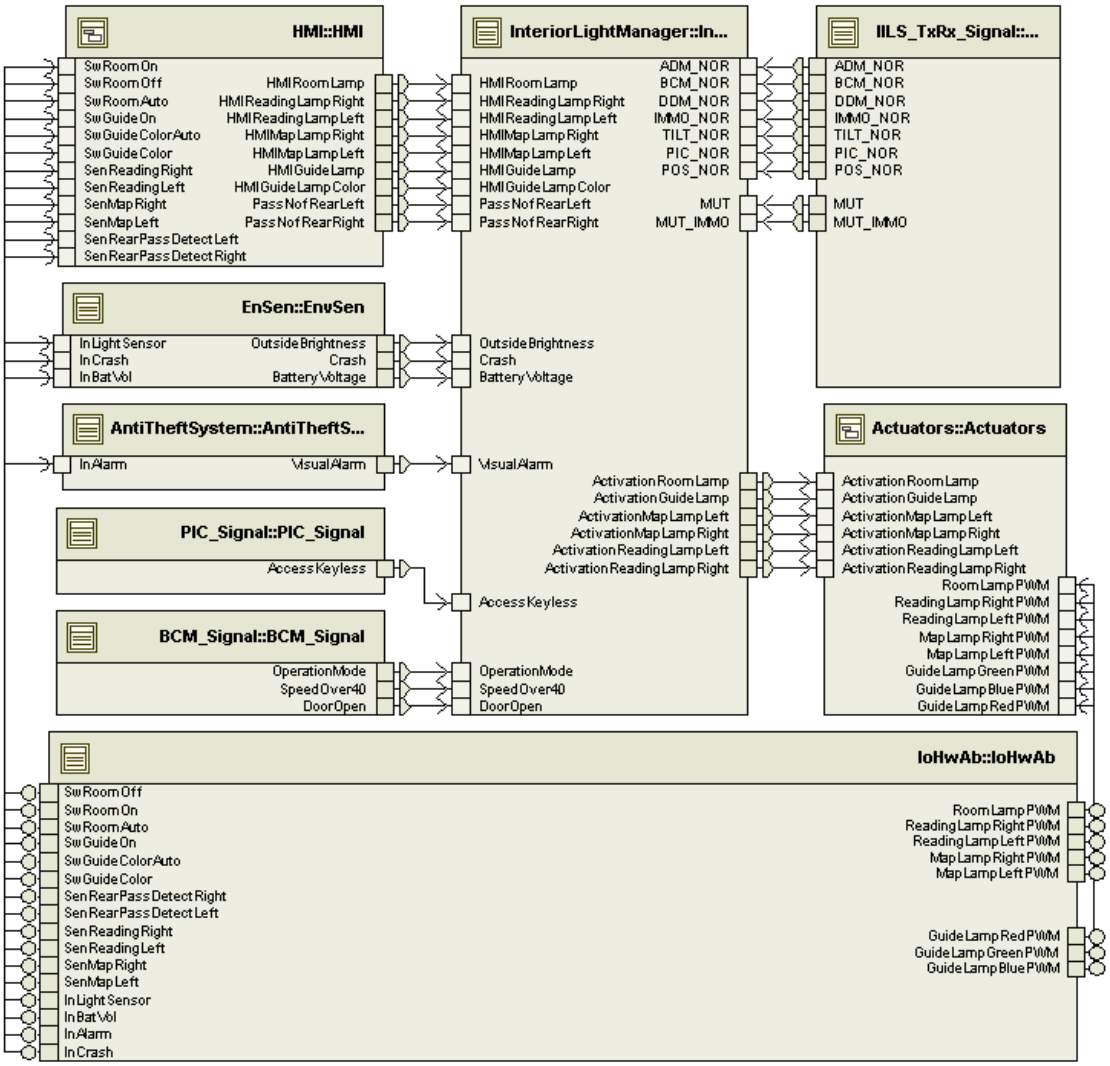


그림 3-52. IILS System Architecture

1) IILS System Architecture

□ IILS 시스템 소프트웨어 컴포넌트 구성

- Sensor SW-C : HMI, EnvSen
- Actuator SW-C : Actuators
- Application SW-C : InteriorLightManager
- Dummy SW-C for Network Signal : PIC_Signal, BCM_Signal, IILSTxRx
- IoHwAb SW-C : IoHwAb

□ 소프트웨어 컴포넌트 설명

- HMI SW-C
  - Composite 소프트웨어 컴포넌트
  - 8개의 Atomic 소프트웨어 컴포넌트로 구성
    - 램프 작동 스위치 및 센서 입력을 처리
- EnvSen SW-C
  - 차량 내외부의 상태를 입력 받음
  - 차량 전압, 외부 밝기, 충돌에 관한 입력
- AntiTheftSystem SW-C
  - 알람시스템으로 부터의 입력
  - 알람 상태를 입력
- Actuators SW-C
  - Composite 소프트웨어 컴포넌트
  - 6개의 Atomic 소프트웨어 컴포넌트로 구성
  - 각 램프의 LED 동작을 위한 출력을 처리
- InteriorLightManager SW-C
  - 메인 제어 알고리즘을 실행
- PIC_Signal SW-C
  - PIC ECU로 부터의 네트워크 신호 전송을 위한 더미 컴포넌트
  - 스마트키에 관한 입력, 차량 잠금 및 잠금 해제
- BCM_Signal SW-C
  - BCM ECU로 부터의 네트워크 신호 전송을 위한 더미 컴포넌트
  - 이그니션 키, 도어 열림, 속도에 관한 상태 입력
- IILS_TxRx_Signal SW-C
  - 네트워크 신호 전송을 위한 더미 컴포넌트

- 다른 ECU의 Sleep/Wakeup 상태, MUT 상태 입력

2) SW-C Description

가) HMI SW-C (Composite SW-C)

- HMI SW-C는 8개의 Sensor SW-C로 구성된다.
- HMI SW-C는 IILS 시스템 작동을 위한 스위치 및 센서 사용자 입력을 위한 SW-C이다
- 각 소프트웨어 컴포넌트의 설계는 다음과 같다
  - 각 소프트웨어 컴포넌트의 Runnable 구성
  - 각 Runnable의 입력 및 출력 포트
  - 각 Runnable의 실행 트리거 조건
- SW-C 설계
  - Trigger mode : Cyclic, 10ms
  - 스위치 및 센서 입력을 받아서 각 Lamp의 동작 요구조건을 판단

나) EnvSen SW-C

☐ InBatVol_RE

- Trigger mode : Cyclic, 10ms
- Description
  - 배터리 전압 입력을 받아서 배터리 상태를 판단한다.

☐ InCrash_RE

- Trigger mode : Cyclic, 10ms
- Description
  - 알람 상태를 판단한다.

☐ InLightSensor_RE

- Trigger mode : Cyclic, 10ms
- Description
  - 조도센서 입력으로 낮과 밤을 구분한다.

다) AntiTheftSystem

☐ AntiTheftSystem_RE

- Trigger mode : Cyclic, 10ms
- Description
  - 차량 도난 알람 상태 판단한다.

라) InteriorLightManager

☐ ControlSystemMode_RE

- Trigger mode : Cyclic, 10ms
- Description
  - 차량 상태, 네트워크 상태를 바탕으로 IILS 시스템의 운용 상태를 판단한다.

☐ ControlRoomLamp_RE

- Trigger mode : Cyclic, 10ms
- Description
  - IILS 시스템의 운용 상태, 차량 상태, RoomLamp 동작 요구조건으로 점등을 판단한다.

☐ ControlGuideLamp_RE

- Trigger mode : Cyclic, 10ms
- Description
  - IILS 시스템의 운용 상태, GuideLamp 동작 요구조건으로 점등 및 색상을 판단한다.

☐ ControlMapLampLeft_RE

- Trigger mode : Cyclic, 10ms
- Description
  - IILS 시스템의 운용 상태, 차량 속도, MapLampLeft 동작 요구조건으로 점등을 판단한다.

☐ ControlMapLampRight_RE

- Trigger mode : Cyclic, 10ms
- Description
  - IILS 시스템의 운용 상태, MapLampRight 동작 요구조건으로 점등을 판단한다.

☐ ControlReadingLampLeft_RE

- Trigger mode : Cyclic, 10ms
- Description
  - IILS 시스템의 운용 상태, 승객 유무, ReadingLampLeft 동작 요구조건으로 점등을 판단한다.

☐ ControlReadingLampRight_RE

- Trigger mode : Cyclic, 10ms

- Description
  - IILS 시스템의 운용 상태, 승객 유무, ReadingLampRight 동작 요구조건으로 점등을 판단한다.

마) Actuators SW-C

- Actuators SW-C는 6개의 Sensor SW-C로 구성된다
- Actuators SW-C는 IILS 램프 작동을 위한 LED 출력을 담당한다.

□ ActLamp*_RE

- Trigger mode : Cyclic, 10ms
- Description
  - 각 램프의 LED 신호를 출력한다.

바) IoHwAb SW-C

- Sensor/Actuator SW-C와 Client/Server 방식으로 통신한다.
- 모든 Runnable의 트리거 조건은 Operation Invocation이다.
- ECU 입출력 신호에 대해서 IO Driver로부터 입출력 값을 전기적인 값으로 변환한다.
- 디바운스 등의 핸들러를 포함한다.

사) PIC_Signal, BCM_Signal and IILS_TxRx_Signal SW-C

- IILS 시스템의 네트워크 수신 메시지 생성을 위한 더미 소프트웨어 컴포넌트이다.

(라) Iehavior Model

- Sensor/Actuator SW-C 및 IoHwAb SW-C는 핸드 코딩함
- InteriorLightManager의 기능은 Matlab/Simulink 기반 모델링으로부터 코드를 생성하
  였다.

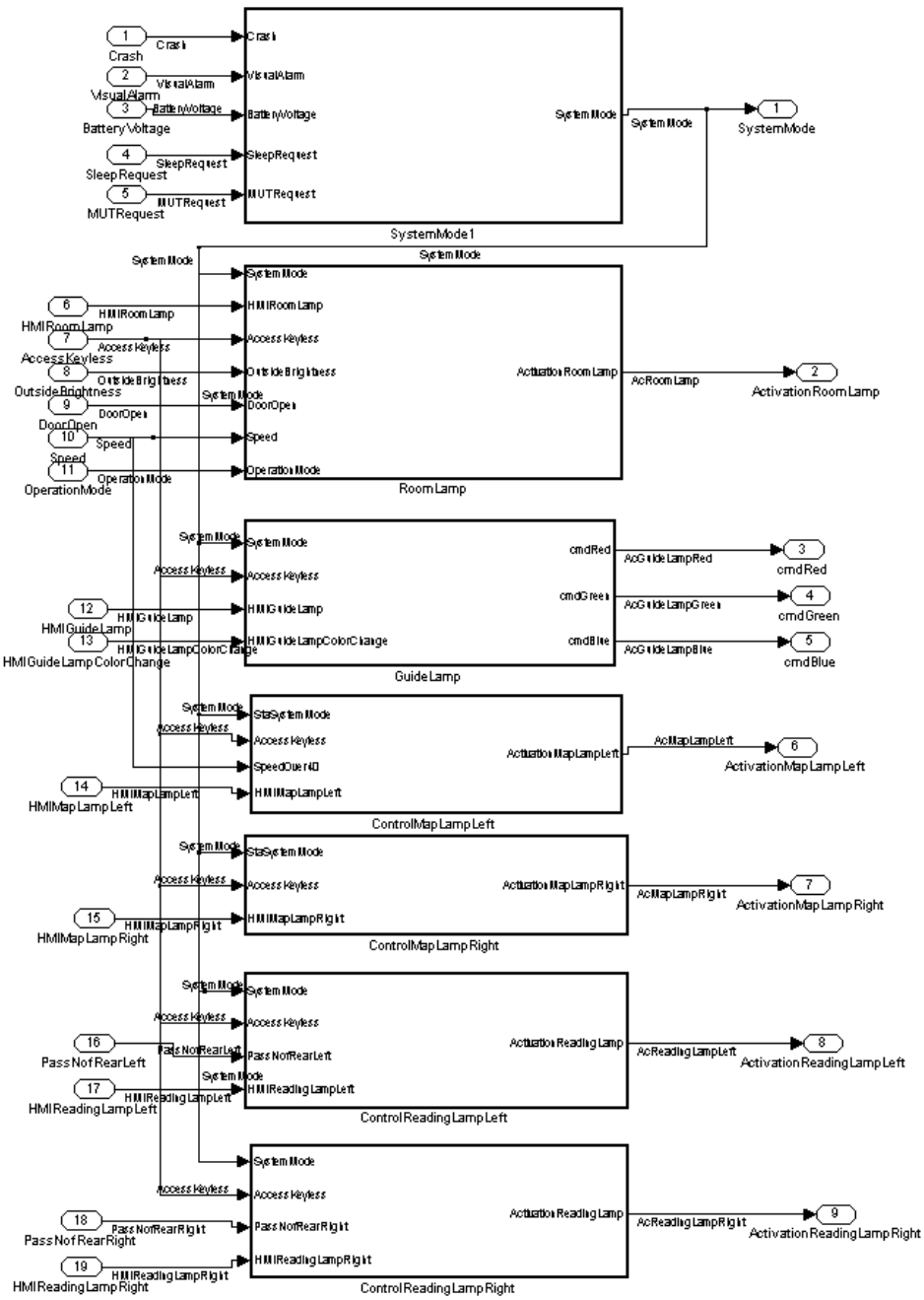


그림 3-53. Simulink Modeling of IILS

- Simulink Model은 크게 7개의 Subsystem으로 구성되어 있으며 각 Subsystem은 다시 내부에 기능별로 Stateflow Chart 및 더 작은 단위의 Subsystem으로 분리되어 있다.

- 각 Subsystem의 내부 구성은 다음과 같다.

1) Control System Mode Subsystem
2) Control Room Lamp Subsystem
3) Control Guide Lamp Subsystem
4) Control Map Lamp Left Subsystem
5) Control Map Lamp Right Subsystem
6) Control Reading Lamp Left Subsystem
7) Control Reading Lamp Right Subsystem

File: D:\10_Projects\06_IILS\05_IILS_AUTOSAR\Matlab\20_IntegratedModel\IILS_Ko_Integration_ert_rtw\ControlGuideLamp.c

```
1   /*
2    * File: ControlGuideLamp.c
3    *
4    * Real-Time Workshop code generated for Simulink model IILS_Ko_Integration.
5    *
6    * Model version                        : 1.62
7    * Real-Time Workshop file version      : 7.0  (R2007b)  02-Aug-2007
8    * Real-Time Workshop file generated on : Mon Mar 17 13:23:03 2008
9    * TLC version                         : 7.0 (Jul 26 2007)
10   * C source code generated on          : Mon Mar 17 13:23:06 2008
11   */
12
13  #include "ControlGuideLamp.h"
14
15  /* Include model header file for global data */
16  #include "IILS_Ko_Integration.h"
17  #include "IILS_Ko_Integration_private.h"
18
19  /* Named constants for block: '<S19>/Delay' */
20  #define IILS_Ko_Integrat_IN_ChangeDelay (2)
21  #define IILS_Ko_Integratio_IN_Dimming_j (3)
22  #define IILS_Ko_Integration_IN_Change  (1)
23
24  /* Named constants for block: '<S20>/ChangeColor' */
25  #define IILS_Ko_Integration_IN_Color5  (5)
26  #define IILS_Ko_Integration_IN_Color1  (1)
```

그림 3-54. Auto-Generated Code From Matlab/Simulink

- Matlab/Simulink 모델로부터 코드를 생성하여 소프트웨어 컴포넌트에 삽입하였다.

(마) Task 설계

1) Task_IoHwAb_Get
- Priority : 10
- Description : IO 드라이버로부터 입력 값을 읽는다.

2) Task_Sensor
- Priority : 9

- Description : 센서의 전기적인 신호를 물리적인 신호로 변환한다

3) Task_Control
- Priority : 8
- Description : 각 Lamp의 메인 제어 알고리즘을 실행한다.

4) Task_Actuator
- Priority : 7
- Description : Lamp의 LED 구동 전압을 출력한다.

5) Task_IoHwAb_Set
- Priority : 6
- Description : Lamp의 LED 구동 신호를 PWM 드라이버로 출력한다.

(바) Basic Software 설계

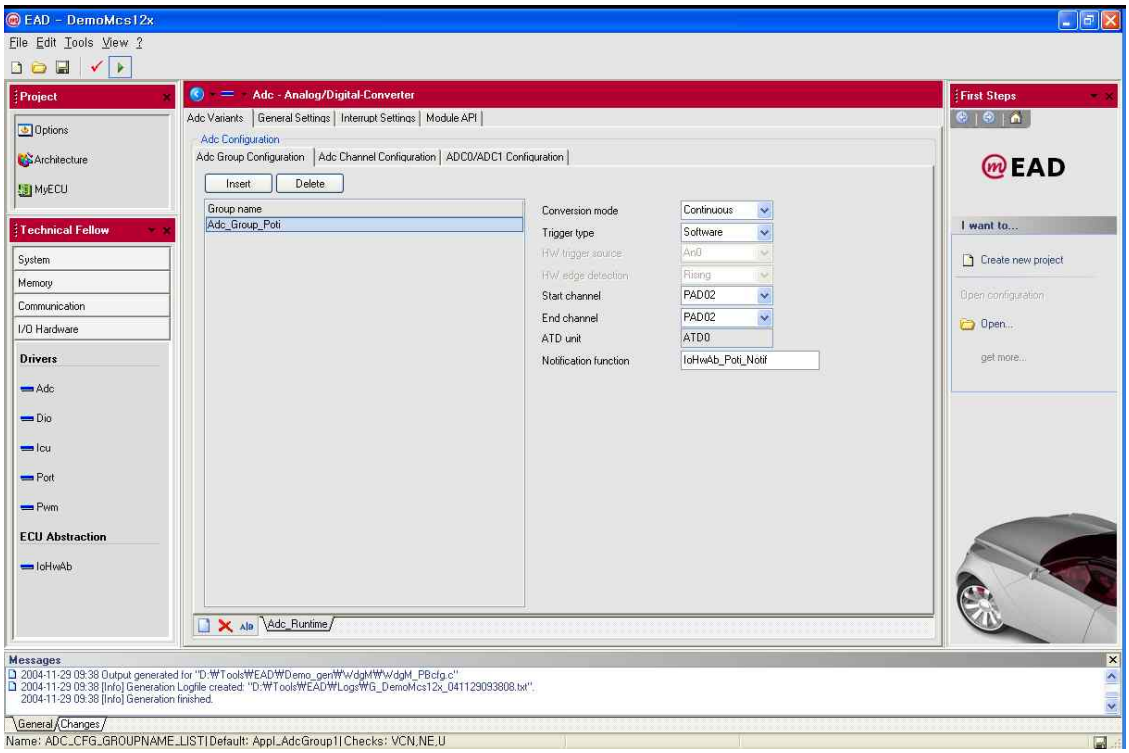


그림 3-55. Basic Software Configuration using MICOSAR EAD

Basic 소프트웨어는 MICROSAR EAD 툴을 활용하여 코드를 생성하였다. 입출력 신호를 구동하기 위해 사용하는 베이직 소프트웨어는 다음과 같다.

1) Microcontroller Abstraction Layer

- Port 드라이버

GPIO, ADC, SPI, PWM 등의 목적으로 사용되는 디지털 입출력 포트, 핀의 전반적인 구성설정과 포트초기화를 담당한다.

- DIO 드라이버

DIO 드라이버는 포트 드라이버에서 설정한 포트, 핀 정보에 의거하여 포트, 핀에 데이터를 읽고 쓰는 모든 작업을 수행한다. IILS에서는 센서/스위치 입력 값(ON/OFF)이 DIO 드라이버에 의해서 구현된다.

- ADC 드라이버

아날로그 - 디지털 변환기는 주로 아날로그 센서 입력 등을 읽어 들이기 위한 장치 드라이버로써 AUTOSAR BSW규격에 맞도록 모드를 정하고 명확한 인터페이스와 API를 사용하여 구동할 수 있다.

- PWM 드라이버

원하는 주파수와 펄스폭으로 주로 모터 등을 구동하기 위하여 구형파를 발생시킨다. IILS에서 1KHz의 PWM 펄스폭제어를 통해 LED를 구동한다.

2) ECU Abstraction Layer

- IO Hardware Abstraction

I/O 하드웨어 추상화 계층은 마이크로컨트롤러 드라이버 계층과 RTE 즉 미들웨어 간의 인터페이스 역할을 한다. I/O 하드웨어 추상화 계층에서는 외부 디바이스와 주변 디바이스의 하드웨어 연결에 관계없이 사용되어지며 데이터의 변환을 위한 API와 인터페이스를 제공함으로서 애플리케이션에게 추상화된 데이터를 제공한다.

다음과 같이 구현하는 드라이버와 ECU 신호에 따라 사용하는 인터페이스가 달라진다.

- ECU discrete 신호의 구현을 위해서는 DIO 드라이버와 PORT 드라이버를 사용해야 한다.
- ECU 아날로그 신호의 구현을 위해서는 최소한 ADC 드라이버를 사용해야 한다.
- ECU PWM 신호의 구현을 위해서는 최소한 PWM 드라이버와 ICU 드라이버를 사용해야 한다.

IO Hardware Abstraction에 대해 아직 EAD에서 많은 부분이 개발자가 직접 API의 내용을 프로그래밍 해야 한다.

(사) 소프트웨어 Integration and Build

현재 AUTOSAR 도구 체인에서는 소프트웨어 Integration과 Build 기능은 지원하지 않는다. 따라서 시스템 모델링의 마지막 과정인 DaVinci Developer에서 생성된 코드를 Real-Target에 맞게 수정하여 Integration하고 Build하는 과정이 필요하다.

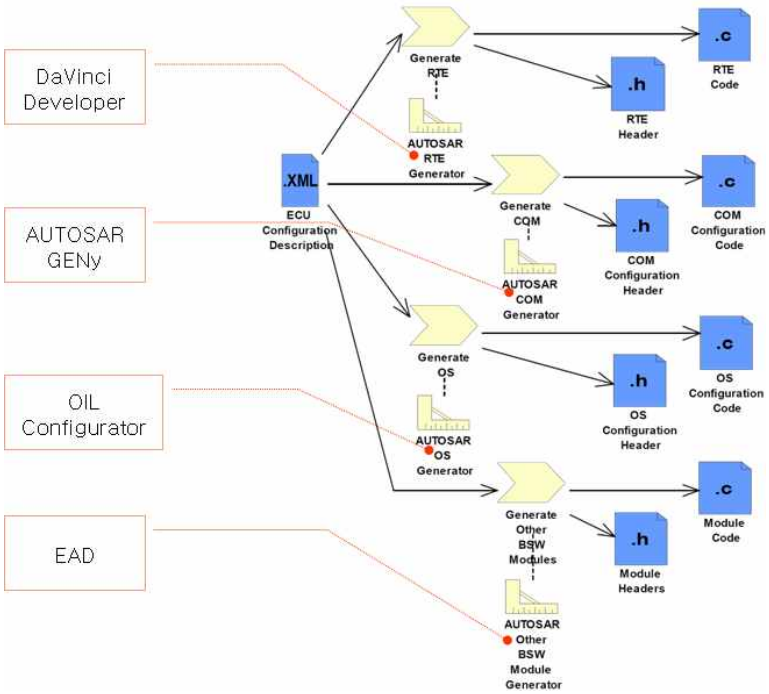


그림 3-56. AUTOSAR Methodology for Integration

위 그림에서 시스템 모델링을 통해 생성된 애플리케이션 소프트웨어 컴포넌트 코드들, Developer에서 생성한 RTE 코드들과 AUTOSAR GENy를 이용하여 생성한 통신관련 코드들, PC 타겟으로 생성된 OIL파일에서 나오는 코드, 마지막으로 베이직 소프트웨어 구성을 위한 EAD를 통해 생성된 코드를 통합하여 Code Warrior 환경에서 디버깅한다.
Integration이 완료되면 타켓 ECU에 다운로딩하고 OS 동작 및 애플리케이션에서 구동되는 TASK들이 제대로 동작하는 과정이 필요하다.

- 에러 없으면 다운로딩하고 osSystemCounter증가하는지 확인
- 만일 에러 발생시 Det에러와 ErrorHook 함수의 에러ID확인
- Rte.c열어서 각 Runnable에서 Brake가 걸리는지 확인
- 사용자 정의된 인터럽트가 제대로 동작하는지 확인

(아) Target ECU Porting

ECU 보드에서 입출력 신호 확인을 확인하기 전에 Driver API를 사용하여 ADC, DIO, PWM 기본 구동이 IO Hardware Abstraction을 통해 이루어지는지 확인한다. 다음과 같이 라이브러리 함수로 정의된 Driver API를 직접 구동하여 Configuration값과 일치하는지 하드웨어적으로 검증한다.

Pwm_SetDutyCycle(PWM_ReadRight, IoReadingLampRightPWM); // PWM API
마찬가지로 다른 IO관련 드라이버들에 대해서도 각각 입출력 신호가 설정된 정보에 맞게 구동되고 있는지 확인한다.

Dio_WriteChannel(DIO_SenPowIn, IoSenPowIn);

필요에 따라 Debouncing, Edge detection 부분까지 추가하여 다음과 같이 입출력 신호를 보상한다.

```
IoRoomOnDIO = !(Dio_ReadChannel(DIO_RoomOn));  // DIO API
// IoHwAb Handler
IoRoomOn = H_Debounce (IoRoomOnDIO,1);  // debounce
IoRoomOn = H_RisingEdge(IoRoomOn,1);  // rising edge
// Write to RTE
*signal = IoRoomOn;
```

각 드라이버는 Application SW-C단에서 Sender-Receiver 방식에 의해 호출되고 구동된다. 디버거에서 기본 입출력 신호가 확인되면 입출력 외부장치들(입력모듈, LED등) 을 부착하고 제어로직에 따라 순차적으로 동작하는지 확인하는 절차를 걸쳐 애플리케이션 개발을 1차 완료한다.

(3) AUTOSAR 기반 IILS 프로토타입 테스트 및 시연

(가) Software 테스트

1) Code Generation from DaVinci

DaVinci Developer의 Generation Parameters 탭에서 CANoe 시뮬레이션을 위한 코드를 생성하기 위해서는 다음과 같이 설정해야 한다.

- 코드 생성 타겟 : PC-Target으로 설정
- Target Generation Steps :

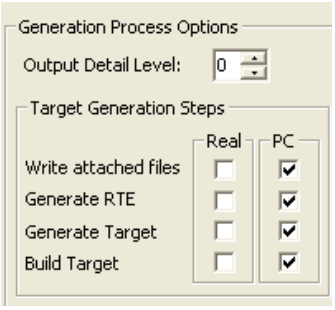


그림 3-57. Target Generatio

Step 설정

- PC-Target 설정 :

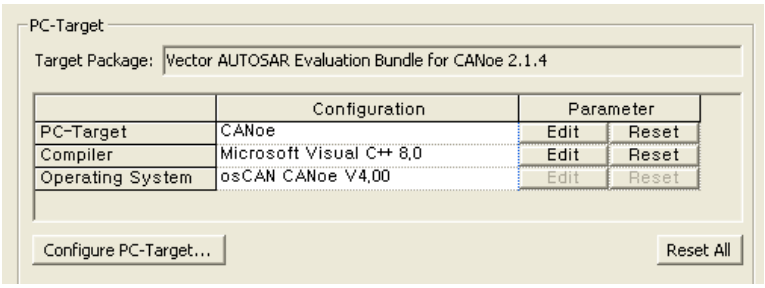


그림 3-58. PC-Target 설정

위와 같이 파라미터를 설정하고 코드를 생성하게 되면 CANoe에서 사용할수 있는 DLL 파일과 DBC 파일이 생성된다.

2) CANoe Simulation 설정

- CANoe Project 생성

새로운 CANoe project를 생성하고, DBC 파일을 import시키고, IILS ECU 노드를 생성하여 DLL 파일을 첨부한다. Simulation을 위한 입력 값과 결과 값을 보기 위해서 CANoe 환경변수를 활용하며, 환경변수가 저장된 DBC 파일을 import 시킨다.

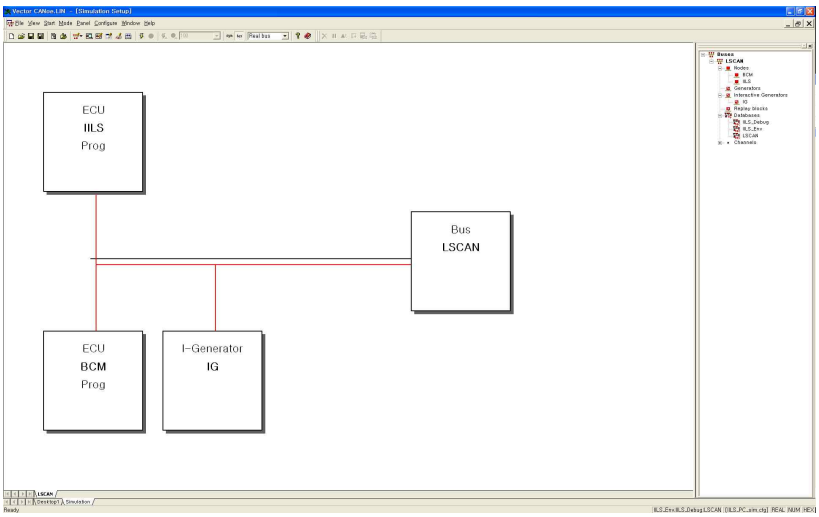


그림 3-59. CANoe Simulation 화면

환경변수를 CANoe에서 읽고 쓰기위해서는 다음과 같은 API를 설정해야 한다.

- 쓰기 : CANoeAPI_PutValueInt("VccEn", IoVccEn);
- 읽기 : CrashDIO = CANoeAPI_GetValueInt("CrashDIO");

입력값과 출력값을 확인하기 위해서 Simulation Panel을 생성해야 한다.
앞서 생성한 환경변수를 Simulation Panel의 스위치, 버튼등과 연결을 위한 설정을 해야 한다.

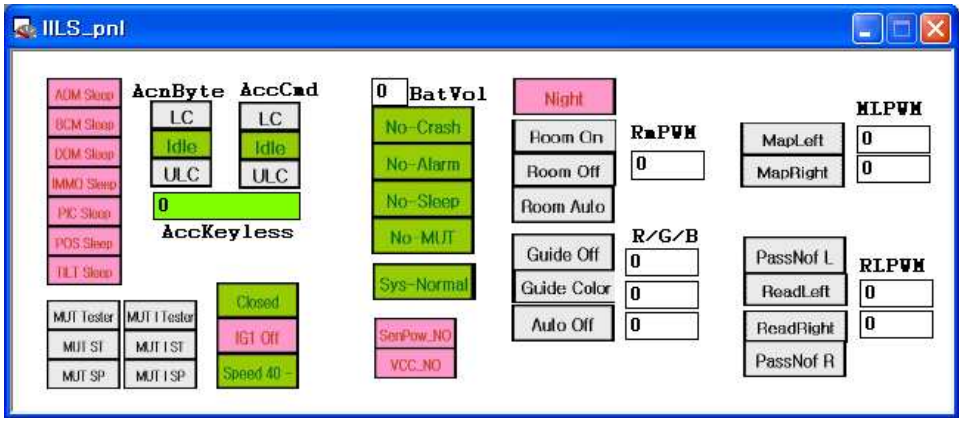


그림 3-60. CANoe 입출력 패널화면

3) Software Testing Result with CANoe

IILS의 요구사항을 기반으로 테스트 케이스를 생성하여 CANoe상에서 테스트 한 결과 모든 요구사항을 만족함을 확인하였다.

(나) ECU 단품 테스트

1) 테스트 환경

IILS 모듈들에 적용된 MCU는 Freescale의 9S12XDP512 제품이며 테스트에 적용된 하드웨어 / 소프트웨어는 다음과 같다.

- 하드웨어
  - HC12용 debugger : P&E USB Multilink
  - CAN통신 H/W : Vector CANcaseXL
  - 테스트 용 입력 모듈 : National Instrument I/O Board(USB-6211)

- 소프트웨어
  - Compiler & Debugger : Freescale Code Warrior
  - CAN Simulation & Test S/W : Vector CANoe.LIN

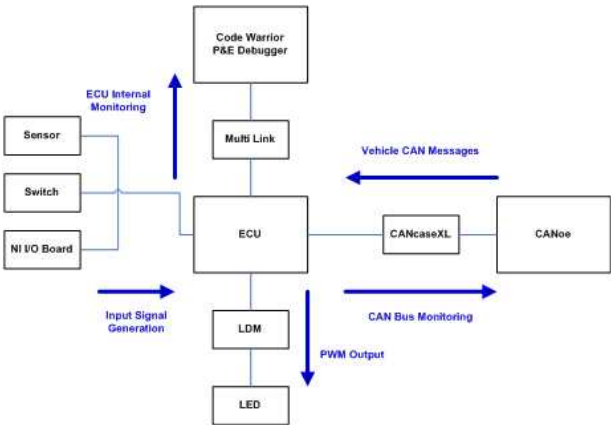


그림 3-61. ECU 단품 테스트 구성

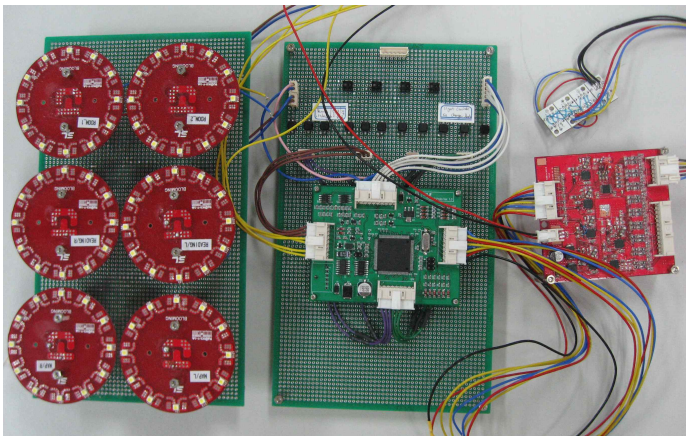


그림 3-62. ECU 단품 테스트 환경

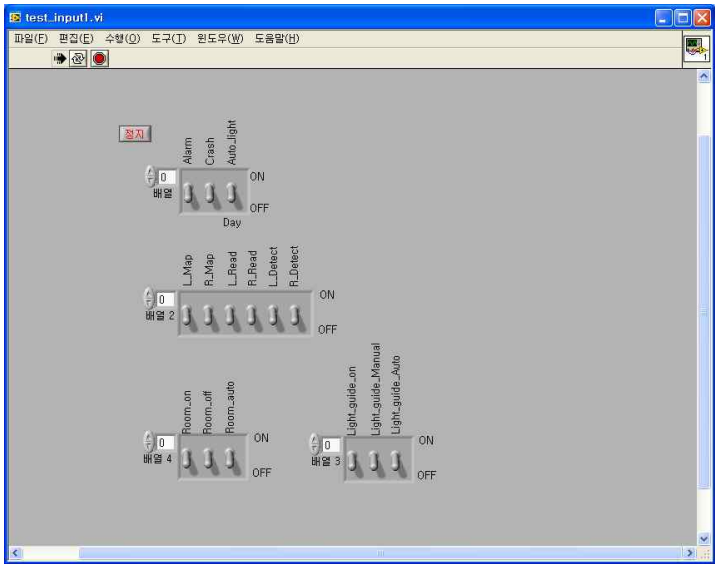


그림 3-63. LabVIEW Test 입력 패널 화면

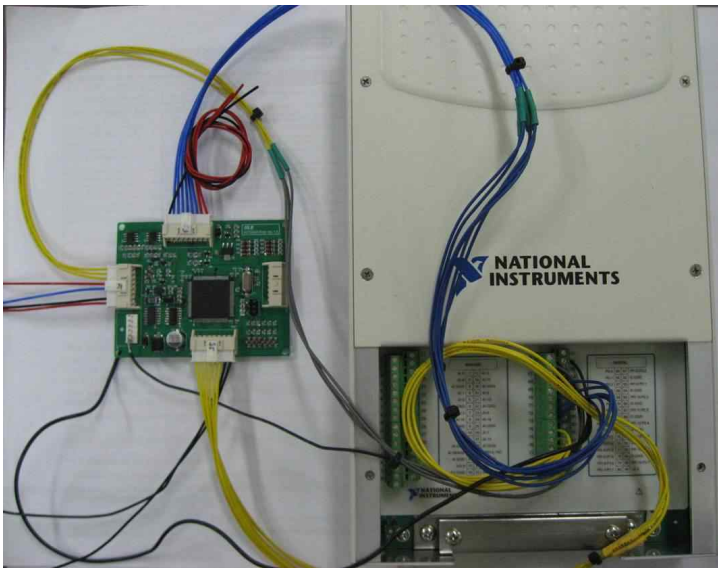


그림 3-64. LabVIEW 테스트 환경

IILS 입력 신호 사양에 맞는 LabVIEW패널을 생성하고 NI사 DAQ Card인 USB-6211를 ECU에 Hard-Wire로 연결하여 ECU 테스트를 시작하였다. LabVIEW 패널에서 입력 디지털 신호를 인가하여 요구사항에 맞는 PWM 출력신호가 발생하는지 테스트 케이스에 따라 확인하였다.

4. ITS를 위한 효율적인 인증 프로토콜 개발

지능형 교통 시스템에서 고속으로 주행하는 차량간 혹은 차량과 기지국간에 송수신되는 메시지 인증을 위한 기반 기술 확보를 위해서, 효율적인 해쉬 체인 운행 알고리즘과 양방 인증을 위한 거리 한정 알고리즘을 개발한다. 효율적인 해쉬 체인 운행(Hash Chain Traversal) 알고리즘은 필요에 따라 일정한 간격으로 위치한 노드들만을 효과적으로 출력할 수 있는 해쉬 체인 운행 알고리즘을 개발하여, 공개키 기반 구조를 이용한 인증 알고리즘들과 비교할 때 송수신되는 메시지의 인증을 위한 지연시간을 효율적으로 개선하였다. 그리고 양방 인증을 위한 거리 한정 알고리즘은, 차량 상호간에 신뢰할 수 있는 거리 정보를 제공하기 위해 양방 인증 기능을 갖는 효율적인 거리 한정 프로토콜 (Distance Bounding Protocol) 개발을 목표로 한다. 이러한 모듈을 이용한 효율적인 데이터 인증 기법의 전체 구조는 아래 그림과 같다.

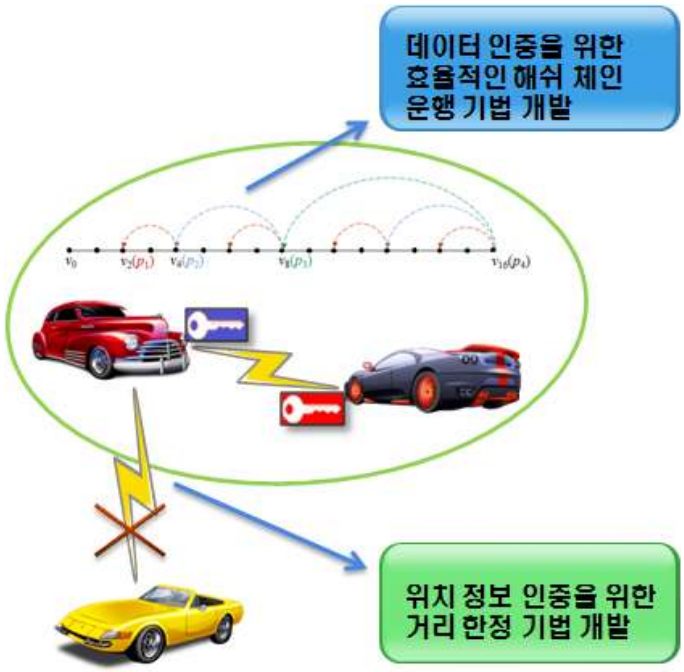


그림 3-65. 효율적 데이터 인증 기법 전체 구조

가. 효율적인 해쉬 체인 운행 알고리즘

해쉬 체인은 무선 네트워크 환경에서 효율적이고 안전한 보안 프로토콜을 설계하기 위한 도구로 많이 이용되어 왔지만, 해쉬 함수가 가지는 일방향성으로 인하여 해쉬 체인의 길이가 길어질수록 해쉬 체인의 해쉬 값들을 계산하는데 많은 연산량이 소모된다는 단점이 있다. 해쉬 체인을 이용하는 암호 알고리즘들은 해쉬 체인의 마지막 해쉬값 v_n을 공개키로 공개하고, v_{n-1}에서 v_0까지의 해쉬값들을 순서대로 비밀값으로 이용한다. 즉, $v_i(0 \le i \le n-1)$는 제한된 일정 시간동안 혹은 일회용으로 대칭키 암호알고리즘의 비밀키 역할, 전송되는 데이터에 대해 인증자 역할 등을 수행한다. 그렇기 때문에 해쉬 체인을 이용하는 보안 프로토콜들은 해쉬 체인의 해쉬값들을 $v_{n-1}, v_{n-2}, \cdots, v_1, v_0$의 순서대로 출력하는 방법이 필요하다. 해쉬 체인 운행 알고리즘은 길이가 긴 해쉬 체인에서 v_{n-1}부터 v_0까지의 해쉬값들을 순서대로 계산하는 효율적인 알고리즘으로 제안되었으며, 최

소한의 메모리(pebble)를 사용하여 v_i를 출력하기 위해 필요한 해쉬 연산 횟수(budget)를 최소화하는 방향으로 연구가 진행되었다.

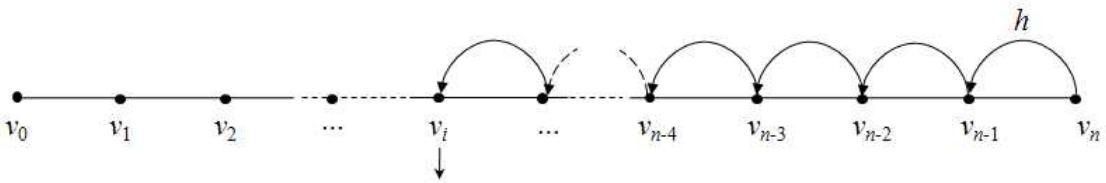


그림 3-66. Trivial hash chain traversal algorithm: re-computation

해쉬 체인 운행 알고리즘은 해쉬 체인을 이용하는 암호 프로토콜들에 있어서 공개값 v_n을 제외한 v_{n-1}부터 v_0까지의 해쉬값들을 순서대로 계산하는 방법을 제공한다. 간단한 해쉬 체인 운행 방법으로는 라운드 $n-i$에서 비밀값 v_0로부터 해쉬 연산을 i번 수행하여 v_i을 계산하는 방법(re-computation)이 있다. 즉, 1개의 메모리 셀에 해쉬 체인의 초기값 v_0에 해당하는 정보만을 저장해두고, v_{n-1}은 v_0로부터 $n-1$번의 해쉬 연산을 반복적으로 수행해서 계산한다. 그리고 다음 두번째 라운드에서 v_{n-2}을 계산하기 위해서는 v_0로부터 $n-2$번의 해쉬 연산을 수행한다. n번째 라운드까지 동일한 과정을 적용하면 $i(1 \le i \le n)$번째 라운드에서 v_{n-i}에 해당하는 해쉬값을 계산할 수 있다. 위의 방법은 1개의 메모리 셀만 있으

면 해쉬 체인의 모든 해쉬값들을 계산할 수 있지만 첫번째 라운드에서 v_{n-1}을 계산하기 위해서는 $n-1$번의 해쉬 연산이 요구된다. 이 방법에서 각 라운드에서 필요한 최대 해쉬 연산 횟수 budget은 해쉬 체인의 길이 n에 비례하여 증가한다.



그림 3-67. Trivial hash chain traversal algorithm: pre-computation

다른 방법으로는 길이가 n인 해쉬 체인의 모든 해쉬값들을 n개의 메모리 셀에 저장해 두고, 각 라운드에서 해당되는 해쉬값을 읽어오는 방법(pre-computation)이 있다. 즉, n개의 메모리 셀에 해쉬 체인의 해쉬 값들이 순서대로 저장되어 있다고 가정하면 i번째 라운드에서는 $n-i$번째 메모리 셀에 저장된 해쉬값을 읽어옴으로써 v_{n-i}를 구할 수 있다. 이 방법은 모든 라운드에서 해쉬 연산을 수행할 필요가 없다. 하지만 모든 해쉬 값들을 각 메모리 셀에 저장해야하기 때문에 센서 노드에서 요구되는 메모리 셀의 개수는 해쉬 체인의 길이 n에 비례하여 증가한다.

이와 같이 해쉬 체인의 해쉬값들을 순서대로 복구하는 문제는 해쉬 체인의 효율성을 떨어뜨리고, 무선 네트워크 환경에서 사용되는 해쉬 체인을 이용하는 암호 알고리즘들의 효율성을 떨어뜨리는 제약 조건으로 작용한다. 그렇기 때문에, 해쉬 체인의 효율성을 유지하고 VANET과 같은 무선 네트워크에 적합한 저전력 기반 보안 프로토콜을 설계하기 위해서는 해쉬 체인의 해쉬 값들을 순서대로 복구하는 효율적인 해쉬 체인 운행 알고리즘이 요구된다. 효율적인 해쉬 체인 운행 알고리즘에 대한 연구 결과는 2002년 Jakobsson에 의해 처음으로 발표되었으며 [Jak02], 2003년에 Coppersmith와 Jakobsson에 의해 준최적화된 연구 알고리즘이 제안되었다[CJ03]. 그리고 2009년도에 본 연구팀의 Yum 등에 의해 준최적화된 Coppersmith와 Jakobsson의 알고리즘을 개선한 해쉬 체인 운행 알고리즘이 발표되었다[YSEL09]. 기존에 발표된 해쉬 체인 운행 알고리즘들 중에서는 Yum 등이 발표한 해쉬 체인 운행 알고리즘이 가장 효율적인 것으로 알려져 있다.

본 연구에서는 일반화된 해쉬 체인 운행 알고리즘을 설계 및 분석하였다. ?절에서는 해쉬 함수, 해쉬 체인, 해쉬 체인의 활용 방안에 대해 배경지식을 설명하고, ?절에서는 기존 연구 내용에 대한 분석 및 검토 내용을 상세히 기술한다. 마지막으로 ?절에서 본 연구에서 제안하는 일반화한 해쉬 체인 운행 알고리즘에

대해 기술한다.

(1) 배경지식

① 해쉬 함수(hash function)

무선 네트워크는 네트워크가 갖는 근본적 특성으로 인해 일반 네트워크 환경보다 훨씬 보안에 취약하다는 특성을 갖는다. 전송되는 데이터에 대한 접근이 용이하기 때문에 전송되는 정보가 쉽게 변경되거나 정당하지 않은 노드가 위조된 데이터를 전송함으로써 전체 정보의 무결성을 쉽게 무너뜨릴 수 있다. 뿐만 아니라 악의적인 노드가 정당한 노드로 가장하여 불필요한 정보를 계속 발생시켜 중간 노드의 자원을 소모시킴으로써 네트워크의 수명을 단축시킬 수도 있다. 이처럼 간단한 공격만으로도 무선 네트워크는 쉽게 와해될 수 있다. 소형 저전력 네트워크 단말기들의 발전과 함께 이러한 보안 문제들은 더욱 크게 대두되고 있다. 무선 네트워크 환경에서 해결해야 하는 보안 문제들은 키 분배 및 관리, 인증, 보안을 위한 네트워크 구조, 라우팅 등 다양한 요소들에서 결합된 형태로 나타난다. 소형 저전력 네트워크 단말기들을 사용하는 VANET과 같은 무선 네트워크 환경에서 보안 문제들을 해결하기 위해서는 기존 네트워크에서 적용되는 보안 기술보다 높은 효율성을 제공하면서 안전성을 보장하는 새로운 보안 기술들에 대한 연구가 요구된다. 이를 위해 무선 네트워크 분야에서는 노드의 제약성을 극복하고 효율적으로 작동할 수 있는 암호학적 프리미티브에 대한 관심이 고조되었다. 해쉬 함수는 이와 같은 맥락에서 무선 네트워크에서 효율적인 보안 기술을 제공하기 위한 암호학적 프리미티브로 대두되었다. 암호학적으로 사용되는 해쉬 함수는 해쉬 함수의 출력으로부터 해쉬 함수의 입력을 구하는 것이 확률적으로 어려운 일방향 함수이다. 본 장에서는 해쉬 함수를 h로 표기하며, 임의의 길이를 가지는 이진 문자열을 입력으로 받아서, 길이가 ℓ인 이진 문자열을 출력하는 함수로 정의하고, 다음과 같이 표기한다.

$$h : 0,1^{*} \rightarrow 0,1^{\ell}$$

해쉬 함수는 일방향 함수로 전자 서명에서 메시지를 압축하거나 MAC(Message Authentication Code) 알고리즘의 설계 등에 사용되어 왔으며, 공개키 암호 알고리즘들에서 사용되는 멱승 연산과 비교하여 그 연산 속도가 1000배 이상 높은 것으로 알려져 있다. 해쉬 함수를 반복적으로 적용해서 얻어지는 해쉬 체인은 해쉬 함수가 제공하는 암호학적 특성과 효율성으

로 인하여 센서 네트워크에서 효율적인 보안 프로토콜 설계에 많이 사용되어 왔다. 자동차로 구성되는 네트워크 단말들은 고속으로 주행하고 있기 때문에 매우 짧은 시간 내에 암호학적 연산을 수행해야 한다는 것을 고려한다면 해쉬 함수는 VANET에서 효율적인 암호 알고리즘 구현을 위한 프리미티브로써 그 활용도가 높을 것으로 기대된다.

② 해쉬 체인(hash chain)

해쉬 체인은 1981년 Lamport에 의해 처음으로 제안되었으며, 초기값 v_0에 해쉬 함수를 반복적으로 적용$(v_i = h(v_{i-1}))$하여 구해지는 일련의 연속된 해쉬값들 $\{v_n, v_{n-1}, \cdots, v_1, v_0\}$로 이루어진다 [Lam81]. 해쉬 체인에서 초기값 v_0와 마지막 해쉬값 v_n은 각각 비밀값(private key)과 공개값(public key)으로 사용되며, 공개값 v_n을 제외한 나머지 해쉬 값들의 개수를 해쉬 체인의 길이로 정의한다. 해쉬 체인에 적용되는 해쉬 함수는 일방향 함수이며, v_i로부터 $v_n, v_{n-1}, \cdots, v_{i+2}, v_{i+1}$을 구하는 것은 쉽지만 비밀값 v_0를 알지 못하는 공격자는 $v_{i-1}, v_{i-2}, \cdots, v_1, v_0$를 구하는 것이 확률적으로 불가능하다. 그리고 v_i를 수신한 수신자는 $n-i$번의 해쉬 연산을 수행하여 v_i의 정당성을 확인할 수 있다. 해쉬 체인은 해쉬 값을 직선 위의 한 점으로 표현함으로써 아래와 같이 나타낼 수 있다.

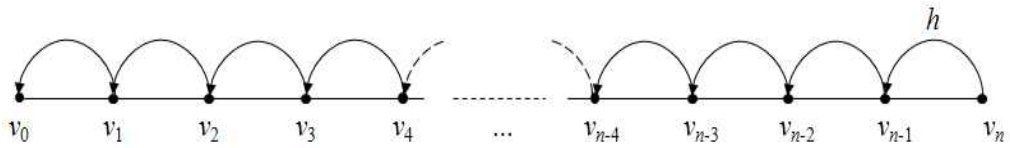


그림 3-68. 해쉬 체인의 도식화

③ 해쉬 체인의 활용 방안

해쉬 체인은 암호학적 특성과 그 효율성으로 인하여 VANET, 센서 네트워크를 비롯하여 유비쿼터스 컴퓨팅 환경에 적합한 보안 기술을 제공하기 위한 도구로 다양한 분야에서 이용되고 있으며, 그 중 대표적인 방법으로는 Micali가 제안한 인증서 체인이 있다 [Mic96]. Micali가 제안한 방법에서 CA(Certificate Authority)는 사용자의 공개키 pk와 해쉬 체인의 마지막 해쉬값 v_n에 대한 인증서를 발급하고, 정해진 시간마다 주기적으로 해쉬 체인의 값 v_i를 사용자에게 순서대로 전송한다. 그리고 사용자는 수신된 해쉬값 v_i를

이용하여 일정 시간동안 인증서의 유효성을 증명하는데 사용할 수 있다.

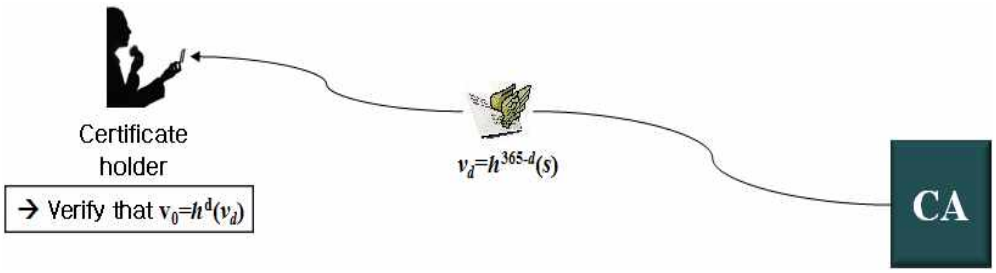


그림 3-69. Micali의 인증서 체인

그림은 Micali의 인증서 체인의 한 예를 도식화 한 것으로 CA는 1년 동안 유효한 인증서를 사용자에게 발급하면서 무작위로 선택된 비밀값 s를 365번 해쉬한 해쉬값 $v_0 = h^{365}(s)$를 인증서에 포함시킨다. 인증서를 발급하고 d일째에 인증서를 확인하기를 원하는 검증자(Verifier)는 CA로부터 $v_d = h^{365-d}(s)$를 받아서 d번 해쉬해서 구한 $v_0^{'}(= h^d(v_d))$가 인증서에 포함된 v_0와 같은지를 비교한다. 검증 알고리즘은 $v_0 = v_0^{'}$일 경우 인증서가 유효하다고 판단하고, 그렇지 않을 경우에는 인증서를 정당하지 못한 것으로 판단한다. 이외에도 해쉬 체인은 전자 지불 시스템 [AMS96, RS96], 일회용 패스워드 시스템 [RFC1760], 온라인 경매 [SS99], 안전한 경로 설정 [HPJ05], multicast authentication [PCST01, PCTS00]등의 다양한 분야에서도 효율적인 보안 프로토콜 설계를 위한 도구로 많이 사용되고 있다. 그리고 해쉬 체인을 이용하는 암호 알고리즘 중 Adrian Perrig 등이 개발한 TESLA(Timed Efficient Stream Loss-tolerant Authentication)는 IETF 표준 [RFC4383]으로 제정되어 산업 전반에서 활용되고 있다.

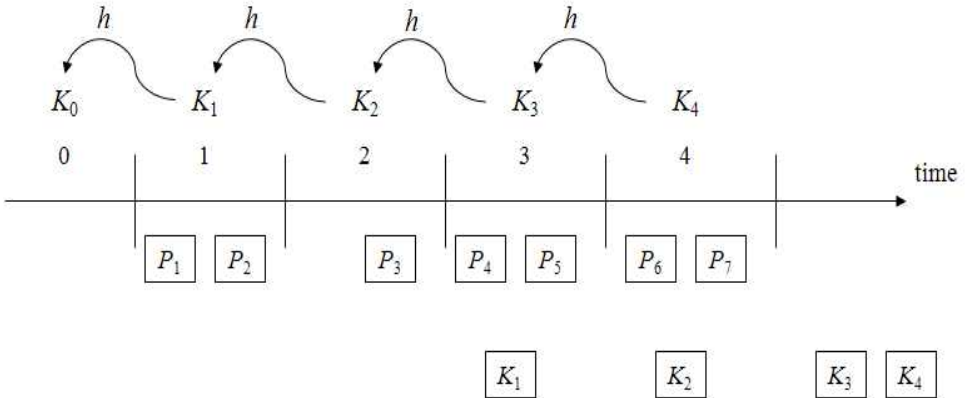


그림 3-70. TESLA(Timed Efficient Stream Loss-tolerant Authentication)

위 그림은 TESLA의 한 예를 도식화하여 표현한 것으로 K_i는 해쉬 체인을 구성하는 해쉬 값으로 K_1부터 순서대로 일정 시간 간격동안 비밀값으로 사용된다. 이때 비밀값으로 사용되는 K_i는 전송되는 데이터$\{P_1, P_2, ...\}$에 대한 암호·인증 알고리즘의 비밀키로 사용되기도 한다. 그리고 일정 시간 이후에 K_i를 공개함으로써 $\{P_1, P_2, ...\}$을 수신한 대상들은 수신된 데이터를 복호·검증할 수 있다.

(2) 기존 연구

① Jakobsson 알고리즘[Jak02]

해쉬 체인 운행 알고리즘은 Itkis와 Reyzin이 제안한 애모티제이션(amortization) 알고리즘에 기반하여 [IR01], 2002년 Jakobsson에 의해 처음으로 제안되었다 [Jak02]. 2개의 pebble이 체인에서 비밀값 v_n과 임의의 해쉬값 v_d를 저장하고 있다면, 해쉬값 v_1을 출력하기 위해서는 $d-1$번의 해쉬 연산이 필요하며, 다음 해쉬값 v_2를 출력하기 위해서는 $d-2$의 해쉬 연산이 필요하다. 그리고 pebble에 저장된 해쉬값 v_d를 출력하고 난 뒤에 다음 해쉬값 v_{d+1}을 출력하기 위해서는 $n-(d+1)$의 해쉬 연산이 필요하다. 따라서, $d=n/2$ 일 때 budget의 크기가 $(n/2)-1$로 최소가 된다. 3개의 pebble $\{p_1, p_2, p_3\}$을 사용할 경우에는 3개의 pebble에 등간격으로 위치하는 해쉬값들 v_n, $v_{2n/3}$, $v_{n/3}$을 저장하는 것보다 p_3에 비밀값 v_n, p_2에는 해쉬 체인의 1/2지점(position=$n/2$)에 해당하는 $v_{n/2}$, p_1에는 해쉬 체인에서 1/4지점(position=$n/4$)에 해당하는 $v_{n/4}$을 저장하는 것이 budget의 크기를 줄일 수

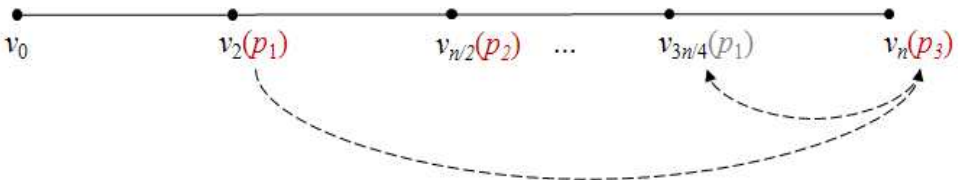


그림 3-71. 3개의 pebble을 이용한 효율적인 해쉬 체인 운행 알고리즘

있다. 전자의 경우에 필요한 budget의 크기는 $n/3$이 되지만, 후자의 경우에는 $n/4$이 된다. 후자의 경우, budget의 크기가 $n/4$이라면 라운드 $n/2$까지는 체인의 해쉬 값들$(v_i, 1 < i \leq n/2)$을 쉽게 출력할 수 있다. 이 때, p_1은 라운드 $n/4$에서 저장하고 있던 $v_{n/4}$을 출력한 이후, n지점에 위치하는 v_n을 저장한다. 그리고, 라운드 i $(n/4 \leq i \leq n/2)$에서 해쉬 값을 출력하고 남는 해쉬 연산량을 이용하여 1번씩만 p_1의 값을 해쉬하여 업데이트한다면, 라운드 $(n/2)+1$이 시작하기 전에 $3n/4$에 위치하는 해쉬값 $v_{3n/4}$을 획득하게 된다. 이 경우, budget $n/4$만을 가지고도 길이가 n인 해쉬 체인을 운행할 수 있게 된다. Jakobsson의 알고리즘에서 pebble $p_i (1 \leq i \leq \log n)$는 초기에는 2^i에 위

```
1. If current.position = n then                    (Arrived at the end?)
     halt                                          (Quit)
   else
     increase current.position by one              (Move pointer forward)
2. For 1 ≤ j ≤ σ do                                (Consider all pebbles)
     If p_j.position ≠ p_j.destination then        (If not arrived ...)
       p_j.position ← p_j.position − 2             (... then move it ...)
       p_j.value ← h(h(p_j.value))                 (... and update value)
3. If current.position is odd then
     output h(p_1.value)                           (Hash and output)
   else
     output p_1.value                              (Output stored value)
     p_1.position ← p_1.position + p_1.StartIncr    (Reached: reassign)
     p_1.destination ← p_1.position + p1.DestIncr   (and reassign)
     If p_1.destination > n then                   (Pebble redundant?)
       p_1.destination ← ⊥                         (Then retire pebble!)
       p_1.position ← ⊥
     else
       p_1.value ← FindValue                       (Call to set value)
     Sort pebbles                                  (Sort w.r.t. destination)
```

그림 3-72. Jakobsson이 제안한 해쉬 체인 운행 알고리즘

치하는 해쉬값 v_{2^i}을 저장하고 있다. 그리고 p_i는 초기 위치(2^i)에서 해쉬 체인을 간격이 2^i인 구간으로 나누면서 오른쪽으로 이동한다. 이 때, $\log n$개의 pebble이 해쉬 체인 상에서 이동하는 궤적은 부분이 전체를 닮아가는 프랙탈(fractal) 형상을 가진다. Jakobsson의 알고리즘에서 pebble p_1은 해쉬 체인을 간격이 2인 구간으로 나누기 때문에 각 라운드 i에서 v_i을 출력하는데 소모되는 해쉬 연산 횟수는 평균 1/2번이 된다. 그리고 다음 라운드에서 해쉬값을 출력하기 위해 이동 중인(=active) pebble에 대해 2번의 해쉬 연산이 할당되며, 알고리즘에 따라 최대 $(\log n)/2$개의 pebble만이 이동하기 때문에 Jakobsson의 알고리즘에서 필요한 budget의 크기는 $\lceil \log n \rceil (= (\log n)/2 \text{ pebble} \times 2 \text{position/pebble})$이 된다. 즉, Jakobsson이 제안한 프랙탈 해쉬 체인 운행 알고리즘은 길이가 n인 해쉬 체인을 운행하는 데 $\lceil \log n \rceil$개의 pebble과 $\lceil \log n \rceil$ budget을 필요로 한다. 위 그림은 Jakobsson이 제안한 해쉬 체인 운행 알고리즘을 나타낸 것이다.

아래 그림은 해쉬 체인 길이(n)가 2^6이고, budget이 6일 경우에 Jakobsson이 제안한 해쉬 체인 운행 알고리즘을 적용한 예를 도식화하여 나타낸 것이다. 아래 그림에서 가로 축은 해쉬 체인을 구성하는 해쉬 값의 위치를 나타낸 것으로 가로축에서 i는 v_i를 나타낸다. 그리고 세로축은 해쉬 체인 운행 알고리즘이 적용되는 각 라운드를 표시한 것으로 $n = 2^6$인 경우 2^6 라운드까지 존재하지만 $33 \sim 64$ 라운드는 앞과 동일한 과정을 반복하기 때문에 표시하지 않는다. 알고리즘에서 사용되는 pebble은 모두 6개로 인덱스를 사용하여 6개의 pebble $\{P_1, P_2, \ldots, P_6\}$을 다르게 표기한다. 그림 "Coppersmith-Jakobsson 알고리즘에서 pebble의 이동"에서 각 pebble P_i은 항상 동일한 역할을 수행하는 것을 확인 할 수 있다. 그리고 C(current value)는 현재 라운드에서 출력되는 해쉬 값을 가리키며 매 라운드마다 1씩 증가한다. 아래 그림에서 Jakobsson의 해쉬 체인 운행 알고리즘은 1 라운드를 시작하기 전에 6개의 pebble은 2^i 간격으로 놓여 있으며(canonical form), 2 라운드에서 pebble P_1은 저장하고 있던 해쉬 값을 출력하고 8번째 해쉬 값으로부터 해쉬 연산을 반복하여 6번째 해쉬 값을 저장하기 위해 이동을 시작한다. 또한 P_2는 4 라운드에서 저장하고 있던 해쉬 값을 출력하고 16번째 해쉬 값으로부터 해쉬 연산을 반복하여 12번째 해쉬 값을 저장하기 위해 이동을 시작한다. 이때 각 라운드에서 각 pebble의 이동에 소모되는 해쉬 연산은 최대 2번씩 수행된다.

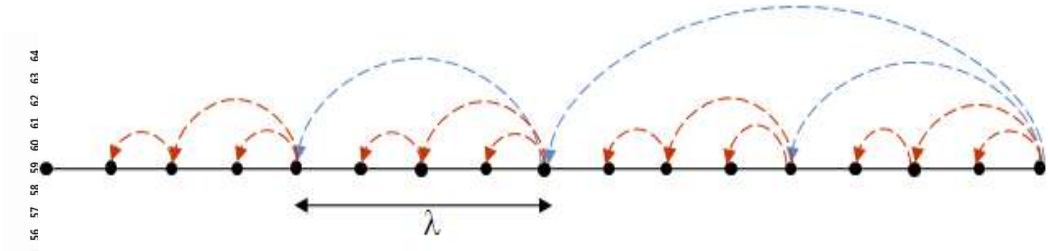


그림 3-74. Coppersmith-Jakobsson 알고리즘에서 pebble의 이동

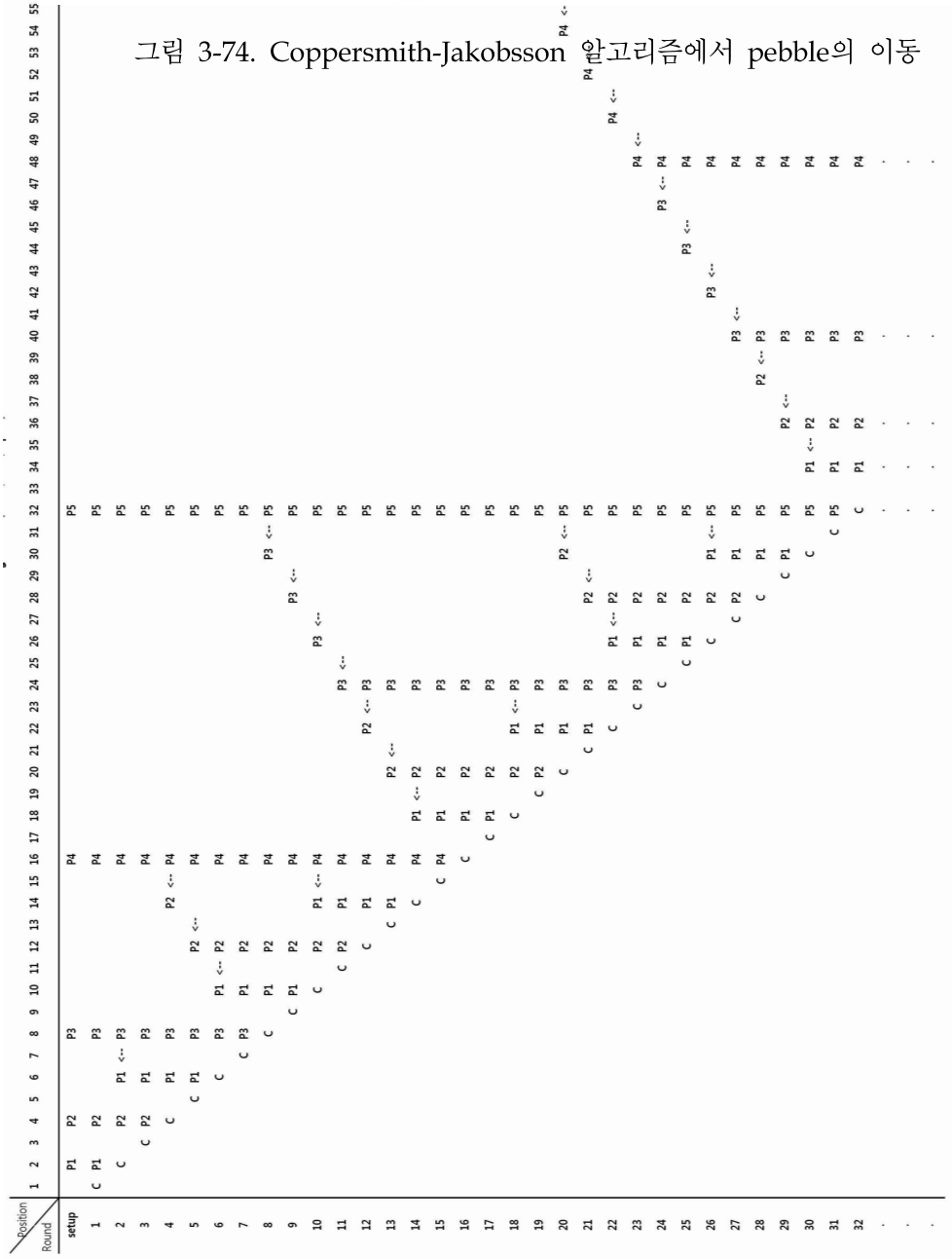


그림 3-73. Jakobsson 알고리즘 적용 예$(n=2^6, b=6)$

② Coppersmith-Jakobsson 알고리즘[CJ03]

Jakobsson의 운행 방법은 알고리즘 측면에서 간단하지만 budget 측면에서는 효율적이지 못하였다. 이와는 반대로 Coppersmith-Jakobsson이 제안한 알고리즘은 budget의 사용에 있어서 준최적화(almost optimal)된 결과를 제공하지만, 운행 방법이 복잡하였다. Coppersmith-Jakobsson은 매 라운드에서 남는 budget을 효율적으로 사용하기 위해 pebble의 역할을 high-priority pebble과 low-priority pebble 2가지로 나누었다. Low-priority pebble은 길이가 n인 해쉬 체인을 간격이 λ인 구간으로 나누는 역할을 수행하고, high-priority pebble은 low-priority pebble에 의해 λ 간격으로 나누어진 구간에서 프랙탈 형상을 이루며 해쉬값을 저장한다. 그렇기 때문에 Coppersmith-Jakobsson의 알고리즘은 double-layer 프랙탈 구조를 가진다.

현재의 해쉬값을 출력하고 high-priority pebble에 할당되고 남는 budget은 모두 low-priority pebble에 할당되어, 각 라운드에서 남는 budget을 모두 소모한다. 이와 같은 방법으로 Coppersmith-Jakobsson은 길이가 n인 해쉬 체인을 운행하기 위해 $\lceil \log n \rceil + \lceil \log(\log n + 1) \rceil$ pebble과 $\lfloor 0.5 \log n \rfloor$ budget을 이용하는 준최적화된 해쉬 체인 운행 알고리즘을 제안하였다. 아래 그림은 Coppersmith-Jakobsson이 제안한 해쉬 체인 운행 알고리즘을 나타낸 것이다.

```
1. Set available ← b.                    (Set the remaining budget.)
2. Increase current by 1.
3. If current is odd then                (No pebble at this position.)
      output h(p1.value),                (Compute and output.)
      decrease available by 1,
   else                                  (A pebble at this position.)
      output p1.value,                   (Output value, set pebble free.)
      set p1.status ← free,
      if current = n, then halt.         (Last value in sequence.)
4. For all free pebbles pj do            (Reassign free pebbles.)
      if backup = 0 then                 (Backup low-priority needed.)
            pj.priority ← low,
            pj.status ← ready,
            BU ← pj,
            backup ← 1,
      else
            Call placeHP (pj)            (Make it high priority.)
5. Sort pebbles.
6. Set j ← 1.                            (First pebble first.)
7. While available > 0 do
      if pj.status = active then                  (Only move active pebbles.)
            decrease pj.position by 1,            (Update its position...)
            pj.value ← h(pj.value),               (... and value...)
            decrease available by 1,              (... and do the accounting.)
            if pj.position = pj.destination then  (Pebble arrived!)
                  pj.status ← arrived,
                  if pj.priority = low then       (A low-priority pebble arrived.)
                        LP ← BU,                  (Backup becomes low priority.)
                        backup ← 0,
                        Call placeLP,             (Activate new low priority pebble!)
                        Sort pebbles.
      increase j by 1.                            (Next pebble!)
8. Sort pebbles.
9. Go to 1.                                       (Next element now.)
```

그림 3-75. Coppersmith-Jakobsson 해쉬 체인 운행 알고리즘

아래 그림은 해쉬 체인 길이(n)가 2^6이고, budget이 3일 경우에 Coppersmith와 Jakobsson이 제안한 해쉬 체인 운행 알고리즘을 적용한 예를 도식화하여 나타낸 것이다. Coppersmith-Jakobsson 알고리즘은 Jakobsson 알고리즘과 비교하여 동일한 길이의 해쉬 체인을 운행하기 위해서 필요로 하는 budget이 $1/2$로 줄어들었다. 하지만 아래 그림에서 나타나듯이 pebble이 가지는 역할이 일정하지 않으며 low priority pebble과 high priority pebble 중에서 알고리즘에서 필요에 따라 역할을 수행하게 된다. 그렇기 때문에 기존 Jakobsson 알고리즘과는 다르게 double layer를 가지며, pebble의 이동이 이루어진다. 주어진 예제에서 $\lambda = 8$로 low priority pebble은 8의 간격으로 pebble이 놓인다. High priority pebble은 간격 8 안에서 이동이 이루어지며 더 작은 2와 4의 간격으로 이동이 이루어진다. 아래 그림에서 가로 축은 해쉬 체인을 구성하는 해쉬 값의 위치를 나타낸 것으로 가로축에서 i는 v_i를 나타낸다. 그리고 세로축은 해쉬 체인 운행 알고리즘이 적용되는 각 라운드를 표시한 것으로 $n = 2^6$인 경우 2^6 라운드까지 존재하지만 $33 \sim 64$ 라운드는 앞과 동일한 과정을 반복하기 때문에 표시하지 않는다. 알고리즘에서 사용되는 pebble은 모두 9개로 이동시에 low priority pebble과 high priority pebble은 각각 L과 H로 표기한다. 그리고 C(current value)는 현재 라운드에서 출력되는 해쉬 값을 가리키며 매 라운드마다 1씩 증가한다. Jakobsson의 해쉬 체인 운행 알고리즘은 1 라운드를 시작하기 전에 6개의 pebble은 2^i 간격으로 놓여 있으며(canonical form), 남은 3개는 대기 상태로 필요에 따라 역할이 할당된다. 각 라운드에서 pebble의 이동에 소모되는 해쉬 연산은 최대 3번씩 수행된다.

③ YSEL 알고리즘

최근 CT-RSA 2009에서 본 연구팀의 Yum 등은 single-layer만을 사용해서 준최적화된 결과를 제공하는 새로운 해쉬 체인 운행 알고리즘(YSEL 알고리즘)을 제안하였다. YSEL 알고리즘은 Coppersmith-Jakobsson이 제안한 알고리즘과 비교할 때 보다 간단한 single-layer 프랙탈 구조를 가질 뿐만 아니라 보다 적은 개수의 pebble을 사용한다. YSEL 알고리즘은 가장 왼쪽에서 이동 중인 pebble에 budget을 우선적으로 할당하는 새로운 애모티제이션 기법을 사용하였으며, 제안한 알고리즘은 길이가 n인 해쉬 체인을 운행하기 위해 $\lceil \log n \rceil$ 개의 pebble과 $\lceil 0.5\log n \rceil$ budget을 사용하였다. YSEL 알고리즘은 현재까지 제안된 해쉬 체인 운행 알고리즘들 중에서 가장 효율적인 것으로 알려져 있다.

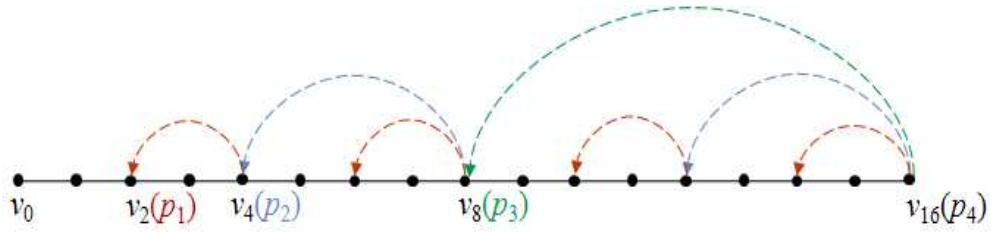


그림 3-76. YSEL 알고리즘에서 pebble의 이동

YSEL 알고리즘에서 pebble이 이동하는 순서는 Jakobsson의 알고리즘과 동일하다. 하지만 Jakobsson의 알고리즘에서 이동 중인 모든 pebble이 매 라운드마다 2번씩 이동하는 반면 YSEL 알고리즘은 주어진 budget 안에서 가장 왼쪽(v_i 방향)에서 이동을 시작하는 pebble부터 budget을 할당한다. YSEL 알고리즘은 Jakobsson 알고리즘 보다 더 효율적인 애모티제이션 기법으로 budget의 활용도를 최대화하였다.

아래 그림에서는 해쉬 체인 길이(n)가 2^6이고, budget이 3일 경우에 YSEL 해쉬 체인 운행 알고리즘을 적용한 예를 도식화하여 나타낸 것이다. 아래 그림에서 가로 축은 해쉬 체인을 구성하는 해쉬 값의 위치를 나타낸 것으로 가로축에서 i는 v_i를 나타낸다. 그리고 세로축은 해쉬 체인 운행알고리즘이 적용되는 각 라운드를 표시한 것으로 $n=2^6$인 경우 2^6 라운드까지 존재하지만 $33 \sim 64$ 라운드는 앞과 동일한 과정을 반복하기 때문에 표시하지 않는다. 알고리즘에서 사용되는 pebble은 모두 6개로 인덱스를 사용하여 6개의 pebble $\{P_1, P_2, \ldots, P_6\}$을 다르게 표기한다. 아래 그림에서 각 pebble P_i은

그림 3-77. Coppersmith-Jakobsson 알고리즘 적용 예($n = 2^6, b = 3$)

```
INPUT: n, h(·) and p_i, where n = 2^κ and 1 ≤ i ≤ κ.
 0. b = ⌈κ/2⌉, current = 0;      // initialization of the budget and the current pointer
 1. IF current = n THEN
        HALT;
    ELSE
        current ← current + 1;          // a new round begins
        available ← b;                  // the remaining budget is set
 2. i ← P(arrived, 0);                  // the leftmost arrived pebble is selected
    IF current mod 2 = 1 THEN           // if current is odd
        OUTPUT h(p_i.value);
        available ← available − 1;
    ELSE                                // if current is even
        OUTPUT p_i.value;
        p_i.position ← p_i.position + 3 · 2^i;          // a new starting position is assigned
        p_i.destination ← p_i.destination + 2 · 2^i;    // a new destination is assigned
        p_i.value ← ⊥;
        p_i.status ← ready;
        j ← P(arrived, p_i.position);
        IF j ≠ ⊥ THEN                   // if the link of the starting position is available
            p_i.value ← p_j.value;
            p_i.status ← active;
 3. i ← P(active, 0);                   // the leftmost active pebble is selected
    IF i = ⊥ THEN                       // if there is no active pebble
        GO TO 1;
    WHILE available > 0 DO
        p_i.position ← p_i.position − 1;    // p_i moves one step towards its destination
        p_i.value ← h(p_i.value);           // p_i upates its associated link
        available ← available − 1;
        IF p_i.position = p_i.destination THEN      // if p_i arrives at its destination
            p_i.status ← arrived;
            j ← P(ready, p_i.destination);
            IF j ≠ ⊥ THEN
                p_j.value ← p_i.value;
                p_j.status ← active;
            GO TO 3;
    GO TO 1;        // this line is reached only if available = 0
```

그림 3-78. YSEL 해쉬 체인 운행 알고리즘

Jakobsson의 알고리즘과 같이 항상 동일한 역할을 수행하는 것을 확인 할 수 있다. 그리고 C(current value)는 현재 라운드에서 출력되는 해쉬 값을 가리키며 매 라운드마다 1씩 증가한다. 그림 16에서 Jakobsson의 해쉬 체인 운행 알고리즘은 1 라운드를 시작하기 전에 6개의 pebble은 2^i 간격으로 놓여 있으며(canonical form), 1 라운드에서 P_1에 저장되어 있는 해쉬 값을 1번 해쉬하여 값을 출력한다. 그리고 2 라운드에서 P_1에 저장되어 있는 해쉬 값을 출력하고, P_1은 P_3에 저장되어 있는 해쉬 값으로부터 2번 해쉬 연산을 수행하여 6번째 해쉬 값을 저장한다. 3 라운드에서는 P_2에 저장되어 있는 해쉬 값을 1번 해쉬하여 값을 출력하고, 4 라운드에서는 P_2에 저장되어 있는 해쉬 값을 출력하고, P_2는 P_4에 저장되어 있는 해쉬 값으로부터 3번의 해쉬 연산을 수행하여 13번째 해쉬 값을 임시로 저장한다. 이때, 주어진 budget이 3이기 때문에 P_2는 목표위치(12)에 도달하지 못한다. 다음 5 라운드에서 P_2는 1번의 해쉬 연산을 수행하여 목표위치(12)에 도달한다. 이후에 운행 과정은 유사한 방법이 반복된다.

(3) 일반화한 해쉬 체인 운행(generalized hash chain traversal) 알고리즘

기존 해쉬 체인 운행 알고리즘들은 해쉬 체인의 해쉬 값을 v_1부터 v_n까지 순서대로 1개씩 출력한다. 하지만 적용되는 보안 프로토콜에 따라 해쉬 체인 운행 알고리즘은 해쉬 값을 일정한 간격에 따라 선택적으로 출력해야 하는 경우도 발생한다. 예를 들어 SEAD(Secure Efficient Ad hoc Distance vector) 라우팅 프로토콜은 라우팅 업데이트를 위해 일련번호(sequence number)와 홉(hop) 간격을 나타내기 위한 metric 정보를 서버에 전송한다. 그렇기 때문에, 해쉬 체인의 해쉬 값을 metric 만큼의 간격으로 선택적으로 출력하는 알고리즘이 요구된다. 하지만, 기존 해쉬 체인 운행 알고리즘들은 $\{v_1, v_2, \ldots, v_n\}$을 순서대로 출력하기 때문에, SEAD 라우팅 프로토콜에서는 m 간격으로 해쉬 값을 출력하기 위해 기존 알고리즘들을 실행해서 $\{v_m, v_{2m}, \ldots, v_n\}$을 추출해야 한다. 본 연구에서는 $m \geq 1$인 모든 경우에 대해서 선택적으로 해쉬 값을 출력하는 해쉬 체인 운행 알고리즘에 대한 연구를 수행하였다. 이전의 연구 결과들은 본 연구 결과에서 $m = 1$인 특수한 경우에 해당하며, 제안하는 연구 결과는 기존 알고리즘을 $m \geq 1$인 모든 경우에 대해서 선택적으로 해쉬 값을 출력하는 해쉬 체인 운행 알고리즘으로 변환하는 것으로 기존 알고리즘들에 적용이 가능하다. 본 절에서는 설명의 편의를 위해 기존 알고리즘 중에서 가장 효율적인 것으로 알려진 YSEL 알고리즘에

그림 3-79. YSEL 알고리즘 적용 예($n=2^6, b=3$)

Input: $(n, h(\cdot), \vec{p}, c, v)$

1: $\kappa \leftarrow \log n$;
2: **if** $c > n$ **then**
3: halt;
4: **else**
5: $available \leftarrow \lceil \frac{\kappa}{2} \rceil$;
6: **end if**
7: $i \leftarrow \mathcal{P}(\vec{p}, arrived, 0)$;
8: **if** $c \bmod 2 = 1$ **then**
9: $v \leftarrow h(p_i.\mathsf{value})$;
10: $available \leftarrow available - 1$;
11: **else**
12: $v \leftarrow p_i.\mathsf{value}$;
13: $p_i.\mathsf{position} \leftarrow p_i.\mathsf{position} + 3 \cdot 2^i$;
14: $p_i.\mathsf{destination} \leftarrow p_i.\mathsf{destination} + 2 \cdot 2^i$;
15: $p_i.\mathsf{value} \leftarrow \perp$;
16: $p_i.\mathsf{status} \leftarrow ready$;
17: $j \leftarrow \mathcal{P}(\vec{p}, arrived, p_i.\mathsf{position})$;
18: **if** $j \neq \perp$ **then**
19: $p_i.\mathsf{value} \leftarrow p_j.\mathsf{value}$;
20: $p_i.\mathsf{status} \leftarrow active$;
21: **end if**
22: **end if**
23: $i \leftarrow \mathcal{P}(active, 0)$;
24: **if** $i = \perp$ **then**
25: halt;
26: **end if**
27: **while** $available > 0$ **do**
28: $p_i.\mathsf{position} \leftarrow p_i.\mathsf{position} - 1$;
29: $p_i.\mathsf{value} \leftarrow h(p_i.\mathsf{value})$;
30: $available \leftarrow available - 1$;
31: **if** $p_i.\mathsf{position} = p_i.\mathsf{destination}$ **then**
32: $p_i.\mathsf{status} \leftarrow arrived$;
33: $j \leftarrow \mathcal{P}(\vec{p}, ready, p_i.\mathsf{destination})$;
34: **if** $j \neq \perp$ **then**
35: $p_j.\mathsf{value} \leftarrow p_i.\mathsf{value}$;
36: $p_j.\mathsf{status} \leftarrow active$;
37: **end if**
38: go to line 23;
39: **end if**
40: **end while**
41: halt;

그림 3-80. YSEL의 *traversal* 알고리즘

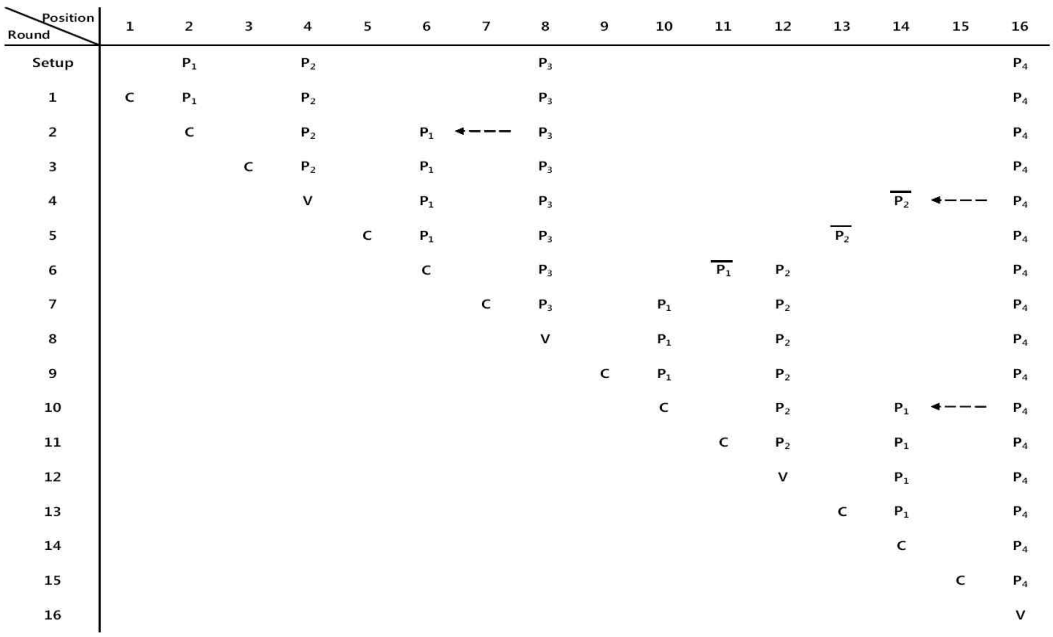


그림 3-81. Simplification of YSEL algorithm (m=4)

기반하여 일반화한 해쉬 체인 알고리즘을 제안 하였다.

YSEL 알고리즘을 실행하여 길이가 n인 해쉬 체인에서 m 간격으로 해쉬 값을 추출할 경우, $\lceil \log n \rceil$ 개의 pebble과 $m\lceil 0.5\log n \rceil$ budget을 필요로 한다. 이는 YSEL 알고리즘이 m 간격으로 해쉬 값을 출력하기 위해서 m번의 라운드를 수행해야 하기 때문이다. 기존 pebble의 개수에는 변화가 없지만 budget의 크기에 대한 요구사항은 m배 증가하게 된다. 본 연구에서는 기존 해쉬 체인 운행 알고리즘을 단순 실행하여 m 간격으로 해쉬 값을 추출할 경우에 발생하는 불필요한 budget과 pebble의 소모량을 최소화한다. 본 과제에 의해 제안되는 알고리즘은 길이가 n인 해쉬 체인 상에서 m 간격으로 해쉬 값을 출력할 경우, $\lceil \log n/m \rceil$ 개의 pebble과 $m\lceil 0.5\log n/m \rceil$ budget을 필요로 한다. 위 그림에서는 본 연구에서 제안하는 일반화한 해쉬 체인 운행(generalized hash chain traversal) 알고리즘을 기술한 것이다.

알고리즘에서 T_{setup}과 $T_{Traversal}$은 기존 알고리즘에서 표기된 $setup$ 알고리즘과 $traversal$ 알고리즘을 나타낸 것으로, 아래에 YSEL 알고리즘의 $setup$ 알고리즘과 $traversal$ 알고리즘을 보기로 제시한다. 아래 그림들에 기술된 알고리즘은 앞에서 소개된 알고리즘과 동일하며, 범용 해쉬 체인 운행 알고리즘의 모듈로 사용하기 위해 알고리즘의 입력과 출력만을 조정하였다.

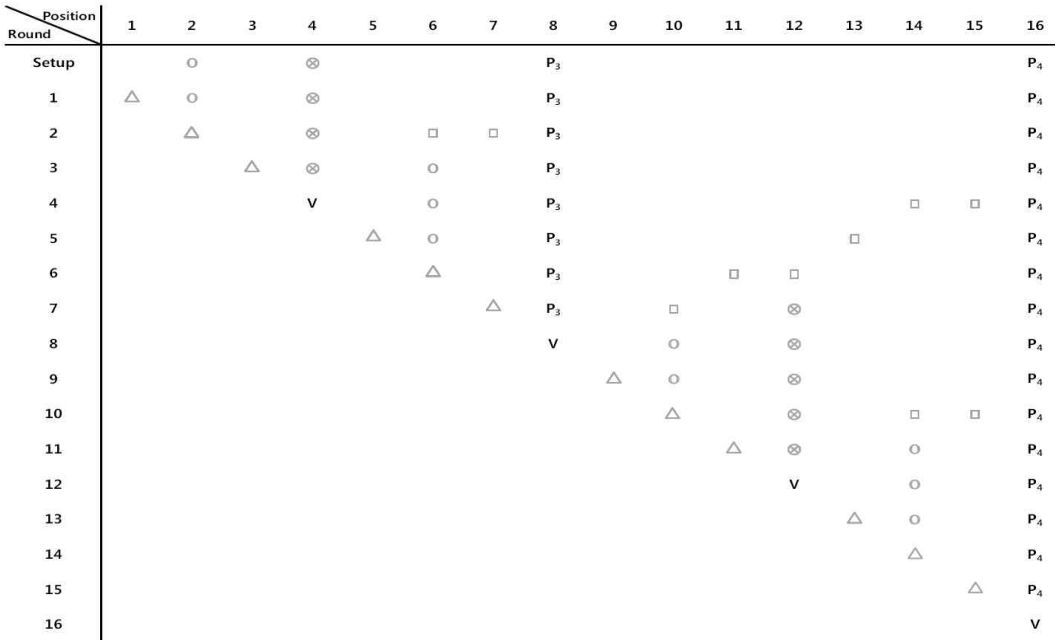


그림 3-82. Generalized hash chain traversal (m=4)

그림은 $m=4$인 경우에 대해서 YSEL 알고리즘을 수행하여 해쉬값 $\{v_4, v_8, v_{12}, v_{16}\}$을 순서대로 추출해내는 것을 나타낸 것이며, [그림 19]는 범용 해쉬 체인 운행 알고리즘에 YSEL 알고리즘을 적용하였을 경우 해쉬 값들의 출력 과정을 나타낸 것이다. 그림에서 가로축과 세로축은 각각 해쉬값의 위치와 라운드를 나타내며 이전 예제들에서와 동일하다. 그리고 본 예제에서는 이동하는 pebble의 경우 YSEL의 예제에서와 다르게 $\overline{P}$로 표기하였다. 그림에서 pebble의 이동은 YSEL에서의 pebble의 이동과 동일하지만 실제 출력되는 해쉬 값은 $\{v_4, v_8, v_{12}, v_{16}\}$만이 출력된다.

아래 그림에서는 범용 해쉬 체인 운행 알고리즘을 사용함으로써 제거되는 연산들을 ○, ⊗, □, △ 으로 표기하였다. 기호 ○는 pebble P_1을 나타내며, 기호 ⊗는 pebble P_2를 나타낸다. 그리고, 기호 □는 pebble P_1과 P_2에 의해 소모되는 해쉬 연산, 즉 pebble의 이동을 나타내며, 기호 △는 출력되지 않는 해쉬값을 나타낸다. 길이가 n인 해쉬 체인을 $m=4$인 간격으로 운행할 경우, YSEL 알고리즘은 1개의 해쉬값을 출력하기 위해 4개의 pebble과 최대 8번의 해쉬 연산을 필요로 하는 반면, 범용 해쉬 체인 운행 알고리즘은 2개의 pebble과 최대 4번의 해쉬 연산만을 필요로 한다. 더욱이 본 연구의 결과는 해쉬값을 출력하기 위해 필요한 pebble과 budget을 최소화하였을 뿐

만 아니라 해쉬 체인 운행에 소모되는 전체 해쉬 연산량을 줄임으로써, 에너지 소모량도 최소화하였다. 아래 그림에서 기호 □의 개수는 pebble P_1과 P_2의 이동시에 해쉬 연산 소모량을 나타낸 것으로 이전에 비해 10번의 해쉬 연산이 제거되었다. 그리고 기호 △는 해쉬값 출력에 소모되는 해쉬 연산량으로 홀수 위치에 존재하는 △는 이전 알고리즘에서 1번의 해쉬 연산을 소모하였다. 즉 그림에서는 8번의 해쉬 연산이 제거되었다. 결국 제시된 예제에서 범용 해쉬 체인 운행 알고리즘은 18번의 해쉬 연산을 제거함으로써 에너지 소모량을 최소화한다. 범용 해쉬 체인 운행 알고리즘은 해쉬 체인의 길이 n이 길어지고 간격 m이 커질수록 더 효율적인 성능을 제공하게 된다.

(4) 알고리즘의 구현

효율적인 해쉬 체인 운행 알고리즘은 마이크로소프트사의 Visual studio 2008의 개발환경 하에서 PolarSSL 암호 라이브러리를 사용하여 C언어를 기반으로 구현하였다. PolarSSL은 open source 암호 라이브러리로 embedded device를 위해 최적화되어 가볍고 모든 함수들은 모듈화 되어 있다. PolarSSL의 홈페이지는 “http://polarssl.org/”이고, 홈페이지에서 소스코드를 다운받을 수 있다.

구현된 프로그램은 해쉬 체인을 운행하기 위한 알고리즘 구현 부분과 해쉬 체인의 공개값을 서명하기 위한 RSA 암호 시스템 부분 및 통합 프로그램 부분으로 나누어지며, DGIST 솔루션[1] 내에 개별 프로젝트로 구성하였다. 각 프로젝트에 대한 설명은 아래와 같다.

1) 구현된 프로그램은 별도의 파일로 제출한다.

프로젝트 명	내 용	비 고
GT1	본 연구에서 제안한 알고리즘과 성능을 비교하기 위해 기존의 해쉬 체인 운행 알고리즘을 구현	해쉬 체인 운행 알고리즘
GT2	본 연구에서 제안하는 해쉬 체인 운행 알고리즘을 구현	
RSA_keygen	길이가 1024 bit인 RSA 암호키 생성	RSA 암호 시스템
RSA_encrypt	RSA_keygen에서 생성한 공개키로 평문을 암호화	
RSA_decrypt	RSA_keygen에서 생성한 비밀키로 암호문을 평문화	
RSA_sign	RSA_keygen에서 생성한 비밀키로 서명 생성	
RSA_verify	RSA_keygen에서 생성한 공개키로 서명 확인	
Prover	RSA 공개키/비밀키 생성 및 해쉬 체인을 생성하여 v_0(공개값) 및 v_0의 서명을 전송하고 주기적으로 해쉬 값을 전송	통합 프로그램
Verifier	수신된 v_0를 v_0의 서명을 통해서 확인하고 주기적으로 수신되는 해쉬 체인 값의 유효성 확인	

표 3-5. 구현 함수 종류

① 해쉬 체인 운행 알고리즘

본 연구에서 제안하는 해쉬 체인 운행 알고리즘의 성능을 확인하기 위해 GT1과 GT2를 구현하였다. GT1은 기존의 알고리즘이고, GT2는 본 연구에서 제안한 알고리즘이다. 해쉬 함수는 SHA1을 사용하였으며, GT1과 GT2의 입력 파라미터는 다음과 같다.

○ N : 해쉬 체인의 길이

○ M : 해쉬 체인의 출력 간격

○ mode : 프로그램 수행 모드. 1은 테스팅 모드이고 2는 시뮬레이션 모드로 100회 Monte carlo 시뮬레이션 수행

[그림 3-83 $N=64$, $M=4$ 일 때의 GT2의 테스팅 모드 수행 결과 예이다. 비밀값 v_n과 공개값 v_0을 출력하고 해쉬 체인의 setup 수행 시간, 해쉬 체인의 출력값과 해쉬값 생성 시간을 출력한다. 수행 시간은 CPU cycle을

측정하였다.

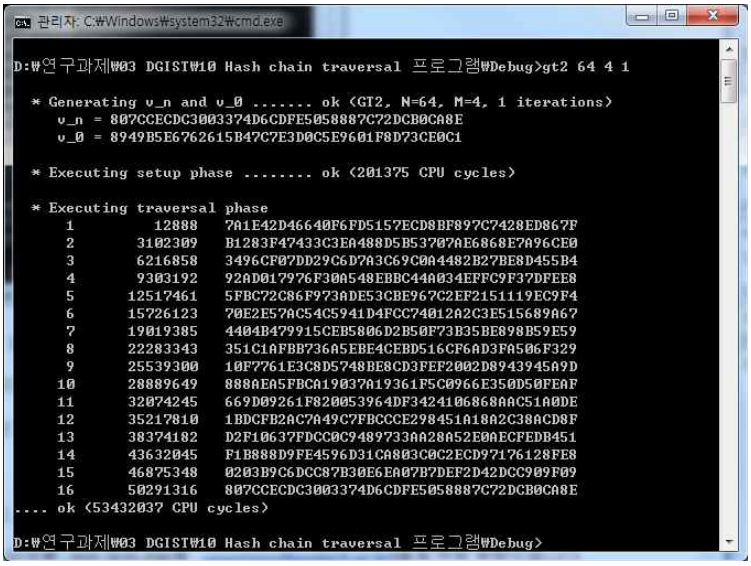


그림 3-83. GT2의 테스팅 모드 수행 예

[그림 3-84]은 GT2의 $N=64$, $M=4$ 일 때의 GT2의 시뮬레이션 모드 수행 결과 예이다. 시뮬레이션 모드에서는 100회 Monte carlo 시뮬레이션을 수행한다.

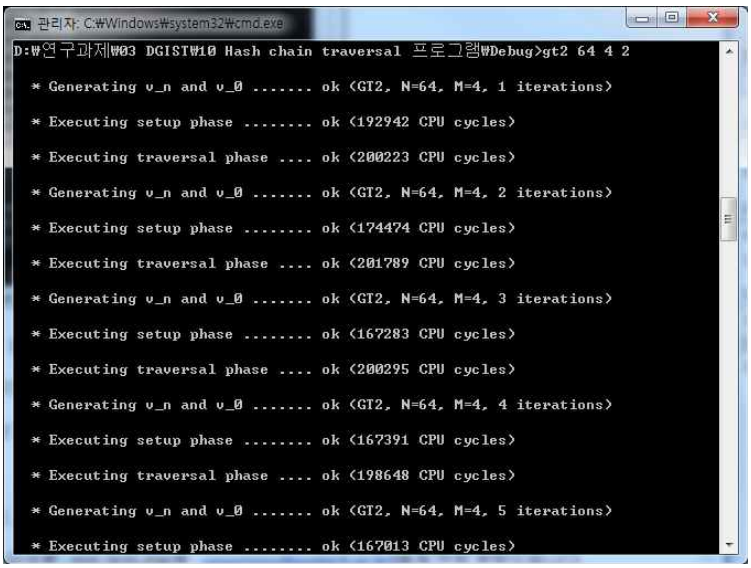


그림 3-84. GT2의 시뮬레이션 모드 수행 예

[그림 3-85]와 [그림 3-86]는 기존 알고리즘과 제안된 알고리즘의 성능을 비교한 것이다. $N=2^{16}$일 때, M을 변화시키면서 100회 Monte carlo 시뮬레이션을 수행하였다. [그림 3-86]는 해쉬 체인의 setup 시간을 비교한 것으로 기존 알고리즘은 출력 간격에 관계없이 setup 소요시간이 일정하지만 제안된 알고리즘은 해쉬 체인의 출력 간격이 늘어 나면 생성하는 pebble의 수가 감소하므로 setup 소요시간이 감소되는 것을 알 수 있다.

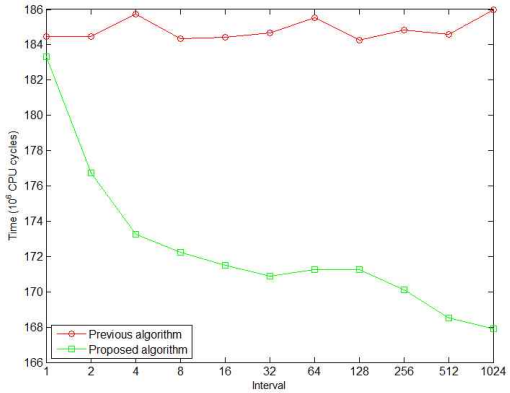


그림 3-85. 해쉬 체인 setup 시간 비교

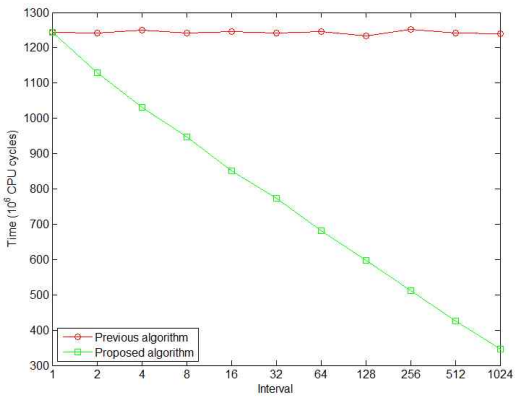


그림 3-86. 해쉬 체인 출력값 생성 시간 비교

[그림 3-86]는 해쉬 체인의 출력값 전체 생성 시간을 비교한 것으로 기존 연구는 출력 간격이 관계없이 전체 해쉬 체인 출력값 생성 시간이 일정하지만, 제안된 알고리즘은 해쉬 체인 출력 간격이 늘어남에 따라 출력값 생성 시간이 감소함을 알 수 있다. 따라서 제안된 알고리즘은 해쉬 체인 출력 간

격 늘어날수록 기존 알고리즘에 비해 더 효율적인 성능을 나타낸다.

② RSA 암호 시스템

RSA 암호 시스템은 RSA_keygen, RSA_encrypt, RSA_decrypt, RSA_sign, RSA_verfiy로 구성되며, 키 길이는 1024 bit이다.

[그림3-87]은 RSA_keygen을 실행한 결과이며, 비밀키를 저장한 rsa_private.key 파일과, 공개키를 저장한 rsa_public.key 파일을 생성한다. 출력파일의 예는 [그림 3-88]와 [그림 3-89]와 같다

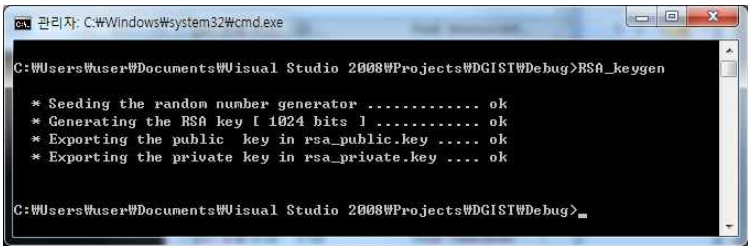


그림 3-87. RSA_keygen 실행 결과

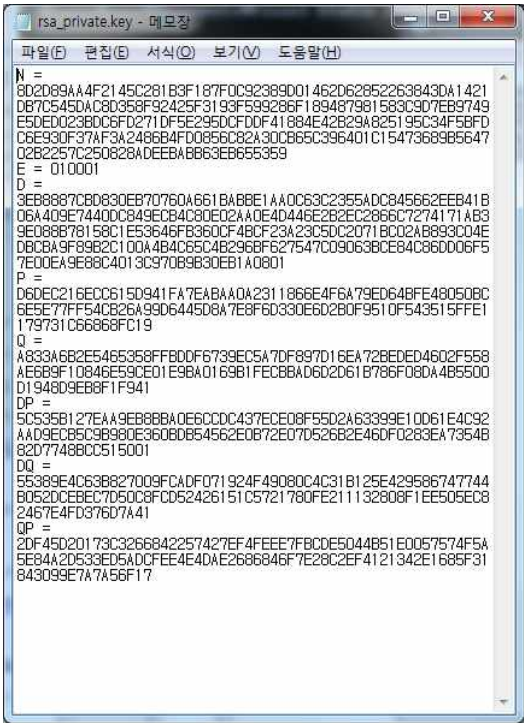


그림 3-88. rsa_private.key 내용

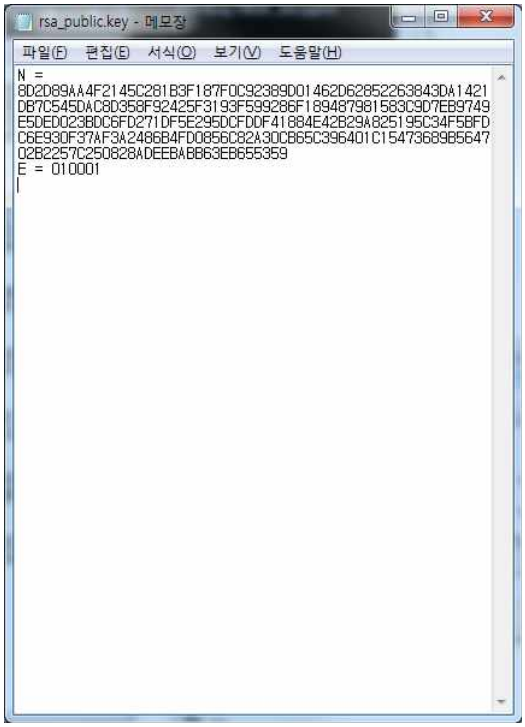


그림 3-89. rsa_public.key 내용

[그림 3-90]은 RSA_encrypt을 실행한 결과이다. 평문을 파일로 입력받아 RSA_keygen에서 생성한 공개키 rsa_public.key를 이용하여 암호화하여 그 결과를 파일로 출력한다. [그림 3-91]은 평문을 암호화-한 결과 예이다.

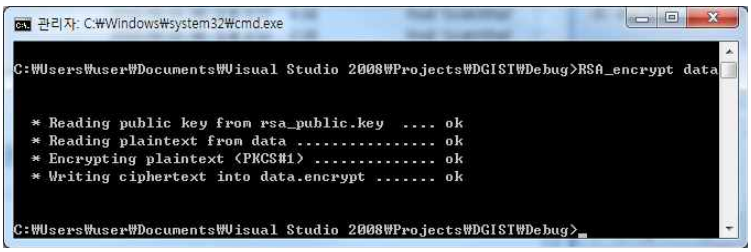


그림 3-90. RSA_encrypt 실행 결과

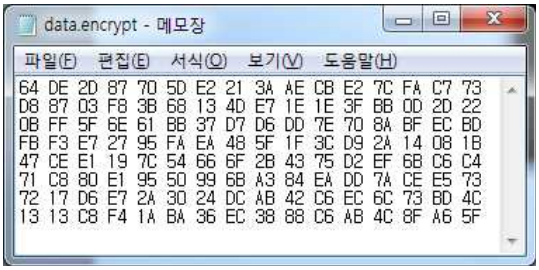


그림 3-91. 암호문 예

[그림 3-92]은 RSA_decrypt를 실행한 결과이다. 암호문을 파일로 입력받아 RSA_keygen에서 생성한 비밀키 rsa_private.key를 이용하여 복호화한 결과를 파일로 출력한다.

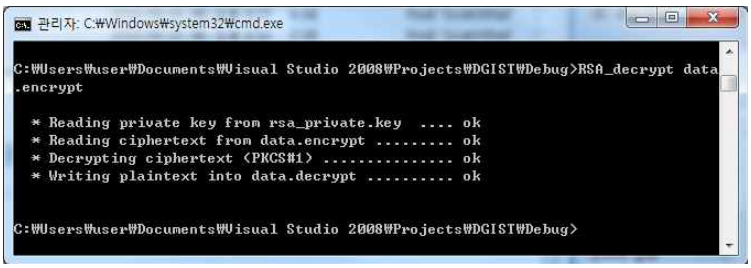


그림 3-92. RSA_decrypt 실행 결과

[그림 3-93]은 RSA_sign을 실행한 결과이다. 평문을 파일로 입력받아 RSA_keygen에서 생성한 비밀키 rsa_private.key를 이용하여 평문의 서명을 생성하여 그 결과를 파일로 출력한다. [그림 3-94]은 평문에 대한 서명을 생성한 결과 예이다.

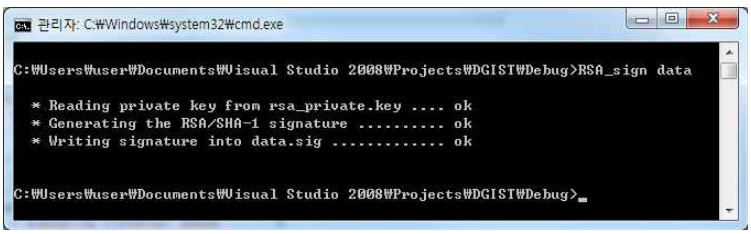


그림 3-93. RSA_sign 실행 결과

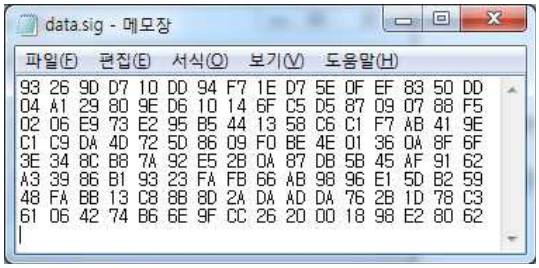


그림 3-94. 평문에 대한 서명 생성 예

[그림 3-95]은 RSA_verify를 실행한 결과이다. 평문 및 평문에 대한 서명을 파일로 입력받아 RSA_keygen에서 생성한 공개키 rsa_public.key를 이용하여 평문의 서명을 확인한다. 만약 서명이 일치하지 않으면 [그림 3-96]와 같은 출력을 발생시킨다.

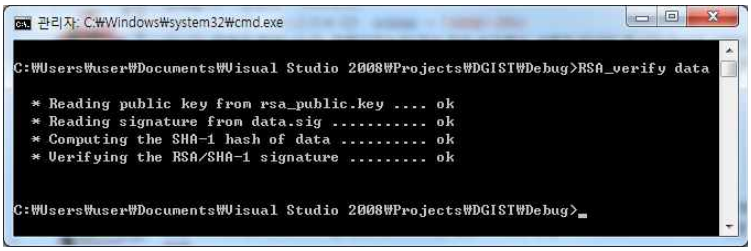


그림 3-95. RSA_verify 실행 결과

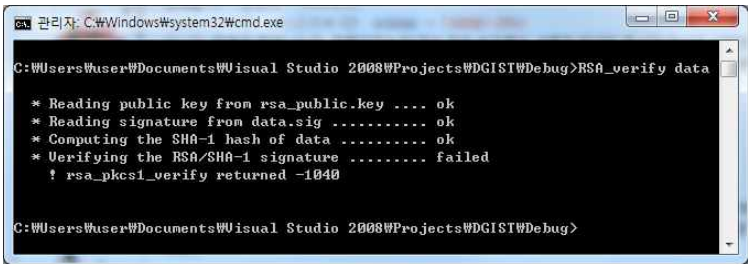


그림 3-96. 서명이 일치하지 않을 겨우 RSA_verify 실행 결과

③ 통합 프로그램

본 연구에서 제안한 해쉬 체인 운행 알고리즘과 RSA 암호 시스템을 통합하여 Prover와 Verifier를 구현하였다. Prover와 Verifier의 입력 파라미터는 다음과 같다.

○ N : 해쉬 체인의 길이

○ M : 해쉬 체인의 출력 간격

[그림 3-97]은 $N=64$, $M=4$ 일 때의 Prover 수행 결과 예이다. Prover는 Setup phase에서 RSA 비밀키/공개키 및 해쉬 체인의 비밀값 v_n과 공개값 v_0를 생성한다. v_0는 비밀키를 이용해 서명하여 Verifier에 공개한다. 그리고 Hash chain traversal phase에서 사용자의 키보드 입력에 따라 해쉬 체인 값을 Verifier에게 전송한다.

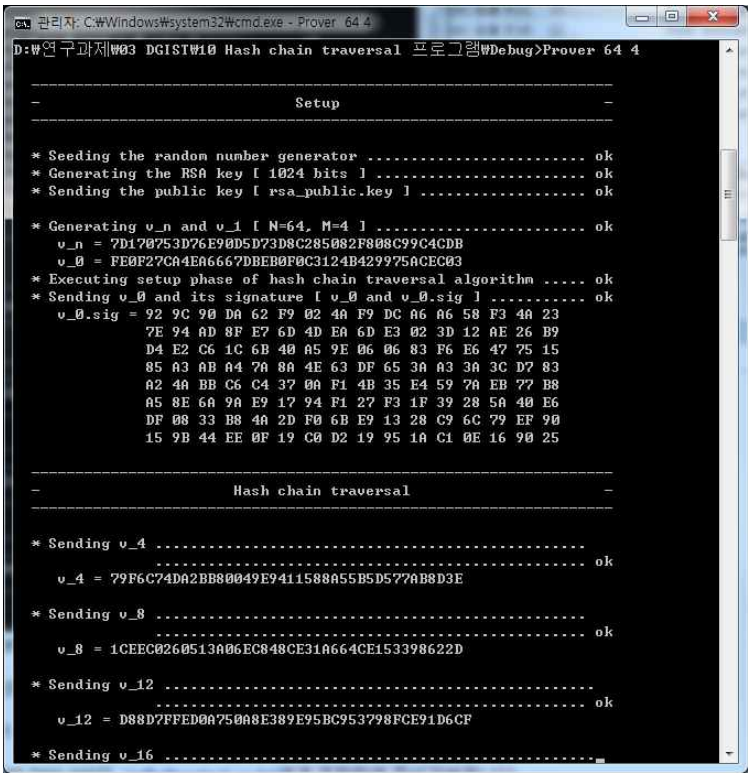


그림 3-97. Prover 수행 예

[그림 3-98]은 $N=64$, $M=4$ 일 때의 Verifier 수행 결과 예이다. Verifier는 Setup phase에서 Prover로부터 해쉬 체인 공개값 v_0, RSA 공개키,

v_0에 대한 RSA 서명을 수신하여 v_0의 유효성을 확인한다. 그리고 Hash chain traversal phase에서 사용자의 키보드 입력에 따라 Prover로부터 수신된 해쉬 체인 값의 유효성을 공개값 v_0를 이용하여 확인한다. 정상동작을 위해서는 Prover를 실행 후 Verifier를 실행하여야 하며, Hash chain traversal phase에서는 Prover에서 해쉬 체인 값을 먼저 전송한 후 Verifier에서 해쉬 체인 값을 수신하도록 하여야 한다.



그림 3-98. Verifier 수행 예

나. 양방 인증을 위한 거리 한정 알고리즘

모바일 네트워크에서 거리정보는 상호 인증 알고리즘에 사용될 수 있다. 특정 위치 내의 디바이스만이 특별한 권한을 가지는 것이다. 예를 들면, 건물 출입을 위해 암호학적 인증 프로토콜을 이용할 경우 출입자의 위치가 수 미터내에 있는지 확인하여야 한다. 그리고 차량간에 교통상황 정보를 교환할 때에도 정보의 유효성을 확인하기 위한 차량간의 위치정보 인증은 반드시 필요하다. 따라서

무선 모바일 네트워크(특히 VANET)에서 안전한 프로토콜을 위해서는 모바일 디바이스(또는 차량)의 위치를 확인하는 것은 매우 중요한 문제이다[CH05][SP07].

모바일 네트워크에서 디바이스의 위치를 확인하는 방법에는 크게 두가지가 있다. 첫 번째 방법은 신호의 크기를 확인하는 것이다. 송신 신호의 크기를 조절하여 송신 거리를 제한하는 것이다. 즉 prover가 메시지를 송신하면 verifier가 메시지를 수신 가능하면 인증을 수행하고, 메시지가 수신되지 않으면 인증을 수행하지 않는 것이다. 그러나 이러한 방법은 물리적 공격에 취약하다. 만약 공격자가 디바이스를 변형하거나 directional 안테나를 사용하여 송신 신호의 크기를 증폭하면 verifier는 보다 먼 위치에서 prover의 메시지를 수신 가능하게 된다. 두 번째 방법은 prover와 verifier사이의 거리를 메시지를 교환하는 동안의 round trip time으로 확인하는 것이다. 이 방법은 기본적으로 prover와 verifier가 challenge-response protocol을 수행한다. Verifier가 challenge와 함께 timer를 시작하면, prover는 verifier의 challenge를 수신하면 간단한 계산을 수행 후 response를 송신한다. Verifier는 response를 수신하면 timer를 멈추고 신호의 전달 속도를 고려하여 prover의 거리를 확인하는 것이다. Verifier는 challenge에 사용될 신호를 ultra-sound 또는 electromagnetic 신호를 사용할 수 있다. Ultra-sound는 전달 속도가 느려서 prover의 프로세싱 지연시간을 고려하지 않아도 되는 장점이 있지만 물리적 공격에 취약하다. 즉 소리는 공기에서 보다 금속에서 속도가 빠르므로 공격자가 소리의 전달매체를 변형하면 prover가 먼거리에서 자신의 거리가 가깝다고 주장할 수 있다. 반면에 electromagnetic은 물리적 공격에는 강하지만 신호의 전달 속도가 빨라서 정확한 위치 정보를 알기 위해서는 prover의 프로세싱 지연시간을 최소로 해야 한다. Fast bit exchange가 이 문제의 해결책이 될 수 있다.

본 연구에서는 차량 상호간에 신뢰할 수 있는 거리 정보를 제공하기 위해 fast bit exchange 기반의 거리 한정 프로토콜(distance bounding protocol)을 설계 및 분석하였다. (1)절에서는 공격 유형을 소개하고, (2)절에서는 기존 연구 내용을 나타내었다. 그리고 (3)절과 (4)절에서는 본 연구에서 설계한 거리 한정 알고리즘을 기술하고 분석하였다.

(1) 공격 유형

Distance bounding protocol에 대한 공격 유형에는 distance fraud attack, mafia fraud attack, terrorist fraud attack이 있다. 본 절에서는 각각의 공격에 대해 기술할 것이다.

그림은 distance fraud attack을 나타낸 것이다. 여기서 P′는 dishonest

prover, V를 verifier이다. 일반적으로 verifier는 특정한 거리내에 prover가 위치하는지 확인하고자 한다. 그러나 distance fraud attack은 dishonest prover가 verifier에게 실제 위치보다 가까이에 있다고 주장하는 공격이다.

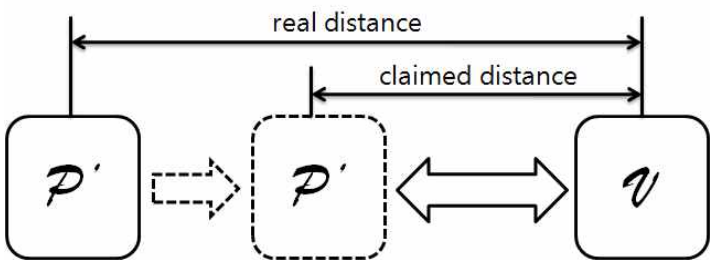


그림 3-99. distance fraud attack

Mafia fraud attack은 man-in-the-middle attack 또는 replay attack이라고도 한다. 아래 그림에서는 mafia fraud attack을 나타낸 것이다. 여기서, P와 V는 prover와 verifier이고, P'와 V'는 malicious prover, verifier, A는 adversary이다. Adversary A는 P'와 V'가 협업하는 형태로 모델링 된다. Malicious prover P'는 honest prover P와 서로 통신하고 동시에 malicious verifier V'는 honest prover P와 통신하여, A가 P의 비밀과 관련된 정보를 이용하여 V에게 자신이 P라고 확신시키는 것이다. 즉 adversary A 자신이 P에게는 V라고 주장하고 V에게는 P라고 주장하는 공격이다. 이때 adversary A는 prover V에 가까이 위치한다.

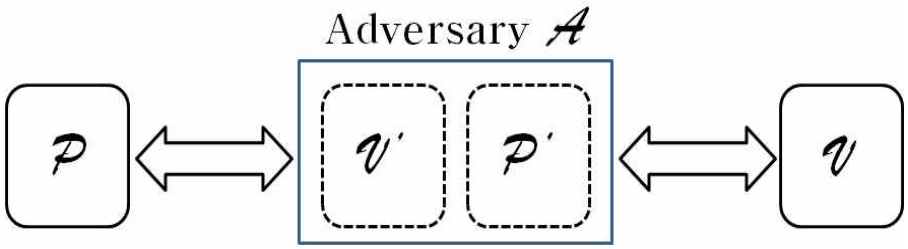


그림 3-100. Mafia fraud attack

Terrorist fraud attack은 mafia fraud attack의 확장이다. 아래 그림에서 보는 바와 같이 terrorist fraud attack은 dishonest prover P'가 다른

dishonest prover P''를 이용하여 verifier V에게 자신의 위치가 P''에 있다고 주장하는 공격이다. 여기서 P''와 P'는 서로 협업은 수행하지만, P''는 P'의 secret key를 모른다고 가정한다. 왜냐하면 P''이 P'의 secret key를 안다고 가정하면 P''와 P'를 구분할 수 없기 때문이다. Terrorist fraud attack에 대한 방어는 hardware적이 접근이 일반적이다. Prover의 secret key를 trusted hardware를 이용하여 안전하게 보호하면 다른 dishonest prover와 협업을 수행할 수 없기 때문이다. 그러므로 본 연구과제에서 distance fraud attack과 mafia fraud attack을 방어하기 위한 연구 수행에 초점을 맞춘다.

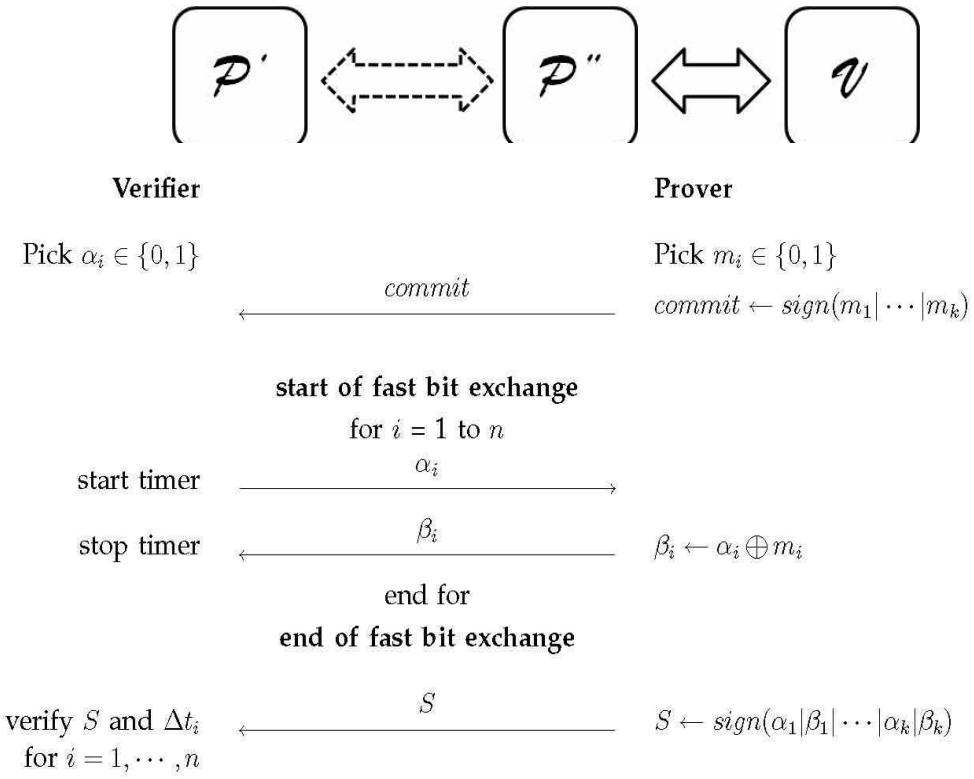


그림 3-101. BC distance bounding protocol

(2) 기존 연구

이 절에서는 최근까지 연구된 distance bounding protocol에 대한 조사 결과를 기술한다. 이 절에 기술된 각 distance bounding protocol은 다음 내

용을 기본적으로 가정한다.

□ 각 프로토콜의 security parameter는 n이다. 즉, 각 프로토콜의 security level은 n에 대한 함수로 표현된다.

□ 각 프로토콜의 prover와 verifier는 기본적으로 동일한 암호체계를 가진다. 다시 말해 동일한 asymmetric 또는 symmetric encryption, 서명 기법(signature scheme), 단방향 함수(one-way function) 등을 가진다.

□ Prover와 verifier는 모두 안전한 난수 발생기(secure random number generator)를 가진다.

□ 모든 프로토콜은 fast bit exchange를 기반으로 설계된다.

① BC 프로토콜

Brands와 Chaum은 distance bounding protocol 분야의 개척자이다. Brands와 Chaum은 mafia fraud attack을 방어 할 수 있는 몇 가지의 프로토콜을 제안하였다[BC93]. 위 그림에서는 Brands가 제안한 distance bounding protocol이다. 이것은 준비 단계, fast bit exchange 단계, 확인 단계로 나눌 수 있다. 준비 단계에서는 $i=1,\cdots,n$에 대해 prover는 $m_i \in \{0,1\}$, verifier는 $\alpha_i \in \{0,1\}$의 n bit의 랜덤값을 생성하고, prover는 생성한 랜덤값에 대해 서명하여 $commit$를 전송한다. fast bit exchange 단계에서는 verifier가 prover에게 α_i값을 전송하면 prover는 $\beta_i = \alpha_i \oplus m_i$를 계산하여 verifier에게 β_i값을 보낸다. 이때 verifier는 time를 이용하여 round trip time Δt_i을 측정한다. 확인 단계에서는 prover는 α_i와 β_i값을 결합한 $2n$ bit의 문자열에 대해 서명하여 S를 verifier에 송신한다. 최종적으로 verifier는 prover가 전송한 서명된 문자열과 측정된 round trip time Δt_i을 계산하여 인증을 수행하게 된다. 이 프로토콜은 prover가 송신하는 서명 메시지 S 때문에 fast bit exchange 단계의 각 round에서 adversary가 distance fraud attack에 대해 성공할 확률과 mafia fraud attack을 성공할 확률은 모두 $\frac{1}{2}$이다. 즉 프로토콜을 모두 수행한 후 adversary가 distance fraud attack과 mafia fraud attack을 성공할 확률은 $\left(\frac{1}{2}\right)^n$이다. 그러나 이 프로토콜은 prover가 $commit$와 S와 같은 서명된 메시지를 생성하기 위해 공개키 기반의 암호 알고리즘을 이용하였다. 다시 말해 공개키 기반의 암호 알고리즘이 prover와 verifier의 프로세싱 오버헤드를 증가시켜 프로토콜 수행이 느리다.

② HK 프로토콜

Hencke와 Kuhn은 RFID 시스템에 가장 많이 사용되는 distance bounding protocol (HK protocol)을 제안하였다[HK05]. HK 프로토콜은 prover와 verifier가 대칭키 K를 공유하고 있다고 가정한다. 아래 그림에서는 HK distance bounding protocol을 나타낸 것이다. 준비 단계에서 prover와 verifier는 각각 랜덤값 N_P와 N_V를 생성하여 서로 교환한다. 그리고 해쉬 함수를 이용하여 n bits 문자열 v^0와 v^1을 생성한다. Fast bit exchange 단계에서는 verifier가 랜덤 bit C_i를 생성하여 prover에게 전송한다. Prover는 C_i가 0이면 v_i^0를 C_i가 1이면 v_i^1을 R_i에 저장하여 verifier에 송신한다. 이러한 과정을 n번 반복하며 round trip time을 측정한다. 확인 단계에서 verifier는 수신된 R_i와 측정된 round trip time Δt_i이 유효한지 확인하여 prover의 인증을 수행한다. 이 프로토콜은 fast bit exchange 단계의 각 round에서 adversary가 distance fraud attack과 mafia fraud attack을 성공할 확률은 $\frac{1}{2}$이 아니고 $\frac{3}{4}$이다. 왜냐하면 distance fraud attack에 대해서 i round 일 때, $v_i^0 = v_i^1$이면 adversary에 올바른 response R_i를 송신할 수 있고, $v_i^0 \neq v_i^1$이면 $\frac{1}{2}$의 확률로 올바른 response를 송신할 수 있으므로, $\frac{1}{2} \cdot 1 + \frac{1}{2} \cdot \frac{1}{2} = \frac{3}{4}$이다. 그리고 mafia fraud attack에 대해 adversary가 준비 단계와 fast bit exchange 단계 사이에 prover에게 C_i'를 송신할 수 있기 때문이다. 이렇게 하면 adversary가 n bits에 대한 기록을 가질 수 있다. 예를 들면, adversary가 prover에게 $C_i' = 0$을 n번 송신하면, adversary는 v^0를 알 수 있다. 이 경우 adversary는 verifier가 $C_i = 0$을 송신하면 항상 올바른 응답을 할 수 있다. 반면에 verifier가 $C_i = 1$을 adversary에게 송신하면 adversary는 $\frac{1}{2}$의 확률로 올바른 응답을 할 수 있다. 그러므로 HK 프로토콜은 adversary의 distance fraud attack과 mafia fraud attack에 대해 $\left(\frac{3}{4}\right)^n$의 확률로 방어에 실패한다.

③ MP 프로토콜

Munilla와 Peinado는 adversary의 공격 성공확률을 줄이기 위해

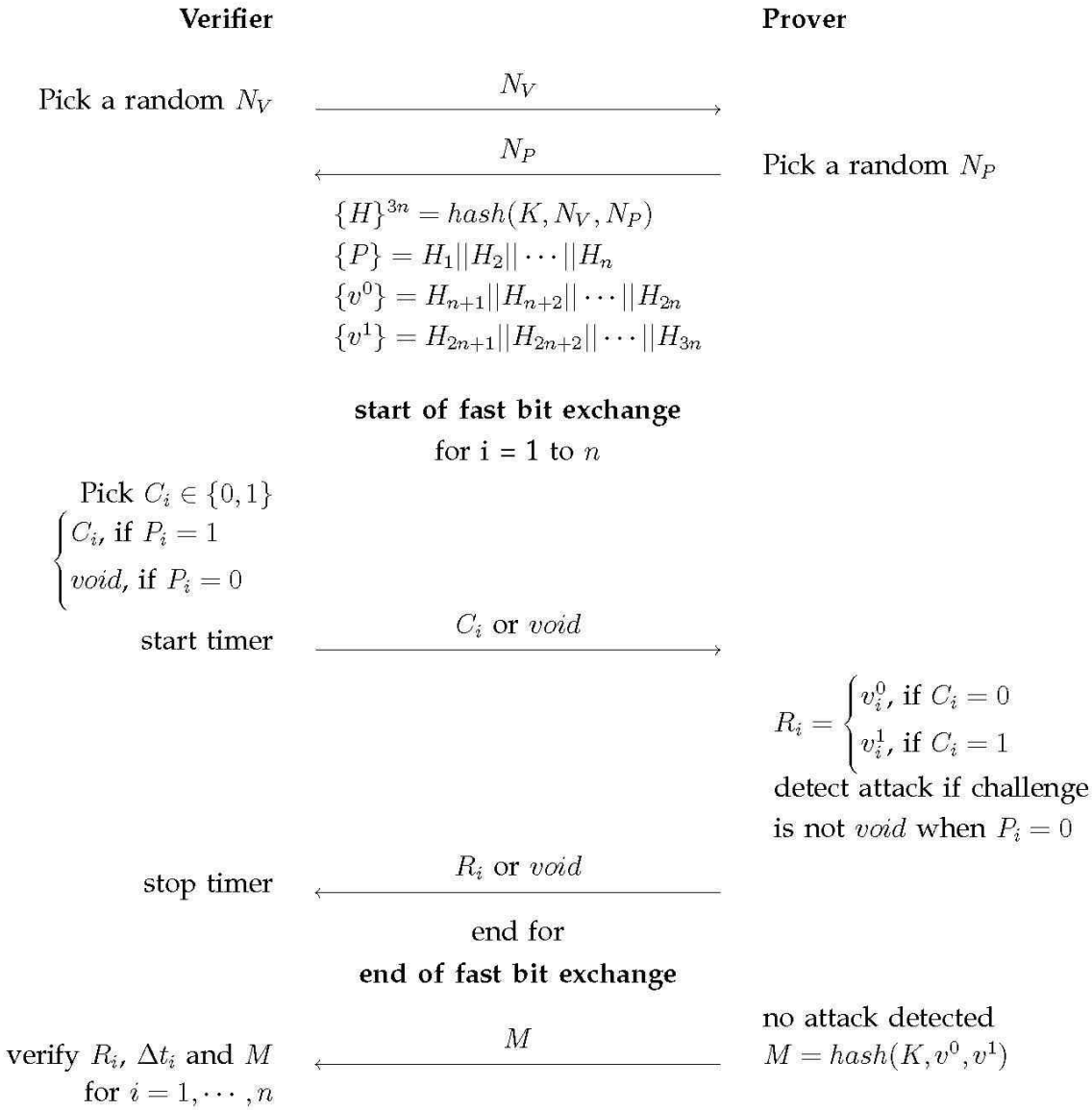


그림 3-102. MP distance bounding protocol

void-challenge를 적용하여 HK distance bounding protocol을 개선하였다 [MP08]. 아래 그림은 Munilla와 Peinado가 제안한 distance bounding protocol (MP 프로토콜)을 나타낸 것으로 verifier는 full-challenge와 void-challenge의 두 가지 형태의 challenge를 수행한다. 준비 단계에서 prover와 verifier는 각각 랜덤값 N_P와 N_V를 생성하여 서로 교환한다. 그리

고 해쉬 함수를 이용하여 n bits 문자열 P와 v^0, v^1을 생성한다. 여기서 P는 void-challenge 여부를 나타낸다. 즉 fast bit exchange 단계에서 $P=1$면 full-challenge로 verifier는 prover에 fast bit exchange를 요청한고, $P=0$이면 verifier는 void-challenge로 prover에게 아무것도 전송하지 않는다. MP 프로토콜의 장점은 void-challenge를 이용하여 prover가 adversary의 공격 진행 여부를 확인할 수 있다는 것이다. Prover가 adversary의 공격을 감지하면 프로토콜의 통신을 중단한다. 최종적으로 확인 단계에서는 prover가 fast bit exchange단계에서 adversary의 공격이 확인되지 않았으면 $M=hash(K,v^0,v^1)$을 verifier에게 송신한다. Munilla와 Peinado는 자신들의 프로토콜에 대해 adversary가 mafia fraud attack을 성공할 확률이 $\left(\frac{9}{16}\right)^n$이라고 주장하였지만, Avoine 등은 MP 프로토콜에서 adversary의 공격 성공 확률을 $\left(\frac{39}{64}\right)^n$으로 재계산 하였다[ABKLM09]. Distance fraud attack에 대해서는 Hancke의 프로토콜과 같이 $\left(\frac{3}{4}\right)^n$의 확률로 adversary가 공격에 성공한다. 이 프로토콜은 추가적인 서명 메시지 없이 HK distance bounding protocol과 비교해서 adversary의 공격 성공 확률을 감소시켰다. 그러나 MP 프로토콜은 세 가지 상태(즉, 0, 1, void)를 필요로 해서 구현을 어렵게 하는 단점이 있다.

④ TP 프로토콜

Tu와 Piramuthu는 MP 프로토콜처럼 adversary의 공격 성공 확률을 감소시키는 distance bounding protocol을 제안하였다[TP07]. 아래 그림은 Tu와 Piramuthu가 제안한 distance bounding protocol이다. Prover와 verifier는 long term secret key K를 공유하고 있다고 가정한다. 준비 단계에서 verifier는 랜덤값 N_V를 생성하여 $K\oplus N_V$를 prover에 송신한다. 또한 prover도 랜덤값 N_P를 생성하여 $K\oplus N_P$를 verifier에 송신한다. prover와 verifier는 이 정보를 이용하여 session key $x=hash(K,N_P,K\|N_V)$를 생성하고, session key와 secret key를 이용하여 $\gamma=x\oplus K$를 계산한다. Tu와 Piramuthu가 제안한 TP 프로토콜은 fast bit exchange 단계에서 2개의 loop를 가진다. Outer loop는 (u,v)에 대해 4가지 경우(즉, (N_V,N_V), (N_P,N_P), (N_P,N_V), $(N_P\oplus N_V,N_P\oplus N_V)$)에 대해 수행하고, inner loop는 각 outer loop에 대해 $n/4$번씩 수행한다. Verifier는 각 round에서 response R_i와 round trip time

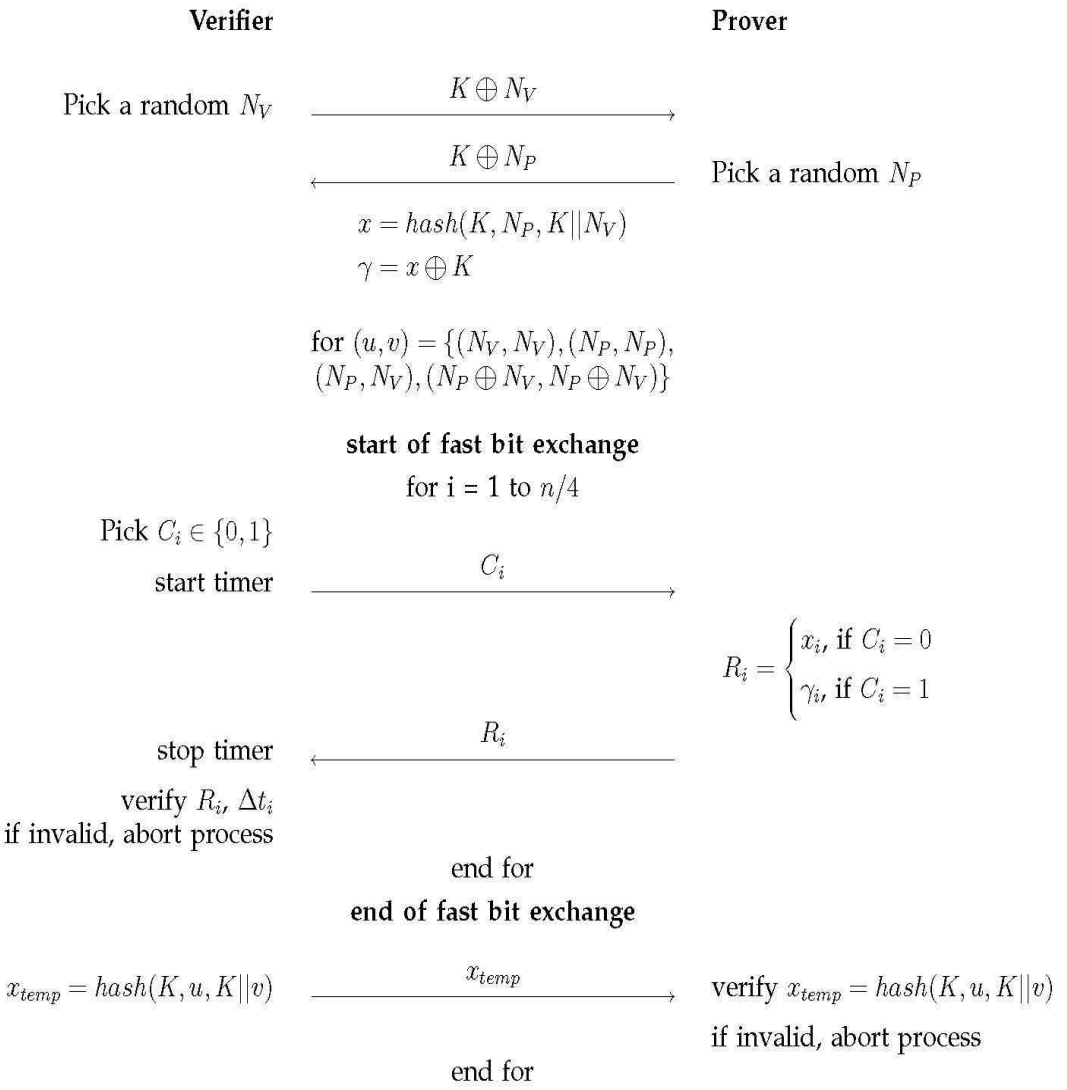


그림 3-103. TP distance bounding protocol

Δt_i를 검사하여 prover 유효한지 확인한다. prover는 각 inner loop의 종료 후 verifier가 전송하는 x_{temp}를 검사하여 verifier가 유효한지 확인한다. TP distance bounding protocol에 대해 adversary가 distance fraud attack에 대해 성공할 확률은 $\left(\frac{3}{4}\right)^n$이고, mafia fraud attack을 성공할 확률은 $\left(\frac{9}{16}\right)^n$이다. 왜냐하면, distance fraud attack에 대해서는 앞의 프로토콜에서 설명한

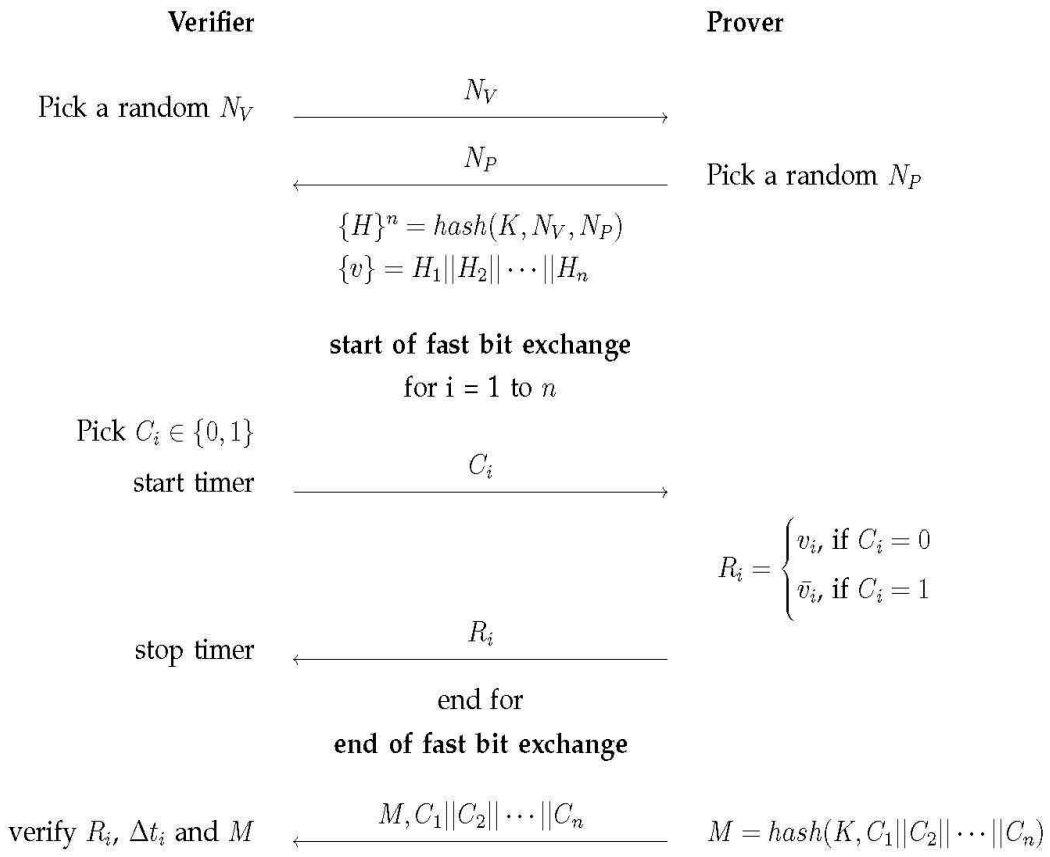


그림 3-104. False accept rate을 최소화한 distance bounding protocol

이유와 같으며, mafia fraud attack에 대해 adversary는 x_{temp}를 계산할 수 없기 때문에 첫 outer loop에 대해서만 prover에게 challenge-response를 사전에 수행할 수 있기 때문이다. 그러나 TP 프로토콜은 active adversary에 대해 안전하지 않다[KAKSP09]. 즉, adversary가 verifier가 prover에 보내는 challenge C_i를 toggle하여 prover에게 전송한 후 prover의 response R_i를 verifier에 중계한다. verifier가 프로토콜을 계속 수행하면 x_i와 $\gamma_i = x_i \oplus K_i$가 같다는 의미이므로, adversary는 $K_i = 0$임을 알 수 있고, 반대로 verifier가 프로토콜 수행을 중지하면 adversary는 $K_i = 1$임을 알 수 있다. 따라서 adversary는 비밀키 K를 추출할 수 있다.

지금까지 distance fraud attack과 mafia fraud attack에 대해 adversary의 공격 성공 확률을 줄이기 위한 distance bounding attack을 살펴보았다.

(3) False accept rate을 최소화한 거리 한정 알고리즘

본 절에서는 distance fraud attack과 mafia fraud attack에 대한 adversary의 공격 성공 확률을 최소로 하는 distance bounding protocol을 제안한다. 그림은 본 연구에서 제안하는 distance bounding protocol이다. 준비 단계에서 prover와 verifier는 각각 랜덤값 N_P와 N_V를 생성하여 서로 교환한다. 그리고 해쉬 함수를 이용하여 n bits 문자열 v를 생성한다. Fast bit exchange 단계에서는 verifier가 랜덤 bit C_i를 생성하여 prover에게 전송한다. Prover는 C_i가 0이면 v_i를 C_i가 1이면 $\overline{v_i}$를 R_i에 저장하여 verifier에 송신한다. 이러한 과정을 n번 반복하며 round trip time을 측정한다. 확인 단계에서 prover는 verifier로부터 수신한 모든 challenge를 이용하여 $M = hash(K, C_1 \| C_2 \| \cdots \| C_n)$을 계산 후 verifier에게 M과 $C_1 \| C_2 \| \cdots \| C_n$를 송신한다. Verifier는 수신된 R_i와 측정된 round trip time Δt_i 및 M이 유효한지 확인하여 prover의 인증을 수행한다.

① Distance fraud attack에 대한 adversary의 공격 성공 확률

그림에서 나타낸 distance bounding protocol에 대한 adversary의 distance fraud attack 성공 확률은 $\left(\frac{1}{2}\right)^n$ 이다. 왜냐하면, distance fraud attack은 prover가 verifier의 challenge를 받기 전에 response를 보내야만 prover의 실제 위치보다 verifier에 가까이 위치한다고 주장할 수 있는데 항상 $v_i \neq \overline{v_i}$ 이므로, prover는 $\frac{1}{2}$의 확률로 verifier의 challenge에 올바른 response를 보낼 수 있기 때문이다.

② Mafia fraud attack에 대한 adversary의 공격 성공 확률

Adversary는 prover와 verifier가 공유하고 있는 secret key K를 알지 못하므로 확인 단계에서 prover가 계산하는 M을 생성할 수 없다. 즉 adversary는 mafia fraud attack을 위해 distance bounding protocol의 준비 단계와 fast bit exchange 단계사이에 prover와 사전에 challenge-response를 수행하지 못한다. 그러므로 그림에서 나타낸 distance bounding protocol에 대한 adversary의 mafia fraud attack 성공 확률은 $\left(\frac{1}{2}\right)^n$ 이다.

프로토콜	adversary 공격 성공 확률		암호 방식	비고
	distance fraud attack	mafia fraud attack		
BC	$\left(\frac{1}{2}\right)^n$	$\left(\frac{1}{2}\right)^n$	공개키	계 산 량 이 많음
HK	$\left(\frac{3}{4}\right)^n$	$\left(\frac{3}{4}\right)^n$	대칭키	
MP	$\left(\frac{3}{4}\right)^n$	$\left(\frac{39}{64}\right)^n$	대칭키	구현이 어려움
TP	$\left(\frac{3}{4}\right)^n$	$\left(\frac{9}{16}\right)^n$	대칭키	안전하 지 않음
proposed	$\left(\frac{1}{2}\right)^n$	$\left(\frac{1}{2}\right)^n$	대칭키	

③ Contribution

그림의 distance bounding protocol과 기존에 제안된 distance bounding protocol에 대한 adversary의 공격 성공 확률은 다음 표에서 비교하였다. 여기서 본 절에서 제안한 프로토콜은 대칭키 기반의 암호 방식을 사용했지만 성능은 공개키 기반의 암호 방식을 사용한 Brands와 동일한 것을 알 수 있다. 다시 말해, 본 절에서 제안한 distance bounding protocol은 대칭키 방식으로 공개키 방식에 비해 오버헤드가 적고 안전하며 구현이 쉽지만 성능은 공개키 기반의 distance bounding protocol과 동일하다.

(4) Mutual challenge를 이용한 거리 한정 알고리즘

이 절에서는 prover와 verifier의 mutual challenge를 통해 mafia fraud attack에 대한 adversary의 공격 성공 확률을 최소화한 distance bounding protocol을 제안한다. 본 연구에서 제안하는 프로토콜은 아래 그림에 나타내었다. 준비 단계에서 prover와 verifier는 각각 랜덤값 N_P와 N_V를 생성하여 서로 교환한다. 그리고 해쉬 함수를 이용하여 n bits 문자열 D와 v^0, v^1을 생성한다. 여기서 D는 challenge를 수행할 주체를 결정하는 변수로 $D=1$이면 verifier가 challenge를 수행하고, $D=0$이면 prover가 challenge를 수행한

다. Fast bit exchange 단계 동안 prover와 verifier는 채널을 모니터링해서 신호의 충돌이 발생하는지 확인하여 충돌이 확인되면 프로토콜의 수행을 중단한다. 그리고, prover(또는 verifier)는 랜덤값을 생성하여 challenge를 수행하고, verifier(또는 prover)는 수신된 값이 0이면 v_i^0을 1이면 v_i^1을 response 값으로 전송한다. 최종적으로 각 round에서 round trip time Δt와 response 값을 확인하여 프로토콜 수행을 계속할 것인지를 결정하고, 프로토콜이 수행 완료되면 prover와 verifier는 서로에 대한 인증을 수행한다.

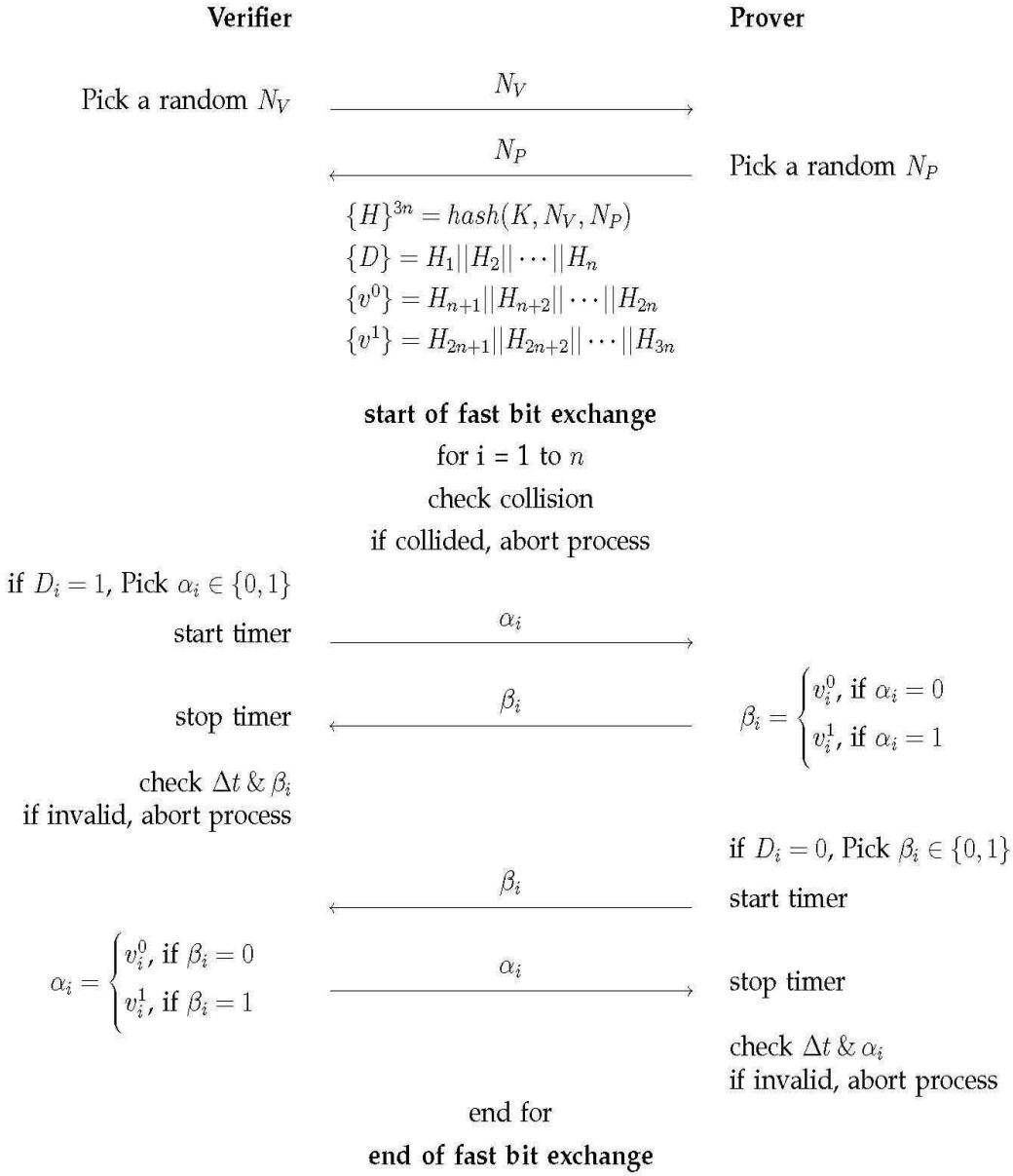


그림 3-105. Mutual challenge를 이용한 distance bounding protocol

① Distance fraud attack에 대한 adversary의 공격 성공 확률

그림에서 나타낸 distance bounding protocol에 대하여 adversary의 각 round에서 distance fraud attack 성공 확률은 다음식과 같이 구할 수 있다.

$$
\begin{aligned}
P_{distance\ fraud\ attack} &= P_{D=1\&decieve} + P_{D=0\&decieve} \\
&= \frac{1}{2}\left(P_{v_i^0 = v_i^1 \& decieve} + P_{v_i^0 \neq v_i^1 \& decieve}\right) + \frac{1}{2} \cdot 1 \\
&= \frac{1}{2}\left(\frac{1}{2} \cdot 1 + \frac{1}{2} \cdot \frac{1}{2}\right) + \frac{1}{2} \\
&= \frac{7}{8}
\end{aligned}
$$

adversary의 distance fraud attack $P_{distance\ fraud\ attack}$ 성공 확률은 $D=1$일 때 공격 성공 확률과 $D=0$일 때 공격 성공 확률의 합으로 나타낼 수 있다. D가 1 또는 0이 될 확률은 각각 $\frac{1}{2}$이고, $D=0$이면 prover가 challenge를 수행하므로 공격 성공 확률은 1이다. 반면 $D=1$이면 verifier가 challenge를 수행하며 이것은 다시 $v_i^0 = v_i^1$일 때와 $v_i^0 \neq v_i^1$일 때 adversary가 공격 성공할 확률로 표현된다. $v_i^0 = v_i^1$이면 adversary가 항상 공격에 성공하고 $v_i^0 \neq v_i^1$이면 공격 성공 확률은 $\frac{1}{2}$이다. 따라서 위의 식에서 보는 바와 같이 각 round에 대한 공격 성공 확률 $P_{distance\ fraud\ attack}$는 $\frac{7}{8}$이다. 그러므로 그림에서 나타낸 프로토콜의 distance fraud attack에 대한 n round 수행 후 adversary의 성공 확률은 $\left(\frac{7}{8}\right)^n$이다.

② Mafia fraud attack에 대한 adversary의 공격 성공 확률

Mafia fraud attack은 두 가지 형태가 존재한다. 하나는 adversary가 verifier와 랜덤 challenge를 통하여 공격하는 것이고, 다른 하나는 준비 단계와 fast bit exchange 단계 사이에 adversary가 prover와 통신하여 사전에 송수신 정보를 수집한 후에 공격하는 것이다. 우선 랜덤 challenge의 의한 공격 성공 확률은 다음 식과 같다.

$$
\begin{aligned}
P_{mafia\ fraud\ attack}^{no-ask} &= P_{no-collision\&decieve} + P_{collision\&decieve} \\
&= \frac{1}{2}\left(P_{v_i^0 = v_i^1 \& not-detect} + P_{v_i^0 \neq v_i^1 \& not-detect}\right) + \frac{1}{2} \cdot 0 \\
&= \frac{1}{2}\left(\frac{1}{2} \cdot 1 + \frac{1}{2} \cdot \frac{1}{2}\right) + \frac{1}{2} \cdot 0 \\
&= \frac{3}{8}
\end{aligned}
$$

adversary의 mafia fraud attack $P_{mafia\ fraud\ attack}$ 성공 확률은 verifier가 충돌을 감지했을 때와 그렇지 않을 때로 구분된다. 충돌이 확인 되면 공격 성공 확률은 0이 되고, 충돌이 확인되지 않으면 다시 $v_i^0 = v_i^1$일 경우와 $v_i^0 \neq v_i^1$인 경우의 adversary가 공격 성공할 확률로 표현된다. $v_i^0 = v_i^1$이면 공격 성공 확률이 1이고, 그렇지 않으면 공격 성공 확률이 0이다. 따라서 위의 식에서 나타낸 바와 같이 랜덤 challenge(no-ask)에 의한 adversary의 각 round에 대한 공격 성공 확률 $P_{mafia\ fraud\ attack}^{no-ask}$은 $\frac{3}{8}$이다. 그러므로 n round 수행 후 adversary의 공격 성공 확률은 $\left(\frac{3}{8}\right)^n$이다.

다음으로 준비 단계와 fast bit exchange 단계 사이에 adversary가 prover와 통신하여 사전에 송수신 정보를 수집한 후에 공격을 수행하면 adversary의 공격 성공 확률은 다음 식과 같다.

$$
\begin{aligned}
P_{mafia\ fraud\ attack}^{ask} &= \left[P_{D=1\&decieve} + P_{D=0\&decieve}\right] \cdot P_{success-ask} \\
&= \left[\frac{1}{2}\left(P_{C_i = C_i'\&not-detect} + P_{C_i \neq C_i'\&not-detect}\right) + \frac{1}{2} \cdot 1\right] \cdot \left(\frac{3}{8}\right)^i \\
&= \left[\frac{1}{2}\left(\frac{1}{2} \cdot 1 + \frac{1}{2} \cdot \frac{1}{2}\right) + \frac{1}{2} \cdot 1\right] \cdot \left(\frac{3}{8}\right)^i \\
&= \frac{7}{8} \cdot \left(\frac{3}{8}\right)^i
\end{aligned}
$$

우선 adversary는 verifier와 fast bit exchange를 수행하기 전에 prover와 fast bit exchange를 수행하여야 한다. 본 절에서 제안된 프로토콜은 mutual challenge를 통한 상호 인증을 수행하므로 adversary는 prover와의 통신도 인증을 통과하여야만 할 수 있다. 그러므로 adversary가 prover에게 i번째 round까지 정보를 획득할 확률 $P_{success-ask}$은 adversary가 i번째 round까지 랜덤 challenge를 성공할 확률과 동일하다. 즉 $\left(P_{mafia\ fraud\ attack}^{no-ask}\right)^i = \left(\frac{3}{8}\right)^i$이다. Adversary가 prover에게 정보의 획득에 성공하면, adversary가 verifier에 대한 공격 성공 확률은 $D=1$과 $D=0$일 때로 구분할 수 있다. 다시 말해 verifier가 challenge를 수행할 경우와 prover가 challenge를 수행할 경우이다. Prover가 challenge를 수행해야 할 경우에는 adversary 공격 성공 확률은 1

이며, verifier가 challenge를 수행할 경우에는 또다시 adversary가 prover에 대해 사전에 수행한 challenge C_i'와 verifier가 시도하는 challenge C_i가 일치할 경우와 일치하지 않을 경우가 있으며, 일치할 경우는 1, 일치하지 않을 경우는 랜덤 response이므로 $\frac{1}{2}$의 공격 성공 확률값을 가진다. 따라서 adversary가 사전에 prover와 통신하여 정보를 수집 후 mafia fraud attack을 수행하면 i 번째 round에서 $\frac{7}{8} \cdot \left(\frac{3}{8}\right)^i$의 공격 성공 확률값을 가진다. 그러므로 n round 수행 후 adversary의 공격 성공 확률은 $\left(\frac{7}{8}\right)^n \prod_{i=1}^{n} \left(\frac{3}{8}\right)^i$이다. ?절에서 설명한 기존 연구에서는 adversary가 verifier에게 인증으로 요청하기 전에 prover와 사전 통신을 통하여 정보를 수집하면 adversary의 공격 성공 확률이 높아지지만, 이 절에서 제안한 distance bounding protocol은 verifier와 prover가 상호 인증을 수행하기 때문에 adversary가 공격을 위해 prover로부터 사전에 정보를 수집하려고 시도하면 오히려 공격 성공확률이 감소함을 알 수 있다.

③ Contribution

그림의 distance bounding protocol과 기존에 제안된 distance bounding protocol에 대한 adversary의 mafia fraud attack의 공격 성공 확률은 그림에서 비교하였다. 이 절에서 제안한 distance bounding protocol은 비록 distance fraud attack에 대해서는 adversary의 공격 성공 확률이 이전 연구에 비해 높지만, adversary의 dominant 공격인 mafia fraud attack에 대해서는 공격 성공 확률이 획기적으로 개선된 것을 알 수 있다.

(5) 릴레이 공격에 강한 효율적인 거리 한정 프로토콜

본 연구에서는 릴레이 공격, 즉 mafia fraud attack에 강한 효율적인 거리 한정 프로토콜을 개발하였다. '①'절에서는 본 연구에서 개발된 distance bounding protocol을 기술할 것이고, '②'절에서는 개발된 distance bounding protocol의 성능 확인을 위한 simulator를 소개하고, '③'절에서는 기존 연구와 개발된 distance bounding protocol과의 성능을 비교하였다.

① 개발된 distance bounding protocol

[그림 3-106]은 본 연구에서 제안하는 distance bounding protocol이다. 준

비 단계에서 prover와 verifier는 각각 랜덤값 N_P와 N_V를 생성하여 서로 교환한다. 그리고 해쉬 함수를 이용하여 $4n$ bits 문자열 α, β, v^0, v^1을 생성한다. Fast bit exchange 단계에서는 α의 값에 따라 prover와 verifier는 두가지

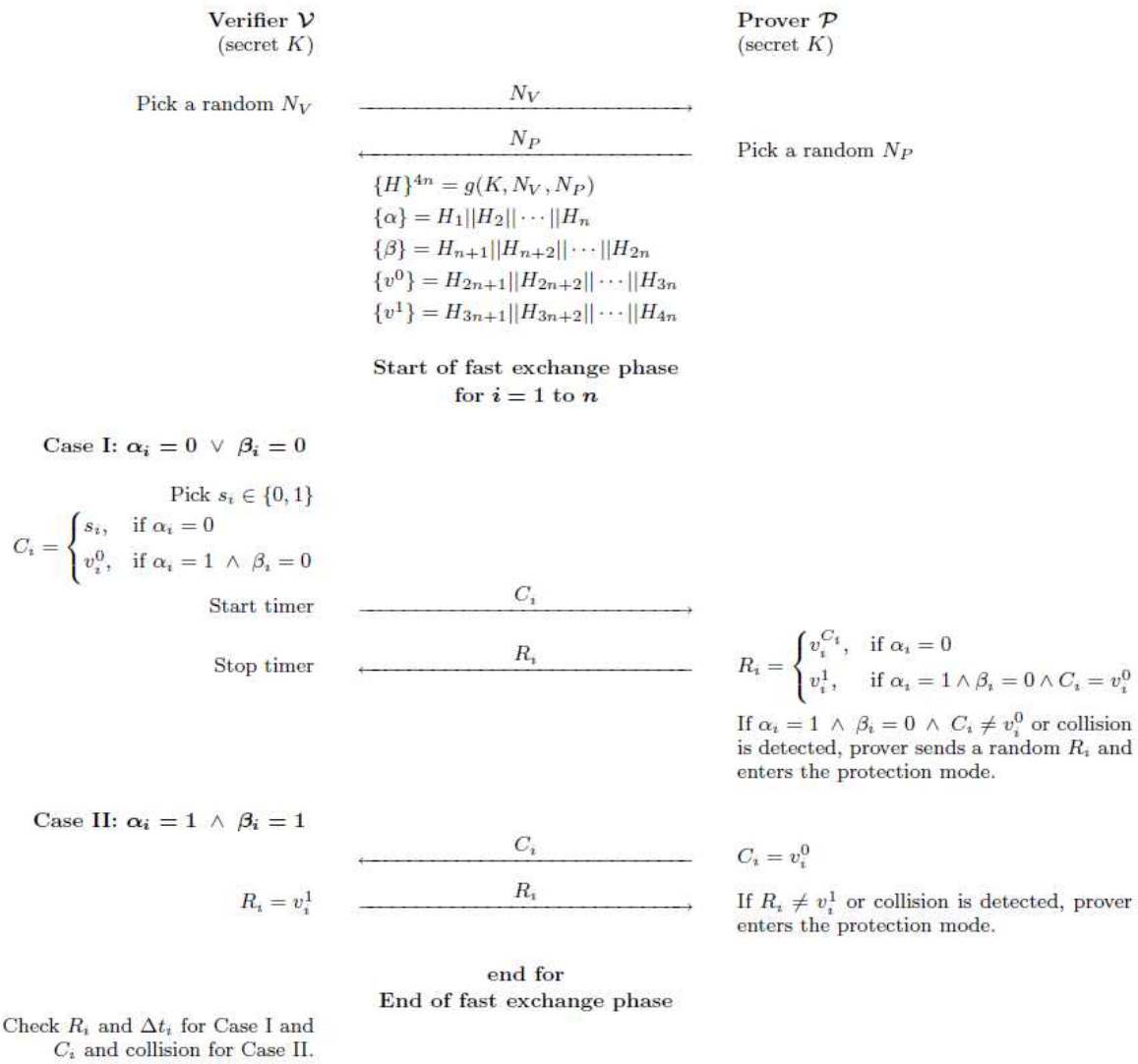


그림 3-106. distance bounding protocol with bidirectional challenges

mode를 가진다. $\alpha=0$ 일 때는 랜덤 challenge 모드이고, $\alpha=1$ 일 때는 테스트 모드이다. 랜덤 challenge 모드에서는 verifier가 prover에게 랜덤값을 송신하여, prover가 랜덤값에 따라 v^0 또는 v^1을 송신한다. 그리고 테스트 모드는 β의 값에 따라, $\beta=0$ 일 때는 verifier가 테스트 bit를 송신하고, $\beta=1$

일 때는 prover가 테스트 bit를 송신하다. 이러한 과정을 n번 반복하여 verifier는 round trip time 및 response 값의 유효성을 확인하여 prover의 인증을 수행한다.

② distance bounding protocol의 성능 확인을 위한 Simulator

제안된 distance bounding protocol의 성능을 확인하기 위하여 Matlab을 이용하여 구현한 simulator를 이용하여 기존 연구와의 성능을 비교하였다. [그림 3-107]는 distance bounding protocol의 성능 확인을 위하여 구현한 simulator를 나타낸 것이다. Simulator의 GUI는 크게 Simulation Parameter 영역, Display Parameter 영역, 버튼 영역, 그래프(결과출력) 영역으로 나누어진다. Simulation Parameter 영역에서는 protocol을 수행할 round 수, Monte Carlo simulation 수행 횟수, simulation 속도를 위해 그래프로 결과 출력을 Monte Carlo 출력을 완료 후 할 것인지를 결정하는 radio 버튼과 [그림 3-108]에서 보는 바와 같이 verifier와 adversary간의 송수신 메시지와 verifier가 adversary의 공격확인 여부를 Matlab의 command windows로 출력할 것인지를 결정하는 radio 버튼이 있다. Display Parameter 영역에는 제안된 protocol과 기존 연구에 대한 성능 비교를 정밀히 하기위한 입력값을 제어하고, 버튼 영역에는 simulation 실행 버튼과 simulator 초기화 버튼이 있다. 마지막으로 그래프 영역에는 제안된 distance bounding protocol과 기존 연구에 대한 adversary의 공격 성공확률을 출력한다.

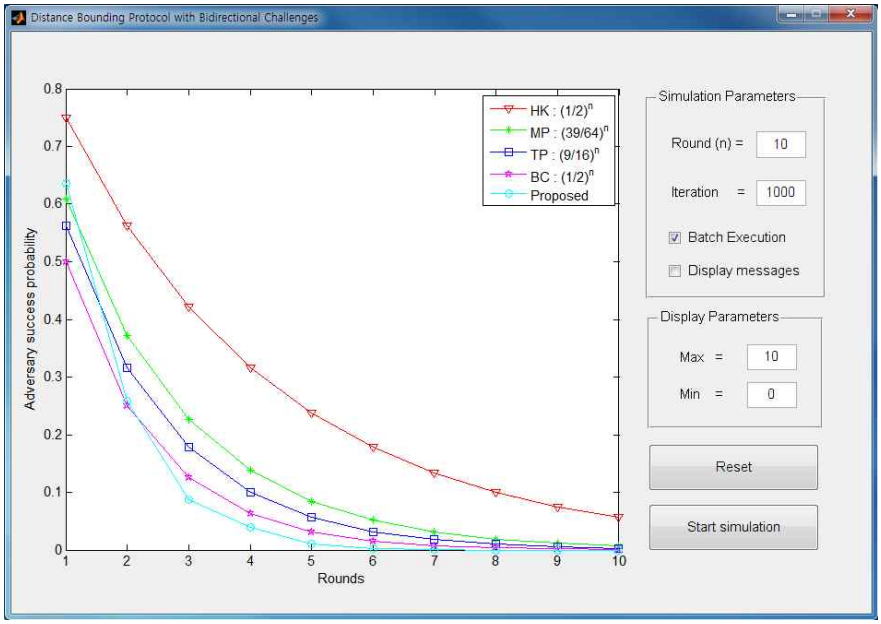


그림 3-107. distance bounding protocol의 성능 확인을 위한 Simulator

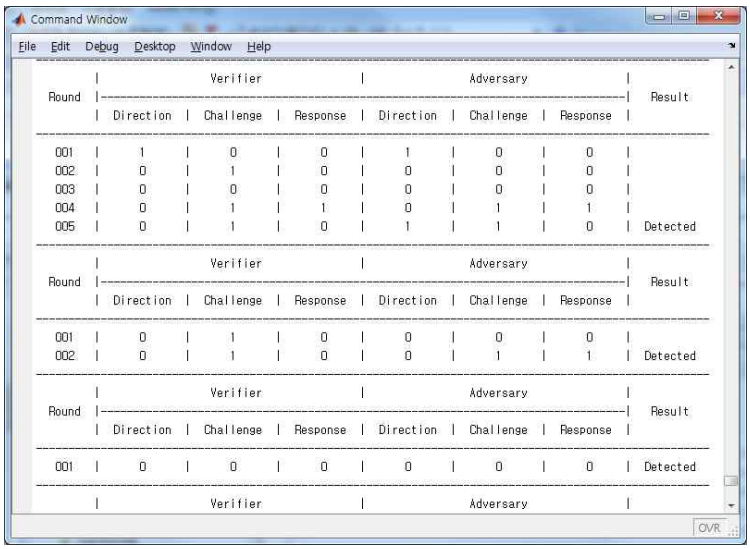


그림 3-108. 제안된 protocol의 송수신 메시지 출력 화면

simulator에 대한 파일 구성은 다음과 같다.

파일명	설명	비고
Main.m	simulator의 Main 파일	
Main.fig	simulator의 GUI 구성 파일	
SHA1.m	hash fuction을 구현	
bin2dec2.m	SHA1.m의 동작을 위한 내부 function	
hex2bin.m		
calculate_new_HSHA1.m		
cls.m		
Adversary.m	adversary의 공격 형태를 모사	
Verifier.m	verifier의 검증 내부 동작을 모사	
ExecuteProtocol.m	제안된 protocol의 수행을 모사	

표 3-6 Simulator 파일 구성

③기존 연구와의 성능 비교

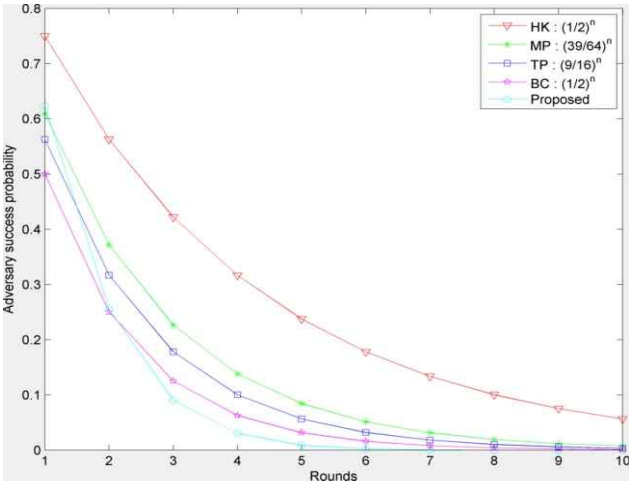


그림 3-109. 이전 연구와 제안한 distance bounding protocol의 공격 성공 확률 비교

[그림 3-109]는 본 연구과제에서 구현한 Simulator를 이용하여 제안한 distance bounding protocol과 이전 연구에 대한 adversary의 mafia fraud attack에 대한 공격 성공 확률을 비교하기 위해 Monte Carlo simulation을 10000회 수행한 결과이다. 그림에서 보는 바와 같이 본 연구과제에서 제안한

protocol이 round 수가 3회 이상이면 기존 연구에 비해 adversary의 공격 성공 확률이 가장 적은 것을 알 수 있다.

5. ITS를 위한 효율적인 차량간 통신 방안 연구

가. BDMA 전송 기술 기반의 차량 통신 시스템

(가) 빔 분할 다중 접속 기술

빔 분할 다중 접속 (Beam Division Multiple Aceess)기술은 기존의 SDMA(space division multiple access) 송신기에서의 치널 정보 필요성 문제 해결을 위하여, 송신기의 채널 정보의 필요 없이 수신기의 위치 정보만으로 적응적 빔을 구현하는 다중 접속방법이다.

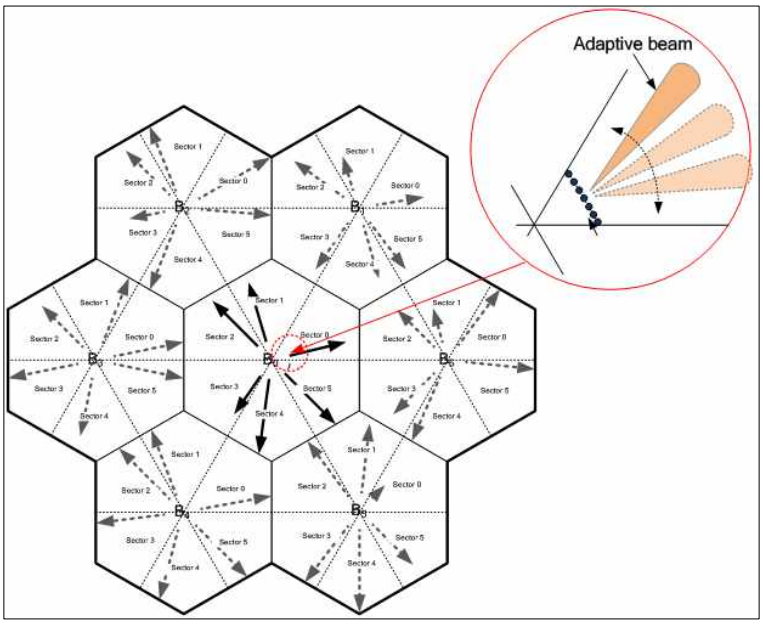


그림 3-110. BDMA 전송 기술을 사용한 통신 시스템

위의 그림과 같이 하량링크에서 각 사용자의 위치를 향하여 다수의 beam을 형성할 수 있으며 같은 주파수로 2개 이상의 빔을 사용하여 주파수 효율을 극대화 할 수 있다.

기존의 셀룰러 시스템에서 빔 분할 다중 접속 기술을 사용하기 위해서는 다음과 같은 문제점이 존재한다. 적응적 빔 기술을 위해 필요한 linear

array antenna는 안테나 크기와 전송 파워 등의 제약 조건 때문에 셀룰러 시스템의 각 단말에서는 적응적 빔 기술이 사용이 어렵다. 또 셀룰러 시스템에서는 실시간으로 단말의 정확한 위치를 파악하는 것이 러렵기 때문에 기지국에서 단말을 향한 정확한 빔 형성에 어려움을 겪는다.

반면에 적응적 빔 기술을 차량 간 통신에 이용하는 경우에는, 차량에서는 위에서 말한 단말의 제약 조건이 존재하지 않아서 linear array antenna의 사용이 가능하여 차량과 인프라 모두 빔 분할 다중 접속 기술을 사용가능하다. 또 차량은 GPS 탑재를 통해 실시간으로 정확한 위치 파악이 가능하며, 도로에 따라 이동경로가 예측 가능하기 때문에 차량과 인프라 혹은 차량 간의 정확한 빔 형성이 가능하다. 따라서 차량 간 통신에 적응적 빔을 사용함으로 충돌 및 간섭을 최소화하고 자원의 재사용이 가능한 장점을 가진다.

기존의 V2I, 즉 차량과 인프라 간의 통신에서는 인프라가 무지향성 안테나를 사용한다. 따라 단말의 위치, 이동 속도, 채널 상태를 고려하여 최적화 된 빔을 형성하지 않는다.

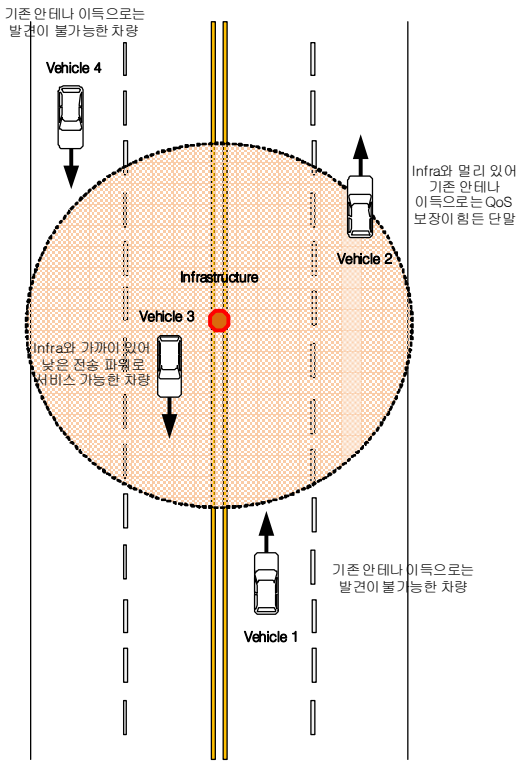


그림 3-111. 무지향성 안테나를 사용한
차량과 인프라의 통신

따라서 사용하는 공간 자원에 비하여 통과하는 차량들에 대하여 높은 서비스의 질(QoS)를 보장해 줄 수 없으며, 기존의 안테나 이득으로는 발견 불가능한 차량이 존재하게 된다.

(나) BDMA 기반의 차량 통신 시스템

○ 시스템 개념도

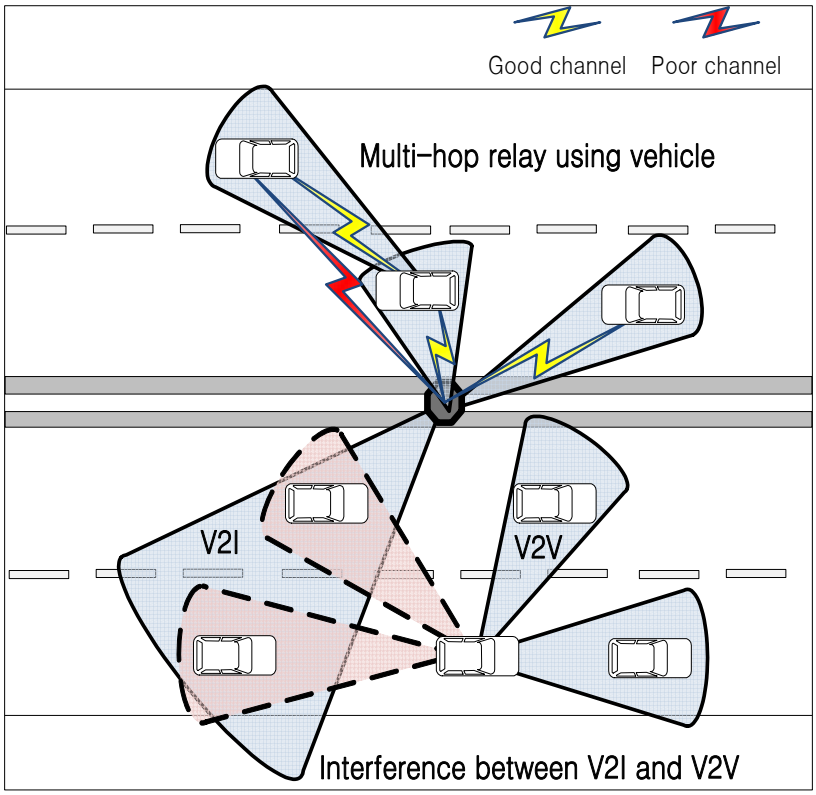


그림 3-112. BDMA 기반의 V2V2I 통신 시스템의 개념도

[그림 3-112]는 제안하는 BDMA 전송 기술 기반의 V2V2I 통신 시스템의 개념도를 나타내며 그 특징은 다음과 같다. 1) 인프라와 차량에서 BDMA 전송 기술을 사용한다. 2) V2I 통신과 V2V 통신이 같은 자원을 공유하는 V2V2I 통신 시스템을 고려하기 때문에 V2I 통신과 V2V 통신 사이에 간섭 문제가 발생한다. 3) 차량과 인프라 사이의 채널 상태가 안 좋은 경우에는 다른 차량을 이용한 multi-hop relay를 고려한다. 따라서 본 연구에서는 BDMA 전송 기술 기반의 V2V2I 통신 시스템의 성능을 향상시키기 위한 스케줄링 방안과 multi-hop relay 방안을 연구하고자 한다.

○ Frame 구조

제안하는 BDMA 기반의 차량 통신 시스템에서는 다음과 같은 이유 때문에 TDMA (time division multiple access) 기반의 통신 방법을 고려한다. 먼저 OFDMA (orthogonal frequency division multiple access) 방식의 경우 각 sub-channel 별로 채널의 상태를 측정하는 것이 쉽지 않을 뿐만 아니라 차량의 빠른 mobility로 인해 측정한 채널이 정확하지 않을 수 있다. CDMA (code division multiple access) 방식의 경우에는 차량의 빠른 mobility 때문에 인프라와 차량사이의 거리에 따른 near-far problem이 더욱 심각해진다.

그림은 BDMA 기반의 V2V2I 통신 시스템에서 frame 구조를 나타낸다. 각각의 frame은 고정된 길이인 Tf를 가지며 V2I 통신을 위한 dowlink subframe과 uplink subframe 그리고 V2V 통신을 위한 V2V subframe으로 구성된다. 각각의 subframe은 Ts의 길이를 가지는 여러 개의 time slot으로 구성되어 있다.

V2I Downlink subframe의 첫 번째 downlink slot은 Frame Preamble (FP)와 Frame Header (FH)를 전송하기 위한 control slot으로 할당된다. Frame Preamble은 매 frame의 시작을 알려주어 frame 동기화를 위해 사용된다. Frame Header는 인프라의 ID와 시스템 정보 그리고 time slot의 사용 및 할당을 위한 제어 정보를 포함하고 있다. Frame Header를 수신함으로써 각 차량은 time slot을 사용하기 위해 필요한 정보 (discovery, contention, V2I uplink/downlink, 그리고 V2V time slot)와 다가오는 data 송수신과 그들의 목적지를 얻을 수 있다.

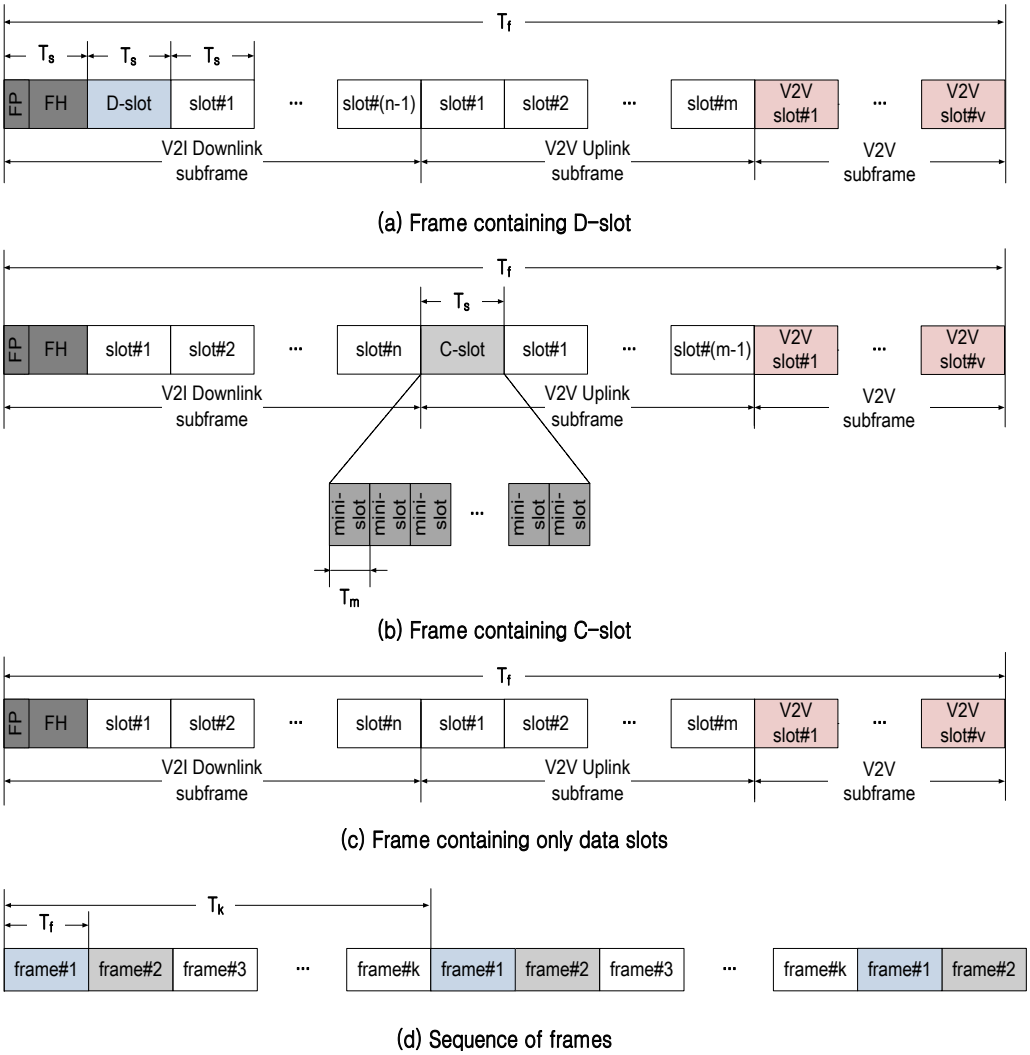


그림 3-113. BDMA 기반의 차량 통신 시스템의 frame 구조

[그림 3-113]의 (b)와 같이 V2I Upnlink subframe의 첫 번째 uplink slot은 contention slot (C-slot) 으로 사용될 수 있다. C-slot은 Ts의 길이를 가지는 여러 개의 mini slot으로 구성되어 있다. 인프라에 초기 접속을 하려는 차량은 mini-slot 중에 하나 선택하고 해당 mini slot 동안 access signal을 인프라에 전송한다.

[그림 3-113]의 (a)와 같이 V2I Downlink subframe의 두 번째 downlink slot은 discovery slot (D-slot) 으로 사용될 수 있다. D-slot의 기간 동안에 인프라는 자신의 서비스 커버리지 내에 새로 도착한 차량들을 발견하기 위해서 여러 개의 discovery beam을 형성한다. Discovery beam은 discovery

preamble (DP), discovery beam의 ID, 그리고 C-slot에 관한 정보를 포함한다. Discovery preamble은 도착하는 차량들에게 인프라의 존재를 알려주고 C-slot 정보는 인프라가 각각의 discovery beam에 할당한 mini slot의 정보를 포함한다. 따라서 인프라가 주변 도로의 상황을 알고 있다고 가정하면 인프라는 각각의 discovery beam의 방향을 결정하여 다른 방향으로부터 도착하는 차량들을 효율적으로 발견할 수 있다.

나머지 V2I Downlink/uplink subframe의 data slot은 V2I 통신을 위해서 할당되며 차량은 uplink slot을 이용하여 인프라에게 서비스를 요청하거나 channel feedback을 보내고 인프라는 downlink slot을 이용하여 다양한 서비스를 차량에게 제공한다. 또한, V2V subframe은 V2V 통신을 위한 V2V data slot으로 구성되어 있으며 차량은 V2V slot을 이용하여 안전, 긴급 및 제어 메시지를 방송한다. V2V subframe 동안에는 V2I 통신은 금지된다.

[그림 3-113]의 (d)와 같이 제안하는 BDMA 기반의 차량 통신 시스템에서는 k개의 frame sequence가 Tp = k Tf 의 주기로 반복된다. 첫 번째 frame의 두 번째 downlink slot은 D-slot으로 사용되어 새로 도착한 차량들을 발견하는데 사용된다. 두 번째 frame의 첫 번째 uplink slot은 C-slot으로 사용되어 도착한 차량들에게 channel access의 기회를 제공한다. Sequence의 다른 frame들은 V2I 통신과 V2V 통신을 위한 data slot만을 포함하고 있다. 따라서 인프라는 Tp의 주기로 discovery beam을 형성한다.

○ Channel access 절차

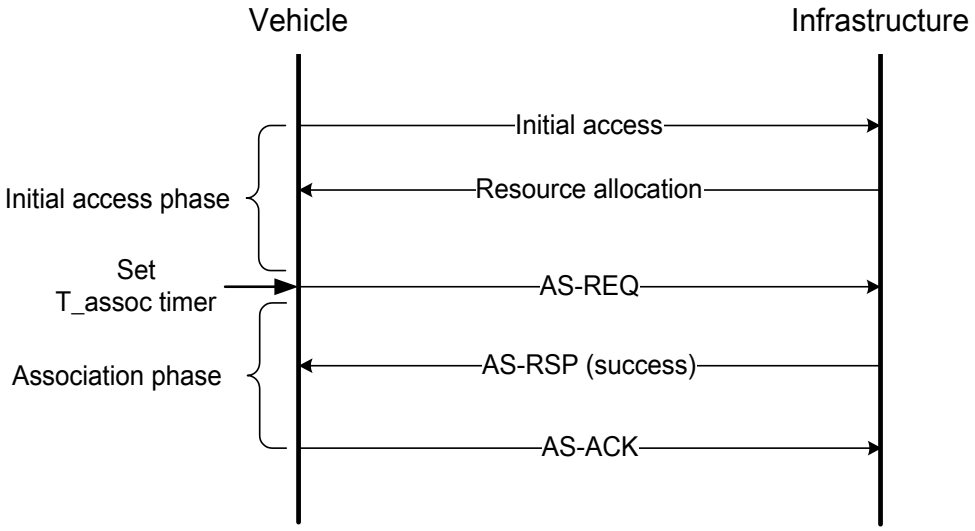


그림 3-114. BDMA 기반의 차량 통신 시스템의 channel access 절차

[그림 3-114] 은 제안하는 BDMA 기반의 차량 통신 시스템에서 channel access 절차를 나타내며 channel access 절차는 initial access와 association의 두 phase로 구성되어 있다. 인프라의 커버리지에 새로 도착한 차량은 discovery beam을 수신한 후에 initial access phase로 들어간다. Initial access phase에서 차량은 먼저 C-slot의 mini slot들 중에서 하나를 선택하고 인프라와 차량 사이의 association을 형성하기 위해서 access signal을 선택된 mini slot 동안에 전송한다. Access signal은 data signal에 비해서 아주 낮은 rate로 전송되며 아무런 data도 포함하지 않는다.

인프라가 특정 mini slot에서 access signal을 탐지하게 되면 association을 위해서 해당 mini slot을 선택한 차량에게 uplink 자원 (uplink slot) 을 할당한다. Association phase에서 해당 차량은 할당 받은 자원을 사용해서 association request (AS-REQ) message를 전송하고 Tassoc timer를 활성화 시킨다.

인프라가 AS-REQ message를 올바르게 수신하면 차량에게 association의 성공을 나타내는 association response (AS-RSP) message를 전송한다. 차량은 해당 AS-RSP message에 대한 응답으로 association acknowledge (AS-ACK) message를 인프라에게 전송한다. 그러면 인프라와 해당 차량 사이의 association이 형성되고 association phase가 완료된다.

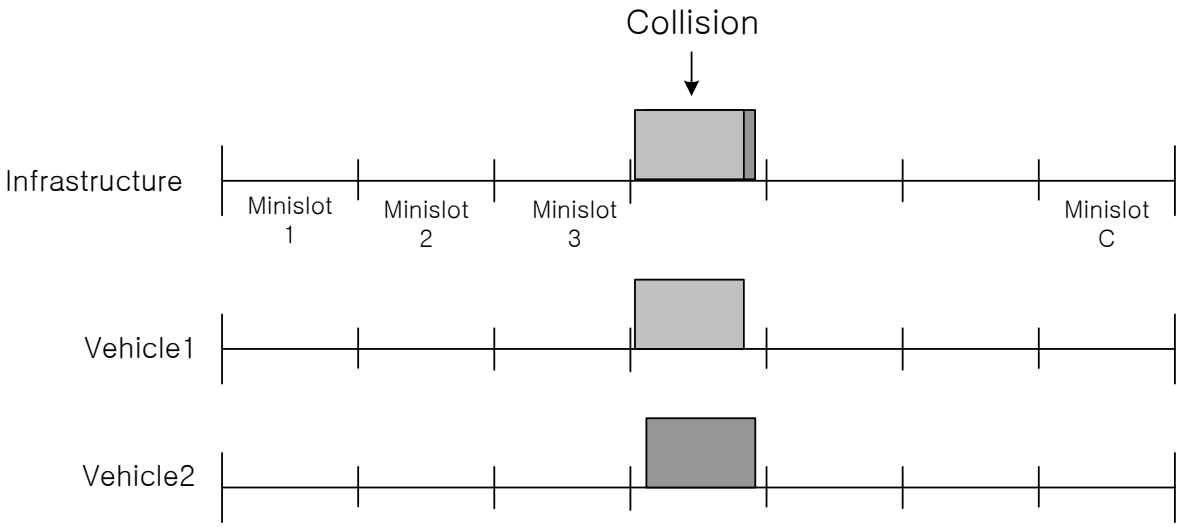


그림 3-115. Initial access에서 충돌 발생

[그림 3-115] 과 같이 initial access phase에서 두 대 이상의 차량이 같은 mini slot을 선택하고 그들의 access signal을 동시에 전송하게 되면 충돌이 발생한다. 인프라에서 on/off detection 기반으로 각각의 mini slot에서 수신한 signal을 발견하기 때문에 만약 충돌이 발생하면 해당 충돌에 관련된 차량들과 인프라는 충돌을 바로 탐지할 수 없다. 그 결과 association phase에서 해당 충돌에 관련된 차량들이 전송한 AS-REQ message가 또다시 충돌할 것이다. 따라서 initial access 동안 충돌은 차량의 access delay를 증가 시킨다.

상기 설명한 충돌 때문에 인프라가 AS-REQ message를 decoding 하는데 실패하면 association의 실패를 나타내는 AS-RSP message를 방송한다. 해당 충돌에 관련된 차량들은 이 실패 message를 수신한 후에 initial access phase로 돌아가서 channel access 절차를 반복하게 된다. 또한, AS-REQ message를 전송한 차량이 Tassoc timer가 종료될 때까지 아무런 AS-RSP message를 수신하지 못하면 해당 차량은 initial access phase로 돌아가서 channel access 절차를 반복하게 된다. Channel access에 성공한 차량은 인프라의 서비스 커버리지 밖으로 나가기 전까지 인프라와 정보를 교환할 수 있다.

나. BDMA 전송 기술 기반의 차량 시스템에서 차량 스케줄링 방안

(가) 기존 연구 및 문제점

○ V2I 통신 시스템

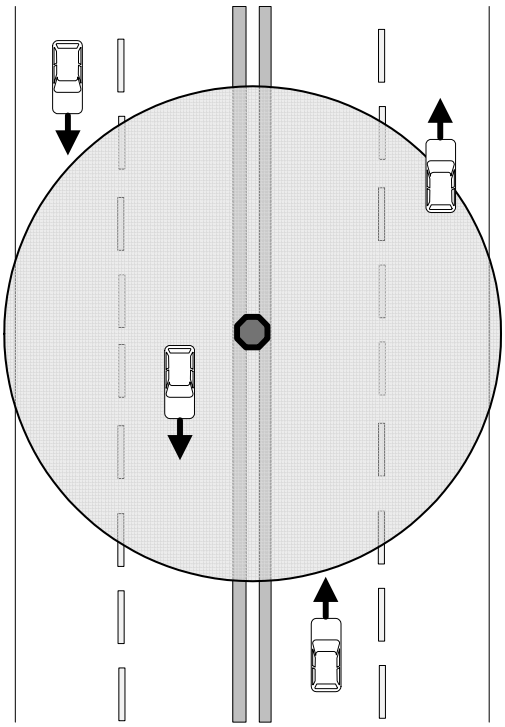


그림 3-116. 기존의 V2I 통신 시스템

V2I 통신 시스템은 최근 들어 활발히 연구가 진행되고 있는 분야로 인프라가 차량에게 다양한 서비스를 제공하는 것을 목표로 하며 다음과 같은 특징을 가지고 있다. 1) [그림 3-116]와 같이 기존의 V2I 통신 시스템에서 인프라는 omni-directional antenna를 사용한다. 2) 차량과 인프라의 접속시간이 제한적이기 때문에 V2I 통신의 기회가 항상 존재하지 않는다. 3) 인프라와 차량의 높은 상대 속도가 야기하는 큰 Doppler effect로 인해 V2I 채널 상태가 좋지 않다.

V2I 통신 시스템에서는 다음과 같은 문제점이 존재한다. 1) 인프라의 서비스 커버리지에 진입하는 차량의 발견이 비효율 적이다. 2) 인프라가 사용하는 공간 자원에 비해서 차량에게 낮은 QoS (Quality of Service)를 제공한다. 3) 인프라가 동시에 많은 수의 차량에게 서비스를 제공할 수 있어야 한다. 4) 차량의 위치, 이동 속도, 채널 상태를 고려한 자원 할당이 곤란하다.

○ V2V 통신 시스템

대부분의 차량 통신 연구가 V2V 통신 시스템에 집중되어 있으며 V2V 통신 시스템은 다음과 같은 특징을 가진다. 1) V2V 통신을 통해서 교통안전 및 편의 향상을 목적으로 한다. 2) 차량 사이의 낮은 상대 속도로 V2V 통신의 기회가 V2I 통신에 비해 상대적으로 많다. 3) 차량 사이의 낮은 상대 속도로 Doppler effect가 작기 때문에 V2V 통신의 채널 상태가 좋다.

V2V 통신 시스템에서는 다음과 같은 문제점이 존재한다. 1) safety application에서는 차량이 주변 차량에게 긴급 message를 broadcast하기 때문에 broadcast storming이 발생하여 중복된 message의 flooding 문제가 발생한다. 2) 차량의 빠른 속도로 네트워크 토폴로지가 빠르게 변하기 때문에 multi-hop 전송에서 최적의 route를 선정하기가 어렵다. 3) 인프라와 같이 central node가 없는 분산 환경의 특징 때문에 차량 간의 coordination이 매우 중요하다. 4) sender와 receiver가 모두 움직이는 환경이기 때문에 기존의 channel model을 그대로 적용하기 어렵다.

○ V2V2I 통신 시스템

최근에 기존의 V2I 통신 시스템과 V2V 통신 시스템의 문제점을 극복하기 위해서 V2V2I 통신 시스템이 활발히 연구 중에 있다. V2V2I 통신 시스템에 대한 연구는 초기 단계로 다음과 같은 특징을 가진다. 1) 인프라의 커버리지 안에서는 V2I 통신과 V2V 통신이 공존하는 환경이다. 2) 기존의 V2I 통신과 V2V 통신의 한계 극복을 위해서 서비스 커버리지 확대 및 시스템 용량 증대를 목적으로 한다.

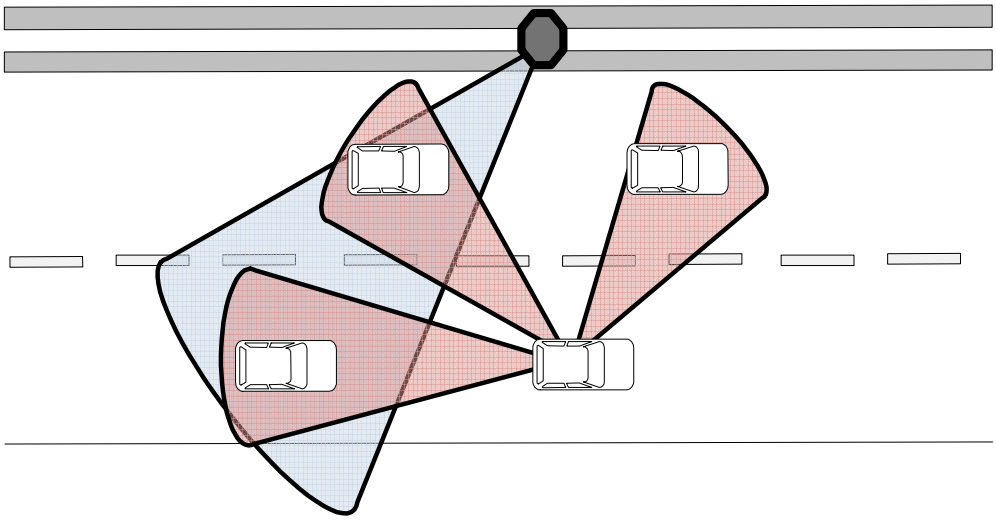


그림 3-117. V2V 통신과 V2I 통신이 서로 다른 자원을 사용하는 경우

[그림 3-117]은 BDMA 전송 기술 기반의 V2V2I 통신 시스템에서 V2V 통신과 V2I 통신이 서로 다른 자원을 사용하는 동작 예시를 나타낸다. 따라서 V2V 통신과 V2I 통신 사이의 간섭이 존재하지 않지만 시스템의 자원 사용 효율 감소로 시스템 용량이 감소한다.

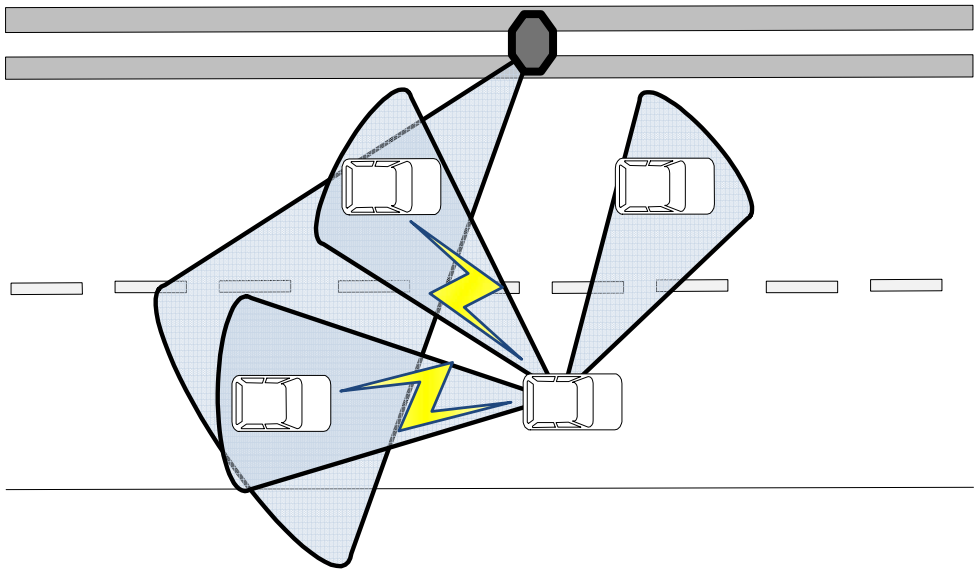


그림 3-118. V2V 통신과 V2I 통신이 서로 같은 자원을 공유하는 경우

[그림 3-118]은 BDMA 전송 기술 기반의 V2V2I 통신 시스템에서 V2V 통신과 V2I 통신이 서로 같은 자원을 공유하는 동작 예시를 나타낸다. 따라

서 V2I 통신과 V2V 통신이 자원의 재사용을 통해서 자원 사용 효율은 증가하지만 V2V 통신과 V2I 통신 사이의 간섭이 발생하기 때문에 시스템 용량이 감소할 수 있다. 따라서 제안 방안에서는 V2V2I 통신 시스템에서 간섭 시나리오를 정립하고 이를 기반으로 V2I 통신과 V2V 통신 사이의 간섭 완화 방안과 V2V2I 통신 시스템의 성능을 최대화하는 스케줄링 방안을 연구한다.

(나) 제안 방안

본 연구에서는 V2V 통신과 V2I 통신이 서로 같은 자원을 공유하는 BDMA 전송 기술 기반의 V2V2I 통신 시스템을 고려하고 있다. 제안방안에서는 보통 인프라는 차량 보다 높은 위치 (예: 가로등, 전신주 등)에 설치되기 때문에 인프라와 차량 간의 채널은 기본적으로 LOS (line of sight) 를 가정하였고 차량간의 채널은 차량 사이의 거리에 따라서 LOS 인지 NLOS (non-line of sight) 인지 구별하였다. 그리고 차량과 인츠라 사이의 채널과 차량 간의 채널은 path loss model과 antenna model을 고려하였다. 또한 차량은 GPS를 탑재하여 자신의 위치, 속도, 및 방향을 알 수 있고 이 정보를 주기적으로 인프라에 보고하여 인프라가 차량의 정보를 알고 있다고 가정하였다.

BDMA 전송 기술 기반의 V2V2I 통신 시스템에서는 다음과 같은 두 가지의 V2V 통신 시나리오를 고려한다. 먼저 V2V 통신의 종류는 안전, 긴급 및 제어 message 전송을 위한 Type I과 일반 data 전송을 위한 Type II로 분류하였다.

시나리오 1) 인프라의 서비스 커버리지 밖에서는 V2I 통신은 존재하지 않고 V2V 통신만 존재하기 때문에 V2V 통신이 시스템의 자원을 모두 사용한다. 이때 V2V 통신은 종류에 따라서 기존의 V2V 통신 프로토콜을 이용한다.

시나리오 2) 인프라의 서비스 커버리지 안에서는 V2I 통신과 V2V 통신이 공존하는 V2V2I 환경이기 때문에 V2V 통신의 종류에 따라서 다른 자원을 사용한다. 교통안전과 직결된 V2V 통신 Type I은 V2V subframe의 V2V slot을 사용하여 언제든지 전송한다. 반면에 일반적인 data 전송인 V2V 통신 Type II는 V2V 통신 Type 1과는 다르게 V2I subframe의 slot을 재사용한다.

V2I uplink slot은 차량이 인프라와 association을 형성하거나 서비스를 요청할 때에 임의적으로 사용하기 때문에 인프라가 V2I uplink 전송 스케줄

을 예측하기 힘들다. 따라서 V2I uplink slot을 V2V 통신에 재사용하는 경우에는 V2I 통신과 V2V 통신 사이의 간섭을 완화하기 힘들다. 반면에 V2I downlink slot은 인프라가 차량에게 서비스를 제공하기 위해서 사용하기 때문에 인프라가 V2I downlink의 전송 스케줄을 예측 가능하다. 따라서 제안 방안에서 V2I downlink slot을 재사용하여 V2V 통신의 자원을 할당하고자 한다.

제안하는 BDMA 전송 기술 기반의 V2V2I 통신 시스템 환경은 총 M 대의 차량이 인프라의 서비스 커버리지 내에 존재하며 차량의 집합은 M={1,...,M} 으로 정의하였다. 한 frame은 N 개의 V2I downlink slot으로 구성되어 있으며 downlink slot의 집합은 N={1,...,N} 으로 정의하였다.

V2I 통신의 자원 할당은 다음과 같이 이루어진다. 인프라는 총 B개의 V2I 빔을 형성할 수 있고 인프라의 V2I beam 자원의 집합은 B = {1,...,B} 으로 정의하였다. 따라서 V2I 통신의 자원 할당은 인프라가 차량 i 에 전송 power ($p_i(n)$) 를 할당하여 V2I beam j ($b_{ij}(n)$) 을 형성하는 것이다.

V2I 통신의 자원 할당 시에 인프라가 차량 i로 할당하는 전송 power $p_i(n)$ 는 두 개의 power constraint를 만족해야 한다.

power constraint 1) $0 \le p_i(n) \le P_{\max}^I \text{ for } \forall i \in M$ [수식 1]

power constraint 2) $\sum_{i=1}^{M}\sum_{j=1}^{B} p_i(n) b_{ij}(n) \le P_{\max}^I$ [수식 2]

첫 번째 power constraint는 인프라가 각각의 차량 i에 할당한 전송 power $p_i(n)$ 가 0 보다는 크고 인프라의 최대 전송 power $P_{\max}^I$ 보다는 작아야 한다는 것이다. 두 번째 power constraint는 인프라가 차량에 할당한 전송 power의 합은 인프라의 최대 전송 power $P_{\max}^I$ 보다는 작아야 한다는 것이다.

V2I 통신의 자원 할당 시에 인프라가 차량 i로 형성하는 V2I beam j $b_{ij}(n)$ 도 두 개의 beam constraint를 만족해야 한다,

beam constraint 1) $\sum_{j=1}^{B} b_{ij}(n) \le 1 \text{ for } \forall i \subseteq \in M$ [수식 3]

beam constraint 2) $\sum_{i=1}^{M}\sum_{j=1}^{B} b_{ij}(n) \le B$ [수식 4]

첫 번째 beam constraint는 각각의 V2I beam은 한 대의 차량에만 할당되어야 하고 두 대 이상의 차량에 같은 V2I beam은 할당할 수 없다는 것이다. 앞으로는 편의상 차량 i에 할당된 V2I beam은 V2I beam i로 정의하겠

다. 두 번째 beam constraint는 인프라가 최대 형성할 수 있 V2I beam의 개수는 최대 B보다 작다는 것이다. 따라서 V2I 통신의 자원할당은 power constraint와 beam constraint를 만족하도록 인프라가 차량에게 전송 power ($p_i(n)$) 를 할당하여 V2I beam ($b_{ij}(n)$) 을 형성하는 것이다.

1) V2V2I 통신 시스템에서 간섭 완화 방안

○ V2I 통신과 V2V 통신 사이의 간섭 시나리오

BDMA 전송 기술 기반의 V2V2I 통신 시스템에서 V2I 통신과 V2V 통신 사이의 간섭 시나리오는 총 세 가지로 다음과 같다.

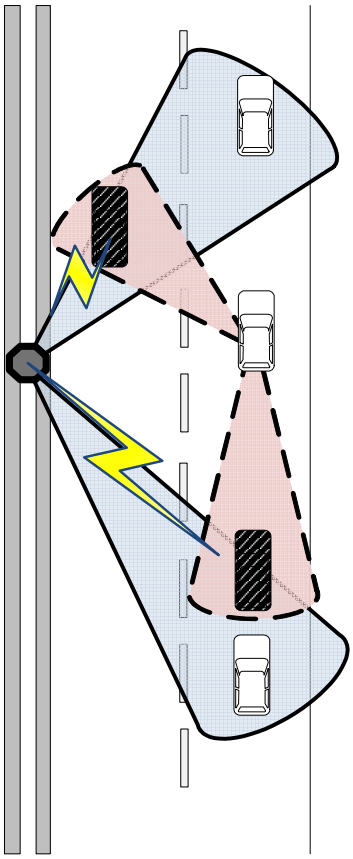


그림 3-119. V2I 통신이

V2V 통신에 주는 간섭

[그림 3-119]는 V2I 통신이 V2V 통신에 간섭을 주는 시나리오 I 을 나타내는 그림으로 V2I beam이 형성된 방향에 위치한 차량이나 V2I 통신 중

인 차량 근처에 위치한 차량과 V2V 통신을 하는 경우에는 V2V 통신이 V2I 통신으로부터 큰 간섭을 받게 된다.

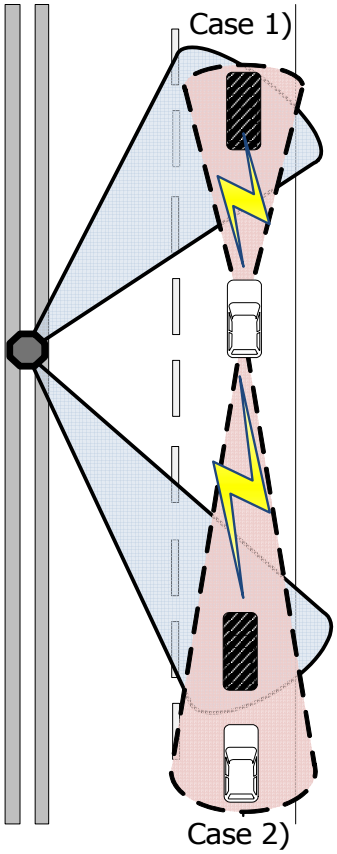


그림 3-120. V2V 통신이 V2I 통신에 주는 간섭

[그림 3-120]은 V2V 통신이 V2I 통신에 간섭을 주는 시나리오 II 를 나타내는 그림으로 Case 1) 과 같이 V2I 통신 중인 차량과 V2V 통신을 하는 경우와 Case 2) 와 같이 V2V beam이 형성된 방향에 V2I 통신 중인 차량이 존재하는 경우에는 V2I 통신이 V2V 통신으로부터 큰 간섭을 받게 된다.

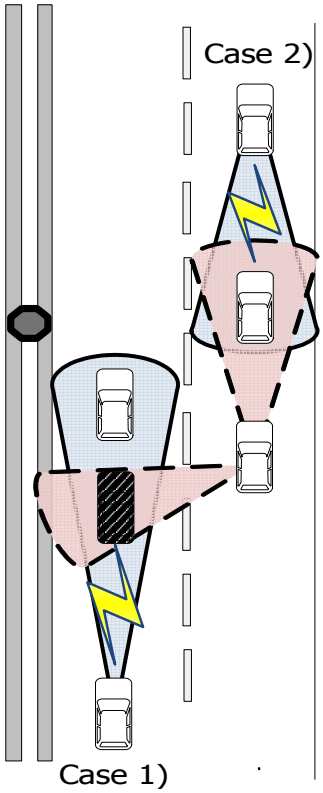


그림 3-121. V2V
통신들 사이의 간섭

[그림 3-121]는 V2V 통신 간에 간섭을 미치는 시나리오 III 을 나타내는 그림으로 Case 1) 과 같이 다른 V2V beam이 형성된 방향에 위치한 차량이나 Case 2) 와 같이 V2V 통신 중인 차량이 다른 V2V 통신을 하는 경우에는 V2V 통신 간에 큰 간섭을 주게 된다. 이와 같이 V2V2I 통신 시스템에서는 V2I 통신과 V2V 통신 사이에 세 가지 시나리오가 존재하고 이러한 간섭은 V2I 통신과 V2V 통신의 용량을 감소시켜 시스템의 전체 성능을 감소시킨다. 따라서 V2I 통신과 V2V 통신간의 간섭 완화 방안을 통해서 시스템 성능을 최대화 하는 스케줄링 방안이 필요하다.

○ V2V2I 통신 시스템에서 간섭 모델

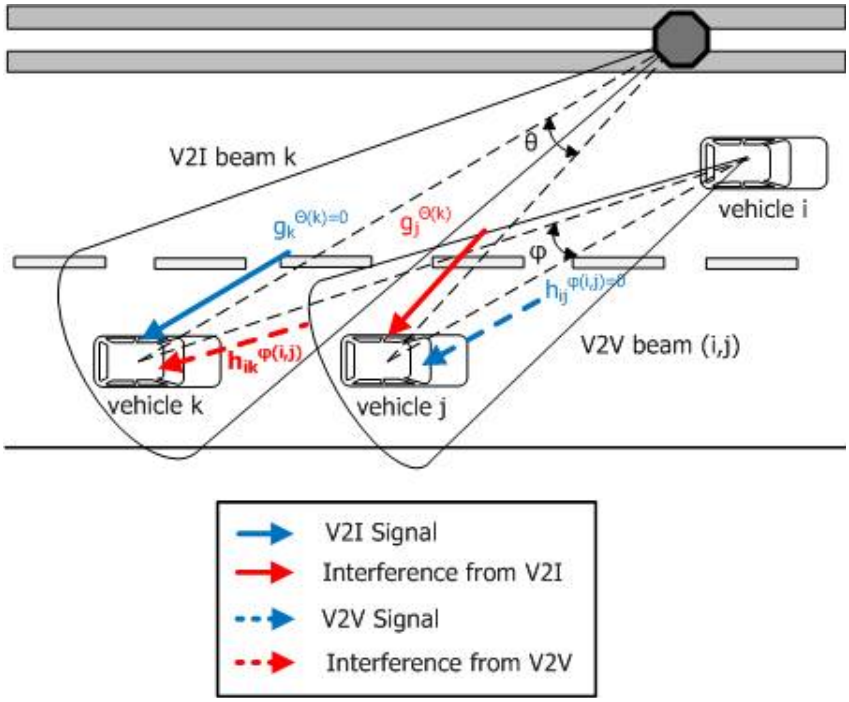


그림 3-122. V2V2I 통신 시스템의 간섭 모델

[그림 3-122]는 BDMA 전송 기술 기반의 V2V2I 통신 시스템에서 V2I 통신과 V2V 통신 사이의 간섭 모델을 나타낸 것으로 이 모델을 확장해서 V2V 통신 간의 간섭 모델도 쉽게 나타낼 수 있다. 인프라는 차량 k 에게 V2I beam k 를 형성하여 V2I 통신을 하고 있고 이와 동시에 차량 i 는 V2V beam (i,j) 를 차량 j에게 형성하여 V2V 통신을 하고 있다. 인프라에서 차량 k 로의 V2I 통신 신호는 파랑 실선으로 차량 i 에서 차량 j 로의 V2V 통신 신호는 파랑 점선으로 표시하였다. 또한 인프라에서 차량 j 로의 V2I 통신 간섭은 파랑 실선으로, 차량 i 에서 차량 k 로의 V2V 통신 간섭은 빨간 점선으로 표시하였다.

V2I 통신의 channel model은 $g_j^{\theta(k)}(n)$ 으로 정의되며 $\theta(k)$ 는 인프라에서 차량 j 의 방향과 V2I beam k 방향 사이의 각도로 정의 된다. 따라서 인프라에서 차량 j 로의 V2I 통신의 간섭 채널은 $g_j^{\theta(k)}(n)$ 으로 정의되며 인프라에서 차량 k 로의 V2I 통신 신호의 채널은 $g_k^{\theta(k)=0}(n)$ 로 정의 되며 이때 인프라는 차량 k 의 방향으로 V2I beam을 형성하기 때문에 $\theta(k)=0$ 을 만족하게 된다.

V2V 통신의 channel model은 $h_{ik}^{\phi(i,j)}(n)$ 으로 정의되며 $\phi(i,j)$ 는 차량 i

에서 차량 k의 방향과 V2V beam (i,j) 방향 사이의 각도로 정의된다. 따라서 차량 i 에서 차량 j 로의 V2V 통신이 차량 k 로 미치는 간섭 채널은 $h_{ik}^{\phi(i,j)}(n)$ 으로 정의되며 차량 i 에서 차량 j 로의 V2V 통신 신호의 채널은 $h_{ij}^{\phi(i,j)=0}(n)$ 으로 정의 되며 이때 차량 i 는 차량 j 의 방향으로 V2V beam을 형성하기 때문에 $\phi(i,j)=0$ 을 만족하게 된다.

제안 방안에서는 V2I 통신과 V2V 통신의 채널은 path loss model과 antenna model을 고려한다. 따라서 V2I 통신의 경우에는 인프라와 차량 사이의 거리와 V2I beam과 차량 사이의 각도 $\theta(k)$ 가 결정되면 channel gain을 구할 수 있다. 마찬가지로 V2V 통신의 경우에는 source 차량과 destination 차량 사이의 거리와 V2V beam과 destination 차량 사이의 각도 $\phi(i,j)$ 가 결정되면 channel gain을 구할 수 있다.

BDMA 전송 기술에서 antenna model 은 다음과 같은 수식으로 표현된다.

$$A(\theta) = -\min[12(\frac{\theta}{\theta_{3dB}})^2, A_m] + G_A \quad \text{[수식 5]}$$

θ 는 beam 방향과 차량 방향의 각도, θ_{3dB} 는 신호가 3dB 감쇠되는 beam-width, A_m 은 최대 감쇠값, 그리고 G_A 은 안테나 이득을 나타낸다. 따라서 antenna model의 gain은 beam 방향과 차량 방향 사이의 각도인 θ 에 따라 결정되며 이때 V2I 통신과 V2V 통신의 경우에는 각각 $\theta=\theta(k)$ 와 $\theta=\phi(i,j)$ 을 만족한다.

○ V2V2I 통신 시스템에서 간섭 완화 방안

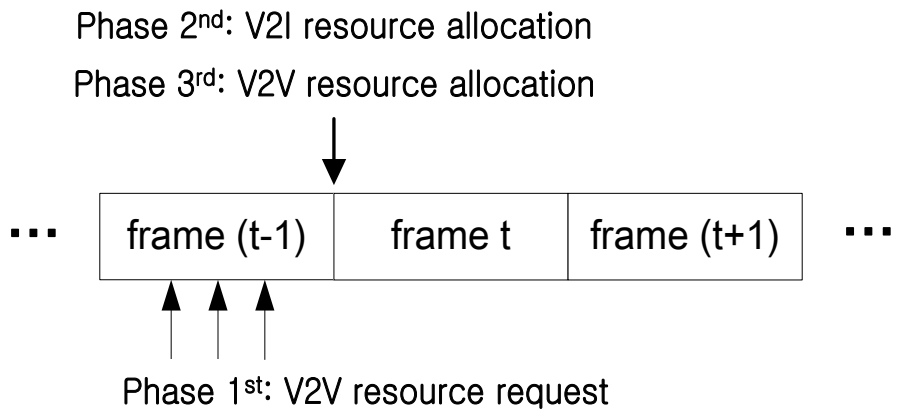


그림 3-123. V2V2I 통신 시스템에서 간섭 완화 방안

[그림 3-123]과 같이 V2V2I 통신 시스템에서 간섭 완화 방안은 다음과 같이 3 단계의 Phase 로 구성된다. Phase 1) 인프라에 접속한 차량은 "V2V 자원 요청 message"를 인프라에 전송함으로써 V2V 통신 자원을 요청한다. "V2V 자원 요청 message"에는 source와 destination 차량 (s,d) (이하 V2V 통신 link (s,d)) 의 정보, 요청하는 V2I slot의 수 $N_{(s,d)}$, V2V 통신 link (s,d) 의 target SINR (signal to intereference plus noise ratio) $\bar{\gamma}_{sd}^{target}$ 이 포함되어 있다. 인프라는 차량으로부터 "V2V 자원 요청 message"를 수신하게 되면 해당 V2V 통신 link (s,d) 의 요청을 V2V 자원 요청의 집합 R 에 포함시킨다.

$R = R \cup (s,d)$ [수식 6]

Phase 2) 인프라는 먼저 V2I 통신의 자원을 할당한다. V2I 통신의 자원 할당은 한 frame 단위로 수행되며 한 frame은 N개의 V2I downlink slot으로 구성되어 있다. 따라서 인프라는 매 frame을 시작할 때마다 각각의 downlink slot에 대해서 V2I 통신의 자원을 할당한다.

Phase 3) 다음으로 인프라는 결정된 V2I 통신의 자원 할당을 기반으로 V2V 통신의 자원을 할당한다. V2V 통신의 자원 할당은 한 frame 단위로 수행되며 인프라는 매 frame을 시작할 때마다 각각의 V2I downlink slot을 재사용하여 V2V 통신의 자원을 할당한다. 이때 V2I 통신과 V2V 통신 사이의 간섭을 최소화함으로서 V2V2I 통신 시스템의 용량을 최대화하고자 한다.

2) V2V2I 통신 시스템에서 V2V 통신 스케줄링 방안

다음으로 인프라가 V2V2I 통신 시스템에서 V2I 통신과 V2V 통신의 간섭 완화를 위한 V2V 통신의 스케줄링 방안에 대해서 자세한 설명을 하고자 한다. 제안하는 V2V 통신 스케줄링 방안은 매 frame의 단위로 수행되며 각각의 V2I downlink slot 마다 V2I 통신의 자원할당이 결정되었을 때에 첫 번째 V2I downlink slot 부터 N 번째 V2I downlink slot의 차례로 V2V 통신 자원을 할당함으로써 전체 시스템 용량을 최대화하는 것을 목표로 한다.

○ Step 1) 초기화

초기화 단계에서는 다음과 같은 파라미터들을 초기값으로 설정한다.

- $n=1$: V2I downlink slot 인덱스
- $P_i = P_{\max}^V \text{ for } i=1,...,M$: 차량의 남은 전송 power를 차량의 최대 전송 power $P_{\max}^V$로 초기화
- $p_{ij}(n)=0 \text{ for } \forall (i,j) \in R$: V2V 자원 요청 집합 R 에 포함된 모든 V2V 통신 link (i,j) 에 할당된 power를 0으로 초기화
- $S(n)=\varnothing$: V2V 통신 자원을 할당 받아 스케줄된 V2V link가 없으므로 공집합으로 초기화

○ Step 2) V2V 자원 요청 집합 R 을 mutual distance의 오름차순으로 정렬한 집합 V 형성

V2V 통신 link에서 source 차량과 destination 차량의 거리가 가까울수록 작은 전송 power를 사용하여 통신하기 때문에 주변에 작은 간섭을 발생시킨다. 따라서 V2V 자원 요청 집합 R 에 포함된 모든 V2V 통신 link (i,j) 에 대해서 차량 i 와 차량 j 사이의 거리가 가까운 순으로 정렬하여 집합 V 를 형성한다. V2V 자원 요청 집합 R 을 정렬하는 방법에는 차량사이의 거리뿐만 아니라 차량의 QoS 등을 고려할 수도 있다.

○ Step 3) V2I 통신 중인 차량을 집합 V 에서 제거

V2V2I 통신 시스템의 간섭 시나리오 II) 의 Case 1) 에서 설명했듯이 V2I 통신 중인 차량에 V2V beam을 형성하는 경우에는 V2V 통신이 V2I 통신 중인 차량에게 큰 간섭을 주게 된다. 또한 V2I 통신 중인 차량은 동시에 V2V beam을 형성하여 V2V 통신을 할 수 없기 때문에 V2I 통신 중인 차량은

V2V 통신의 source 차량이나 destination 차량이 될 수 없다. 따라서 다음과 같이 time slot n에서 V2I 통신 중인 차량의 집합 $I(n)$ 을 계산한다.

$$I(n)=\left\{ i\in M|\ \sum_{j=1}^{B} b_{ij}(n)=1 \right\}$$ [수식 7]

각 차량 i 에 대해서 V2I beam j 가 할당되었으면 $b_{ij}(n)=1$ 을 만족하기 때문에 V2I 통신 중인 차량의 집합을 구할 수 있다. V2I 통신 중인 차량은 다른 V2V 통신의 source 차량이나 destination 차량이 될 수 없기 때문에 $I(n)$ 에 포함된 각각의 V2I 통신 중인 차량 i 에 대해서 동시에 V2V 통신이 불가능한 V2V link의 집합 T_i 을 계산한다.

$$T_i=\{(n_1,n_2)\in V\mid i=n_1 \text{ or } i=n_2\}$$ [수식 8]

그리고 동시에 V2V 통신이 불가능한 V2V link의 집합 T_i 을 집합 V 에서 제거 한다. $V = V - T_i$ [수식 9]

○ Step 4) 집합 V 에 포함된 V2V link 중에 하나의 V2V link (i,j) 를 선택하여 Step 5), 6) 7)을 수행하여 V2V2I 통신 시스템의 간섭을 문제를 고려하여 선택된 V2V link (i,j) 를 스케줄링할지 결정한다.

○ Step 5) V2V link (i,j) 가 V2I 통신과 V2V 통신에게 받는 간섭 고려

먼저 V2V link (i,j) 의 target SINR $\bar{\gamma}_{ij}^{target}$ 을 만족하는 전송 power p_{ij}^{*} 를 다음의 수식을 이용해서 계산한다.

$$\frac{p_{ij}^{*}h_{ij}^{\phi(i,j)}(n)}{WN_0+\sum_{k\in I(n)}p_k(n)g_j^{\theta(k)}(n)+\sum_{(u,v)\in S(n)}p_{uv}(n)h_{uj}^{\phi(u,v)}(n)}=\bar{\gamma}_{ij}^{target}+\triangle\gamma^{V2V}$$ [수식 10]

좌변의 분자는 V2V link (i,j) 의 수신신호세기이고 분모의 첫 번째 항은 노이즈 (W : channel bandwidth, N_0 : power spectral density of noise), 두 번째 항은 V2I 통신으로부터의 간섭, 세 번째 항은 스케줄된 V2V 통신으로부터의 간섭을 나타낸다. 우변은 V2V link (i,j) 의 target SINR $\bar{\gamma}_{ij}^{target}$ 과 V2V link의 SINR margin $\triangle\gamma^{V2V}$ 의 합으로 표현된다. 이때 V2V link의 SINR margin $\triangle\gamma^{V2V}$ 은 V2V link (i,j) 가 V2I 통신과 다른 V2V 통신으로부터 받을 간섭을 고려해서 target SINR $\bar{\gamma}_{ij}^{target}$ 에 margin을 두어 전송 power p_{ij}^{*} 를 할당하도록 한다.

만약 V2V link (i,j) 의 source 차량 i 의 남은 전송 power로 target SINR을 만족할 수 있다면 ($p_{ij}^{*} \le P_i$) Step 6) 을 실행하고 만족할 수 없다면 ($p_{ij}^{*} > P_i$) Step 4) 로 돌아간다.

○ Step 6) V2I 통신이 V2V 통신에게 받는 간섭 고려

먼저 V2I 통신 중인 차량 k 가 V2V 통신으로부터 간섭을 받지 않는 경우의 SNR (signal to noise ratio) γ_k 을 다음의 수식을 이용해서 계산한다.

$$\gamma_k = \frac{p_k(n) g_k^{\theta(k)}(n)}{WN_0} \quad \text{[수식 11]}$$

우변의 분자는 V2I link k 의 수신 신호세기이고 분모는 노이즈이다. 다음으로 V2I 통신 중인 차량 k 가 V2V 통신으로 간섭을 받는 경우의 SINR $\overline{\gamma_k}$ 을 다음의 수식을 이용해서 계산한다.

$$\bar{\gamma}_k = \frac{p_k(n) g_k^{\theta(k)}(n)}{WN_0 + \sum_{(u,v)\in S(n)} p_{uv}(n) h_{uj}^{\phi(u,v)}(n) + p_{ij}^{*} h_{ik}^{\phi(i,j)}(n)} \quad \text{[수식 12]}$$

우변의 분자는 V2I link k 의 수신신호세기이고 분모의 첫 번째 항은 노이즈, 두 번째 항은 V2V 통신 자원을 할당 받아 스케줄된 V2V link로부터의 간섭, 세 번째 항은 V2V link (i,j) 를 추가적으로 스케줄 하였을 때 받는 간섭을 나타낸다.

만약 $I(n)$ 에 포함된 모든 V2I link k 의 SNR과 SINR 값의 차이 ($\gamma_k - \overline{\gamma_k}$) 가 V2V link의 SINR margin $\triangle\gamma^{V2I}$ 보다 작다면 ($\gamma_k - \overline{\gamma_k} \le \triangle\gamma^{V2I}$) Step 7)을 실행하고 $I(n)$ 에 포함된 V2I link 중 하나라도 SINR margin $\triangle\gamma^{V2I}$ 보다 큰 간섭을 받으면 ($\gamma_k - \overline{\gamma_k} > \triangle\gamma^{V2I}$) Step 4) 로 돌아간다.

○ Step 7) V2V link (i,j) 가 다른 V2V 통신에게 주는 간섭 고려

먼저 V2V link (i,j) 를 추가적으로 스케줄 하였을 때에 기존에 스케줄 된 V2V link (s,d) 의 SINR $\bar{\gamma}_{sd}$ 을 다음의 수식을 이용해서 계산한다.

$$\bar{\gamma}_{sd} = \frac{p_{sd}^{*} h_{sd}^{\phi(s,d)}(n)}{WN_0 + \sum_{k\in I(n)} p_k(n) g_d^{\theta(k)}(n) + \sum_{(u,v)\in S(s,d)} p_{uv}(n) h_{ud}^{\phi(u,v)}(n) + p_{ij}^{*} h_{ik}^{\phi(i,j)}(n)} \quad \text{[수식 13]}$$

우변의 분자는 V2V link (s,d) 의 수신신호세기이고 분모의 첫 번째 항은 노이즈, 두 번째 항은 V2I 통신으로부터의 간섭, 세 번째 항은 V2V link

(s,d) 자신을 제외하고 V2V 통신 자원을 할당 받아 스케줄된 V2V link로부터의 간섭, 네 번째 항은 V2V link (i,j) 를 추가적으로 스케줄 하였을 때 받는 간섭을 나타낸다. 이 때 분모의 세 번째 항에서 $S_{(s,d)}$ 는 스케줄된 V2V link의 집합 $S(n)$ 중에서 V2V link (s,d) 를 제외한 V2V link의 집합으로 $S_{(s,d)} = S(n) - (s,d)$ 로 정의 된다.

만약 V2V link (i,j) 를 추가하였을 때에 기존에 스케줄링 된 V2V link (s,d) 가 SINR margin $\triangle\gamma^{V2V}$ 보다 작은 간섭을 받으면 target SINR $\bar{\gamma}_{sd}^{target}$ 을 만족할 수 있다 ($\bar{\gamma}_{sd} \geq \bar{\gamma}_{sd}^{target}$). 반면에 V2V link (i,j) 를 추가함으로써 SINR margin $\triangle\gamma^{V2V}$ 보다 더 큰 간섭을 받는 경우에는 target SINR $\bar{\gamma}_{sd}^{target}$ 을 만족할 수 없다 ($\bar{\gamma}_{sd} < \bar{\gamma}_{sd}^{target}$). 따라서 만약 집합 $S(n)$ 에 포함된 모든 V2V link (s,d) 가 target SINR $\bar{\gamma}_{sd}^{target}$ 을 만족하면 Step 8) 을 실행하고 집합 $S(n)$ 에 포함된 V2V link (s,d) 중 하나라도 target SINR $\bar{\gamma}_{sd}^{target}$ 을 만족하지 못하면 Step 4) 를 실행한다.

○ Step 8) V2V link (i,j) 스케줄링 및 업데이트

만약 V2V link (i,j) 가 추가적으로 스케줄 되더라도 V2I 통신과 다른 V2V 통신과의 간섭 문제를 피할 수 있다면 다음과 같이 V2V link (i,j) 를 V2I Downlink slot n에 스케줄링하고 관련 변수들을 업데이트 한 후에 Step 9) 를 실행한다.

- $S(n) = S(n) \cup (i,j)$: V2V link (i,j) 를 V2I slot n 에 스케줄링
- $p_{ij}(n) = p_{ij}^{*}$: V2V link (i,j) 의 target SINR을 만족하는 전송 power p_{ij}^{*} 를 할당
- $P_i = P_i - p_{ij}^{*}$: V2V link (i,j) 의 source 차량의 남은 power를 업데이트
- $N_{(i,j)} = N_{(i,j)} - 1$: V2V link (i,j) 가 요청한 V2I slot의 수를 감소
- $V = V - (i,j)$: 스케줄된 V2V link (i,j) 를 집합 V 에서 제거

○ Step 9) V2V link (i,j) 와 동시에 스케줄이 불가능한 V2V link를 집합 V 에서 제거

차량은 half-duplex로 통신을 하기 때문에 data의 송수신을 동시에 할 수 없다. 따라서 한 V2V link의 source 차량은 다른 V2V link의 destination 차

량이 될 수 없다. 먼저, 스케줄된 V2V link (i,j) 의 source 차량이 다른 V2V link의 destination 차량인 V2V link의 집합 T_{ij} 을 다음의 수식을 이용해서 계산한다.

$T_{ij} = \{(n_1, n_2) \in V \mid i = n_2\}$ [수식 14]

그리고 집합 T_{ij} 를 집합 V 에서 제거하고 ($V = V - T_{ij}$) Step 10) 을 실행한다.

○ Step 10) V2V 자원 요청 집합 R 을 업데이트

만약 V2V link (i,j) 가 요청한 수의 V2I slot을 모두 할당 받으면 ($N_{(i,j)} = 0$) 해당 V2V link (i,j) 를 V2V 자원 요청 집합 R 에서 제거하고 ($R = R - (i,j)$) Step 11) 을 수행한다.

○ Step 11) time slot 인덱스 n을 업데이트

집합 V 에 아직 스케줄 할 수 있는 V2V link가 존재한다면 ($V \neq \varnothing$) Step 4) 를 수행하고 집합 V 에 스케줄 가능한 V2V link가 존재하지 않다면 ($V = \varnothing$) time slot의 인덱스를 증가시킨 후 ($n = n + 1$) Step 1) 로 돌아가서 V2V 통신의 스케줄링을 반복한다. 만약 현재 time slot이 frame의 마지막 time slot 이라면 ($n = N$) 현재 frame에서 V2V 통신의 스케줄링을 종료한다.

(다) 성능 분석

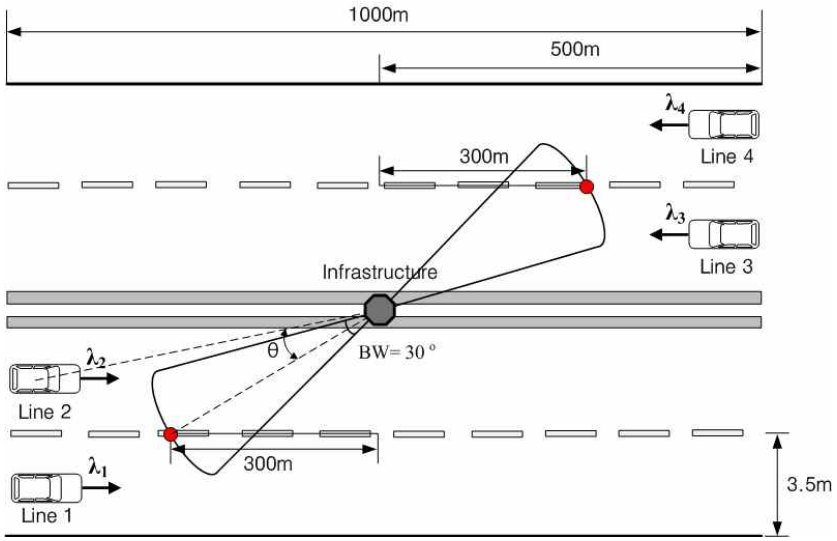


그림 3-124. V2V2I 통신 시스템의 시뮬레이션 환경

제안하는 BDMA 기반의 V2V2I 통신 시스템에서 스케줄링 방안의 성능을 확인하기 위하여 그림 [3-124]과 같은 시뮬레이션 환경을 가정하여 시뮬레이션을 수행하였다. 인프라를 중심으로 길이가 1000 m, 각 차선의 폭은 3.5 m 인 왕복 2 차선 도로를 고려하였으며 아래쪽의 1, 2 차선의 차량은 왼쪽에서 오른쪽 방향으로 진행하며 위쪽의 3, 4 차선의 차량은 오른쪽에서 왼쪽 방향으로 진행한다. 각 차량의 이동속도는 각 차선 1, 2, 3, 4 의 순서대로 각각 60 km/h, 45 km/h, 30 km/h, 55 km/h의 속도로 고정되어 이동하게 된다. 차량의 생성은 각 차선의 차량 arrival rate λ [vehicles/s] 에 따라서 차선의 끝에서 임의로 생성된다.

제안 방안에서는 symmetric traffic과 asymmetric traffic의 두 가지 경우에 대해서 시뮬레이션을 수행하였다. Symmetric traffic 경우에는 모든 차선에서 같은 arrival rate λ 로 차량을 생성하며 각 차선의 arrival rate λ 를 0.05 에서 0.5 vehicle/s 로 조절해가면서 시뮬레이션을 수행하였다. Asymmetric traffic 경우에는 한 방향의 arrival rate는 고정 시키고 (차선 3과 4의 arrival rate $\lambda_3 = \lambda_4$ 는 0.25로 고정) 다른 방향의 arrival rate를 변화시켰다 (차선 1과 2의 arrival rate $\lambda_1 = \lambda_2$ 0.05부터 0.5까지 변화). Discovery beam은 각 방향 별로 각각 형성되며 discovery beam을 수신한 차량은 인프라에 접속하게 된다.

Simulation time	10000s
Frame length	1s
Number of V2I slots in a frame	1
Max. tx power of infra & vehicle	43 dBm & 27 dBm
V2I & V2V beam width	30°
Number of V2I beams (B)	4
Number of requested V2I slots ($N_{(s,d)}$)	1
Range of V2V comm.	50 m
Path-loss model for V2I	$31.5 + 35\log_{10}(d) + 8$
Path-loss model for V2V	$59.7 + 18.5\log_{10}(d)$

그림 3-125. 시뮬레이션 파라미터

자세한 시뮬레이션 파라미터는 그림 [3-125]과 같이 정리하였다. 각각의 시뮬레이션의 기간은 10000 초이며 frame의 길이는 1 초이고 한 frame은 하나의 slot으로 구성되어 있다고 가정하였다. 인프라와 차량의 최대 전송 power는 각각 43 dBm과 27 dBm으로 설정하였다. 인프라가 형성하는 V2I beam과 차량이 형성하는 V2V beam의 beam width는 30 도로 동일하며 인프라가 동시에 형성 가능한 V2I beam의 개수는 최대 4 개로 설정하였다. 30 도의 beam을 사용할 때의 BDMA 전송 기술의 antenna model $A(\theta) = -\min[12(\frac{\theta}{\theta_{3dB}})^2, A_m] + G_A$ 에서 신호가 3dB 감쇠되는 beam-width $\theta_{3dB} = 17.5^\circ$, 최대 감쇠값 $A_m = 26dB$, 그리고 안테나 이득 $G_A = 20dBi$ 로 설정된다. 또한 V2V 통신의 전송 range를 50 m로 설정하여 V2V link의 source 차량과 destination 차량의 거리를 제한하였다. 또한 각각 V2I 통신과 V2V 통신의 path-loss을 다르게 설정하였다.

본 연구에서는 기존 방안으로 V2I 통신 시스템을 고려하고 제안 방안으로 V2V2I 통신 시스템을 고려하여 성능을 비교하고자 한다. 기존의 V2I 통신 시스템에서는 V2I 통신만 존재하기 때문에 기존 방안의 시스템 용량 $C_{con}(n)$ 은 V2I 통신의 용량 $C_{con}^{V2I}(n)$ 과 같다 ($C_{con}(n) = C_{con}^{V2I}(n)$). 기존 방안에서 V2I 통신은 V2V 통신으로부터 간섭을 받지 않기 때문에 기존 방안의 V2I 통신의 용량 $C_{con}^{V2I}(n)$ 은 다음과 같은 수식을 통해 구할 수 있다.

$$C_{con}^{V2I}(n) = \sum_{k \in I(n)} \frac{p_k(n) g_k^{\theta(k)}(n)}{WN_0} \quad \text{[수식 15]}$$

V2V2I 통신 시스템에서는 V2I 통신과 V2V 통신이 공존하기 때문에 제안 방안의 시스템 용량 $C_{pro}(n)$ 은 V2I 통신과 V2V 통신의 용량의 합 ($C_{pro}^{V2I}(n) + C_{pro}^{V2V}(n)$) 과 같다 ($C_{pro}(n) = C_{pro}^{V2I}(n) + C_{pro}^{V2V}(n)$). 제안 방안에서 V2I 통신은 V2V 통신으로부터 간섭을 받기 때문에 제안 방안의 V2I 통신의 용량 $C_{pro}^{V2I}(n)$ 은 다음과 같은 수식을 통해 구할 수 있다.

$$C_{pro}^{V2I}(n) = \sum_{k \in I(n)} \left(\frac{p_k(n) g_k^{\theta(k)}(n)}{WN_0 + \sum_{(i,j) \in S(n)} p_{ij}(n) h_{ik}^{\phi(i,j)}(n)} \right) \quad \text{[수식 16]}$$

분모의 첫 번째 항은 노이즈, 두 번째 항은 V2V 통신으로부터의 간섭을 나타낸다. 제안 방안에서 V2V 통신은 V2I 통신과 다른 V2V 통신으로부터 간섭을 받기 때문에 제안 방안의 V2V 통신의 용량 $C_{pro}^{V2V}(n)$ 은 다음과 같은 수식을 통해 구할 수 있다.

$$C_{pro}^{V2V}(n)=\sum_{(i,j)\in S(n)}\left(\frac{p_{ij}(n)h_{ij}^{\phi(i,j)}(n)}{WN_0+\sum_{k\in I(n)}p_k(n)g_j^{\theta(k)}(n)+\sum_{(u,v)\in S(u,v)}p_{uv}(n)h_{uj}^{\phi(u,v)}(n)}\right)$$ [수식 17]

분모의 첫 번째 항은 노이즈, 두 번째 항은 V2I 통신으로부터의 간섭, 세 번째 항은 다른 V2V 통신으로 부터의 간섭을 나타낸다.

시뮬레이션 결과에서 기존 방안 (Conventional scheme, V2I) 은 V2I 통신 시스템의 용량 $C_{con}(n)=C_{con}^{V2I}(n)$ 을 나타내며 제안 방안 (Proposed scheme) 은 각각 V2V2I 통신 시스템에서 V2I 통신의 용량 $C_{pro}^{V2I}(n)$ (Proposed scheme, V2I) 과 V2V2I 통신 시스템의 전체 용량 $C_{pro}(n)=C_{pro}^{V2I}(n)+C_{pro}^{V2V}(n)$ (Proposed scheme, V2I+V2V) 을 각각 나타내었다.

시뮬레이션에서 다음과 같은 시스템 파라미터를 바꿔가면서 경향성을 살펴보고자 한다. 1) p : 인프라에 접속한 차량의 "V2V 자원 요청 message"의 arrival rate [request/s] 으로 시뮬레이션에서 모든 차량의 p 는 동일하게 설정하였다. 2) γ^{target} : V2V link의 target SINR로 시뮬레이션에서 모든 V2V link의 target SINR은 $\bar{\gamma}_{ij}^{target}$ 로서 동일하게 설정하였다 ($\bar{\gamma}_{ij}^{target}=\gamma^{target}$). 3) $\triangle\gamma^{V2I}$: V2I link의 SINR margin으로서 각각의 V2I link 의 SINR margin은 동일하게 설정하였다. 4) $\triangle\gamma^{V2V}$: V2V link의 SINR margin으로서 각각의 V2V link의 SINR margin은 동일하게 설정하였다.

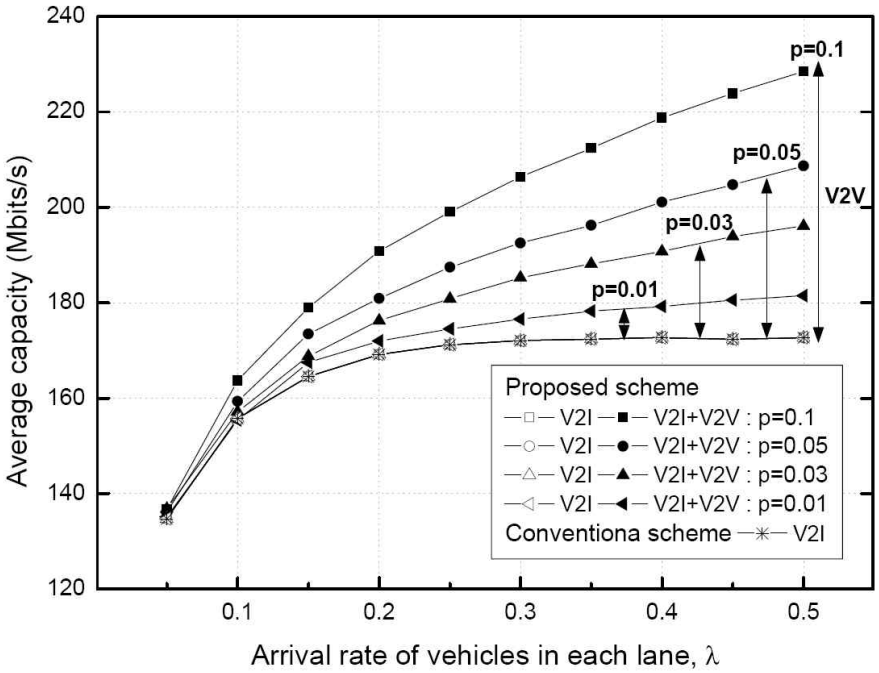


그림 3-126. Symmetric traffic에서 V2V 통신 요청의 arrival rate에 따른 평균 시스템 용량

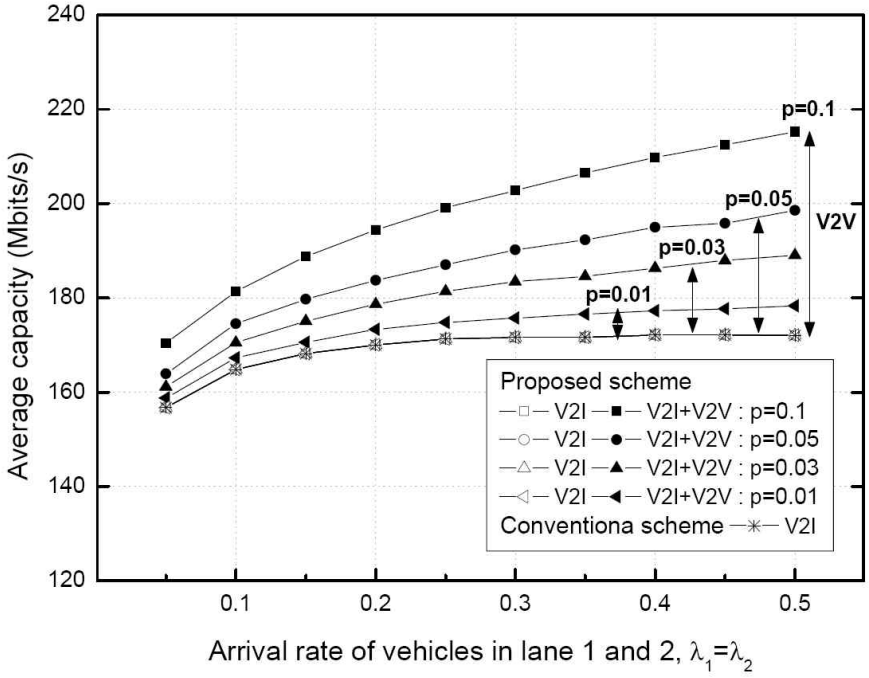


그림 3-127. Asymmetric traffic에서 V2V 통신 요청의 arrival rate에 따른 평균 시스템 용량

[그림 3-126]와 [그림 3-127]은 각각 symmetric traffic과 asymmetric traffic에서 V2V 통신 요청 arrival rate p 에 따른 V2I 통신 시스템 (기존 방안) 과 V2V2I 통신 시스템 (제안 방안) 의 평균 용량을 보여준다. 이 때 V2V 통신 요청 arrival rate p 은 각각 0.01, 0.03, 0.05, 0.1로 변화 시켰고 다른 시스템 파라미터들은 다음의 값으로 고정하였다.

- V2V link의 target SINR $\gamma^{target} = 5dB$
- V2I link의 SINR margin $\triangle\gamma^{V2I} = 1dB$
- V2V link의 SINR margin $\triangle\gamma^{V2V} = 1dB$

기본적으로 traffic load가 증가함에 따라서 (각 차선에서 차량의 arrival rate가 증가) 기존 방안과 제안 방안에서 V2I 통신의 용량은 모두 포화 상태에 도달한다. V2V2I 통신 시스템에서 제안하는 V2V 통신 스케줄링 방안을 사용함으로써 traffic load가 증가하거나 V2V 통신 요청 arrival rate p 이 증가함에 따라서 동시에 더 많은 V2V link를 스케줄링 할 수 있다. 따라서 V2V 통신 요청 arrival rate p 이 증가함에 따라서 V2V 통신의 용량이 증가하고 제안 방안의 전체 시스템 용량을 기존 방안 대비 증가시킬 수 있음을

확인할 수 있다. 이때 V2V 통신 요청 arrival rate p 값과 상관없이 제안 방안의 V2I 통신 용량은 기존 방안의 V2I 통신 용량과 거의 동일하게 유지됨을 확인할 수 있다. 그 이유는 제안 방안에서 V2V 통신 요청이 많이 들어오더라도 V2I link의 SINR margin $\triangle\gamma^{V2I}$ 을 고려하여 V2V 통신을 스케줄링하기 때문이다. 제안 방안은 symmetric traffic과 asymmetric traffic에 상관없이 모두 비슷한 성능 패턴을 보여준다.

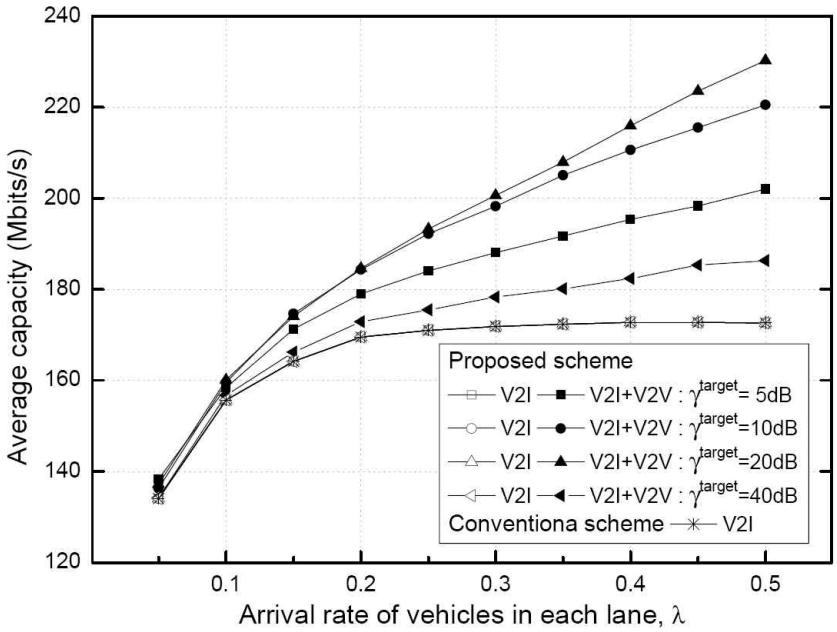


그림 128. Symmetric traffic에서 V2V target SINR에 따른 평균 시스템 용량

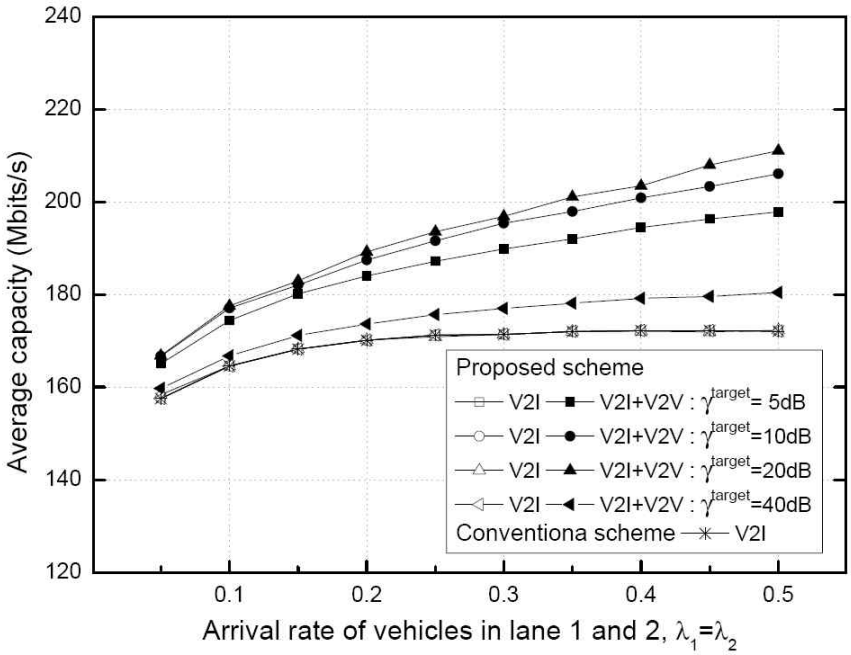


그림 3-129. Asymmetric traffic에서 V2V target SINR에 따른 평균시스템 용량

[그림 3-128]과 [그림 3-129]는 각각 symmetric traffic과 asymmetric traffic에서 V2V link의 target SINR γ^{target} 에 따른 V2I 통신 시스템 (기존 방안) 과 V2V2I 통신 시스템 (제안 방안) 의 평균 용량을 보여준다. 이 때 V2V link의 target SINR γ^{target} 은 각각 5 dB, 10 dB, 20 dB, 40 dB로 변화 시켰고 다른 시스템 파라미터들은 다음의 값으로 고정하였다.

- V2V 통신 요청 arrival rate $p = 0.05$
- V2I link의 SINR margin $\triangle\gamma^{V2I} = 1dB$
- V2V link의 SINR margin $\triangle\gamma^{V2V} = 1dB$

기본적으로 V2V link의 target SINR γ^{target} 이 증가함에 따라서 각각의 V2V link의 용량이 증가한다. 따라서 V2V link의 target SINR γ^{target} 이 증가함에 따라서 V2V 통신의 용량이 증가하고 제안 방안의 전체 시스템 용량을 기존 방안 대비 증가시킬 수 있음을 확인할 수 있다. 이때 V2V link의 target SINR γ^{target}의 값과는 상관없이 제안 방안의 V2I 통신 용량은 기존 방안의 V2I 통신 용량과 거의 동일하게 유지됨을 확인할 수 있다. 그 이유는 제안 방안에서 V2V link의 target SINR이 증가함에 따라서 V2I 통신에게 주

는 간섭이 증가해도 V2I link의 SINR margin $\triangle\gamma^{V2I}$ 을 고려하여 V2V 통신을 스케줄링하기 때문이다.

하지만 V2V link의 target SINR γ^{target} 이 증가함에 따라서 V2V 통신의 용량 증가의 폭은 점점 작아지고 V2V link의 target SINR이 너무 큰 경우 ($\gamma^{target} = 40dB$) 에는 오히려 V2V 통신의 용량이 감소한다. 그 이유는 V2V link의 target SINR이 너무 큰 경우에는 V2V 통신이 V2I 통신이나 다른 V2V 통신에게 주는 간섭이 커져서 V2I link의 SINR margin $\triangle\gamma^{V2I}$ 혹은 V2V link의 SINR margin $\triangle\gamma^{V2V}$ 을 만족시킬 수 없기 때문에 해당 V2V link를 스케줄링하지 않기 때문이다. 제안 방안은 symmetric traffic과 asymmetric traffic에 상관없이 모두 비슷한 성능 패턴을 보여준다.

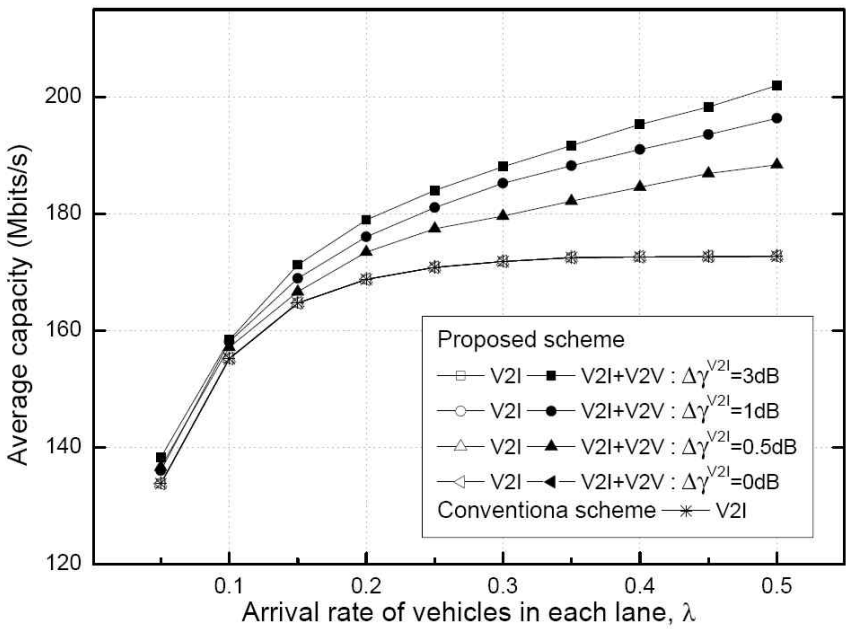


그림 3-130. Symmetric traffic에서 V2I link의 SINR margin에 따른 평균 시스템 용량

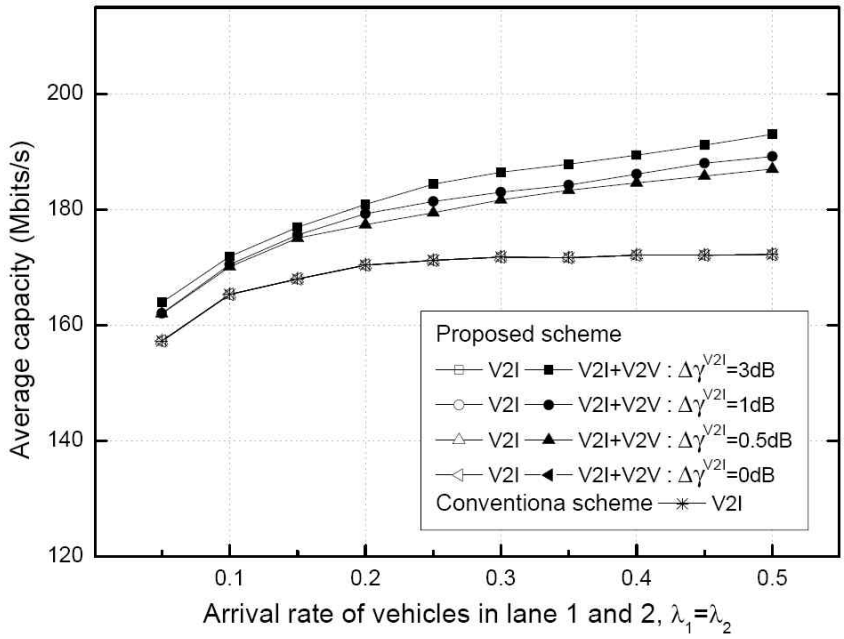


그림 3-131. Asymmetric traffic에서 V2I link의 SINR margin에 따른 평균 시스템 용량

[그림 3-130]과 [그림 3-131]는 각각 symmetric traffic과 asymmetric traffic에서 V2I link의 SINR margin $\triangle\gamma^{V2I}$ 에 따른 V2I 통신 시스템 (기존 방안) 과 V2V2I 통신 시스템 (제안 방안) 의 평균 용량을 보여준다. 이 때 V2I link의 SINR margin $\triangle\gamma^{V2I}$ 은 각각 3 dB, 1 dB, 0.5 dB, 0dB로 변화시켰고 다른 시스템 파라미터들은 다음의 값으로 고정하였다.

- V2V 통신 요청 arrival rate $p = 0.03$
- V2V link의 target SINR $\gamma^{target} = 5dB$
- V2V link의 SINR margin $\triangle\gamma^{V2V} = 1dB$

V2I link의 SINR margin $\triangle\gamma^{V2I}$ 이 증가함에 따라서 V2I 통신이 V2V 통신으로부터 허용 가능한 간섭의 양이 증가하기 때문에 V2I 통신에 더 큰 간섭을 미치는 V2V link를 스케줄링 할 수 있다. 따라서 V2I link의 SINR margin $\triangle\gamma^{V2I}$ 이 증가함에 따라서 V2V 통신의 용량이 증가하고 제안 방안의 전체 시스템 용량을 기존 방안 대비 증가시킬 수 있음을 확인할 수 있다. 이때 V2I link의 SINR margin $\triangle\gamma^{V2I}$ 의 값과는 상관없이 제안 방안의 V2I 통신 용량은 기존 방안의 V2I 통신 용량과 거의 동일하게 유지됨을 확인할 수 있다. 그 이유는 제안하는 V2V 통신 스케줄링 방안에서 V2I 통신 중인

차량이 V2V link의 destination 차량이 되지 않도록 스케줄링하기 때문에 V2I 통신이 V2V 통신으로부터 큰 간섭을 피할 수 있기 때문이다. 하지만 V2I link의 SINR margin $\triangle\gamma^{V2I}=0dB$ 인 경우에는 V2I 통신이 V2V 통신으로부터 간섭을 전혀 허용하지 않기 때문에 V2V 통신의 용량은 0이 되고 기존 방안과 제안 방안의 시스템 용량은 같아지게 된다. 제안 방안은 symmetric traffic과 asymmetric traffic에 상관없이 모두 비슷한 성능 패턴을 보여준다.

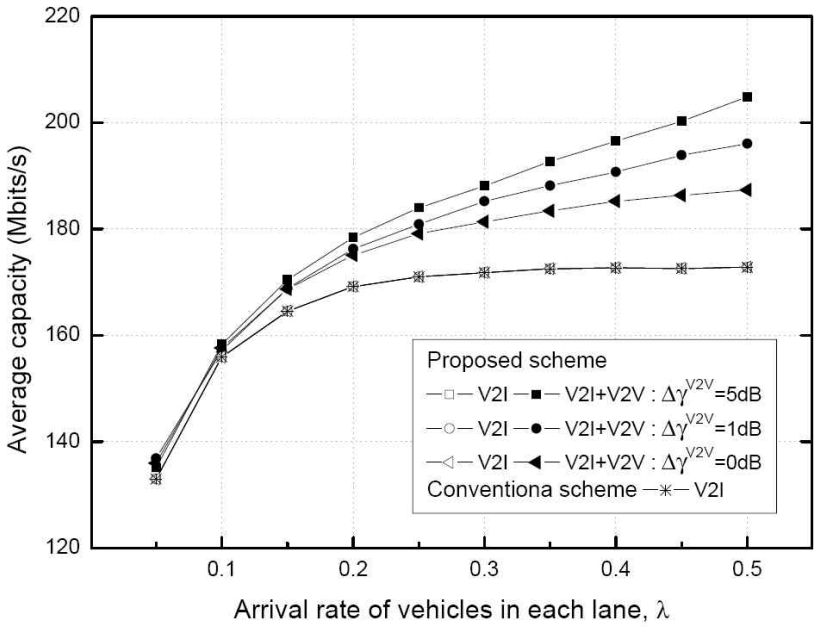


그림 3-132. Symmetric traffic에서 V2V link의 SINR margin에 따른 평균 시스템 용량

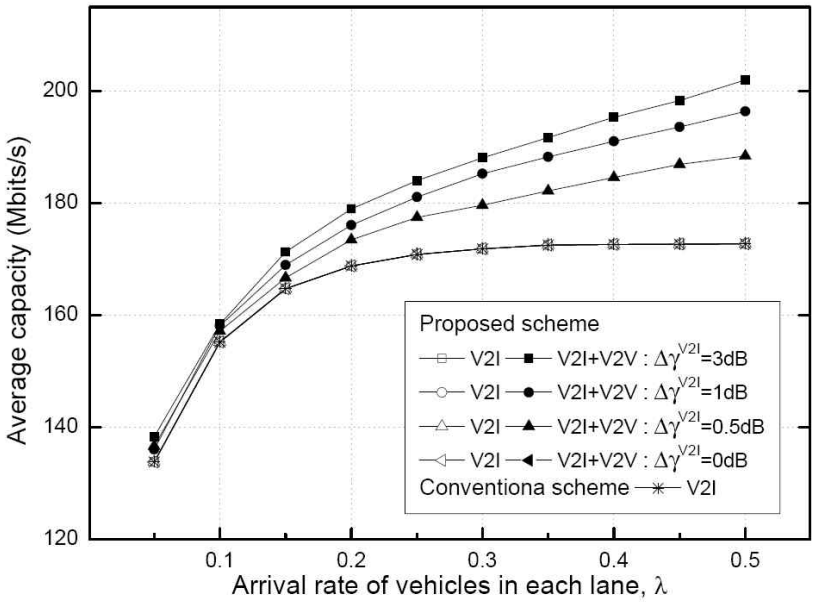


그림 3-133. Asymmetric traffic에서 V2V link의 SINR margin에 따른 평균 시스템 용량

[그림 3-132]와 [그림 3-133]은 각각 symmetric traffic과 asymmetric traffic에서 V2V link의 SINR margin $\triangle\gamma^{V2V}$ 에 따른 V2I 통신 시스템 (기존 방안) 과 V2V2I 통신 시스템 (제안 방안) 의 평균 용량을 보여준다. 이 때 V2V link의 SINR margin $\triangle\gamma^{V2V}$ 은 각각 5 dB, 1 dB, 0 dB로 변화 시켰고 다른 시스템 파라미터들은 다음의 값으로 고정하였다.

- V2V 통신 요청 arrival rate $p = 0.03$
- V2V link의 target SINR $\gamma^{target} = 5dB$
- V2I link의 SINR margin $\triangle\gamma^{V2I} = 1dB$

V2V link의 SINR margin $\triangle\gamma^{V2V}$ 이 증가함에 따라서 V2V 통신이 V2I 통신과 다른 V2V 통신으로부터 허용 가능한 간섭의 양이 증가하기 때문에 더 많은 V2V link를 스케줄링 할 수 있다. 따라서 V2V link의 SINR margin $\triangle\gamma^{V2V}$ 이 증가함에 따라서 V2V 통신의 용량이 증가하고 제안 방안의 전체 시스템 용량을 기존 방안 대비 증가시킬 수 있음을 확인할 수 있다. 이때 V2V link의 SINR margin $\triangle\gamma^{V2V}$ 의 값과는 상관없이 제안 방안의 V2I 통신 용량은 기존 방안의 V2I 통신 용량과 거의 동일하게 유지됨을 확인할 수 있다. 그 이유는 제안 방안에서 더 많은 V2V link를 스케줄링 하더라도 V2I

link의 SINR margin $\triangle\gamma^{V2I}$ 을 고려하기 때문이다.

V2V link의 SINR margin $\triangle\gamma^{V2V}=0dB$ 인 경우에는 V2V 통신이 V2I 통신이나 다른 V2V 통신으로부터 간섭을 전혀 허용하지 않는다. 하지만 제안하는 V2V 통신 스케줄링 방안을 사용하면 V2V2I 통신 시스템의 공간 자원을 최대한 재활용하여 간섭을 주지 않는 V2V link를 찾아서 스케줄링하기 때문에 V2V 통신의 용량이 0이 되지 않는다. 제안 방안은 symmetric traffic 과 asymmetric traffic에 상관없이 모두 비슷한 성능 패턴을 보여준다.

다. BDMA 기반의 차량 통신 시스템에서 multi-hop relay 방안

(가) 기존 연구 및 문제점

○ 차량 통신의 multi-hop relay 기존 연구

차량 통신의 multi-hop relay는 데이터를 전송하려는 source 차량과 데이터를 수신하는 destination 차량 사이에 중계를 담당하는 relay 차량이 존재하여 데이터의 전송을 효율적으로 돕는 데에 그 목적이 있다.

기존의 multi-hop relay 연구는 대부분 V2V 환경을 중심으로 집중되어있으며 multi-hop routing이 그 주를 이루고 있다. 기존 연구의 multi-hop routing 기법은 크게 세 가지로 분류할 수 있다. 1) Topology-based routing 2) Position-based routing 3) Cluster-based routing. Topology-based routing은 V2V communication network의 topology 변화에 적응적으로 최적의 routing path를 찾는 것을 목표로 하고 있고 position-based routing은 차량의 위치를 기반으로 routing path를 찾는다. 그리고 Cluster-based routing은 일정한 기준으로 차량들 사이에 cluster를 형성하여 cluster 간의 데이터 전송을 multi-hop routing으로 수행한다.

V2I 환경에서는 차량과 인프라와의 접속시간이 인프라의 커버리지와 차량의 속도에 의해 제한되어 있다. 또한, 차량의 이동 속도가 빨라서 차량이 인프라의 커버리지 내에 머무르는 시간이 상대적으로 짧다. 따라서 multi-hop relay를 통해서 인프라의 커버리지를 넓혀주는 연구가 주로 수행되었다 . 이때 relay는 인프라의 커버리지 밖에서 커버리지 쪽으로 접근하는 차량이 인프라로 접속하는 것을 도와준다. 따라서 해당 차량이 커버리지 내에 들어오기 전부터 relay를 통한 데이터 전송이 이루어 질 수 있어서 차량과 인프라의 접속시간을 증가시킬 수 있다는 장점이 있다.

○ 차량 통신의 multi-hop relay 특징

차량 통신에서 multi-hop relay 전송은 다음과 같은 특징을 갖는다. 1) V2I 통신 환경과 V2V 통신 환경 모두에서 multi-hop relay 전송이 가능하다. 2) V2I 통신 환경에서는 V2I 통신의 direct link의 채널보다 propagation loss가 상대적으로 적은 V2V 통신의 multi-hop link의 채널이 더 좋은 경우가 있다. 3) V2V 통신 환경의 multi-hop relay는 주로 safety application에서 데이터의 broadcasting을 위해 사용된다.

○ 차량 통신의 multi-hop relay의 문제점

기존에 연구가 많이 이루어 졌던 V2V 환경에서의 multi-hop relay에서는 많은 차량이 같은 데이터를 broadcasting 하다 보면 불필요한 전송으로 자원의 낭비가 발생할 수 있다. 또한, 차량의 수가 많을수록 다양한 path가 존재하고 그 중에서 최적의 route를 찾는 것이 어렵다는 문제가 있다.

BDMA 전송 기술 기반의 V2V2I 통신 시스템 환경에서는 인프라에서 차량으로 데이터를 전송할 때 direct로 전송할 것인지, relay를 통해 전송할 것인지를 판단할 수 있는 기준이 필요하다. 또한, 차량의 고속 이동성으로 인해 도로 위의 traffic이 동적으로 변하고 있기 때문에 multi-hop 전송을 위한 최적의 relay를 찾는 방안이 필요하다.

○ V2V2I 통신 시스템에서 multi-hop relay 동작 시나리오

1) Multi-hop relay를 이용한 시스템 용량 증대

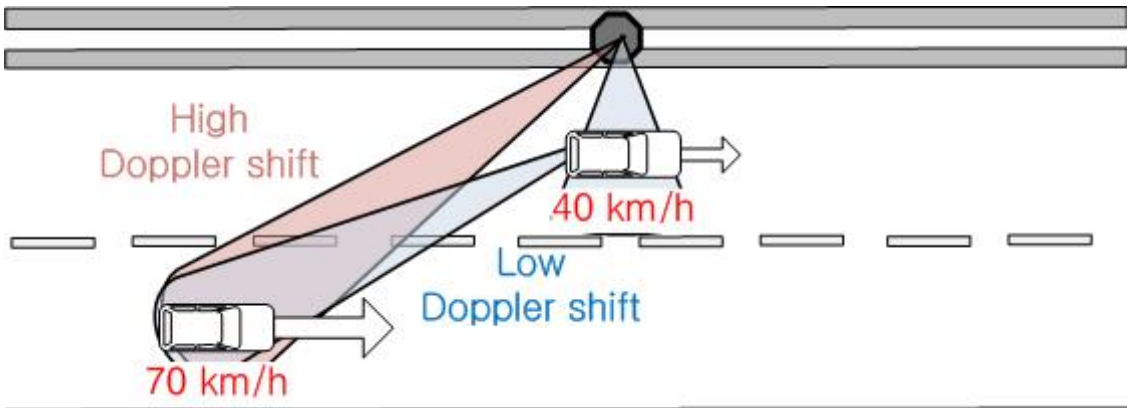


그림 3-134. Multi-hop relay 이용한 시스템 용량 증대

V2V2I 통신 시스템에서는 차량의 고속 이동성 V2I 채널에서 큰 Doppler shift 영향을 받게 된다. 이러한 Doppler shift 영향은 차량의 속도가 빠를수록 더 심해져 차량의 수신 신호 세기를 낮추기 때문에 차량의 QoS를 만족시키지 못할 수도 있다. 하지만 V2V 채널의 경우에는 차량들이 같은 방향으로 이동하고 있기 때문에 상대적인 속도가 V2I 채널의 경우보다 적고 따라서 Doppler shift의 영향도 상대적으로 덜하다. [그림 3-134]에서 두 번째 차선 (일반 차선) 에서 70 km/h의 비교적 고속으로 움직이는 승용차에게 데이터를 전송하려고하면 인프라와 차량 사이의 채널은 차량의 높은 이동 속도 때문에 Doppler shift의 영향이 크다. 반면에 첫 번째 차선 (버스 전용차선) 에서 40 km/h의 비교적 저속으로 움직이는 버스는 인프라에 상대적으로 가까이 위치하여 해당 차량을 relay로 이용하여 multi-hop 전송을 하게 되면 상대 속도가 낮고 path-loss로 인한 신호의 감쇄효과도 적기 때문에 더 좋은 채널을 통해 전송을 할 수 있게 된다.

따라서 상대속도가 낮은 relay 차량을 이용하여 multi-hop 전송을 하게 되면 시스템의 용량을 증대시킬 수 있는 효과를 가져 온다. 하지만 relay 차량을 선택하는 과정에서 destination 차량보다 인프라와의 거리가 더 먼 차량을 relay로 선택할 경우 path-loss가 증가하여 오히려 시스템의 성능이 저하될 가능성이 있으므로 최적의 relay를 선택하는 방안이 필요하다.

2) Multi-hop relay를 이용한 인프라의 coverage extension

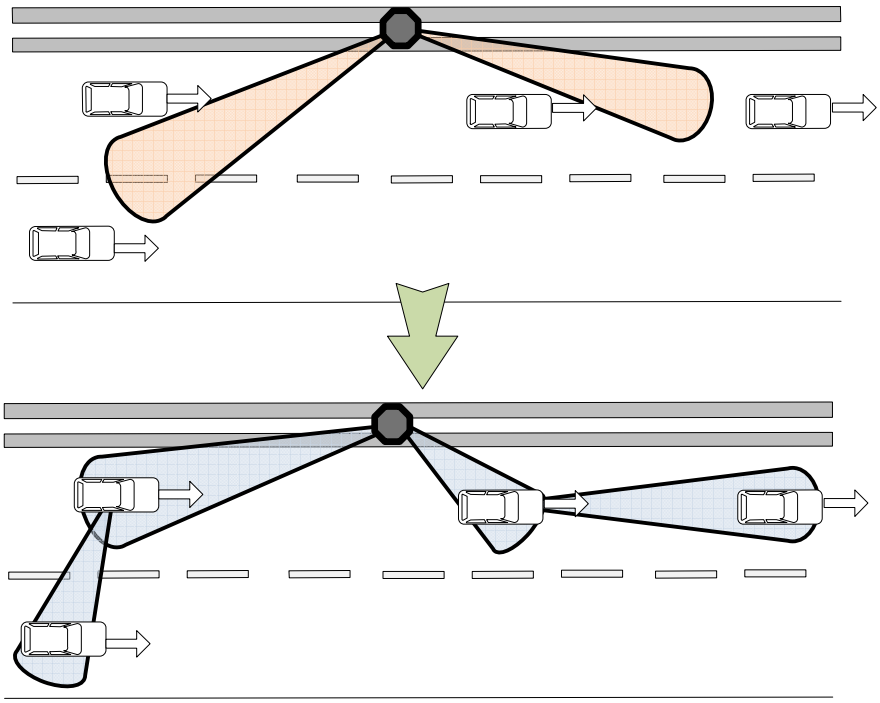


그림 3-135. Multi-hop relay를 이용한 인프라의 coverage extension

일반적으로 인프라에서 전송한 신호는 인프라에서 거리가 멀어짐에 따라 그 세기가 감소하게 되므로 신호가 약하게 도달하는 음영 지역이 존재할 수 있다. 이 음영 지역은 인프라의 커버리지 근처에서 주로 나타나고 커버리지 밖의 영역에서는 수신 신호의 세기가 너무 약해서 신호를 제대로 수신할 수 없다. V2I 통신 시스템에서는 차량이 인프라의 커버리지 내에 존재할 때에만 차량과 인프라 간의 통신이 이루어 질 수 있는데 일반적으로 차량의 속도가 빠르기 때문에 차량이 인프라의 커버리지 내에 머무르는 시간은 제한적이다. 따라서 relay를 통한 multi-hop 전송을 하게 되면 인프라의 제한된 커버리지 영역을 확장하는 효과를 가져 올 수 있고 이것은 차량이 인프라에 접속되어 있는 시간을 증가시켜준다. 따라서 차량과 인프라는 더 많은 데이터를 주고받을 수 있기 때문에 시스템의 성능을 향상시킬 수 있다.

[그림 3-135]은 multi-hop relay 사용을 통해 인프라의 커버리지를 증가시키는 예를 보여주고 있다. 먼저 인프라의 커버리지 밖에서 커버리지 안쪽으로 진입하려고 하는 차량에 대해서 커버리지 내에 있는 relay 차량을 통해 multi-hop 전송을 이용하여 인프라로의 접속을 도와준다. 또한, 커버리지를 빠져나가는 차량에 대해서 커버리지 내에 존재하는 relay 차량을 통해

multi-hop으로 전송하면 인프라의 커버리지를 확장할 수 있다. 하지만 multi-hop relay를 통해서 인프라의 커버리지를 확대하기 위해서는 인프라의 커버리지 근처에 relay 차량이 존재해야한다.

(나) 제안 방안

본 연구에서는 BDMA 전송 기술 기반의 V2V2I 통신 시스템에서 relay 차량을 사용한 multi-hop 전송을 통해 인프라에서 전송한 데이터를 destination 차량까지 효율적으로 전송하는 방안을 제안하고자 한다. 특히, 간섭을 고려하여 최적의 relay를 선택하는 방안을 제안하고 선택된 relay의 자원할당도 고려하여 V2V2I 통신 시스템의 용량을 개선하는 것을 목표로 한다.

○ BDMA 기반의 V2V2I 통신 시스템의 서비스 환경

인프라는 기존의 cellular 또는 long-range broadband (e.g. IEEE 802.16 standard) network에서 사용하는 BS (Base Station) 를 가정하였다. 인프라와 차량은 모두 적응적으로 빔을 형성할 수 있는 BDMA 안테나를 장착하였다고 가정한다. 차량은 rooftop에 안테나를 장착하고 일정한 속력으로 이동한다.

○ BDMA 기반의 V2V2I 통신 시스템의 서비스 시나리오

BDMA 기반의 V2V2I 통신 시스템에서는 인프라와 차량, 차량과 차량의 두 가지 통신 방식이 존재한다. 먼저 discovery beam을 수신하여 인프라에 접속한 차량은 인프라에게 서비스를 요청하고 인프라는 서비스를 요청한 차량들에게 non-safety 데이터 (e.g. Internet data) 를 V2I data slot을 통해 전송한다. 차량은 GPS를 탑재하여 자신의 위치, 속도 및 방향을 포함하고 있는 차량 정보 메시지를 주기적으로 인프라에 전송하여 인프라가 차량의 정보를 알고 있다고 가정하였다. 인프라는 차량으로부터 받은 차량 정보 메시지를 통해 차량의 위치 및 채널 상태를 추정할 수 있다. V2V 통신에서는 V2V slot을 사용해 차량끼리 주기적으로 control 메시지를 주고받기 때문에 차량 사이의 채널을 추정할 수 있다고 가정한다.

○ 시스템 모델

본 연구에서는 하나의 인프라가 존재하고 그 인프라의 커버리지 영역 내

에 다수의 차량이 존재하는 single cell V2V2I 통신 시스템을 고려한다. 제안 방안은 TDMA 방식으로 동작하고 매 time slot 마다 인프라에서 스케줄링을 통해 서비스를 제공할 destination 차량을 선택한다. 제안 방안은 매 time slot마다 스케줄링이 된 destination 차량에 대해 적절한 relay를 선택하고 선택된 relay의 자원 할당 방안을 제안함으로써 V2V2I 통신 시스템의 용량을 최대화 하고자 한다.

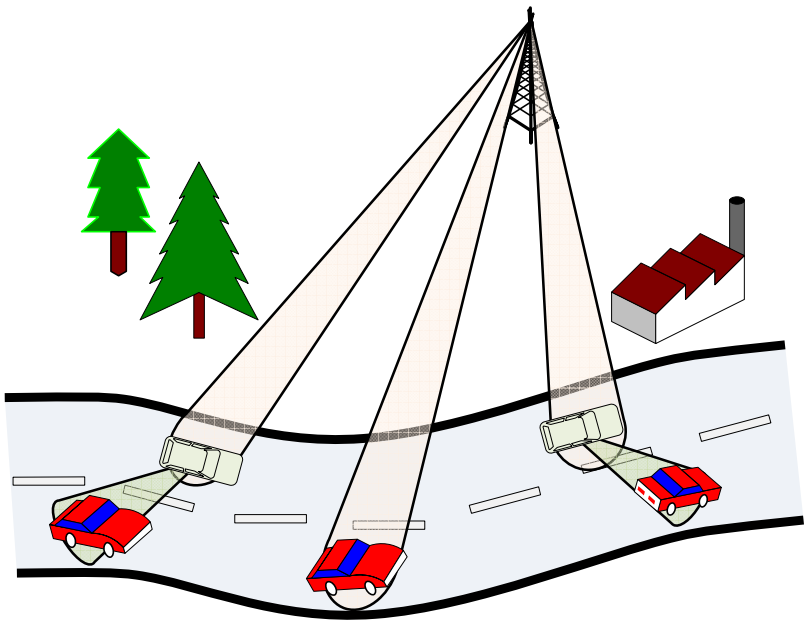


그림 3-136. BDMA 기반 V2V2I 통신 시스템에서 multi-hop 전송과 direct 전송의 예

[그림 3-136]는 본 연구에서 고려하는 BDMA기반 V2V2I 통신 시스템의 전송 시나리오 예를 나타낸다. 본 연구에서 언급하는 multi-hop relay는 한 대의 relay 차량을 이용하여 전송하는 two-hop relay를 의미한다. 제안 방안에서 인프라가 multi-hop으로 전송하는 경우 relay 차량은 인프라로 부터 수신한 신호를 decoding하여 destination 차량으로 전송하는 decode and forward (DF) 방식을 사용한다. 이때 relay는 인프라로부터 전송되는 신호를 항상 정확하게 decoding 할 수 있다고 가정하였다.

인프라에서 destination 차량으로 직접 전송하는 경우를 "direct V2I 통신"이라고 정의하였다. 한편, 인프라에서 destination 차량으로 전송 시에 relay 차량을 이용하는 경우, 두 번의 전송 과정이 필요한데 인프라에서 relay 차량으로의 전송 과정을 1st phase 라 부르고 그 때의 전송을 “multi-hop 1st

phase 통신" 이라고 정의한다. 그리고 relay 차량에서 destination 차량으로의 전송과정을 2nd phase라 부르고 그 때의 전송을 "multi-hop 2nd phase 통신" 이라고 정의한다. BDMA 기반의 V2V2I 통신 시스템에서는 TDMA 방식을 사용하므로 multi-hop으로 전송 시 한 time slot을 1st phase와 2nd phase에서 나누어 사용하게 된다.

인프라가 destination 차량 i 에게 direct로 데이터를 전송하는 경우와 relay를 통하여 multi-hop으로 전송하는 경우 각각 전송에 사용되는 time slot 자원의 분배는 다음과 같다.

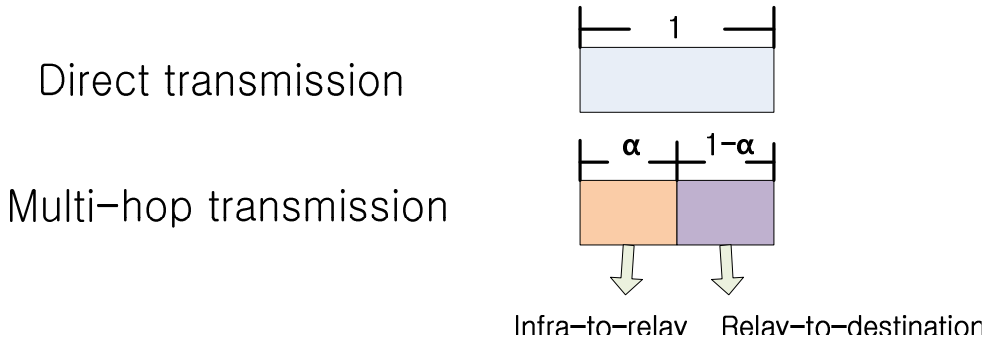


그림 3-137. Direct 전송과 multi-hop 전송의 자원 사용 비교

[그림 3-137]에서 보는 바와 같이 인프라가 relay를 사용하지 않고 direct 전송을 하는 경우에는 하나의 time slot을 전부 사용한다. 반면에 relay를 사용하여 multi-hop으로 전송하는 경우에는 하나의 time slot을 인프라와 relay가 나눠서 사용하게 된다. 이 때 인프라가 relay 차량으로의 전송에 사용하는 time slot의 비율을 α 라고 하면 relay 차량이 destination 차량으로의 전송에 사용하는 time slot의 비율은 $1-\alpha$ 이다. 본 연구의 제안 방안에서는 $\alpha = 0.5$ 의 값을 사용하여 multi-hop 전송 시에 인프라가 사용하는 자원의 양과 relay 차량이 사용하는 자원의 양은 같도록 설정하였다 (direct 전송 시 인프라가 사용하는 time slot 자원의 양의 절반).

Time slot t 에서 인프라의 커버리지 영역 내에 존재하는 차량들 중에서 인프라에 의해 스케줄링 된 destination 차량은 총 V 개 존재하며 그 집합을 V(t) = {1,..., V} 라고 정의한다. 반면에 time slot t 에서 인프라의 커버리지 영역 내에 존재하지만 인프라에 의해 스케줄링 되지 않은 차량은 총 R 개 존재하며 그 집합을 R(t) = {1,..., R} 라고 정의한다. 집합 R(t) 에 포함된 차량은 집합 V(t) 에 포함된 차량의 relay로 사용될 수 있는 relay candidate set

이다. 집합 V(t) 와 집합 R(t) 에 포함된 차량은 각각 i 와 j 를 index로 사용한다.

인프라에서 최대 형성 가능한 beam 의 개수는 안테나 개수와 인프라의 전송 power에 의해 제한된다. 또한 인프라가 형성하는 beam 의 개수가 너무 많으면 beam 간 간섭이 심해지기 때문에 인프라에서 beam 의 개수를 계속 늘릴 수는 없다. 따라서 본 연구에서는 인프라가 한 time slot에서 동시에 형성 가능한 beam 의 개수를 B 개로 제한하였다. 그리고 인프라의 최대 전송 파워를 P 라고 정의하고 차량의 최대 전송 파워를 Pv로 정의하였다. 그리고 인프라가 time slot t 에 스케줄링 된 차량에 형성하는 beam의 파워는 p(t) 로 동일하게 할당하였다 (한 time slot에서 빔의 개수를 최대로 형성 시: p(t) = P/B). Destination 차량의 relay 차량이 정해지면 해당 relay 차량은 1st phase에서 인프라로부터 받은 데이터를 2nd phase에서 destination 차량에 전달해주어야 한다. 이 때 relay 차량의 전송 파워를 p(t) = (pij(t)) 라고 정의하며 pij(t) 는 destination 차량 i 에 대한 relay 차량 j 의 전송 파워이다.

본 연구에서 집합 V(t) 에 포함된 각 destination 차량은 채널 상황에 따라서 인프라로부터 direct로 데이터를 수신할 수도 있고 relay 차량을 통해서 multi-hop으로 서비스를 수신할 수도 있다. 만약, relay 차량을 통해서 데이터를 수신한다면 어떤 destination 차량이 어떤 relay 차량을 통해서 multi-hop으로 서비스를 수신하는지를 나타내야 한다. 이를 위해서 relay usage indicator matrix A(t) = (ai,j(t)) 를 정의하였고 A(t) 는 $V \times R$의 size를 갖는다. 따라서 집합 R(t) 에 포함된 차량 j 가 destination 차량 i 의 relay 차량으로 할당되면 ai,j(t) 는 1의 값을 갖고 아니면 0의 값을 갖는다.

위에서 정의한 relay 차량의 전송 파워 pij(t) 와 relay usage indicator ai,j(t) 는 다음과 같은 constraints를 만족해야 한다.

1) $0 \le p_i^j(t) \le P_v \; \forall i \in$ V(t) , $\forall j \in$ R(t) [수식 18]

2) $\sum_{j=1}^{R} a_{i,j}(t) \le 1 \;\; \forall i \in$ V(t) [수식 19]

3) $\sum_{j=1}^{V} a_{i,j}(t) \le 1 \;\; \forall j \in$ R(t) [수식 20]

Constraint 1)은 relay 차량에서 사용하는 power의 constraint이다. 집합 R(t) 에 포함된 차량 j 가 destination 차량 i의 relay 차량으로 선택 되지 않았다면 pij(t) 는 0의 값을 가져야 하고 차량 j 가 relay 차량으로 선택 되었다면 pij(t) 는 0 보다 크고 relay에서 사용할 수 있는 최대 파워 P_v 보다 작

아야 한다.

Constraint 2)는 하나의 destination 차량 i 는 relay 차량을 최대 하나만 사용될 수 있다는 constraint이다. 즉, 각각의 destination 차량 i 에 두 대 이상의 relay 차량이 할당될 수 없다는 것이다.

Constraint 3)는 하나의 relay 차량 j 는 최대 하나의 destination 차량에 데이터를 전송해야 한다는 constraint이다. 즉, 각각의 relay 차량 j 는 두 대 이상의 destination 차량에게 동시에 데이터를 전송할 수 없다.

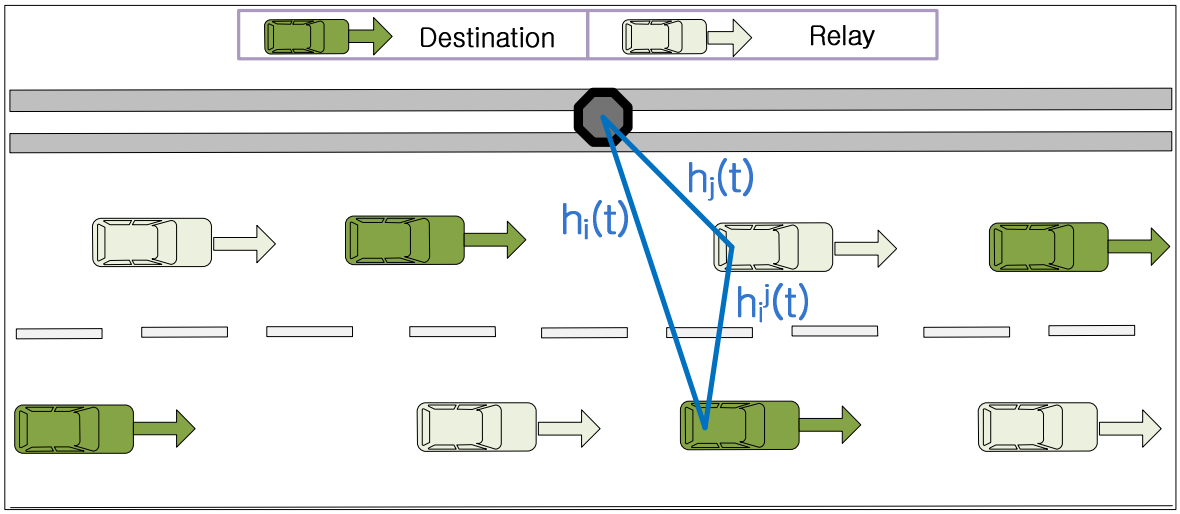


그림 3-138. V2V2I multi-hop 통신에서 사용되는 채널

[그림 3-138]은 제안 방안에서 사용되는 채널을 나타낸 그림으로 인프라와 destination 차량, 인프라와 relay 차량, relay 차량과 destination 차량 간의 채널은 각각 hi(t), hj(t), hij(t)로 정의하였다. 인프라와 차량 사이의 채널인 hi(t)와 hj(t)는 차량으로부터 주기적으로 전송되는 uplink feedback을 통해서 인프라가 알 수 있고, destination 차량 i와 relay 차량 j 사이의 채널인 hij(t)는 주기적인 V2V 통신을 통해 추정 가능하며 해당 정보를 인프라에 보고한다. 따라서 인프라는 인프라와 차량 사이의 채널 정보와 차량 사이의 채널 정보를 다 알고 있는 상태라고 가정한다. 위에서 언급한 세 가지의 채널 이득 h(t)는 path-loss model과 antenna mode을 동시에 고려하였다.

일반적으로 path loss model은 다음과 같은 수식으로 표현된다.

$$PL(l) = 10\log_{10}\frac{1}{K}\left(\frac{l}{l_0}\right)^{\gamma} \quad \text{[수식 21]}$$

위의 식에서 l 은 신호가 생성된 지점으로부터 path-loss를 측정하고자 하

는 지점까지의 거리이고 l_0 는 안테나 far-field에 대한 기준 거리이다. 또한 γ는 path-loss exponent이고 K 는 상수이다.

BDMA 전송 기술에서 antenna model 은 다음과 같은 수식으로 표현된다 [6].

$$A(\theta,\theta_{3dB}) = -\min[12(\frac{\theta}{\theta_{3dB}})^2, A_m] + G_A \quad \text{[수식 22]}$$

θ 는 beam 방향과 차량 방향의 각도, θ_{3dB} 는 신호가 3dB 감쇠되는 beam-width, $A_m = 20 + 10\log10(\frac{70}{\theta_{3dB}})$ 은 최대 감쇠값을 나타낸다. [수식 22] 의 우변의 첫 번째 항은 normalization된 값이고 이 값을 원래 값으로 보상해주는 안테나 이득 G_A 는 $G_A = 14 + 10\log_{10}(\frac{70}{\theta_{3dB}})$ 로 표현된다. 따라서 antenna model의 gain은 beam 방향과 차량 방향 사이의 각도인 θ 에 따라 결정되고 []그림 3-139]는 $\theta_{3dB} = 70^\circ$ 일 때의 안테나 이득을 dB 단위로 나타낸 그래프이다.

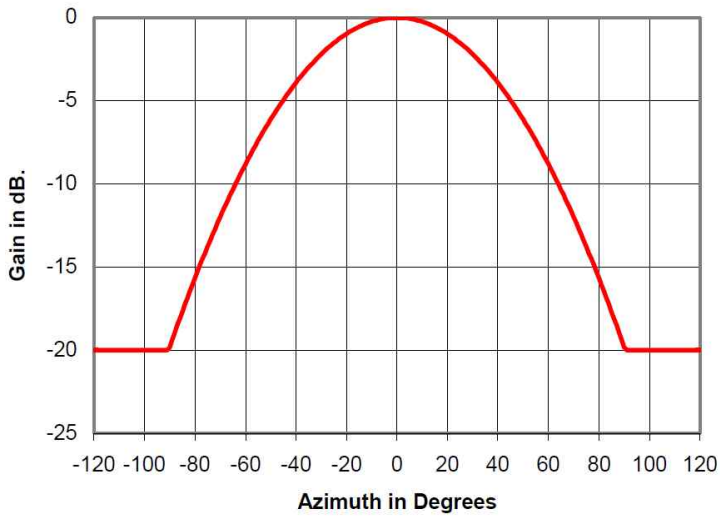


그림 3-139. 안테나 이득 모델의 예 (Θ_{3dB} = 70°)

기존의 cellular 또는 long-range broadband network 의 BS이 제안 방안에서 인프라로 사용되기 때문에 인프라는 차량 보다 높은 위치에 설치되지만 일반적으로 도로에서 멀리 떨어져 있다. 따라서 인프라와 destination 차량 사이에는 산이나 건물, 주변 차량 등의 장애물이 존재할 수 있고, 인프

라와 destination 차량 사이의 거리가 멀어서 NLOS 환경이 형성되어 채널 상황이 좋지 않은 차량이 존재할 수 있다. 반면, 차량과 차량 사이는 비교적 거리가 가깝고 주변 장애물이 적은 LOS 환경이 형성되기 때문에 채널 상황이 비교적 좋다.

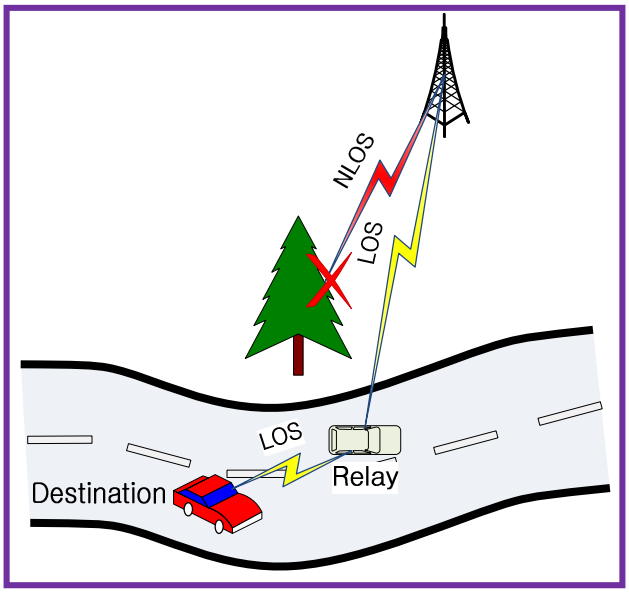


그림 3-140. Multi-hop relay 채널 환경

[그림 3-140]은 BDMA 기반의 V2V2I 통신 시스템에서 multi-hop relay 채널 환경을 나타낸다. 본 연구에서 인프라와 거리가 멀거나 주변 장애물 등에 의해서 채널 상황이 좋지 않은 destination 차량에 대해서는 인프라와의 채널을 NLOS로 가정하였다. 특히, 승용차의 경우에는 차량의 높이가 낮아서 안테나의 위치도 비교적 낮기 때문에 인프라와의 채널이 NLOS일 확률이 더 크다. 따라서 NLOS 채널 환경인 destination 차량이 인프라로부터 직접 데이터를 수신할 경우에는 destination 차량에서 수신 신호의 강도가 약해진다. 반면에 인프라와의 거리가 비교적 가깝고 안테나의 위치가 높은 차량 (예를 들면 버스나 트럭) 의 경우에는 인프라와 LOS 채널이 보장될 수 있다. 이러한 LOS 채널 환경의 차량이 NLOS 채널 환경의 destination 차량과 가까이 있는 경우에는 두 차량 사이의 채널은 LOS 환경이 보장될 수 있다. 따라서 인프라가 LOS 채널 환경의 차량을 relay 차량으로 선택하여 NLOS 채널 환경의 destination 차량에게 multi-hop으로 데이터를 전송을 하게 되면 1st phase에서와 2nd phase에서의 propagation loss를 줄일 수 있기 때문에

destination 차량의 수신 신호의 세기가 증가하게 된다.

1) (다) 간섭을 고려한 최적의 relay 차량 선택 방안

○ V2V2I multi-hop 통신 간섭 모델

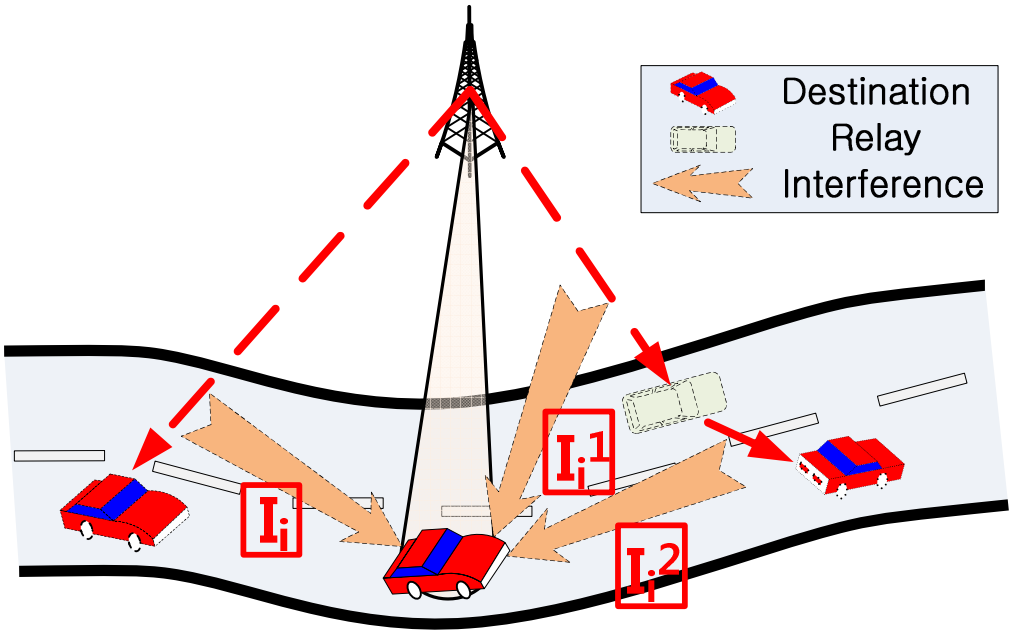


그림 3-141. V2V2I multi-hop 간섭 모델

[그림 3-141]는 제안하는 BDMA 전송 기술 기반의 V2V2I 시스템에서 multi-hop 전송의 간섭 모델을 나타낸 것이다. 인프라가 destination 차량 i 에게 direct로 전송하는 경우에 다른 destination 차량에 미치는 간섭을 I_i 로 정의하였다. 인프라가 destination 차량 i 에게 multi-hop으로 전송하는 경우에 multi-hop 1st phase 통신이 다른 destination 차량에 미치는 간섭을 I_i^1 로 정의하였다. 마찬가지로 인프라가 destination 차량 i 에게 multi-hop으로 전송하는 경우에 multi-hop 2nd phase 통신이 다른 destination 차량에 미치는 간섭을 I_i^2 로 정의하였다.

○ V2V2I multi-hop 통신의 채널 용량

1) Direct V2I 전송 시

인프라가 destination 차량 i 에게 direct로 전송하는 경우에 다른 destination 차량들로부터 받는 간섭을 고려한 destination 차량 i 의 SINR은 다음과 같이 1st phase와 2nd phase로 나누어 계산할 수 있다.

$$SINR_i^{D1} = \frac{p(t)h_i(t)}{\sum_{l=1(l\neq i)}^{V}[(1-\sum_{j=1}^{R}a_{l,j}(t))I_l \; + \; \sum_{j=1}^{R}a_{l,j}(t)I_l^1] + \sigma^2} \quad \text{[수식 23]}$$

$$SINR_i^{D2} = \frac{p(t)h_i(t)}{\sum_{l=1(l\neq i)}^{V}[(1-\sum_{j=1}^{R}a_{l,j}(t))I_l \; + \; \sum_{j=1}^{R}a_{l,j}(t)I_l^2] + \sigma^2} \quad \text{[수식 24]}$$

Destination 차량 i 가 다른 destination 차량 $l(\in V(t))$ 의 통신으로부터 받는 간섭은 인프라가 차량 l 에게 direct로 전송할 때와 relay를 통해 multi-hop으로 전송할 때를 모두 고려해야 한다. 인프라가 destination 차량 i 에게 direct로 전송하는 동시에 인프라가 차량 l 에게 multi-hop으로 전송하는 경우에는 차량 l 의 1st phase와 2nd phase 전송에서 destination i가 받는 간섭이 각각 다르기 때문이다.

Destination 차량 i 가 1st phase에서 받는 총 간섭은 인프라가 다른 destination 차량에게 direct로 전송하는 direct V2I 통신으로부터의 간섭 ([수식 23]에서 $\sum_{l=1(l\neq i)}^{V}(1-\sum_{j=1}^{R}a_{l,j}(t))I_l$) 과 인프라가 다른 destination 차량에게 multi-hop relay를 통해 전송하는 multi-hop 1st phase 통신으로부터의 간섭 ([수식 23]에서 $\sum_{l=1(l\neq i)}^{V}\sum_{j=1}^{R}a_{l,j}(t)I_l^1$) 의 합이다.

Destination 차량 i 가 2nd phase에서 받는 총 간섭은 인프라가 다른 destination 차량에게 direct로 전송하는 direct V2I 통신으로부터의 간섭 ([수식 24]에서 $\sum_{l=1(l\neq i)}^{V}(1-\sum_{j=1}^{R}a_{l,j}(t))I_l$) 과 인프라가 다른 destination 차량에게 multi-hop relay를 통해 전송하는 multi-hop 2nd phase 통신으로부터의 간섭 ([수식 24]에서 $\sum_{l=1(l\neq i)}^{V}\sum_{j=1}^{R}a_{l,j}(t)I_l^2$) 의 합이다.

Destination 차량 i 의 $SINR_i^{D1}$ 과 $SINR_i^{D2}$ 을 바탕으로 destination 차량 i

의 1st phase의 채널 용량 C_i^{D1} 과 2nd phase의 채널 용량 C_i^{D1} 을 다음과 같이 계산할 수 있다.

$$C_i^{D1} = \frac{1}{2} log_2(1 + SINR_i^{D1}) \quad [수식\ 25]$$

$$C_i^{D2} = \frac{1}{2} log_2(1 + SINR_i^{D2}) \quad [수식\ 26]$$

그리고 인프라가 destination 차량 i 에게 direct로 전송할 때의 채널 용량 C_i^D 은 1st phase와 2nd phase의 채널 용량의 합으로 나타낼 수 있다.

$$C_i^D = C_i^{D1} + C_i^{D2} \quad [수식\ 27]$$

2) Multi-hop 전송 시

인프라가 destination 차량 i 에게 relay 차량 j 를 사용하여 multi-hop으로 전송하는 경우에 다른 destination 차량들로부터 받는 간섭을 고려한 destination 차량 i의 SINR은 다음과 같이 1st phase와 2nd phase로 나누어 계산할 수 있다.

$$SINR_{ij}^1 = \frac{p(t)h_j(t)}{\sum_{l=1(l\neq i)}^{V}[(1-\sum_{j=1}^{R}a_{l,j}(t))I_l \ + \ \sum_{j=1}^{R}a_{l,j}(t)I_l^1] + \sigma^2} \quad [수식\ 28]$$

$$SINR_{ij}^2 = \frac{p_i^j(t)h_i^j(t)}{\sum_{l=1(l\neq i)}^{V}[(1-\sum_{j=1}^{R}a_{l,j}(t))I_l \ + \ \sum_{j=1}^{R}a_{l,j}(t)I_l^2] + \sigma^2} \quad [수식\ 29]$$

Destination 차량 i 가 다른 destination 차량 l 의 통신으로부터 받는 간섭은 인프라가 차량 l 에게 direct로 전송할 때와 relay를 통해 multi-hop으로 전송할 때를 모두 고려해야 한다.

Destination 차량 i 의 multi-hop 1st phase 통신에서 relay 차량 j 가 받는 총 간섭은 인프라가 다른 destination 차량에게 direct로 전송하는 direct V2I 통신으로부터의 간섭 ([수식 28]에서 $\sum_{l=1(l\neq i)}^{V}(1-\sum_{j=1}^{R}a_{l,j}(t))I_l$) 과 인프라가 다른 destination 차량에게 multi-hop relay를 통해 전송하는 multi-hop 1st phase 통신으로부터의 간섭 ([수식 28]에서 $\sum_{l=1(l\neq i)}^{V}\sum_{j=1}^{R}a_{l,j}(t)I_l^1$) 의 합이다.

Destination 차량 i 의 multi-hop 2nd phase 통신에서 destination 차량 i 가 받는 총 간섭은 인프라가 다른 destination 차량에게 direct로 전송하는

direct V2I 통신으로부터의 간섭 ([수식 29]에서 $\sum_{l=1(l\neq i)}^{V}(1-\sum_{j=1}^{R}a_{l,j}(t))I_l$) 과 인프라가 다른 destination 차량에게 multi-hop relay를 통해 전송하는 multi-hop 1st phase 통신으로부터의 간섭 ([수식 29]에서 $\sum_{l=1(l\neq i)}^{V}\sum_{j=1}^{R}a_{l,j}(t)I_l^1$) 의 합이다.

따라서 multi-hop relay를 사용하는 destination 차량 i 의 1st phase 채널 용량 C_{ij}^1 과 2nd phase 채널 용량 C_{ij}^2 은 다음과 같이 계산할 수 있다.

$C_{ij}^1 = \frac{1}{2}log_2(1+SINR_{ij}^1)$ [수식 30]

$C_{ij}^2 = \frac{1}{2}log_2(1+SINR_{ij}^2)$ [수식 31]

인프라가 relay 차량을 통해서 destination 차량에게 데이터를 전송하는 경우의 채널 용량 C_{ij}^M 은 인프라가 destination 차량 i 에게 direct로 전송할 때의 채널 용량 C_i^D 과는 다르게 계산된다. 그 이유는 1st phase와 2nd phase의 채널 용량 중 더 작은 값을 갖는 통신 link가 bottleneck이 되어 end-to-end 채널 용량은 multi-hop 1st phase 통신과 multi-hop 2nd phase 통신의 두 채널 용량 중에서 최소값이 되기 때문이다. 따라서 인프라가 destination 차량 i 에게 multi-hop으로 전송할 때의 채널 용량C_{ij}^M 은 다음과 같이 나타낼 수 있다.

$C_{ij}^M = \min(C_{ij}^1, C_{ij}^2)$ [수식 32]

○ 간섭을 고려한 최적의 relay 차량 선택 방안

본 연구에서 제안하는 최적의 relay 차량 선택 방안은 relay를 사용하게 됨으로써 multi-hop 통신에서 받는 간섭과 다른 destination 차량에게 미치는 간섭을 모두 고려한다. 먼저 인프라에서 direct로 전송하는 경우에 얻을 수 있는 채널 용량 C_i^D 과 relay 차량을 사용하여 multi-hop으로 전송하는 경우에 얻을 수 있는 채널 용량 C_{ij}^M 을 계산한다. 그리고 multi-hop으로 전송하는 경우의 채널 용량이 더 높을 경우에는 relay 차량을 사용하여 multi-hop으로 전송을 하고 그렇지 않으면 direct로 전송한다. Relay 차량을 사용하여 multi-hop으로 전송하는 경우, 사용가능한 relay 차량 중에서 가장

높은 multi-hop 통신의 채널 용량을 달성할 수 있는 relay 차량을 선택한다. 제안 방안은 다음과 같이 크게 네 단계로 구성된다.

1) Direct 전송 시 채널 용량 계산
2) Multi-hop 전송 시 채널 용량 계산
3) 사용가능한 relay 후보 subset 계산
4) 최적의 relay 차량 선택

한 time slot 에서 인프라에 의해 스케줄링된 destination 차량들이 결정되면 인프라는 동시에 다수의 beam 을 형성하여 해당 destination 차량들에게 데이터를 전송한다. 이 때 beam 간의 간섭이 발생하기 때문에 간섭으로 인해 각 destination 차량의 수신 SINR이 감소하므로 전체 시스템의 용량이 저하될 수 있다. 따라서 relay 차량의 사용 여부를 판단할 때에 인프라와 destination 차량의 통신 간에 서로 미치는 간섭을 고려하여 인프라와 destination 차량의 통신에서 얻을 수 있는 채널 용량을 최대화하는 것을 목표로 한다.

제안하는 최적의 relay 선택 방안은 하나의 destination 차량에 대해 relay 차량의 사용 여부와 relay 차량을 사용할 경우에 relay 차량의 자원 할당을 결정한다. 다음으로 destination 차량 i 에 대해서 제안하는 최적의 relay 선택 방안을 각 단계별로 자세히 설명하고자 한다.

Step 1) Direct 전송 시 채널 용량 계산

Step 1) 에서는 인프라가 destination 차량 i 에게 direct V2I 통신을 할 경우에 destination 차량 i 의 채널 용량 C_i^D 을 다음과 같이 계산한다. 다른 destination 차량이 direct V2I 통신을 할 경우에는 direct V2I 통신으로부터의 간섭을 계산하고 multi-hop 통신을 할 경우에는 multi-hop 1st phase 통신과 multi-hop 2nd phase 통신에서 오는 간섭을 각각 계산한다. 그리고 계산한 간섭으로부터 destination 차량 i 의 1st phase의 SINR $SINR_i^{D1}$ 과 2nd phase의 SINR $SINR_i^{D2}$ 을 각각 계산하고 이로부터 1st phase의 채널 용량 C_i^{D1} 과 2nd phase의 채널 용량 C_i^{D2} 을 각각 계산한다. 그리고 최종적으로 인프라가 destination 차량 i 에게 direct로 전송할 경우의 채널 용량 $C_i^D = C_i^{D1} + C_i^{D2}$ 을 계산한다.

Step 2) Multi-hop 전송 시 채널 용량 계산

Step 2) 에서는 인프라가 relay 차량 j 를 통해서 destination 차량 i 에게

multi-hop으로 전송할 경우에 destination 차량 i 의 채널 용량 C_{ij}^{M} 을 계산하고 동시에 relay 차량 j의 전송파워를 할당한다.

인프라가 destination 차량 i 로 multi-hop 전송을 할 때 사용할 수 있는 relay 차량은 집합 R(t)에 포함되어 있는 차량 중의 하나이다. Destination 차량 i 와의 거리가 D 이상인 차량들은 destination 차량 i 와 거리가 멀어서 multi-hop 2nd phase 통신의 propagation loss가 크다. 또한, destination 차량 i 와의 거리가 D 이상인 차량들은 인프라와의 거리가 멀어서 multi-hop 1st phase 통신의 propagation loss가 크다. 따라서 destination 차량 i 와의 거리가 D 이상인 차량들을 relay 차량으로 선택하였을 때의 이득이 없다. 따라서 제안 방안에서는 집합 R(t)에 포함된 차량 중에서 destination 차량 i 와의 거리가 D 이하인 차량들만 relay 차량으로 선택을 고려한다.

Step 1) 과 같은 방법으로 relay 차량 j 를 선택하였을 때에 destination 차량 i 가 다른 destination 차량으로부터 받는 간섭을 계산하여 destination 차량 i 의 1st phase의 SINR $SINR_{ij}^{1}$ 을 먼저 계산하고 2nd phase의 SINR $SINR_{i}^{D2}$ 이 $SINR_{ij}^{1} = SINR_{ij}^{2}$ 을 만족하도록 relay 차량 j 의 파워를 결정한다. 만약 결정된 파워가 차량의 최대 전송 파워 P_v 보다 크면 relay 차량 j 의 파워를 P_v로 설정하여 2nd phase의 SINR $SINR_{ij}^{2}$ 을 계산한다. 상기 결정된 destination 차량 i 의 1st phase와 2nd phase의 SINR로 부터 1st phase의 채널 용량 C_{ij}^{1} 과 2nd phase의 채널 용량 C_{ij}^{2} 을 각각 계산하고 최종적으로 인프라가 relay 차량 j 를 통해서 destination 차량 i 에게 multi-hop으로 전송할 경우의 채널 용량 $C_{ij}^{M} = \min(C_{ij}^{1}, C_{ij}^{2})$ 을 계산한다.

Step 3) 사용가능한 relay 후보 subset 계산

인프라가 relay 차량 j 를 통해서 destination 차량 i 에게 multi-hop으로 전송할 경우의 채널 용량 C_{ij}^{M} 이 인프라가 destination 차량 i 에게 direct로 전송할 경우의 채널 용량 C_{i}^{D} 보다 큰 경우에는 relay 차량 j 를 destination 차량 i 의 relay 후보의 집합인 Ri(t) 에 포함시킨다.

Step 4) 최적의 relay 차량 선택

Destination 차량 i 의 relay 후보의 집합인 Ri(t) 중에서 최적의 relay 차량을 선택하기 위해서 먼저 인프라가 destination 차량 i 에게 direct로 전송

할 때의 시스템 용량 C^D 을 다음과 같이 계산한다. Destination 차량 i 의 채널 용량은 Step 1)에서 결정된 C_i^D 이고 다른 destination 차량의 채널 용량은 인프라가 해당 차량에게 direct로 전송하는지 multi-hop으로 전송하는지에 따라서 계산한다. 만약, 인프라가 다른 destination 차량에게 direct로 전송할지 multi-hop으로 전송할지 아직 결정하지 않았다면 해당 destination 차량의 채널 용량은 direct로 전송하였을 때의 채널 용량을 계산한다. 따라서 모든 destination 차량의 채널 용량이 구해지면 destination 차량의 채널 용량의 합인 시스템 용량 C^D 을 계산한다.

만약 Step 3) 에서 구한 destination 차량 i 의 relay 후보 차량의 집합 Ri(t) 이 공집합이면 인프라는 destination 차량 i 에게 direct 전송을 하기 때문에 시스템 용량은 C^D 가 된다. 반면에 relay 후보 차량의 집합 Ri(t) 이 공집합이 아니라면 각각의 relay 후보 차량 j에 대해서 인프라가 relay 차량을 통해서 destination 차량 i 에게 multi-hop으로 전송할 때의 시스템 용량 C^M 을 계산한다. Destination 차량 i 의 채널 용량은 Step 2) 에서 결정된 C_{ij}^M 이고 다른 destination 차량의 채널 용량은 인프라가 해당 차량에게 direct로 전송하는지 multi-hop으로 전송하는지에 따라서 계산한다. 만약, 인프라가 다른 destination 차량에게 direct로 전송할지 multi-hop으로 전송할지 아직 결정하지 않았다면 해당 destination 차량의 채널 용량은 direct로 전송하였을 때의 채널 용량을 계산한다. 따라서 모든 destination 차량의 채널 용량이 구해지면 destination 차량의 채널 용량의 합인 시스템 용량 C^M 을 계산한다. 다음으로 relay 후보 차량 중에서 가장 큰 시스템 용량 C^M 을 달성하는 차량을 destination 차량의 최적의 relay 차량으로 선택하고 해당 relay 차량에게 Step 2) 에서 결정된 전송 파워를 사용한다. 만약 Ri(t) 에 포함된 모든 relay 후보 차량에 대해서 $C^M > C^D$ 을 만족하지 않으면 인프라는 destination 차량 i에게 direct로 전송한다.

(라) 선택된 relay 차량의 자원 할당 방안

제안 방안에서는 간섭을 고려한 최적의 relay 차량 선택 방안의 Step 2)에서 destination 차량 i 의 multi-hop 1st phase 통신의 SINR $SINR_{ij}^1$ 을 계산한 후에 multi-hop 2nd phase 통신의 SINR $SINR_i^{D2}$ 이 $SINR_{ij}^1 = SINR_{ij}^2$ 을 만족하도록 relay 차량 j 의 전송 파워를 결정한다. 만약 결정된 relay 차량의 파워가 차량의 최대 전송 파워 P_v 보다 크다면 relay 차량 j 의 파워를

P_v로 설정한다.

제안 방안에서 만약 relay 차량 j 에서 $SINR_{ij}^1 = SINR_{ij}^2$ 의 조건을 만족하도록 결정한 파워가 P_v 를 넘지 않더라도 결정된 파워를 그대로 사용한다. Relay 차량 j 에서 더 많은 전송 파워를 사용하여 multi-hop 2nd phase 통신의 채널 용량이 multi-hop 1st phase 채널 용량보다 크더라도 ($C_{ij}^1 < C_{ij}^2$) destination 차량 i의 end-to-end multi-hop 채널 용량은 $C_{ij}^M = \min(C_{ij}^1, C_{ij}^2) = C_{ij}^1$ 와 같이 multi-hop 1st phase 통신의 채널 용량으로 제한된다. 또한 relay 차량 j 가 multi-hop 2nd phase 통신에서 전송 파워를 많이 사용할 경우에는 다른 차량에게 미치는 간섭이 커지게 되므로 전체 네트워크 측면에서 좋지 않은 결과를 가져오게 된다. 따라서 제안 방안에서는 relay 차량의 전송 파워를 낭비하지 않으면서 다른 차량에 미치는 간섭을 최소화 하도록 자원 할당을 수행할 수 있다.

(마) 성능 분석

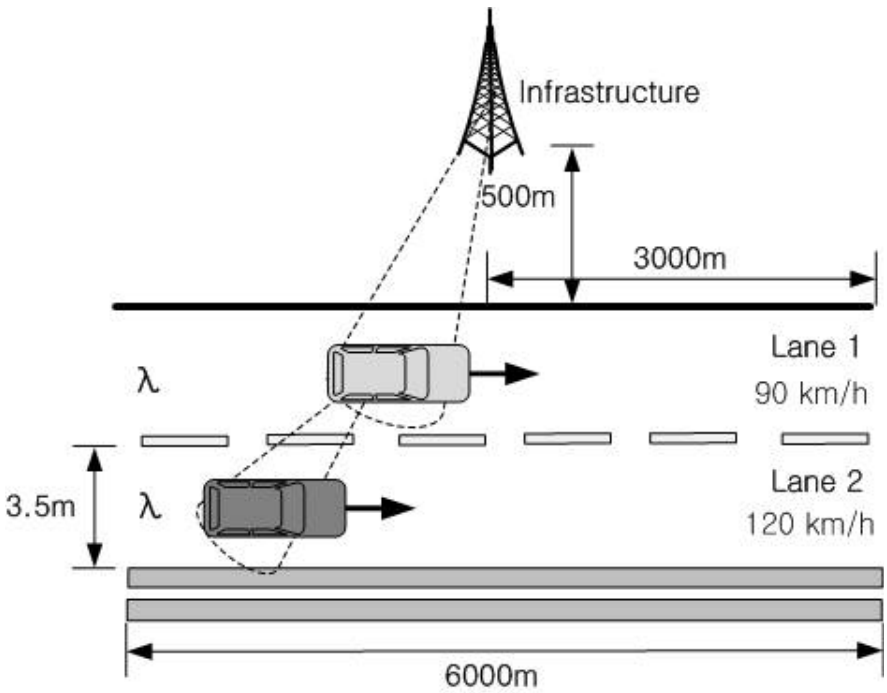


그림 3-142. V2V2I multi-hop relay 시뮬레이션 환경

제안하는 BDMA 기반의 V2V2I 통신 시스템에서 multi-hop relay 방안의 성능을 확인하기 위하여 [그림 3-142]와 같은 시뮬레이션 환경을 가정하

여 시뮬레이션을 수행하였다. 인프라를 중심으로 길이가 6000 m, 각 차선의 폭은 3.5 m인 편도 2 차선 도로를 고려하였으며 인프라에 더 가까운 차선을 Lane 1이라 정의하고 나머지 차선을 Lane 2라 정의하였다. Lane 1에서 모든 차량은 90km/h의 고정된 속도로 이동하고 Lane 2에서 모든 차량은 120km/h의 고정된 속도로 이동하며 차량은 차선 변경을 하지 않는다고 가정하였다. 인프라는 도로의 중앙인 3000 m 지점에서 도로에 수직으로 500m 떨어진 지점에 위치한다.

Lane 1과 Lane 2의 traffic 양은 각 차선의 차량 arrival rate λ [vehicles/s] 에 따라서 결정되며 차량은 λ 에 따라서 차선의 끝에서 임의로 생성된다. 제안 방안에서는 모든 차선에서 같은 arrival rate λ 로 차량을 생성하는 symmetric traffic을 고려하였으며 각 차선의 arrival rate λ 를 0.05에서 0.5 vehicle/s로 조절해가면서 시뮬레이션을 수행하였다.

Simulation time	5000s
Frame length	1s
Number of time slots in a frame	1
Number of beams (B)	4
Max. tx power of infra and vehicle	43 dBm & 23 dBm
Beam width of antenna	30°
Path-loss model for direct V2I (NLOS)	$15.17 + 38\log_{10}(d)$
Path-loss model for multi-hop 1st phase (LOS)	$38.18 + 25\log_{10}(d)$
Path-loss model for multi-hop 2nd phase (LOS)	$59.7 + 18.5\log_{10}(d)$

그림 3-143. 시뮬레이션 파라미터

자세한 시뮬레이션 파라미터는 [그림 3-143]과 같이 정리하였다. 시뮬레이션에서 차량 arrival rate λ 값을 변화시켜가며 제안 방안의 성능을 확인하는데 각각의 λ 에 대해서 총 5000초 동안 시뮬레이션이 수행하였다. 인프라는 최대 4개의 beam 을 형성할 수 있고 (B = 4) 차량은 하나의 beam 만 형성하도록 설정하였다. 인프라와 차량에서 사용할 수 있는 최대 파워는 각각 43 dBm 과 23 dBm 이고 인프라에서 형성하는 beam의 개수에 따라 최대

파워를 각 beam에 동일하게 나눠서 사용한다. 인프라에서는 30 도의 beam width를 가지는 beam을 형성하며 30 도의 beam을 사용할 때의 BDMA 전송 기술의 antenna model $A(\theta) = -\min[12(\frac{\theta}{\theta_{3dB}})^2, A_m] + G_A$ 에서 신호가 3dB 감쇠되는 beam-width $\theta_{3dB} = 17.5^\circ$, 최대 감쇠값 $A_m = 26dB$, 그리고 안테나 이득 $G_A = 20dBi$ 로 설정된다. Direct V2I 통신의 path-loss 모델은 NLOS 모델인 $PL(d) = 15.17 + 38\log_{10}(d)$ [7]을 사용하였다. Multi-hop 으로 전송하는 경우 multi-hop 1st phase 통신의 path-loss 모델은 LOS 모델인 $PL(d) = 38.18 + 25\log_{10}(d)$ 을 사용하고 multi-hop 2nd phase 통신의 path-loss 모델은 $PL(d) = 59.7 + 18.5\log_{10}(d)$ 을 사용하였다.

제안 방안에서 인프라가 destination 차량에게 데이터를 전송할 때 relay 차량을 사용할지의 여부는 destination 차량과 인프라와의 거리에 의존한다. 즉, 인프라와의 거리가 상대적으로 가까운 destination 차량의 경우에는 LOS 환경일 확률이 높기 때문에 인프라와의 채널 상황이 좋고 propagation loss 가 적다. 따라서 relay를 사용하는 이득이 없는 경우가 많다. 반면에 인프라와의 거리가 비교적 먼 destination 차량의 경우에는 NLOS 환경일 확률이 높기 때문에 인프라의 음영 지역에 위치하여 인프라와의 채널 상황이 좋지 않고 propagation loss가 크다. 따라서 relay를 사용함으로써 성능 이득을 기대할 수 있다. 이와 같은 특성을 반영하여 제안 방안의 성능을 확인하기 위해서 [그림 3-144]과 같이 도로를 일정한 간격의 구간으로 나누고 인프라가 특정 구간에서 destination을 선택하면서 시뮬레이션을 수행하고자 한다.

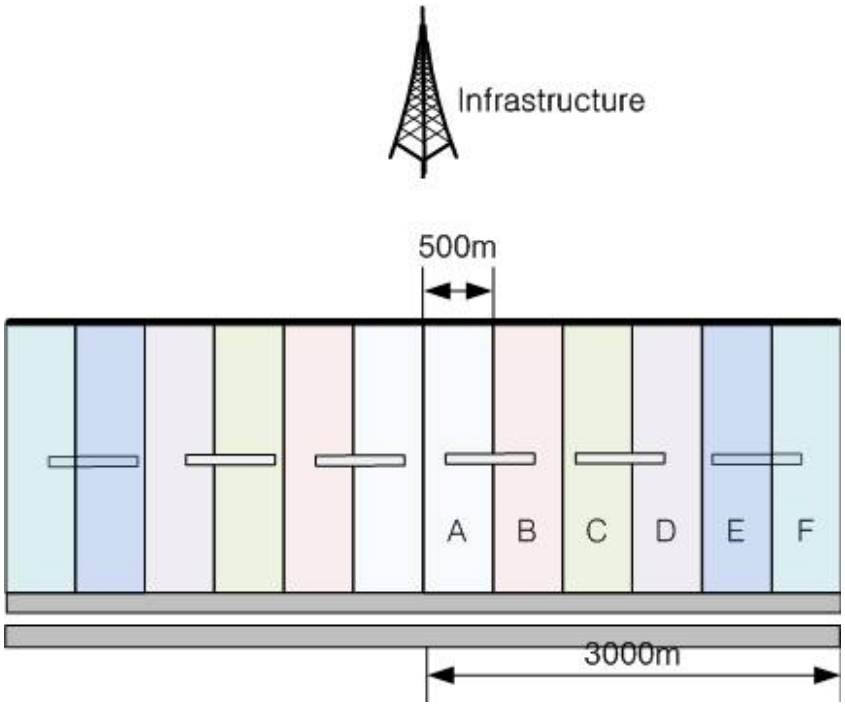


그림 3-144. Destination 차량 선택 구간

[그림 3-144]과 같이 3000m 길이의 도로 절반을 500m의 동일한 길이를 갖는 A, B, C, D, E, F 의 6개의 구간으로 나눈다. 구간 A는 도로 중앙에서 500m 떨어진 지점까지 해당되고 구간 B는 도로 중앙에서부터 500~1000m 지점, 구간 C는 1000~1500m, 구간 D는 1500~2000m, 구간 E는 2000~2500m 지점, 구간 F는 2500~3000m 떨어진 지점에 해당한다. 같은 방식으로 나머지 도로 절반에 대해서도 A, B, C, D, E, F 의 순서로 동일하게 구간을 나누면 [그림 3-144] 과 같이 전체 도로를 A, B, C, D, E, F 의 여섯 구간으로 나눌 수 있다. 즉, 위의 [그림 3-144] 에서 동일한 색상을 갖는 부분은 같은 구간임을 나타낸다.

이와 같이 구간을 나눈 후에 인프라는 연속된 두 구간 내에서 destination 차량을 임의로 선택하는데 두 구간 중에서 도로 중앙에 더 가까운 구간에서 먼저 선택한 후, 더 이상 선택할 차량이 없을 때 다음 구간에서 선택한다. 만약 연속된 두 구간 A-B 에서 destination 차량을 선택한다고 하면 도로의 중앙에 더 가까운 구간 A에서 destination 차량을 먼저 선택한 후에 더 이상 선택할 수 있는 차량이 존재하지 않으면 구간 B에서 선택한다. 이처럼 구간을 나누어 destination 차량을 선택하는 것은 인프라와의 거리가 다른 각 구간에서 destination 차량을 선택하였을 때의 제안 방안의 성능을 분석하기

위해서 이다. 또한, 한 구간이 아닌 연속된 두 구간에서 destination 차량을 선택하는 이유는 한 구간 내에서 모든 destination 차량을 선택할 경우 destination 차량 사이에 발생하는 간섭이 커서 전체 시스템의 성능을 저하시키기 때문이다.

제안 방안은 BDMA 기반의 V2V2I 통신 시스템에서 인프라와의 채널 상태가 좋지 않은 destination 차량에 대해서 인프라가 최적의 relay 차량을 선택해 multi-hop으로 전송함으로써 destination 차량의 채널 용량을 증가시키는 것을 목적으로 한다. 본 연구에서는 제안 방안의 성능을 비교하기 위해서 기존 방안으로 인프라가 모든 destination 차량에게 direct로 전송하였을 때를 고려하였다.

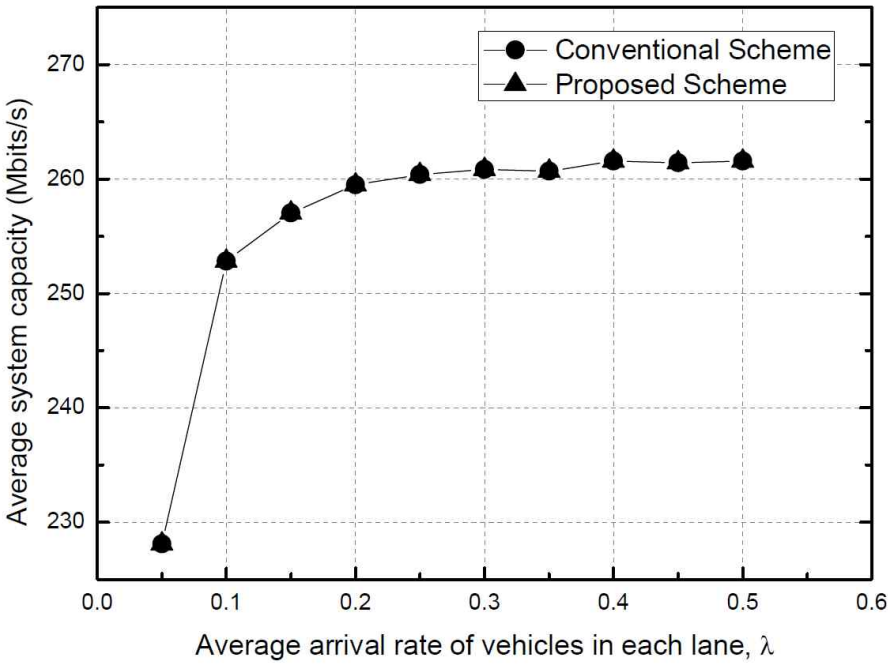


그림 3-145. 구간 A-B 에서 average arrival rate에 따른 평균 시스템 용량

[그림 3-145]은 구간 A-B 에서 destination 차량을 선택했을 때에 traffic load에 따른 기존 방안과 제안방안의 평균 시스템 용량을 보여준다. 기본적으로 도로의 traffic load이 증가함에 따라서 채널 상황이 좋은 차량이 많이 존재한다. 인프라와 가까운 구간 A-B 에서 destination 차량을 선택하였기

때문에 기존 방안과 제안 방안 모두 traffic load이 증가함에 따라서 시스템 용량도 증가하고 traffic 양이 일정 수준에 도달하면 시스템 용량은 일정 값에 수렴한다. [그림 3-146]에서 보는바와 같이 구간 A-B 에서 destination 차량을 선택하는 경우에는 destination 차량과 인프라의 거리가 가까워서 인프라와의 채널 상황이 좋기 때문에 relay를 사용하였을 때의 이득이 없다. 따라서 구간 A-B 에서 destination 차량을 선택하는 경우에 제안 방안은 기존 방안과 성능이 동일하다.

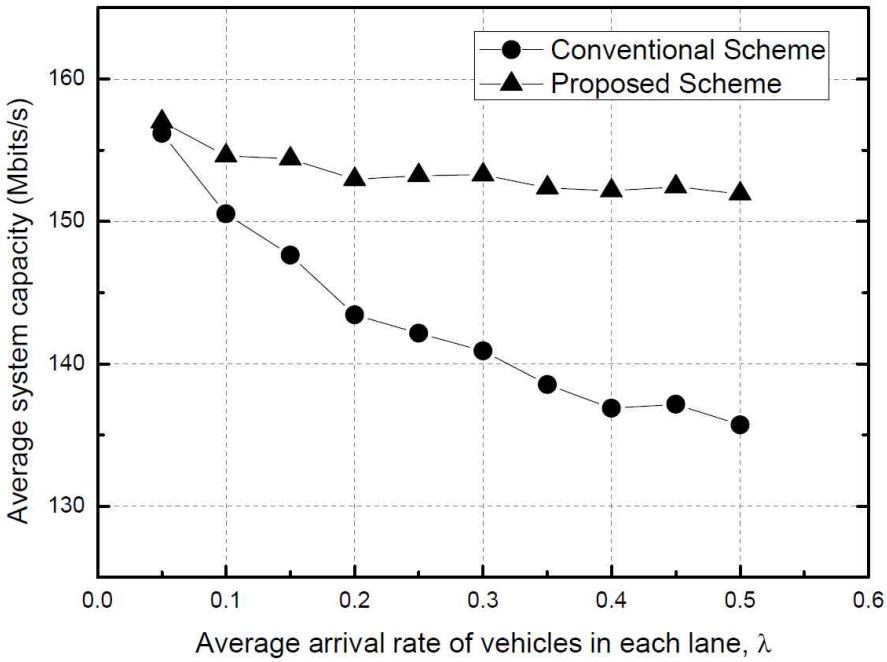


그림 3-146. 구간 B-C 에서 average arrival rate에 따른 평균 시스템 용량

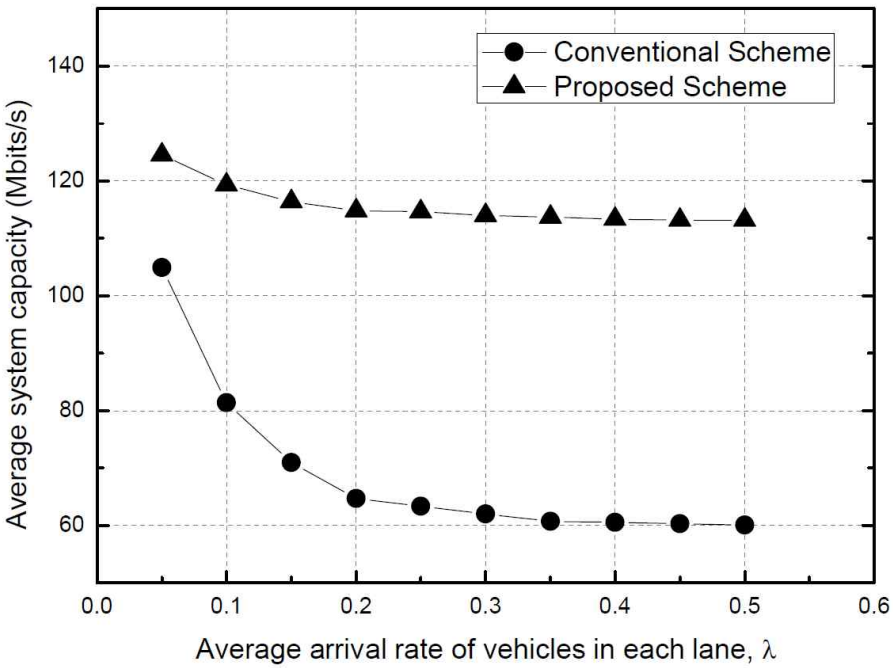


그림 3-147. 구간 C-D 에서 average arrival rate에 따른 평균 시스템 용량

[그림 3-146] 와 [그림 3-147] 은 각각 구간 B-C 와 구간 C-D 에서 destination 차량을 선택했을 때에 traffic load에 따른 기존 방안과 제안방안의 평균 시스템 용량을 보여준다. 그림에서 보는바와 같이 destination 차량과 인프라의 거리가 멀어지면서 인프라와의 채널 상황이 좋지 않은 차량이 존재하기 때문에 그러한 차량에 대해서 relay를 사용하였을 경우 이득이 존재한다. 또한 traffic load가 증가함에 따라서 더 좋은 환경의 relay 차량이 존재하기 때문에 제안 방안의 이득이 증가하는 것을 확인할 수 있다. 기존 방안 대비 제안 방안의 이득은 구간 B-C 의 경우에는 최대 약 12%의 이득이 존재하며 구간 C-D 의 경우에는 최대 약 88%의 이득이 존재한다. 구간 B-C 와 구간 C-D 에서 destination 차량을 선택했을 때의 시스템 용량은 채널 상황이 더 좋은 A-B 구간의 시스템 용량보다 더 작은 것을 알 수 있다.

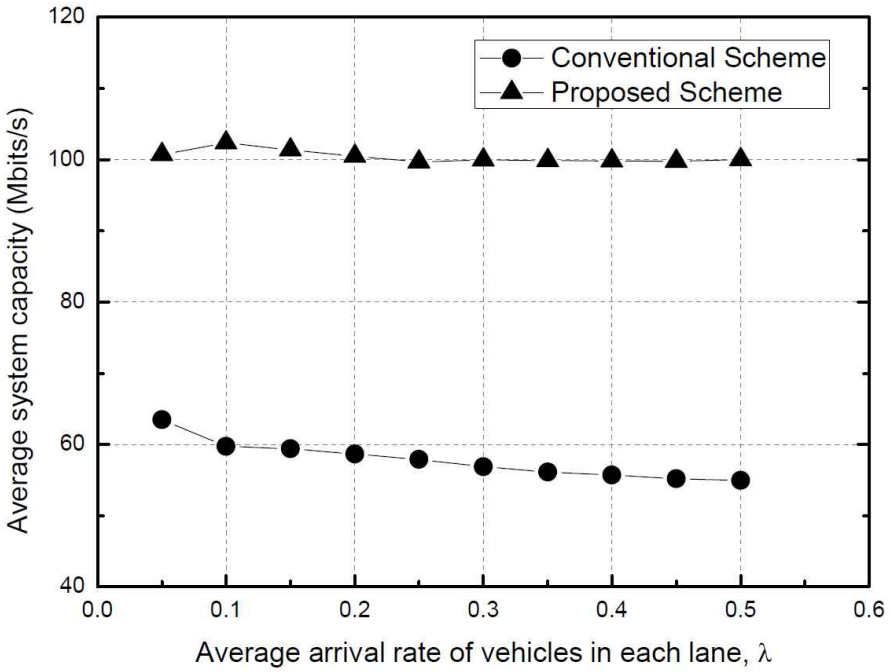


그림 3-148. 구간 D-E 에서 average arrival rate에 따른 평균 시스템 용량

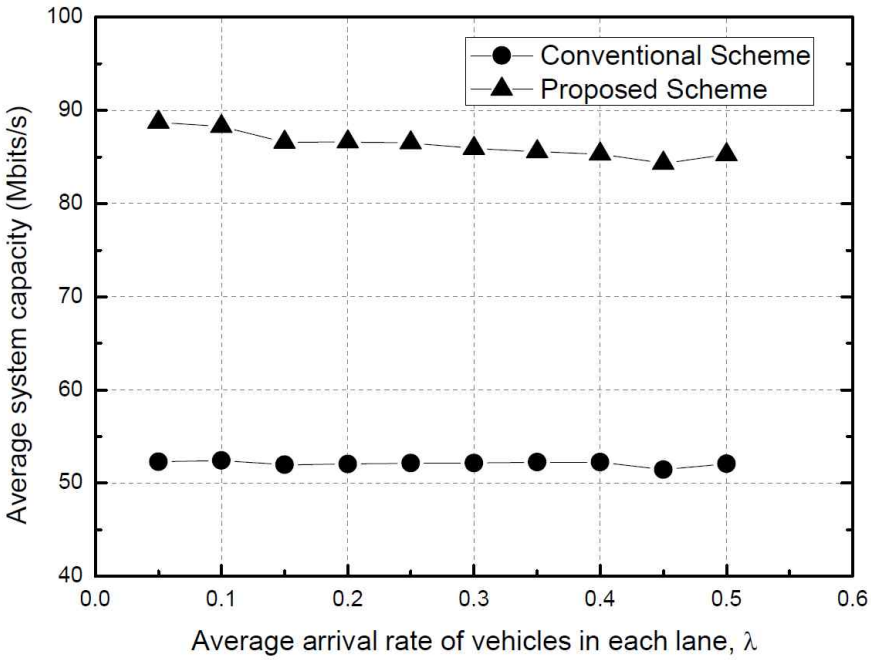


그림 3-149. 구간 E-F 에서 average arrival rate에 따른 평균 시스템 용량

[그림 3-148] 과 [그림 3-149] 는 구간 D-E 와 구간 E-F 에서 destination 차량을 선택했을 때에 traffic load에 따른 기존 방안과 제안방안의 평균 시스템 용량을 보여준다. 구간 D-E 와 구간 E-F 에서 destination 차량을 선택하면 구간 B-C 와 구간 C-D 에서 destination 차량을 선택했을 때보다 destination 차량과 인프라의 거리가 멀어지기 때문에 인프라와의 채널 상황이 좋지 않은 차량이 더 많이 존재한다. 따라서 기존 방안 대비 제안 방안의 이득이 두드러지게 나타나고 구간 D-E 의 경우에는 평균 80%의 이득이 존재하며 구간 E-F 의 경우에는 평균 약 70%의 이득이 존재한다. 또한 traffic load가 증가함에 따라서 좋은 환경의 relay 차량의 수가 빠르게 포화상태에 도달하기 때문에 제안 방안의 이득이 거의 일정하게 유지되는 것을 확인할 수 있다.

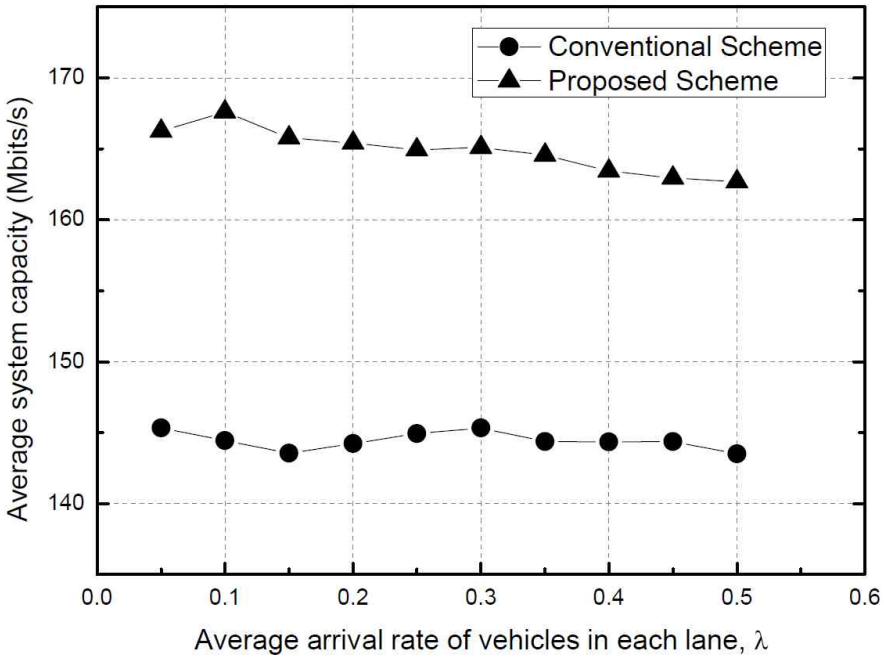


그림 3-150. 전 구간에서 average arrival rate에 따른 평균 시스템 용량

[그림 3-150] 은 전 구간에서 균일하게 destination 차량을 선택했을 때에 traffic load에 따른 기존 방안과 제안방안의 평균 시스템 용량을 보여준다. 도로를 여섯 개가 아닌 네 개의 구간으로 나누고 각 구간에서 destination 차량을 하나씩 선택하였다. 따라서 도로의 전 구간에서 균일하게 destination 차량을 선택하였기 때문에 [그림 3-150] 은 차량들의 fairness를 보장해 줄 수 있는 일반적인 환경에서 제안 방안의 이득을 나타낸다. 인프라와 가까운 곳에 위치한 차량을 destination 차량으로 선택하는 경우에는 제안 방안의 이득이 존재하지 않고 간섭 문제 때문에 해당 차량의 주변 차량을 relay 차량으로 선택할 수 없기 때문에 제안 방안의 이득은 구간 D-E 와 구간 E-F 의 경우보다는 떨어지지만 평균 15% 정도의 이득이 존재함을 확인할 수 있었다.

라. 인프라와 릴레이의 협력을 통한 재전송 기술

(가) 차량간 통신에서 C-ARQ mechanism

본 연구에서는 source node인 인프라(infrastructure), 다수의 relay node(RN) 그리고 destination node인 차량(vehicle)으로 이루어진 two-way relaying 시스템을 고려하고 있다. 본 연구에서 고려하는 차량 통신에서 cooperative ARQ 메커니즘의 동작은 [그림 3-151]와 같다. 먼저 인프라가 ARQ packet을 차량으로 전송한다. 이때, 인프라는 항상 전송할 ARQ packet을 가지고 있고, 빔은 항상 인프라로부터의 ARQ packet을 엿들을 수 있다고 가정하였다. 또한, 인프라와 RN 사이의 channel이 LOS이고 안정적이라고 가정하였다. ARQ packet을 수신한 차량은 해당 ARQ packet에 대한 ACK/NACK 정보를 포함한 feedback을 인프라로 전송한다. 인프라는 전송한 ARQ packet에 대한 feedback을 round trip delay (T1) 후에 차량으로부터 수신할 수 있다. 만약 인프라가 NACK를 수신하거나 재전송 timer (T2)가 종료되기 전에 feedback을 수신하지 못하면 해당 ARQ packet은 재전송된다. 이때, 인프라는 현재 busy하지 않은 RN와 함께 ARQ packet을 재전송한다. RN는 인프라에서 사용했던 AMC mode를 사용해서 ARQ packet을 전송한다. 이러한 재전송 과정은 인프라가 ACK를 수신할 때까지 반복 된다.

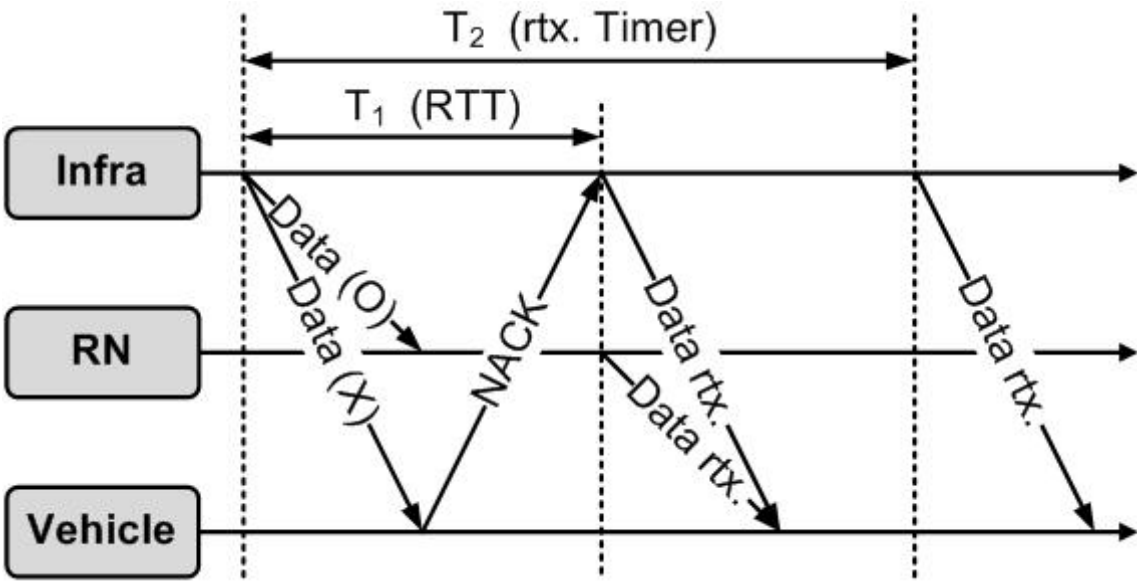


그림 3-151. Cooperative ARQ 메커니즘의 동작

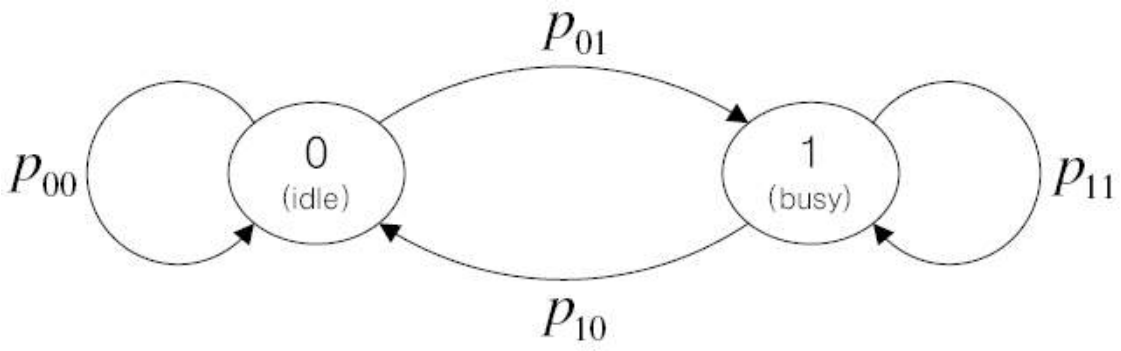


그림 3-152. RN의 state transition 다이어그램

다음으로 본 연구에서 고려하고 있는 cooperation model에 대해서 설명하겠다. 각각의 RN은 [그림 3-152]와 같이 two state의 Markov chain으로 모델링 되었다. 여기서, state 0과 1은 각각 RN의 idle state와 busy state를 나타내고, Ps1s2는 RN의 state transition이 s1에서 s2로 발생할 확률을 나타낸다. 따라서 각 RN의 state transition matrix Q는 수식 1과 같이 표현될 수 있다.

$$\mathbf{Q} = \begin{bmatrix} p_{00} & p_{01} \\ p_{10} & p_{11} \end{bmatrix} \qquad \text{[수식1]}$$

Q를 기본으로 하여 steady state의 probability vector를 유도할 수 있고, 그에 따라 RN이 idle일 확률인 ε을 수식 2와 같이 계산할 수 있다.

$$\varepsilon = \pi^{(0)} = \frac{p_{10}}{p_{10} + p_{01}} \qquad \text{[수식2]}$$

여기서 π(0)는 RN이 idle state일 steady state probability이다. 각각 RN의 state는 서로 독립이며 재전송에 협력하는 RN의 숫자는 N라고 가정하면, frame t에서 idle 상태인 RN의 수를 나타내는 Rt를 정의할 수 있다. Rt는 0부터 N까지의 정수를 가질 수 있다. Rt가 r1에서 r2로 변할 transition probability는 수식 3과 같이 계산되어 진다.

$$\Pr(R_{t+1}=r_2|R_t=r_1)=\sum_{i=\max(0,r_1-r_2)}^{\min(r_1,N-r_2)}\begin{pmatrix} r_1 \\ r_1-i \end{pmatrix} p_{11}^{r_1-i}p_{10}^{i} \times \begin{pmatrix} N-r_1 \\ r_2-r_1+i \end{pmatrix} p_{00}^{N-r_2-i}p_{01}^{r_2-r_1+i} \quad \text{[수식3]}$$

이를 이용해서 state transition의 probability matrix인 PR을 형성 할 수 있다.

본 연구에서 천천히 변하는 channel model에 잘 맞는 finite state Markov chain (FSMC)으로 AMC를 모델링 하였다. FSMC 모델에 따라 signal-to-noise ratio (SNR)의 범위는 K개의 겹쳐지지 않는 연속적인 interval로 나눠진다. 각각의 interval은 특정한 AMC mode로 mapping이 된다. 이때, 각각의 AMC mode의 target packet error rate(pt)를 보장할 수 있도록 interval을 나누는 경계점을 결정하여야 한다. 분석을 간단하게 하기 위해서 다음과 같이 각 AMC mode의 PER(packet error rate)은 curve fitting을 통해 정확한 PER로 근사하였다.

$$PER_n(\gamma)\approx\begin{cases}1, & if \quad 0<\gamma<\gamma_{pn} \\ a_n\exp(-g_n\gamma), & if \quad \gamma\geq\gamma_{pn}\end{cases} \quad \text{[수식4]}$$

따라서 주어진 fading condition에서 SNR이 결정되면 AMC mode간 state transition의 probability matrix (PAMC)와 각 AMC 모드의 average PER(p)을 찾을 수 있다. PERsd,k와 PERrd,k는 각각 k 번째 AMC mode를 사용했을 때 SN-DN(인프라-차량) 링크와 RN-DN(RN-차량) 링크의 average PER으로 정의되며 SNR이 결정되었을 때 계산할 수 있다.

PR과 PAMC의 Kronecker product 연산을 수행하여 새로운 state transition의 probability matrix P를 생성 할 수 있다. P의 Markov state 정의는 (Rt, Γt),로 Γt는 frame t에 사용된 AMC mode를 의미한다. 따라서 Π · P = Π과 Π · 1 = 1 의 관계를 이용하여 steady state의 probability vector인 Π를 구할 수 있다. 여기서 1은 원소가 1인 one-column matrix 이다.

Error mode은 데이터와 feedback의 전송 error probability를 계산하기 위한 것으로 forward channel과 backward channel이 서로 mutually independent하며 symmetric하다고 가정하였다. Downlink 데이터 전송의 경우에는 각각의 Markov state를 위해서 다음과 같은 세 개의 평균 PER을 정

의 하였다: p(f)tx(r,k), p(f)rtx(r,k), p(b)(r,k). p(f)tx(r,k)는 새로운 ARQ packet이 Markov state (r, k)에서 forward channel을 통해서 인프라에서 차량으로 전송되었을 때의 평균 PER로 PERsd,k의 값을 가진다. p(f)rtx(r,k)는 재전송되는 ARQ packet이 Markov state (r, k)에서 forward channel을 통해 인프라와 RN에서 차량으로 전송되었을 때의 평균 PER로 PERsd,k × PERrsd,k의 값을 가진다. 여기서 PERsd,k을 r번 곱한 이유는 r개의 RN이 재전송을 도와주기 때문이다. p(b)(r,k)는 feedback이 Markov state (r, k)에서 backward channel을 통하여 차량에서 인프라로 전송되었을 때의 평균 PER로 PERsd,k의 값을 가진다. Uplink 데이터 전송의 경우에도 같은 방법으로 평균 PER을 구할 수 있다. 따라서 상기 계산된 PER들을 가지고 다음과 같은 probability matrix들을 정의할 수 있다.

$$\begin{aligned} \mathbf{M}_F^{(f)} &= diag\{\mathbf{p_{tx}^{(f)}}\} \\ \mathbf{M}_T^{(f)} &= diag\{1-\mathbf{p_{tx}^{(f)}}\} \end{aligned} \quad \text{[수식5]}$$

M(f)F M(f)T는 새로운 ARQ packet이 forward channel을 통해 전송되었을 때에 error가 발생할 확률과 error가 나지 않을 확률의 probability matrix이다. 또한, p(f)tx는 다음과 같이 정의되었다.

$$\mathbf{p_{tx}^{(f)}} = [\overline{p}_{tx}^{(f)}(0,1), \overline{p}_{tx}^{(f)}(0,2), ..., \overline{p}_{tx}^{(f)}(N,K-1), \overline{p}_{tx}^{(f)}(N,K)] \quad \text{[수식6]}$$

같은 방법으로 재전송된 packet이 forward channel을 통해 전송되었을 때에 error날 확률과 error가 나지 않을 확률의 probability matrix인 Q(f)F와 Q(f)T를 구할 수 있고, feedback이 backward channel을 통해 전송 되었을 때에 error날 확률과 error가 나지 않을 확률의 probability matrix인 M(b)F와 M(b)T를 구할 수 있다. 상기 유도된 probability matrix들을 기반으로 하여 forward와 backward channel을 다음과 같이 동시에 고려할 수 있다.

$$\begin{aligned} \mathbf{P}_{xy} &= \mathbf{P} \cdot \mathbf{M}_x^{(f)} \cdot \mathbf{M}_y^{(b)} \\ \mathbf{R}_{xy} &= \mathbf{P} \cdot \mathbf{Q}_x^{(f)} \cdot \mathbf{M}_y^{(b)} \end{aligned} \quad \text{[수식7]}$$

Pxy는 새로운 ARQ packet이 전송되었을 때의 state transition probability이고, Rxy는 ARQ packet이 재전송 되었을 때의 state transition probability를 나타낸다. 또한, subscript인 x 와 y는 각각 forward와 backward의 전송에서 에러가 났을 때와 나지 않았을 때를 의미하고 F, T 그리고 D로 표시된다. 이때, F는 에러가 났을 때 T는 에러가 나지 않았을 때의 전송을 나타내며 D는 'don't care'를 나타낸다. C-ARQ 메커니즘의 성능

을 분석하기 위해서 본 연구에서는 matrix signal flow graph (MSFG) 기술을 이용하였다. C-ARQ 메커니즘은 [그림 3-153]와 같은 MSFG로 모델링될 수 있다.

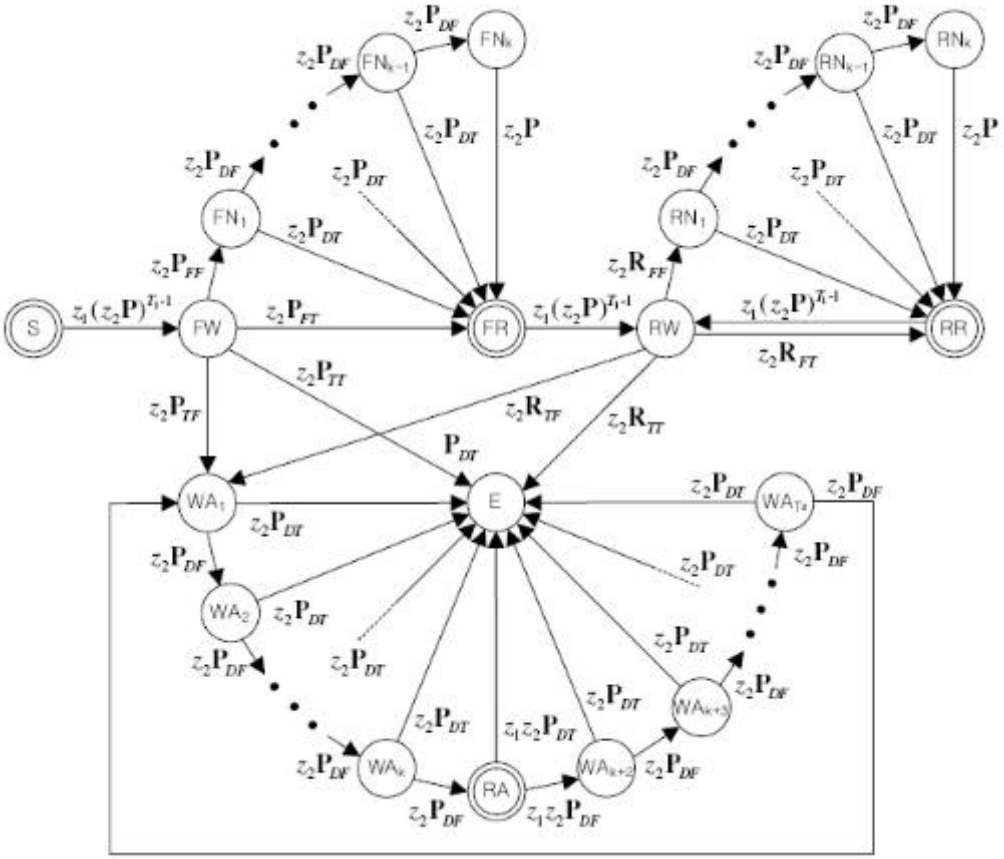


그림 3-153. C-ARQ mechanism의 MSFG

[그림 3-153]에서 S 와 E 는 각각 packet전송의 시작과 끝을 나타내는 state다. FW 는 인프라가 첫 번째 packet을 전송한 후에 feedback을 기다리는 state다. FN 과 FR 는 인프라가 전송한 첫 번째 packet이 성공적으로 전송되지 않았을 때, 인프라가 각각 NACK와 재전송(NACK를 수신 혹은 재전송 timer가 종료)을 기다리는 state이다. FNk에서 재전송 timer는 종료되어가고 k = T2 − T1 이다. RW는 인프라가 재전송 이후의 feedback을 기다리는 state이다. RN 과 RR 는 RW 에서 인프라가 전송한 packet이 성공적으로 전송되지 않았을 때, 인프라가 각각 NACK와 재전송(NACK를 수신 혹은 재전

송 timer가 종료)을 기다리는 state이다. RNk에서 재전송 timer는 종료되어 가고 k = T2 − T1 이다. 인프라가 전송한 packet이 성공적으로 전송되었을 때, WA는 인프라가 ACK를 기다리는 state이고 RA 는 재전송 timer가 종료되어 재전송이 발생되는 state이다. WAk에서 재전송 timer는 종료되어가고 k = T2 − T1 이다.

[그림 3-154]에서 전송 효율과 delay를 계산하기 위한 파라미터인 z1과 z2를 찾아 볼 수 있다. z1 은 ARQ packet의 전송을 의미하기 때문에 packet이 전송 혹은 재전송되는 state의 outgoing flow에서 z1이 곱해지며 z1 을 이용하여 packet 당 전송 횟수를 셀 수 있다. Packet은 S, FR, RR, 그리고 RA 의 네 가지 state에서만 전송될 수 있기 때문에 packet이 전송되었다는 의미를 가진 z1 은 S, FR, RR, 그리고 RA 의 outgoing flow에만 곱해지게 된다. z2 는 한 frame의 종료를 의미하기 때문에 state transition이 발생할 때마다 z2 가 해당 state transition의 outgoing flow에 곱해지게 되고 그 의미는 해당 state transition으로 가는데 필요한 frame의 숫자이다.

Flow-reduction 기술을 사용하여, 제안하는 MSFG의 characteristic function인, F(z1, z2)를 다음과 같이 유도할 수 있다.

$$\mathbf{F}(z_1,z_2)=z_1(z_2\mathbf{P})^{T_1-1}[z_2(\mathbf{P}_{TT}+\mathbf{P}_1\mathbf{P}_2\mathrm{R}_{TT})+z_2^2(\mathbf{P}_{TF}+\mathbf{P}_1\mathbf{P}_2\mathbf{R}_{FT}) \times(\mathbf{I}-z_1(z_2\mathbf{P}_{DF})^{T_2})^{-1}(\sum_{i=0}^{k-1}(z_2\mathbf{P}_{DF})^i+\sum_{i=k}^{T_2-1}z_1(z_2\mathbf{P}_{DF})^i)\mathbf{P}_{DT}] \quad \text{[수식8]}$$

여기서 P1 과 P2 는 다음과 같이 정의 되었다.

$$\mathbf{P}_1=z_1(z_2\mathbf{P}_{FT}+z_2^2\mathbf{P}_{FF}(\sum_{i=0}^{k-2}(z_2\mathbf{P}_{DF})^i\mathbf{P}_{DT}+z_2^{k-1}\mathbf{P}_{DF}^{k-1}\mathbf{P}))(z_2\mathbf{P})^{T_1-1}$$

$$\mathbf{P}_2=(\mathbf{I}-z_1(z_2\mathbf{R}_{FT}+z_2^2\mathbf{R}_{FF}(\sum_{i=0}^{k-2}(z_2\mathbf{P}_{DF})^i\mathbf{P}_{DT}+z_2^{k-1}\mathbf{P}_{DF}^{k-1}\mathbf{P}))(z_2\mathbf{P})^{T_1-1})^{-1} \quad \text{[수식9]}$$

F(z1, z2)를 이용해 새 packet의 전송을 위한 probability vector로 normalize된 characteristic function인 G(z1, z2)를 다음과 같이 계산할 수 있다.

$$\mathbf{G}(z_1,z_2)=\frac{\mathbf{\Pi_S}\mathbf{F}(z_1,z_2)\mathbf{1}}{\mathbf{\Pi_S}\mathbf{1}} \quad \text{[수식10]}$$

여기서 ΠX 는 state X의 stationary vector를 의미한다. 따라서 state S 에서 새로운 ARQ packet이 전송되므로 ΠS 는 새로운 packet의 전송을 위한 probability vector를 의미한다. ΠS 를 계산하기 위해서 [그림 3-154]의 MSFG는 z1과 z2에 각각 1을 대입하고 packet이 전송되지 않는 state들을 제거함으로써 그림 과 같이 간단해 질 수 있다.

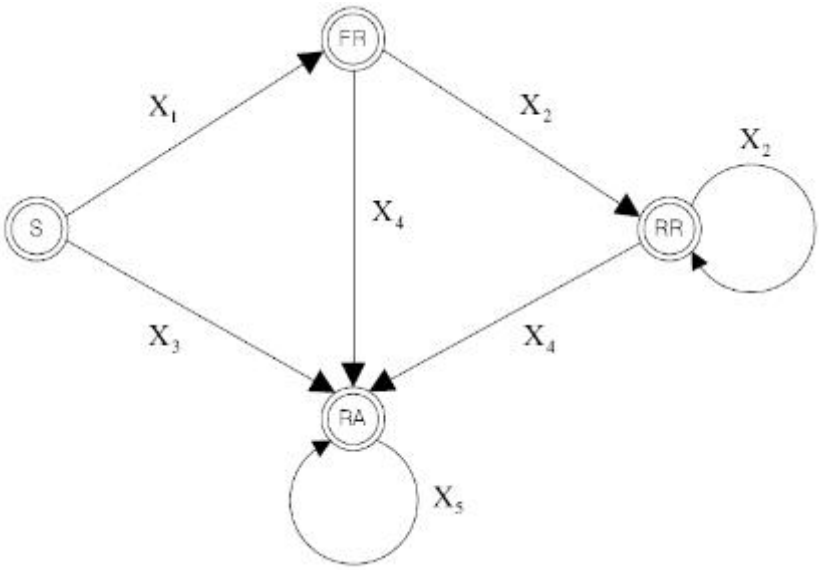


그림 3-154. ΠS를 계산하기 위해 간이화된 MSFG

$$X_1 = P^{T_1-1}(P_{FT} + P_{FF}(\sum_{i=0}^{k-2} P_{DF}^{i} P_{DT} + P_{DF}^{k-1} P))$$

$$X_2 = P^{T_1-1}(R_{FT} + R_{FF}(\sum_{i=0}^{k-2} P_{DF}^{i} P_{DT} + P_{DF}^{k-1} P))$$

$$X_3 = P^{T_1-1} P_{TF} P_{DF}^{k}$$

$$X_4 = P^{T_1-1} R_{TF} P_{DF}^{k}$$

$$X_5 = P_{DF}^{T_2}$$

ΠS 는 다음과 같이 state S, FR, RR 그리고 RA 사이의 관계를 이용해서 구할 수 있다.

$$\begin{aligned} \Pi_{FR} &= \Pi_S X_1 \\ \Pi_{RR} &= (\Pi_{FR} + \Pi_{RR}) X_2 \\ \Pi_{RA} &= \Pi_S X_3 + (\Pi_{FR} + \Pi_{RR}) X_4 \\ \Pi &= \Pi_S + \Pi_{FR} + \Pi_{RR} + \Pi_{RA} \end{aligned} \qquad \text{[수식11]}$$

상기의 공식 중 마지막 공식은 인프라가 항상 packet을 전송한다는 가정에서 유도된 것 이다. 상기 G(z1, z2)를 이용해서 packet당 평균 전송 횟수인 NT를 다음과 같이 구할 수 있다.

$$N_t = \frac{d}{dz_1}\mathbf{G}(z_1,1)\Big|_{z_1=1} \qquad \text{[수식12]}$$

다음으로 총 데이터 전송에서 재전송 전송을 뺀 순수한 데이터 효율인 η 를 다음과 같이 계산할 수 있다.

$$\eta = 1/N_t \qquad \text{[수식13]}$$

전송 delay는 처음 전송된 packet이 성공적으로 전송될 때까지 걸리는 frame의 평균 숫자로 정의된다. 여기서 성공적인 전송은 ACK를 받음으로써 이루어진다. Packet의 round trip delay와 재전송으로 인한 delay가 전송 delay에 포함된다. 전송 delay(Dtx)는 다음과 같이 계산될 수 있다.

$$D_{tx} = \frac{d}{dz_2}\mathbf{G}(1,z_2)\Big|_{z_2=1} \qquad \text{[수식14]}$$

ARQ packet 전송을 위해 사용된 자원(U)은 다음과 같이 유도할 수 있다.

$$U = N_t \sum_{i=0}^{N}\sum_{j=1}^{K} \frac{b_K}{b_i}\cdot \pi^{(i,j)} \qquad \text{[수식15]}$$

여기서 π(i,j) 는 Markov state (i,j)의 stationary probability이고 bj 는 j 번째 AMC mode를 사용하였을 때 bits/symbol 이다. 가장 높은 AMC mode에서 사용한 resource가 1이라고 가정했을 때 각 AMC mode에서 사용된 resource를 bits/symbol 비에 따라 normalize 하였다. 따라서 bK/bj 는 K 번째 AMC mode를 사용했을 경우 하나의 symbol에 전송되는 총 bit 수를 전송하기위해 j 번째 AMC mode 사용했을 때 필요한 symbol의 개수를 나타낸다.

제 4 장 목표 달성도 및 관련분야에의 기여도

제 1 절 연구개발목표의 달성도

1. 목표 대비 실적

2010년 목표치	실적	달성율
● 표준화된 차량용 소프트웨어 플랫폼 구조 연구 o 컴포넌트 실행환경 개발 - AUTOSAR R3.0기반의 RTE 세부 기능 구현 o OS 개발 - AUTOSAR OS 테스트 및 안정화 - OSEK OS 인증 획득 - 플랫폼 확장 및 상용화 o Safety Critical 서비스 개발 - 시스템 서비스 모듈 안정화 - 내결함 통신 서비스 설계 및 구현 - 내결함 통신 서비스 안정화 o Safety Critical 하드웨어 추상화 기법 연구 - 내결함 통신 하드웨어 추상화 계층 안정화 - 내결함 통신 드라이버 안정화 ● 차량용 소프트웨어 플랫폼 설계 도구 개발 o ECU 환경 설정 도구 개발 - 다중플랫폼 지원 o BSW 코드 생성 도구 개발 - BSW 요구사항 반영 코드생성 기능 개선	● 표준화된 차량용 소프트웨어 플랫폼 구조 연구 o 컴포넌트 실행환경 개발 - RTE 세부 기능 구현 o OS 개발 - AUTOSAR OS 동적 커버리지 테스트 - OSEK OS 인증 획득 및 상표사용 협약 - 8/32비트 플랫폼 지원 개발 및 상용화를 위한 라이브러리화 o Safety Critical 서비스 개발 - 시스템 서비스 모듈 안정화 - 내결함 통신 서비스 설계 및 구현 - 내결함 통신 서비스 안정화 o Safety Critical 하드웨어 추상화 기법 연구 - 내결함 통신 하드웨어 추상화 계층 안정화 - 내결함 통신 드라이버 안정화 ● 차량용 소프트웨어 플랫폼 설계 도구 개발 o ECU 환경 설정 도구 개발 - 8/16/32비트 다중플랫폼 지원 설정 기능 개발 o BSW 코드 생성 도구 개발 - 코드생성기능 개선 및 최적화	100% 100%
● ITS를 위한 효율적인 인증 프로토콜	● ITS를 위한 효율적인 인증 프로토콜	100%

2010년 목표치	실적	달성율
개발 o 효율적인 해쉬 체인 운행 알고리즘 - 해쉬체인 운행기법의 기존연구 분석 - 일반화한 해쉬 체인 운행 알고리즘 설계 o 양방 인증을 위한 거리 한정 알고리즘 - 거리한정 알고리즘의 기존연구분석 - False accept rate를 최소화한 거리 한정 알고리즘 설계 - Mutual challenge를 이용한 거리 한정 알고리즘 설계 ● ITS를 위한 효율적인 통신 방안 o BDMA 기반의 V2I 통신 시스템 연구 - 적용적 안테나를 사용하여 간섭 현상을 감소시키는 차량간 Multi-hp 시스템 분석 - 차량 간 통신의 특징을 적용한 빔 분할 다중 접속 방식 제안 o 인프라와 릴레이의 협력을 통한 재전송 기술 연구 - 도로 주변 인프라를 이용한 정보의 효율적 전달을 위한 분석	개발 o 효율적인 해쉬 체인 운행 알고리즘 - 해쉬체인 운행기법의 기존연구 분석 - 일반화한 해쉬 체인 운행 알고리즘 설계 o 양방 인증을 위한 거리 한정 알고리즘 - 거리한정 알고리즘의 기존연구분석 - False accept rate를 최소화한 거리 한정 알고리즘 설계 - Mutual challenge를 이용한 거리 한정 알고리즘 설계 ● ITS를 위한 효율적인 통신 방안 o BDMA 기반의 V2I 통신 시스템 연구 - 효율적인 초기 접속 및 충돌 회피 알고리즘 설계 - 차량의 grouping을 이용한 자원할당 알고리즘 설계 o 인프라와 릴레이의 협력을 통한 재전송 기술 연구 - 차량간 통신에서 C-ARQ mechanism의 성능 분석	100%

2. 논문 및 특허 실적

가. 논문 실적

번호	논문제목	국내/국외	학술지명	학술대회명	발표일자
1	AUTOSAR 소프트웨어 플랫폼 호환성 테스트에 관한 연구	국내		2008대한임베디드공학회 추계학술대회	2008.11.14

2	AUTOSAR 베이직 소프트웨어 모듈의 타이밍 설계	국내		2008대한임베디드공학회 추계학술대회	2008.11.14
3	AUTOSAR I/O 드라이버 클러스터 분석 및 설계	국내		2008년 한국자동차공학회 학술대회	2008.11.20
4	차량 전장시스템의 적용을 위한 AUTOSAR 플랫폼의 단계적 전이방법	국내		2008년 한국자동차공학회 학술대회	2008.11.20
5	AUTOSAR Migration from Existing Automotive Software	국외	-	International Conference on Control, Automation and Systems 2008	2008.10.15
6	AUTOSAR 소프트웨어 아키텍처의 스케줄링 방법	국내	-	2008년 한국자동차공학회 학술대회	2008.11.15
7	자동차 기술과 IT 기술 융합 SW 표준화 동향	국내	한국통신학회지		2008.6
8	Technology trends for automotive embedded system and a case study	국외		The 3rd International Sysposium on Embedded Technology	2008.5.23
9	Implementation of AUTOSAR I/O Driver Modules for a SSPS System	국외	-	International Conference on Control, Automation and Systems 2008	2008.10.15
10	차량 간 통신이 적용된 초기 환경에 적용 가능한 응용	국내		2008대한임베디드공학회	2008.11.14

	서비스			추계학술대회	
11	상태표시 골롬보 압축 알고리즘을 이용한 효율적인 플래쉬 메모리 역리스트 재구성 기법	국내		2008대한임베디드공학회 추계학술대회	2008.11.14
12	The Design of Remote Vehicle Management System based on OMA DM Protocol and AUTOSAR S/W Architecture	국외		ALPIT 2008	2008.7
13	The Remote Vehicle Management System based on standardized Interfaces	국외		MITA2008	2008.7
14	차량용 임베디드 S/W 런타임 업그레이드 시스템 설계	국내		2008년 한국자동차공학회 학술대회	2008.11.20
15	AMI-C 표준 기반의 원격 차량 관리 시스템 설계	국내		정보처리학회	2008,6
16	Priority-Based Scheduling of Dynamic Segment in FlexRay Network	국외		International Conference on Control, Automation and Systems 2008	2008.10.16
17	차량용 임베디드 시스템을 위한 표준 운영체제의 구현	국내		대한 임베디드 공학회 추계학술대회	2009.11.20
18	차량용 임베디드 소프트웨어의 신뢰성 확보를 위한 소프트웨어 테스팅 사례 연구	국내		대한 임베디드 공학회 추계학술대회	2009.11.20
19	차량용 임베디드 미들웨어를 위한 코드생성 도구의 설계	국내		대한 임베디드 공학회	2009.11.20

				추계학술대회	
20	OSEK/VDX 기반 차량용 운영체제의 구현 및 신뢰성 확보 방안	국내		한국자동차공학회	2009.11.24
21	OSEK/VDX 기반 차량용 운영체제의 소프트웨어 테스팅을 통한 신뢰성 확보 연구	국내		UCWIT2009	2009,12,11
22	Generalized Hash Chain Traversal with Selective Output	국내		IEICE	제출
23	효율적인 인증을 제공하는 차량간 통신 기법	국내		2009대한임베디드공학회 추계학술대회	2009.11.20
24	차량간 통신에서 효율적인 자원할당 방안	국내		2009대한임베디드공학회 추계학술대회	2009.11.20
25	차량용 임베디드 시스템을 위한 표준 운영체제의 구현	국내		대한 임베디드 공학회 추계학술대회	2009.11.20
26	차량용 임베디드 소프트웨어의 신뢰성 확보를 위한 소프트웨어 테스팅 사례 연구	국내		대한 임베디드 공학회 추계학술대회	2009.11.20
27	차량용 임베디드 미들웨어를 위한 코드생성 도구의 설계	국내		대한 임베디드 공학회 추계학술대회	2009.11.20
28	OSEK/VDX 기반 차량용 운영체제의 구현 및 신뢰성 확보 방안	국내		한국자동차공학회	2009.11.24
29	OSEK/VDX 기반 차량용 운영체제의 소프트웨어 테스팅을 통한 신뢰성 확보	국내		UCWIT2009	2009,12,11

	연구				
30	효율적인 인증을 제공하는 차량간 통신 기법	국내		2009대한임베디드공학회 추계학술대회	2009.11.20
31	차량간 통신에서 효율적인 자원할당 방안	국내		2009대한임베디드공학회 추계학술대회	2009.11.20
32	차량간 통신에서 사고 차량을 위한 효율적인 무선 자원 할당 방안	국내		2010 대한임베디드 공학회추계학술대회	2010.11.19
33	기지국을 이용한 차량간 GPS 정보교환을 위한 효율적인 인증 프로토콜	국내	2010 대한임베디드 공학회논문지		2010.9.30
34	기지국을 이용하여 차량 주변 위성 사진을 안전하게 제공하는 실시간 프로토콜	국내		2010 대한임베디드 공학회추계학술대회	2010.11.19
35	지능형 자동차용 임베디드 플랫폼 소프트웨어 테스팅 연구	국내	대한전자공학회지 제47권 SC편 제6호		2010.11월호
36	Design of a Vehicular Code Generator for Distributed Automotive Systems	국외		2010 ITNG Conference	2010.04.14
37	AUTOSAR 방법론에 따른 코드 생성기 모델 연구	국내		2010 한국통신학회 하계학술대회	2010.06.24
38	중첩 셀 네트워크에서 부하 균등화를 위한 채널 관리 기법	국내		2010 한국통신학회 추계학술대회	2010.11.13
39	Generalized Hash Chain Traversal with Selective Output	국외	IEICE		2010.5.1

나. 특허 실적

번호	제 목	출원/등록
1	네트워크 매니지먼트 메시지 중재 시스템	출원
2	실시간 시스템의 최악응답시간 예측방법	출원
3	윈도우 안전 장치를 구비한 도어	출원
4	윈도우 제어 장치	출원
5	AUTOSAR 기반의 자동차용 사고기록장치의 컴포넌트 기반 소프트웨어 시스템	출원
6	휴대용 슬롯 기반의 차량용 ECU 확장 방법	출원
7	지능형 자동차를 위한 교통 표지판 검출 방법 및 상기 방법을 실행하는 시스템	출원
8	네트워크 데이터 관리 장치 및 관리 방법	출원
9	차량 제어 방법 및 제어 장치	출원
10	차량용 연료센더의 더미스터의 고장 진단	출원
1	근거리 차량간의 대상 차량을 지정하는 정보 전달 방법	출원
2	버스전용차로의 유료 이용 방법 및 시스템	출원
3	전력 거래 서비스 방법 및 시스템	출원
4	차량주변 위성사진 제공 시스템 및 방법	출원
5	기지국을 이용한 주위 차량 위치 인식 시스템 및 기지국과 차량 사이의 인증방법	출원
6	임베디드 멀티미디어 기기에 적합한 MPEG-7메타-데이터 저장 방법	등록
7	질의 처리 방법, 역 리스트 관리 방법, 역 리스트 관리를 위한 압축 방법, 및 구문 역 리스트 관리 방법	등록
8	확장 정렬 칼럼과 확장 투플 식별자 기반의 정렬외부-유니온 방법을 사용하여 엑스쿼리 문을 에스큐엘문으로 변환하는 방법 및 장치	등록
9	차량 주변 영상 생성 장치 및 방법	출원
10	인프라 기반 차량 통신 시스템에서 통신 그룹 형성 및 자원 할당 방안	출원

11	인프라 기반 차량 통신 시스템에서 적응적 빔을 이용한 통신 방안	출원
12	차량용 연료센더의 더미스터의 고장 진단	출원
13	차량진단방법 및 그 시스템과, 정보단말기	출원
14	이종 프로트콜 통신에서의 메시지 전송 지연을 보상하는 방법 및 장치	출원
15	이종 프로토콜 통신에서 프로세스의 우선 순위를 조절하여 지연을 보상하는 방법 및 장치	출원
16	이종 프로토콜 통신에서 병렬메세지를 처리하는 방법 및 장치	출원
17	차량의 주행모드의 결정장치	출원
18	오류 정정을 위한 인코딩 및 디코딩 방법 및 장치	출원
19	암호화된 교통 제어 정보를 송수신하는 방법 및 장치	출원
20	동적셀 선택장치 및 동적설 선택방법	출원
21	메모리 보호를 위한 컨트롤러 장치 및 컨트롤러 장치의 동작 방법	출원
22	운영체제 태스크 프레임워크의 자동 생성 방법 및 시스템	출원
23	적응성 빔 기반 통신 시스템 및 그 멀티 홉 릴레이 전송방법	출원
24	적응성 빔 기반 통신 시스템 및 그 스케줄링 방법	출원
25	버퍼 관리방법, 그 장치, 서버 및 전장 시스템	등록
26	네트워크 데이터 관리 장치 및 관리 방법	등록
27	연료 경고등의 고장 감지 장치 및 방법	등록
28	정적 태스크 정의 기능을 가진 시스템을 위한 우선순위 재정의 및 대기큐 관리방법과 상기 방법을 실행하는 시스템	등록

다. 소프트웨어 등록

번호	제 목	기 관	
1	BSW 코드 생성 도구	컴퓨터 프로그램 보호 위원회	등록
2	ECU SW 플랫폼 설정 도구	컴퓨터 프로그램 보호 위원회	등록
3	AUTOSAR 환경 설정 및 코드 생성 도구	컴퓨터 프로그램 보호 위원회	등록
4	OSCAR-OSEK 핵심 커널 모듈	컴퓨터 프로그램 보호 위원회	등록
5	OSCAR-OSEK 핵심 커널 모듈	컴퓨터 프로그램 보호 위원회	등록
6	AUTOSAR 기반 차량용 응용시스템 개발 도구	컴퓨터 프로그램 보호 위원회	등록

라. 이술 이전

번호	이 전 기 술 명	계약일
1	차량 임베디드시스템용 RTOS	2010.11.09

제 2 절 관련분야의 기여도

1. 기술적 측면
  - 지능형 자동차 개발을 위한 핵심 기반 기술력을 확보함
  - 자동차 산업과 최신 IT 기술의 융합을 가속화 시키며 시너지 효과를 발생시킴
  - 컴포넌트 기반의 소프트웨어 플랫폼을 사용함으로써 소프트웨어의 재사용성 증대, 신뢰성 향상, Time-to-Market에 기여함
  - 표준 소프트웨어 플랫폼 기반의 다양한 응용 서비스의 개발이 예상됨
  - 모델 기반 개발 방법과 도구 지원으로 응용 서비스 개발 모델의 지식화가 가능해짐
  - 지식 모델의 지원으로 소프트웨어의 재사용성이 극대화되므로 신규 응용 서비스의 개발 주기가 단축되는 이점 제공
  - 자동차용 임베디드 소프트웨어 설계 및 보증 기술 확보
  - 세계 표준에 대한 분석 및 연구를 통하여 국내 기업의 기술경쟁력 및 대응능력 제고
  - 차량 임베디드 부품의 신뢰성 향상 기술 확보
  - 차량 임베디드 시스템의 안정적인 설계 프로세스 기술 확보
  - 표준 소프트 웨어 플랫폼 및 네트워크 응용한 임베디드 부품 설계 기술 확보
- 확장형 해쉬 체인 운행 기법은 기존 운행 기법에 비해 실행 시간을 단축할 수 있는 장점 보유
- 거리한정 기법의 개발은 위치 정보에 기반한 기술보다 진보된 교통 서비스 제공에 대한 신뢰도 보장
- 고속으로 주행하는 차량에서의 데이터 무결성 검증 인프라 구축
- 확보된 차량 통신 기술과 ITS 주행 시험장 인프라를 바탕으로 지능형 교통 시스템 실증 사업 추진 및 사업화 모델 발굴
- 주행 차량간 혹은 차량과 기지국 간의 통신망 안전성 확보를 통한 ITS 보안 원천기술의 기술이전 추진

2. 경제・산업적 측면
  - 해외 의존적인 자동차 전장 소프트웨어 플랫폼 및 도구의 국산화로 수입대체에 기여
  - 완성차 업계에서 사용하는 다양한 도구들 중 서비스 요구사항 모델링, 모델

검증, 코드 생성을 위한 도구의 개발로 관련 시장 창출에 기여
- 자동차 부품 산업의 고부가가치화, 국제 경쟁력 강화에 기여
- 표준 소프트웨어 기반 부품 모듈에 시장 적용 확대의 기회를 제공하여 부품 재사용성 증대 및 시장 요구사항 반영 주기의 단축에 기여
- 향후 자동차 부품 시장에서 전개될 글로벌 비딩 체계 하에서 표준 소프트웨어 기반의 부품 모듈에 대한 채택의 확대가 예상되므로 조기 시장 선점에 기여가 가능함
- 자동차 산업의 IT 개발 인력에 대한 인식 제고로 연구 개발 인력의 고용 효과 창출
- 자동차 산업에서 통신망의 안전성을 보장하여 녹색 성장의 중요한 기술들의 효율적인 활용을 가능하게 하고, 관련 기반 기술들의 사회 보급을 촉진
- 차량 통신 기술의 ITS에 효율적인 인증 기법을 적용함으로써, 안전하고 편리한 교통 환경 제공
- 차량 통신 망의 신뢰도 확보를 통하여 ITS의 조기 상용화에 기여
- 악의적인 사용자로부터 교통 인프라를 보호하는 국가 보안 기술력 확보

제 5 장 연구개발결과의 활용계획

본 연구에서는 복잡하고 거대해진 차량용 소프트웨어를 효과적으로 개발하기 위한 표준화된 차량용 임베디드 소프트웨어 플랫폼과 설계 도구 및 응용 시스템을 개발하였다.

최근 전 세계적으로 거의 모든 산업분야에 걸쳐 네트워크 기술과 지능형 부품 기술을 적용하려는 시도가 이루어지고 있는데, 응용 시스템 개발을 통하여 얻은 시스템 설계 기술 및 시스템 구현기술은 이러한 폭넓은 산업 분야의 기초기술로 활용될 수 있다. 대표적인 사례로써 첨단 수술용 로봇 등 네트워크 기반 의료기기에 적용되어 신뢰성을 향상 시키거나, 철도, 선박, 항공・우주, 제철, 화학 플랜트 등 수 많은 산업분야에 적용되어 제품이나 시스템의 신뢰도와 안전성 확보에 기여하여 경쟁력 강화에 이바지할 수 있을 것으로 판단된다.
특히, 국내에서 차량용 임베디드 소프트웨어 플랫폼을 최초로 개발하였다는 것과 국제 표준에 준하는 소프트웨어 플랫폼과 도구의 개발을 통하여 자동차 응용 서비스를 조기에 구현하여 기술적 선도성을 확보하였다는 점에서 향후 관련 기술의 상용화에 유리한 위치를 선점하였다고 판단된다.

본 연구의 연구 결과를 바탕으로 심화 연구를 수행함으로써 연구결과를 구체화하고, 완성차 및 부품 제조 업계와의 공조를 통해 보유 기술을 양산에 적용하기 위한 활동이 요구된다.

또한, 본 연구에서는 ITS를 위한 효율적인 인증 프로토콜 개발을 목표로 효율적인 해쉬 체인 운행 알고리즘 개발과 양방 인증을 위한 거리 한정 알고리즘 개발의 세부 연구를 수행하였다.

개발 중인 확장형 해쉬 체인 운행 기법은 기존 운행 기법에 비해 실행 시간을 단축하고, 거리한정 기법의 개발은 위치 정보에 기반한 기술보다 진보된 교통 서비스 제공에 대한 신뢰도를 보장할 수 있으므로, 고속으로 주행하는 차량에서의 데이터 무결성 검증을 위한 최적의 인프라를 구축할 수가 있다.

이 모듈들을 ITS 차량 통신 기술의 메시지 인증을 위해 적용하면, 효율적이면서도 안전하고 편리한 교통 환경을 제공할 수가 있다. 그리고 주행 차량간 혹은 차량과 기지국간의 통신망 안전성 확보를 가능하게 하므로, ITS 보안 원천 기술

을 필요로하는 자동차 부품 생산 업체로의 기술이전도 추진할 수 있을 것으로 판단된다.

제 6 장 연구개발과정에서 수집한 해외과학기술정보

1. Vector

- 독일의 대표적인 자동차 임베디드 소프트웨어 및 툴 업체로 "DaVinci"라는 독자적인 AUTOSAR 플랫폼 및 개발도구를 개발 완료
- AUTOSAR Release 2.0 규격 호환
- DaVinci tools와 Basic software module configuration tools로 구성

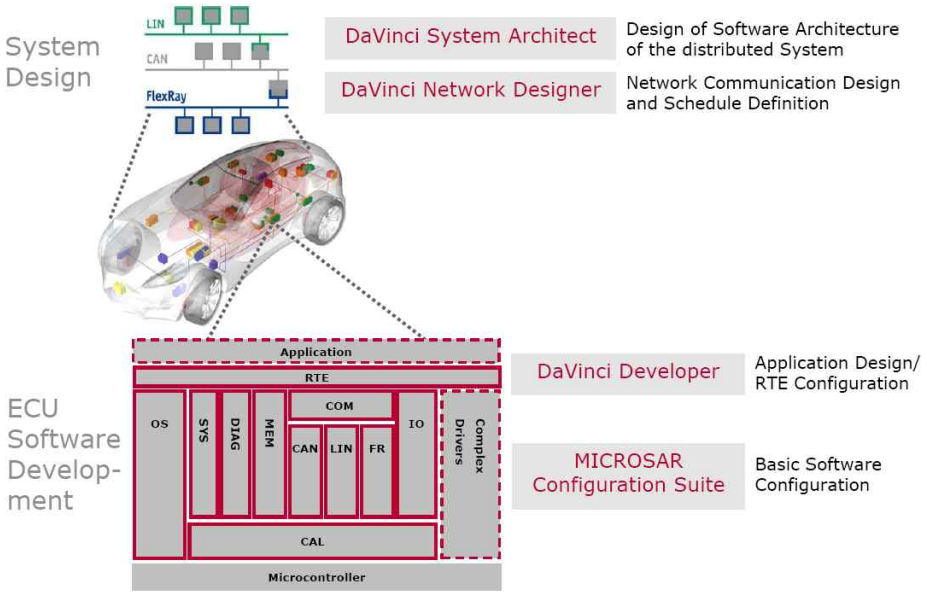


그림 6-1 DaVinci Tools

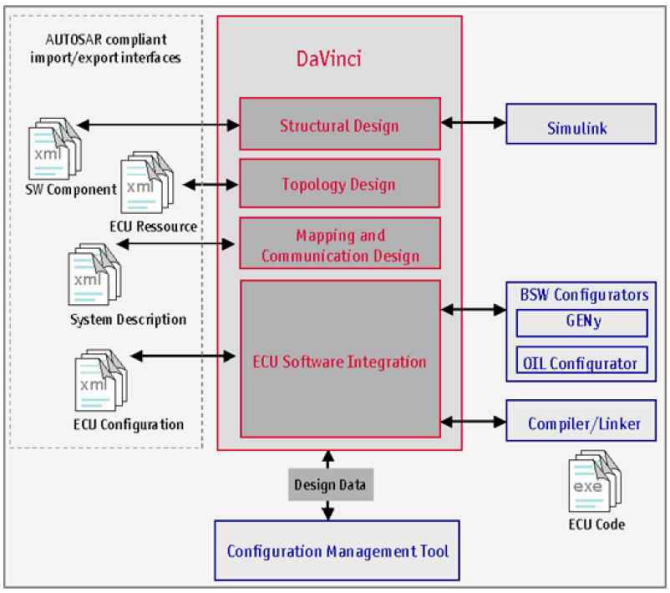


그림 6-2. Vector Davinci Tool Construction

2. TNI-Software

TNI-Software사는 AUTOSAR 프리미엄 멤버로서 자동차 분야 뿐만 아니라 항공, 에너지, 산업자동화 등의 다양한 분야에 대하여 실시간 모델링과 시뮬레이션을 제공한다. 요구사항 기반 엔지니어링, 모델기반 설계 및 AUTOSAR Builder 등의 통합전장 시스템을 지원하는 도구 라인을 갖추고 있다.

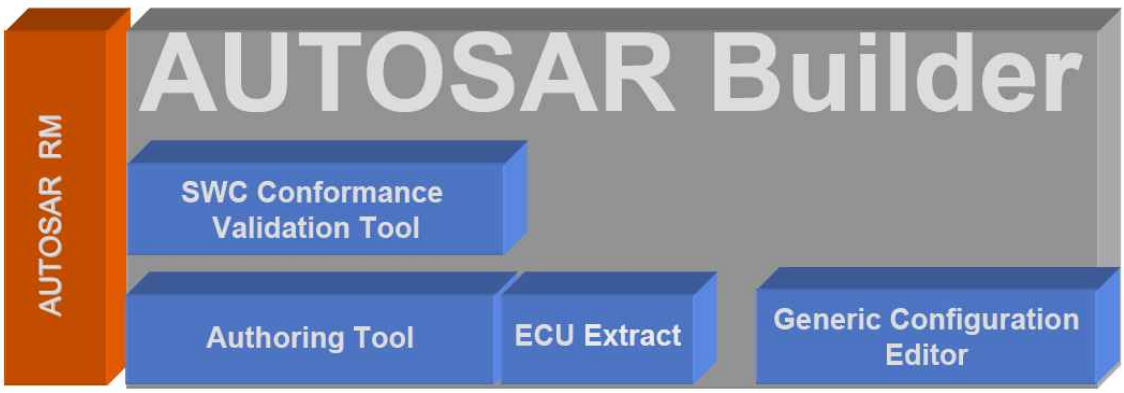


그림 6-3. AUTOSAR Builder Tool Components

AUTOSAR Builder를 이용하여 시스템 구성과 ECU 구성 개발과 검증을 통해

ECU Extract까지 간편하고 자동화된 개발과정을 지원한다. 또한 자바기반 플랫폼을 사용하여 전이성과 확장성 측면에서 우수하며 다양한 비즈니스 모델을 지원한다.

3. dSPACE

dSPACE사는 실부하 타겟 개발 및 구현과 ECU 테스팅 등의 강력한 개발도구와 서비스를 제공하며 전자제어 유닛 및 메카트로닉스 애플리케이션 소프트웨어 개발부문에서 통합솔루션을 제공하는 대표적인 임베디드 SW개발 업체이다.



그림 6-4. V 싸이클을 이용한 자동차 응용 ECU 개발과정

위 그림과 같이 dSPACE사의 도구라인은 V 프로세스에 적합한 모델기반 설계와 오프라인 시뮬레이션을 제공한다. 또한 자동코드 생성, 개발기간 단축 및 직관적인 Calibration을 제공하는 등 개발검증 부문에서 독자적인 기술을 보유하고 있다.

4. Elektrobit

Elektrobit사는 센서와 제어시스템 및 통합기술 그리고 무선 통신 분야에서 신뢰성 있는 기술을 보유하고 있다. 특히 자동차 산업의 AUTOSAR 표준에 맞추어서 다양한 회사의 마이크로컨트롤러에 대하여 MCAL(Microcontroller Abstraction Layer) 등의 베이직 소프트웨어를 제공한다는 점에서 강점을 보인다.

또한 Elektrobit사의 AUTOSAR 소프트웨어 번들은 자동차 바디, 섀시, 파워트레인 전반에 걸친 다양한 애플리케이션을 개발할 수 있기 때문에 확장성이 뛰어나고, ECU 의존적인 부분을 최대한 배제한 소프트웨어 솔루션을 제공한다. 현재 Freescale사 등의 반도체업체들과 연계하여 시장성을 확보하고 있어서 앞으로의 Elektrobit사의 행보가 주목되고 있다.

5. WAVE

WAVE는 Wireless Access Vehicle Environment의 약자로 IEEE를 중심으로 진행되고 있는 공공 안전과 ITS 서비스를 위한 V2V/V2I 네트워크 기술의 기술 표준이다. WAVE 표준의 구성은 PHY계층과 일부 MAC계층에 대하여 IEEE 802.11p를 기반으로 하고 있으면 상위 계층의 통신 표준은 IEEE 1609.x시리즈에서 정의된다.

WAVE의 특징은 다음과 같다. 최대 200km/h의 차량 이동속도를 지원하며 최대 거리 1km에 대하여 1Mbps이상의 서비스를 지원한다. 사용하는 주파수 대역은 5.850GHz에서 5.925GHz까지 75MHz의 영역을 포함한다. 이 영역 안에 각 10MHz에 해당되는 7개의 channel들이 존재한다. Modulation방식은 QPSK OFDM 방식을 기본으로 하여 16QAM과 64QAM도 제공한다. Data Rate는 10MHz channel을 사용할 경우에는 각 6, 9, 12, 18, 24, 27 Mbps이고 20MHz channel을 사용할 경우에는 6, 9, 12, 18, 24, 36, 48, 54Mbps이다.

WAVE에서의 802.11p에서의 핵심 요소 기술들에 대해 살펴보자. 위에서 언급한 것과 같이 미국의 FCC (Federal Communication Commission)는 5.9GHz 대역에서 75MHz의 대역폭을 V2X통신을 위한 전용으로 아래 그림과 같이 7개의 10MHz 채널을 할당하였다.

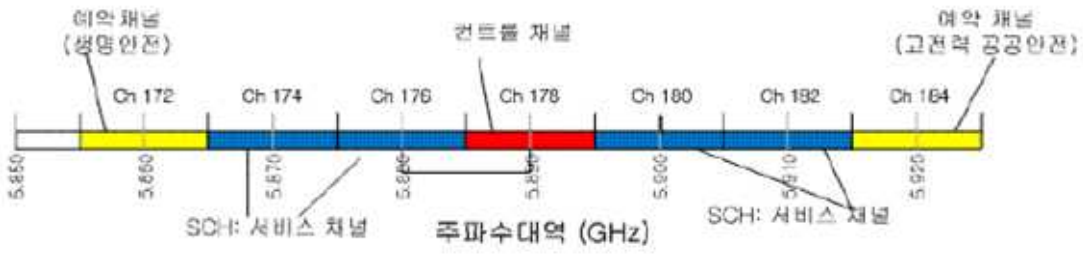


그림 6-5. 주파수 대역과 채널 할당

채널 178번은 교통안전 전용의 컨트롤 채널(CCH: Control Channel)이며, 주파수 양끝 단의 두 채널을 특수한 목적을 위해 예약되었다. 그 외 4개의 채널을 서비스 채널(SCH: Service channel)로 할당되어, 교통안전 또는 일반 상업적 서비스를 위해 사용된다.

기존의 IEEE802.11 무선랜 표준은 단말과 AP가 통신하기 위하여 AP가 보내는 Beacon 메시지를 듣고 BSS에 가입하게 된다. BSS에 가입하는 과정은 Association과 Authentication등 몇 단계를 거치게 되므로 교통안전 통신을 위에서는 위와 같은 가입 과정은 시간이 너무 오래 걸린다는 단점이 존재하게 된다. 따라서 WAVE에서의 802.11p에서는 WAVE 모드가 존재한다. WAVE 모드에서는 BSS의 설정 없이 station간 통신을 지원하거나 WBSS가 있는 상태에서 station간 통신을 지원한다. WBSS(WAVE BSS)는 WAVE mode로 동작하는 하나의 WAVE service provider와 복수의 WAVE service user로 구성되는 station의 집합이다. 기존의 Infrastructure mode와 ad hoc mode와는 달리 authentication과 association과정이 없으며 joining과정만 존재한다. 이 과정에서 management (beacon)과 control(RTS/CTS, ACK), data frame이 사용된다. WAVE동작을 위하여 WAVE provider는 비주기적으로 필요 시에만 WAVE advertisement frame을 생성하여 사용한다. 이 때 station간의 동기는 WAVE announcement프레임을 수신함으로 이루어진다. WAVE announcement 프레임을 GPS시간을 기준으로 하는 Time stamp를 이용하여 64bit로 구성되어 있다.

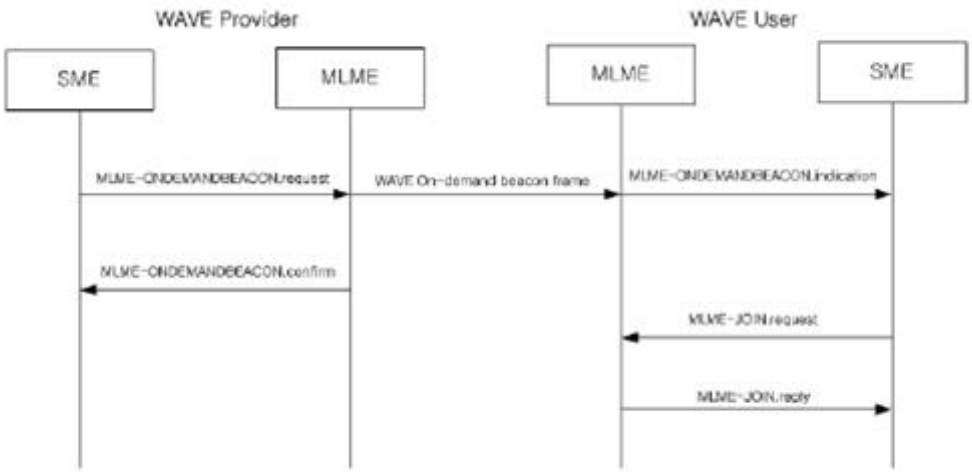


그림 6-6. IEEE 802.11p 에서 제공하는 WAVE Advertisement 과정

IEEE 1609.x시리즈는 MAC계층과 네트워크 상위 계층의 다양한 영역을 포함하는데 각각의 기능은 다음과 같다. IEEE 1609.1에서는 응용 서비스의 RM(Resource Manager)에 대해 정의하고 있다. WAVE를 사용하는 장치는 크게 도로변에 설치되어 정적인 특성을 가지는 RSU (Roadside Unit)와 움직이는 각 차량에 설치되는 OBU (Onboard Unit)로 구분된다. 일반적으로RSU의 통신 범위 내에 들어온 OBU는 서비스를 요청하고, RSU는 이 요청에 해당하는 서비스를 제공하게 된다. RM은 RSU에 존재하며 RM과 통신하는 RCP (Resource Command Processor)는 OBU에 위치한다. RSU와 별도로 원거리에 위치한 응용 서비스인 RMA (Resource Manager Application)은 RM을 통하여 RCP와 통신하게 된다. 즉, RM은 마치 RMA의 응용 계층처럼 동작하게 된다.

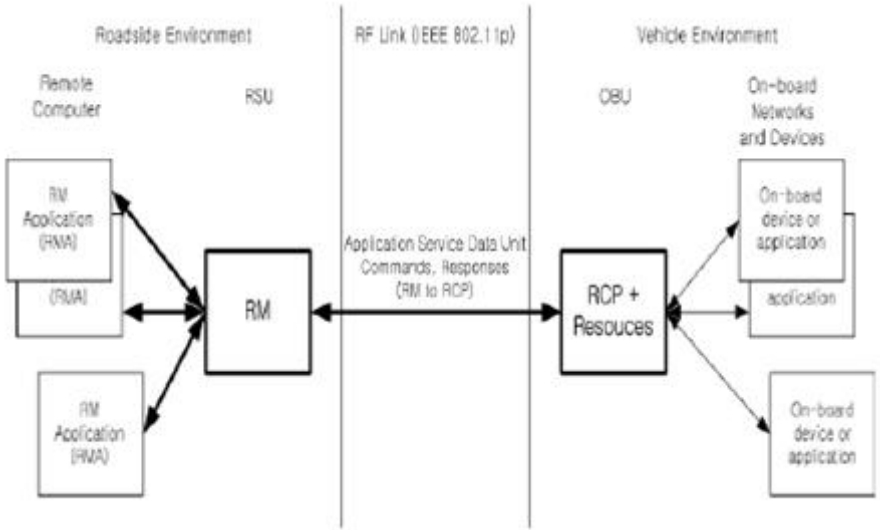


그림 6-7. IEEE 1609.1 의 구성요소

IEEE 1609.2는 WAVE 네트워크와 응용 서비스를 위해 제공하는 보안 서비스를 정의하고 Security가 보장되는 WAVE 메시지를 생성한다. 제공하는 보안 서비스에는 공개키(public key)를 이용한 Encryption기법과 비익명 인증(Non-anonymous Authentication)을 포함한다.

IEEE 1609.3은 서비스 provider와 user간의 안전한 데이터 교환을 위한 네트워크 계층 및 전송 계층을 정의하고 있다. 여기서 새로 추가된 부분은 WSM과 WSMP이다. WSM은 긴급 데이터 전송 등을 위해 최소 채널 용량 (Minimal Channel Capacity) 사용을 목적으로 하여 IP 트래픽이 SCH로만 전송 가능한 것과 달리 CCH와 SCH를 모두 사용 가능하다. WSMP(WAVE Short Message Protocol)는 WSM의 전송을 위해 설계 되었으며 채널 번호, 전송 파워와 같은 물리적 특성을 응용 서비스가 직접 제어할 수 있다. WBSS 설정 없이 데이터 전송을 하기 위해서는 WSMP를 사용하여 CCH만을 통해 전송하게 되면, user에서 PSID (Provider Service Identifier)를 통해 해당 응용 서비스에 전달된다.

IEEE 1609.4는 다중 채널 조정 기능이 가능하도록 802.11 표준을 기능적으로 확장하였다. WAVE에서는 정보 교환을 위하여 Management Frame과 Data Frame 두 가지 종류의 Frame이 존재한다. Primary Management Frame인 WAVE Announcement는 CCH에서만 전송이 허용되며, 그 밖의 정의된 다른 Management Frame들은 SCH에서 사용된다. WSM을 포함한 Data Frame은 CCH와 SCH 모두에서 교환될 수 있다. 하지만 IP Data Frame은 SCH에서만 전송이 허용된다. Channel routing을 위하여 MSDU가 LLC로부터 MAC 계층으로

전달될 때 MAC은 MSDU가 WSM인지 IP datagram인지 결정하기 위해 802.2의 헤더의 EtherType필드를 조사한다. WSMP 헤더는 채널, 파워 레벨, 전송 속도 등을 포함한다. WSMP 데이터가 MAC으로 전달되면 MAC은 WSMP 헤더에 포함된 채널 넘버에 상응하는 적절한 Queue로 packet을 routing한다. 채널 넘버가 적절한 채널 넘버가 아니면 해당 데이터 packet을 버려진다. IP Datagram의 경우에는, IP 데이터 교환을 시작하기 전에 송신자 프로파일이 MLME에 등록된다. 송신자 프로파일은 SCH 넘버, 파워 레벨, 전송 속도 등의 정보를 포함한다. IP 데이터가 LLC로부터 MAC으로 도착하면 MAC은 패킷을 현재 SCH에 해당하는 버퍼로 routing한다. 송신자 프로파일에 등록되지 않았거나 전송하는 WAVE Device에 해당하는 WBSS 멤버가 아니라면 그 데이터는 버려진다.

6. A Node-Cooperative ARQ Scheme For Wireless ad Hoc Networks

이 논문은 높은 frame error rate를 가지는 Ad hoc 네트워크의 무선 채널에서 cooperative ARQ방식을 사용하여 이에 대항하는 방법에 관한 논문이다. 기존의 유선 연결로 이루어진 채널에서는 frame error가 임의로(random) 발생하게 되지만 무선 채널의 경우에는 frame error가 임의보다는 순간적 폭발하듯이(burst) 발생하게 된다. 협력 통신(cooperative communication)은 각 node들이 송신과 수신을 동시에 수행하여 다른 node사이에 통로(relay)역할을 하는 방법이다. 제안 방안에서는 SW(stop and wait)방식의 ARQ 방식을 가정하였는데, 이는 실제 Ad hoc network에서 node간의 전송 지연(propagation delay)가 frame duration에 비해 매우 적으므로 한 frame을 보내고 ACK 혹은 NACK로 확인하는 SW 방식 좀더 복잡한 방식인 GBN(go back N), SR(Selective Repeat) ARQ 방식보다 더 효과적이다. 따라서 sender node는 ACK를 통해 이전에 전송된 frame의 올바른 수신이 확인될 때까지 다음 frame을 전송하지 않는다.

협력 그룹 (cooperation group)은 서로 각 node들이 single hop으로 통신할 수 있는 node들로 정의된다. 즉, 협력 그룹은 각 노드들이 서로의 커버리지 영역 안에 존재하고 있는 노드들의 그룹이다. 협력 그룹의 생성은 초기의 connection setup stage동안 형성되며 관리에는 특별한 부가적 신호 없이 자율적으로 형성되며 유지된다. 즉 협력 그룹은 구체적인 조건에 의해 단단하게(tight) 구성된 것이 아니라 자유롭게(loosely) 구성된다.

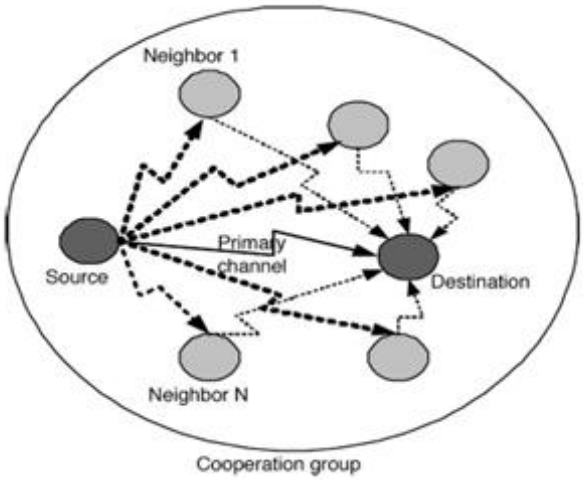


그림 6-8. 협력 그룹을 사용한 ARQ 동작 과정

[그림 6-8]은 협력 그룹을 사용한 ARQ의 동작과정을 나타낸다. 임의의 순간에 특정 source 노드가 의도된 destination 노드에 frame을 전송하기 위하여 이를 위한 특정 channel 자원을 획득한다. 같은 협력 그룹에 속하는 주변 노드들은 error가 발생하면 도움을 주기 위하여 전송을 위해 사용되는 channel을 주시하여 가장 최근에 전송한 frame의 복사본을 디코딩하여 저장한다. 앞에서 정한 것과 같이 SW방식의 ARQ를 사용하므로 sender는 현재 frame의 ACK를 받을 때까지 다른 frame을 전송하지 않으므로 주변 노드들이 많은 frame에 대한 복사본을 저장할 필요가 없다. 만약 전송된 frame에 대하여 destination node가 ACK를 발송하면 협력 그룹내의 node들을 저장하였던 해당 frame의 복사본을 삭제한다. 반대로 destination node가 NACK을 발송하게 되면 sender와 주변 node들이 재전송 과정에서 cooperation을 수행한다. 이와 같은 node들의 협력을 통하여 성공적인 재전송 확률을 개선할 수 있다.

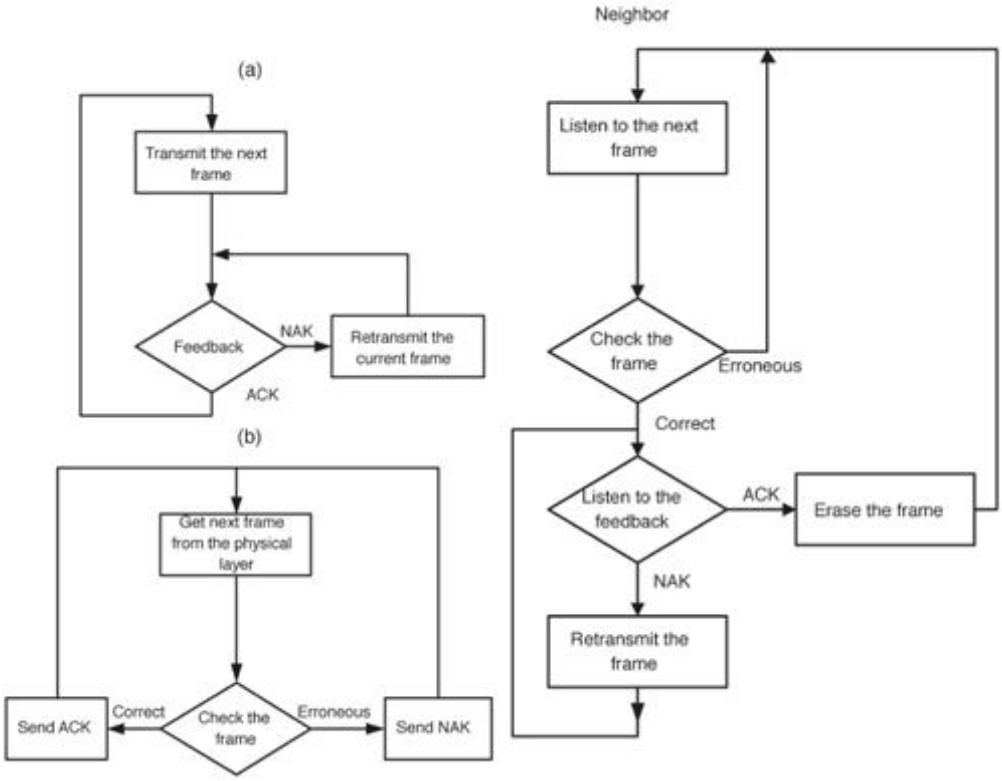


그림 6-9. (a) Source (b) Receiver (c) Cooperative node의 동작 과정

이 논문을 통하여, 주위에 존재하는 노드들을 one-hop 영역과 같이 특정한 기준에 따라 하나의 협력 그룹으로 묶는 방안을 차량간 통신에 적용할 수 있는 것을 확인할 수 있었다. 비슷한 속도로 이동하는 차량 그룹들에 대하여 하나의 협력 그룹으로 지정하게 되면, V2V뿐 아니라 V2I와의 통신에서도 재전송 기법을 통하여 원활한 데이터 통신을 구현할 수 있을 것이다.

7. A Comparison of Reliable Multicast Protocol for Mobile Ad Hoc Networks

이 논문은 MANET에 존재하는 다양한 protocol들에 대하여 장단점과 동작을 비교한다. MANET은 다수의 모바일 노드들로 구성되어 있으며, 모바일 노드들을 multi-hop 라우팅 방법을 통해 순간적인 네트워크를 구성한다. 이중 ARQ를 바탕으로 하는 multicast protocol들에 대한 요약과 비교를 제시하였다.

RMA (Reliable Multicast Algorithm) 방식에서는 source들이 그룹에 대하여 완벽한 정보를 가진다고 가정한다. Source는 전송해야 되는 그룹 멤버들에 대하여 MKNOWN과 MUNKNOWN이라 이름 붙여진 multicast메시지 둘 중 하나를 전송한다. MKNOWN은 알려진 route를 가진 receiver에게 전송되며 MUNKNOWN은 전송 route가 알려지지 않은 receiver에게 전송된다. 만약 source가 모든 그룹 멤버들로부터 ACK를 받지 못할 경우 재전송 과정이 시작되어 모든 ACK를 받을 때까지 반복된다. 모든 노드들은 메시지를 이용해 routing table을 초기화시킨다. Source는 연결의 지속시간에 따라 가장 reliable한 경로를 선택하게 된다.

RALM (Reliable Adaptive Light Weight Multicast Transport protocol)방식은 congestion control을 적용한 것이다. Source가 receiver의 목록에서 하나의 노드에 대하여round robin방시에 따라 feedback receiver로 선택하여 데이터를 전송하며 이를 알려준다. Feedback receiver는 ACK 혹은 NANK를 통해 전송결과를 알려준다. Source는 NACK를 받으면 먼저 전송 속도를 감소시키고 손실된 데이터들을 재전송한다. 이 방법은 데이터 손실의 이유가 bottleneck 연결의 congestion을 발생한 경우에 효과적이다. 하지만 데이터 손실의 주요 원인이 이동성인 경우에는 불필요하게 속도가 줄어들 수 있다.

ReAct (Reliable, Adaptive, Congestion-Controlled Ad hoc Multicast protocol) 방식은 RALM에 local 재전송 기능을 추가한다. Receiver는 손실된 데이터를 먼저 주위의 그룹 멤버들에게 요청하고 몇 번 실패할 경우에 재전송을 위한 NACK를 발송하게 된다. 이 경우 local 재전송이 source가 재전송하는 것 보다 시간과 충돌을 줄이게 된다. 하지만 local재전송이 빈번하게 실패하는 경우 오히려 delay가 길어지게 된다.

Protocol	Recovery Method	Delivery Ratio	Feedback Control	Repair Message	Congestion Control	Local Recovery
RMA	ARQ-based with sender-initiation	guaranteed	ACKs	Multicast	-	No
RALM	ARQ-based with receiver-initiation	guaranteed	NACKs ACKs	Multicast	Yes. Window-adjustable	No
ReAct	ARQ-based with receiver-initiation	guaranteed	NACKs ACKs	Multicast, Unicast for local recovery	Yes. Window-adjustable	Yes

그림 6-10. ARQ 기반 multicast protocol들의 특성 정리

이 논문을 multi-hop으로 구성된 Ad-Hoc 네트워크에서 데이터의 원활한 전송을 위하여 ARQ 기술을 사용하는 경우에 다양한 protocol들이 존재한다는 것을 알 수 있었다. 차량간 통신의 경우에는 특히 각 차량들의 빠른 이동성이 큰 특징 중 하나이므로 이에 적합한 재전송 기술을 필요로 할 것이다.

제 7 장 참고문헌

[1] http://www.vdo.com/products_solutions/cars/safety/braking-technology

[2] http://www.autosar.org, AUTomotive Open System ARchitecture

[3] http://www.vector-informatik.com, Vector Informatik GmbH

[4] http://www.osek-vdx.org, Open Systems and the Corresponding Interfaces for Automotive Electronics

[5] G. Avoine, M. A. Bingol, S. Kardas, C. Lauradoux, and B. Martin, "A formal framework for cryptanalyzing RFID distance bounding protocols," Cryptology ePrint Archive, Report 2009/543, 2009, http://eprint.iacr.org/.

[6] R. J. Anderson, C. Manifavas, and C. Sutherland, "Netcard - a practical electronic-cash system", In Security Protocols Workshop, LNCS, vol. 1189, 1996.

[7] S. Brands and D. Chaum, "Distance-bounding protocols (extended abstract)," in International Conference on the Theory and Applications of Cryptographic Techniques, EUROCRYPT, 1993, pp. 344-359.

[8] S. Capkun and J.-P. Hubaux, "secure positioning of wireless devices with application to sensor networks," in Joint Conference of the IEEE Computer and Communications Societies, INFOCOM, 2005, pp. 1917-928.

[9] D. Coppersmith and M. Jakobsson, "Almost optimal hash sequence traversal", In Financial Cryptography(FC), LNCS, vol. 2357, 2002.

[10] G. Hancke and M. Kuhn, "An RFID distance bounding protocol," in International Conference on Security and Privacy for Emerging Areas in Communications Networks. SecureComm, Sept. 2005, pp. 67-73.

[11] J. Munilla and A. Peinado, "Distance bounding protocols for RFID enhanced by using void-challenges and analysis in noisy channels," Wireless Communications and Mobile Computing, vol. 8, no. 9, pp. 1227-1232, 2008.

[12] Y.-C. Hu, A. Perrig, and D. B. Johnson, "Ariadne: A secure on-demand routing protocol for ad hoc networks", Wireless Networks, vol. 11, issue 1-2, 2005.

[13] G. Itkis and L. Reyzin, "Forward-secure signatures with optimal signing and verifying", In CRYPTO 2001, ser. Lecture Notes in Computer Science 2139, pp. 332-354. Springer, 2001.

[14] M. Jakobsson, “Fractal hash sequence representation and traversal”, In IEEE International Symposium on Information Theory, 2002.

[15] L. Lamport. “Password Authentication with Insecure Communication,” Communications of the ACM, vol. 24, issue 11, 1981.

[16] S. Micali. “Efficient Certificate Revocation,” Technical Report MIT/LCS/TM-542b, MIT Laboratory for Computer Science, 1996.

[17] J. Munilla and A. Peinado, “Distance bounding protocols for RFID enhanced by using void-challenges and analysis in noisy channels,” Wireless Communications and Mobile Computing, vol. 8, no. 9, pp. 1227 – 1232, 2008.

[18] A. Perrig, R. Canetti, D. X. Song, and J. D. Tygar, “Efficient and secure source authentication for multicast”, In Network and Distributed System Security Symposium(NDSS), ACM, 2001.

[19] A. Perrig, R. Canetti, J. D. Tygar, and D. X. Song, “Efficient authentication and signing of multicast streams over lossy channels”, In IEEE Symposium on Security and Privacy, IEEE Computer Society, 2000.

[20] N. Haller, “The s/key one-time password system”, In RFC 1760. Internet Engineering Task Force, 1995.

[21] M. Baugher and E. Carrara. “The Use of Timed Efficient Stream Loss-Tolerant Authentication (TESLA) in the Secure Real-time Transport Protocol (SRTP)”, In RFC 4383. Internet Engineering Task Force, 2006.

[22] R. L. Rivest and A. Shamir, “Payword and micromint: Two simple micropayment schemes”, In Security Protocols Workshop, ser. Lecture Notes in Computer Science, vol. 1189, 1996.

[23] D. Singelee and B. Preneel, “Distance bounding in noisy environments,” in European Workshop on Security and Privacy in Ad-hoc and Sensor Networks, ESAS 2007, ser. Lecture Notes in Computer Science, vol. 4572. Springer, 2007, pp. 101-115.

[24] S. G. Stubblebine and P. F. Syverson, “Fair on-line auctions without special trusted parties”, In Financial Cryptography(FC), LNCS, vol. 1648, 1999.

[25] Y.-J. Tu and S. Piramuthu, “RFID distance bounding protocols,” in International EURASIP Workshop on RFID Technology, RFID, 2007.

[26] C. H. Kim, G. Avoine, F. Koeune, F.-X. Standaert, and O. Pereira, “The

swiss-knife RFID distance bounding protocol," in International Conference on Information Security and Cryptology, ICISC 2008, ser. Lecture Notes in Computer Science, vol. 5461. Springer, 2009, pp. 98-115.

[27] D. H. Yum, J. W. Seo and S. W. Eom, P. J. Lee, "Single-Layer Fractal Hash Chain Traversal with Almost Optimal Complexity", In The Cryptographers' Track at the RSA Conference(CT-RSA), ser. Lecture Notes in Computer Science, vol. 5473, 2009

주 의

1. 이 보고서는 과학기술부에서 시행한 특정연구개발사업의 연구보고서입니다.
2. 이 보고서 내용을 발표할 때에는 반드시 과학기술부에서 시행한 특정연구개발사업의 연구결과임을 밝혀야 합니다.
3. 국가과학기술 기밀유지에 필요한 내용은 대외적으로 발표 또는 공개하여서는 아니됩니다.